
Gareth James ■ Daniela Witten ■ Trevor Hastie
Robert Tibshirani ■ Jonathan Taylor

Nhập môn Statistical Learning

với ứng dụng trong Python

Lần in đầu: ngày 5 tháng 7 năm 2023

Kính tặng cha mẹ chúng tôi:

Alison và Michael James

Chiara Nappi và Edward Witten

Valerie và Patrick Hastie

Vera và Sami Tibshirani

John và Brenda Taylor

và gửi tặng gia đình chúng tôi:

Michael, Daniel và Catherine

Tessa, Theo, Otto và Ari

Samantha, Timothy và Lynda

Charlie, Ryan, Julie và Cheryl

Lee-Ann và Isobel

Lời tựa

Statistical learning là một tập hợp các công cụ giúp hiểu những *dataset phức tạp*. Trong những năm gần đây, quy mô và phạm vi thu thập dữ liệu đã tăng lên đáng kinh ngạc trong hầu hết mọi lĩnh vực khoa học và công nghiệp. Vì vậy, statistical learning đã trở thành một bộ công cụ thiết yếu đối với bất kỳ ai muốn hiểu dữ liệu. Khi ngày càng nhiều công việc hiện nay liên quan đến dữ liệu, điều đó cũng có nghĩa là statistical learning đang nhanh chóng trở thành một bộ công cụ thiết yếu cho *tất cả mọi người*.

Một trong những cuốn sách đầu tiên về statistical learning, *The Elements of Statistical Learning* (ESL, của Hastie, Tibshirani và Friedman), được xuất bản năm 2001; ấn bản thứ hai ra mắt năm 2009. ESL đã trở thành một giáo trình phổ biến không chỉ trong thống kê mà còn trong các lĩnh vực liên quan. Một lý do khiến ESL được đón nhận rộng rãi là cách trình bày tương đối dễ tiếp cận. Tuy nhiên, ESL phù hợp nhất với những người đã được đào tạo chuyên sâu về các ngành khoa học toán học.

An Introduction to Statistical Learning, With Applications in R (ISLR), xuất bản lần đầu năm 2013 và có ấn bản thứ hai năm 2021, ra đời từ nhu cầu rõ rệt về một cách trình bày bao quát hơn và ít đi sâu vào kỹ thuật hơn đối với các chủ đề cốt lõi của statistical learning. Bên cạnh việc ôn lại linear regression, ISLR đề cập đến nhiều phương pháp thống kê và machine learning quan trọng nhất hiện nay, bao gồm resampling, sparse methods cho classification và regression, generalized additive models, tree-based methods, support vector machines, deep learning, survival analysis, clustering và multiple testing.

Kể từ khi xuất bản năm 2013, ISLR đã trở thành giáo trình chủ đạo trong nhiều lớp học bậc đại học và sau đại học trên toàn thế giới, đồng thời là một tài liệu tham khảo quan trọng cho các data scientist. Một trong những yếu tố làm nên thành công của sách là, bắt đầu từ Chương 2, mỗi chương đều có một bài thực hành bằng R minh họa cách triển khai các phương pháp statistical learning được trình bày trong chương, mang lại cho người đọc kinh nghiệm thực hành quý giá.

Tuy nhiên, trong những năm gần đây, **Python** ngày càng trở nên phổ biến trong data science, và nhu cầu về một phiên bản thay thế ISLR sử dụng **Python** cũng ngày càng tăng. Vì vậy, cuốn sách này, *An Introduction to Statistical Learning, With Applications in Python* (ISLP), trình bày cùng nội dung với ISLR nhưng triển khai các bài thực hành bằng **Python**. Thành quả này có được nhờ sự tham gia của đồng tác giả mới, Jonathan Taylor. Một số bài thực hành sử dụng **ISLP**, một Python package do chúng tôi xây dựng để hỗ trợ việc triển khai bằng **Python** các phương pháp statistical learning trong từng chương. Những bài thực hành này hữu ích cho cả người mới bắt đầu lẫn người đã có kinh nghiệm với **Python**.

ISLP (cũng như ISLR) hướng đến việc tập trung nhiều hơn vào ứng dụng của các phương pháp và ít hơn vào các chi tiết toán học. Vì thế, sách phù hợp với sinh viên đại học năm cuối hoặc học viên cao học ngành thống kê hay các lĩnh vực định lượng liên quan, cũng như những người ở các chuyên ngành khác muốn sử dụng công cụ statistical learning để phân tích dữ liệu của mình. Sách có thể được dùng làm giáo trình cho một học phần kéo dài hai học kỳ.

Chúng tôi biết ơn những độc giả sau đây vì các ý kiến đóng góp quý báu cho ấn bản đầu tiên của ISLR: Pallavi Basu, Alexandra Chouldechova, Patrick Danaher, Will Fithian, Luella Fu, Sam Gross, Max Grazier G'Sell, Courtney Paulson, Xinghao Qiao, Elisa Sheng, Noah Simon, Kean Ming Tan, Xin Lu Tan. Chúng tôi cảm ơn những độc giả sau vì các góp ý hữu ích cho ấn bản thứ hai của ISLR: Alan Agresti, Iain Carmichael, Yiqun Chen, Erin Craig, Daisy Ding, Lucy Gao, Ismael Lemhadri, Bryan Martin, Anna Neufeld, Geoff Tims, Carsten Voelkmann, Steve Yadlowsky và James Zou. Chúng tôi đặc biệt biết ơn Balasubramanian “Naras” Narasimhan vì sự hỗ trợ của ông đối với cả ISLR và ISLP.

Với chúng tôi, được chứng kiến ảnh hưởng đáng kể của ISLR đối với việc ứng dụng statistical learning, cả trong và ngoài môi trường học thuật, là một vinh dự và đặc ân. Chúng tôi hy vọng ấn bản **Python** mới này sẽ tiếp tục cung cấp cho các nhà thống kê ứng dụng và data scientist hôm nay cũng như mai sau những công cụ cần thiết để thành công trong một thế giới vận hành dựa trên dữ liệu.

Đưa ra dự đoán thật khó, nhất là dự đoán về tương lai.

-Yogi Berra

Mục lục

Lời tựa	vii
1 Giới thiệu	1
2 Statistical Learning	15
2.1 Statistical Learning là gì?	15
2.1.1 Tại sao cần ước lượng f ?	17
2.1.2 Ước lượng f như thế nào?	20
2.1.3 Trade-Off giữa Prediction Accuracy và Model Interpretability	23
2.1.4 So sánh Supervised và Unsupervised Learning	25
2.1.5 So sánh bài toán Regression và Classification	27
2.2 Đánh giá độ chính xác của mô hình	27
2.2.1 Đo lường chất lượng fit	28
2.2.2 Bias-Variance Trade-Off	31
2.2.3 Bối cảnh Classification	34
2.3 Thực hành: Nhập môn Python	40
2.3.1 Bắt đầu	40
2.3.2 Các lệnh cơ bản	40
2.3.3 Giới thiệu Numerical Python	42
2.3.4 Đồ họa	48
2.3.5 Sequence và ký hiệu Slice	51
2.3.6 Đánh chỉ số dữ liệu	51
2.3.7 Nạp dữ liệu	55
2.3.8 Vòng lặp For	59
2.3.9 Các cách tóm tắt bổ sung bằng đồ thị và số	61
2.4 Bài tập	63
3 Linear Regression	69
3.1 Simple Linear Regression	70
3.1.1 Ước lượng các hệ số	71
3.1.2 Đánh giá độ chính xác của các ước lượng hệ số	72

3.1.3	Đánh giá độ chính xác của mô hình	77
3.2	Multiple Linear Regression	80
3.2.1	Ước lượng các hệ số Regression	81
3.2.2	Một số câu hỏi quan trọng	83
3.3	Những vấn đề khác trong mô hình Regression	91
3.3.1	Qualitative Predictors	91
3.3.2	Các mở rộng của Linear Model	94
3.3.3	Những vấn đề có thể gặp	100
3.4	Kế hoạch marketing	109
3.5	So sánh Linear Regression với K-Nearest Neighbors	111
3.6	Thực hành: Linear Regression	116
3.6.1	Import các package	116
3.6.2	Simple Linear Regression	117
3.6.3	Multiple Linear Regression	122
3.6.4	Multivariate Goodness of Fit	123
3.6.5	Interaction Terms	124
3.6.6	Các phép biến đổi phi tuyến của Predictor	125
3.6.7	Qualitative Predictors	126
3.7	Bài tập	127
4	Classification	135
4.1	Tổng quan về Classification	135
4.2	Tại sao không dùng Linear Regression?	136
4.3	Logistic Regression	138
4.3.1	Logistic Model	139
4.3.2	Ước lượng các hệ số Regression	140
4.3.3	Đưa ra dự đoán	141
4.3.4	Multiple Logistic Regression	142
4.3.5	Multinomial Logistic Regression	144
4.4	Generative Models cho Classification	146
4.4.1	Linear Discriminant Analysis với $p = 1$	147
4.4.2	Linear Discriminant Analysis với $p > 1$	150
4.4.3	Quadratic Discriminant Analysis	156
4.4.4	Naive Bayes	158
4.5	So sánh các phương pháp Classification	161
4.5.1	So sánh bằng phân tích lý thuyết	161
4.5.2	So sánh bằng thực nghiệm	164
4.6	Generalized Linear Models	167
4.6.1	Linear Regression trên dữ liệu Bikeshare	167
4.6.2	Poisson Regression trên dữ liệu Bikeshare	169
4.6.3	Generalized Linear Models trong bối cảnh tổng quát hơn	172
4.7	Thực hành: Logistic Regression, LDA, QDA, và KNN	173
4.7.1	Dữ liệu thị trường chứng khoán	173
4.7.2	Logistic Regression	174
4.7.3	Linear Discriminant Analysis	179
4.7.4	Quadratic Discriminant Analysis	181
4.7.5	Naive Bayes	182
4.7.6	K-Nearest Neighbors	183

4.7.7	Linear và Poisson Regression trên dữ liệu Bikeshare .	188
4.8	Bài tập	193
5	Các phương pháp Resampling	201
5.1	Cross-Validation	202
5.1.1	Phương pháp Validation Set	202
5.1.2	Leave-One-Out Cross-Validation	204
5.1.3	k-Fold Cross-Validation	206
5.1.4	Bias-Variance Trade-Off for k-Fold Cross-Validation .	208
5.1.5	Cross-Validation cho bài toán Classification	209
5.2	Bootstrap	212
5.3	Thực hành: Cross-Validation và Bootstrap	215
5.3.1	Phương pháp Validation Set	216
5.3.2	Cross-Validation	217
5.3.3	Bootstrap	220
5.4	Bài tập	224
6	Lựa chọn Linear Model và Regularization	229
6.1	Subset Selection	231
6.1.1	Best Subset Selection	231
6.1.2	Stepwise Selection	233
6.1.3	Lựa chọn mô hình tối ưu	235
6.2	Các phương pháp Shrinkage	240
6.2.1	Ridge Regression	240
6.2.2	Lasso	244
6.2.3	Lựa chọn Tuning Parameter	252
6.3	Các phương pháp Dimension Reduction	253
6.3.1	Principal Components Regression	254
6.3.2	Partial Least Squares	260
6.4	Những vấn đề khi số chiều lớn	262
6.4.1	High-Dimensional Data	262
6.4.2	Điều gì có thể xảy ra khi số chiều lớn?	263
6.4.3	Regression khi số chiều lớn	265
6.4.4	Diễn giải kết quả khi số chiều lớn	266
6.5	Thực hành: Linear Models và các phương pháp Regularization	267
6.5.1	Các phương pháp Subset Selection	268
6.5.2	Ridge Regression và Lasso	273
6.5.3	PCR và PLS Regression	280
6.6	Bài tập	283
7	Vượt ra ngoài tính tuyến tính	289
7.1	Polynomial Regression	290
7.2	Step Functions	292
7.3	Basis Functions	293
7.4	Regression Splines	294
7.4.1	Piecewise Polynomials	294
7.4.2	Các ràng buộc và Spline	296
7.4.3	Biểu diễn bằng Spline Basis	296
7.4.4	Chọn số lượng và vị trí của Knot	297

7.4.5	So sánh với Polynomial Regression	299
7.5	Smoothing Splines	300
7.5.1	Tổng quan về Smoothing Splines	300
7.5.2	Lựa chọn Smoothing Parameter λ	301
7.6	Local Regression	303
7.7	Generalized Additive Models	305
7.7.1	GAM cho bài toán Regression	306
7.7.2	GAM cho bài toán Classification	308
7.8	Thực hành: Mô hình hóa phi tuyến	309
7.8.1	Polynomial Regression và Step Functions	310
7.8.2	Splines	315
7.8.3	Smoothing Splines và GAMs	317
7.8.4	Local Regression	324
7.9	Bài tập	325
8	Các phương pháp Tree-Based	331
8.1	Cơ sở của Decision Trees	331
8.1.1	Regression Trees	331
8.1.2	Classification Trees	337
8.1.3	So sánh Tree với Linear Model	341
8.1.4	Ưu và nhược điểm của Tree	341
8.2	Bagging, Random Forests, Boosting và Bayesian Additive Regression Trees	343
8.2.1	Bagging	343
8.2.2	Random Forests	346
8.2.3	Boosting	347
8.2.4	Bayesian Additive Regression Trees	350
8.2.5	Tổng kết các phương pháp Tree Ensemble	353
8.3	Thực hành: Các phương pháp Tree-Based	354
8.3.1	Fit Classification Trees	355
8.3.2	Fit Regression Trees	358
8.3.3	Bagging và Random Forests	360
8.3.4	Boosting	361
8.3.5	Bayesian Additive Regression Trees	362
8.4	Bài tập	363
9	Support Vector Machines	367
9.1	Maximal Margin Classifier	367
9.1.1	Hyperplane là gì?	368
9.1.2	Classification bằng Separating Hyperplane	368
9.1.3	Maximal Margin Classifier	370
9.1.4	Xây dựng Maximal Margin Classifier	372
9.1.5	Trường hợp Non-separable	372
9.2	Support Vector Classifiers	373
9.2.1	Tổng quan về Support Vector Classifier	373
9.2.2	Chi tiết về Support Vector Classifier	374
9.3	Support Vector Machines	377
9.3.1	Classification với Decision Boundary phi tuyến	378
9.3.2	Support Vector Machine	379

9.3.3	Ứng dụng vào dữ liệu bệnh tim	382
9.4	SVM với nhiều hơn hai Class	383
9.4.1	One-Versus-One Classification	384
9.4.2	One-Versus-All Classification	384
9.5	Mối liên hệ với Logistic Regression	384
9.6	Thực hành: Support Vector Machines	387
9.6.1	Support Vector Classifier	387
9.6.2	Support Vector Machine	390
9.6.3	ROC Curves	392
9.6.4	SVM với nhiều Class	393
9.6.5	Ứng dụng vào dữ liệu biểu hiện gene	394
9.7	Bài tập	395
10	Deep Learning	399
10.1	Neural Network một Layer	400
10.2	Neural Network nhiều Layer	402
10.3	Convolutional Neural Networks	406
10.3.1	Convolution Layers	407
10.3.2	Pooling Layers	410
10.3.3	Kiến trúc của một Convolutional Neural Network . .	410
10.3.4	Data Augmentation	411
10.3.5	Kết quả khi dùng Pretrained Classifier	412
10.4	Document Classification	413
10.5	Recurrent Neural Networks	416
10.5.1	Sequential Models cho Document Classification . . .	418
10.5.2	Time Series Forecasting	420
10.5.3	Tổng kết về RNN	424
10.6	Khi nào nên dùng Deep Learning	425
10.7	Fit một Neural Network	427
10.7.1	Backpropagation	428
10.7.2	Regularization và Stochastic Gradient Descent . . .	429
10.7.3	Dropout Learning	431
10.7.4	Network Tuning	431
10.8	Interpolation và Double Descent	432
10.9	Thực hành: Deep Learning	435
10.9.1	Network một Layer trên dữ liệu Hitters	437
10.9.2	Network nhiều Layer trên dữ liệu chữ số MNIST . .	444
10.9.3	Convolutional Neural Networks	448
10.9.4	Sử dụng các mô hình Pretrained CNN	452
10.9.5	Document Classification trên IMDB	454
10.9.6	Recurrent Neural Networks	458
10.10	Bài tập	465
11	Survival Analysis và Censored Data	469
11.1	Survival Time và Censoring Time	470
11.2	Tìm hiểu kỹ hơn về Censoring	470
11.3	Kaplan–Meier Survival Curve	472
11.4	Log-Rank Test	474
11.5	Mô hình Regression với Survival Response	476

11.5.1	Hazard Function	476
11.5.2	Proportional Hazards	478
11.5.3	Ví dụ: Dữ liệu ung thư não	482
11.5.4	Ví dụ: Dữ liệu Publication	482
11.6	Shrinkage cho Cox Model	484
11.7	Các chủ đề bổ sung	486
11.7.1	Area Under the Curve cho Survival Analysis	486
11.7.2	Lựa chọn thang thời gian	487
11.7.3	Time-Dependent Covariates	488
11.7.4	Kiểm tra giả định Proportional Hazards	488
11.7.5	Survival Trees	488
11.8	Thực hành: Survival Analysis	489
11.8.1	Dữ liệu ung thư não	489
11.8.2	Dữ liệu Publication	493
11.8.3	Dữ liệu tổng đài	494
11.9	Bài tập	498
12	Unsupervised Learning	503
12.1	Thách thức của Unsupervised Learning	503
12.2	Principal Components Analysis	504
12.2.1	Principal Components là gì?	505
12.2.2	Một cách diễn giải khác về Principal Components	508
12.2.3	Proportion of Variance Explained	510
12.2.4	Tìm hiểu thêm về PCA	512
12.2.5	Các ứng dụng khác của Principal Components	515
12.3	Missing Values và Matrix Completion	515
12.4	Các phương pháp Clustering	520
12.4.1	K-Means Clustering	521
12.4.2	Hierarchical Clustering	525
12.4.3	Những vấn đề thực tiễn trong Clustering	532
12.5	Thực hành: Unsupervised Learning	535
12.5.1	Principal Components Analysis	535
12.5.2	Matrix Completion	539
12.5.3	Clustering	542
12.5.4	Ví dụ với dữ liệu NCI60	546
12.6	Bài tập	552
13	Multiple Testing	557
13.1	Ôn nhanh về Hypothesis Testing	558
13.1.1	Kiểm định một giả thuyết	558
13.1.2	Type I Error và Type II Error	562
13.2	Thách thức của Multiple Testing	563
13.3	Family-Wise Error Rate	565
13.3.1	Family-Wise Error Rate là gì?	565
13.3.2	Các cách kiểm soát Family-Wise Error Rate	567
13.3.3	Trade-Off giữa FWER và Power	572
13.4	False Discovery Rate	573
13.4.1	Trực giác về False Discovery Rate	573
13.4.2	Quy trình Benjamini-Hochberg	575

13.5	Phương pháp Re-Sampling cho p-Value và False Discovery Rate	577
13.5.1	Phương pháp Re-Sampling cho p-Value	578
13.5.2	Phương pháp Re-Sampling cho False Discovery Rate	579
13.5.3	Khi nào các phương pháp Re-Sampling hữu ích?	581
13.6	Thực hành: Multiple Testing	583
13.6.1	Ôn lại các Hypothesis Test	583
13.6.2	Family-Wise Error Rate	585
13.6.3	False Discovery Rate	588
13.6.4	Phương pháp Re-Sampling	590
13.7	Bài tập	593
Chỉ mục		597



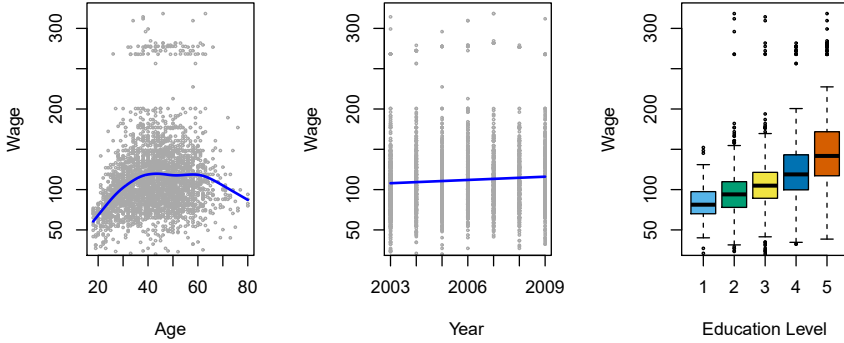
Giới thiệu

Tổng quan về Statistical Learning

Statistical learning là một tập hợp rộng lớn các công cụ giúp *hiểu dữ liệu*. Các công cụ này có thể được phân loại thành *supervised* hoặc *unsupervised*. Nói khái quát, supervised statistical learning xây dựng một mô hình thống kê để dự đoán hoặc ước lượng một *output* dựa trên một hay nhiều *input*. Những bài toán như vậy xuất hiện trong nhiều lĩnh vực khác nhau, từ kinh doanh, y học, vật lý thiên văn đến chính sách công. Với unsupervised statistical learning, ta có các input nhưng không có output đóng vai trò giám sát; dù vậy, ta vẫn có thể học được các mối quan hệ và cấu trúc từ dữ liệu đó. Để minh họa một số ứng dụng của statistical learning, chúng tôi trình bày ngắn gọn ba dataset thực tế được xem xét trong cuốn sách này.

Dữ liệu Wage

Trong ứng dụng này, được gọi là dataset **Wage** xuyên suốt cuốn sách, chúng tôi xem xét một số yếu tố liên quan đến tiền lương của một nhóm nam giới sống ở vùng ven Đại Tây Dương của Hoa Kỳ. Cụ thể, chúng tôi muốn tìm hiểu mối liên hệ của tuổi, trình độ học vấn của một người lao động, cùng với năm dương lịch, với tiền lương của người đó. Chẳng hạn, hãy xem ô bên trái của Hình 1.1, biểu diễn **wage** theo **age** của từng người trong dataset. Có dấu hiệu cho thấy **wage** tăng theo **age**, nhưng lại giảm sau khoảng 60 tuổi. Đường màu xanh lam, ước lượng **wage** trung bình tại mỗi **age**, làm xu hướng này rõ hơn. Khi biết **age** của một người lao động, ta có thể dùng đường cong này để dự đoán **wage** của họ. Tuy nhiên, Hình 1.1 cũng cho thấy rõ ràng các giá trị dao động đáng kể quanh mức trung bình này, nên chỉ riêng **age** khó có thể đưa ra dự đoán chính xác về **wage** của một người cụ thể.

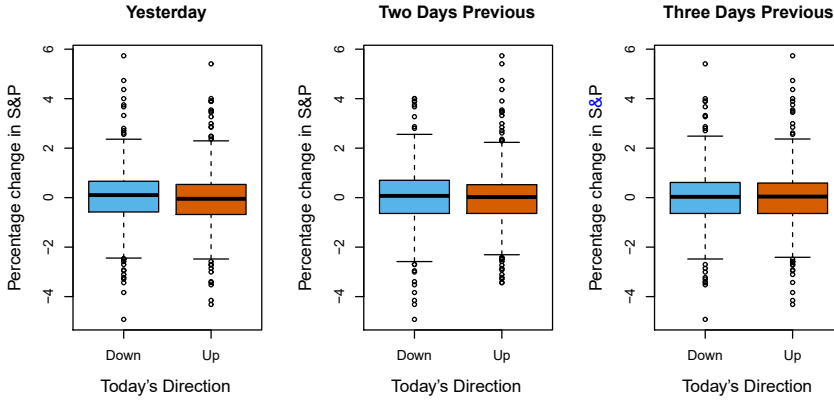


HÌNH 1.1. Dữ liệu *Wage*, chứa thông tin khảo sát thu nhập của nam giới ở vùng Trung Đại Tây Dương của Hoa Kỳ. Trái: *wage* là một hàm của *age*. Trung bình, *wage* tăng theo *age* đến khoảng 60 tuổi, rồi bắt đầu giảm. Giữa: *wage* là một hàm của *year*. Từ năm 2003 đến 2009, *wage* trung bình tăng chậm nhưng đều, tổng cộng khoảng \$10,000. Phải: Các boxplot biểu diễn *wage* theo *education*, trong đó 1 là trình độ thấp nhất (chưa tốt nghiệp trung học phổ thông) và 5 là cao nhất (có bằng sau đại học ở bậc cao). Trung bình, *wage* tăng theo trình độ *education*.

Chúng tôi cũng có thông tin về trình độ học vấn của từng người lao động và *year* mà họ nhận mức *wage* đó. Ở giữa và ô bên phải của Hình 1.1, biểu diễn *wage* theo *year* và *education*, cho thấy cả hai yếu tố này đều có liên hệ với *wage*. Tiền lương tăng khoảng \$10,000 theo dạng gần tuyến tính, tức gần một đường thẳng, từ năm 2003 đến 2009; tuy nhiên, mức tăng này rất nhỏ so với độ biến thiên của dữ liệu. Tiền lương cũng thường cao hơn ở những người có trình độ học vấn cao hơn: nam giới thuộc mức học vấn thấp nhất (1) thường có lương thấp hơn đáng kể so với những người thuộc mức cao nhất (5). Rõ ràng, dự đoán chính xác nhất về *wage* của một người sẽ có được bằng cách kết hợp *age*, *education* và *year*. Trong Chương 3, chúng tôi trình bày linear regression, phương pháp có thể dùng để dự đoán *wage* từ dataset này. Lý tưởng nhất, ta nên dự đoán *wage* theo cách có tính đến mối quan hệ phi tuyến giữa *wage* và *age*. Trong Chương 7, chúng tôi thảo luận một nhóm phương pháp để giải quyết vấn đề này.

Dữ liệu thị trường chứng khoán

Dữ liệu *Wage* đặt ra yêu cầu dự đoán một giá trị output *continuous* hoặc *quantitative*. Đây thường được gọi là bài toán *regression*. Tuy nhiên, trong một số trường hợp, ta muốn dự đoán một giá trị không phải số, tức một output *categorical* hoặc *qualitative*. Chẳng hạn, trong Chương 4, chúng tôi xem xét một dataset thị trường chứng khoán chứa biến động hằng ngày của chỉ số Standard & Poor's 500 (S&P) trong giai đoạn 5 năm từ 2001 đến 2005. Chúng tôi gọi đây là dữ liệu *Smarket*. Mục tiêu là dự đoán chỉ số sẽ *tăng* hay *giảm* vào một ngày nhất định, dựa trên tỷ lệ phần trăm thay đổi của chỉ số trong 5 ngày trước đó. Ở đây, bài toán statistical learning không yêu cầu dự đoán một giá trị số. Thay vào đó, ta dự đoán diễn biến của thị trường chứng khoán trong một ngày sẽ thuộc nhóm *Up* hay nhóm



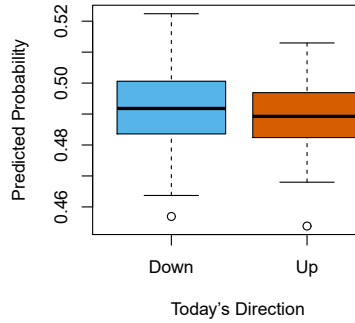
HÌNH 1.2. Trái: Các boxplot của tỷ lệ phần trăm thay đổi chỉ số S&P trong ngày trước đó, chia theo những ngày thị trường tăng hoặc giảm, được lấy từ dữ liệu *Smarket*. Giữa và phải: Tương tự ở bên trái, nhưng biểu diễn tỷ lệ phần trăm thay đổi của 2 và 3 ngày trước đó.

Down. Đây được gọi là bài toán *classification*. Một mô hình có thể dự đoán chính xác hướng biến động của thị trường hần sẽ rất hữu ích!

Ở bên trái của Hình 1.2 có hai boxplot biểu diễn tỷ lệ phần trăm thay đổi của chỉ số chứng khoán trong ngày trước đó: một boxplot cho 648 ngày mà thị trường tăng trong ngày tiếp theo, và một boxplot cho 602 ngày mà thị trường giảm. Hai đồ thị trông gần như giống hệt nhau, cho thấy không có một chiến lược đơn giản nào để dùng biến động S&P của hôm qua dự đoán mức sinh lời hôm nay. Những ô còn lại, biểu diễn các boxplot của tỷ lệ phần trăm thay đổi cách hôm nay 2 và 3 ngày, cũng cho thấy ít liên hệ giữa mức sinh lời trong quá khứ và hiện tại. Tất nhiên, việc không thấy một quy luật rõ ràng như vậy là điều có thể dự kiến: nếu tồn tại tương quan mạnh giữa mức sinh lời của các ngày liên tiếp, ta có thể áp dụng một chiến lược giao dịch đơn giản để kiếm lợi nhuận từ thị trường. Dù vậy, trong Chương 4, chúng tôi vẫn khảo sát dữ liệu này bằng một số phương pháp statistical learning khác nhau. Điều thú vị là dữ liệu hé lộ một vài xu hướng yếu, cho thấy ít nhất trong giai đoạn 5 năm này, ta có thể dự đoán đúng hướng biến động thị trường trong khoảng 60% số trường hợp (Hình 1.3).

Dữ liệu biểu hiện gene

Hai ứng dụng trước minh họa những dataset có cả biến input và biến output. Tuy nhiên, một nhóm bài toán quan trọng khác liên quan đến tình huống ta chỉ quan sát được các biến input mà không có output tương ứng. Chẳng hạn, trong marketing, ta có thể có thông tin nhân khẩu học của một số khách hàng hiện tại hoặc tiềm năng. Ta muốn biết những kiểu khách hàng nào giống nhau bằng cách nhóm các cá nhân theo đặc điểm quan sát được của họ. Đây được gọi là bài toán *clustering*. Khác với các ví dụ trước, ở đây ta không tìm cách dự đoán một biến output.

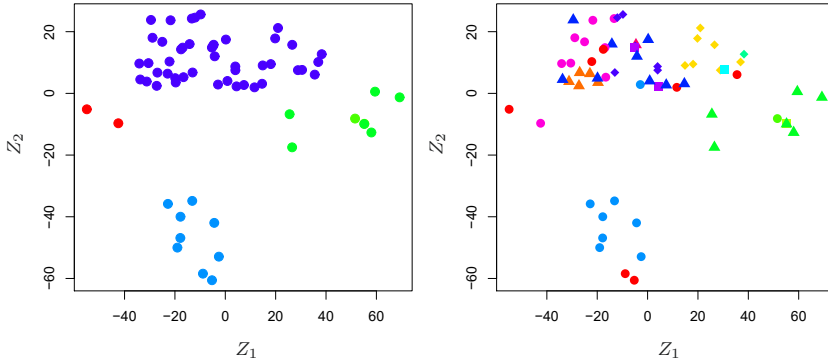


HÌNH 1.3. Chúng tôi fit một mô hình *quadratic discriminant analysis* trên tập con của dữ liệu *Smarket* tương ứng với giai đoạn 2001–2004, rồi dự đoán xác suất thị trường chứng khoán giảm bằng dữ liệu năm 2005. Trung bình, xác suất giảm được dự đoán cao hơn vào những ngày thị trường thực sự giảm. Dựa trên các kết quả này, chúng tôi dự đoán đúng hướng biến động thị trường trong 60% số trường hợp.

Chúng tôi dành Chương 12 để thảo luận các phương pháp statistical learning cho những bài toán không có sẵn một biến output tự nhiên. Chúng tôi xem xét dataset *NCI60*, gồm 6,830 phép đo biểu hiện gene cho mỗi dòng trong số 64 dòng tế bào ung thư. Thay vì dự đoán một biến output cụ thể, chúng tôi muốn xác định xem có những nhóm, hay *cluster*, giữa các dòng tế bào dựa trên các phép đo biểu hiện gene của chúng hay không. Đây là một câu hỏi khó giải quyết, một phần vì mỗi dòng tế bào có hàng nghìn phép đo biểu hiện gene, khiến việc trực quan hóa dữ liệu trở nên khó khăn.

Ở bên trái của Hình 1.4 giải quyết vấn đề này bằng cách biểu diễn mỗi dòng trong số 64 dòng tế bào chỉ bằng hai số, Z_1 và Z_2 . Đây là hai *principal components* đầu tiên của dữ liệu, tóm lược 6,830 phép đo biểu hiện gene của mỗi dòng tế bào thành hai số, hay hai chiều. Dù việc *dimension reduction* này có thể đã làm mất một phần thông tin, giờ đây ta có thể quan sát dữ liệu để tìm dấu hiệu của clustering. Quyết định số lượng cluster thường là một bài toán khó. Tuy nhiên, ở bên trái của Hình 1.4 gợi ý có ít nhất bốn nhóm dòng tế bào, được chúng tôi biểu diễn bằng các màu riêng biệt.

Trong dataset cụ thể này, các dòng tế bào thực ra tương ứng với 14 loại ung thư khác nhau. (Tuy nhiên, thông tin này không được dùng để tạo ô bên trái của Hình 1.4.) Ở bên phải của Hình 1.4 giống ô bên trái, ngoại trừ việc 14 loại ung thư được thể hiện bằng những ký hiệu có màu khác nhau. Có bằng chứng rõ ràng rằng các dòng tế bào thuộc cùng một loại ung thư thường nằm gần nhau trong biểu diễn hai chiều này. Hơn nữa, dù thông tin về loại ung thư không được dùng để tạo ô bên trái, kết quả clustering thu được vẫn phần nào tương đồng với một số loại ung thư thực tế quan sát được ở ô bên phải. Điều này cung cấp một sự kiểm chứng độc lập nhất định về độ chính xác của phân tích clustering.



HÌNH 1.4. Trái: Biểu diễn dataset biểu hiện gene *NCI60* trong không gian hai chiều Z_1 và Z_2 . Mỗi điểm tương ứng với một trong 64 dòng tế bào. Dường như có bốn nhóm dòng tế bào, được chúng tôi thể hiện bằng các màu khác nhau. Phải: Tương tự ô bên trái, nhưng mỗi loại trong số 14 loại ung thư được biểu diễn bằng một ký hiệu có màu riêng. Những dòng tế bào thuộc cùng một loại ung thư thường nằm gần nhau trong không gian hai chiều.

Lược sử Statistical Learning

Dù thuật ngữ *statistical learning* còn khá mới, nhiều khái niệm nền tảng của lĩnh vực này đã được phát triển từ lâu. Đầu thế kỷ XIX, phương pháp *least squares* được xây dựng, triển khai dạng thức sớm nhất của phương pháp ngày nay được gọi là *linear regression*. Phương pháp này lần đầu được áp dụng thành công cho các bài toán thiên văn học. Linear regression được dùng để dự đoán những giá trị quantitative, chẳng hạn tiền lương của một người. Để dự đoán những giá trị qualitative, như một bệnh nhân sống hay tử vong, hoặc thị trường chứng khoán tăng hay giảm, *linear discriminant analysis* đã được đề xuất vào năm 1936. Trong thập niên 1940, nhiều tác giả đưa ra một phương pháp thay thế là *logistic regression*. Đầu thập niên 1970, thuật ngữ *generalized linear model* được xây dựng để mô tả cả một lớp phương pháp statistical learning, trong đó linear regression và logistic regression là những trường hợp riêng.

Đến cuối thập niên 1970, đã có thêm nhiều kỹ thuật học từ dữ liệu. Tuy nhiên, chúng gần như chỉ là các phương pháp tuyến tính, bởi việc fit những mối quan hệ phi tuyến khi đó còn khó khăn về mặt tính toán. Đến thập niên 1980, công nghệ máy tính cuối cùng đã tiến bộ đủ để chi phí tính toán của các phương pháp phi tuyến không còn là trở ngại quá lớn. Vào giữa thập niên 1980, *classification and regression trees* được phát triển; ngay sau đó là *generalized additive models*. *Neural networks* trở nên phổ biến trong thập niên 1980, còn *support vector machines* xuất hiện trong thập niên 1990.

Từ đó, statistical learning nổi lên như một nhánh mới của thống kê, tập trung vào mô hình hóa và dự đoán theo hướng supervised và unsupervised. Trong những năm gần đây, tiến bộ của statistical learning gắn liền với việc ngày càng có nhiều phần mềm mạnh và tương đối dễ sử dụng, chẳng hạn hệ thống *Python* phổ biến và được cung cấp miễn phí. Điều này có thể

tiếp tục thúc đẩy sự chuyển đổi của lĩnh vực, từ một tập hợp các kỹ thuật do các nhà thống kê và nhà khoa học máy tính sử dụng, phát triển, thành một bộ công cụ thiết yếu cho một cộng đồng rộng lớn hơn nhiều.

Về cuốn sách này

The Elements of Statistical Learning (ESL) của Hastie, Tibshirani và Friedman được xuất bản lần đầu vào năm 2001. Kể từ đó, cuốn sách đã trở thành một tài liệu tham khảo quan trọng về nền tảng của statistical machine learning. Thành công của sách bắt nguồn từ cách trình bày toàn diện và chi tiết nhiều chủ đề quan trọng của statistical learning, cũng như việc sách có thể tiếp cận một lượng độc giả rộng rãi nếu so với nhiều giáo trình thống kê nâng cao. Tuy nhiên, yếu tố lớn nhất làm nên thành công của ESL là tính thời sự. Vào thời điểm sách ra đời, sự quan tâm đến statistical learning bắt đầu bùng nổ. ESL cung cấp một trong những phần nhập môn đầu tiên vừa dễ tiếp cận vừa toàn diện về chủ đề này.

Kể từ lần xuất bản đầu tiên của ESL, statistical learning tiếp tục phát triển mạnh. Sự mở rộng diễn ra theo hai hướng. Rõ rệt nhất là việc phát triển các phương pháp statistical learning mới và cải tiến nhằm trả lời nhiều câu hỏi khoa học trong nhiều lĩnh vực. Tuy nhiên, statistical learning cũng mở rộng cộng đồng người sử dụng. Trong thập niên 1990, năng lực tính toán tăng lên đã tạo ra một làn sóng quan tâm từ những người không chuyên về thống kê, những người mong muốn dùng các công cụ thống kê tiên tiến để phân tích dữ liệu của mình. Đáng tiếc, tính kỹ thuật cao của các phương pháp này khiến cộng đồng người dùng chủ yếu vẫn giới hạn ở các chuyên gia thống kê, khoa học máy tính và những lĩnh vực liên quan, vốn có đủ nền tảng đào tạo và thời gian để hiểu cũng như triển khai chúng.

Trong những năm gần đây, các software package mới và được cải tiến đã giảm đáng kể gánh nặng triển khai của nhiều phương pháp statistical learning. Đồng thời, ngày càng nhiều lĩnh vực, từ kinh doanh, chăm sóc sức khỏe, di truyền học đến khoa học xã hội và xa hơn nữa, nhận ra rằng statistical learning là một công cụ mạnh với những ứng dụng thực tiễn quan trọng. Vì vậy, lĩnh vực này đã chuyển từ chỗ chủ yếu được quan tâm trong giới học thuật thành một ngành được sử dụng rộng rãi, với lượng độc giả tiềm năng rất lớn. Xu hướng này chắc chắn sẽ tiếp diễn khi những khối lượng dữ liệu khổng lồ và phần mềm phân tích chúng ngày càng sẵn có.

Mục đích của *An Introduction to Statistical Learning* (ISL) là hỗ trợ statistical learning chuyển từ một lĩnh vực học thuật sang một lĩnh vực được sử dụng rộng rãi. ISL không nhằm thay thế ESL, vốn toàn diện hơn nhiều cả về số lượng phương pháp được xem xét lẫn mức độ chuyên sâu. Chúng tôi xem ESL là một tài liệu đồng hành quan trọng cho những người làm chuyên môn có bằng sau đại học về thống kê, machine learning hoặc các lĩnh vực liên quan, những người cần hiểu chi tiết kỹ thuật đứng sau các phương pháp statistical learning. Tuy nhiên, cộng đồng sử dụng các kỹ thuật statistical learning đã mở rộng, bao gồm những người có mối quan tâm và nền tảng đa dạng hơn. Vì thế, vẫn có chỗ cho một phiên bản ít chuyên sâu về kỹ thuật hơn và dễ tiếp cận hơn của ESL.

Qua nhiều năm giảng dạy các chủ đề này, chúng tôi nhận thấy chúng thu hút học viên cao học và nghiên cứu sinh thuộc những lĩnh vực khác nhau như quản trị kinh doanh, sinh học và khoa học máy tính, cũng như sinh viên đại học các năm cuối có định hướng định lượng. Điều quan trọng với nhóm độc giả đa dạng này là hiểu được mô hình, trực giác, điểm mạnh và điểm yếu của từng phương pháp. Tuy nhiên, nhiều chi tiết kỹ thuật phía sau các phương pháp statistical learning, như các thuật toán tối ưu hóa và những tính chất lý thuyết, không phải mối quan tâm chính của họ. Chúng tôi tin rằng những người học này không cần hiểu thật sâu các khía cạnh đó để sử dụng các phương pháp một cách có hiểu biết, cũng như đóng góp cho lĩnh vực đã chọn bằng các công cụ statistical learning.

ISL được xây dựng trên bốn tiền đề sau.

1. *Nhiều phương pháp statistical learning có liên quan và hữu ích trong rất nhiều lĩnh vực học thuật lẫn ngoài học thuật, vượt ra ngoài phạm vi các ngành thống kê.* Chúng tôi tin rằng nhiều phương pháp statistical learning đương đại nên được, và sẽ được, phổ biến cũng như sử dụng rộng rãi như các phương pháp cổ điển, chẳng hạn linear regression, hiện nay. Vì thế, thay vì cố gắng xem xét mọi phương pháp có thể có, một việc bất khả thi, chúng tôi tập trung trình bày những phương pháp mà mình tin là có khả năng ứng dụng rộng nhất.
2. *Không nên nhìn nhận statistical learning như một chuỗi các black box.* Không có phương pháp đơn lẻ nào hoạt động tốt trong mọi ứng dụng có thể có. Nếu không hiểu các bánh răng bên trong chiếc hộp hoặc cách chúng tương tác với nhau, ta không thể chọn được chiếc hộp tốt nhất. Vì vậy, chúng tôi cố gắng mô tả cẩn thận mô hình, trực giác, giả định và những trade-off đứng sau mỗi phương pháp được xem xét.
3. *Dù cần biết mỗi bánh răng làm nhiệm vụ gì, ta không nhất thiết phải có kỹ năng chế tạo cả cỗ máy bên trong chiếc hộp!* Do đó, chúng tôi hạn chế tối đa việc thảo luận các chi tiết kỹ thuật liên quan đến quy trình fit và các tính chất lý thuyết. Chúng tôi giả định người đọc nắm được những khái niệm toán học cơ bản, nhưng không giả định họ có bằng sau đại học trong các ngành khoa học toán học. Chẳng hạn, chúng tôi gần như hoàn toàn tránh sử dụng đại số ma trận, và người đọc có thể hiểu toàn bộ cuốn sách mà không cần kiến thức chi tiết về ma trận và vector.
4. *Chúng tôi giả định người đọc quan tâm đến việc áp dụng các phương pháp statistical learning vào những bài toán thực tế.* Để hỗ trợ điều đó và tạo động lực cho các kỹ thuật được thảo luận, chúng tôi dành một phần trong mỗi chương cho các bài thực hành trên máy tính. Trong mỗi bài, chúng tôi hướng dẫn người đọc thực hiện một ứng dụng thực tế của các phương pháp được xem xét trong chương. Khi giảng dạy nội dung này, chúng tôi dành khoảng một phần ba thời gian trên lớp để làm các bài thực hành và nhận thấy chúng cực kỳ hữu ích. Nhiều sinh viên ít thiên về tính toán, ban đầu còn e ngại các bài thực hành, đã dần thành thạo trong suốt học kỳ hoặc học kỳ quý. Cuốn sách này ban đầu được xuất bản năm 2013, với ấn bản thứ hai năm 2021,

kèm các bài thực hành viết bằng ngôn ngữ **R**. Kể từ đó, nhu cầu về các bản triển khai bằng **Python** của những kỹ thuật quan trọng trong statistical learning ngày càng tăng. Do vậy, phiên bản này có các bài thực hành bằng **Python**. Số lượng Python package hiện có tăng rất nhanh; khi xem các lệnh import ở đầu mỗi bài thực hành, người đọc sẽ thấy chúng tôi đã lựa chọn cẩn thận và sử dụng những package phù hợp nhất. Chúng tôi cũng cung cấp thêm một số mã lệnh và chức năng trong package **ISLP**. Tuy nhiên, các bài thực hành trong ISL là những phần độc lập; người đọc có thể bỏ qua nếu muốn dùng một software package khác hoặc không muốn áp dụng các phương pháp được thảo luận vào những bài toán thực tế.

Ai nên đọc cuốn sách này?

Cuốn sách dành cho bất kỳ ai quan tâm đến việc dùng các phương pháp thống kê hiện đại để mô hình hóa và dự đoán từ dữ liệu. Nhóm này bao gồm nhà khoa học, kỹ sư, data analyst, data scientist và chuyên gia phân tích định lượng, nhưng cũng bao gồm những người ít thiên về kỹ thuật, có bằng cấp thuộc các lĩnh vực không định lượng như khoa học xã hội hoặc kinh doanh. Chúng tôi kỳ vọng người đọc đã học ít nhất một học phần thống kê nhập môn. Nền tảng về linear regression cũng hữu ích, dù không bắt buộc, vì chúng tôi ôn lại các khái niệm cốt lõi của linear regression trong Chương 3. Mức độ toán học của sách vừa phải và không đòi hỏi kiến thức chi tiết về các phép toán ma trận. Sách có phần nhập môn **Python**. Việc từng tiếp xúc với một ngôn ngữ lập trình, chẳng hạn **MATLAB** hoặc **R**, là hữu ích nhưng không bắt buộc.

Ấn bản đầu tiên của giáo trình này đã được dùng để giảng dạy cho học viên cao học và nghiên cứu sinh trong kinh doanh, kinh tế, khoa học máy tính, sinh học, khoa học Trái Đất, tâm lý học và nhiều ngành khác thuộc khoa học tự nhiên cũng như khoa học xã hội. Sách cũng được dùng để dạy sinh viên đại học các năm cuối đã học một học phần về linear regression. Trong một học phần chặt chẽ hơn về toán học, sử dụng ESL làm giáo trình chính, ISL có thể đóng vai trò tài liệu bổ trợ để giảng dạy những khía cạnh tính toán của các phương pháp khác nhau.

Ký hiệu và đại số ma trận cơ bản

Lựa chọn ký hiệu cho một giáo trình luôn là việc khó. Phần lớn, chúng tôi áp dụng cùng những quy ước ký hiệu như ESL.

Chúng tôi dùng n để biểu thị số điểm dữ liệu phân biệt, hay *observations*, trong mẫu. Chúng tôi dùng p để chỉ số biến có thể sử dụng cho việc dự đoán. Ví dụ, dataset **Wage** gồm 11 biến của 3,000 người, nên ta có $n = 3,000$ observations và $p = 11$ biến, chẳng hạn **year**, **age**, **race** và những biến khác. Lưu ý rằng trong toàn bộ cuốn sách, tên biến được biểu thị bằng chữ có màu: **Variable Name**.

Trong một số ví dụ, p có thể khá lớn, ở mức hàng nghìn hoặc thậm chí hàng triệu. Tình huống này xuất hiện tương đối thường xuyên, chẳng hạn khi phân tích dữ liệu sinh học hiện đại hoặc dữ liệu quảng cáo trên web.

Nói chung, chúng tôi dùng x_{ij} để biểu thị giá trị của biến thứ j ở observation thứ i , trong đó $i = 1, 2, \dots, n$ và $j = 1, 2, \dots, p$. Trong toàn bộ cuốn sách, i được dùng để đánh chỉ số các mẫu hoặc observations (từ 1 đến n), còn j được dùng để đánh chỉ số các biến (từ 1 đến p). Chúng tôi ký hiệu $\mathbf{X}$ là một ma trận $n \times p$ có phần tử (i, j) bằng x_{ij} . Nghĩa là,

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1p} \\ x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \dots & x_{np} \end{pmatrix}.$$

Với những độc giả chưa quen với ma trận, có thể hình dung $\mathbf{X}$ như một bảng tính chứa các số, có n hàng và p cột.

Đôi khi ta quan tâm đến các hàng của $\mathbf{X}$, được viết là $x_1, x_2, \dots, x_n$. Ở đây, x_i là một vector có độ dài p , chứa các giá trị đo của p biến ở observation thứ i . Nghĩa là,

$$x_i = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{ip} \end{pmatrix}. \quad (1.1)$$

(Theo mặc định, các vector được biểu diễn dưới dạng cột.) Chẳng hạn, với dữ liệu **Wage**, x_i là một vector có độ dài 11, gồm **year**, **age**, **race** và những giá trị khác của người thứ i . Những lúc khác, ta lại quan tâm đến các cột của $\mathbf{X}$, được viết là $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_p$. Mỗi cột là một vector có độ dài n . Nghĩa là,

$$\mathbf{x}_j = \begin{pmatrix} x_{1j} \\ x_{2j} \\ \vdots \\ x_{nj} \end{pmatrix}.$$

Ví dụ, với dữ liệu **Wage**, $\mathbf{x}_1$ chứa $n = 3,000$ giá trị của **year**.

Với ký hiệu này, ma trận $\mathbf{X}$ có thể được viết là

$$\mathbf{X} = (\mathbf{x}_1 \quad \mathbf{x}_2 \quad \dots \quad \mathbf{x}_p),$$

hoặc

$$\mathbf{X} = \begin{pmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_n^T \end{pmatrix}.$$

Ký hiệu T biểu thị *transpose* của một ma trận hoặc vector. Do đó, chẳng hạn,

$$\mathbf{X}^T = \begin{pmatrix} x_{11} & x_{21} & \dots & x_{n1} \\ x_{12} & x_{22} & \dots & x_{n2} \\ \vdots & \vdots & & \vdots \\ x_{1p} & x_{2p} & \dots & x_{np} \end{pmatrix},$$

còn

$$x_i^T = (x_{i1} \quad x_{i2} \quad \dots \quad x_{ip}).$$

Chúng tôi dùng y_i để biểu thị observation thứ i của biến mà ta muốn dự đoán, chẳng hạn **wage**. Vì thế, tập hợp tất cả n observations được viết dưới dạng vector như sau:

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}.$$

Khi đó, dữ liệu quan sát của ta gồm $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, trong đó mỗi x_i là một vector có độ dài p . (Nếu $p = 1$, x_i chỉ là một scalar.)

Trong cuốn sách này, một vector có độ dài n luôn được ký hiệu bằng *chữ thường in đậm*; ví dụ,

$$\mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}.$$

Tuy nhiên, các vector có độ dài khác n , chẳng hạn feature vector có độ dài p như trong (1.1), được ký hiệu bằng *chữ thường không in đậm*, ví dụ a . Scalar cũng được ký hiệu bằng chữ thường không in đậm, ví dụ a . Trong những trường hợp hiếm hoi mà hai cách sử dụng này gây mơ hồ, chúng tôi sẽ làm rõ ý nghĩa được dùng. Ma trận được ký hiệu bằng *chữ hoa in đậm*, chẳng hạn **A**. Random variable được ký hiệu bằng chữ hoa không in đậm, ví dụ A , bất kể số chiều của chúng.

Đôi khi ta muốn chỉ rõ số chiều của một đối tượng. Để cho biết một đối tượng là scalar, chúng tôi dùng ký hiệu $a \in \mathbb{R}$. Để cho biết đó là một vector có độ dài k , chúng tôi dùng $a \in \mathbb{R}^k$ (hoặc $\mathbf{a} \in \mathbb{R}^n$ nếu vector có độ dài n). Một đối tượng là ma trận $r \times s$ được biểu thị bằng $\mathbf{A} \in \mathbb{R}^{r \times s}$.

Chúng tôi tránh dùng đại số ma trận bất cứ khi nào có thể. Tuy nhiên, trong một vài trường hợp, việc tránh hoàn toàn khiến cách trình bày trở nên quá rườm rà. Khi đó, điều quan trọng là hiểu khái niệm nhân hai ma trận. Giả sử $\mathbf{A} \in \mathbb{R}^{r \times d}$ và $\mathbf{B} \in \mathbb{R}^{d \times s}$. Tích của $\mathbf{A}$ và $\mathbf{B}$ được ký hiệu là **AB**. Phần tử (i, j) của **AB** được tính bằng cách nhân từng phần tử của hàng thứ i trong $\mathbf{A}$ với phần tử tương ứng của cột thứ j trong $\mathbf{B}$ rồi cộng lại. Nghĩa là, $(\mathbf{AB})_{ij} = \sum_{k=1}^d a_{ik}b_{kj}$. Chẳng hạn, xét

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad \text{và} \quad \mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Khi đó,

$$\mathbf{AB} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 1 \times 5 + 2 \times 7 & 1 \times 6 + 2 \times 8 \\ 3 \times 5 + 4 \times 7 & 3 \times 6 + 4 \times 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

Lưu ý rằng phép toán này tạo ra một ma trận $r \times s$. Chỉ có thể tính $\mathbf{AB}$ khi số cột của $\mathbf{A}$ bằng số hàng của $\mathbf{B}$.

Cấu trúc cuốn sách

Chương 2 giới thiệu các thuật ngữ và khái niệm cơ bản của statistical learning. Chương này cũng trình bày K-nearest neighbor classifier, một phương pháp rất đơn giản nhưng hoạt động tốt một cách đáng ngạc nhiên trên nhiều bài toán. Chương 3 và 4 đề cập đến những phương pháp tuyến tính cổ điển cho regression và classification. Cụ thể, Chương 3 ôn lại linear regression, điểm khởi đầu nền tảng của mọi phương pháp regression. Trong Chương 4, chúng tôi thảo luận hai trong số những phương pháp classification cổ điển quan trọng nhất: logistic regression và linear discriminant analysis.

Một vấn đề trung tâm trong mọi tình huống statistical learning là lựa chọn phương pháp tốt nhất cho một ứng dụng cụ thể. Vì vậy, trong Chương 5, chúng tôi giới thiệu cross-validation và bootstrap, những công cụ có thể dùng để ước lượng độ chính xác của nhiều phương pháp khác nhau nhằm chọn ra phương pháp tốt nhất.

Phần lớn nghiên cứu gần đây về statistical learning tập trung vào các phương pháp phi tuyến. Tuy nhiên, các phương pháp tuyến tính thường có lợi thế so với những phương pháp phi tuyến cạnh tranh về khả năng diễn giải, và đôi khi cả độ chính xác. Vì vậy, trong Chương 6, chúng tôi xem xét nhiều phương pháp tuyến tính, cả cổ điển lẫn hiện đại hơn, có khả năng cải thiện so với linear regression tiêu chuẩn. Những phương pháp này bao gồm stepwise selection, ridge regression, principal components regression và lasso.

Các chương còn lại đi vào thế giới statistical learning phi tuyến. Trước tiên, trong Chương 7, chúng tôi giới thiệu một số phương pháp phi tuyến hoạt động tốt với những bài toán chỉ có một biến input. Sau đó, chúng tôi chỉ ra cách dùng chúng để fit các mô hình cộng phi tuyến có nhiều hơn một input. Trong Chương 8, chúng tôi khảo sát các tree-based methods, bao gồm bagging, boosting và random forests. Support vector machines, một tập hợp các phương pháp thực hiện classification tuyến tính lẫn phi tuyến, được thảo luận trong Chương 9. Trong Chương 10, chúng tôi trình bày deep learning, một hướng tiếp cận regression và classification phi tuyến thu hút nhiều sự chú ý trong những năm gần đây. Chương 11 khảo sát survival analysis, một hướng tiếp cận regression chuyên dùng cho trường hợp biến output bị censored, tức không được quan sát đầy đủ.

Trong Chương 12, chúng tôi xem xét bối cảnh unsupervised, nơi ta có các biến input nhưng không có biến output. Cụ thể, chúng tôi trình bày principal components analysis, K-means clustering và hierarchical clustering. Cuối cùng, trong Chương 13, chúng tôi đề cập đến chủ đề rất quan trọng là multiple hypothesis testing.

Tên	Mô tả
Auto	Quãng đường trên một đơn vị nhiên liệu, công suất và thông tin khác về ô tô.
Bikeshare	Mức sử dụng theo giờ của một chương trình xe đạp dùng chung tại Washington, DC.
Boston	Giá trị nhà ở và thông tin khác về các khu vực điều tra dân số tại Boston.
BrainCancer	Thời gian sống của bệnh nhân được chẩn đoán ung thư não.
Caravan	Thông tin về những người được chào bán bảo hiểm caravan.
Carseats	Thông tin về doanh số ghế ô tô tại 400 cửa hàng.
College	Đặc điểm nhân khẩu học, học phí và thông tin khác về các trường đại học ở Hoa Kỳ.
Credit	Thông tin về dư nợ thẻ tín dụng của 400 khách hàng.
Default	Hồ sơ vỡ nợ của khách hàng tại một công ty thẻ tín dụng.
Fund	Mức sinh lời của 2,000 nhà quản lý hedge fund trong 50 tháng.
Hitters	Thành tích và tiền lương của các cầu thủ bóng chày.
Khan	Các phép đo biểu hiện gene của bốn loại ung thư.
NCI60	Các phép đo biểu hiện gene của 64 dòng tế bào ung thư.
NYSE	Mức sinh lời, volatility và khối lượng giao dịch của Sở Giao dịch Chứng khoán New York.
OJ	Thông tin bán hàng của nước cam Citrus Hill và Minute Maid.
Portfolio	Giá trị quá khứ của tài sản tài chính, dùng cho phân bổ danh mục đầu tư.
Publication	Thời gian đến khi công bố của 244 thử nghiệm lâm sàng.
Smarket	Mức sinh lời phần trăm hằng ngày của S&P 500 trong một giai đoạn 5 năm.
USArrests	Thống kê tội phạm trên 100,000 dân tại 50 tiểu bang của Hoa Kỳ.
Wage	Dữ liệu khảo sát thu nhập của nam giới ở vùng Trung Đại Tây Dương của Hoa Kỳ.
Weekly	1,089 mức sinh lời hằng tuần của thị trường chứng khoán trong 21 năm.

BẢNG 1.1. Danh sách các dataset cần để thực hiện những bài thực hành và bài tập trong giáo trình này. Tất cả đều có trong package **ISLP**, ngoại trừ **USArrests**, thuộc bản phân phối **R** cơ bản nhưng có thể truy cập từ **Python**.

Ở cuối mỗi chương, chúng tôi trình bày một hoặc nhiều phần thực hành **Python**, hướng dẫn có hệ thống các ứng dụng của những phương pháp được thảo luận trong chương. Những bài thực hành này minh họa điểm mạnh và điểm yếu của các phương pháp, đồng thời là tài liệu tham khảo hữu ích về cú pháp cần dùng để triển khai chúng. Người đọc có thể thực hiện theo tốc độ của riêng mình, hoặc dùng các bài thực hành làm nội dung chính cho những buổi học nhóm trên lớp. Trong mỗi bài thực hành **Python**, chúng tôi trình bày những kết quả thu được khi thực hiện bài đó vào thời điểm viết sách. Tuy nhiên, các phiên bản **Python** mới liên tục được phát hành, và theo thời gian, các package được gọi trong bài thực hành cũng sẽ được cập nhật. Vì thế, trong tương lai, những kết quả ở các phần thực hành có thể không còn khớp hoàn toàn với kết quả mà người đọc thu được khi tự thực hiện. Khi cần thiết, chúng tôi sẽ đăng các bản cập nhật bài thực hành trên website của sách.

Chúng tôi dùng ký hiệu  để đánh dấu những mục hoặc bài tập chứa các khái niệm khó hơn. Người đọc có thể dễ dàng bỏ qua chúng nếu không muốn tìm hiểu sâu đến mức đó hoặc chưa có nền tảng toán học cần thiết.

Dataset dùng trong bài thực hành và bài tập

Trong giáo trình này, chúng tôi minh họa các phương pháp statistical learning bằng những ứng dụng trong marketing, tài chính, sinh học và các lĩnh vực khác. Package **ISLP** chứa một số dataset cần thiết để thực hiện các bài thực hành và bài tập đi kèm sách. Một dataset khác, **USArrests**, thuộc bản phân phối **R** cơ bản; chúng tôi hướng dẫn cách truy cập dữ liệu này từ **Python** trong Mục 12.5.1. Bảng 1.1 tóm tắt các dataset cần dùng cho bài thực hành và bài tập. Một vài dataset trong số này cũng được cung cấp dưới dạng tệp văn bản trên website của sách để sử dụng trong Chương 2.

Website của sách

Website của cuốn sách này có địa chỉ

www.statlearning.com

Trang web cung cấp nhiều tài nguyên, bao gồm Python package đi kèm sách và một số dataset bổ sung.

Lời cảm ơn

Một vài đồ thị trong cuốn sách này được lấy từ ESL: Hình 6.7, 8.3 và 12.14. Tất cả các đồ thị khác được tạo cho phiên bản **R** của ISL, ngoại trừ Hình 13.10, vốn khác đi do phần mềm **Python** dùng để hỗ trợ vẽ đồ thị đó.



Statistical Learning

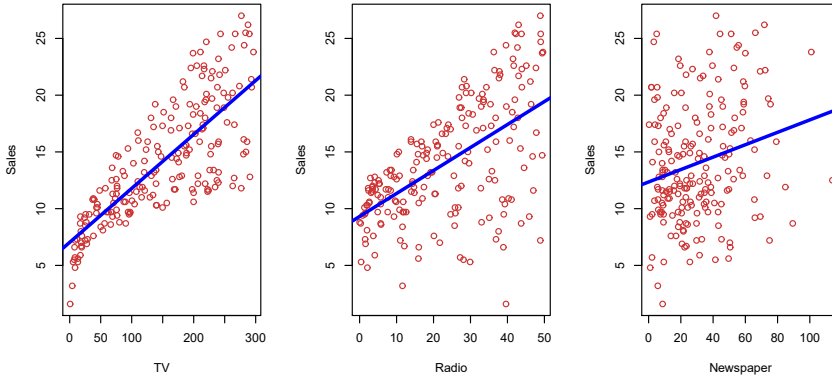
2.1 Statistical Learning là gì?

Để tạo động lực cho việc tìm hiểu statistical learning, ta bắt đầu bằng một ví dụ đơn giản. Giả sử chúng ta là những chuyên gia tư vấn thống kê, được một khách hàng thuê để khảo sát mối liên hệ giữa quảng cáo và doanh số của một sản phẩm cụ thể. Dataset **Advertising** gồm **sales** của sản phẩm đó tại 200 thị trường khác nhau, cùng với ngân sách quảng cáo sản phẩm ở mỗi thị trường trên ba phương tiện: **TV**, **radio** và **newspaper**. Dữ liệu được thể hiện trong Hình 2.1. Khách hàng không thể trực tiếp làm tăng doanh số sản phẩm. Tuy nhiên, họ có thể kiểm soát chi tiêu quảng cáo trên từng phương tiện trong ba phương tiện này. Vì vậy, nếu xác định được mối liên hệ giữa quảng cáo và doanh số, ta có thể hướng dẫn khách hàng điều chỉnh ngân sách quảng cáo, qua đó gián tiếp tăng doanh số. Nói cách khác, mục tiêu của chúng ta là xây dựng một mô hình chính xác để dự đoán doanh số dựa trên ngân sách dành cho ba phương tiện.

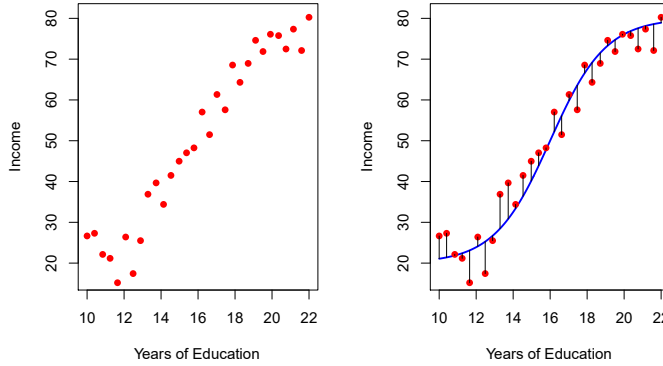
Trong bối cảnh này, ngân sách quảng cáo là các *input variable*, còn **sales** là một *output variable*. Các input variable thường được ký hiệu bằng X , kèm chỉ số dưới để phân biệt. Chẳng hạn, X_1 có thể là ngân sách **TV**, X_2 là ngân sách **radio** và X_3 là ngân sách **newspaper**. Input còn được gọi bằng nhiều tên khác, như *predictor*, *independent variable*, *feature*, hoặc đôi khi đơn giản là *variable*. Output variable, trong trường hợp này là **sales**, thường được gọi là *response* hoặc *dependent variable*, và thường được ký hiệu bằng Y . Trong suốt cuốn sách, chúng tôi dùng các thuật ngữ này thay thế cho nhau.

Tổng quát hơn, giả sử ta quan sát một quantitative response Y và p predictor khác nhau $X_1, X_2, \dots, X_p$. Ta giả định có một mối quan hệ nào đó giữa Y và $X = (X_1, X_2, \dots, X_p)$, có thể viết dưới dạng rất tổng quát:

$$Y = f(X) + \epsilon. \quad (2.1)$$



HÌNH 2.1. Dataset *Advertising*. Đồ thị biểu diễn *sales*, tính bằng nghìn đơn vị sản phẩm, theo ngân sách *TV*, *radio* và *newspaper*, tính bằng nghìn đô la, của 200 thị trường khác nhau. Trong mỗi đồ thị, chúng tôi trình bày kết quả fit *sales* theo biến tương ứng bằng simple least squares, được mô tả trong Chương 3. Nói cách khác, mỗi đường màu xanh lam là một mô hình đơn giản có thể dùng để dự đoán *sales* lần lượt từ *TV*, *radio* và *newspaper*.



HÌNH 2.2. Dataset *Income*. Trái: Các chấm đỏ là giá trị quan sát được của *income* (tính bằng nghìn đô la) và *years of education* ở 30 người. Phải: Đường cong xanh lam thể hiện mối quan hệ thực sự giữa *income* và *years of education*, thường là chưa biết (nhưng đã biết trong trường hợp này vì dữ liệu được mô phỏng). Các đoạn thẳng đen biểu diễn sai số gần với từng observation. Một số sai số dương (nếu observation nằm trên đường cong xanh) và một số sai số âm (nếu observation nằm dưới đường cong). Xét tổng thể, những sai số này có trung bình xấp xỉ bằng không.

Ở đây, f là một hàm cố định nhưng chưa biết của $X_1, \dots, X_p$, còn ϵ là một *error term* ngẫu nhiên, độc lập với X và có trung bình bằng không. Trong cách biểu diễn này, f thể hiện thông tin mang tính *systematic* mà X cung cấp về Y .

Một ví dụ khác là ô bên trái của Hình 2.2, biểu diễn *income* theo *years of education* của 30 người trong dataset *Income*. Đồ thị gợi ý rằng ta có thể dự đoán *income* bằng *years of education*. Tuy nhiên, hàm f liên kết input variable với output variable nói chung là chưa biết. Trong trường hợp

error term
systematic

này, ta phải ước lượng f dựa trên những điểm quan sát được. Vì **Income** là một dataset mô phỏng, f đã được biết và được biểu diễn bằng đường cong xanh lam ở ô bên phải của Hình 2.2. Các đoạn thẳng đứng biểu diễn những error term ϵ . Ta thấy một số trong 30 observations nằm phía trên đường cong xanh và một số nằm phía dưới; xét tổng thể, các sai số có trung bình xấp xỉ bằng không.

Nói chung, hàm f có thể liên quan đến nhiều hơn một input variable. Trong Hình 2.3, chúng tôi biểu diễn **income** như một hàm của **years of education** và **seniority**. Ở đây, f là một mặt hai chiều cần được ước lượng từ dữ liệu quan sát.

Về bản chất, statistical learning là một tập hợp các phương pháp để ước lượng f . Trong chương này, chúng tôi phác thảo một số khái niệm lý thuyết cốt lõi xuất hiện khi ước lượng f , cùng với những công cụ đánh giá các ước lượng thu được.

2.1.1 Tại sao cần ước lượng f ?

Có hai lý do chính khiến ta muốn ước lượng f : prediction và inference. Chúng tôi lần lượt thảo luận từng lý do.

Prediction

Trong nhiều tình huống, ta dễ dàng có được một tập input X , nhưng lại không dễ thu được output Y . Trong bối cảnh này, vì error term có trung bình bằng không, ta có thể dự đoán Y bằng

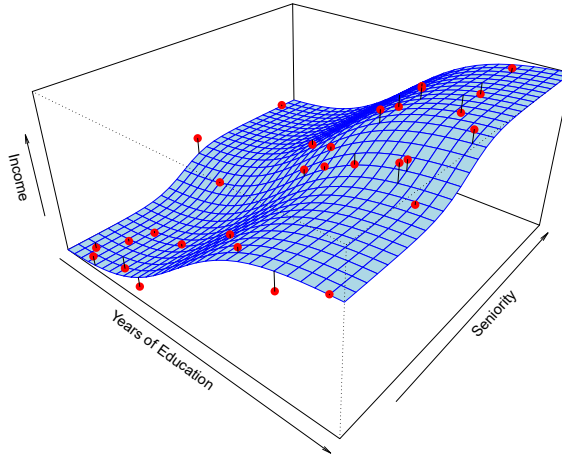
$$\hat{Y} = \hat{f}(X), \quad (2.2)$$

trong đó $\hat{f}$ biểu thị ước lượng của ta đối với f , còn $\hat{Y}$ biểu thị dự đoán tương ứng cho Y . Ở đây, $\hat{f}$ thường được xem là một *black box*: ta thường không quan tâm đến dạng chính xác của $\hat{f}$, miễn là nó tạo ra những dự đoán chính xác cho Y .

Ví dụ, giả sử $X_1, \dots, X_p$ là các đặc điểm của mẫu máu bệnh nhân có thể dễ dàng đo được trong phòng xét nghiệm, còn Y là một biến mã hóa nguy cơ bệnh nhân gặp phản ứng bất lợi nghiêm trọng với một loại thuốc cụ thể. Việc tìm cách dự đoán Y từ X là tự nhiên, vì khi đó ta có thể tránh dùng thuốc đang xét cho những bệnh nhân có nguy cơ phản ứng bất lợi cao, tức những người có giá trị ước lượng của Y cao.

Độ chính xác của $\hat{Y}$ khi dự đoán Y phụ thuộc vào hai đại lượng, mà chúng tôi gọi là *reducible error* và *irreducible error*. Nói chung, $\hat{f}$ không phải là một ước lượng hoàn hảo của f , và sự thiếu chính xác này tạo ra một phần sai số. Sai số đó là *reducible* vì ta có khả năng cải thiện độ chính xác của $\hat{f}$ bằng cách dùng kỹ thuật statistical learning phù hợp nhất để ước lượng f . Tuy nhiên, ngay cả khi có thể ước lượng f một cách hoàn hảo, sao cho response ước lượng có dạng $\hat{Y} = f(X)$, dự đoán của ta vẫn còn sai số! Nguyên nhân là Y còn là hàm của ϵ , mà theo định nghĩa, không thể dự đoán bằng X . Vì vậy, sự biến thiên gắn với ϵ cũng ảnh hưởng đến độ chính xác của dự đoán. Đây được gọi là *irreducible error*, bởi dù ước lượng f tốt đến đâu, ta cũng không thể giảm phần sai số do ϵ gây ra.

reducible
error
irreducible
error



HÌNH 2.3. Đồ thị biểu diễn *income* theo *years of education* và *seniority* trong dataset *Income*. Mặt màu xanh lam thể hiện mối quan hệ thực sự giữa *income* với *years of education* và *seniority*; ta biết mối quan hệ này vì dữ liệu được mô phỏng. Các chấm đỏ biểu thị những giá trị quan sát được của các đại lượng này ở 30 người.

Tại sao irreducible error lại lớn hơn không? Đại lượng ϵ có thể chứa những biến chưa được đo nhưng hữu ích cho việc dự đoán Y : vì ta không đo chúng, f không thể dùng chúng để dự đoán. Đại lượng ϵ cũng có thể chứa sự biến thiên không thể đo được. Chẳng hạn, nguy cơ gặp phản ứng bất lợi của một bệnh nhân vào một ngày nhất định có thể thay đổi tùy theo biến thiên trong quá trình sản xuất thuốc, hoặc tùy theo tình trạng sức khỏe tổng thể mà bệnh nhân cảm nhận trong ngày hôm đó.

Xét một ước lượng $\hat{f}$ và một tập predictor X đã cho, tạo ra dự đoán $\hat{Y} = \hat{f}(X)$. Tạm giả định cả $\hat{f}$ lẫn X đều cố định, để sự biến thiên chỉ đến từ ϵ . Khi đó, dễ dàng chỉ ra rằng

$$\begin{aligned} E(Y - \hat{Y})^2 &= E[f(X) + \epsilon - \hat{f}(X)]^2 \\ &= \underbrace{[f(X) - \hat{f}(X)]^2}_{\text{Reducible}} + \underbrace{\text{Var}(\epsilon)}_{\text{Irreducible}}, \end{aligned} \quad (2.3)$$

trong đó $E(Y - \hat{Y})^2$ là giá trị trung bình, hay *expected value*, của bình phương chênh lệch giữa giá trị dự đoán và giá trị thực tế của Y ; còn $\text{Var}(\epsilon)$ là *variance* gắn với error term ϵ .

Cuốn sách tập trung vào các kỹ thuật ước lượng f nhằm giảm thiểu reducible error. Cần nhớ rằng irreducible error luôn đặt ra một giới hạn trên cho độ chính xác khi dự đoán Y . Trong thực tế, giới hạn này hầu như luôn chưa biết.

Inference

Ta thường quan tâm đến việc hiểu mối liên hệ giữa Y và $X_1, \dots, X_p$. Trong tình huống này, ta muốn ước lượng f , nhưng mục tiêu không nhất thiết là dự đoán Y . Khi đó, không thể xem $\hat{f}$ như một black box, vì ta cần biết

dạng chính xác của nó. Trong bối cảnh này, ta có thể muốn trả lời các câu hỏi sau:

- *Những predictor nào có liên hệ với response?* Thường chỉ một phần nhỏ trong số các predictor sẵn có liên hệ đáng kể với Y . Tùy theo ứng dụng, việc xác định một vài predictor quan trọng trong một tập lớn các biến có thể dùng sẽ vô cùng hữu ích.
- *Mối quan hệ giữa response và từng predictor là gì?* Một số predictor có thể có quan hệ thuận với Y : giá trị predictor lớn hơn đi kèm giá trị Y lớn hơn. Những predictor khác có thể có quan hệ theo chiều ngược lại. Tùy vào độ phức tạp của f , mối quan hệ giữa response và một predictor cụ thể còn có thể phụ thuộc vào giá trị của các predictor khác.
- *Có thể mô tả đầy đủ mối quan hệ giữa Y và từng predictor bằng một phương trình tuyến tính, hay mối quan hệ đó phức tạp hơn?* Trước đây, phần lớn phương pháp ước lượng f sử dụng dạng tuyến tính. Trong một số tình huống, giả định này hợp lý, thậm chí đáng mong muốn. Nhưng mối quan hệ thực sự thường phức tạp hơn; khi đó, một mô hình tuyến tính có thể không biểu diễn chính xác mối quan hệ giữa các biến input và output.

Trong cuốn sách, ta sẽ gặp nhiều ví dụ thuộc bối cảnh prediction, inference, hoặc kết hợp cả hai.

Chẳng hạn, xét một công ty muốn thực hiện chiến dịch marketing trực tiếp. Mục tiêu là xác định những người có khả năng phản hồi tích cực với thư quảng cáo, dựa trên các biến nhân khẩu học quan sát được ở từng người. Ở đây, những biến nhân khẩu học đóng vai trò predictor, còn phản hồi với chiến dịch marketing, tích cực hoặc tiêu cực, đóng vai trò outcome. Công ty không quan tâm đến việc hiểu sâu mối quan hệ giữa từng predictor và response; họ chỉ muốn dự đoán response một cách chính xác từ các predictor. Đây là ví dụ về mô hình hóa phục vụ prediction.

Ngược lại, xét dữ liệu **Advertising** được minh họa trong Hình 2.1. Ta có thể quan tâm đến những câu hỏi như:

- Những phương tiện truyền thông nào có liên hệ với doanh số?
- Phương tiện nào tạo ra mức tăng doanh số lớn nhất? Hoặc,
- Một mức tăng nhất định trong quảng cáo **TV** đi kèm mức tăng doanh số lớn đến đâu?

Tình huống này thuộc bối cảnh inference. Một ví dụ khác là mô hình hóa thương hiệu sản phẩm mà khách hàng có thể mua dựa trên các biến như giá, vị trí cửa hàng, mức giảm giá, giá của đối thủ cạnh tranh, v.v. Trong trường hợp này, điều ta quan tâm nhất có thể chính là mối liên hệ giữa từng biến và xác suất mua hàng. Chẳng hạn, giá sản phẩm liên hệ với doanh số ở mức độ nào? Đây là ví dụ về mô hình hóa phục vụ inference.

Cuối cùng, một số mô hình có thể được xây dựng cho cả prediction lẫn inference. Ví dụ, trong bất động sản, ta có thể tìm cách liên hệ giá trị nhà ở với các input như tỷ lệ tội phạm, quy hoạch phân khu, khoảng cách đến

sông, chất lượng không khí, trường học, mức thu nhập của cộng đồng, diện tích nhà, v.v. Ở đây, ta có thể quan tâm đến mối liên hệ giữa từng input variable và giá nhà: chẳng hạn, một căn nhà có tầm nhìn ra sông sẽ có giá trị cao hơn bao nhiêu? Đây là bài toán inference. Hoặc ta chỉ muốn dự đoán giá trị của một căn nhà khi biết các đặc điểm của nó: căn nhà này đang được định giá thấp hay cao? Đây là bài toán prediction.

Tùy mục tiêu cuối cùng là prediction, inference hay kết hợp cả hai, những phương pháp ước lượng f khác nhau có thể phù hợp. Chẳng hạn, *linear model* cho phép thực hiện inference tương đối đơn giản và dễ diễn giải, nhưng có thể dự đoán kém chính xác hơn một số phương pháp khác. Ngược lại, một số phương pháp có tính phi tuyến cao được trình bày ở những chương sau có thể đưa ra dự đoán khá chính xác cho Y , nhưng phải đánh đổi bằng một mô hình khó diễn giải hơn, khiến inference trở nên khó khăn hơn.

linear model

2.1.2 Ước lượng f như thế nào?

Trong suốt cuốn sách, chúng tôi khảo sát nhiều phương pháp tuyến tính và phi tuyến để ước lượng f . Tuy nhiên, các phương pháp này thường có một số đặc điểm chung. Mục này trình bày tổng quan những đặc điểm đó. Ta luôn giả định đã quan sát một tập gồm n điểm dữ liệu khác nhau. Chẳng hạn, trong Hình 2.2, ta quan sát $n = 30$ điểm dữ liệu. Những observations này được gọi là *training data*, vì ta sẽ dùng chúng để huấn luyện, hay dạy, phương pháp cách ước lượng f . Ký hiệu x_{ij} là giá trị của predictor, hay input, thứ j tại observation i , với $i = 1, 2, \dots, n$ và $j = 1, 2, \dots, p$. Tương ứng, ký hiệu y_i là response variable của observation thứ i . Khi đó, training data gồm $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, trong đó $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})^T$.

training data

Mục tiêu của ta là áp dụng một phương pháp statistical learning vào training data để ước lượng hàm f chưa biết. Nói cách khác, ta muốn tìm một hàm $\hat{f}$ sao cho $Y \approx \hat{f}(X)$ đối với bất kỳ observation (X, Y) nào. Nói khái quát, phần lớn phương pháp statistical learning cho nhiệm vụ này có thể được phân loại thành *parametric* hoặc *non-parametric*. Sau đây, chúng tôi thảo luận ngắn gọn hai nhóm phương pháp này.

parametric
non-
parametric

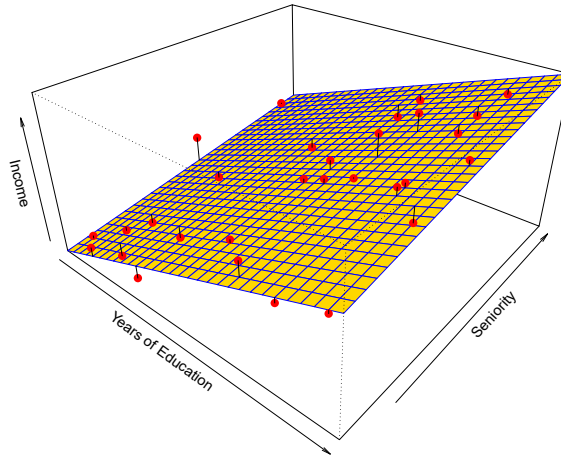
Các phương pháp Parametric

Các phương pháp parametric sử dụng cách tiếp cận dựa trên mô hình, gồm hai bước.

1. Trước hết, ta giả định dạng hàm, hay hình dạng, của f . Chẳng hạn, một giả định rất đơn giản là f tuyến tính theo X :

$$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p. \quad (2.4)$$

Đây là một linear model, sẽ được thảo luận chi tiết trong Chương 3. Khi đã giả định f tuyến tính, bài toán ước lượng f được đơn giản hóa đáng kể. Thay vì phải ước lượng một hàm p chiều hoàn toàn tùy ý $f(X)$, ta chỉ cần ước lượng $p + 1$ hệ số $\beta_0, \beta_1, \dots, \beta_p$.



HÌNH 2.4. Một linear model được fit bằng least squares trên dữ liệu *Income* ở Hình 2.3. Các observations có màu đỏ, còn mặt phẳng màu vàng thể hiện kết quả fit dữ liệu bằng least squares.

- Sau khi chọn mô hình, ta cần một quy trình sử dụng training data để *fit*, hay *train*, mô hình. Với linear model (2.4), ta cần ước lượng các parameter $\beta_0, \beta_1, \dots, \beta_p$. Nghĩa là, ta muốn tìm các giá trị của những parameter này sao cho

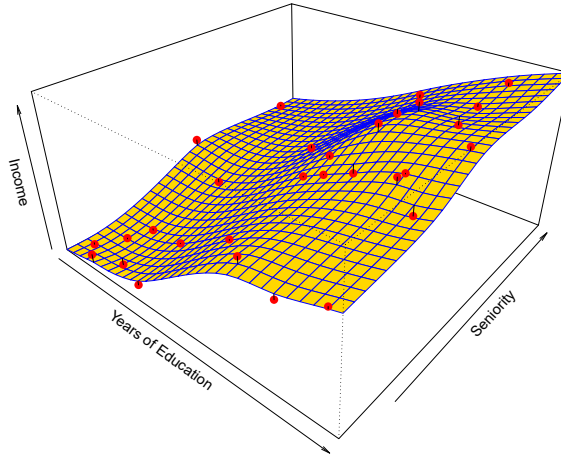
$$Y \approx \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p.$$

Phương pháp phổ biến nhất để fit mô hình (2.4) được gọi là (*ordinary*) *least squares*, sẽ được thảo luận trong Chương 3. Tuy nhiên, least squares chỉ là một trong nhiều cách có thể dùng để fit linear model. Trong Chương 6, chúng tôi trình bày các cách khác để ước lượng những parameter trong (2.4).

Cách tiếp cận dựa trên mô hình vừa mô tả được gọi là *parametric*: nó đưa bài toán ước lượng f về bài toán ước lượng một tập parameter. Giả định f có dạng parametric giúp đơn giản hóa bài toán, bởi nói chung việc ước lượng một tập parameter, chẳng hạn $\beta_0, \beta_1, \dots, \beta_p$ trong linear model (2.4), dễ hơn nhiều so với fit một hàm f hoàn toàn tùy ý. Nhược điểm tiềm tàng của cách tiếp cận parametric là mô hình được chọn thường không khớp với dạng thực sự, chưa biết của f . Nếu mô hình đã chọn quá khác f thực, ước lượng sẽ kém. Ta có thể thử khắc phục bằng cách chọn những mô hình *flexible*, có thể fit nhiều dạng hàm khác nhau của f . Nhưng nói chung, fit một mô hình linh hoạt hơn đòi hỏi phải ước lượng nhiều parameter hơn. Những mô hình phức tạp hơn này có thể dẫn đến hiện tượng *overfitting* dữ liệu: về cơ bản, chúng bám quá sát sai số, hay *noise*. Những vấn đề này được thảo luận xuyên suốt cuốn sách.

Hình 2.4 minh họa một cách tiếp cận parametric áp dụng cho dữ liệu *Income* ở Hình 2.3. Chúng tôi đã fit một linear model có dạng

$$\text{income} \approx \beta_0 + \beta_1 \times \text{education} + \beta_2 \times \text{seniority}.$$



HÌNH 2.5. Kết quả fit dữ liệu *Income* ở Hình 2.3 bằng một *thin-plate spline* trơn được thể hiện bằng màu vàng; các *observations* có màu đỏ. Spline được thảo luận trong Chương 7.

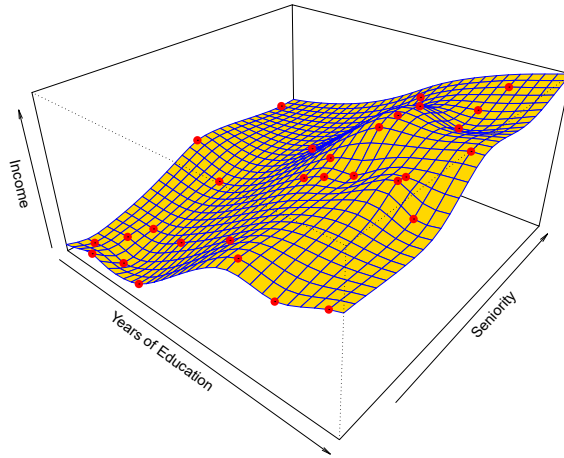
Vì đã giả định mối quan hệ tuyến tính giữa response và hai predictor, toàn bộ bài toán fit được đưa về việc ước lượng β_0 , β_1 và β_2 , thực hiện bằng least squares linear regression. So sánh Hình 2.3 với Hình 2.4, ta thấy kết quả linear fit trong Hình 2.4 chưa hoàn toàn đúng: f thực có độ cong mà linear fit không nắm bắt được. Tuy vậy, linear fit dường như vẫn thể hiện tương đối tốt mối quan hệ thuận giữa số năm đi học và thu nhập, cũng như mối quan hệ thuận yếu hơn đôi chút giữa thâm niên và thu nhập. Với số lượng observations ít như vậy, đây có thể là kết quả tốt nhất ta đạt được.

Các phương pháp Non-Parametric

Các phương pháp non-parametric không đặt ra giả định tường minh về dạng hàm của f . Thay vào đó, chúng tìm một ước lượng của f nằm gần các điểm dữ liệu nhất có thể, nhưng không quá gồ ghề hay uốn lượn. Những cách tiếp cận này có thể có lợi thế lớn so với các phương pháp parametric: nhờ tránh giả định một dạng hàm cụ thể cho f , chúng có khả năng fit chính xác một phạm vi rộng hơn những hình dạng có thể có của f . Mọi cách tiếp cận parametric đều đi kèm khả năng dạng hàm dùng để ước lượng f rất khác f thực; khi đó, mô hình thu được sẽ không fit dữ liệu tốt. Ngược lại, các phương pháp non-parametric tránh hoàn toàn nguy cơ này vì về cơ bản không giả định dạng của f . Nhưng chúng có một nhược điểm lớn: do không rút gọn bài toán ước lượng f thành một số ít parameter, ta cần lượng observations rất lớn, nhiều hơn hẳn mức thường cần cho một phương pháp parametric, để thu được ước lượng chính xác của f .

Hình 2.5 minh họa một cách tiếp cận non-parametric để fit dữ liệu *Income*. Một *thin-plate spline* được dùng để ước lượng f . Cách tiếp cận này không áp đặt một mô hình định trước lên f . Thay vào đó, nó tìm cách tạo ra một ước lượng của f gần dữ liệu quan sát nhất có thể, với ràng buộc rằng kết quả fit, tức mặt màu vàng trong Hình 2.5, phải trơn. Trong trường

thin-plate
spline



HÌNH 2.6. Kết quả fit dữ liệu *Income* ở Hình 2.3 bằng một thin-plate spline gồ ghề. Kết quả fit này không tạo ra sai số nào trên training data.

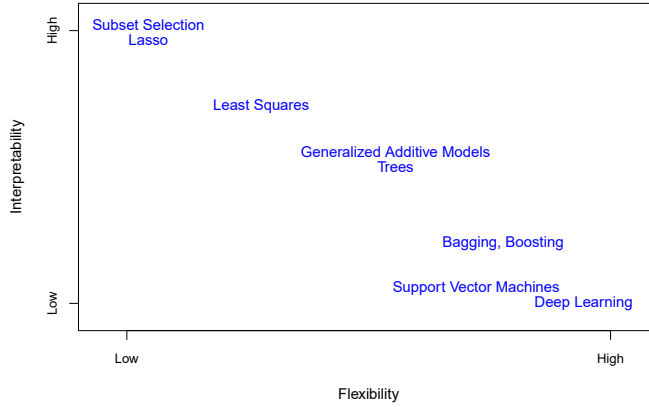
hợp này, non-parametric fit đã tạo ra một ước lượng rất chính xác của f thực được thể hiện trong Hình 2.3. Để fit một thin-plate spline, người phân tích dữ liệu phải lựa chọn mức độ trơn. Hình 2.6 thể hiện cùng thin-plate spline đó nhưng được fit với mức độ trơn thấp hơn, cho phép kết quả gồ ghề hơn. Ước lượng thu được fit hoàn hảo dữ liệu quan sát! Tuy nhiên, spline fit trong Hình 2.6 biến thiên mạnh hơn nhiều so với hàm f thực ở Hình 2.3. Đây là ví dụ về overfitting dữ liệu, như đã thảo luận trước đó. Tình huống này không mong muốn, vì kết quả fit sẽ không ước lượng chính xác response cho những observations mới không thuộc training dataset ban đầu. Các phương pháp lựa chọn mức độ trơn phù hợp được thảo luận trong Chương 5. Spline được trình bày trong Chương 7.

Như ta đã thấy, các phương pháp parametric và non-parametric cho statistical learning đều có ưu và nhược điểm. Chúng tôi khảo sát cả hai nhóm phương pháp xuyên suốt cuốn sách.

2.1.3 Trade-Off giữa Prediction Accuracy và Model Interpretability

Trong nhiều phương pháp được xem xét trong sách, một số ít linh hoạt hơn, hay có nhiều ràng buộc hơn: chúng chỉ có thể tạo ra một phạm vi tương đối nhỏ các hình dạng để ước lượng f . Chẳng hạn, linear regression là một cách tiếp cận tương đối kém linh hoạt, vì chỉ tạo được những hàm tuyến tính như các đường thẳng trong Hình 2.1 hoặc mặt phẳng trong Hình 2.4. Những phương pháp khác, như thin-plate spline trong Hình 2.5 và 2.6, linh hoạt hơn đáng kể vì có thể tạo ra nhiều hình dạng hơn hẳn để ước lượng f .

Một câu hỏi hợp lý là: *tại sao ta lại chọn một phương pháp nhiều ràng buộc hơn thay vì một phương pháp rất linh hoạt?* Có nhiều lý do để ưu tiên một mô hình nhiều ràng buộc hơn. Nếu quan tâm chủ yếu đến inference, các mô hình đó dễ diễn giải hơn nhiều. Chẳng hạn, khi mục tiêu là inference,



HÌNH 2.7. Minh họa trade-off giữa *flexibility* và *interpretability* của các phương pháp statistical learning khác nhau. Nói chung, khi *flexibility* của một phương pháp tăng, *interpretability* của nó giảm.

linear model có thể là lựa chọn tốt vì ta khá dễ hiểu mối quan hệ giữa Y với $X_1, X_2, \dots, X_p$. Ngược lại, các phương pháp rất linh hoạt, như spline được thảo luận trong Chương 7 và thể hiện ở Hình 2.5, 2.6, cũng như các phương pháp boosting trong Chương 8, có thể dẫn đến những ước lượng f phức tạp đến mức khó hiểu một predictor riêng lẻ liên hệ với response như thế nào.

Hình 2.7 minh họa trade-off giữa *flexibility* và *interpretability* của một số phương pháp trong sách. Least squares linear regression, được trình bày trong Chương 3, tương đối kém linh hoạt nhưng khá dễ diễn giải. *Lasso*, được trình bày trong Chương 6, dựa trên linear model (2.4), nhưng dùng một quy trình fit khác để ước lượng các hệ số $\beta_0, \beta_1, \dots, \beta_p$. Quy trình mới đặt ra nhiều ràng buộc hơn khi ước lượng hệ số và đưa một số hệ số về đúng bằng không. Do đó, theo nghĩa này, lasso kém linh hoạt hơn linear regression. Nó cũng dễ diễn giải hơn linear regression, vì trong mô hình cuối cùng, response variable chỉ liên hệ với một tập con nhỏ của các predictor, cụ thể là những predictor có ước lượng hệ số khác không. Ngược lại, *generalized additive model* (GAM), được trình bày trong Chương 7, mở rộng linear model (2.4) để cho phép một số mối quan hệ phi tuyến. Vì thế, GAM linh hoạt hơn linear regression. Chúng cũng khó diễn giải hơn đôi chút, vì mối quan hệ giữa từng predictor và response giờ được mô hình hóa bằng một đường cong. Cuối cùng, các phương pháp hoàn toàn phi tuyến như *bagging*, *boosting*, *support vector machine* với non-linear kernel và neural network (deep learning), được trình bày trong các Chương 8, 9, 10, là những cách tiếp cận rất linh hoạt nhưng khó diễn giải hơn.

Ta đã thấy rằng khi mục tiêu là inference, việc dùng các phương pháp statistical learning đơn giản và tương đối kém linh hoạt có những lợi thế rõ ràng. Tuy nhiên, trong một số bối cảnh, ta chỉ quan tâm đến prediction và hoàn toàn không quan tâm đến *interpretability* của mô hình dự đoán. Chẳng hạn, khi xây dựng một thuật toán dự đoán giá cổ phiếu, yêu cầu duy nhất có thể là dự đoán chính xác; *interpretability* không phải điều

Lasso

generalized
additive
modelbagging
boosting
support
vector
machine

đáng quan tâm. Trong bối cảnh này, ta có thể nghĩ rằng mô hình linh hoạt nhất sẽ là lựa chọn tốt nhất. Điều đáng ngạc nhiên là không phải lúc nào cũng vậy! Ta thường thu được dự đoán chính xác hơn bằng một phương pháp kém linh hoạt hơn. Hiện tượng thoát nhìn có vẻ trái trực giác này liên quan đến khả năng overfitting của những phương pháp rất linh hoạt. Ta đã thấy một ví dụ overfitting trong Hình 2.6. Khái niệm rất quan trọng này sẽ được thảo luận thêm trong Mục 2.2 và xuyên suốt cuốn sách.

2.1.4 So sánh Supervised và Unsupervised Learning

Phần lớn bài toán statistical learning thuộc một trong hai nhóm: *supervised* hoặc *unsupervised*. Các ví dụ đã thảo luận trong chương này đều thuộc supervised learning. Với mỗi observation gồm các giá trị đo predictor x_i , $i = 1, \dots, n$, có một giá trị đo response tương ứng y_i . Ta muốn fit một mô hình liên hệ response với các predictor, nhằm dự đoán chính xác response cho những observations trong tương lai (prediction), hoặc hiểu rõ hơn mối quan hệ giữa response và các predictor (inference). Nhiều phương pháp statistical learning cổ điển như linear regression và *logistic regression* (Chương 4), cũng như các cách tiếp cận hiện đại hơn như GAM, boosting và support vector machines, hoạt động trong bối cảnh supervised learning. Phần lớn cuốn sách dành cho bối cảnh này.

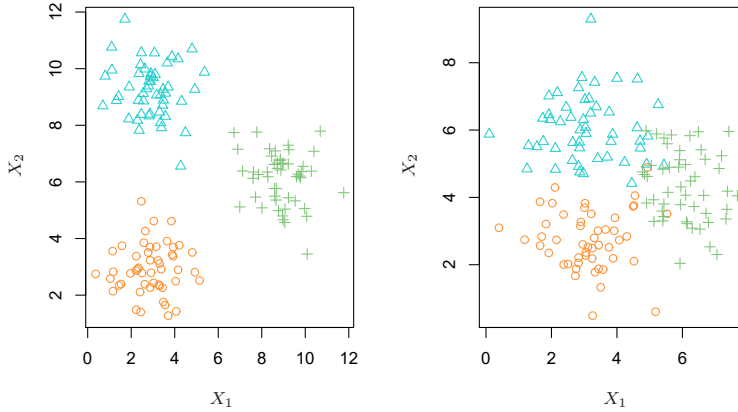
Ngược lại, unsupervised learning mô tả tình huống khó hơn đôi chút: với mỗi observation $i = 1, \dots, n$, ta quan sát một vector giá trị đo x_i nhưng không có response tương ứng y_i . Không thể fit một mô hình linear regression vì không có response variable để dự đoán. Theo một nghĩa nào đó, ta đang làm việc mà không có thông tin định hướng; tình huống được gọi là unsupervised vì thiếu một response variable có thể giám sát việc phân tích. Khi ấy, ta có thể thực hiện loại phân tích thống kê nào? Ta có thể tìm hiểu mối quan hệ giữa các biến hoặc giữa các observations. Một công cụ statistical learning dùng được trong bối cảnh này là *cluster analysis*, hay clustering. Mục tiêu của cluster analysis là dựa trên $x_1, \dots, x_n$ để xác định các observations có thuộc những nhóm tương đối riêng biệt hay không. Chẳng hạn, trong nghiên cứu phân khúc thị trường, ta có thể quan sát nhiều đặc điểm, hay biến, của khách hàng tiềm năng, như mã bưu chính, thu nhập gia đình và thói quen mua sắm. Ta có thể tin rằng khách hàng thuộc những nhóm khác nhau, như nhóm chi tiêu nhiều và nhóm chi tiêu ít. Nếu có thông tin về kiểu chi tiêu của từng khách hàng, ta có thể thực hiện phân tích supervised. Nhưng thông tin này không có sẵn, tức ta không biết từng khách hàng tiềm năng có thuộc nhóm chi tiêu nhiều hay không. Trong bối cảnh đó, ta có thể thử cluster khách hàng dựa trên các biến đã đo để xác định những nhóm khách hàng tiềm năng riêng biệt. Việc xác định các nhóm như vậy có thể hữu ích vì chúng có thể khác nhau về một đặc điểm đáng quan tâm, chẳng hạn thói quen chi tiêu.

Hình 2.8 minh họa đơn giản bài toán clustering. Chúng tôi biểu diễn 150 observations với các giá trị đo của hai biến X_1 và X_2 . Mỗi observation thuộc một trong ba nhóm riêng biệt. Để minh họa, thành viên của mỗi nhóm được vẽ bằng màu và ký hiệu khác nhau. Tuy nhiên, trong thực tế, ta không biết các observations thuộc nhóm nào; mục tiêu là xác định nhóm

supervised
unsupervised

logistic
regression

cluster
analysis



HÌNH 2.8. Một dataset clustering gồm ba nhóm. Mỗi nhóm được biểu diễn bằng một ký hiệu có màu khác nhau. Trái: Ba nhóm tách biệt rõ. Trong trường hợp này, một phương pháp clustering có thể xác định thành công cả ba nhóm. Phải: Các nhóm có phần chồng lấn. Khi đó, nhiệm vụ clustering khó hơn.

của từng observation. Ở ô bên trái của Hình 2.8, đây là nhiệm vụ tương đối dễ vì các nhóm tách biệt rõ. Ngược lại, ô bên phải minh họa bối cảnh khó hơn, trong đó các nhóm có phần chồng lấn. Ta không thể kỳ vọng một phương pháp clustering gắn đúng tất cả các điểm chồng lấn vào nhóm của chúng: xanh lam, xanh lá hoặc cam.

Trong các ví dụ ở Hình 2.8, chỉ có hai biến, nên ta có thể quan sát trực tiếp scatterplot của các observations để nhận diện cluster. Tuy nhiên, trên thực tế, ta thường gặp dataset có nhiều hơn hai biến rất nhiều. Khi ấy, không dễ biểu diễn các observations bằng đồ thị. Chẳng hạn, nếu dataset có p biến, ta có thể tạo ra $p(p-1)/2$ scatterplot khác nhau; việc quan sát trực quan không còn là cách khả thi để nhận diện cluster. Vì vậy, các phương pháp clustering tự động rất quan trọng. Clustering và những cách tiếp cận unsupervised learning khác được thảo luận trong Chương 12.

Nhiều bài toán tự nhiên thuộc bối cảnh supervised hoặc unsupervised learning. Tuy nhiên, đôi khi việc xác định một phân tích là supervised hay unsupervised lại không rõ ràng như vậy. Chẳng hạn, giả sử ta có n observations. Với m observations, trong đó $m < n$, ta có cả các giá trị đo predictor và một giá trị đo response. Với $n - m$ observations còn lại, ta có các giá trị đo predictor nhưng không có giá trị đo response. Tình huống này có thể xuất hiện khi chi phí đo predictor tương đối thấp nhưng việc thu thập response tương ứng tốn kém hơn nhiều. Ta gọi đây là bài toán *semi-supervised learning*. Trong bối cảnh này, ta muốn dùng một phương pháp statistical learning có thể kết hợp cả m observations có giá trị đo response lẫn $n - m$ observations không có. Dù là một chủ đề thú vị, nó nằm ngoài phạm vi cuốn sách.

semi-supervised learning

2.1.5 So sánh bài toán Regression và Classification

Các biến có thể được phân loại thành *quantitative* hoặc *qualitative*, còn

quantitative
qualitative

gọi là *categorical*. Quantitative variable nhận giá trị số. Ví dụ gồm tuổi, chiều cao hoặc thu nhập của một người, giá trị một căn nhà và giá một cổ phiếu. Ngược lại, qualitative variable nhận giá trị thuộc một trong K class, hay category, khác nhau. Ví dụ gồm tình trạng hôn nhân (đã kết hôn hay chưa), thương hiệu sản phẩm được mua (A, B hoặc C), một người có vợ hay không (có hoặc không), hoặc chẩn đoán ung thư (Acute Myelogenous Leukemia, Acute Lymphoblastic Leukemia hoặc No Leukemia). Ta thường gọi những bài toán có quantitative response là bài toán *regression*, còn những bài toán có qualitative response là bài toán *classification*. Tuy nhiên, sự phân biệt không phải lúc nào cũng rạch ròi. Least squares linear regression (Chương 3) dùng cho quantitative response, còn logistic regression (Chương 4) thường dùng cho qualitative response gồm hai class, hay *binary* response. Vì vậy, dù mang tên regression, logistic regression là một phương pháp classification. Nhưng vì ước lượng xác suất của các class, nó cũng có thể được xem như một phương pháp regression. Một số phương pháp thống kê, như K-nearest neighbors (Chương 2 và 4) và boosting (Chương 8), có thể dùng cho cả quantitative response lẫn qualitative response.

Ta thường chọn phương pháp statistical learning dựa trên việc response là quantitative hay qualitative: chẳng hạn, dùng linear regression khi response là quantitative và logistic regression khi response là qualitative. Tuy nhiên, predictor là qualitative hay quantitative thường được xem là ít quan trọng hơn. Phần lớn phương pháp statistical learning trong sách có thể được áp dụng bất kể kiểu predictor variable, miễn là mọi qualitative predictor được mã hóa đúng trước khi phân tích. Vấn đề này được thảo luận trong Chương 3.

2.2 Đánh giá độ chính xác của mô hình

Một mục tiêu cốt lõi của cuốn sách là giới thiệu cho người đọc nhiều phương pháp statistical learning, vượt xa cách tiếp cận linear regression tiêu chuẩn. Tại sao cần giới thiệu nhiều phương pháp statistical learning khác nhau, thay vì chỉ một phương pháp tốt nhất? Trong thống kê, không có “bữa trưa miễn phí”: không phương pháp nào vượt trội mọi phương pháp khác trên tất cả các dataset có thể có. Trên một dataset cụ thể, một phương pháp có thể hoạt động tốt nhất, nhưng trên một dataset tương tự mà khác biệt, phương pháp khác lại có thể tốt hơn. Vì vậy, xác định phương pháp cho kết quả tốt nhất với một tập dữ liệu đã cho là nhiệm vụ quan trọng. Lựa chọn cách tiếp cận tốt nhất có thể là một trong những phần khó nhất khi thực hành statistical learning.

Trong mục này, chúng tôi thảo luận một số khái niệm quan trọng nhất khi lựa chọn một quy trình statistical learning cho dataset cụ thể. Ở những phần tiếp theo của sách, chúng tôi sẽ giải thích cách áp dụng các khái niệm này vào thực tế.

2.2.1 Đo lường chất lượng fit

Để đánh giá hiệu quả của một phương pháp statistical learning trên dataset đã cho, ta cần một cách đo mức độ các dự đoán thực sự khớp với dữ liệu quan sát. Nghĩa là, ta cần định lượng mức độ gần nhau giữa response dự đoán và response thực của một observation. Trong bối cảnh regression, thước đo được dùng phổ biến nhất là *mean squared error* (MSE), được xác định bởi

mean squared
error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2, \quad (2.5)$$

trong đó $\hat{f}(x_i)$ là dự đoán mà $\hat{f}$ đưa ra cho observation thứ i . MSE nhỏ nếu các response dự đoán rất gần response thực, và lớn nếu ở một số observations, response dự đoán khác đáng kể response thực.

MSE trong (2.5) được tính bằng chính training data đã dùng để fit mô hình, nên tên gọi chính xác hơn là *training MSE*. Nhưng nói chung, điều ta thực sự quan tâm không phải là phương pháp hoạt động tốt đến đâu trên training data. Ta quan tâm đến độ chính xác của những dự đoán khi áp dụng phương pháp vào *test data* chưa từng gặp. Tại sao lại như vậy? Giả sử ta muốn xây dựng một thuật toán dự đoán giá cổ phiếu dựa trên mức sinh lời trước đó. Ta có thể train phương pháp bằng mức sinh lời của cổ phiếu trong 6 tháng qua. Nhưng ta không thực sự quan tâm phương pháp dự đoán giá cổ phiếu tuần trước tốt đến đâu; ta quan tâm nó sẽ dự đoán giá ngày mai hay tháng sau tốt đến đâu. Tương tự, giả sử ta có các số đo lâm sàng, như cân nặng, huyết áp, chiều cao, tuổi và tiền sử bệnh trong gia đình, của một số bệnh nhân, cùng thông tin từng người có mắc đái tháo đường hay không. Ta có thể dùng những bệnh nhân này để train một phương pháp statistical learning dự đoán nguy cơ đái tháo đường từ các số đo lâm sàng. Trong thực tế, ta muốn phương pháp dự đoán chính xác nguy cơ cho những bệnh nhân tương lai dựa trên số đo của họ. Ta không quá quan tâm việc phương pháp có dự đoán chính xác nguy cơ ở những bệnh nhân dùng để train mô hình hay không, vì đã biết ai trong số họ mắc bệnh.

training MSE

test data

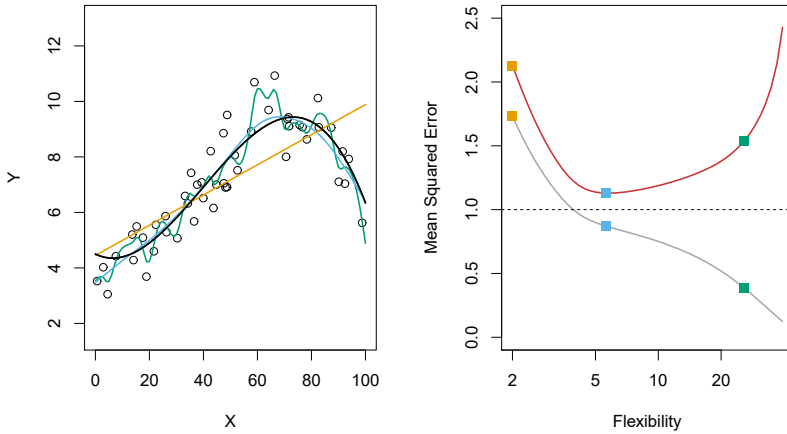
Diễn đạt bằng toán học, giả sử ta fit phương pháp statistical learning trên các training observations $\{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ và thu được ước lượng $\hat{f}$. Khi đó, ta có thể tính $\hat{f}(x_1), \hat{f}(x_2), \dots, \hat{f}(x_n)$. Nếu các giá trị này xấp xỉ $y_1, y_2, \dots, y_n$, training MSE trong (2.5) sẽ nhỏ. Tuy nhiên, điều ta thực sự quan tâm không phải là $\hat{f}(x_i) \approx y_i$; ta muốn biết $\hat{f}(x_0)$ có xấp xỉ y_0 hay không, trong đó (x_0, y_0) là một test observation chưa từng gặp, không được dùng để train phương pháp. Ta muốn chọn phương pháp có *test MSE* thấp nhất, chứ không phải training MSE thấp nhất. Nói cách khác, nếu có nhiều test observations, ta có thể tính

test MSE

$$\text{Ave}(y_0 - \hat{f}(x_0))^2, \quad (2.6)$$

là trung bình bình phương sai số dự đoán trên các test observations (x_0, y_0) . Ta muốn chọn mô hình sao cho đại lượng này nhỏ nhất có thể.

Ta có thể lựa chọn một phương pháp giảm thiểu test MSE bằng cách nào? Trong một số bối cảnh, ta có sẵn test dataset, tức một tập observations chưa được dùng để train phương pháp statistical learning. Khi đó, ta chỉ



HÌNH 2.9. Trái: Dữ liệu được mô phỏng từ f , thể hiện bằng màu đen. Ba ước lượng của f được biểu diễn: đường linear regression màu cam, cùng hai kết quả fit bằng smoothing spline màu xanh lam và xanh lá. Phải: Training MSE (đường xám), test MSE (đường đỏ) và test MSE nhỏ nhất có thể đạt được trên mọi phương pháp (đường đứt nét). Các hình vuông biểu diễn training MSE và test MSE của ba kết quả fit ở ô bên trái.

cần tính (2.6) trên các test observations và chọn phương pháp có test MSE nhỏ nhất. Nhưng nếu không có test observations thì sao? Ta có thể nghĩ đến việc chọn phương pháp statistical learning giảm thiểu training MSE trong (2.5). Cách này có vẻ hợp lý, vì training MSE và test MSE dường như liên hệ chặt chẽ. Đáng tiếc, chiến lược đó có một vấn đề căn bản: không có gì bảo đảm phương pháp có training MSE thấp nhất cũng có test MSE thấp nhất. Nói khái quát, nhiều phương pháp thống kê ước lượng hệ số với mục đích cụ thể là giảm thiểu MSE trên training set. Với những phương pháp này, training MSE có thể rất nhỏ nhưng test MSE thường lớn hơn nhiều.

Hình 2.9 minh họa hiện tượng này bằng một ví dụ đơn giản. Ở ô bên trái, chúng tôi tạo ra các observations từ (2.1), với f thực được biểu diễn bằng đường cong đen. Các đường cam, xanh lam và xanh lá minh họa ba ước lượng có thể có của f , thu được bằng những phương pháp có mức độ linh hoạt tăng dần. Đường cam là kết quả fit bằng linear regression, tương đối kém linh hoạt. Các đường xanh lam và xanh lá được tạo bằng smoothing spline, sẽ được trình bày trong Chương 7, với những mức độ

smoothing
spline

Xét tiếp ô bên phải của Hình 2.9. Đường xám biểu diễn training MSE trung bình theo độ linh hoạt, hay chính xác hơn là *degrees of freedom*, của một số smoothing spline. Degrees of freedom là đại lượng tóm tắt độ linh hoạt của một đường cong, sẽ được trình bày đầy đủ hơn trong Chương 7. Các hình vuông màu cam, xanh lam và xanh lá biểu thị MSE của những

degrees of
freedom

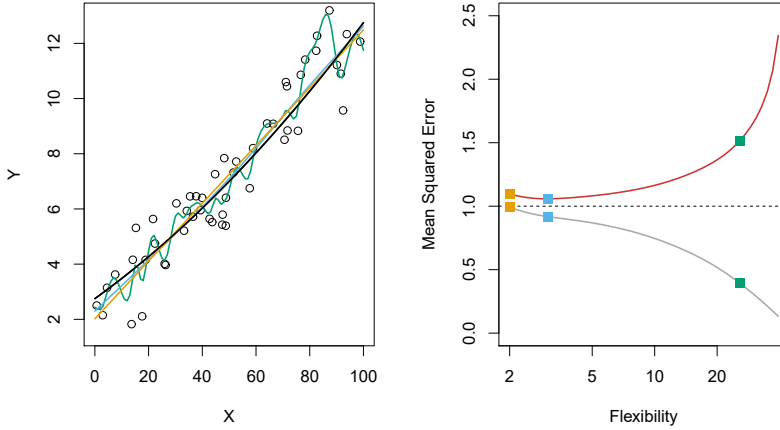
đường tương ứng ở ô bên trái. Một đường cong bị ràng buộc nhiều hơn, do đó trơn hơn, có ít degrees of freedom hơn một đường uốn lượn. Lưu ý rằng trong Hình 2.9, linear regression nằm ở đầu có nhiều ràng buộc nhất, với hai degrees of freedom. Training MSE giảm đơn điệu khi độ linh hoạt tăng. Trong ví dụ này, f thực là phi tuyến, nên linear fit màu cam không đủ linh hoạt để ước lượng tốt f . Đường xanh lá có training MSE thấp nhất trong ba phương pháp, vì nó là đường linh hoạt nhất trong ba đường được fit ở ô bên trái.

Trong ví dụ này, ta biết hàm f thực, nên cũng có thể tính test MSE trên một test set rất lớn như một hàm của độ linh hoạt. (Tất nhiên, nói chung f chưa biết nên không thể làm điều này.) Test MSE được biểu diễn bằng đường đỏ ở ô bên phải của Hình 2.9. Tương tự training MSE, ban đầu test MSE giảm khi độ linh hoạt tăng. Tuy nhiên, đến một mức nào đó, test MSE chững lại rồi bắt đầu tăng. Vì vậy, cả đường cam và đường xanh lá đều có test MSE cao. Đường xanh lam đạt test MSE nhỏ nhất, điều không đáng ngạc nhiên vì ở ô bên trái của Hình 2.9, nó dường như ước lượng f tốt nhất. Đường đứt nét nằm ngang biểu thị $\text{Var}(\epsilon)$, irreducible error trong (2.3), tương ứng với test MSE thấp nhất có thể đạt được trong mọi phương pháp. Do đó, smoothing spline được biểu diễn bằng đường xanh lam gần với mức tối ưu.

Ở ô bên phải của Hình 2.9, khi độ linh hoạt của phương pháp statistical learning tăng, ta quan sát thấy training MSE giảm đơn điệu và test MSE có dạng chữ U. Đây là một tính chất nền tảng của statistical learning, đúng bất kể dataset cụ thể hay phương pháp thống kê được sử dụng. Khi độ linh hoạt của mô hình tăng, training MSE giảm, nhưng test MSE chưa chắc giảm. Khi một phương pháp cho training MSE nhỏ nhưng test MSE lớn, ta nói rằng nó đang *overfitting* dữ liệu. Điều này xảy ra vì quy trình statistical learning cố gắng quá mức để tìm quy luật trong training data, và có thể nhận nhầm những mẫu hình do ngẫu nhiên thành các tính chất thực sự của hàm f chưa biết. Khi overfit training data, test MSE sẽ rất lớn vì những quy luật mà phương pháp tưởng đã tìm thấy trong training data đơn giản là không tồn tại trong test data. Lưu ý rằng dù có overfitting hay không, ta hầu như luôn kỳ vọng training MSE nhỏ hơn test MSE, vì phần lớn phương pháp statistical learning trực tiếp hoặc gián tiếp tìm cách giảm thiểu training MSE. Overfitting nói riêng đến trường hợp một mô hình kém linh hoạt hơn lẽ ra sẽ cho test MSE nhỏ hơn.

Hình 2.10 đưa ra một ví dụ khác, trong đó f thực xấp xỉ tuyến tính. Ta lại thấy training MSE giảm đơn điệu khi độ linh hoạt tăng, và test MSE có dạng chữ U. Tuy nhiên, vì quan hệ thực gần tuyến tính, test MSE chỉ giảm nhẹ trước khi tăng trở lại, nên least squares fit màu cam tốt hơn đáng kể so với đường xanh lá rất linh hoạt. Cuối cùng, Hình 2.11 minh họa trường hợp f có tính phi tuyến cao. Các đường training MSE và test MSE vẫn có những dạng tổng quát như trước, nhưng giờ cả hai giảm nhanh trước khi test MSE bắt đầu tăng chậm.

Trong thực tế, ta thường tính training MSE tương đối dễ, nhưng việc ước lượng test MSE khó hơn đáng kể vì thường không có test data. Như ba ví dụ trên cho thấy, độ linh hoạt tương ứng với mô hình có test MSE nhỏ nhất có thể khác nhau đáng kể giữa các dataset. Xuyên suốt sách, chúng tôi



HÌNH 2.10. Các chi tiết giống Hình 2.9, nhưng dùng một hàm f thực khác, gần tuyến tính hơn nhiều. Trong trường hợp này, linear regression fit dữ liệu rất tốt.

thảo luận nhiều cách có thể dùng trong thực tế để ước lượng điểm cực tiểu này. Một phương pháp quan trọng là *cross-validation* (Chương 5), dùng training data để ước lượng test MSE.

cross-
validation

2.2.2 Bias-Variance Trade-Off

Dạng chữ U quan sát được ở các đường test MSE trong Hình 2.9–2.11 thực ra là kết quả của hai tính chất cạnh tranh nhau trong các phương pháp statistical learning. Dù chứng minh toán học nằm ngoài phạm vi cuốn sách, có thể chỉ ra rằng expected test MSE tại một giá trị x_0 đã cho luôn phân tích được thành tổng của ba đại lượng cơ bản: *variance* của $\hat{f}(x_0)$, bình phương *bias* của $\hat{f}(x_0)$ và variance của các error term ϵ . Nghĩa là,

variance
bias

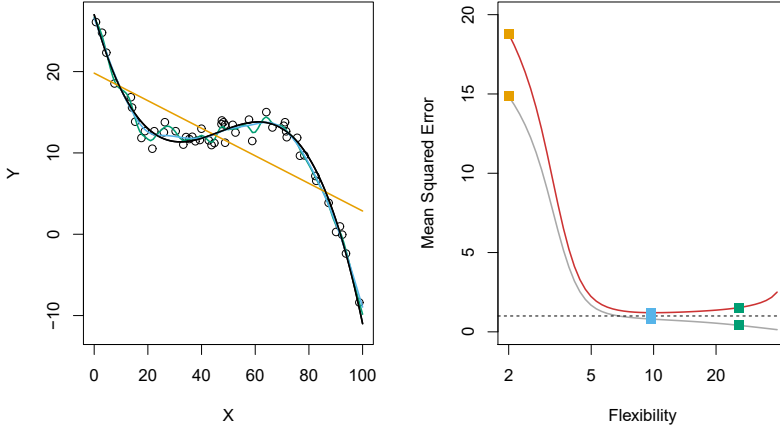
$$E(y_0 - \hat{f}(x_0))^2 = \text{Var}(\hat{f}(x_0)) + [\text{Bias}(\hat{f}(x_0))]^2 + \text{Var}(\epsilon). \quad (2.7)$$

Ở đây, $E(y_0 - \hat{f}(x_0))^2$ xác định *expected test MSE* tại x_0 , là test MSE trung bình mà ta sẽ thu được nếu liên tục ước lượng f bằng một số lượng lớn training set và đánh giá mỗi ước lượng tại x_0 . Expected test MSE tổng thể có thể được tính bằng cách lấy trung bình $E(y_0 - \hat{f}(x_0))^2$ trên mọi giá trị x_0 có thể có trong test set.

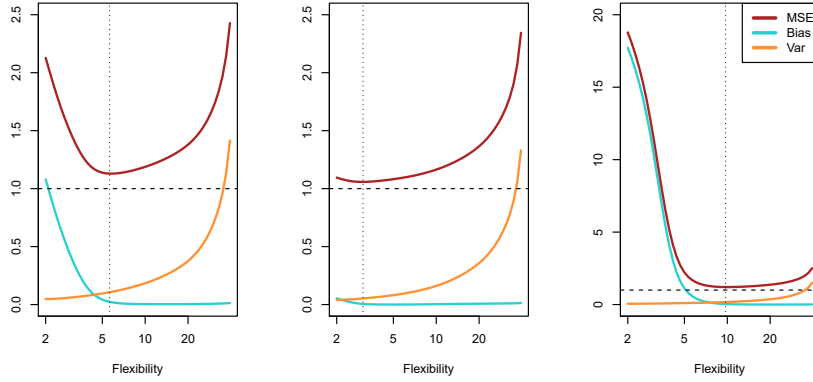
expected test
MSE

Phương trình 2.7 cho biết để giảm thiểu expected test error, ta cần chọn một phương pháp statistical learning đồng thời có variance thấp và bias thấp. Lưu ý rằng variance vốn là một đại lượng không âm, và bình phương bias cũng không âm. Vì thế, expected test MSE không bao giờ nhỏ hơn $\text{Var}(\epsilon)$, tức irreducible error trong (2.3).

Variance và bias của một phương pháp statistical learning có nghĩa là gì? *Variance* là mức độ $\hat{f}$ thay đổi nếu được ước lượng bằng một training dataset khác. Vì training data được dùng để fit phương pháp, các training dataset khác nhau sẽ cho $\hat{f}$ khác nhau. Nhưng lý tưởng nhất, ước lượng của f không nên thay đổi quá nhiều giữa các training set. Nếu một phương pháp có variance cao, những thay đổi nhỏ trong training data có thể khiến



HÌNH 2.11. Các chỉ tiết giống Hình 2.9, nhưng dùng một hàm f khác, rất khác với dạng tuyến tính. Trong trường hợp này, linear regression fit dữ liệu rất kém.



HÌNH 2.12. Bình phương bias (đường xanh lam), variance (đường cam), $\text{Var}(\epsilon)$ (đường đứt nét) và test MSE (đường đỏ) của ba dataset trong Hình 2.9–2.11. Đường chấm dọc chỉ mức độ linh hoạt tương ứng với test MSE nhỏ nhất.

$\hat{f}$ thay đổi lớn. Nói chung, các phương pháp thống kê linh hoạt hơn có variance cao hơn. Xét các đường xanh lá và cam trong Hình 2.9. Đường xanh lá linh hoạt bám rất sát các observations. Nó có variance cao vì thay đổi bất kỳ một điểm dữ liệu nào cũng có thể khiến ước lượng $\hat{f}$ thay đổi đáng kể. Ngược lại, đường least squares màu cam tương đối kém linh hoạt và có variance thấp, vì di chuyển một observation bất kỳ nhiều khả năng chỉ làm vị trí đường thẳng thay đổi đôi chút.

Mặt khác, *bias* là sai số được đưa vào khi xấp xỉ một bài toán thực tế, vốn có thể cực kỳ phức tạp, bằng một mô hình đơn giản hơn nhiều. Chẳng hạn, linear regression giả định mỗi quan hệ tuyến tính giữa Y với $X_1, X_2, \dots, X_p$. Khó có bài toán thực tế nào thực sự có một mối quan hệ tuyến tính đơn giản như vậy, nên thực hiện linear regression chắc chắn sẽ tạo ra một phần bias trong ước lượng f . Trong Hình 2.11, f thực có tính

phi tuyến đáng kể; vì vậy, dù có bao nhiêu training observations, ta cũng không thể tạo ra một ước lượng chính xác bằng linear regression. Nói cách khác, linear regression tạo ra bias cao trong ví dụ này. Tuy nhiên, trong Hình 2.10, f thực rất gần tuyến tính, nên với đủ dữ liệu, linear regression có thể tạo ra một ước lượng chính xác. Nhìn chung, các phương pháp linh hoạt hơn tạo ra bias thấp hơn.

Theo quy luật chung, khi dùng những phương pháp linh hoạt hơn, variance tăng và bias giảm. Tốc độ thay đổi tương đối của hai đại lượng này quyết định test MSE tăng hay giảm. Khi tăng độ linh hoạt của một lớp phương pháp, ban đầu bias thường giảm nhanh hơn mức variance tăng. Vì thế, expected test MSE giảm. Tuy nhiên, đến một thời điểm, tăng độ linh hoạt gần như không còn tác động đến bias nhưng bắt đầu làm variance tăng đáng kể. Khi đó, test MSE tăng. Ta đã quan sát dạng test MSE giảm rồi tăng này ở các ô bên phải của Hình 2.9–2.11.

Ba đồ thị trong Hình 2.12 minh họa Phương trình 2.7 cho các ví dụ ở Hình 2.9–2.11. Trong mỗi trường hợp, đường liền màu xanh lam biểu diễn bình phương bias ở các mức độ linh hoạt khác nhau, còn đường cam tương ứng với variance. Đường đứt nét nằm ngang biểu thị $\text{Var}(\epsilon)$, irreducible error. Cuối cùng, đường đỏ, tương ứng với MSE trên test set, là tổng của ba đại lượng này. Trong cả ba trường hợp, variance tăng và bias giảm khi độ linh hoạt tăng. Tuy nhiên, mức độ linh hoạt tương ứng với test MSE tối ưu khác nhau đáng kể giữa ba dataset, bởi bình phương bias và variance thay đổi với tốc độ khác nhau trong từng dataset. Ở ô bên trái của Hình 2.12, bias ban đầu giảm nhanh, khiến expected test MSE giảm mạnh lúc đầu. Ngược lại, ở ô giữa, f thực gần tuyến tính nên bias chỉ giảm ít khi độ linh hoạt tăng, và test MSE chỉ giảm nhẹ trước khi tăng nhanh theo variance. Cuối cùng, ở ô bên phải, bias giảm mạnh khi độ linh hoạt tăng vì f thực có tính phi tuyến rất cao. Variance cũng chỉ tăng rất ít khi độ linh hoạt tăng. Do đó, test MSE giảm đáng kể trước khi tăng nhẹ lúc mô hình tiếp tục trở nên linh hoạt hơn.

Mối quan hệ giữa bias, variance và test set MSE trong Phương trình 2.7, được thể hiện ở Hình 2.12, được gọi là *bias-variance trade-off*. Để một phương pháp statistical learning hoạt động tốt trên test set, cần có variance thấp đồng thời với bình phương bias thấp. Đây được gọi là một trade-off vì ta dễ tạo ra một phương pháp có bias cực thấp nhưng variance cao, chẳng hạn vẽ một đường cong đi qua mọi training observation, hoặc một phương pháp có variance rất thấp nhưng bias cao, như fit một đường nằm ngang vào dữ liệu. Thách thức là tìm ra phương pháp có cả variance và bình phương bias thấp. Trade-off này là một trong những chủ đề quan trọng nhất, lặp lại nhiều lần trong sách.

Trong tình huống thực tế, khi không quan sát được f , nói chung ta không thể tính tường minh test MSE, bias hay variance của một phương pháp statistical learning. Dù vậy, ta luôn nên ghi nhớ bias-variance trade-off. Trong sách, chúng tôi khảo sát những phương pháp cực kỳ linh hoạt, do đó về cơ bản có thể loại bỏ bias. Nhưng điều này không bảo đảm chúng tốt hơn một phương pháp đơn giản hơn nhiều như linear regression. Xét một ví dụ cực đoan: giả sử f thực là tuyến tính. Khi đó, linear regression không có bias, khiến phương pháp linh hoạt hơn rất khó cạnh tranh. Ngược lại, nếu

bias-variance
trade-off

f thực rất phi tuyến và ta có nhiều training observations, một cách tiếp cận rất linh hoạt có thể tốt hơn, như trong Hình 2.11. Trong Chương 5, chúng tôi thảo luận cross-validation, một cách ước lượng test MSE bằng training data.

2.2.3 Bối cảnh Classification

Đến đây, việc thảo luận độ chính xác của mô hình mới tập trung vào regression. Tuy nhiên, nhiều khái niệm đã gặp, như bias-variance trade-off, cũng áp dụng cho classification với một vài điều chỉnh do y_i không còn là quantitative. Giả sử ta muốn ước lượng f từ các training observations $\{(x_1, y_1), \dots, (x_n, y_n)\}$, trong đó $y_1, \dots, y_n$ là qualitative. Cách phổ biến nhất để định lượng độ chính xác của ước lượng $\hat{f}$ là training *error rate*, tức tỷ lệ lỗi khi áp dụng $\hat{f}$ vào các training observations:

$$\frac{1}{n} \sum_{i=1}^n I(y_i \neq \hat{y}_i). \quad (2.8)$$

Ở đây, $\hat{y}_i$ là class label dự đoán cho observation thứ i bằng $\hat{f}$. Còn $I(y_i \neq \hat{y}_i)$ là một *indicator variable*, bằng 1 nếu $y_i \neq \hat{y}_i$ và bằng không nếu $y_i = \hat{y}_i$. Nếu $I(y_i \neq \hat{y}_i) = 0$, observation thứ i được phương pháp classification phân loại đúng; nếu không, nó bị phân loại sai. Vì vậy, Phương trình 2.8 tính tỷ lệ các trường hợp phân loại sai.

Phương trình 2.8 được gọi là *training error rate*, vì tính trên chính dữ liệu đã dùng để train classifier. Giống trong regression, điều ta quan tâm nhất là error rate khi áp dụng classifier vào những test observations chưa dùng để train. *Test error rate* gắn với một tập test observations dạng (x_0, y_0) được xác định bởi

$$\text{Ave}(I(y_0 \neq \hat{y}_0)), \quad (2.9)$$

trong đó $\hat{y}_0$ là class label dự đoán khi áp dụng classifier vào test observation có predictor x_0 . Một classifier tốt là classifier có test error (2.9) nhỏ nhất.

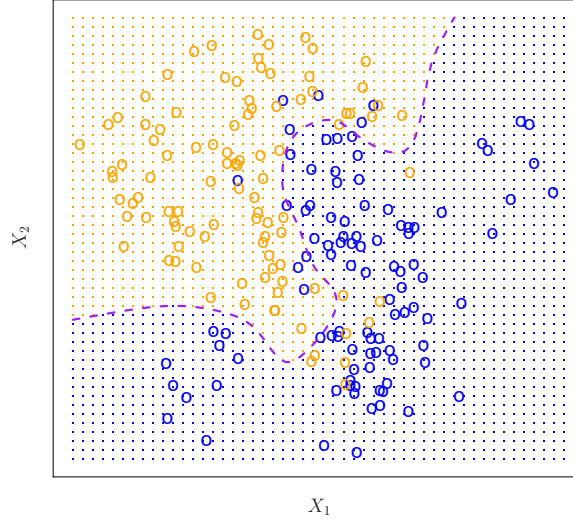
Bayes Classifier

Có thể chỉ ra rằng, dù chứng minh nằm ngoài phạm vi sách, test error rate trong (2.9) được giảm thiểu, xét trung bình, bởi một classifier rất đơn giản: gán mỗi observation vào class có khả năng xảy ra cao nhất khi biết các giá trị predictor của nó. Nói cách khác, ta chỉ cần gán một test observation có predictor vector x_0 vào class j sao cho

$$\Pr(Y = j \mid X = x_0) \quad (2.10)$$

lớn nhất. Lưu ý rằng (2.10) là một *conditional probability*: xác suất $Y = j$ khi đã quan sát predictor vector x_0 . Classifier rất đơn giản này được gọi là *Bayes classifier*. Trong bài toán hai class, chỉ có hai giá trị response có thể có, chẳng hạn class 1 hoặc class 2, Bayes classifier dự đoán class 1 nếu $\Pr(Y = 1 \mid X = x_0) > 0.5$, và class 2 trong trường hợp còn lại.

Hình 2.13 đưa ra một ví dụ dùng dataset mô phỏng trong không gian hai chiều gồm các predictor X_1 và X_2 . Những vòng tròn màu cam và xanh



HÌNH 2.13. Một dataset mô phỏng gồm 100 observations trong mỗi nhóm của hai nhóm, thể hiện bằng màu xanh lam và cam. Đường đứt nét tím biểu diễn Bayes decision boundary. Lưới nền cam chỉ vùng mà một test observation được gán vào class cam, còn lưới nền xanh chỉ vùng mà một test observation được gán vào class xanh.

lam tương ứng với các training observations thuộc hai class khác nhau. Ở mỗi giá trị X_1 và X_2 , xác suất response thuộc class cam hay xanh là khác nhau. Vì đây là dữ liệu mô phỏng, ta biết cách dữ liệu được tạo ra và có thể tính conditional probability ở từng giá trị X_1, X_2 . Vùng nền cam là tập các điểm có $\Pr(Y = \text{orange} \mid X)$ lớn hơn 50%, còn vùng nền xanh chỉ các điểm có xác suất đó dưới 50%. Đường đứt nét tím biểu diễn những điểm có xác suất đúng bằng 50%. Đây được gọi là *Bayes decision boundary*. Dự đoán của Bayes classifier được quyết định bởi Bayes decision boundary: observation nằm về phía cam của đường biên sẽ được gán vào class cam, và tương tự, observation nằm về phía xanh sẽ được gán vào class xanh.

Bayes decision boundary

Bayes classifier tạo ra test error rate thấp nhất có thể, gọi là *Bayes error rate*. Vì Bayes classifier luôn chọn class làm (2.10) lớn nhất, error rate tại $X = x_0$ sẽ là $1 - \max_j \Pr(Y = j \mid X = x_0)$. Nói chung, Bayes error rate tổng thể được xác định bởi

Bayes error rate

$$1 - \mathbb{E} \left[\max_j \Pr(Y = j \mid X) \right], \quad (2.11)$$

trong đó phép lấy kỳ vọng tính trung bình xác suất trên mọi giá trị có thể của X . Với dữ liệu mô phỏng đang xét, Bayes error rate là 0.133. Giá trị này lớn hơn không vì các class chồng lấn trong tổng thể thực, nghĩa là $\max_j \Pr(Y = j \mid X = x_0) < 1$ tại một số giá trị x_0 . Bayes error rate tương tự irreducible error đã thảo luận trước đó.

K-Nearest Neighbors

Về lý thuyết, ta luôn muốn dự đoán qualitative response bằng Bayes classifier. Nhưng với dữ liệu thực, ta không biết conditional distribution của Y khi biết X , nên không thể tính Bayes classifier. Vì vậy, Bayes classifier đóng vai trò một chuẩn tham chiếu lý tưởng, không thể đạt tới, để so sánh các phương pháp khác. Nhiều cách tiếp cận tìm cách ước lượng conditional distribution của Y khi biết X , rồi gán một observation vào class có xác suất ước lượng cao nhất. Một phương pháp như vậy là classifier *K-nearest neighbors* (KNN). Cho một số nguyên dương K và một test observation x_0 , trước tiên KNN classifier xác định K điểm trong training data gần x_0 nhất, ký hiệu là $\mathcal{N}_0$. Sau đó, nó ước lượng conditional probability của class j bằng tỷ lệ các điểm trong $\mathcal{N}_0$ có response bằng j :

K-nearest
neighbors

$$\Pr(Y = j \mid X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j). \quad (2.12)$$

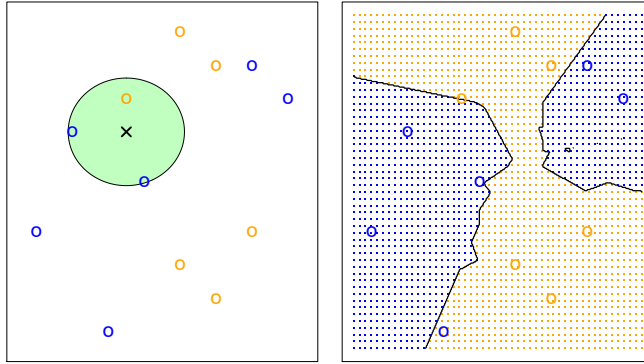
Cuối cùng, KNN phân loại test observation x_0 vào class có xác suất trong (2.12) lớn nhất.

Hình 2.14 minh họa cách tiếp cận KNN. Ở ô bên trái, chúng tôi vẽ một training dataset nhỏ gồm sáu observations xanh và sáu observations cam. Mục tiêu là dự đoán cho điểm được đánh dấu bằng dấu chữ thập đen. Giả sử chọn $K = 3$. Khi đó, KNN trước hết xác định ba observations gần dấu chữ thập nhất. Vùng lân cận này được biểu diễn bằng một đường tròn. Nó gồm hai điểm xanh và một điểm cam, cho xác suất ước lượng bằng $2/3$ với class xanh và $1/3$ với class cam. Do đó, KNN dự đoán dấu chữ thập đen thuộc class xanh. Ở ô bên phải của Hình 2.14, chúng tôi áp dụng KNN với $K = 3$ tại mọi giá trị có thể của X_1, X_2 và vẽ KNN decision boundary tương ứng.

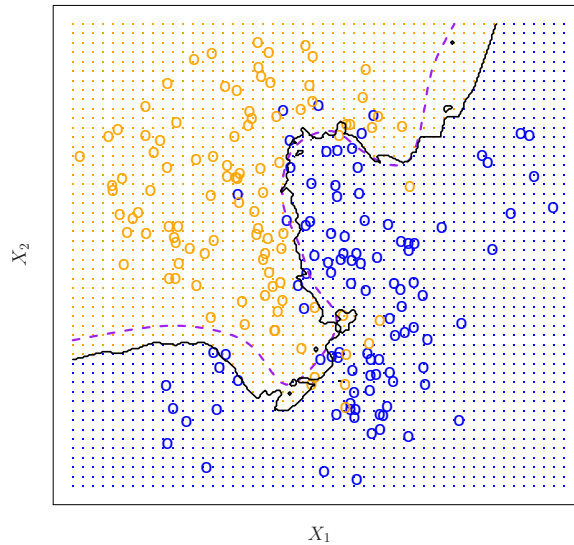
Dù rất đơn giản, KNN thường tạo được các classifier gần với Bayes classifier tối ưu một cách đáng ngạc nhiên. Hình 2.15 thể hiện KNN decision boundary với $K = 10$, áp dụng cho dataset mô phỏng lớn hơn ở Hình 2.13. Lưu ý rằng dù KNN classifier không biết phân phối thực, KNN decision boundary vẫn rất gần đường biên của Bayes classifier. Test error rate của KNN là 0.1363, gần với Bayes error rate bằng 0.1304.

Việc chọn K ảnh hưởng mạnh đến KNN classifier thu được. Hình 2.16 biểu diễn hai KNN fit trên dữ liệu mô phỏng ở Hình 2.13, với $K = 1$ và $K = 100$. Khi $K = 1$, decision boundary quá linh hoạt và tìm thấy những mẫu hình trong dữ liệu không tương ứng với Bayes decision boundary. Đây là classifier có bias thấp nhưng variance rất cao. Khi K tăng, phương pháp kém linh hoạt hơn và tạo ra decision boundary gần tuyến tính. Đây là classifier có variance thấp nhưng bias cao. Trên dataset mô phỏng này, cả $K = 1$ lẫn $K = 100$ đều không dự đoán tốt: test error rate tương ứng là 0.1695 và 0.1925.

Giống trong regression, training error rate và test error rate không có mối liên hệ mạnh. Với $K = 1$, training error rate của KNN bằng 0, nhưng test error rate có thể khá cao. Nói chung, khi dùng các phương pháp classification linh hoạt hơn, training error rate giảm nhưng test error rate chưa chắc giảm. Trong Hình 2.17, chúng tôi biểu diễn test error và training error của KNN theo $1/K$. Khi $1/K$ tăng, phương pháp linh hoạt hơn. Giống



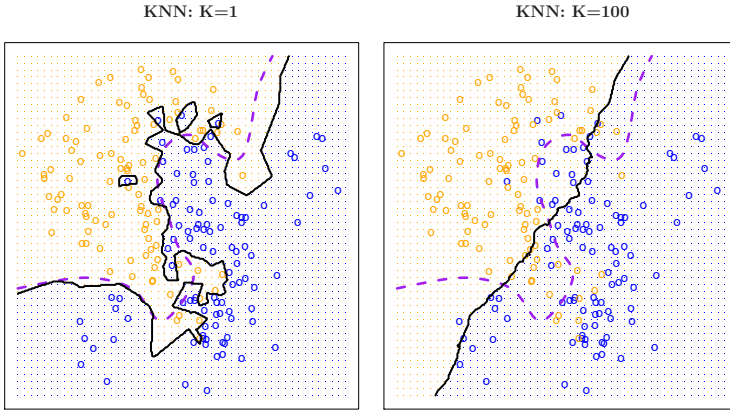
HÌNH 2.14. Cách tiếp cận KNN với $K = 3$, minh họa trong tình huống đơn giản gồm sáu observations xanh và sáu observations cam. Trái: Test observation cần dự đoán class label được thể hiện bằng dấu chữ thập đen. Ba điểm gần test observation nhất được xác định, và test observation được dự đoán thuộc class xuất hiện nhiều nhất, ở đây là xanh lam. Phải: KNN decision boundary của ví dụ được vẽ màu đen. Lưới xanh chỉ vùng mà test observation được gán vào class xanh, còn lưới cam chỉ vùng mà test observation được gán vào class cam.



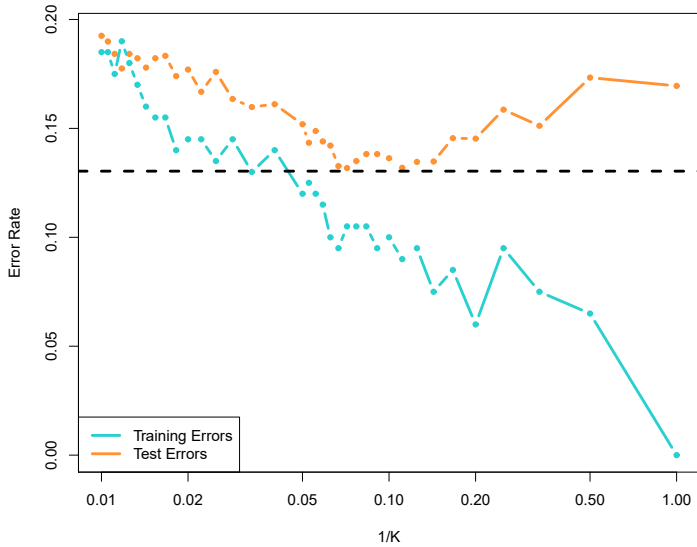
HÌNH 2.15. Đường cong đen biểu thị KNN decision boundary trên dữ liệu ở Hình 2.13, với $K = 10$. Bayes decision boundary được thể hiện bằng đường đứt nét tím. Hai decision boundary của KNN và Bayes rất giống nhau.

trong regression, training error rate liên tục giảm khi độ linh hoạt tăng. Tuy nhiên, test error có dạng chữ U đặc trưng: ban đầu giảm, đạt cực tiểu tại khoảng $K = 10$, rồi tăng trở lại khi phương pháp quá linh hoạt và overfit.

Trong cả regression lẫn classification, lựa chọn đúng độ linh hoạt có vai trò quyết định đối với thành công của bất kỳ phương pháp statistical learning nào. Bias-variance trade-off và dạng chữ U của test error khiến



HÌNH 2.16. So sánh các KNN decision boundary (đường liền màu đen) thu được với $K = 1$ và $K = 100$ trên dữ liệu ở Hình 2.13. Với $K = 1$, decision boundary quá linh hoạt, còn với $K = 100$, nó không đủ linh hoạt. Bayes decision boundary được thể hiện bằng đường đứt nét tím.



HÌNH 2.17. Training error rate của KNN (màu xanh, 200 observations) và test error rate (màu cam, 5,000 observations) trên dữ liệu ở Hình 2.13, khi độ linh hoạt tăng, được đánh giá bằng $1/K$ trên thang log, hay tương đương khi số lượng neighbor K giảm. Đường đứt nét đen biểu thị Bayes error rate. Các đường có dạng nhấp nhô do training dataset có kích thước nhỏ.

đây có thể là nhiệm vụ khó khăn. Trong Chương 5, chúng tôi quay lại chủ đề này và thảo luận nhiều phương pháp ước lượng test error rate, qua đó lựa chọn độ linh hoạt tối ưu cho một phương pháp statistical learning đã cho.

2.3 Lab: Giới thiệu Python

2.3.1 Bắt đầu

Để thực hiện các lab trong sách, bạn cần hai thứ:

1. Một bản cài đặt **Python3**, phiên bản Python được dùng trong các lab.
2. Quyền truy cập **Jupyter**, một giao diện Python rất phổ biến, chạy code thông qua tệp gọi là *notebook*.

notebook

Bạn có thể tải và cài đặt **Python3** theo hướng dẫn tại anaconda.com.

Có nhiều cách để sử dụng **Jupyter**. Sau đây là một vài cách:

1. Dùng dịch vụ Colaboratory của Google: colab.research.google.com/.
2. Dùng **JupyterHub**, tại jupyter.org/hub.
3. Dùng bản cài đặt **jupyter** của riêng bạn. Hướng dẫn cài đặt có tại jupyter.org/install.

Vui lòng xem thông tin cập nhật về cách thiết lập Python và Jupyter trên máy tính tại trang tài nguyên Python trên website của sách, statlearning.com.

Bạn cần cài package **ISLP**, cho phép truy cập các dataset và những hàm được chúng tôi xây dựng riêng. Trong terminal của macOS hoặc Linux, nhập `pip install ISLP`; lệnh này cũng cài phần lớn các package khác cần dùng trong lab. Trang tài nguyên Python có liên kết đến website tài liệu của **ISLP**.

Để thực hiện lab này, tải tệp `Ch2-statlearn-lab.ipynb` từ trang tài nguyên Python. Sau đó chạy lệnh sau tại command line: `jupyter lab Ch2-statlearn-lab.ipynb`.

Nếu dùng Windows, bạn có thể mở anaconda từ menu Start rồi làm theo các liên kết. Chẳng hạn, để cài **ISLP** và chạy lab này, bạn có thể thực hiện các lệnh trên trong anaconda shell.

2.3.2 Các lệnh cơ bản

Trong lab này, chúng tôi giới thiệu một số lệnh Python đơn giản. Để tìm thêm tài nguyên về Python nói chung, bạn đọc có thể tham khảo hướng dẫn tại docs.python.org/3/tutorial/.

Giống phần lớn ngôn ngữ lập trình, Python dùng *function* để thực hiện các phép toán. Để chạy một function tên `fun`, ta nhập `fun(input1, input2)`, trong đó các đầu vào (hay *argument*) `input1` và `input2` chỉ dẫn Python cách chạy function. Một function có thể có số đầu vào bất kỳ. Chẳng hạn, function `print()` xuất biểu diễn dạng văn bản của tất cả các argument ra console. `print()`

```
In [1]: print('fit a model with', 11, 'variables')
```

```
fit a model with 11 variables
```

Lệnh sau cung cấp thông tin về function `print()`.

```
In [2]: print?
```

Cộng hai số nguyên trong Python khá trực quan.

```
In [3]: 3 + 5
```

```
Out[3]: 8
```

Trong Python, dữ liệu văn bản được xử lý bằng *string*. Chẳng hạn, `string` `"hello"` và `'hello'` là các string. Ta có thể nối chúng bằng dấu cộng `+`.

```
In [4]: "hello" + " " + "world"
```

```
Out[4]: 'hello world'
```

String thực ra là một kiểu *sequence*: tên gọi chung cho một danh sách có thứ tự. Ba kiểu sequence quan trọng nhất là list, tuple và string. Bây giờ chúng tôi giới thiệu list.

Lệnh sau chỉ dẫn Python ghép các số 3, 4 và 5 lại, rồi lưu chúng dưới dạng một *list* tên `x`. Khi nhập `x`, ta nhận lại list đó. list

```
In [5]: x = [3, 4, 5]
x
```

```
Out[5]: [3, 4, 5]
```

Lưu ý rằng ta đã dùng dấu ngoặc vuông `[]` để tạo list này.

Ta thường muốn cộng hai tập hợp số với nhau. Có vẻ hợp lý khi thử đoạn code sau, dù nó sẽ không cho kết quả mong muốn.

```
In [6]: y = [4, 9, 7]
x + y
```

```
Out[6]: [3, 4, 5, 4, 9, 7]
```

Kết quả có thể hơi trái trực giác: tại sao Python không cộng các phần tử của hai list theo từng vị trí? Trong Python, list chứa những object bất kỳ, và phép cộng giữa các list được thực hiện bằng *concatenation*. Thực ra, concatenation chính là hành vi ta đã thấy khi nhập `"hello" + " " + "world"`.

Ví dụ này phản ánh việc Python là một ngôn ngữ lập trình đa dụng. Phần lớn chức năng chuyên về dữ liệu của Python đến từ các package khác, nổi bật là `numpy` và `pandas`. Trong mục tiếp theo, chúng tôi giới thiệu package `numpy`. Xem docs.scipy.org/doc/numpy/user/quickstart.html để biết thêm về `numpy`.

2.3.3 Giới thiệu Numerical Python

Như đã nói, sách sử dụng các chức năng có trong library, hay package, **numpy**. Package là một tập hợp các module không nhất thiết có trong bản phân phối Python cơ bản. Tên **numpy** là cách viết tắt của “numerical Python”.

Để sử dụng **numpy**, trước hết ta phải import package này.

```
In [7]: import numpy as np
```

Ở dòng trên, ta đặt tên **np** cho module **numpy**; đây là cách viết tắt giúp tham chiếu dễ hơn.

Trong **numpy**, **array** là tên gọi chung cho một tập hợp số nhiều chiều. Ta dùng function **np.array()** để định nghĩa **x** và **y**, là các array một chiều, tức vector.

```
In [8]: x = np.array([3, 4, 5])
        y = np.array([4, 9, 7])
```

Nếu quên gọi **import numpy as np** trước đó, lời gọi **np.array()** sẽ báo lỗi. Cú pháp **np.array()** cho biết ta đang gọi một function thuộc module **numpy**, được viết tắt là **np**.

Giờ đây, khi cộng **x** và **y**, ta thu được kết quả hợp lý hơn, khác với kết quả khi cộng hai list trước đó.

```
In [9]: x + y
```

```
Out[9]: array([ 7, 13, 12])
```

Trong **numpy**, matrix thường được biểu diễn bằng array hai chiều, còn vector bằng array một chiều.¹ Ta có thể tạo một array hai chiều như sau.

```
In [10]: x = np.array([[1, 2], [3, 4]])
        x
```

```
Out[10]: array([[1, 2],
               [3, 4]])
```

Object **x** có một số *attribute*, tức những object gắn với nó. Để truy cập một attribute của **x**, ta nhập **x.attribute**, thay **attribute** bằng tên attribute mong muốn. Chẳng hạn, ta có thể truy cập attribute **ndim** của **x** như sau.

```
In [11]: x.ndim
```

```
Out[11]: 2
```

Kết quả cho biết **x** là array hai chiều. Tương tự, **x.dtype** là attribute *data type* của object **x**. Nó cho biết **x** gồm các số nguyên 64 bit:

¹Ta cũng có thể tạo matrix bằng **np.matrix()**. Tuy nhiên, trong các lab của sách, chúng tôi sẽ dùng **np.array()** để tạo matrix.

```
In [12]: x.dtype
```

```
Out[12]: dtype('int64')
```

Tại sao `x` gồm các số nguyên? Vì ta tạo `x` bằng cách chỉ truyền các số nguyên vào function `np.array()`. Nếu truyền vào bất kỳ số có phần thập phân nào, ta sẽ nhận được một array các số *floating point*, tức số thực. floating

```
In [13]: np.array([[1, 2], [3.0, 4]]).dtype
```

```
Out[13]: dtype('float64')
```

Nhập `fun?` sẽ hiển thị tài liệu của function `fun`, nếu có. Hãy thử với `np.array`.

```
In [14]: np.array?
```

Tài liệu cho biết ta có thể dùng argument `dtype` để tạo một array có kiểu dấu phẩy động.

```
In [15]: np.array([[1, 2], [3, 4]], float).dtype
```

```
Out[15]: dtype('float64')
```

Array `x` có hai chiều. Attribute `shape` cho biết số hàng và số cột của nó.

```
In [16]: x.shape
```

```
Out[16]: (2, 2)
```

Method là một function gắn với một object. Chẳng hạn, với array `x`, biểu thức `x.sum()` tính tổng tất cả các phần tử, bằng method `sum()` dành cho array. Lờ gọi `x.sum()` tự động truyền `x` vào làm argument đầu tiên của method `sum()`. Method
.sum()

```
In [17]: x = np.array([1, 2, 3, 4])
x.sum()
```

```
Out[17]: 10
```

Ta cũng có thể tính tổng bằng `np.sum(x)`.

```
In [18]: x = np.array([1, 2, 3, 4])
np.sum(x)
```

```
Out[18]: 10
```

Xét một ví dụ khác: method `reshape()` trả về một array mới có cùng các phần tử với `x`, nhưng khác shape. Ta thực hiện bằng cách truyền một *tuple* .reshape()
tuple

vào lời gọi `reshape()`, ở đây là `(2, 3)`. Tuple này chỉ định rằng ta muốn tạo array hai chiều có 2 hàng và 3 cột.²

Trong đoạn code sau, ký tự `\n` tạo một dòng mới.

```
In [19]: x = np.array([1, 2, 3, 4, 5, 6])
print('beginning x:\n', x)
x_reshape = x.reshape((2, 3))
print('reshaped x:\n', x_reshape)
```

```
beginning x:
[1 2 3 4 5 6]
reshaped x:
[[1 2 3]
 [4 5 6]]
```

Kết quả trên cho thấy array của `numpy` được xác định dưới dạng một sequence các hàng. Cách này được gọi là *row-major ordering*, đối lập với *column-major ordering*.

Python, và do đó cả `numpy`, đánh chỉ số từ 0. Nghĩa là để truy cập phần tử ở góc trên bên trái của `x_reshape`, ta nhập `x_reshape[0,0]`.

```
In [20]: x_reshape[0, 0]
```

```
Out[20]: 1
```

Tương tự, `x_reshape[1,2]` trả về phần tử ở hàng thứ hai, cột thứ ba của `x_reshape`.

```
In [21]: x_reshape[1, 2]
```

```
Out[21]: 6
```

Tương tự, `x[2]` trả về phần tử thứ ba của `x`.

Bây giờ, hãy sửa phần tử ở góc trên bên trái của `x_reshape`. Thật bất ngờ, ta thấy phần tử đầu tiên của `x` cũng thay đổi!

²Giống list, tuple biểu diễn một sequence các object. Tại sao cần nhiều hơn một cách tạo sequence? Tuple và list có vài điểm khác nhau, nhưng có lẽ quan trọng nhất là các phần tử của tuple không thể bị sửa đổi, còn các phần tử của list thì có thể.

```
In [22]: print('x before we modify x_reshape:\n', x)
print('x_reshape before we modify x_reshape:\n', x_reshape)
x_reshape[0, 0] = 5
print('x_reshape after we modify its top left element:\n',
      x_reshape)
print('x after we modify top left element of x_reshape:\n', x)
```

```
Out[22]: x before we modify x_reshape:
[1 2 3 4 5 6]
x_reshape before we modify x_reshape:
[[1 2 3]
 [4 5 6]]
x_reshape after we modify its top left element:
[[5 2 3]
 [4 5 6]]
x after we modify top left element of x_reshape:
[5 2 3 4 5 6]
```

Sửa `x_reshape` cũng làm thay đổi `x` vì hai object dùng chung một vùng bộ nhớ.

Ta vừa thấy rằng có thể sửa một phần tử của array. Vậy có sửa được tuple không? Hóa ra là không: cố làm như vậy sẽ gây ra một *exception*, hay *exception* lỗi.

```
In [23]: my_tuple = (3, 4, 5)
my_tuple[0] = 2
```

```
TypeError: 'tuple' object does not support item assignment
```

Bây giờ, chúng tôi điểm qua vài attribute hữu ích của array. Attribute `shape` chứa kích thước của array; nó luôn là một tuple. Attribute `ndim` trả về số chiều, còn `T` cho biết transpose của array.

```
In [24]: x_reshape.shape, x_reshape.ndim, x_reshape.T
```

```
Out[24]: ((2, 3),
          2,
          array([[5, 4],
                 [2, 5],
                 [3, 6]]))
```

Lưu ý rằng ba kết quả riêng lẻ `(2,3)`, `2` và `array([[5, 4], [2, 5], [3, 6]])` lại được trả về cùng nhau dưới dạng một tuple.

Ta thường muốn áp dụng các function lên array. Chẳng hạn, ta có thể tính căn bậc hai của các phần tử bằng function `np.sqrt()`:

`np.sqrt()`

```
In [25]: np.sqrt(x)
```

```
Out[25]: array([2.24, 1.41, 1.73, 2., 2.24, 2.45])
```

Ta cũng có thể bình phương các phần tử:

```
In [26]: x**2
```

```
Out[26]: array([25, 4, 9, 16, 25, 36])
```

Ta có thể tính căn bậc hai bằng cùng ký hiệu đó, nhưng nâng lên lũy thừa $1/2$ thay vì 2.

```
In [27]: x**0.5
```

```
Out[27]: array([2.24, 1.41, 1.73, 2., 2.24, 2.45])
```

Trong suốt cuốn sách, ta thường muốn tạo dữ liệu ngẫu nhiên. Function `np.random.normal()` tạo một vector các biến ngẫu nhiên normal. Ta có thể tìm hiểu thêm bằng cách xem trang trợ giúp, qua lời gọi `np.random.normal?`. Dòng đầu tiên của trang trợ giúp là `normal(loc=0.0, scale=1.0, size=None)`. Dòng *signature* này cho biết các argument `loc`, `scale` và `size` là *keyword argument*: ta truyền chúng theo tên, với thứ tự tùy ý.³ Giá trị mặc định của mean `loc` là 0, của standard deviation `scale` là 1. Nếu không chỉ định `size`, function trả về một số ngẫu nhiên duy nhất.

`np.random.
normal()`

signature

Lệnh sau tạo 50 biến ngẫu nhiên độc lập có phân phối $N(0, 1)$.

```
In [28]: x = np.random.normal(size=50)  
x
```

```
Out[28]: array([-1.19,  0.41,  0.9 , -0.44, -0.9 , -0.38,  0.13,  1.87,  
                -0.35,  1.16,  0.79, -0.97, -1.21,  0.06, -1.62, -0.6 ,  
                -0.77, -2.12,  0.38, -1.22, -0.06, -1.97, -1.74, -0.56,  
                1.7 , -0.95,  0.56,  0.35,  0.87,  0.88, -1.66, -0.32,  
                -0.3 , -1.36,  0.92, -0.31,  1.28, -1.94,  1.07,  0.07,  
                0.79, -0.46,  2.19, -0.27, -0.64,  0.85,  0.13,  0.46,  
                -0.09,  0.7 ])
```

Tiếp theo, ta tạo `y` bằng cách cộng vào mỗi phần tử của `x` một biến ngẫu nhiên độc lập có phân phối $N(50, 1)$.

```
In [29]: y = x + np.random.normal(loc=50, scale=1, size=50)
```

Function `np.corrcoef()` tính correlation matrix giữa `x` và `y`. Các phần tử ngoài đường chéo cho biết correlation giữa chúng.

`np.corrcoef()`

```
In [30]: np.corrcoef(x, y)
```

```
Out[30]: array([[1.   , 0.69],  
                [0.69, 1.   ]])
```

Nếu thực hiện đoạn code này trong Jupyter notebook, bạn sẽ thu được kết quả khác. Đó là vì mỗi lần gọi `np.random.normal()`, ta nhận được những giá trị khác nhau.

³Python cũng có *positional argument*. Chẳng hạn, xem `np.sum?`: argument đầu tiên, `a`, là array cần tính tổng và có thể được truyền vào mà không cần ghi tên. Các keyword argument như `axis` và `dtype` có thể được truyền theo thứ tự bất kỳ.

```
In [31]: print(np.random.normal(scale=5, size=2))
print(np.random.normal(scale=5, size=2))
```

```
Out[31]: [4.28  2.59]
[4.62 -2.54]
```

Để bảo đảm code cho chính xác cùng kết quả mỗi lần chạy, ta có thể đặt *random seed* bằng function `np.random.default_rng()`. Function này nhận một argument là số nguyên tùy ý do người dùng chọn. Nếu đặt random seed trước khi tạo dữ liệu ngẫu nhiên, chạy lại code sẽ cho cùng kết quả. Object `rng` về cơ bản có tất cả các method tạo số ngẫu nhiên có trong `np.random`. Vì vậy, để tạo dữ liệu normal, ta dùng `rng.normal()`.

```
In [32]: rng = np.random.default_rng(1303)
print(rng.normal(scale=5, size=2))
rng2 = np.random.default_rng(1303)
print(rng2.normal(scale=5, size=2))
```

```
Out[32]: [4.09 -1.07 ]
[4.09 -1.07 ]
```

Trong các lab của sách, chúng tôi dùng `np.random.default_rng()` mỗi khi thực hiện phép tính liên quan đến đại lượng ngẫu nhiên trong `numpy`. Về nguyên tắc, điều này cho phép bạn đọc tái lập chính xác các kết quả được trình bày. Tuy nhiên, khi có phiên bản `numpy` mới, có thể xuất hiện một vài khác biệt nhỏ giữa kết quả trong lab và kết quả từ `numpy`.

Các function `np.mean()`, `np.var()` và `np.std()` được dùng để tính mean, variance và standard deviation của array. Các function này cũng có sẵn dưới dạng method của array. `np.mean()`
`np.var()`
`np.std()`

```
In [33]: rng = np.random.default_rng(3)
y = rng.standard_normal(10)
np.mean(y), y.mean()
```

```
Out[33]: (-0.11, -0.11)
```

```
In [34]: np.var(y), y.var(), np.mean((y - y.mean())**2)
```

```
Out[34]: (2.72, 2.72, 2.72)
```

Lưu ý rằng theo mặc định, `np.var()` chia cho cỡ mẫu n , thay vì $n - 1$; xem argument `ddof` trong `np.var?`.

```
In [35]: np.sqrt(np.var(y)), np.std(y)
```

```
Out[35]: (1.65, 1.65)
```

Các function `np.mean()`, `np.var()` và `np.std()` cũng có thể được áp dụng cho các hàng và cột của matrix. Để minh họa, ta tạo một matrix 10×3 gồm các biến ngẫu nhiên $N(0, 1)$ và xét việc tính tổng theo hàng.

```
In [36]: X = rng.standard_normal((10, 3))
X
```

```
Out[36]: array([[ 0.23, -0.35, -0.28],
                [-0.67, -1.06, -0.39],
                [ 0.48, -0.24,  0.96],
                [-0.2 ,  0.02,  1.55],
                [ 0.55, -0.51, -0.18],
                [ 0.54,  1.94, -0.27],
                [-0.24,  1. , -0.89],
                [-0.29,  0.88,  0.58],
                [ 0.09,  0.67, -2.83],
                [ 1.02, -0.96, -1.67]])
```

Vì array được sắp theo row-major ordering, trục đầu tiên, tức `axis=0`, tương ứng với các hàng. Ta truyền argument này vào method `mean()` của object `X`. `.mean()`

```
In [37]: X.mean(axis=0)
```

```
Out[37]: array([0.15,  0.14, -0.34])
```

Lệnh sau cho cùng kết quả.

```
In [38]: X.mean(0)
```

```
Out[38]: array([0.15,  0.14, -0.34])
```

2.3.4 Đồ họa

Trong Python, người ta thường dùng library `matplotlib` để vẽ đồ thị. Tuy nhiên, vì Python không được viết riêng cho phân tích dữ liệu, việc vẽ đồ thị không phải là chức năng nội tại của ngôn ngữ. Ta sẽ dùng function `subplots()` của `matplotlib.pyplot` để tạo một figure và axes dùng để vẽ dữ liệu. Để xem thêm nhiều ví dụ về cách vẽ đồ thị trong Python, bạn đọc nên truy cập matplotlib.org/stable/gallery/. `matplotlib`

Trong `matplotlib`, một đồ thị gồm một *figure* và một hoặc nhiều *axes*. Có thể hình dung figure là tấm nền trống để hiển thị một hay nhiều đồ thị: đó là toàn bộ cửa sổ đồ họa. Axes chứa các thông tin quan trọng của từng đồ thị, chẳng hạn nhãn trục *x* và *y*, tiêu đề, cùng nhiều thông tin khác. (Lưu ý rằng trong `matplotlib`, từ *axes* không phải dạng số nhiều của *axis*: axes của một đồ thị chứa nhiều thông tin hơn chỉ riêng trục *x* và trục *y*.) `figure`
`axes`

Ta bắt đầu bằng cách import function `subplots()` từ `matplotlib`. Ta sẽ dùng function này xuyên suốt khi tạo figure. Function trả về một tuple có độ dài hai: một object figure và object axes tương ứng. Thông thường, ta truyền `figsize` dưới dạng keyword argument. Sau khi tạo axes, ta thử vẽ đồ thị đầu tiên bằng method `plot()` của nó. Để tìm hiểu thêm, nhập `ax.plot?`. `subplots()`
`.plot()`

```
In [39]: from matplotlib.pyplot import subplots
fig, ax = subplots(figsize=(8, 8))
x = rng.standard_normal(100)
y = rng.standard_normal(100)
ax.plot(x, y);
```

Ở đây, cần lưu ý rằng ta đã *unpack* tuple có độ dài hai do `subplots()` trả về thành hai biến riêng biệt, `fig` và `ax`. Người ta thường ưu tiên `unpack` thay vì đoạn code tương đương nhưng dài hơn một chút dưới đây:

```
In [40]: output = subplots(figsize=(8, 8))
fig = output[0]
ax = output[1]
```

Ta thấy ô lệnh trước đó tạo ra một line plot, là kiểu đồ thị mặc định. Để tạo scatterplot, ta truyền thêm một argument cho `ax.plot()`, chỉ định rằng các điểm cần được hiển thị bằng hình tròn.

```
In [41]: fig, ax = subplots(figsize=(8, 8))
ax.plot(x, y, 'o');
```

Các giá trị khác nhau của argument bổ sung này có thể tạo ra các đường với màu sắc và kiểu nét khác nhau.

Một cách khác là dùng function `ax.scatter()` để tạo scatterplot.

`.scatter()`

```
In [42]: fig, ax = subplots(figsize=(8, 8))
ax.scatter(x, y, marker='o');
```

Lưu ý rằng trong các khối code trên, ta kết thúc dòng cuối bằng dấu chấm phẩy. Điều này ngăn `ax.plot(x, y)` in văn bản ra notebook, nhưng vẫn cho phép tạo đồ thị. Nếu bỏ dấu chấm phẩy ở cuối, ta nhận được kết quả sau:

```
In [43]: fig, ax = subplots(figsize=(8, 8))
ax.scatter(x, y, marker='o')
```

```
Out[43]: <matplotlib.collections.PathCollection at 0x7fb3d9c8f310>
Figure(432x288)
```

Ở những phần tiếp theo, ta sẽ dùng dấu chấm phẩy cuối dòng mỗi khi phần văn bản được xuất ra không liên quan đến nội dung đang thảo luận.

Để đặt nhãn cho đồ thị, ta dùng các method `set_xlabel()`, `set_ylabel()` và `set_title()` của `ax`.

`.set_xlabel()`
`.set_ylabel()`
`.set_title()`

```
In [44]: fig, ax = subplots(figsize=(8, 8))
ax.scatter(x, y, marker='o')
ax.set_xlabel("this is the x-axis")
ax.set_ylabel("this is the y-axis")
ax.set_title("Plot of X vs Y");
```

Việc có thể truy cập chính object figure `fig` cho phép ta thay đổi một số thuộc tính rồi hiển thị lại. Ở đây, ta đổi kích thước từ `(8, 8)` thành `(12, 3)`.

```
fig.set_size_inches(12,3)
fig
```

Đôi khi ta muốn tạo nhiều đồ thị trong một figure. Có thể làm điều này bằng cách truyền thêm các argument cho `subplots()`. Dưới đây, ta tạo một lưới đồ thị 2×3 trong một figure có kích thước do argument `figsize` xác định. Trong những trường hợp này, axes của các đồ thị thường có liên hệ với nhau. Chẳng hạn, tất cả các đồ thị có thể dùng chung trục x . Function `subplots()` có thể tự xử lý tình huống đó khi nhận keyword argument `sharex=True`. Object axes dưới đây là một array trỏ đến các đồ thị khác nhau trong figure.

```
In [45]: fig, axes = subplots(nrows=2,
                             ncols=3,
                             figsize=(15, 5))
```

Bây giờ, ta tạo một scatterplot dùng 'o' ở cột thứ hai của hàng thứ nhất và một scatterplot dùng '+' ở cột thứ ba của hàng thứ hai.

```
In [46]: axes[0,1].plot(x, y, 'o')
axes[1,2].scatter(x, y, marker='+')
fig
```

Nhập `subplots?` để tìm hiểu thêm về `subplots()`.

Để lưu kết quả của `fig`, ta gọi method `savefig()` của nó. Argument `dpi` `.savefig()` là số điểm trên mỗi inch, dùng để xác định kích thước của figure tính bằng pixel.

```
In [47]: fig.savefig("Figure.png", dpi=400)
fig.savefig("Figure.pdf", dpi=200);
```

Ta có thể tiếp tục sửa `fig` qua từng bước cập nhật; chẳng hạn, thay đổi phạm vi trục x , lưu lại figure, và thậm chí hiển thị lại nó.

```
In [48]: axes[0,1].set_xlim([-1,1])
fig.savefig("Figure_updated.jpg")
fig
```

Bây giờ, ta tạo một số đồ thị phức tạp hơn. Method `ax.contour()` `.contour()` tạo *contour plot* để biểu diễn dữ liệu ba chiều, tương tự bản đồ địa hình. Nó `contour plot` nhận ba argument:

- Một vector các giá trị x (chiều thứ nhất),
- Một vector các giá trị y (chiều thứ hai), và
- Một matrix có các phần tử tương ứng với giá trị z (chiều thứ ba) tại mỗi cặp tọa độ (x,y) .

Để tạo x và y , ta dùng lệnh `np.linspace(a, b, n)`, trả về một vector gồm `np.linspace()` n số bắt đầu tại a và kết thúc tại b .

```
In [49]: fig, ax = subplots(figsize=(8, 8))
x = np.linspace(-np.pi, np.pi, 50)
y = x
f = np.multiply.outer(np.cos(y), 1 / (1 + x**2))
ax.contour(x, y, f);
```

Ta có thể tăng độ phân giải bằng cách thêm nhiều mức hơn vào hình.

```
In [50]: fig, ax = subplots(figsize=(8, 8))
ax.contour(x, y, f, levels=45);
```

Để tinh chỉnh kết quả của function `ax.contour()`, hãy xem tệp trợ giúp bằng cách nhập `?plt.contour`.

Method `ax.imshow()` tương tự `ax.contour()`, nhưng tạo một đồ thị mã hóa bằng màu, trong đó màu sắc phụ thuộc vào giá trị `z`. Đồ thị này được gọi là *heatmap*, đôi khi dùng để biểu diễn nhiệt độ trong dự báo thời tiết.

```
In [51]: fig, ax = subplots(figsize=(8, 8))
ax.imshow(f);
```

2.3.5 Sequence và ký hiệu *Slice*

Như đã thấy, function `np.linspace()` có thể được dùng để tạo một sequence các số.

```
In [52]: seq1 = np.linspace(0, 10, 11)
seq1
```

```
Out[52]: array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9., 10.])
```

Function `np.arange()` trả về một sequence các số cách nhau một khoảng `step`. Nếu không chỉ định `step`, giá trị mặc định là 1. Hãy tạo một sequence bắt đầu tại 0 và kết thúc tại 10.

```
In [53]: seq2 = np.arange(0, 10)
seq2
```

```
Out[53]: array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Tại sao kết quả trên không có 10? Điều này liên quan đến ký hiệu *slice* trong Python. Ký hiệu slice được dùng để đánh chỉ số cho các sequence như list, tuple và array. Giả sử ta muốn lấy các phần tử từ thứ tư đến thứ sáu, bao gồm cả hai đầu, của một string. Ta lấy một slice của string bằng ký hiệu chỉ số `[3:6]`.

```
In [54]: "hello world"[3:6]
```

```
Out[54]: 'lo '
```

Trong khối code trên, ký hiệu `3:6` là cách viết tắt của `slice(3,6)` khi dùng bên trong `[]`.


```
In [55]: "hello world"[slice(3,6)]
```

```
Out[55]: 'lo '
```

Có thể bạn đã nghĩ `slice(3,6)` sẽ trả về các ký tự từ thứ tư đến thứ bảy của string, vì Python bắt đầu đánh chỉ số từ 0. Nhưng nó chỉ trả về các ký tự từ thứ tư đến thứ sáu. Điều này cũng giải thích vì sao lệnh `np.arange(0, 10)` trước đó chỉ trả về các số nguyên từ 0 đến 9. Xem tài liệu [slice?](#) để biết những tùy chọn hữu ích khi tạo slice.

2.3.6 Đánh chỉ số dữ liệu

Trước hết, ta tạo một array `numpy` hai chiều.

```
In [56]: A = np.array(np.arange(16)).reshape((4, 4))
          A
```

```
Out[56]: array([[ 0,  1,  2,  3],
                [ 4,  5,  6,  7],
                [ 8,  9, 10, 11],
                [12, 13, 14, 15]])
```

Nhập `A[1,2]` sẽ lấy phần tử ở hàng thứ hai và cột thứ ba. (Như thường lệ, Python đánh chỉ số từ 0.)

```
In [57]: A[1,2]
```

Out [57]: 6

Số đầu tiên sau dấu ngoặc vuông mở [chỉ hàng, còn số thứ hai chỉ cột.

Đánh chỉ số hàng, cột và submatrix

Để chọn nhiều hàng cùng lúc, ta có thể truyền vào một list chỉ định các hàng cần chọn. Chẳng hạn, `[1, 3]` sẽ lấy hàng thứ hai và thứ tư:

```
In [58]: A[[1,3]]
```

```
Out[58]: array([[ 4,  5,  6,  7],
                 [12, 13, 14, 15]])
```

Để chọn cột thứ nhất và thứ ba, ta truyền `[0,2]` vào làm argument thứ hai trong dấu ngoặc vuông. Trong trường hợp này, ta cần truyền argument đầu tiên là `:` để chọn tất cả các hàng.

```
In [59]: A[:, [0, 2]]
```

```
Out[59]: array([[ 0,  2],
                [ 4,  6],
                [ 8, 10],
                [12, 14]])
```

Bây giờ, giả sử ta muốn chọn submatrix gồm hàng thứ hai và thứ tư, cùng cột thứ nhất và thứ ba. Đến đây, việc đánh chỉ số trở nên hơi phức tạp. Một cách nghĩ tự nhiên là dùng các list để lấy hàng và cột:

```
In [60]: A[[1,3],[0,2]]
```

```
Out[60]: array([ 4, 14])
```

Ồ, điều gì đã xảy ra? Ta nhận được một array một chiều có độ dài hai, giống hệt

```
In [61]: np.array([A[1,0],A[3,2]])
```

```
Out[61]: array([ 4, 14])
```

Tương tự, đoạn code sau cũng không lấy được submatrix gồm hàng thứ hai và thứ tư cùng cột thứ nhất, thứ ba và thứ tư:

```
In [62]: A[[1,3],[0,2,3]]
```

```
IndexError: shape mismatch: indexing arrays could not be broadcast
together with shapes (2,) (3,)
```

Ta có thể thấy vấn đề ở đây. Khi nhận hai list chỉ số, `numpy` hiểu rằng chúng cung cấp các cặp chỉ số i, j cho một loạt phần tử. Đó là lý do hai list phải có cùng độ dài. Tuy nhiên, điều này không đúng với ý định của ta, vì ta đang muốn lấy một submatrix.

Có một cách đơn giản như sau. Trước hết, ta tạo một submatrix bằng cách chọn một tập con các hàng của `A`, rồi ngay sau đó tạo tiếp một submatrix bằng cách chọn một tập con các cột.

```
In [63]: A[[1,3]][:,[0,2]]
```

```
Out[63]: array([[ 4,  6],
               [12, 14]])
```

Có những cách hiệu quả hơn để đạt được cùng kết quả.

Convenience function `np.ix_()` cho phép trích xuất một submatrix bằng các list, thông qua việc tạo một object *mesh* trung gian.

Convenience
function
`np.ix_()`
mesh

```
In [64]: idx = np.ix_([1,3],[0,2,3])
         A[idx]
```

```
Out[64]: array([[ 4,  6,  7],
               [12, 14, 15]])
```

Một cách khác là chọn tập con của matrix một cách hiệu quả bằng slice. Slice `1:4:2` lấy phần tử thứ hai và thứ tư của sequence, còn slice `0:3:2` lấy phần tử thứ nhất và thứ ba. (Thành phần thứ ba trong một slice là độ lớn bước nhảy.)

```
In [65]: A[1:4:2,0:3:2]
```

```
Out [65]: array([[ 4,  6],
                [12, 14]])
```

Tại sao ta có thể lấy trực tiếp một submatrix bằng slice nhưng không thể làm như vậy bằng list? Đó là vì chúng là những kiểu Python khác nhau và được **numpy** xử lý khác nhau. Slice có thể được dùng để lấy object từ những sequence bất kỳ, như string, list và tuple, còn việc dùng list để đánh chỉ số bị hạn chế hơn.

Boolean Indexing

Trong **numpy**, *Boolean* là kiểu có giá trị **True** hoặc **False**, cũng được biểu diễn lần lượt bằng 1 và 0. Dòng tiếp theo tạo một vector các số 0, được biểu diễn dưới dạng Boolean, có độ dài bằng kích thước chiều thứ nhất của **A**. Boolean

```
In [66]: keep_rows = np.zeros(A.shape[0], bool)
        keep_rows
```

```
Out [66]: array([False,  False,  False,  False])
```

Bây giờ, ta đặt hai phần tử thành **True**.

```
In [67]: keep_rows[[1,3]] = True
        keep_rows
```

```
Out [67]: array([False,  True,  False,  True])
```

Lưu ý rằng nếu xem như số nguyên, các phần tử của **keep_rows** giống với các giá trị trong **np.array([0,1,0,1])**. Dưới đây, ta dùng **==** để kiểm tra sự bằng nhau. Khi áp dụng cho hai array, phép toán **==** được thực hiện theo từng phần tử.

```
In [68]: np.all(keep_rows == np.array([0,1,0,1]))
```

```
Out [68]: True
```

(Ở đây, function **np.all()** kiểm tra xem tất cả các phần tử của một array có phải là **True** hay không. Một function tương tự, **np.any()**, có thể dùng để kiểm tra xem có phần tử nào của array là **True** hay không.) np.all()
np.any()

Tuy nhiên, dù **np.array([0,1,0,1])** và **keep_rows** bằng nhau theo **==**, chúng lại chọn các tập hàng khác nhau! Biểu thức thứ nhất lấy lần lượt hàng thứ nhất, thứ hai, thứ nhất và thứ hai của **A**.

```
In [69]: A[np.array([0,1,0,1])]
```

```
Out [69]: array([[0, 1, 2, 3],
                [4, 5, 6, 7],
                [0, 1, 2, 3],
                [4, 5, 6, 7]])
```

Ngược lại, `keep_rows` chỉ lấy hàng thứ hai và thứ tư của `A`, tức các hàng mà Boolean bằng `TRUE`.

```
In [70]: A[keep_rows]
```

```
Out[70]: array([[ 4,  5,  6,  7],
                [12, 13, 14, 15]])
```

Ví dụ này cho thấy `numpy` xử lý Boolean và số nguyên khác nhau.

Ta lại dùng function `np.ix_()` để tạo một mesh chứa hàng thứ hai và thứ tư, cùng cột thứ nhất, thứ ba và thứ tư. Lần này, ta áp dụng function cho các Boolean thay vì các list.

```
In [71]: keep_cols = np.zeros(A.shape[1], bool)
keep_cols[[0, 2, 3]] = True
idx_bool = np.ix_(keep_rows, keep_cols)
A[idx_bool]
```

```
Out[71]: array([[ 4,  6,  7],
                [12, 14, 15]])
```

Ta cũng có thể kết hợp một list với một array Boolean trong các argument của `np.ix_()`:

```
In [72]: idx_mixed = np.ix_([1,3], keep_cols)
A[idx_mixed]
```

```
Out[72]: array([[ 4,  6,  7],
                [12, 14, 15]])
```

Để biết thêm chi tiết về việc đánh chỉ số trong `numpy`, bạn đọc có thể tham khảo hướng dẫn `numpy` đã được nhắc đến trước đó.

2.3.7 Nạp dữ liệu

Dataset thường chứa nhiều kiểu dữ liệu khác nhau, và có thể có tên gắn với các hàng hoặc cột. Vì những lý do này, *data frame* thường là cách phù hợp nhất để chứa chúng. Có thể hình dung data frame là một sequence các array có cùng độ dài; đó chính là các cột. Các phần tử ở những array khác nhau được kết hợp để tạo thành một hàng. Library `pandas` cho phép tạo và làm việc với các object data frame.

Đọc một dataset

Bước đầu tiên của phần lớn các phân tích là đưa dataset vào Python. Trước khi thử nạp dataset, ta phải bảo đảm Python biết tìm tệp chứa nó ở đâu. Nếu tệp nằm cùng thư mục với tệp notebook này thì mọi thứ đã sẵn sàng. Nếu không, có thể dùng lệnh `os.chdir()` để đổi thư mục. (Bạn cần gọi `import os` trước khi gọi `os.chdir()`.)

Ta bắt đầu bằng cách đọc tệp `Auto.csv`, có trên website của sách. Đây là tệp dùng dấu phẩy để phân tách các giá trị, và có thể được đọc bằng `pd.read_csv():`

`pd.read_csv()`

```
In [73]: import pandas as pd
Auto = pd.read_csv('Auto.csv')
Auto
```

Website của sách cũng có một phiên bản dữ liệu này dùng khoảng trắng làm dấu phân tách, tên là `Auto.data`. Có thể đọc tệp đó như sau:

```
In [74]: Auto = pd.read_csv('Auto.data', delim_whitespace=True)
```

Cả `Auto.csv` lẫn `Auto.data` đều chỉ là tệp văn bản. Trước khi nạp dữ liệu vào Python, nên xem dữ liệu bằng trình soạn thảo văn bản hoặc phần mềm khác, chẳng hạn Microsoft Excel.

Bây giờ, ta xem cột của `Auto` tương ứng với biến `horsepower`:

```
In [75]: Auto['horsepower']
```

```
Out[75]: 0      130.0
1      165.0
2      150.0
3      150.0
4      140.0
...
392    86.00
393    52.00
394    84.00
395    79.00
396    82.00
Name: horsepower, Length: 397, dtype: object
```

Ta thấy `dtype` của cột này là `object`. Hóa ra, khi đọc dữ liệu, tất cả các giá trị trong cột `horsepower` đều được hiểu là string. Ta có thể tìm nguyên nhân bằng cách xem các giá trị duy nhất của cột.

```
In [76]: np.unique(Auto['horsepower'])
```

Để tiết kiệm chỗ, chúng tôi lược bỏ kết quả của khối code trước. Ta thấy thủ phạm là giá trị `?`, được dùng để mã hóa các giá trị thiếu.

Để khắc phục, ta cần truyền cho `pd.read_csv()` một argument tên `na_values`. Khi đó, mỗi lần `?` xuất hiện trong tệp, nó sẽ được thay bằng `np.nan`, nghĩa là *not a number*:

```
In [77]: Auto = pd.read_csv('Auto.data',
                           na_values=['?'],
                           delim_whitespace=True)
Auto['horsepower'].sum()
```

```
Out[77]: 40952.0
```

Attribute `Auto.shape` cho biết dữ liệu có 397 observations, tức hàng, và chín biến, tức cột.

```
In [78]: Auto.shape
```

```
Out[78]: (397, 9)
```

Có nhiều cách xử lý dữ liệu thiếu. Trong trường hợp này, vì chỉ có năm hàng chứa observations bị thiếu, ta chọn dùng method `Auto.dropna()` để đơn giản là loại bỏ các hàng đó. `.dropna()`

```
In [79]: Auto_new = Auto.dropna()
Auto_new.shape
```

```
Out[79]: (392, 9)
```

Cơ bản về cách chọn hàng và cột

Ta có thể dùng `Auto.columns` để kiểm tra tên các biến.

```
In [80]: Auto = Auto_new # overwrite the previous value
Auto.columns
```

```
Out[80]: Index(['mpg', 'cylinders', 'displacement', 'horsepower',
               'weight', 'acceleration', 'year', 'origin', 'name'],
              dtype='object')
```

Truy cập hàng và cột của data frame tương tự, nhưng không hoàn toàn giống, việc truy cập hàng và cột của array. Nhớ rằng argument đầu tiên của method `[]` luôn được áp dụng cho các hàng của array. Tương tự, truyền một slice vào method `[]` sẽ tạo ra một data frame có các hàng được xác định bởi slice đó:

```
In [81]: Auto[:3]
```

```
Out[81]:
```

	mpg	cylinders	displacement	horsepower	weight	...
0	18.0	8	307.0	130.0	3504.0	...
1	15.0	8	350.0	165.0	3693.0	...
2	18.0	8	318.0	150.0	3436.0	...

Tương tự, ta có thể dùng một array Boolean để chọn một tập con các hàng:

```
In [82]: idx_80 = Auto['year'] > 80
Auto[idx_80]
```

Tuy nhiên, nếu truyền một list các string vào method `[]`, ta nhận được một data frame chứa tập cột tương ứng.

```
In [83]: Auto[['mpg', 'horsepower']]
```

```
Out[83]:      mpg      horsepower
0       18.0      130.0
1       15.0      165.0
2       18.0      150.0
3       16.0      150.0
4       17.0      140.0
...      ...      ...
392     27.0      86.0
393     44.0      52.0
394     32.0      84.0
395     28.0      79.0
396     31.0      82.0
392 rows x 2 columns
```

Vì không chỉ định cột chỉ số khi nạp data frame, các hàng được gán nhãn bằng những số nguyên từ 0 đến 396.

```
In [84]: Auto.index
```

```
Out[84]: Int64Index([ 0,   1,   2,   3,   4,   5,   6,   7,   8,   9,
                    ...,
                    387, 388, 389, 390, 391, 392, 393, 394, 395, 396],
                  dtype='int64', length=392)
```

Ta có thể dùng method `set_index()` để đổi tên các hàng theo nội dung của `Auto['name']`.

```
In [85]: Auto_re = Auto.set_index('name')
Auto_re
```

```
Out[85]:      mpg  cylinders  displacement  ...
          name
chevrolet chevelle malibu    18.0         8         307.0  ...
buick skylark 32            15.0         8         350.0  ...
plymouth satellite          18.0         8         318.0  ...
amc rebel sst               16.0         8         304.0  ...
```

```
In [86]: Auto_re.columns
```

```
Out[86]: Index(['mpg', 'cylinders', 'displacement', 'horsepower',
                'weight', 'acceleration', 'year', 'origin'],
              dtype='object')
```

Ta thấy cột `'name'` không còn ở đó.

Bây giờ, khi index đã được đặt thành `name`, ta có thể truy cập các hàng của data frame theo tên bằng method `loc[]` của `Auto`:

```
.loc[]
```

```
In [87]: rows = ['amc rebel sst', 'ford torino']
Auto_re.loc[rows]
```

```
Out [87]:
```

	mpg	cylinders	displacement	horsepower	...
name					
amc rebel sst	16.0	8	304.0	150.0	...
ford torino	17.0	8	302.0	140.0	...

Thay vì dùng tên trong index, ta có thể lấy hàng thứ tư và thứ năm của `Auto` bằng method `iloc[]`:

`.iloc[]`

```
In [88]: Auto_re.iloc[[3,4]]
```

Ta cũng có thể dùng method này để lấy cột thứ nhất, thứ ba và thứ tư của `Auto_re`:

```
In [89]: Auto_re.iloc[:, [0,2,3]]
```

Ta có thể lấy hàng thứ tư và thứ năm, cùng cột thứ nhất, thứ ba và thứ tư, bằng một lời gọi `iloc[]` duy nhất:

```
In [90]: Auto_re.iloc[[3,4], [0,2,3]]
```

```
Out [90]:
```

	mpg	displacement	horsepower
name			
amc rebel sst	16.0	304.0	150.0
ford torino	17.0	302.0	140.0

Các giá trị trong index không nhất thiết phải duy nhất: trong data frame có một số xe cùng mang tên `ford galaxie 500`.

```
In [91]: Auto_re.loc['ford galaxie 500', ['mpg', 'origin']]
```

```
Out [91]:
```

	mpg	origin
name		
ford galaxie 500	15.0	1
ford galaxie 500	14.0	1
ford galaxie 500	14.0	1

Tìm hiểu thêm về cách chọn hàng và cột

Bây giờ, giả sử ta muốn tạo một data frame gồm `weight` và `origin` của tập con các xe có `year` lớn hơn 80, tức được sản xuất sau năm 1980. Để làm điều này, trước hết ta tạo một array Boolean đánh chỉ số các hàng. Method `loc[]` chấp nhận cả Boolean lẫn string:

```
In [92]: idx_80 = Auto_re['year'] > 80
Auto_re.loc[idx_80, ['weight', 'origin']]
```

Để viết gọn hơn, ta có thể dùng một function không tên gọi là `lambda`:

`lambda`

```
In [93]: Auto_re.loc[lambda df: df['year'] > 80, ['weight', 'origin']]
```


Lời gọi `lambda` tạo một function nhận một argument duy nhất, ở đây là `df`, và trả về `df['year'] > 80`. Vì function được tạo bên trong method `loc[]` của data frame `Auto_re`, chính data frame này sẽ được truyền vào làm argument. Xét thêm một ví dụ dùng `lambda`: giả sử ta muốn lấy tất cả các xe sản xuất sau năm 1980 và đi được hơn 30 mile trên mỗi gallon nhiên liệu:

```
In [94]: Auto_re.loc[lambda df: (df['year'] > 80) & (df['mpg'] > 30),
                ['weight', 'origin']]
```

Ký hiệu `&` thực hiện phép *and* theo từng phần tử. Xét một ví dụ khác: giả sử ta muốn lấy tất cả các xe Ford và Datsun có `displacement` nhỏ hơn 300. Ta kiểm tra xem mỗi giá trị `name` có chứa string `ford` hoặc `datsun` hay không bằng method `str.contains()` của attribute `index` trong data frame:

```
In [95]: Auto_re.loc[lambda df: (df['displacement'] < 300)
                        & (df.index.str.contains('ford')
                           | df.index.str.contains('datsun')),
                        ['weight', 'origin']]
```

Ở đây, ký hiệu `|` thực hiện phép *or* theo từng phần tử.

Như vậy, ta có một tập hợp phép toán mạnh để đánh chỉ số hàng và cột của data frame. Với truy vấn dựa trên số nguyên, dùng method `iloc[]`. Với phép chọn bằng string hoặc Boolean, dùng method `loc[]`. Với truy vấn dùng function để lọc hàng, dùng method `loc[]` và truyền một function, thường là `lambda`, vào argument dành cho hàng.

2.3.8 Vòng lặp For

Vòng lặp `for` là công cụ tiêu chuẩn trong nhiều ngôn ngữ, thực hiện lặp lại một khối code trong khi thay đổi các giá trị bên trong. Chẳng hạn, giả sử ta lặp qua các phần tử của một list và tính tổng của chúng.

```
In [96]: total = 0
for value in [3,2,19]:
    total += value
print('Total is: {}'.format(total))
```

Total is: 24

Phần code thụt vào bên dưới dòng chứa câu lệnh `for` được chạy với mỗi giá trị trong sequence được chỉ định ở câu lệnh `for`. Vòng lặp kết thúc khi ô lệnh kết thúc, hoặc khi code trở về cùng mức thụt dòng với câu lệnh `for` ban đầu. Ta thấy dòng cuối ở trên, dùng để in tổng, chỉ được thực hiện một lần sau khi vòng lặp `for` kết thúc. Có thể lồng các vòng lặp bằng cách thụt dòng thêm một mức.

```
In [97]: total = 0
for value in [2,3,19]:
    for weight in [3, 2, 1]:
        total += value * weight
print('Total is: {0}'.format(total))
```

```
Total is: 144
```

Ở trên, ta tính tổng trên mọi tổ hợp của **value** và **weight**. Ta cũng sử dụng ký hiệu *increment* trong Python: biểu thức **a += b** tương đương với **a = a + b**. Ngoài sự tiện lợi về cách viết, ký hiệu này còn có thể tiết kiệm thời gian trong những tác vụ tính toán nặng, khi không cần tạo tường minh giá trị trung gian của **a+b**.

Có lẽ một tác vụ thường gặp hơn là tính tổng trên các cặp (**value**, **weight**). Chẳng hạn, để tính giá trị trung bình của một biến ngẫu nhiên có thể nhận các giá trị 2, 3 hoặc 19 với xác suất tương ứng là 0.2, 0.3, 0.5, ta tính tổng có trọng số. Những tác vụ như vậy thường có thể được thực hiện bằng function **zip()**, lặp qua một sequence các tuple.

zip()

```
In [98]: total = 0
for value, weight in zip([2,3,19],
                        [0.2,0.3,0.5]):
    total += weight * value
print('Weighted average is: {0}'.format(total))
```

```
Weighted average is: 10.8
```

Định dạng string

Trong khối code trên, ta cũng in ra một string hiển thị tổng. Tuy nhiên, object **total** là một số nguyên chứ không phải string. Chèn giá trị của một đại lượng vào string là tác vụ phổ biến, được đơn giản hóa nhờ những công cụ định dạng string mạnh của Python. Nhiều tác vụ làm sạch dữ liệu liên quan đến việc thao tác và tạo string bằng chương trình.

Chẳng hạn, ta có thể muốn lặp qua các cột của một data frame và in tỷ lệ phần trăm giá trị thiếu trong từng cột. Hãy tạo một data frame **D** có các cột mà 20% số phần tử bị thiếu, tức được đặt thành **np.nan**. Ta tạo các giá trị trong **D** từ phân phối normal có mean 0 và variance 1 bằng **rng.standard_normal()**, rồi ghi đè một số phần tử ngẫu nhiên bằng **rng.choice()**.

np.nan

```
In [99]: rng = np.random.default_rng(1)
A = rng.standard_normal((127, 5))
M = rng.choice([0, np.nan], p=[0.8,0.2], size=A.shape)
A += M
D = pd.DataFrame(A, columns=['food',
                             'bar',
                             'pickle',
                             'snack',
                             'popcorn'])

D[:3]
```

```
Out [99]:      food      bar  pickle      snack  popcorn
0  0.345584  0.821618  0.330437 -1.303157      NaN
1      NaN -0.536953  0.581118  0.364572  0.294132
2      NaN  0.546713      NaN -0.162910 -0.482119
```

```
In [100]: for col in D.columns:
            template = 'Column "{0}" has {1:.2%} missing values'
            print(template.format(col,
                                   np.isnan(D[col]).mean()))
```

```
Column "food" has 16.54% missing values
Column "bar" has 25.98% missing values
Column "pickle" has 29.13% missing values
Column "snack" has 21.26% missing values
Column "popcorn" has 22.83% missing values
```

Ta thấy method `template.format()` nhận hai argument `{0}` và `{1:.2%}`, trong đó argument thứ hai kèm thông tin định dạng. Cụ thể, nó quy định argument thứ hai phải được biểu diễn dưới dạng phần trăm với hai chữ số thập phân.

Tài liệu tại docs.python.org/3/library/string.html có nhiều ví dụ hữu ích và phức tạp hơn.

2.3.9 Các cách tóm tắt bổ sung bằng đồ thị và số liệu

Ta có thể dùng các function `ax.plot()` hoặc `ax.scatter()` để biểu diễn các biến quantitative. Tuy nhiên, chỉ nhập tên biến sẽ gây ra thông báo lỗi, vì Python không biết phải tìm những biến đó trong dataset `Auto`.

```
In [101]: fig, ax = subplots(figsize=(8, 8))
ax.plot(horsepower, mpg, 'o');
```

```
NameError: name 'horsepower' is not defined
```

Ta có thể xử lý bằng cách truy cập trực tiếp các cột:

```
In [102]: fig, ax = subplots(figsize=(8, 8))
ax.plot(Auto['horsepower'], Auto['mpg'], 'o');
```

Một cách khác là dùng method `plot()` qua lời gọi `Auto.plot()`. Với `.plot()` method này, ta có thể truy cập các biến theo tên. Các method vẽ đồ thị của data frame trả về một object quen thuộc: axes. Ta có thể dùng nó để cập nhật đồ thị như đã làm trước đó:

```
In [103]: ax = Auto.plot.scatter('horsepower', 'mpg');
          ax.set_title('Horsepower vs. MPG')
```

Nếu muốn lưu figure chứa một axes đã cho, ta có thể tìm figure tương ứng bằng cách truy cập attribute `figure`:

```
In [104]: fig = ax.figure
          fig.savefig('horsepower_mpg.png');
```

Ta còn có thể yêu cầu data frame vẽ vào một object axes cụ thể. Trong trường hợp này, method `plot()` tương ứng sẽ trả về axes đã được sửa đổi mà ta truyền vào làm argument. Lưu ý rằng khi yêu cầu một lưới đồ thị một chiều, object `axes` cũng có một chiều. Ta đặt scatterplot vào ô giữa của một hàng gồm ba đồ thị trong cùng figure.

```
In [105]: fig, axes = subplots(ncols=3, figsize=(15, 5))
          Auto.plot.scatter('horsepower', 'mpg', ax=axes[1]);
```

Cũng lưu ý rằng có thể truy cập các cột của data frame dưới dạng attribute: hãy thử nhập `Auto.horsepower`.

Bây giờ, xét biến `cylinders`. Nhập `Auto.cylinders.dtype` cho thấy nó đang được xử lý như một biến quantitative. Tuy nhiên, vì biến này chỉ có một số ít giá trị có thể có, ta có thể muốn xem nó là qualitative. Dưới đây, ta thay cột `cylinders` bằng một phiên bản categorical của `Auto.cylinders`. Function `pd.Series()` có tên như vậy vì `pandas` thường được dùng trong các ứng dụng time series. `pd.Series()`

```
In [106]: Auto.cylinders = pd.Series(Auto.cylinders, dtype='category')
          Auto.cylinders.dtype
```

Vì `cylinders` giờ là qualitative, ta có thể biểu diễn nó bằng method `boxplot()`. `.boxplot()`

```
In [107]: fig, ax = subplots(figsize=(8, 8))
          Auto.boxplot('mpg', by='cylinders', ax=ax);
```

Method `hist()` được dùng để vẽ histogram. `.hist()`

```
In [108]: fig, ax = subplots(figsize=(8, 8))
          Auto.hist('mpg', ax=ax);
```

Có thể thay đổi màu của các cột và số lượng bin:

```
In [109]: fig, ax = subplots(figsize=(8, 8))
          Auto.hist('mpg', color='red', bins=12, ax=ax);
```

Xem `Auto.hist?` để biết thêm các tùy chọn vẽ đồ thị.

Ta có thể dùng function `pd.plotting.scatter_matrix()` để tạo một scatterplot matrix, qua đó trực quan hóa mọi mối quan hệ từng cặp giữa các cột của data frame. `pd.plotting.scatter_matrix()`

```
In [110]: pd.plotting.scatter_matrix(Auto);
```

Ta cũng có thể tạo scatterplot cho một tập con các biến.

```
In [111]: pd.plotting.scatter_matrix(Auto[['mpg',
                                           'displacement',
                                           'weight']]);
```

Method `describe()` tạo phần tóm tắt bằng số liệu cho từng cột trong data frame.

```
In [112]: Auto[['mpg', 'weight']].describe()
```

Ta cũng có thể tạo phần tóm tắt cho một cột duy nhất.

```
In [113]: Auto['cylinders'].describe()
Auto['mpg'].describe()
```

Để thoát Jupyter, chọn `File / Close and Halt`.

2.4 Bài tập

Lý thuyết

1. Với từng ý từ (a) đến (d), hãy cho biết nhìn chung ta kỳ vọng một phương pháp statistical learning linh hoạt sẽ hoạt động tốt hơn hay kém hơn một phương pháp kém linh hoạt. Giải thích câu trả lời.
 - (a) Cỡ mẫu n cực kỳ lớn, còn số predictor p nhỏ.
 - (b) Số predictor p cực kỳ lớn, còn số observation n nhỏ.
 - (c) Mỗi quan hệ giữa các predictor và response có tính phi tuyến rất cao.
 - (d) Variance của các error term, tức $\sigma^2 = \text{Var}(\epsilon)$, cực kỳ lớn.
2. Giải thích mỗi tình huống sau là bài toán classification hay regression, và cho biết ta quan tâm chủ yếu đến inference hay prediction. Cuối cùng, xác định n và p .
 - (a) Ta thu thập dữ liệu về 500 công ty hàng đầu tại Hoa Kỳ. Với mỗi công ty, ta ghi nhận lợi nhuận, số nhân viên, ngành hoạt động và lương CEO. Ta muốn hiểu những yếu tố nào ảnh hưởng đến lương CEO.
 - (b) Ta đang cân nhắc tung ra một sản phẩm mới và muốn biết sản phẩm sẽ thành công hay thất bại. Ta thu thập dữ liệu về 20 sản phẩm tương tự đã được tung ra trước đây. Với mỗi sản phẩm, ta ghi nhận thành công hay thất bại, giá bán sản phẩm, ngân sách marketing, giá của đối thủ cạnh tranh và mười biến khác.
 - (c) Ta muốn dự đoán phần trăm thay đổi của tỷ giá USD/Euro dựa trên những thay đổi hằng tuần của các thị trường chứng khoán trên thế giới. Vì vậy, ta thu thập dữ liệu theo tuần cho toàn bộ năm 2012. Với mỗi tuần, ta ghi nhận phần trăm thay đổi của USD/Euro, phần trăm thay đổi của thị trường Hoa Kỳ, phần trăm thay đổi của thị trường Anh và phần trăm thay đổi của thị trường Đức.
3. Bây giờ, ta xem lại bias-variance decomposition.
 - (a) Trên cùng một đồ thị, phác họa các đường điển hình của bình phương bias, variance, training error, test error và Bayes error (hay irreducible error), khi chuyển từ các phương pháp statistical learning ít linh hoạt sang các phương pháp linh hoạt hơn. Trục x biểu diễn mức độ linh hoạt của phương pháp, còn trục y biểu diễn giá trị của từng đường. Cần có năm đường. Hãy đặt nhãn cho từng đường.
 - (b) Giải thích vì sao mỗi đường trong năm đường có hình dạng như ở ý (a).
4. Bây giờ, hãy nghĩ đến một số ứng dụng thực tế của statistical learning.

- (a) Mô tả ba ứng dụng thực tế mà classification có thể hữu ích. Mô tả response và các predictor. Mục tiêu của từng ứng dụng là inference hay prediction? Giải thích câu trả lời.
- (b) Mô tả ba ứng dụng thực tế mà regression có thể hữu ích. Mô tả response và các predictor. Mục tiêu của từng ứng dụng là inference hay prediction? Giải thích câu trả lời.
- (c) Mô tả ba ứng dụng thực tế mà cluster analysis có thể hữu ích.
5. Một cách tiếp cận rất linh hoạt có những ưu điểm và nhược điểm gì so với một cách tiếp cận ít linh hoạt hơn trong regression hoặc classification? Trong những hoàn cảnh nào nên ưu tiên cách tiếp cận linh hoạt hơn? Khi nào nên ưu tiên cách tiếp cận ít linh hoạt hơn?
6. Mô tả sự khác nhau giữa cách tiếp cận statistical learning parametric và non-parametric. Trong regression hoặc classification, cách tiếp cận parametric có những ưu điểm gì so với cách tiếp cận non-parametric? Nó có những nhược điểm gì?
7. Bảng dưới đây cho một training dataset gồm sáu observations, ba predictors và một biến response qualitative.

Obs.	X_1	X_2	X_3	Y
1	0	3	0	Red
2	2	0	0	Red
3	0	1	3	Red
4	0	1	2	Green
5	-1	0	1	Green
6	1	1	1	Red

Giả sử ta muốn dùng dataset này để dự đoán Y khi $X_1 = X_2 = X_3 = 0$ bằng K -nearest neighbors.

- (a) Tính Euclidean distance giữa từng observation và test point $X_1 = X_2 = X_3 = 0$.
- (b) Dự đoán của ta khi $K = 1$ là gì? Tại sao?
- (c) Dự đoán của ta khi $K = 3$ là gì? Tại sao?
- (d) Nếu Bayes decision boundary trong bài toán này có tính phi tuyến rất cao, ta kỳ vọng giá trị tốt nhất của K là lớn hay nhỏ? Tại sao?

Ứng dụng

8. Bài tập này liên quan đến dataset **College**, có trong tệp **College.csv** trên website của sách. Dataset chứa một số biến về 777 trường đại học và cao đẳng khác nhau tại Hoa Kỳ. Các biến gồm:
- **Private**: Biến chỉ báo trường công/tư
 - **Apps**: Số hồ sơ đăng ký nhận được

- **Accept**: Số ứng viên được chấp nhận
- **Enroll**: Số sinh viên mới nhập học
- **Top10perc**: Sinh viên mới thuộc nhóm 10% đứng đầu lớp ở trung học
- **Top25perc**: Sinh viên mới thuộc nhóm 25% đứng đầu lớp ở trung học
- **F.Undergrad**: Số sinh viên bậc đại học học toàn thời gian
- **P.Undergrad**: Số sinh viên bậc đại học học bán thời gian
- **Outstate**: Học phí đối với sinh viên ngoài tiểu bang
- **Room.Board**: Chi phí ăn ở
- **Books**: Chi phí sách ước tính
- **Personal**: Chi tiêu cá nhân ước tính
- **PhD**: Tỷ lệ phần trăm giảng viên có bằng Ph.D.
- **Terminal**: Tỷ lệ phần trăm giảng viên có terminal degree
- **S.F.Ratio**: Tỷ lệ sinh viên/giảng viên
- **perc.alumni**: Tỷ lệ phần trăm cựu sinh viên đóng góp tài chính
- **Expend**: Chi phí giảng dạy trên mỗi sinh viên
- **Grad.Rate**: Tỷ lệ tốt nghiệp

Trước khi đọc dữ liệu vào Python, có thể xem dữ liệu bằng Excel hoặc trình soạn thảo văn bản.

- Dùng function `pd.read_csv()` để đọc dữ liệu vào Python. Đặt tên dữ liệu đã nạp là `college`. Bảo đảm bạn đã đặt thư mục đúng vị trí chứa dữ liệu.
- Xem dữ liệu dùng trong notebook bằng cách tạo và chạy một ô mới chỉ chứa lệnh `college`. Bạn sẽ thấy cột đầu tiên chỉ là tên các trường, nằm trong một cột có tên tương tự `Unnamed: 0`. Ta không thực sự muốn `pandas` xem phần này là dữ liệu. Tuy nhiên, những tên đó có thể hữu ích về sau. Hãy thử các lệnh sau và tương tự, xem các data frame thu được:

```
college2 = pd.read_csv('College.csv', index_col=0)
college3 = college.rename({'Unnamed: 0': 'College'},
                           axis=1)
college3 = college3.set_index('College')
```

Cách này dùng cột đầu tiên của tệp làm index cho data frame. Nghĩa là `pandas` đã đặt tên cho mỗi hàng theo trường tương ứng. Bây giờ, bạn sẽ thấy cột dữ liệu đầu tiên là `Private`. Lưu ý rằng tên các trường xuất hiện bên trái bảng. Ở trên, ta cũng giới thiệu một object Python mới: *dictionary*, được xác định bằng các cặp (key, value). Giữ lại phiên bản dữ liệu đã sửa bằng lệnh sau:

```
college = college3
```

dictionary

- (c) Dùng method `describe()` để tạo phần tóm tắt bằng số liệu cho các biến trong dataset.
- (d) Dùng function `pd.plotting.scatter_matrix()` để tạo một scatterplot matrix của các cột đầu tiên `[Top10perc, Apps, Enroll]`. Nhớ rằng ta có thể tham chiếu đến một list `C` gồm các cột của data frame `A` bằng `A[C]`.
- (e) Dùng method `boxplot()` của `college` để tạo các boxplot cạnh nhau của `Outstate` theo `Private`.
- (f) Tạo một biến qualitative mới tên `Elite`, bằng cách chia biến `Top10perc` thành hai nhóm dựa trên việc tỷ lệ sinh viên thuộc nhóm 10% đứng đầu lớp trung học có vượt quá 50% hay không.

```
college['Elite'] = pd.cut(college['Top10perc'],
                        [0,0.5,1],
                        labels=['No', 'Yes'])
```

Dùng method `value_counts()` của `college['Elite']` để xem có bao nhiêu trường thuộc nhóm elite. Cuối cùng, dùng lại method `boxplot()` để tạo các boxplot cạnh nhau của `Outstate` theo `Elite`.

- (g) Dùng method `plot.hist()` của `college` để tạo một số histogram có số bin khác nhau cho vài biến quantitative. Lệnh `plt.subplots(2, 2)` có thể hữu ích: nó chia cửa sổ đồ họa thành bốn vùng để có thể vẽ bốn đồ thị cùng lúc. Thay đổi các argument cho phép chia màn hình theo những tổ hợp khác.
 - (h) Tiếp tục khám phá dữ liệu và trình bày ngắn gọn những điều bạn phát hiện.
9. Bài tập này sử dụng dataset `Auto` đã được nghiên cứu trong lab. Bảo đảm các giá trị thiếu đã được loại khỏi dữ liệu.
- (a) Predictor nào là quantitative, predictor nào là qualitative?
 - (b) Miền giá trị của từng predictor quantitative là gì? Bạn có thể trả lời bằng các method `min()` và `max()` trong `numpy`. `.min()`
`.max()`
 - (c) Mean và standard deviation của từng predictor quantitative là gì?
 - (d) Bây giờ, loại bỏ các observations từ thứ 10 đến thứ 85. Miền giá trị, mean và standard deviation của từng predictor trong tập con dữ liệu còn lại là gì?
 - (e) Dùng toàn bộ dataset để khảo sát các predictor bằng đồ thị, sử dụng scatterplot hoặc công cụ khác do bạn chọn. Tạo một số đồ thị làm nổi bật mối quan hệ giữa các predictor. Nhận xét những điều bạn tìm thấy.
 - (f) Giả sử ta muốn dự đoán số mile đi được trên mỗi gallon nhiên liệu (`mpg`) dựa trên các biến khác. Các đồ thị của bạn có gợi ý

rằng biến nào khác có thể hữu ích để dự đoán `mpg` không? Giải thích câu trả lời.

10. Bài tập này sử dụng dataset nhà ở `Boston`.

- (a) Trước hết, nạp dataset `Boston`, thuộc library `ISLP`.
- (b) Dataset có bao nhiêu hàng? Bao nhiêu cột? Các hàng và cột biểu diễn điều gì?
- (c) Vẽ một số scatterplot theo từng cặp của các predictor (cột) trong dataset này. Mô tả những điều bạn tìm thấy.
- (d) Có predictor nào liên hệ với tỷ lệ tội phạm tính trên đầu người không? Nếu có, giải thích mối quan hệ.
- (e) Có khu ngoại ô nào của Boston dường như có tỷ lệ tội phạm đặc biệt cao không? Thuế suất thì sao? Tỷ lệ học sinh/giáo viên thì sao? Nhận xét miền giá trị của từng predictor.
- (f) Có bao nhiêu khu ngoại ô trong dataset này giáp sông Charles?
- (g) Median của tỷ lệ học sinh/giáo viên giữa các thị trấn trong dataset là bao nhiêu?
- (h) Khu ngoại ô nào của Boston có median giá trị nhà do chủ sở hữu cư trú thấp nhất? Các predictor khác của khu ngoại ô đó có giá trị bao nhiêu, và những giá trị này nằm ở đâu so với miền giá trị chung của từng predictor? Nhận xét những điều bạn tìm thấy.
- (i) Trong dataset này, có bao nhiêu khu ngoại ô có trung bình hơn bảy phòng trên mỗi căn nhà? Hơn tám phòng trên mỗi căn nhà? Nhận xét các khu ngoại ô có trung bình hơn tám phòng trên mỗi căn nhà.



Linear Regression

Chương này trình bày linear regression, một cách tiếp cận rất đơn giản cho supervised learning. Cụ thể, linear regression là công cụ hữu ích để dự đoán một quantitative response. Phương pháp này đã tồn tại từ lâu và là chủ đề của vô số sách giáo khoa. Dù có vẻ hơi nhàm chán khi so với một số cách tiếp cận statistical learning hiện đại hơn được mô tả ở những chương sau, linear regression vẫn là một phương pháp statistical learning hữu ích và được dùng rộng rãi. Hơn nữa, nó là điểm khởi đầu tốt để tìm hiểu các cách tiếp cận mới hơn: như ta sẽ thấy trong các chương sau, nhiều phương pháp statistical learning phức tạp có thể được xem là dạng tổng quát hóa hoặc mở rộng của linear regression. Vì vậy, không thể xem nhẹ tầm quan trọng của việc hiểu vững linear regression trước khi nghiên cứu những phương pháp learning phức tạp hơn. Trong chương này, chúng tôi ôn lại một số ý tưởng chính làm nền tảng cho linear regression model, cùng cách tiếp cận least squares thường được dùng nhất để fit mô hình này.

Nhớ lại dữ liệu **Advertising** ở Chương 2. Hình 2.1 biểu diễn **sales**, tính bằng nghìn đơn vị, của một sản phẩm cụ thể theo ngân sách quảng cáo, tính bằng nghìn đô la, trên các phương tiện **TV**, **radio** và **newspaper**. Giả sử với vai trò chuyên gia tư vấn thống kê, ta được yêu cầu dựa trên dữ liệu này để đề xuất một kế hoạch marketing cho năm tới nhằm tạo ra doanh số cao. Những thông tin nào sẽ hữu ích để đưa ra đề xuất như vậy? Sau đây là một số câu hỏi quan trọng cần giải quyết:

1. *Có mối quan hệ giữa ngân sách quảng cáo và doanh số không?* Mục tiêu đầu tiên là xác định dữ liệu có cung cấp bằng chứng về association giữa chi tiêu quảng cáo và doanh số hay không. Nếu bằng chứng yếu, người ta có thể lập luận rằng không nên chi tiền cho quảng cáo!
2. *Mối quan hệ giữa ngân sách quảng cáo và doanh số mạnh đến đâu?* Giả sử quảng cáo và doanh số có quan hệ với nhau, ta muốn biết

độ mạnh của mối quan hệ ấy. Biết ngân sách quảng cáo có cung cấp nhiều thông tin về doanh số sản phẩm không?

3. *Những phương tiện nào liên hệ với doanh số?* Cả ba phương tiện – **TV**, **radio** và **newspaper** – đều liên hệ với doanh số, hay chỉ một hoặc hai phương tiện có liên hệ? Để trả lời, ta phải tách riêng contribution của từng phương tiện đối với doanh số khi đã chi tiền cho cả ba.
4. *Mức độ association giữa từng phương tiện và doanh số lớn đến đâu?* Với mỗi đô la chi cho quảng cáo trên một phương tiện cụ thể, doanh số sẽ tăng bao nhiêu? Ta có thể dự đoán mức tăng này chính xác đến đâu?
5. *Ta có thể dự đoán doanh số tương lai chính xác đến đâu?* Với một mức chi quảng cáo **TV**, **radio** hoặc **newspaper** đã cho, dự đoán doanh số là bao nhiêu và dự đoán ấy chính xác đến đâu?
6. *Mối quan hệ có tuyến tính không?* Nếu chi tiêu quảng cáo trên các phương tiện khác nhau và doanh số có quan hệ gần như đường thẳng, linear regression là công cụ phù hợp. Nếu không, vẫn có thể transform predictor hoặc response để dùng linear regression.
7. *Có synergy giữa các phương tiện quảng cáo không?* Có thể việc chi 50.000 đô la cho quảng cáo truyền hình và 50.000 đô la cho radio đi kèm doanh số cao hơn so với dồn cả 100.000 đô la cho riêng truyền hình hoặc radio. Trong marketing, hiện tượng này được gọi là *synergy effect*; trong thống kê, nó được gọi là *interaction effect*.

synergy effect
interaction effect

Hóa ra linear regression có thể trả lời từng câu hỏi trên. Trước hết, chúng tôi sẽ thảo luận các câu hỏi trong bối cảnh tổng quát, rồi quay lại tình huống cụ thể này ở Mục 3.4.

3.1 Simple Linear Regression

Đúng như tên gọi, *simple linear regression* là một cách tiếp cận rất trực tiếp để dự đoán quantitative response Y dựa trên một predictor variable X . Phương pháp giả định X và Y có mối quan hệ gần tuyến tính. Về toán học, ta viết

simple linear regression

$$Y \approx \beta_0 + \beta_1 X. \quad (3.1)$$

Có thể đọc dấu “ $\approx$ ” là “được mô hình hóa gần đúng bởi”. Đôi khi, ta mô tả (3.1) bằng cách nói rằng ta regress Y trên X .

Chẳng hạn, X có thể biểu diễn quảng cáo **TV**, còn Y biểu diễn **sales**. Khi đó, ta có thể regress **sales** trên **TV** bằng cách fit mô hình

$$\text{sales} \approx \beta_0 + \beta_1 \times \text{TV}.$$

Trong Phương trình 3.1, β_0 và β_1 là hai hằng số chưa biết, lần lượt biểu diễn *intercept* và *slope* trong linear model. Gộp lại, β_0 và β_1 được gọi là các *model coefficient* hoặc *parameter*. Sau khi dùng training data để thu được

intercept
slope
model coefficient
parameter

các ước lượng $\hat{\beta}_0$ và $\hat{\beta}_1$ của model coefficient, ta có thể dự đoán doanh số tương lai với một giá trị quảng cáo **TV** cụ thể bằng cách tính

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x, \quad (3.2)$$

trong đó $\hat{y}$ là dự đoán của Y dựa trên $X = x$. Ở đây, dấu mũ “ $\hat{\cdot}$ ” ký hiệu giá trị ước lượng của một parameter hay coefficient chưa biết, hoặc giá trị dự đoán của response.

3.1.1 Ước lượng các coefficient

Trong thực tế, β_0 và β_1 chưa biết. Vì vậy, trước khi dùng (3.1) để dự đoán, ta phải dùng dữ liệu để ước lượng các coefficient. Gọi

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

là n cặp observation, mỗi cặp gồm một phép đo X và một phép đo Y . Trong ví dụ **Advertising**, dataset này gồm ngân sách quảng cáo **TV** và doanh số sản phẩm tại $n = 200$ thị trường khác nhau. (Nhớ rằng dữ liệu được biểu diễn trong Hình 2.1.) Mục tiêu là tìm các ước lượng coefficient $\hat{\beta}_0$ và $\hat{\beta}_1$ sao cho linear model (3.1) fit tốt dữ liệu sẵn có, tức $y_i \approx \hat{\beta}_0 + \hat{\beta}_1 x_i$ với $i = 1, \dots, n$. Nói cách khác, ta muốn tìm intercept $\hat{\beta}_0$ và slope $\hat{\beta}_1$ sao cho đường thẳng thu được gần nhất có thể với $n = 200$ điểm dữ liệu. Có nhiều cách đo độ gần. Tuy nhiên, cách phổ biến nhất là giảm thiểu tiêu chuẩn *least squares*; đó là cách tiếp cận được dùng trong chương này. Các cách khác sẽ được xem xét ở Chương 6.

Gọi $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$ là dự đoán cho Y dựa trên giá trị thứ i của X . Khi đó, $e_i = y_i - \hat{y}_i$ là *residual* thứ i – chênh lệch giữa response quan sát thứ i và response thứ i được linear model dự đoán. Ta định nghĩa *residual sum of squares* (RSS) là

$$\text{RSS} = e_1^2 + e_2^2 + \dots + e_n^2,$$

hoặc tương đương,

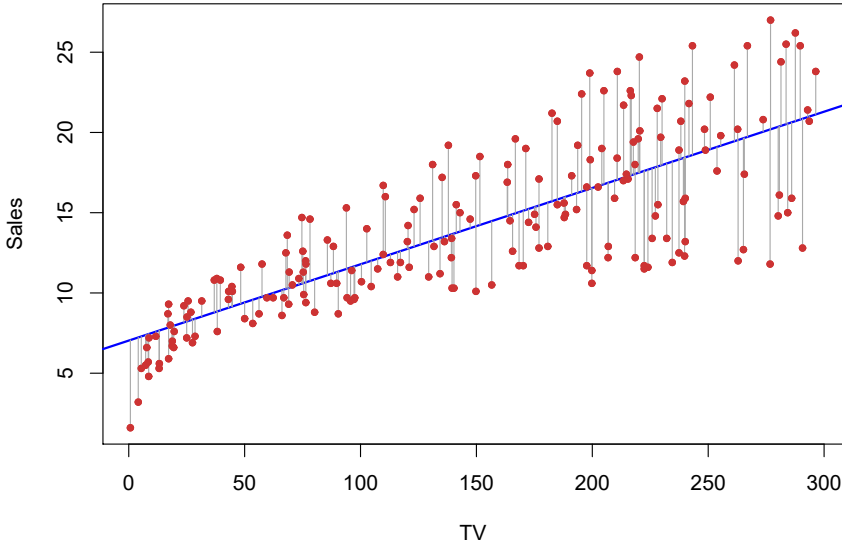
$$\text{RSS} = (y_1 - \hat{\beta}_0 - \hat{\beta}_1 x_1)^2 + \dots + (y_n - \hat{\beta}_0 - \hat{\beta}_1 x_n)^2. \quad (3.3)$$

Cách tiếp cận least squares chọn $\hat{\beta}_0$ và $\hat{\beta}_1$ để giảm thiểu RSS. Dùng một số phép tính giải tích, có thể chứng minh rằng các giá trị giảm thiểu là

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}, \quad (3.4)$$

trong đó $\bar{y} \equiv n^{-1} \sum_{i=1}^n y_i$ và $\bar{x} \equiv n^{-1} \sum_{i=1}^n x_i$ là các sample mean. Nói cách khác, (3.4) xác định các least squares coefficient estimate cho simple linear regression.

Hình 3.1 biểu diễn simple linear regression fit cho dữ liệu **Advertising**, với $\hat{\beta}_0 = 7.03$ và $\hat{\beta}_1 = 0.0475$. Nói cách khác, theo phép xấp xỉ này, mỗi 1.000 đô la chi thêm cho quảng cáo **TV** đi kèm khoảng 47,5 đơn vị sản phẩm bán thêm. Trong Hình 3.2, chúng tôi tính RSS tại một số giá trị β_0 và β_1 , dùng dữ liệu advertising với **sales** là response và **TV** là predictor. Trong mỗi đồ thị, chấm đỏ biểu diễn cặp least squares estimate ($\hat{\beta}_0, \hat{\beta}_1$) từ (3.4). Rõ ràng, các giá trị này giảm thiểu RSS.



HÌNH 3.1. Với dữ liệu *Advertising*, hình biểu diễn *least squares fit* của phép regression *sales* trên *TV*. Kết quả fit được tìm bằng cách giảm thiểu *residual sum of squares*. Mỗi đoạn thẳng xám biểu diễn một *residual*. Trong trường hợp này, *linear fit* nắm bắt được bản chất của mối quan hệ, dù ước lượng quá cao trend ở phía trái đồ thị.

3.1.2 Đánh giá độ chính xác của các coefficient estimate

Nhớ lại từ (2.1), ta giả định mối quan hệ thực giữa X và Y có dạng $Y = f(X) + \epsilon$ với một hàm f chưa biết, trong đó ϵ là error term ngẫu nhiên có mean bằng không. Nếu xấp xỉ f bằng một hàm tuyến tính, ta viết

$$Y = \beta_0 + \beta_1 X + \epsilon. \quad (3.5)$$

Ở đây, β_0 là intercept, tức expected value của Y khi $X = 0$; còn β_1 là slope, tức mức tăng trung bình của Y đi kèm mức tăng một đơn vị của X . Error term gom tất cả những gì mô hình đơn giản này bỏ sót: mối quan hệ thực có lẽ không tuyến tính, có thể tồn tại những biến khác làm Y biến thiên, và có thể có measurement error. Ta thường giả định error term độc lập với X .

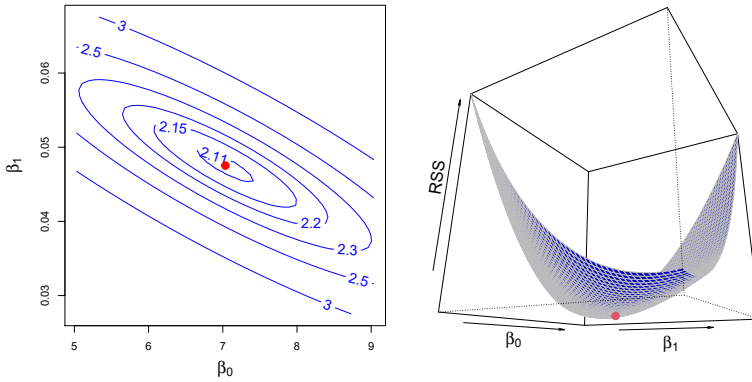
Mô hình (3.5) xác định *population regression line*, là phép xấp xỉ tuyến tính tốt nhất cho mối quan hệ thực giữa X và Y .¹ Các least squares regression coefficient estimate trong (3.4) xác định *least squares line*. Ở bên trái của Hình 3.3 biểu diễn hai đường này trong một ví dụ mô phỏng đơn giản. Chúng tôi tạo 100 giá trị X ngẫu nhiên và sinh 100 giá trị Y tương ứng từ mô hình

$$Y = 2 + 3X + \epsilon, \quad (3.6)$$

trong đó ϵ được sinh từ phân phối normal có mean bằng không. Đường đỏ ở ô trái Hình 3.3 biểu diễn mối quan hệ thực $f(X) = 2 + 3X$, còn đường xanh

population
regression line
least squares
line

¹Giả định tuyến tính thường là một working model hữu ích. Tuy nhiên, trái với điều nhiều sách giáo khoa có thể nói, hiếm khi ta tin mỗi quan hệ thực sự là tuyến tính.



HÌNH 3.2. Contour plot và đồ thị ba chiều của RSS trên dữ liệu *Advertising*, dùng *sales* làm response và *TV* làm predictor. Các chấm đỏ tương ứng với least squares estimate $\hat{\beta}_0$ và $\hat{\beta}_1$ trong (3.4).

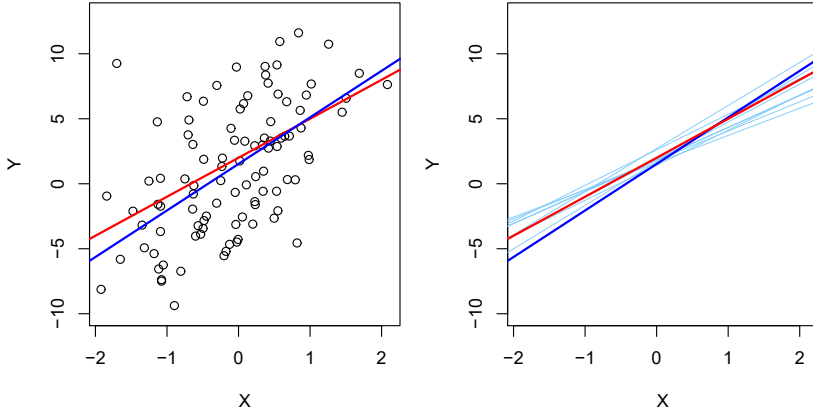
làm là least squares estimate dựa trên dữ liệu quan sát. Với dữ liệu thực, mỗi quan hệ thực thường không được biết, nhưng ta luôn tính được least squares line bằng các coefficient estimate trong (3.4). Nói cách khác, trong ứng dụng thực tế, ta có một tập observations để tính least squares line; còn population regression line không quan sát được. Ở ô phải Hình 3.3, chúng tôi sinh mười dataset khác nhau từ mô hình (3.6) và vẽ mười least squares line tương ứng. Lưu ý rằng các dataset khác nhau được sinh từ cùng mô hình thực tạo ra các least squares line hơi khác nhau, nhưng population regression line không đổi.

Thoạt nhìn, khác biệt giữa population regression line và least squares line có thể có vẻ tinh tế và khó hiểu. Ta chỉ có một dataset, vậy việc hai đường khác nhau cùng mô tả quan hệ giữa predictor và response có nghĩa gì? Về căn bản, khái niệm hai đường này là phần mở rộng tự nhiên của cách tiếp cận thống kê tiêu chuẩn: dùng thông tin từ một sample để ước lượng đặc trưng của population lớn.

Chẳng hạn, giả sử ta muốn biết population mean μ của một biến ngẫu nhiên Y . Tiếc rằng μ chưa biết, nhưng ta có n observations $y_1, \dots, y_n$ từ Y để ước lượng μ . Một ước lượng hợp lý là $\hat{\mu} = \bar{y}$, trong đó $\bar{y} = n^{-1} \sum_{i=1}^n y_i$ là sample mean. Sample mean và population mean khác nhau, nhưng nhìn chung sample mean cung cấp một ước lượng tốt của population mean. Tương tự, các coefficient chưa biết β_0 và β_1 trong linear regression xác định population regression line. Ta tìm cách ước lượng chúng bằng $\hat{\beta}_0$ và $\hat{\beta}_1$ trong (3.4); các coefficient estimate này xác định least squares line.

Phép so sánh giữa linear regression và việc ước lượng mean của một biến ngẫu nhiên phù hợp dựa trên khái niệm *bias*. Nếu dùng sample mean $\hat{\mu}$ để ước lượng μ , ước lượng này là *unbiased*, theo nghĩa xét trung bình ta kỳ vọng $\hat{\mu}$ bằng μ . Chính xác thì điều này có nghĩa gì? Trên một tập observations $y_1, \dots, y_n$ cụ thể, $\hat{\mu}$ có thể ước lượng quá cao μ ; trên một tập khác, $\hat{\mu}$ có thể ước lượng quá thấp μ . Nhưng nếu có thể lấy trung bình của rất nhiều ước lượng μ thu được từ rất nhiều tập observations, trung bình đó

bias
unbiased



HÌNH 3.3. Một dataset mô phỏng. Trái: Đường đỏ biểu diễn mối quan hệ thực $f(X) = 2 + 3X$, gọi là *population regression line*. Đường xanh lam là *least squares line*, tức *least squares estimate* của $f(X)$ dựa trên dữ liệu quan sát màu đen. Phải: *Population regression line* vẫn được vẽ màu đỏ và *least squares line* màu xanh lam đậm. Mười *least squares line* màu xanh nhạt được tính từ mười tập observations ngẫu nhiên riêng biệt. Các *least squares line* khác nhau, nhưng xét trung bình, chúng khá gần *population regression line*.

sẽ đúng bằng μ . Vì thế, một unbiased estimator không ước lượng quá cao hay quá thấp parameter thực một cách có hệ thống. Tính unbiased cũng đúng với các least squares coefficient estimate trong (3.4): khi ước lượng β_0 và β_1 từ một dataset cụ thể, các ước lượng không hoàn toàn bằng β_0 và β_1 . Nhưng nếu có thể lấy trung bình các ước lượng từ rất nhiều dataset, giá trị trung bình sẽ chính xác. Thật vậy, ở phải của Hình 3.3 cho thấy trung bình của nhiều least squares line, mỗi đường được ước lượng từ một dataset riêng, khá gần population regression line thực.

Tiếp tục phép so sánh với việc ước lượng population mean μ của biến ngẫu nhiên Y . Một câu hỏi tự nhiên là: sample mean $\hat{\mu}$ ước lượng μ chính xác đến đâu? Ta đã xác định rằng trung bình của $\hat{\mu}$ trên nhiều dataset sẽ rất gần μ , nhưng một ước lượng $\hat{\mu}$ đơn lẻ có thể thấp hơn hoặc cao hơn μ đáng kể. Ước lượng đơn lẻ đó sẽ lệch bao xa? Nói chung, ta trả lời bằng cách tính *standard error* của $\hat{\mu}$, viết là $SE(\hat{\mu})$. Công thức quen thuộc là

standard error

$$\text{Var}(\hat{\mu}) = SE(\hat{\mu})^2 = \frac{\sigma^2}{n}, \quad (3.7)$$

trong đó σ là standard deviation của mỗi realization y_i của Y .² Nói gần đúng, standard error cho biết mức chênh lệch trung bình giữa ước lượng $\hat{\mu}$ và giá trị thực μ . Phương trình 3.7 cũng cho thấy độ lệch này giảm theo n : càng có nhiều observations, standard error của $\hat{\mu}$ càng nhỏ.

²Công thức này đúng với điều kiện n observations không correlated.

Tương tự, ta có thể hỏi $\hat{\beta}_0$ và $\hat{\beta}_1$ gần các giá trị thực β_0 và β_1 đến đâu. Để tính standard error của $\hat{\beta}_0$ và $\hat{\beta}_1$, ta dùng

$$\text{SE}(\hat{\beta}_0)^2 = \sigma^2 \left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_{i=1}^n (x_i - \bar{x})^2} \right], \quad \text{SE}(\hat{\beta}_1)^2 = \frac{\sigma^2}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad (3.8)$$

trong đó $\sigma^2 = \text{Var}(\epsilon)$. Để các công thức này hoàn toàn chính xác, ta cần giả định error ϵ_i của mỗi observation có cùng variance σ^2 và không correlated. Điều này rõ ràng không đúng trong Hình 3.1, nhưng công thức vẫn là một phép xấp xỉ tốt. Lưu ý rằng $\text{SE}(\hat{\beta}_1)$ nhỏ hơn khi các x_i phân tán rộng hơn; trực giác là khi đó ta có nhiều leverage hơn để ước lượng slope. Ta cũng thấy $\text{SE}(\hat{\beta}_0)$ sẽ bằng $\text{SE}(\hat{\mu})$ nếu $\bar{x} = 0$, khi đó $\hat{\beta}_0 = \bar{y}$. Nhìn chung, σ^2 chưa biết nhưng có thể được ước lượng từ dữ liệu. Ước lượng của σ được gọi là *residual standard error*, cho bởi $\text{RSE} = \sqrt{\text{RSS}/(n-2)}$. Nói chặt chẽ, khi σ^2 được ước lượng từ dữ liệu, ta nên viết $\widehat{\text{SE}}(\hat{\beta}_1)$ để thể hiện đã dùng một ước lượng; để ký hiệu đơn giản, chúng tôi bỏ dấu mũ bỏ sung này.

residual
standard error

Standard error có thể dùng để tính *confidence interval*. Confidence interval 95% được định nghĩa là một khoảng sao cho với xác suất 95%, khoảng đó chứa giá trị thực chưa biết của parameter. Cận dưới và cận trên của khoảng được tính từ sample dữ liệu. Confidence interval 95% có tính chất sau: nếu lấy sample lặp lại và dựng confidence interval cho từng sample, 95% số khoảng sẽ chứa giá trị thực chưa biết của parameter. Với linear regression, confidence interval 95% của β_1 có dạng xấp xỉ

confidence
interval

$$\hat{\beta}_1 \pm 2 \cdot \text{SE}(\hat{\beta}_1). \quad (3.9)$$

Nghĩa là có xấp xỉ 95% khả năng khoảng

$$[\hat{\beta}_1 - 2 \cdot \text{SE}(\hat{\beta}_1), \hat{\beta}_1 + 2 \cdot \text{SE}(\hat{\beta}_1)] \quad (3.10)$$

chứa giá trị thực của β_1 .³ Tương tự, confidence interval của β_0 có dạng xấp xỉ

$$\hat{\beta}_0 \pm 2 \cdot \text{SE}(\hat{\beta}_0). \quad (3.11)$$

Với dữ liệu advertising, confidence interval 95% của β_0 là [6.130, 7.935], còn của β_1 là [0.042, 0.053]. Vì vậy, ta kết luận rằng khi không quảng cáo, doanh số trung bình nằm đâu đó từ 6.130 đến 7.935 đơn vị. Hơn nữa, với mỗi 1.000 đô la tăng thêm cho quảng cáo truyền hình, doanh số trung bình tăng từ 42 đến 53 đơn vị.

Standard error cũng được dùng để thực hiện *hypothesis test* trên các coefficient. Hypothesis test phổ biến nhất kiểm tra *null hypothesis*

hypothesis
test
null
hypothesis

$$H_0 : \text{Không có mối quan hệ giữa } X \text{ và } Y \quad (3.12)$$

so với *alternative hypothesis*

alternative
hypothesis

$$H_a : \text{Có một mối quan hệ giữa } X \text{ và } Y. \quad (3.13)$$

³Đây là xấp xỉ vì nhiều lý do. Phương trình 3.10 dựa trên giả định error có phân phối Gaussian. Ngoài ra, hệ số 2 trước $\text{SE}(\hat{\beta}_1)$ thay đổi đôi chút theo số observations n . Chính xác hơn, thay vì 2, (3.10) phải dùng quantile 97,5% của phân phối t với $n-2$ degrees of freedom. Cách tính chính xác confidence interval 95% bằng R sẽ được trình bày sau trong chương.

BẢNG 3.1. Với dữ liệu *Advertising*, các coefficient của least squares model cho phép regression số đơn vị bán được trên ngân sách quảng cáo *TV*. Ngân sách *TV* tăng 1.000 đô la đi kèm doanh số tăng khoảng 50 đơn vị. Nhờ rằng biến *sales* tính bằng nghìn đơn vị, còn biến *TV* tính bằng nghìn đô la.

	Coefficient	Std. error	<i>t</i> -statistic	<i>p</i> -value
Intercept	7.0325	0.4578	15.36	< 0.0001
TV	0.0475	0.0027	17.67	< 0.0001

Về toán học, điều này tương ứng với kiểm tra $H_0 : \beta_1 = 0$ so với $H_a : \beta_1 \neq 0$, vì nếu $\beta_1 = 0$ thì mô hình (3.5) rút gọn thành $Y = \beta_0 + \epsilon$, và X không associated với Y . Để kiểm tra null hypothesis, ta cần xác định liệu ước lượng $\hat{\beta}_1$ có cách 0 đủ xa để có thể tin rằng β_1 khác 0 hay không. Xa đến đâu mới là đủ? Điều này phụ thuộc vào độ chính xác của $\hat{\beta}_1$, tức phụ thuộc vào $SE(\hat{\beta}_1)$. Nếu standard error nhỏ, ngay cả giá trị $\hat{\beta}_1$ tương đối nhỏ cũng có thể cung cấp bằng chứng mạnh rằng $\beta_1 \neq 0$, nên X và Y có quan hệ. Ngược lại, nếu standard error lớn, $|\hat{\beta}_1|$ phải lớn mới có thể bác bỏ null hypothesis. Trong thực tế, ta tính *t*-statistic

t-statistic

$$t = \frac{\hat{\beta}_1 - 0}{SE(\hat{\beta}_1)}, \quad (3.14)$$

đo số standard deviation từ $\hat{\beta}_1$ đến 0. Nếu thực sự không có quan hệ giữa X và Y , ta kỳ vọng (3.14) có phân phối t với $n - 2$ degrees of freedom. Phân phối t có dạng chuông và khi n lớn hơn khoảng 30, nó khá giống phân phối standard normal. Vì vậy, với giả định $\beta_1 = 0$, ta dễ dàng tính xác suất quan sát được một giá trị có độ lớn tuyệt đối bằng hoặc vượt $|t|$. Xác suất này được gọi là *p*-value. Nói gần đúng, *p*-value nhỏ cho biết nếu không có association thực giữa predictor và response, khả năng tình cờ quan sát được một association mạnh như vậy là thấp. Vì thế, nếu thấy *p*-value nhỏ, ta có thể suy ra predictor và response có association. Ta bác bỏ null hypothesis, tức tuyên bố X và Y có quan hệ, nếu *p*-value đủ nhỏ. Các ngưỡng *p*-value thường dùng là 5% hoặc 1%, dù chủ đề này sẽ được khảo sát kỹ hơn trong Chương 13. Khi $n = 30$, chúng tương ứng với *t*-statistic (3.14) xấp xỉ 2 và 2,75.

p-value

Bảng 3.1 trình bày chi tiết least squares model cho phép regression số đơn vị bán được trên ngân sách quảng cáo *TV* của dữ liệu *Advertising*. Lưu ý rằng các coefficient $\hat{\beta}_0$ và $\hat{\beta}_1$ rất lớn so với standard error tương ứng, nên *t*-statistic cũng lớn; nếu H_0 đúng, xác suất thấy các giá trị như vậy gần như bằng không. Vì thế, ta có thể kết luận $\beta_0 \neq 0$ và $\beta_1 \neq 0$.⁴

3.1.3 Đánh giá độ chính xác của mô hình

Sau khi bác bỏ null hypothesis (3.12) để ủng hộ alternative hypothesis (3.13), ta tự nhiên muốn định lượng mức độ mô hình fit dữ liệu. Chất

⁴Trong Bảng 3.1, *p*-value nhỏ của intercept cho phép bác bỏ null hypothesis $\beta_0 = 0$, còn *p*-value nhỏ của *TV* cho phép bác bỏ null hypothesis $\beta_1 = 0$. Bác bỏ giả thuyết sau cho phép kết luận *TV* và *sales* có quan hệ. Bác bỏ giả thuyết trước cho phép kết luận rằng khi không chỉ cho *TV*, doanh số khác không.

BẢNG 3.2. Với dữ liệu *Advertising*, thông tin bổ sung về least squares model cho phép regression số đơn vị bán được trên ngân sách quảng cáo *TV*.

Quantity	Value
Residual standard error	3.26
R^2	0.612
F -statistic	312.1

lượng của linear regression fit thường được đánh giá bằng hai đại lượng liên quan: residual standard error (RSE) và thống kê R^2 .

Bảng 3.2 trình bày RSE, thống kê R^2 , và F -statistic (sẽ được mô tả ở Mục 3.2.2) cho linear regression của số đơn vị bán được trên ngân sách quảng cáo *TV*.

Residual Standard Error

Nhớ lại mô hình (3.5): mỗi observation có một error term ϵ . Do các error term này tồn tại, ngay cả khi biết regression line thực, tức biết β_0 và β_1 , ta vẫn không thể dự đoán hoàn hảo Y từ X . RSE là một ước lượng của standard deviation của ϵ . Nói gần đúng, nó là mức trung bình mà response lệch khỏi regression line thực. RSE được tính bằng

$$\text{RSE} = \sqrt{\frac{1}{n-2} \text{RSS}} = \sqrt{\frac{1}{n-2} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (3.15)$$

Lưu ý rằng RSS đã được định nghĩa ở Mục 3.1.1, với công thức

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (3.16)$$

Với dữ liệu advertising, kết quả linear regression trong Bảng 3.2 cho RSE bằng 3,26. Nói cách khác, doanh số thực tại mỗi thị trường lệch khỏi regression line thực trung bình khoảng 3.260 đơn vị. Một cách diễn giải khác: ngay cả khi mô hình đúng và biết chính xác các giá trị chưa biết β_0, β_1 , mọi dự đoán doanh số dựa trên quảng cáo *TV* vẫn lệch trung bình khoảng 3.260 đơn vị. Tất nhiên, sai số dự đoán 3.260 đơn vị có chấp nhận được hay không phụ thuộc vào bối cảnh. Trong dữ liệu advertising, mean doanh số trên mọi thị trường xấp xỉ 14.000 đơn vị, nên phần trăm sai số là $3.260/14.000 = 23\%$.

RSE được xem là thước đo mức độ mô hình (3.5) không fit dữ liệu. Nếu các dự đoán từ mô hình rất gần outcome thực, tức $\hat{y}_i \approx y_i$ với $i = 1, \dots, n$, thì (3.15) nhỏ và ta có thể kết luận mô hình fit dữ liệu rất tốt. Ngược lại, nếu một hay nhiều $\hat{y}_i$ rất xa y_i , RSE có thể khá lớn, cho thấy mô hình không fit tốt dữ liệu.

Thống kê R^2

RSE cung cấp thước đo tuyệt đối về mức độ mô hình (3.5) không fit dữ liệu. Nhưng vì được đo theo đơn vị của Y , không phải lúc nào cũng rõ RSE

như thế nào là tốt. Thống kê R^2 cung cấp một thước đo khác. Nó có dạng một tỷ lệ – proportion of variance explained – nên luôn nhận giá trị từ 0 đến 1 và không phụ thuộc vào scale của Y . Để tính R^2 , ta dùng

$$R^2 = \frac{\text{TSS} - \text{RSS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}}, \quad (3.17)$$

trong đó $\text{TSS} = \sum_i (y_i - \bar{y})^2$ là *total sum of squares*, còn RSS được định nghĩa trong (3.16). TSS đo total variance trong response Y , có thể xem là lượng variability vốn có của response trước khi thực hiện regression. Ngược lại, RSS đo variability còn chưa được giải thích sau regression. Vì thế, $\text{TSS} - \text{RSS}$ đo variability trong response được giải thích, hay loại bỏ, nhờ regression; còn R^2 đo tỷ lệ variability trong Y có thể được giải thích bằng X . R^2 gần 1 cho thấy regression giải thích phần lớn variability của response. Giá trị gần 0 cho thấy regression không giải thích nhiều variability; điều này có thể xảy ra vì linear model sai, error variance σ^2 cao, hoặc cả hai. Trong Bảng 3.2, $R^2 = 0.61$, nên chưa đến hai phần ba variability trong **sales** được giải thích bằng linear regression trên **TV**.

So với RSE (3.15), thống kê R^2 (3.17) có lợi thế về diễn giải vì luôn nằm từ 0 đến 1. Tuy nhiên, xác định thế nào là một giá trị R^2 tốt vẫn có thể khó và nói chung phụ thuộc vào ứng dụng. Chẳng hạn, trong một số bài toán vật lý, ta có thể biết dữ liệu thực sự đến từ linear model với residual error nhỏ. Khi đó, ta kỳ vọng R^2 cực gần 1; một giá trị nhỏ hơn đáng kể có thể báo hiệu vấn đề nghiêm trọng trong thí nghiệm tạo dữ liệu. Trái lại, trong các ứng dụng điển hình của sinh học, tâm lý học, marketing và nhiều lĩnh vực khác, linear model (3.5) tốt nhất cũng chỉ là phép xấp xỉ rất thô cho dữ liệu, còn residual error do những yếu tố không đo được khác thường rất lớn. Trong bối cảnh này, ta kỳ vọng predictor chỉ giải thích một tỷ lệ nhỏ variance của response; R^2 thấp hơn nhiều so với 0,1 có thể thực tế hơn.

Thống kê R^2 đo mối quan hệ tuyến tính giữa X và Y . Nhớ rằng *correlation*, được định nghĩa là

$$\text{Cor}(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}, \quad (3.18)$$

cũng là thước đo mối quan hệ tuyến tính giữa X và Y .⁵ Điều này gợi ý có thể dùng $r = \text{Cor}(X, Y)$ thay cho R^2 để đánh giá linear model fit. Thật vậy, có thể chỉ ra rằng trong bối cảnh simple linear regression, $R^2 = r^2$. Nói cách khác, bình phương correlation và thống kê R^2 là một. Tuy nhiên, ở mục tiếp theo, ta thảo luận multiple linear regression, dùng đồng thời nhiều predictor để dự đoán response. Khái niệm correlation giữa các predictor và response không tự động mở rộng sang bối cảnh này, vì correlation định lượng association giữa một cặp biến duy nhất chứ không phải một số lượng biến lớn hơn. Ta sẽ thấy R^2 đảm nhiệm vai trò đó.

⁵Về phải của (3.18) thực ra là sample correlation; vì thế, viết $\widehat{\text{Cor}}(X, Y)$ sẽ chính xác hơn. Tuy nhiên, để ký hiệu đơn giản, chúng tôi bỏ dấu mũ.

BẢNG 3.3. Các simple linear regression model khác cho dữ liệu *Advertising*. Coefficient của mô hình dự đoán số đơn vị bán được theo ngân sách quảng cáo *radio* (trên) và *newspaper* (dưới). Chi thêm 1.000 đô la cho *radio* đi kèm doanh số trung bình tăng khoảng 203 đơn vị; cùng mức tăng cho *newspaper* đi kèm doanh số tăng khoảng 55 đơn vị.

Simple regression of <i>sales</i> on <i>radio</i>				
	Coefficient	Std. error	<i>t</i> -statistic	<i>p</i> -value
Intercept	9.312	0.563	16.54	< 0.0001
radio	0.203	0.020	9.92	< 0.0001
Simple regression of <i>sales</i> on <i>newspaper</i>				
Intercept	12.351	0.621	19.88	< 0.0001
newspaper	0.055	0.017	3.30	0.00115

3.2 Multiple Linear Regression

Simple linear regression hữu ích khi dự đoán response dựa trên một predictor variable. Nhưng trong thực tế, ta thường có nhiều hơn một predictor. Chẳng hạn, với dữ liệu *Advertising*, ta đã khảo sát mối quan hệ giữa *sales* và quảng cáo *TV*. Ta còn có dữ liệu về số tiền chi cho quảng cáo *radio* và *newspaper*, và có thể muốn biết hai phương tiện này có associated với doanh số hay không. Làm thế nào mở rộng phân tích dữ liệu advertising để chứa thêm hai predictor này?

Một lựa chọn là chạy ba simple linear regression riêng biệt, mỗi mô hình dùng một phương tiện quảng cáo khác nhau làm predictor. Chẳng hạn, ta có thể fit simple linear regression để dự đoán *sales* dựa trên số tiền chi cho quảng cáo *radio*. Kết quả nằm ở phần trên Bảng 3.3. Ta thấy chi thêm 1.000 đô la cho quảng cáo radio đi kèm doanh số tăng khoảng 203 đơn vị. Phần dưới Bảng 3.3 chứa các least squares coefficient của simple linear regression giữa *sales* và ngân sách *newspaper*. Ngân sách quảng cáo newspaper tăng 1.000 đô la đi kèm doanh số tăng khoảng 55 đơn vị.

Tuy nhiên, cách fit một simple linear regression model riêng cho từng predictor chưa hoàn toàn thỏa đáng. Thứ nhất, không rõ phải đưa ra một dự đoán *sales* duy nhất như thế nào khi biết ngân sách của cả ba phương tiện, vì mỗi ngân sách gắn với một regression equation riêng. Thứ hai, mỗi regression equation bỏ qua hai phương tiện còn lại khi ước lượng regression coefficient. Ta sẽ sớm thấy rằng nếu ngân sách các phương tiện correlated với nhau trên 200 thị trường trong dataset, điều này có thể dẫn đến những ước lượng rất sai lệch về association giữa ngân sách từng phương tiện và doanh số.

Thay vì fit một simple linear regression model riêng cho từng predictor, cách tốt hơn là mở rộng simple linear regression model (3.5) để trực tiếp chứa nhiều predictor. Ta làm điều này bằng cách gán một slope coefficient riêng cho từng predictor trong cùng một mô hình. Tổng quát, giả sử ta có p predictor khác nhau. Khi đó, multiple linear regression model có dạng

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon, \quad (3.19)$$

BẢNG 3.4. Với dữ liệu *Advertising*, các least squares coefficient estimate của multiple linear regression giữa số đơn vị bán được với ngân sách quảng cáo *TV*, *radio* và *newspaper*.

	Coefficient	Std. error	t-statistic	p-value
Intercept	2.939	0.3119	9.42	< 0.0001
TV	0.046	0.0014	32.81	< 0.0001
radio	0.189	0.0086	21.89	< 0.0001
newspaper	-0.001	0.0059	-0.18	0.8599

trong đó X_j là predictor thứ j , còn β_j định lượng association giữa biến đó và response. Ta diễn giải β_j là ảnh hưởng trung bình lên Y khi X_j tăng một đơn vị, trong khi giữ cố định tất cả predictor khác. Trong ví dụ advertising, (3.19) trở thành

$$\text{sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{radio} + \beta_3 \times \text{newspaper} + \epsilon. \quad (3.20)$$

3.2.1 Ước lượng regression coefficient

Giống bối cảnh simple linear regression, các regression coefficient $\beta_0, \beta_1, \dots, \beta_p$ trong (3.19) chưa biết và phải được ước lượng. Khi có các ước lượng $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$, ta dự đoán bằng

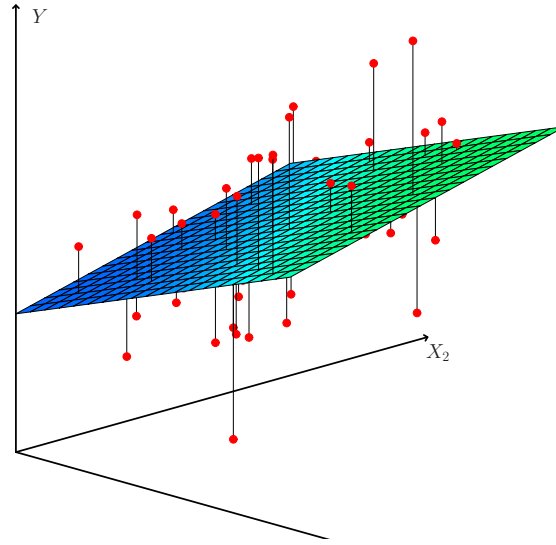
$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \dots + \hat{\beta}_p x_p. \quad (3.21)$$

Các parameter được ước lượng bằng cùng cách tiếp cận least squares đã gặp trong simple linear regression. Ta chọn $\beta_0, \beta_1, \dots, \beta_p$ để giảm thiểu tổng bình phương residual

$$\begin{aligned} \text{RSS} &= \sum_{i=1}^n (y_i - \hat{y}_i)^2 \\ &= \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \hat{\beta}_2 x_{i2} - \dots - \hat{\beta}_p x_{ip})^2. \end{aligned} \quad (3.22)$$

Các giá trị $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ giảm thiểu (3.22) là multiple least squares regression coefficient estimate. Khác với simple linear regression estimate trong (3.4), multiple regression coefficient estimate có dạng khá phức tạp và dễ biểu diễn nhất bằng matrix algebra. Vì vậy, chúng tôi không trình bày công thức ở đây. Mọi package thống kê đều có thể tính các coefficient estimate này; phần sau của chương sẽ chỉ ra cách thực hiện trong R. Hình 3.4 minh họa least squares fit cho một toy dataset có $p = 2$ predictor.

Bảng 3.4 trình bày multiple regression coefficient estimate khi dùng ngân sách quảng cáo *TV*, *radio* và *newspaper* để dự đoán doanh số sản phẩm trong dữ liệu *Advertising*. Ta diễn giải kết quả như sau: với ngân sách *TV* và *newspaper* không đổi, chỉ thêm 1.000 đô la cho quảng cáo *radio* đi kèm khoảng 189 đơn vị doanh số tăng thêm. So sánh những coefficient estimate này với Bảng 3.1 và Bảng 3.3, ta thấy multiple regression



HÌNH 3.4. Trong không gian ba chiều với hai predictor và một response, least squares regression line trở thành một mặt phẳng. Mặt phẳng được chọn để giảm thiểu tổng bình phương khoảng cách theo phương thẳng đứng giữa mỗi observation, biểu diễn bằng màu đỏ, và mặt phẳng.

BẢNG 3.5. Correlation matrix của TV, radio, newspaper và sales trong dữ liệu Advertising.

	TV	radio	newspaper	sales
TV	1.0000	0.0548	0.0567	0.7822
radio		1.0000	0.3541	0.5762
newspaper			1.0000	0.2283
sales				1.0000

coefficient estimate của TV và radio khá giống simple linear regression coefficient estimate. Tuy nhiên, dù coefficient estimate của newspaper trong Bảng 3.3 khác không có ý nghĩa thống kê, coefficient estimate của newspaper trong multiple regression model lại gần 0, và p -value tương ứng không còn significant, vào khoảng 0,86. Điều này minh họa rằng simple regression coefficient và multiple regression coefficient có thể khá khác nhau. Khác biệt bắt nguồn từ việc trong simple regression, slope biểu diễn mức tăng trung bình của doanh số sản phẩm đi kèm mức tăng 1.000 đô la cho quảng cáo newspaper, bỏ qua các predictor khác như TV và radio. Trái lại, trong multiple regression, coefficient của newspaper biểu diễn mức tăng trung bình của doanh số đi kèm mức tăng 1.000 đô la chỉ cho newspaper trong khi giữ TV và radio cố định.

Việc multiple regression cho rằng sales không có quan hệ với newspaper, trong khi simple linear regression lại gợi ý điều ngược lại, có hợp lý không? Thực ra là có. Xét correlation matrix của ba predictor variable và response variable trong Bảng 3.5. Lưu ý correlation giữa radio và newspaper là 0,35.

Điều này cho thấy các thị trường chi nhiều cho quảng cáo newspaper thường cũng chi nhiều cho radio. Bây giờ, giả sử multiple regression là đúng: quảng cáo newspaper không associated với doanh số, còn quảng cáo radio thì có. Khi đó, ở những thị trường chi nhiều hơn cho radio, doanh số có xu hướng cao hơn; như correlation matrix cho thấy, tại chính những thị trường này ta cũng thường chi nhiều hơn cho newspaper. Vì vậy, trong simple linear regression chỉ xét **sales** theo **newspaper**, ta quan sát thấy giá trị newspaper cao thường đi kèm sales cao, dù quảng cáo newspaper không trực tiếp associated với sales. Như vậy, newspaper là một surrogate cho radio; newspaper được “ghi công” cho association giữa radio và sales.

Kết quả hơi trái trực giác này rất phổ biến trong đời thực. Xét một ví dụ phi lý để minh họa. Regress số vụ cá mập tấn công trên doanh số kem bằng dữ liệu thu thập tại một cộng đồng ven biển theo thời gian sẽ cho thấy mối quan hệ dương, tương tự mối quan hệ giữa sales và newspaper. Dĩ nhiên, chưa ai đề xuất cấm kem ở bãi biển để giảm cá mập tấn công. Thực tế là nhiệt độ cao khiến nhiều người đến bãi biển hơn, từ đó làm tăng cả doanh số kem lẫn số vụ cá mập tấn công. Multiple regression số vụ cá mập tấn công trên doanh số kem và nhiệt độ cho thấy, đúng như trực giác, doanh số kem không còn là predictor significant sau khi điều chỉnh theo nhiệt độ.

3.2.2 Một số câu hỏi quan trọng

Khi thực hiện multiple linear regression, ta thường quan tâm đến một số câu hỏi quan trọng:

1. Có ít nhất một predictor $X_1, X_2, \dots, X_p$ hữu ích để dự đoán response không?
2. Tất cả predictor đều giúp giải thích Y , hay chỉ một tập con là hữu ích?
3. Mô hình fit dữ liệu tốt đến đâu?
4. Với một tập giá trị predictor, nên dự đoán response bằng giá trị nào, và dự đoán chính xác đến đâu?

Sau đây, ta lần lượt giải quyết từng câu hỏi.

Một: Response và các predictor có quan hệ không?

Nhớ lại trong simple linear regression, để xác định response và predictor có quan hệ hay không, ta chỉ cần kiểm tra $\beta_1 = 0$. Trong multiple regression với p predictor, ta cần hỏi mọi regression coefficient có đồng thời bằng 0 hay không, tức $\beta_1 = \beta_2 = \dots = \beta_p = 0$. Giống simple linear regression, ta dùng hypothesis test để trả lời. Null hypothesis là

$$H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0,$$

so với alternative

$$H_a : \text{có ít nhất một } \beta_j \text{ khác } 0.$$

Hypothesis test này được thực hiện bằng cách tính *F-statistic*

F-statistic

BẢNG 3.6. Thông tin bổ sung về least squares model giữa số đơn vị bán được với ngân sách quảng cáo TV, newspaper và radio trong dữ liệu Advertising. Thông tin khác về mô hình nằm trong Bảng 3.4.

Quantity	Value
Residual standard error	1.69
R^2	0.897
F -statistic	570

$$F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n - p - 1)}, \quad (3.23)$$

trong đó, giống simple linear regression, $\text{TSS} = \sum_i (y_i - \bar{y})^2$ và $\text{RSS} = \sum_i (y_i - \hat{y}_i)^2$. Nếu các giả định của linear model đúng, có thể chỉ ra rằng

$$E\{\text{RSS}/(n - p - 1)\} = \sigma^2,$$

và với điều kiện H_0 đúng,

$$E\{(\text{TSS} - \text{RSS})/p\} = \sigma^2.$$

Vì thế, khi response và các predictor không có quan hệ, ta kỳ vọng F -statistic gần 1. Trái lại, nếu H_a đúng thì $E\{(\text{TSS} - \text{RSS})/p\} > \sigma^2$, nên ta kỳ vọng $F > 1$.

F -statistic của multiple linear regression model giữa sales với radio, TV và newspaper được trình bày trong Bảng 3.6. Trong ví dụ này, F -statistic bằng 570. Vì giá trị này lớn hơn 1 rất nhiều, nó cung cấp bằng chứng thuyết phục chống lại null hypothesis H_0 . Nói cách khác, F -statistic lớn gợi ý rằng ít nhất một phương tiện quảng cáo phải liên hệ với doanh số. Nhưng nếu F -statistic gần 1 hơn thì sao? F -statistic phải lớn đến đâu mới có thể bác bỏ H_0 và kết luận tồn tại mối quan hệ? Hóa ra câu trả lời phụ thuộc vào n và p . Khi n lớn, F -statistic chỉ lớn hơn 1 một chút vẫn có thể cung cấp bằng chứng chống lại H_0 . Ngược lại, nếu n nhỏ, cần F -statistic lớn hơn để bác bỏ H_0 . Khi H_0 đúng và error ϵ_i có phân phối normal, F -statistic tuân theo phân phối F .⁶ Với giá trị n, p bất kỳ, software thống kê có thể dùng phân phối này để tính p -value của F -statistic. Dựa vào p -value, ta xác định có bác bỏ H_0 hay không. Với dữ liệu advertising, p -value gần với F -statistic trong Bảng 3.6 gần như bằng 0, nên có bằng chứng cực mạnh rằng ít nhất một phương tiện associated với doanh số tăng.

Trong (3.23), ta kiểm tra H_0 rằng tất cả coefficient đều bằng 0. Đôi khi, ta muốn kiểm tra một tập con gồm q coefficient bằng 0. Điều này tương ứng với null hypothesis

$$H_0 : \beta_{p-q+1} = \beta_{p-q+2} = \cdots = \beta_p = 0,$$

trong đó để tiện trình bày, ta đặt các biến được chọn để loại bỏ ở cuối danh sách. Khi đó, ta fit một mô hình thứ hai dùng tất cả các biến trừ q biến

⁶Ngay cả khi error không có phân phối normal, F -statistic vẫn xấp xỉ phân phối F với điều kiện cỡ mẫu n lớn.

cuối. Giả sử residual sum of squares của mô hình đó là RSS_0 . F -statistic phù hợp là

$$F = \frac{(RSS_0 - RSS)/q}{RSS/(n - p - 1)}. \quad (3.24)$$

Lưu ý rằng trong Bảng 3.4, mỗi predictor riêng lẻ có một t -statistic và một p -value. Chúng cho biết từng predictor có liên hệ với response hay không, sau khi điều chỉnh theo các predictor khác. Hóa ra mỗi kiểm tra đó hoàn toàn tương đương⁷ với F -test loại một biến đó khỏi mô hình trong khi giữ nguyên tất cả biến khác, tức $q = 1$ trong (3.24). Vì vậy, nó báo cáo partial effect của việc thêm biến đó vào mô hình. Chẳng hạn, như đã thảo luận, các p -value cho thấy **TV** và **radio** liên hệ với **sales**, nhưng không có bằng chứng **newspaper** associated với **sales** khi giữ **TV**, **radio** cố định.

Khi đã có p -value riêng của từng biến, tại sao còn phải xem F -statistic tổng thể? Có vẻ nếu bất kỳ p -value riêng lẻ nào rất nhỏ thì ít nhất một predictor phải liên hệ với response. Tuy nhiên, lập luận này có sai sót, đặc biệt khi số predictor p lớn.

Chẳng hạn, xét ví dụ có $p = 100$ và $H_0 : \beta_1 = \beta_2 = \dots = \beta_p = 0$ đúng, tức không biến nào thực sự associated với response. Trong tình huống này, do ngẫu nhiên, khoảng 5% số p -value gần với từng biến sẽ nhỏ hơn 0,05. Nói cách khác, ta kỳ vọng thấy xấp xỉ năm p -value nhỏ ngay cả khi predictor và response không có association thực.⁸ Thật vậy, khả năng ta tình cờ quan sát được ít nhất một p -value dưới 0,05 là cao. Vì thế, nếu dùng các t -statistic riêng lẻ và p -value tương ứng để quyết định có association giữa các biến và response hay không, khả năng kết luận sai là rất cao. F -statistic không gặp vấn đề này vì nó điều chỉnh theo số predictor. Do đó, nếu H_0 đúng, xác suất F -statistic tạo ra p -value dưới 0,05 chỉ là 5%, bất kể số predictor hay số observations.

Cách dùng F -statistic để kiểm tra association bất kỳ giữa các predictor và response hoạt động khi p tương đối nhỏ, và chắc chắn nhỏ so với n . Nhưng đôi khi ta có số biến rất lớn. Nếu $p > n$, số coefficient β_j cần ước lượng nhiều hơn số observations dùng để ước lượng chúng. Khi đó, ta thậm chí không thể fit multiple linear regression model bằng least squares, nên không thể dùng F -statistic hay phần lớn khái niệm đã gặp đến đây trong chương. Khi p lớn, có thể dùng một số cách tiếp cận được trình bày ở mục tiếp theo, chẳng hạn forward selection. Bối cảnh *high-dimensional* này được thảo luận chi tiết hơn trong Chương 6.

high-
dimensional

Hai: Quyết định các biến quan trọng

Như đã thảo luận, bước đầu tiên trong phân tích multiple regression là tính F -statistic và xem p -value tương ứng. Nếu dựa vào p -value ta kết luận ít nhất một predictor liên hệ với response, câu hỏi tự nhiên tiếp theo là predictor nào! Ta có thể xem p -value riêng lẻ như trong Bảng 3.4, nhưng như đã thảo luận, và sẽ được khảo sát thêm ở Chương 13, nếu p lớn thì khả năng có một số false discovery là cao.

Có thể tất cả predictor đều associated với response, nhưng thường response chỉ associated với một tập con predictor. Nhiệm vụ xác định

⁷Bình phương của mỗi t -statistic chính là F -statistic tương ứng.

⁸Điều này liên hệ với khái niệm quan trọng multiple testing, trọng tâm của Chương 13.

những predictor nào associated với response để fit một mô hình duy nhất chỉ chứa các predictor đó được gọi là *variable selection*. Bài toán variable selection được nghiên cứu kỹ ở Chương 6; ở đây, chúng tôi chỉ phác thảo một số cách tiếp cận cổ điển.

variable
selection

Lý tưởng nhất, ta muốn thực hiện variable selection bằng cách thử nhiều mô hình khác nhau, mỗi mô hình chứa một tập con predictor khác nhau. Chẳng hạn, nếu $p = 2$, ta có thể xét bốn mô hình: (1) không chứa biến nào; (2) chỉ chứa X_1 ; (3) chỉ chứa X_2 ; và (4) chứa cả X_1, X_2 . Sau đó, ta chọn mô hình tốt nhất trong số đã xét. Làm sao xác định mô hình nào tốt nhất? Có thể dùng nhiều thống kê để đánh giá chất lượng mô hình, gồm *Mallow's C_p* , *Akaike information criterion (AIC)*, *Bayesian information criterion (BIC)*, và *adjusted R^2* . Các đại lượng này được thảo luận chi tiết hơn ở Chương 6. Ta cũng có thể xác định mô hình tốt nhất bằng cách vẽ các output của mô hình, như residual, để tìm pattern.

Mallow's C_p
Akaike
information
criterion
Bayesian
information
criterion
adjusted R^2

Tiếc rằng có tổng cộng 2^p mô hình chứa các tập con của p biến. Nghĩa là ngay cả với p vừa phải, việc thử mọi tập con predictor cũng bất khả thi. Chẳng hạn, nếu $p = 2$, có $2^2 = 4$ mô hình cần xét. Nhưng nếu $p = 30$, ta phải xét $2^{30} = 1.073.741.824$ mô hình! Điều này không thực tế. Vì vậy, trừ khi p rất nhỏ, ta không thể xét tất cả 2^p mô hình mà cần một cách tiếp cận tự động, hiệu quả để chọn một tập mô hình nhỏ hơn. Có ba cách tiếp cận cổ điển:

- *Forward selection.* Ta bắt đầu với *null model*, tức mô hình có intercept nhưng không có predictor. Sau đó, ta fit p simple linear regression và thêm vào null model biến tạo ra RSS thấp nhất. Tiếp theo, thêm biến tạo ra RSS thấp nhất cho mô hình mới có hai biến. Tiếp tục cho đến khi thỏa một stopping rule nào đó.
- *Backward selection.* Ta bắt đầu với tất cả biến trong mô hình và loại biến có p -value lớn nhất, tức biến ít statistically significant nhất. Fit mô hình mới có $p - 1$ biến rồi loại biến có p -value lớn nhất. Tiếp tục đến khi đạt stopping rule; chẳng hạn, dừng khi mọi biến còn lại có p -value thấp hơn một threshold.
- *Mixed selection.* Đây là sự kết hợp giữa forward và backward selection. Ta bắt đầu không có biến nào trong mô hình; giống forward selection, thêm biến tạo fit tốt nhất và tiếp tục thêm từng biến. Như đã thấy với ví dụ **Advertising**, p -value của biến có thể tăng khi thêm predictor mới. Vì thế, nếu tại bất kỳ thời điểm nào p -value của một biến trong mô hình vượt threshold, ta loại biến đó. Tiếp tục các bước tiến và lùi cho đến khi mọi biến trong mô hình đều có p -value đủ thấp, còn mọi biến ngoài mô hình sẽ có p -value lớn nếu được thêm vào.

Forward
selection
null model

Backward
selection

Mixed
selection

Backward selection không thể dùng nếu $p > n$, còn forward selection luôn dùng được. Forward selection là một greedy approach và có thể sớm đưa vào những biến về sau trở nên redundant. Mixed selection có thể khắc phục điều này.

Ba: Model fit

Hai thước đo bằng số phổ biến nhất của model fit là RSE và R^2 , tức tỷ lệ variance được giải thích. Chúng được tính và diễn giải giống như với simple linear regression.

Nhớ rằng trong simple regression, R^2 là bình phương correlation giữa response và biến. Trong multiple linear regression, nó bằng $\text{Cor}(Y, \hat{Y})^2$, tức bình phương correlation giữa response và linear model đã fit. Thực ra, một tính chất của linear model đã fit là nó tối đa hóa correlation này trong mọi linear model có thể có.

R^2 gần 1 cho thấy mô hình giải thích phần lớn variance của response variable. Chẳng hạn, trong Bảng 3.6, mô hình dùng cả ba phương tiện quảng cáo để dự đoán sales có $R^2 = 0.8972$. Mặt khác, mô hình chỉ dùng TV và radio có $R^2 = 0.89719$. Nói cách khác, thêm quảng cáo newspaper vào mô hình đã có TV, radio chỉ làm R^2 tăng rất ít, dù trước đó ta thấy p -value của newspaper trong Bảng 3.4 không significant. Hóa ra R^2 luôn tăng khi thêm biến vào mô hình, ngay cả khi những biến ấy chỉ associated yếu với response. Nguyên nhân là thêm biến luôn làm residual sum of squares trên training data giảm, dù chưa chắc giảm trên test data. Vì R^2 cũng được tính trên training data, nó bắt buộc phải tăng. Việc thêm newspaper vào mô hình chỉ có TV và radio làm R^2 tăng rất nhỏ cung cấp thêm bằng chứng rằng có thể loại newspaper. Về cơ bản, newspaper không thực sự cải thiện model fit trên training sample; đưa nó vào có thể gây kết quả kém trên independent test sample do overfitting.

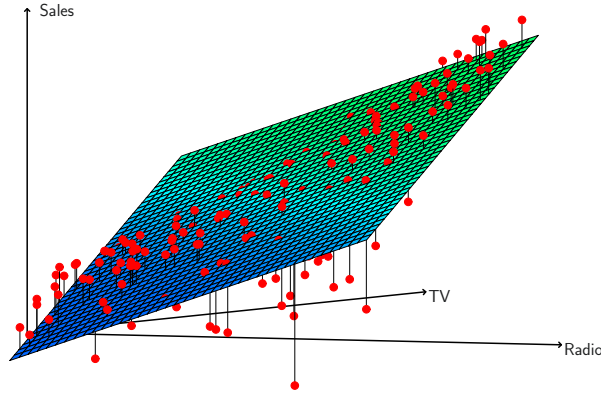
Trái lại, mô hình chỉ chứa TV có $R^2 = 0.61$ (Bảng 3.2). Thêm radio làm R^2 cải thiện đáng kể. Điều này cho thấy mô hình dùng chỉ tiêu TV, radio để dự đoán sales tốt hơn đáng kể mô hình chỉ dùng quảng cáo TV. Ta có thể định lượng thêm mức cải thiện bằng cách xem p -value của coefficient radio trong mô hình chỉ chứa TV và radio.

Mô hình chỉ có TV, radio làm predictor có RSE bằng 1,681, còn mô hình có thêm newspaper có RSE bằng 1,686 (Bảng 3.6). Ngược lại, mô hình chỉ chứa TV có RSE bằng 3,26 (Bảng 3.2). Điều này xác nhận kết luận trước đó: mô hình dùng chỉ tiêu TV và radio để dự đoán sales chính xác hơn nhiều trên training data so với mô hình chỉ dùng TV. Hơn nữa, khi TV và radio đã là predictor, không có lý do gì thêm chỉ tiêu newspaper vào mô hình. Bạn đọc tinh ý có thể thắc mắc tại sao RSE lại tăng khi thêm newspaper, dù RSS bắt buộc giảm. Nói chung,

$$\text{RSE} = \sqrt{\frac{1}{n - p - 1} \text{RSS}}, \quad (3.25)$$

và công thức này rút gọn thành (3.15) cho simple linear regression. Do đó, mô hình có nhiều biến hơn có thể có RSE cao hơn nếu mức giảm RSS nhỏ so với mức tăng p .

Ngoài RSE và R^2 , việc vẽ dữ liệu cũng hữu ích. Tóm tắt bằng đồ thị có thể bộc lộ những vấn đề của mô hình mà thống kê bằng số không cho thấy. Chẳng hạn, Hình 3.5 biểu diễn đồ thị ba chiều của TV, radio và sales. Một số observations nằm trên, số khác nằm dưới least squares regression plane. Cụ thể, linear model dường như ước lượng sales quá cao khi phần lớn tiền quảng cáo chỉ được chi riêng cho TV hoặc radio. Nó ước lượng sales quá



HÌNH 3.5. Với dữ liệu *Advertising*, linear regression fit cho *sales* dùng *TV* và *radio* làm predictor. Pattern của residual cho thấy dữ liệu có mối quan hệ phi tuyến rõ rệt. Residual dương, nhìn thấy phía trên mặt phẳng, thường nằm dọc đường 45 độ nơi ngân sách TV và radio được chia đều. Residual âm, phần lớn không nhìn thấy, thường nằm xa đường này, nơi ngân sách nghiêng hẳn về một phương tiện.

thấp khi ngân sách được chia cho hai phương tiện. Pattern phi tuyến rõ rệt này gợi ý *synergy* hay *interaction effect* giữa các phương tiện: kết hợp các phương tiện tạo ra mức tăng doanh số lớn hơn dùng riêng một phương tiện. Trong Mục 3.3.2, ta sẽ mở rộng linear model để chứa synergy effect bằng interaction term.

synergy
interaction
effect

Bốn: Prediction

Sau khi fit multiple regression model, ta dễ dàng áp dụng (3.21) để dự đoán response Y dựa trên một tập giá trị predictor $X_1, X_2, \dots, X_p$. Tuy nhiên, dự đoán này có ba loại uncertainty:

1. Các coefficient estimate $\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_p$ là ước lượng của $\beta_0, \beta_1, \dots, \beta_p$. Nghĩa là least squares plane

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p$$

chỉ là một ước lượng của population regression plane thực

$$f(X) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p.$$

Độ thiếu chính xác trong coefficient estimate liên hệ với reducible error ở Chương 2. Ta có thể tính confidence interval để xác định $\hat{Y}$ gần $f(X)$ đến đâu.

2. Trong thực tế, giả định linear model cho $f(X)$ hầu như luôn là một phép xấp xỉ thực tế, nên có thêm một nguồn reducible error tiềm tàng gọi là *model bias*. Vì vậy, khi dùng linear model, thực chất ta đang ước lượng phép xấp xỉ tuyến tính tốt nhất của bề mặt thực. Tuy nhiên, ở đây ta bỏ qua khác biệt này và xử lý như thể linear model là đúng.

model bias

3. Ngay cả khi biết $f(X)$, tức biết giá trị thực của $\beta_0, \beta_1, \dots, \beta_p$, ta vẫn không thể dự đoán hoàn hảo response do random error ϵ trong mô hình (3.20). Trong Chương 2, đây được gọi là irreducible error. Y sẽ biến thiên bao nhiêu quanh $\hat{Y}$? Ta dùng prediction interval để trả lời. Prediction interval luôn rộng hơn confidence interval vì nó bao gồm cả sai số trong ước lượng $f(X)$ (reducible error) lẫn uncertainty về mức độ một điểm riêng lẻ khác population regression plane (irreducible error).

Ta dùng *confidence interval* để định lượng uncertainty quanh doanh số trung bình trên một số lượng lớn thành phố. Chẳng hạn, giả sử tại mỗi thành phố chi 100.000 đô la cho quảng cáo TV và 20.000 đô la cho radio, confidence interval 95% là [10.985, 11.528]. Ta diễn giải rằng 95% số khoảng có dạng này sẽ chứa giá trị thực của $f(X)$.⁹ Mặt khác, có thể dùng *prediction interval* để định lượng uncertainty quanh doanh số của một thành phố cụ thể. Nếu tại thành phố đó chi 100.000 đô la cho quảng cáo TV và 20.000 đô la cho radio, prediction interval 95% là [7.930, 14.580]. Ta diễn giải rằng 95% số khoảng có dạng này sẽ chứa giá trị thực của Y cho thành phố đó. Lưu ý cả hai khoảng đều có tâm tại 11.256, nhưng prediction interval rộng hơn đáng kể confidence interval, phản ánh uncertainty lớn hơn về doanh số tại một thành phố cụ thể so với doanh số trung bình trên nhiều địa điểm.

confidence
interval

prediction
interval

⁹Nói cách khác, nếu thu thập nhiều dataset giống *Advertising*, rồi từ mỗi dataset dựng một confidence interval của doanh số trung bình với 100.000 đô la cho TV và 20.000 đô la cho radio, thì 95% số confidence interval đó sẽ chứa giá trị thực của doanh số trung bình.

3.3 Các cân nhắc khác trong Regression Model



3.3.1 Qualitative Predictors

Trong phần thảo luận đến đây, ta giả định rằng mọi biến trong linear regression model đều là quantitative. Nhưng trong thực tế điều này không nhất thiết đúng; một số predictor thường là qualitative.

Chẳng hạn, dataset **Credit** trong Hình 3.6 ghi lại các biến của một số chủ thẻ tín dụng. Response là **balance** (dư nợ thẻ tín dụng trung bình của mỗi người), cùng một số quantitative predictor: **age**, **cards** (số thẻ tín dụng), **education** (số năm học), **income** (nghìn đô la), **limit** (hạn mức tín dụng) và **rating** (điểm tín dụng). Mỗi panel trong Hình 3.6 là scatterplot của một cặp biến, được xác định bằng nhãn hàng và cột tương ứng. Ví dụ, scatterplot ngay bên phải từ “Balance” biểu diễn **balance** theo **age**, còn đồ thị ngay bên phải “Age” biểu diễn **age** theo **cards**. Ngoài các quantitative variable này, ta còn có bốn qualitative variable: **own** (sở hữu nhà), **student** (tình trạng sinh viên), **status** (tình trạng hôn nhân) và **region** (East, West hoặc South).

Predictor chỉ có hai level

Giả sử ta muốn khảo sát khác biệt về dư nợ thẻ tín dụng giữa người sở hữu nhà và người không sở hữu nhà, tạm thời bỏ qua các biến khác. Nếu một qualitative predictor (còn gọi là *factor*) chỉ có hai *level*, hay hai giá trị khả dĩ, việc đưa nó vào regression model rất đơn giản. Ta chỉ cần tạo một *indicator variable* hay *dummy variable* nhận hai giá trị số khả dĩ.¹ Chẳng hạn, từ biến **own**, ta tạo biến mới

$$x_i = \begin{cases} 1 & \text{nếu người thứ } i \text{ sở hữu nhà,} \\ 0 & \text{nếu người thứ } i \text{ không sở hữu nhà,} \end{cases} \quad (3.26)$$

rồi dùng biến này làm predictor trong regression equation. Khi đó ta có mô hình

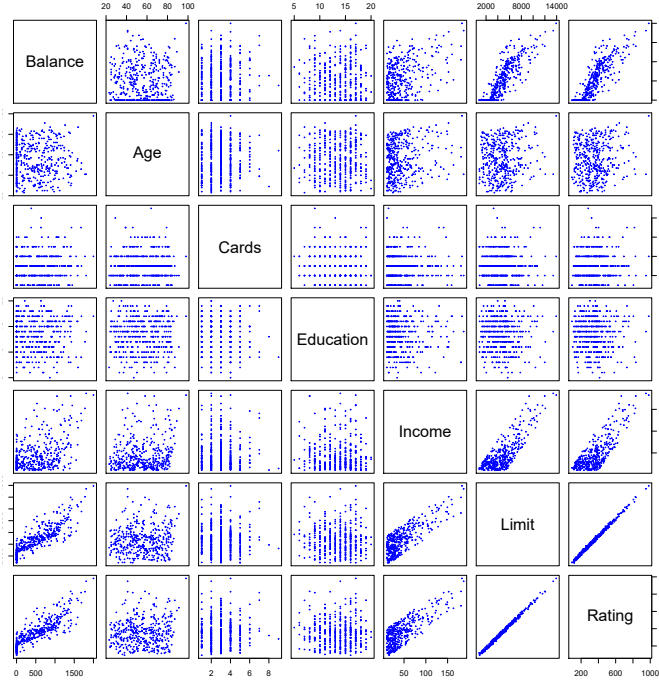
$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{nếu người thứ } i \text{ sở hữu nhà,} \\ \beta_0 + \epsilon_i & \text{nếu người thứ } i \text{ không sở hữu nhà.} \end{cases} \quad (3.27)$$

Giờ đây, β_0 được diễn giải là dư nợ thẻ tín dụng trung bình của nhóm không sở hữu nhà; $\beta_0 + \beta_1$ là dư nợ trung bình của nhóm sở hữu nhà; và β_1 là chênh lệch trung bình về dư nợ giữa hai nhóm.

Bảng 3.7 trình bày coefficient estimate và các thông tin khác của mô hình (3.27). Dư nợ thẻ tín dụng trung bình của người không sở hữu nhà được ước lượng là 509,80 đô la; nhóm sở hữu nhà được ước lượng có thêm 19,73 đô la dư nợ, tổng cộng $509.80 + 19.73 = 529.53$ đô la. Tuy nhiên, *p*-value của dummy variable rất cao. Vì vậy không có bằng chứng thống kê về khác biệt trong dư nợ trung bình dựa trên quyền sở hữu nhà.

Việc mã hóa nhóm sở hữu nhà là 1 và nhóm không sở hữu là 0 trong (3.27) hoàn toàn tùy ý. Nó không ảnh hưởng regression fit nhưng làm thay

¹Trong cộng đồng machine learning, việc tạo dummy variable để xử lý qualitative predictor được gọi là “one-hot encoding”.



HÌNH 3.6. Dataset *Credit* chứa thông tin về *balance*, *age*, *cards*, *education*, *income*, *limit* và *rating* của một số khách hàng tiềm năng.

BẢNG 3.7. Least squares coefficient estimate của regression *balance* trên *own* trong dataset *Credit*. Linear model được cho bởi (3.27); quyền sở hữu được mã hóa thành dummy variable như trong (3.26).

	Coefficient	Std. error	t-statistic	p-value
Intercept	509.80	33.13	15.389	< 0.0001
own[Yes]	19.73	46.05	0.429	0.6690

đổi cách diễn giải coefficient. Nếu mã hóa người không sở hữu là 1 và người sở hữu là 0, các ước lượng của β_0 và β_1 lần lượt sẽ là 529,53 và $-19,73$, vẫn cho dự đoán $529.53 - 19.73 = 509.80$ đô la với người không sở hữu và 529,53 đô la với người sở hữu. Một lựa chọn khác là dùng dummy variable

$$x_i = \begin{cases} 1 & \text{nếu người thứ } i \text{ sở hữu nhà,} \\ -1 & \text{nếu người thứ } i \text{ không sở hữu nhà,} \end{cases}$$

trong regression equation, tạo thành mô hình

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{nếu người thứ } i \text{ sở hữu nhà,} \\ \beta_0 - \beta_1 + \epsilon_i & \text{nếu người thứ } i \text{ không sở hữu nhà.} \end{cases}$$

Lúc này β_0 có thể được diễn giải là dư nợ thẻ tín dụng trung bình tổng thể (bỏ qua ảnh hưởng của quyền sở hữu nhà), còn β_1 là mức mà dư nợ

BẢNG 3.8. *Least squares coefficient estimate của regression **balance** trên **region** trong dataset **Credit**. Linear model được cho bởi (3.30); **region** được mã hóa bằng hai dummy variable (3.28) và (3.29).*

	Coefficient	Std. error	t-statistic	p-value
Intercept	531.00	46.32	11.464	< 0.0001
region[South]	-12.50	56.68	-0.221	0.8260
region[West]	-18.69	65.02	-0.287	0.7740

của nhóm sở hữu và không sở hữu lần lượt nằm trên và dưới trung bình.² Trong ví dụ này, $\hat{\beta}_0 = 519.665$ đô la, nằm giữa hai trung bình 509,80 và 529,53 đô la; $\hat{\beta}_1 = 9.865$ đô la, bằng một nửa chênh lệch trung bình 19,73 đô la. Dù dùng cách mã hóa nào, dự đoán cuối cùng cho hai nhóm vẫn giống hệt nhau; chỉ cách diễn giải coefficient thay đổi.

Qualitative predictor có nhiều hơn hai level

Khi qualitative predictor có nhiều hơn hai level, một dummy variable không thể biểu diễn mọi giá trị khả dĩ. Ta có thể tạo thêm dummy variable. Ví dụ, với biến **region**, ta tạo hai biến. Biến thứ nhất là

$$x_{i1} = \begin{cases} 1 & \text{nếu người thứ } i \text{ đến từ South,} \\ 0 & \text{nếu người thứ } i \text{ không đến từ South,} \end{cases} \quad (3.28)$$

và biến thứ hai là

$$x_{i2} = \begin{cases} 1 & \text{nếu người thứ } i \text{ đến từ West,} \\ 0 & \text{nếu người thứ } i \text{ không đến từ West.} \end{cases} \quad (3.29)$$

Đưa cả hai biến vào regression equation cho mô hình

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \epsilon_i = \begin{cases} \beta_0 + \beta_1 + \epsilon_i & \text{nếu người thứ } i \text{ đến từ South,} \\ \beta_0 + \beta_2 + \epsilon_i & \text{nếu người thứ } i \text{ đến từ West,} \\ \beta_0 + \epsilon_i & \text{nếu người thứ } i \text{ đến từ East.} \end{cases} \quad (3.30)$$

Ta diễn giải β_0 là dư nợ thể tín dụng trung bình của người ở East; β_1 là chênh lệch dư nợ trung bình giữa South và East; còn β_2 là chênh lệch giữa West và East. Số dummy variable luôn ít hơn số level một đơn vị. Level không có dummy variable—trong ví dụ này là East—được gọi là *baseline*. baseline

Từ Bảng 3.8, estimated balance của baseline East là 531,00 đô la. Nhóm South được ước lượng có dư nợ thấp hơn East 12,50 đô la, còn nhóm West thấp hơn East 18,69 đô la. Tuy nhiên, p -value của hai dummy variable đều rất lớn, cho thấy không có bằng chứng thống kê về khác biệt thực sự trong dư nợ trung bình giữa South với East hay West với East.³ Baseline được chọn tùy ý và dự đoán cuối cùng của mỗi nhóm không phụ thuộc lựa chọn

²Chính xác hơn, β_0 bằng một nửa tổng dư nợ trung bình của hai nhóm. Do đó, β_0 chỉ đúng bằng trung bình tổng thể khi hai nhóm có cùng số thành viên.

³Về lý thuyết vẫn có thể có khác biệt giữa South và West, dù dữ liệu ở đây không gợi ý khác biệt nào.

đó, nhưng coefficient và p -value của chúng thì có. Thay vì dựa vào từng coefficient, ta có thể dùng F -test cho $H_0 : \beta_1 = \beta_2 = 0$; kiểm định này không phụ thuộc cách mã hóa. F -test cho p -value 0,96, nên ta không thể bác bỏ null hypothesis rằng **balance** không liên hệ với **region**.

Dummy-variable approach không gây khó khăn khi đưa đồng thời quantitative và qualitative predictor vào mô hình. Chẳng hạn, để regress **balance** trên quantitative variable **income** và qualitative variable **student**, ta chỉ cần tạo dummy variable cho **student**, rồi fit multiple regression model dùng **income** và dummy variable đó để dự đoán **balance**.

Có nhiều cách mã hóa qualitative variable ngoài dummy-variable approach. Tất cả cho model fit tương đương, nhưng coefficient khác nhau, có cách diễn giải khác nhau và được thiết kế để đo những *contrast* cụ thể. Chủ đề này nằm ngoài phạm vi cuốn sách.

3.3.2 Mở rộng Linear Model

Linear regression model chuẩn (3.19) cho kết quả dễ diễn giải và hoạt động khá tốt trên nhiều bài toán thực tế. Tuy nhiên, nó đặt ra một số giả định rất hạn chế và thường bị vi phạm. Hai giả định quan trọng nhất nói rằng quan hệ giữa predictor và response có tính *additive* và *linear*. Giả định additivity nghĩa là association giữa predictor X_j và response Y không phụ thuộc giá trị của predictor khác. Giả định linearity nói rằng mức thay đổi của Y gắn với một đơn vị thay đổi của X_j là hằng số, bất kể giá trị của X_j . Ở các chương sau, ta sẽ gặp những phương pháp tinh vi hơn nhằm nói lỏng hai giả định này. Trước hết, ta xem nhanh một số cách mở rộng cổ điển thường dùng.

Loại bỏ giả định Additive

Trong phân tích dữ liệu **Advertising** trước đó, ta kết luận cả **TV** và **radio** dường như associated với **sales**. Các linear model làm nền tảng cho kết luận này giả định ảnh hưởng của việc tăng chi tiêu ở một phương tiện không phụ thuộc số tiền dành cho phương tiện kia. Ví dụ, mô hình (3.20) nói rằng mức tăng sales trung bình khi TV tăng một đơn vị luôn là β_1 , bất kể chi cho radio bao nhiêu.

Nhưng mô hình đơn giản ấy có thể sai. Giả sử chi tiền quảng cáo radio thực sự làm quảng cáo TV hiệu quả hơn, khiến slope của TV tăng khi radio tăng. Khi có ngân sách cố định 100.000 đô la, chia một nửa cho radio và một nửa cho TV có thể làm sales tăng nhiều hơn dồn toàn bộ cho một trong hai. Trong marketing, đây là *synergy effect*; trong statistics, nó được gọi là *interaction effect*. Hình 3.5 gợi ý hiệu ứng này có thể tồn tại: khi TV hoặc radio thấp, sales thực thấp hơn dự đoán của linear model; khi ngân sách được chia giữa hai phương tiện, mô hình lại có xu hướng dự đoán thấp hơn sales thực.

Xét linear regression model chuẩn với hai biến,

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \epsilon.$$

Theo mô hình này, X_1 tăng một đơn vị đi kèm mức tăng trung bình β_1 đơn vị của Y . Sự có mặt của X_2 không làm thay đổi phát biểu đó. Một

BẢNG 3.9. Với dữ liệu *Advertising*, least squares coefficient estimate của regression *sales* trên *TV* và *radio*, có interaction term như trong (3.33).

	Coefficient	Std. error	t-statistic	p-value
Intercept	6.7502	0.248	27.23	< 0.0001
TV	0.0191	0.002	12.70	< 0.0001
radio	0.0289	0.009	3.24	0.0014
TV×radio	0.0011	0.000	20.73	< 0.0001

cách mở rộng là thêm predictor thứ ba, gọi là *interaction term*, được tạo bằng tích X_1X_2 : interaction term

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2 + \epsilon. \quad (3.31)$$

Vì sao interaction term giúp nới lỏng giả định additive? Ta có thể viết lại (3.31) thành

$$\begin{aligned} Y &= \beta_0 + (\beta_1 + \beta_3 X_2) X_1 + \beta_2 X_2 + \epsilon \\ &= \beta_0 + \tilde{\beta}_1 X_1 + \beta_2 X_2 + \epsilon, \end{aligned} \quad (3.32)$$

trong đó $\tilde{\beta}_1 = \beta_1 + \beta_3 X_2$. Vì $\tilde{\beta}_1$ nay là hàm của X_2 , association giữa X_1 và Y không còn cố định: thay đổi X_2 sẽ làm thay đổi association giữa X_1 và Y . Lập luận tương tự cho thấy thay đổi X_1 cũng làm thay đổi association giữa X_2 và Y .

Ví dụ, giả sử ta nghiên cứu năng suất nhà máy và muốn dự đoán số đơn vị sản phẩm từ số dây chuyền cùng tổng số công nhân. Ảnh hưởng của việc tăng số dây chuyền có vẻ sẽ phụ thuộc số công nhân, bởi nếu không có người vận hành thì thêm dây chuyền không thể làm sản lượng tăng. Vì vậy, linear model dự đoán *units* nên có interaction term giữa *lines* và *workers*. Giả sử fit cho kết quả

$$\begin{aligned} \text{units} &\approx 1.2 + 3.4 \times \text{lines} + 0.22 \times \text{workers} + 1.4 \times (\text{lines} \times \text{workers}) \\ &= 1.2 + (3.4 + 1.4 \times \text{workers}) \times \text{lines} + 0.22 \times \text{workers}. \end{aligned}$$

Nói cách khác, thêm một dây chuyền làm số đơn vị sản xuất tăng $3.4 + 1.4 \times \text{workers}$. Do đó càng nhiều công nhân, ảnh hưởng của dây chuyền càng mạnh.

Trở lại ví dụ *Advertising*, linear model dùng *radio*, *TV* và interaction giữa hai biến để dự đoán *sales* có dạng

$$\begin{aligned} \text{sales} &= \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{radio} + \beta_3 \times (\text{radio} \times \text{TV}) + \epsilon \\ &= \beta_0 + (\beta_1 + \beta_3 \times \text{radio}) \times \text{TV} + \beta_2 \times \text{radio} + \epsilon. \end{aligned} \quad (3.33)$$

Ta có thể diễn giải β_3 là mức tăng hiệu quả của quảng cáo TV gắn với việc quảng cáo radio tăng một đơn vị (hoặc ngược lại). Các coefficient thu được khi fit (3.33) nằm trong Bảng 3.9.

Kết quả trong Bảng 3.9 cho thấy rõ mô hình chứa interaction term tốt hơn mô hình chỉ có *main effect*. *p*-value của *TV×radio* cực thấp, cung cấp bằng chứng mạnh cho $H_a : \beta_3 \neq 0$; quan hệ thực không phải additive. R^2 của mô hình (3.33) là 96,8%, so với 89,7% khi dự đoán sales bằng TV và main effect

radio mà không có interaction. Như vậy, $(96.8 - 89.7)/(100 - 89.7) = 69\%$ phần variability của sales còn lại sau additive model đã được interaction term giải thích. Các coefficient gợi ý rằng quảng cáo TV tăng 1.000 đô la gắn với sales tăng $(\hat{\beta}_1 + \hat{\beta}_3 \times \text{radio}) \times 1,000 = 19 + 1.1 \times \text{radio}$ đơn vị; quảng cáo radio tăng 1.000 đô la gắn với sales tăng $29 + 1.1 \times \text{TV}$ đơn vị.

Trong ví dụ này, TV, radio và interaction term đều statistically significant. Nhưng đôi khi interaction term có p -value rất nhỏ trong khi main effect tương ứng lại không. *Hierarchical principle* nói rằng nếu đưa interaction vào mô hình, ta cũng nên đưa các main effect vào, ngay cả khi p -value của coefficient không significant. Nếu $X_1 X_2$ liên hệ với response thì việc coefficient riêng của X_1 hay X_2 có đúng bằng 0 hay không không mấy quan trọng. Hơn nữa, $X_1 X_2$ thường correlated với X_1 và X_2 , nên bỏ chúng làm thay đổi ý nghĩa của interaction.

Hierarchical
principle

Khái niệm interaction cũng áp dụng cho qualitative variable hoặc kết hợp qualitative và quantitative variable. Thực ra, interaction giữa một qualitative variable và một quantitative variable có cách diễn giải rất tự nhiên. Xét dataset **Credit**; giả sử ta dự đoán **balance** bằng quantitative variable **income** và qualitative variable **student**. Không có interaction term, mô hình là

$$\text{balance}_i \approx \beta_0 + \beta_1 \text{income}_i + \begin{cases} \beta_2 & \text{nếu người thứ } i \text{ là student,} \\ 0 & \text{nếu không phải student,} \end{cases} \quad (3.34)$$

tức là fit hai đường song song: một cho student, một cho non-student. Hai đường có intercept khác nhau, $\beta_0 + \beta_2$ và β_0 , nhưng cùng slope β_1 , như panel trái Hình 3.7. Điều đó có nghĩa ảnh hưởng trung bình của một đơn vị tăng trong income lên balance không phụ thuộc tình trạng student. Đây có thể là hạn chế nghiêm trọng vì thay đổi income có thể ảnh hưởng rất khác đến balance của student và non-student.

Ta khắc phục bằng cách thêm interaction variable, tạo bởi tích của **income** với dummy variable của **student**:

$$\text{balance}_i \approx \begin{cases} (\beta_0 + \beta_2) + (\beta_1 + \beta_3) \text{income}_i & \text{nếu là student,} \\ \beta_0 + \beta_1 \text{income}_i & \text{nếu không phải student.} \end{cases} \quad (3.35)$$

Ta vẫn có hai regression line, nhưng giờ chúng khác cả intercept lẫn slope. Mô hình cho phép thay đổi income ảnh hưởng khác nhau tới hai nhóm.

Panel phải Hình 3.7 cho thấy quan hệ ước lượng giữa income và balance trong mô hình (3.35). Slope của student thấp hơn slope của non-student, gợi ý rằng income tăng associated với mức tăng balance nhỏ hơn ở student so với non-student.

Quan hệ phi tuyến

Linear regression model (3.19) giả định quan hệ tuyến tính giữa response và predictor, nhưng quan hệ thực đôi khi phi tuyến. Một cách mở rộng trực tiếp, rất đơn giản là *polynomial regression*. Các chương sau sẽ trình bày những cách tổng quát và phức tạp hơn.

polynomial
regression

Hình 3.8 biểu diễn mpg (miles per gallon) theo horsepower của các xe trong dataset **Auto**. Đường cam là linear regression fit. Dữ liệu cho thấy

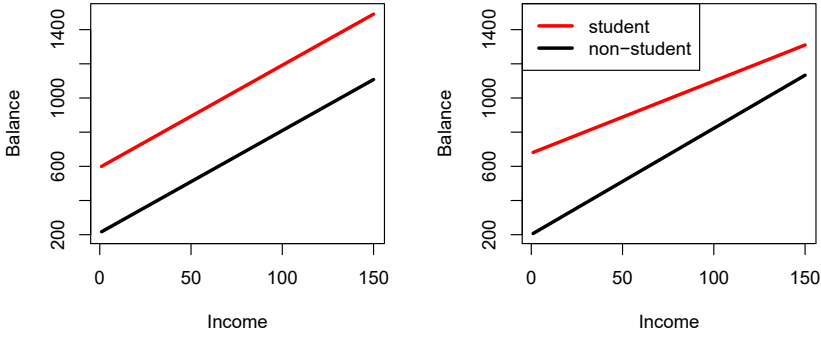


FIGURE 3.7. For the **Credit** data, the least squares lines are shown for pre-

HÌNH 3.7. Với dữ liệu **Credit**, các least squares line dự đoán **balance** từ **income** được biểu diễn cho **student** và **non-student**. Trái: Fit mô hình (3.34), không có interaction giữa **income** và **student**. Phải: Fit mô hình (3.35), có interaction term giữa **income** và **student**.

BẢNG 3.10. Với dataset **Auto**, least squares coefficient estimate của regression **mpg** trên **horsepower** và **horsepower²**.

	Coefficient	Std. error	<i>t</i> -statistic	<i>p</i> -value
Intercept	56.9001	1.8004	31.6	< 0.0001
horsepower	-0.4662	0.0311	-15.0	< 0.0001
horsepower ²	0.0012	0.0001	10.1	< 0.0001

quan hệ rõ rệt nhưng bị uốn cong, không phải tuyến tính. Ta có thể đưa các phiên bản biến đổi của predictor vào linear model. Hình dạng gần quadratic gợi ý mô hình

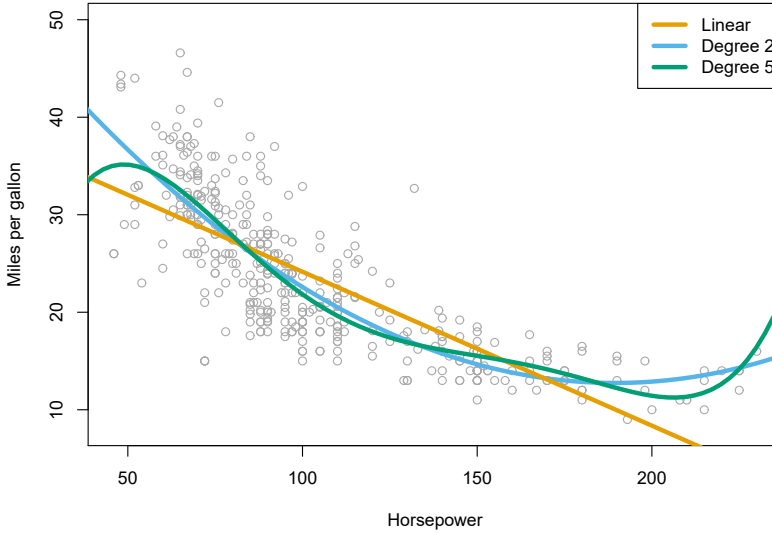
$$\text{mpg} = \beta_0 + \beta_1 \times \text{horsepower} + \beta_2 \times \text{horsepower}^2 + \epsilon. \quad (3.36)$$

Mặc dù (3.36) dự đoán mpg bằng một hàm phi tuyến của horsepower, đây vẫn là linear model: chỉ là multiple linear regression với $X_1 = \text{horsepower}$ và $X_2 = \text{horsepower}^2$. Do đó ta vẫn dùng software linear regression chuẩn để ước lượng $\beta_0, \beta_1, \beta_2$.

Đường cong xanh lam trong Hình 3.8 là quadratic fit. Nó tốt hơn đáng kể so với fit chỉ có linear term: $R^2 = 0.688$, so với 0,606 của linear fit, và *p*-value của quadratic term trong Bảng 3.10 rất significant.

Nếu thêm **horsepower²** cải thiện mô hình nhiều như vậy, tại sao không thêm bậc ba, bậc bốn hay bậc năm? Đường xanh lục trong Hình 3.8 là fit chứa mọi polynomial đến bậc năm. Fit thu được có vẻ gọn quá mức; chưa rõ các term bổ sung có thực sự cải thiện fit cho dữ liệu hay không.

Cách mở rộng linear model bằng polynomial function của predictor được gọi là polynomial regression. Ta sẽ khảo sát sâu hơn phương pháp này và các mở rộng phi tuyến khác của linear model trong Chương 7.



HÌNH 3.8. Dataset *Auto. mpg* và *horsepower* được biểu diễn cho một số xe. Linear regression fit màu cam. Fit của mô hình có horsepower^2 là đường cong xanh lam. Fit của mô hình chứa mọi polynomial của *horsepower* đến bậc năm có màu xanh lục.

3.3.3 Các vấn đề tiềm ẩn

Khi fit linear regression model cho một dataset cụ thể, nhiều vấn đề có thể xảy ra. Phổ biến nhất gồm:

1. Quan hệ response–predictor phi tuyến.
2. Các error term correlated.
3. Variance của error term không hằng.
4. Outlier.
5. High-leverage point.
6. Collinearity.

Trong thực tế, nhận diện và khắc phục những vấn đề này vừa là nghệ thuật vừa là khoa học. Vô số sách đã dành nhiều trang cho chủ đề đó. Vì linear regression model không phải trọng tâm chính ở đây, chúng tôi chỉ tóm lược một số điểm then chốt.

1. Tính phi tuyến của dữ liệu

Linear regression model giả định có quan hệ đường thẳng giữa predictor và response. Nếu quan hệ thực khác xa tuyến tính, gần như mọi kết luận từ model fit đều đáng ngờ và prediction accuracy có thể suy giảm đáng kể.

Residual plot là công cụ đồ họa hữu ích để nhận diện tính phi tuyến. Với simple linear regression, ta vẽ residual $e_i = y_i - \hat{y}_i$ theo predictor x_i . Với multiple regression có nhiều predictor, ta vẽ residual theo predicted hay

Residual plot

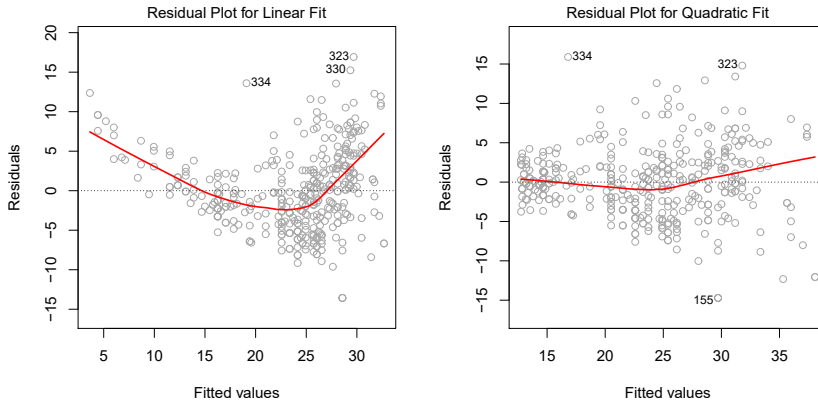


FIGURE 3.9. Plots of residuals versus predicted (or fitted) values for the **Auto** data set. In each plot, the red line is a smooth fit to the residuals, intended to make it easier to identify a trend. Left: A linear regression of **mpg** on **horsepower**. A strong pattern in the residuals indicates non-linearity in the data. Right: A linear regression of **mpg** on **horsepower** and **horsepower**². There is little pattern in the

HÌNH 3.9. Đồ thị residual theo predicted (hay fitted) value cho dataset **Auto**. Trong mỗi đồ thị, đường đỏ là smooth fit của residual nhằm làm trend dễ nhận thấy hơn. Trái: Linear regression của **mpg** trên **horsepower**; pattern mạnh trong residual cho thấy dữ liệu phi tuyến. Phải: Linear regression của **mpg** trên **horsepower** và **horsepower**²; residual hầu như không còn pattern.

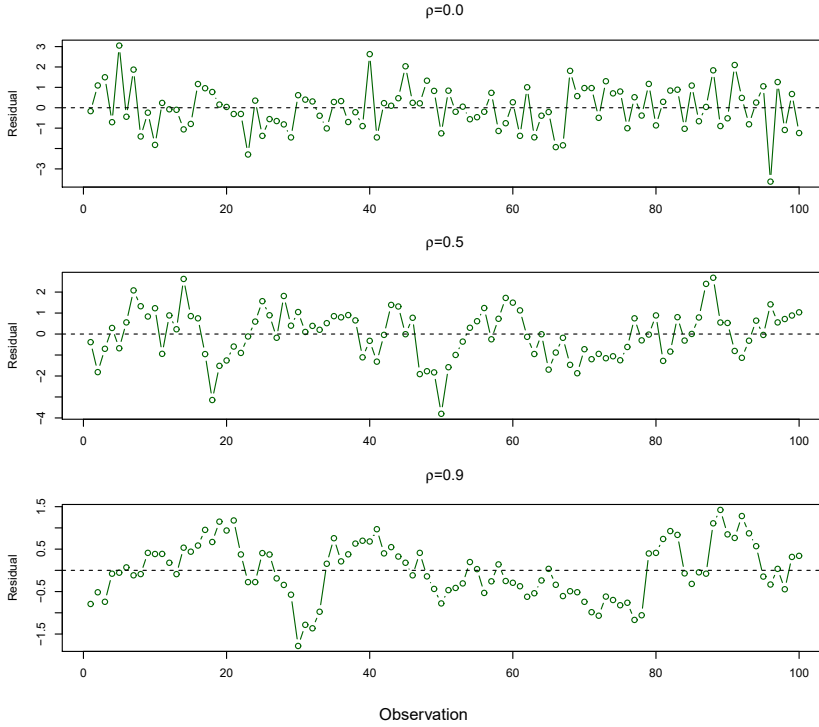
fitted value $\hat{y}_i$. Lý tưởng nhất, residual plot không có pattern dễ nhận biết; nếu có, một khía cạnh nào đó của linear model có thể đang gặp vấn đề.

Panel trái Hình 3.9 là residual plot của linear regression **mpg** trên **horsepower** trong dataset **Auto**. Đường đỏ là smooth fit giúp dễ thấy trend. Residual có hình chữ U rõ rệt, là dấu hiệu mạnh của tính phi tuyến. Panel phải là residual plot của mô hình (3.36) có quadratic term. Hầu như không còn pattern, cho thấy quadratic term cải thiện model fit.

Nếu residual plot cho thấy association phi tuyến, cách đơn giản là đưa các phép biến đổi phi tuyến của predictor như $\log X$, $\sqrt{X}$ và X^2 vào regression model. Các chương sau sẽ bàn về những phương pháp phi tuyến tiên tiến hơn.

2. Correlation của Error Term

Một giả định quan trọng của linear regression model là các error term $\epsilon_1, \epsilon_2, \dots, \epsilon_n$ không correlated. Nếu error không correlated, việc ϵ_i dương hầu như không cung cấp thông tin về dấu của ϵ_{i+1} . Standard error của regression coefficient estimate và fitted value được tính dựa trên giả định này. Nếu error term thực sự correlated, standard error ước lượng thường thấp hơn standard error thực, làm confidence interval và prediction interval hẹp hơn mức cần thiết. Chẳng hạn, confidence interval 95% trên thực tế có thể có xác suất chứa parameter thực thấp hơn 0,95 nhiều. p -value cũng thấp



HÌNH 3.10. Đồ thị residual của các time-series dataset mô phỏng, được tạo với các mức correlation ρ khác nhau giữa error term tại hai thời điểm liên kế.

hơn mức đúng, khiến ta có thể kết luận nhầm một parameter statistically significant. Tóm lại, error correlated có thể tạo ra sự tự tin không có cơ sở vào mô hình.

Một ví dụ cực đoan: giả sử ta vô tình nhân đôi dữ liệu, tạo ra từng cặp observation và error term giống hệt nhau. Nếu bỏ qua điều này, phép tính standard error sẽ hành xử như ta có sample size $2n$, trong khi thực tế chỉ có n sample. Parameter estimate vẫn giống nhau, nhưng confidence interval sẽ hẹp đi theo hệ số $\sqrt{2}$.

Correlation giữa error term thường xuất hiện trong *time series data*, nơi observation được đo tại các thời điểm rời rạc. Observation ở hai thời điểm liên kế thường có error correlated dương. Để kiểm tra, ta vẽ residual của mô hình theo thời gian. Nếu error không correlated, đồ thị không nên có pattern rõ rệt. Nếu correlated dương, residual có thể xuất hiện *tracking*: các residual liên kế nhận giá trị tương tự nhau.

Panel trên Hình 3.10 dùng error không correlated và không cho thấy trend theo thời gian. Panel dưới dùng correlation 0,9 giữa các error liên kế, nên residual có pattern rõ: các giá trị gần nhau có xu hướng giống nhau. Panel giữa là trường hợp vừa phải với correlation 0,5; tracking vẫn tồn tại nhưng kém rõ hơn.

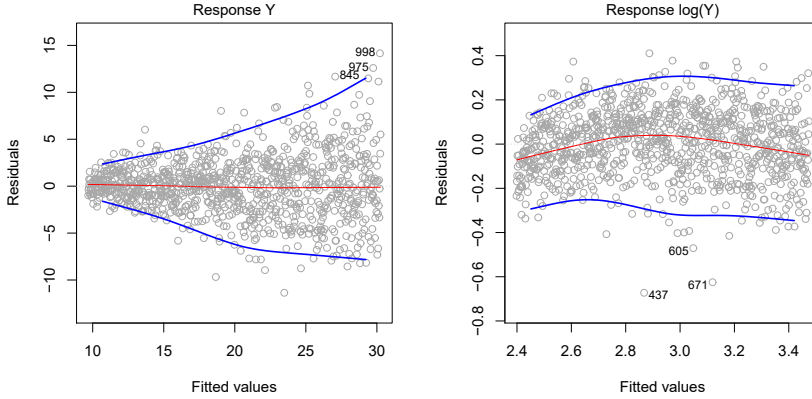


FIGURE 3.11. Residual plots. In each plot, the red line is a smooth fit to the residuals, intended to make it easier to identify a trend. The blue lines track the outer quantiles of the residuals, and emphasize patterns. Left: The funnel shape

HÌNH 3.11. Residual plot. Trong mỗi đồ thị, đường đỏ là smooth fit của residual; các đường xanh theo outer quantile và nhấn mạnh pattern. Trái: Hình phễu cho thấy heteroscedasticity. Phải: Response đã được log-transform và không còn bằng chứng về heteroscedasticity.

Nhiều phương pháp đã được phát triển để xử lý đúng correlation của error term trong time-series data. Hiện tượng này cũng có thể xuất hiện ngoài time series. Ví dụ, trong nghiên cứu dự đoán chiều cao từ cân nặng, giả định error không correlated có thể bị vi phạm nếu một số người cùng gia đình, cùng chế độ ăn hoặc cùng tiếp xúc với yếu tố môi trường. Giả định error không correlated cực kỳ quan trọng đối với linear regression và nhiều phương pháp thống kê khác; experimental design tốt là yếu tố then chốt để giảm nguy cơ correlation như vậy.

3. Variance của Error Term không hằng

Một giả định quan trọng khác là error term có variance hằng, $\text{Var}(\epsilon_i) = \sigma^2$. Standard error, confidence interval và hypothesis test của linear model đều dựa vào giả định này.

Thực tế, variance của error term thường không hằng; nó có thể tăng theo response. Ta nhận diện non-constant variance, hay *heteroscedasticity*, qua hình phễu trong residual plot. Panel trái Hình 3.11 cho thấy độ lớn residual tăng cùng fitted value. Một giải pháp là biến đổi response Y bằng hàm lõm như $\log Y$ hoặc $\sqrt{Y}$. Phép biến đổi co các response lớn nhiều hơn và làm giảm heteroscedasticity. Panel phải biểu diễn residual sau khi dùng $\log Y$; variance trông đã hằng, dù vẫn có chút bằng chứng về quan hệ phi tuyến.

Đôi khi ta biết khá rõ variance của từng response. Ví dụ, response thứ i có thể là trung bình của n_i raw observation. Nếu mỗi raw observation không correlated và có variance σ^2 , trung bình của chúng có variance $\sigma_i^2 = \sigma^2/n_i$. Khi đó, ta có thể fit mô hình bằng *weighted least squares*, với weight tỷ

weighted least
squares

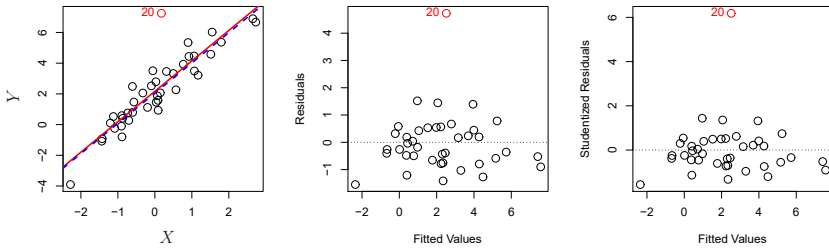


FIGURE 3.12. Left: The least squares regression line is shown in red, and the regression line after removing the outlier is shown in blue. Center: The residual plot clearly identifies the outlier. Right: The outlier has a studentized residual of 6; typically we expect values between -3 and 3 .

HÌNH 3.12. Trái: Least squares regression line màu đỏ; đường sau khi bỏ outlier màu xanh. Giữa: Residual plot nhận diện rõ outlier. Phải: Outlier có studentized residual bằng 6; thông thường ta kỳ vọng giá trị nằm giữa -3 và 3 .

lệ nghịch variance—trong ví dụ này $w_i = n_i$. Phần lớn software linear regression đều hỗ trợ observation weight.

4. Outlier

Outlier là một điểm có y_i cách xa giá trị mô hình dự đoán. Outlier có thể xuất hiện vì nhiều nguyên nhân, chẳng hạn ghi sai observation khi thu thập dữ liệu.

Điểm đỏ (observation 20) ở panel trái Hình 3.12 là một outlier điển hình. Đường đỏ liền là least squares fit; đường xanh đứt là fit sau khi bỏ outlier. Việc bỏ điểm này gần như không đổi slope và chỉ giảm intercept rất ít. Một outlier không có predictor value bất thường thường ít ảnh hưởng least squares fit. Nhưng nó vẫn có thể gây vấn đề khác: RSE bằng 1,09 khi giữ outlier và chỉ 0,77 khi bỏ nó. Vì RSE được dùng để tính mọi confidence interval và p -value, một điểm duy nhất làm tăng RSE mạnh có thể thay đổi cách diễn giải fit. Tương tự, outlier làm R^2 giảm từ 0,892 xuống 0,805.

Residual plot giúp nhận diện outlier, nhưng khó quyết định residual phải lớn đến đâu mới được coi là outlier. Thay vì residual thô, ta có thể vẽ *studentized residual*, tính bằng cách chia mỗi residual e_i cho estimated standard error của nó. Observation có trị tuyệt đối studentized residual lớn hơn 3 là outlier khả dĩ. Trong panel phải Hình 3.12, studentized residual của outlier vượt 6, còn mọi observation khác nằm giữa -2 và 2 .

Nếu tin rằng outlier xuất phát từ lỗi thu thập hay ghi dữ liệu, một giải pháp là bỏ observation. Tuy nhiên phải thận trọng: outlier có thể phản ánh khiếm khuyết của mô hình, chẳng hạn thiếu một predictor.

5. High-Leverage Point

Outlier là observation có response y_i bất thường khi đã biết predictor x_i . Ngược lại, observation có *high leverage* mang giá trị x_i bất thường. Observation 41 trong panel trái Hình 3.13 có predictor value lớn so với

Outlier

studentized
residual

high leverage

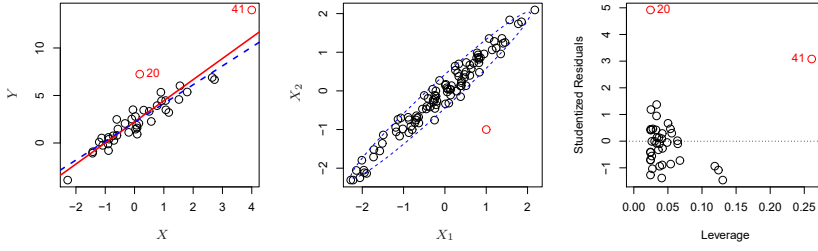


FIGURE 3.13. Left: Observation 41 is a high leverage point, while 20 is not. The red line is the fit to all the data, and the blue line is the fit with observation 41 removed. Center: The red observation is not unusual in terms of its X_1 value or its X_2 value, but still falls outside the bulk of the data, and hence has high leverage. Right: Observation 41 has a high leverage and a high residual.

HÌNH 3.13. Trái: Observation 41 là high-leverage point, còn observation 20 thì không. Đường đỏ fit toàn bộ dữ liệu; đường xanh fit sau khi bỏ observation 41. Giữa: Observation đỏ không bất thường nếu xét riêng X_1 hay X_2 , nhưng nằm ngoài phần lớn dữ liệu nên có high leverage. Phải: Observation 41 vừa có leverage cao vừa có residual cao.

các observation khác. Dữ liệu giống Hình 3.12, chỉ thêm một high-leverage observation. Đường đỏ liền là fit cho toàn bộ dữ liệu, đường xanh đứt là fit khi bỏ observation 41.

So sánh panel trái của Hình 3.12 và Hình 3.13, ta thấy bỏ high-leverage observation ảnh hưởng đến least squares line nhiều hơn hẳn bỏ outlier. High-leverage observation thường ảnh hưởng đáng kể đến estimated regression line. Nếu chỉ vài observation chi phối mạnh đường fit, mọi vấn đề ở các điểm đó có thể làm toàn bộ fit mất hiệu lực; vì vậy cần nhận diện chúng.

Trong simple linear regression, high-leverage observation khá dễ thấy vì predictor value nằm ngoài range bình thường. Trong multiple regression, một observation có thể nằm trong range của từng predictor riêng lẻ nhưng vẫn bất thường khi xét toàn bộ predictor. Panel giữa Hình 3.13 minh họa dataset có X_1, X_2 : phần lớn observation nằm trong ellipse xanh đứt, còn điểm đỏ ở ngoài dù cả X_1 lẫn X_2 của nó đều không bất thường khi xét riêng. Với hơn hai predictor, vấn đề còn rõ hơn vì không có cách đơn giản để vẽ đồng thời mọi dimension.

Để định lượng leverage của một observation, ta tính *leverage statistic*. Giá trị lớn cho biết high leverage. Với simple linear regression,

leverage
statistic

$$h_i = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{\sum_{i'=1}^n (x_{i'} - \bar{x})^2}. \quad (3.37)$$

Rõ ràng h_i tăng theo khoảng cách giữa x_i và $\bar{x}$. Công thức có thể mở rộng cho multiple predictor. Leverage statistic luôn nằm giữa $1/n$ và 1, còn leverage trung bình của mọi observation luôn là $(p+1)/n$. Nếu một observation có leverage vượt xa $(p+1)/n$, ta có thể nghi ngờ đó là high-leverage point.

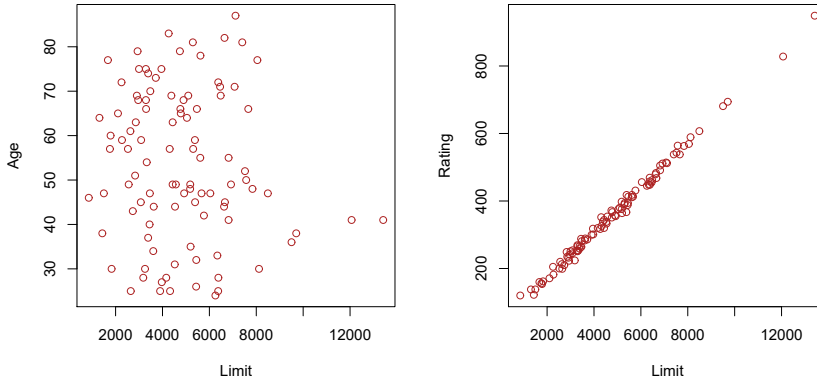


FIGURE 3.14. Scatterplots of the observations from the **Credit** data set. Left: A plot of **age** versus **limit**. These two variables are not collinear. Right: A plot of **rating** versus **limit**. There is high collinearity.

HÌNH 3.14. Scatterplot của observation trong dataset **Credit**. Trái: **age** theo **limit**; hai biến không collinear. Phải: **rating** theo **limit**; collinearity rất cao.

Panel phải Hình 3.13 vẽ studentized residual theo h_i . Observation 41 nổi bật vì vừa có leverage statistic rất cao vừa có studentized residual lớn—nó đồng thời là outlier và high-leverage observation, một tổ hợp đặc biệt nguy hiểm. Đồ thị cũng giải thích vì sao observation 20 ít ảnh hưởng fit trong Hình 3.12: leverage của nó thấp.

6. Collinearity

Collinearity là tình huống hai hay nhiều predictor variable liên hệ chặt với nhau. Hình 3.14 minh họa bằng dataset **Credit**. Ở panel trái, **limit** và **age** không có quan hệ rõ. Ở panel phải, **limit** và **rating** correlated rất mạnh nên được gọi là collinear. Collinearity gây khó khăn vì khó tách ảnh hưởng riêng của từng biến lên response. Do **limit** và **rating** có xu hướng cùng tăng hoặc cùng giảm, ta khó xác định riêng từng biến associated với **balance** như thế nào.

Collinearity

Hình 3.15 cho thấy khó khăn do collinearity. Panel trái là contour plot của RSS (3.22) theo các coefficient estimate khả dĩ của regression **balance** trên **limit** và **age**. Mỗi ellipse là tập coefficient có cùng RSS; ellipse càng gần tâm thì RSS càng thấp. Chấm đen và đường đứt tương ứng với coefficient estimate cho RSS nhỏ nhất, tức least squares estimate. Trục **limit** và **age** được scale để chứa giá trị đến bốn standard error ở hai phía của estimate, nên gồm mọi giá trị coefficient hợp lý. Chẳng hạn, coefficient thật của **limit** gần như chắc chắn nằm giữa 0,15 và 0,20.

Panel phải Hình 3.15 là contour plot RSS của regression **balance** trên hai biến highly collinear là **limit** và **rating**. Contour chạy dọc một thung lũng hẹp; một range rộng của coefficient estimate cho cùng RSS. Một thay đổi nhỏ trong dữ liệu có thể làm cặp least squares estimate di chuyển tới

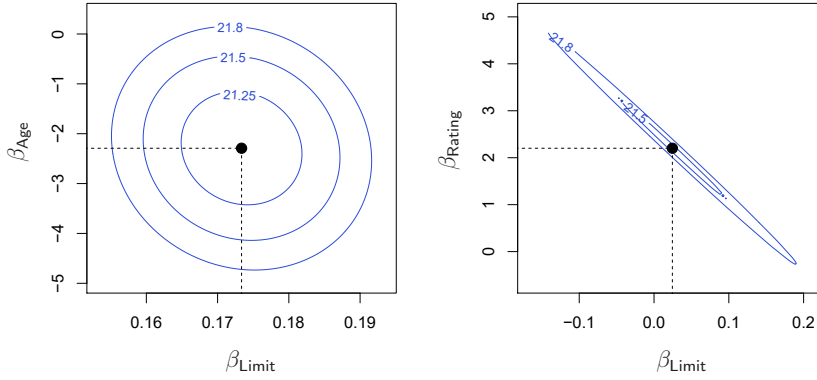


FIGURE 3.15. Contour plots for the RSS values as a function of the parameters β for various regressions involving the **Credit** data set. In each plot, the black dots represent the coefficient values corresponding to the minimum RSS. Left: A contour plot of RSS for the regression of **balance** onto **age** and **limit**. The

HÌNH 3.15. Contour plot của RSS theo parameter β cho các regression dùng dataset **Credit**. Chấm đen là coefficient value cho RSS nhỏ nhất. Trái: Regression **balance** trên **age** và **limit**; minimum được xác định rõ. Phải: Regression **balance** trên **rating** và **limit**; do collinearity, nhiều cặp $(\beta_{\text{Limit}}, \beta_{\text{Rating}})$ có RSS tương tự nhau.

BẢNG 3.11. Kết quả của hai multiple regression model dùng dataset **Credit**. Model 1 regress **balance** trên **age** và **limit**; Model 2 regress **balance** trên **rating** và **limit**. Standard error của $\hat{\beta}_{\text{limit}}$ tăng 12 lần trong regression thứ hai do collinearity.

		Coefficient	Std. error	t-statistic	p-value
Model 1	Intercept	-173.411	43.828	-3.957	< 0.0001
	age	-2.292	0.672	-3.407	0.0007
	limit	0.173	0.005	34.496	< 0.0001
Model 2	Intercept	-377.537	45.254	-8.343	< 0.0001
	rating	2.202	0.952	2.312	0.0213
	limit	0.025	0.064	0.384	0.7012

bất kỳ đầu dọc thung lũng, tạo uncertainty lớn. Scale của coefficient limit nay từ khoảng -0.2 đến 0.2 , rộng gấp tám lần range hợp lý khi regression cùng age. Dù từng coefficient có uncertainty lớn hơn nhiều, cặp coefficient gần như chắc chắn vẫn nằm đâu đó trong thung lũng contour này. Ví dụ, ta không kỳ vọng coefficient limit và rating thực lẫn lượt là -0.1 và 1 , dù từng giá trị riêng lẻ đều có vẻ hợp lý.

Vì collinearity làm coefficient estimate kém chính xác, standard error của $\hat{\beta}_j$ tăng. t -statistic được tính bằng $\hat{\beta}_j$ chia standard error, nên collinearity làm t -statistic giảm. Khi đó ta có thể không bác bỏ được $H_0 : \beta_j = 0$. Nói cách khác, collinearity làm giảm *power* của hypothesis test—xác suất phát hiện đúng một coefficient khác 0.

Bảng 3.11 so sánh hai multiple regression model. Trong mô hình thứ nhất, age và limit đều highly significant với p -value rất nhỏ. Trong mô hình thứ

hai, collinearity giữa limit và rating làm standard error của coefficient limit tăng 12 lần và p -value tăng lên 0,701. Tầm quan trọng của limit đã bị che khuất bởi collinearity. Vì vậy, cần nhận diện và xử lý vấn đề collinearity khi fit mô hình.

Cách đơn giản để phát hiện collinearity là xem correlation matrix của predictor. Phần tử có trị tuyệt đối lớn cho biết một cặp biến highly correlated. Nhưng correlation matrix không phát hiện được mọi trường hợp: ba hay nhiều biến có thể collinear dù không cặp nào có correlation đặc biệt cao. Tình huống này gọi là *multicollinearity*.

Cách tốt hơn để đánh giá multicollinearity là tính *variance inflation factor* (VIF): tỷ số giữa variance của $\hat{\beta}_j$ khi fit full model và variance của $\hat{\beta}_j$ khi fit riêng. VIF nhỏ nhất bằng 1, tương ứng hoàn toàn không có collinearity. Trong thực tế thường có một lượng nhỏ collinearity; theo rule of thumb, VIF vượt 5 hoặc 10 cho thấy mức collinearity có vấn đề. Với mỗi biến,

variance
inflation
factor

$$\text{VIF}(\hat{\beta}_j) = \frac{1}{1 - R_{X_j|X_{-j}}^2},$$

trong đó $R_{X_j|X_{-j}}^2$ là R^2 của regression X_j trên mọi predictor khác. Nếu giá trị này gần 1 thì có collinearity và VIF lớn.

Trong dữ liệu **Credit**, regression **balance** trên **age**, **rating** và **limit** cho VIF lần lượt là 1,01; 160,67; và 160,59. Đúng như nghi ngờ, dữ liệu có collinearity đáng kể.

Khi gặp collinearity, có hai giải pháp đơn giản. Thứ nhất, bỏ một biến có vấn đề khỏi regression. Thường việc này không làm model fit giảm nhiều vì collinearity ngụ ý thông tin biến đó cung cấp về response là redundant khi đã có các biến còn lại. Ví dụ, nếu regress balance trên age và limit mà bỏ rating, VIF giảm gần mức tối thiểu 1, còn R^2 chỉ giảm từ 0,754 xuống 0,75. Như vậy, bỏ rating giải quyết collinearity mà hầu như không ảnh hưởng fit. Giải pháp thứ hai là gộp các biến collinear thành một predictor duy nhất. Chẳng hạn, ta có thể lấy trung bình các phiên bản standardized của limit và rating để tạo biến mới đo credit worthiness.

3.4 Kế hoạch Marketing



Giờ ta quay lại ngắn gọn với bảy câu hỏi về dữ liệu **Advertising** đã đặt ra ở đầu chương.

1. *Sales và advertising budget có quan hệ không?*

Ta trả lời bằng cách fit multiple regression model của **sales** trên **TV**, **radio**, **newspaper**, như trong (3.20), rồi kiểm định $H_0 : \beta_{TV} = \beta_{radio} = \beta_{newspaper} = 0$. Trong Mục 3.2.2, F -statistic được dùng để quyết định có bác bỏ null hypothesis hay không. Ở đây, p -value của F -statistic trong Bảng 3.6 rất thấp, cho bằng chứng rõ ràng về quan hệ giữa advertising và sales.

2. *Mối quan hệ mạnh đến đâu?*

Ta đã bàn hai thước đo model accuracy ở Mục 3.1.3. RSE ước lượng standard deviation của response quanh population regression line. Với dữ liệu **Advertising**, RSE là 1,69 đơn vị, còn response trung bình là 14,022, tương ứng percentage error khoảng 12%. Thứ hai, R^2 ghi nhận tỷ lệ variability của response được predictor giải thích. Các predictor giải thích gần 90% variance của sales. RSE và R^2 được trình bày trong Bảng 3.6.

3. *Phương tiện nào associated với sales?*

Ta xem p -value gắn với t -statistic của từng predictor (Mục 3.1.2). Trong multiple linear regression ở Bảng 3.4, p -value của TV và radio thấp, còn newspaper thì không. Điều này gợi ý chỉ TV và radio liên hệ với sales. Chương 6 sẽ khảo sát sâu hơn.

4. *Association giữa từng phương tiện và sales lớn đến đâu?*

Trong Mục 3.1.2, standard error của $\hat{\beta}_j$ được dùng để dựng confidence interval cho β_j . Từ Bảng 3.4, confidence interval 95% của coefficient trong multiple regression model dùng cả ba ngân sách là (0.043, 0.049) cho TV, (0.172, 0.206) cho radio và (-0.013, 0.011) cho newspaper. Khoảng của TV và radio hẹp, cách xa 0, cho bằng chứng rằng hai phương tiện liên hệ với sales. Khoảng của newspaper chứa 0, nên biến này không statistically significant khi đã biết TV và radio.

Collinearity có thể làm standard error rất rộng. Liệu đó có phải lý do confidence interval của newspaper rộng? VIF của TV, radio và newspaper lần lượt là 1,005; 1,145; và 1,145, nên không có bằng chứng collinearity.

Để đánh giá riêng association của từng phương tiện với sales, ta có thể thực hiện ba simple linear regression. Kết quả trong Bảng 3.1 và Bảng 3.3 cho bằng chứng về association cực mạnh giữa TV với sales và giữa radio với sales. Nếu bỏ qua giá trị TV và radio, newspaper có association nhẹ với sales.

5. *Có thể dự đoán future sales chính xác đến đâu?*

Response được dự đoán bằng (3.21). Accuracy phụ thuộc vào việc ta muốn dự đoán individual response $Y = f(X) + \epsilon$ hay average response $f(X)$ (Mục 3.2.2). Trường hợp đầu dùng prediction interval; trường

hợp sau dùng confidence interval. Prediction interval luôn rộng hơn vì tính cả uncertainty gắn với ϵ , tức irreducible error.

6. *Quan hệ có tuyến tính không?*

Residual plot có thể nhận diện tính phi tuyến. Nếu quan hệ tuyến tính, residual plot không nên có pattern. Với dữ liệu **Advertising**, ta thấy nonlinear effect trong Hình 3.5; hiệu ứng này cũng có thể quan sát trên residual plot. Trong Mục 3.3.2, ta đã đưa transformation của predictor vào linear regression model để xử lý quan hệ phi tuyến.

7. *Có synergy giữa các advertising media không?*

Linear regression model chuẩn giả định quan hệ additive giữa predictor và response. Additive model dễ diễn giải vì association giữa từng predictor với response không phụ thuộc giá trị predictor khác. Tuy nhiên, giả định này có thể không thực tế. Trong Mục 3.3.2, interaction term được thêm vào regression model để biểu diễn quan hệ non-additive; p -value nhỏ của interaction term chỉ ra sự hiện diện của quan hệ đó. Hình 3.5 gợi ý dữ liệu **Advertising** có thể không additive. Thêm interaction term làm R^2 tăng đáng kể, từ khoảng 90% lên gần 97%.

3.5 So sánh Linear Regression với K-Nearest Neighbors

Như đã thảo luận trong Chương 2, linear regression là một parametric approach vì nó giả định functional form tuyến tính cho $f(X)$. Parametric method có một số ưu điểm: thường dễ fit vì chỉ cần ước lượng ít coefficient; trong linear regression, coefficient có cách diễn giải đơn giản và dễ thực hiện statistical significance test. Nhưng chúng cũng có nhược điểm: ngay từ cách xây dựng, chúng đặt giả định mạnh lên dạng của $f(X)$. Nếu functional form được chỉ định khác xa sự thật và mục tiêu là prediction accuracy, parametric method sẽ hoạt động kém. Chẳng hạn, nếu giả định quan hệ tuyến tính giữa X và Y trong khi quan hệ thực khác xa tuyến tính, model fit sẽ kém và mọi kết luận đều đáng ngờ.

Trái lại, non-parametric method không giả định tường minh một parametric form cho $f(X)$, do đó cung cấp cách tiếp cận regression linh hoạt hơn. Cuốn sách sẽ bàn nhiều phương pháp non-parametric. Ở đây, ta xét một phương pháp đơn giản và nổi tiếng: *K-nearest neighbors regression* (KNN regression), họ hàng gần của KNN classifier trong Chương 2. Với K và prediction point x_0 , KNN regression trước hết xác định K training observation gần x_0 nhất, ký hiệu $\mathcal{N}_0$, rồi ước lượng $f(x_0)$ bằng trung bình mọi training response trong $\mathcal{N}_0$:

$$\hat{f}(x_0) = \frac{1}{K} \sum_{x_i \in \mathcal{N}_0} y_i.$$

Hình 3.16 minh họa hai KNN fit trên dataset có $p = 2$ predictor. Panel trái dùng $K = 1$, panel phải dùng $K = 9$. Khi $K = 1$, KNN fit nội suy hoàn hảo training observation và có dạng step function gồ gề. Với $K = 9$,

*K-nearest
neighbors
regression*

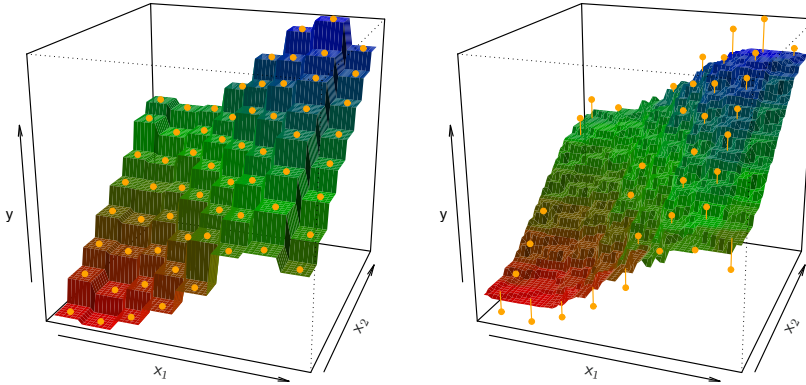


FIGURE 3.16. Plots of $\hat{f}(X)$ using KNN regression on a two-dimensional data set with 64 observations (orange dots). Left: $K = 1$ results in a rough step function fit. Right: $K = 9$ produces a much smoother fit.

HÌNH 3.16. Đồ thị $\hat{f}(X)$ bằng KNN regression trên dataset hai chiều có 64 observation (chấm cam). Trái: $K = 1$ tạo một step-function fit gồ ghề. Phải: $K = 9$ tạo fit mượt hơn nhiều.

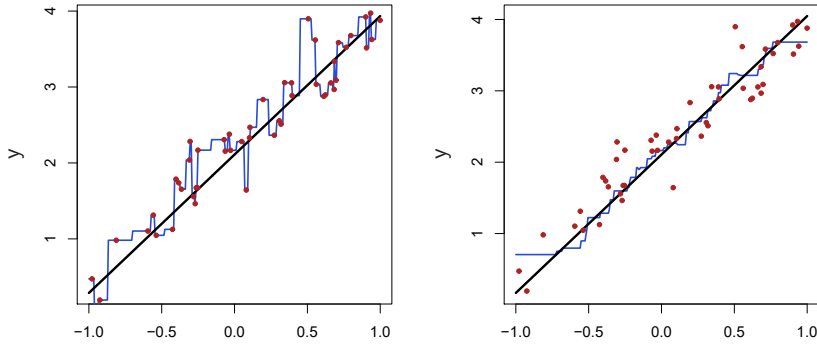
nó vẫn là step function, nhưng việc lấy trung bình chín observation tạo các vùng dự đoán hằng nhỏ hơn và fit mượt hơn.

Nói chung, K tối ưu phụ thuộc bias–variance tradeoff đã giới thiệu ở Chương 2. K nhỏ cho fit linh hoạt nhất, bias thấp nhưng variance cao, vì prediction trong một vùng hoàn toàn phụ thuộc chỉ một observation. K lớn cho fit mượt và ít biến thiên hơn; prediction là trung bình của nhiều điểm nên thay đổi một observation ít ảnh hưởng hơn. Tuy nhiên, smoothing có thể tạo bias do che khuất cấu trúc trong $f(X)$. Chương 5 giới thiệu một số cách ước lượng test error rate, có thể dùng để tìm K tối ưu.

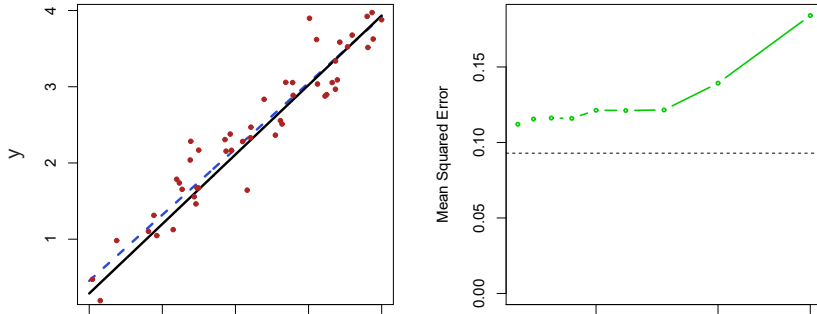
Khi nào parametric approach như least squares linear regression vượt non-parametric approach như KNN? Câu trả lời đơn giản: khi parametric form được chọn gần dạng thực của f . Hình 3.17 minh họa dữ liệu sinh từ one-dimensional linear regression model. Đường đen liền là $f(X)$, đường xanh là KNN fit với $K = 1$ và $K = 9$. Prediction khi $K = 1$ biến thiên quá mạnh, còn fit $K = 9$ mượt hơn và gần $f(X)$ hơn. Nhưng vì quan hệ thực tuyến tính, non-parametric approach khó cạnh tranh với linear regression: nó phải trả giá bằng variance mà không được bù lại bởi bias thấp hơn.

Đường xanh đứt ở panel trái Hình 3.18 là linear regression fit trên cùng dữ liệu và gần như hoàn hảo. Panel phải cho thấy linear regression vượt KNN: đường xanh lục biểu diễn test-set mean squared error (MSE) của KNN theo $1/K$, còn đường đen đứt là test MSE của linear regression. KNN error cao hơn; với K lớn nó chỉ kém least squares một chút, nhưng kém xa khi K nhỏ.

Trong thực tế, quan hệ giữa X và Y hiếm khi tuyến tính hoàn toàn. Hình 3.19 so sánh least squares regression với KNN khi mức phi tuyến



HÌNH 3.17. Đồ thị $\hat{f}(X)$ bằng KNN regression trên dataset một chiều có 50 observation. Quan hệ thực là đường đen liền. Trái: Đường xanh với $K = 1$ nội suy training data. Phải: Đường xanh với $K = 9$ cho fit mượt hơn.



HÌNH 3.18. Khảo sát tiếp dataset trong Hình 3.17. Trái: Đường xanh đứt là least squares fit; vì $f(X)$ thực sự tuyến tính (đường đen), regression line ước lượng $f(X)$ rất tốt. Phải: Đường ngang đứt là least-squares test MSE; đường xanh lục là KNN MSE theo $1/K$ trên log scale. Linear regression có test MSE thấp hơn. KNN tốt nhất khi K rất lớn, tương ứng $1/K$ nhỏ.

tăng. Hàng trên có quan hệ gần tuyến tính: test MSE của linear regression tốt hơn KNN khi K nhỏ, nhưng KNN vượt linear regression khi $K \geq 4$. Hàng dưới lệch tuyến tính mạnh hơn; KNN vượt đáng kể linear regression với mọi K . Khi mức phi tuyến tăng, test MSE của KNN gần như không đổi, còn test MSE của linear regression tăng mạnh.

Hình 3.18 và Hình 3.19 cho thấy KNN kém linear regression một chút khi quan hệ tuyến tính nhưng tốt hơn nhiều khi phi tuyến. Khi không biết quan hệ thực, có vẻ nên ưu tiên KNN: tệ nhất chỉ kém nhẹ nếu quan hệ tuyến tính, nhưng có thể tốt hơn đáng kể nếu phi tuyến. Tuy nhiên, ngay cả

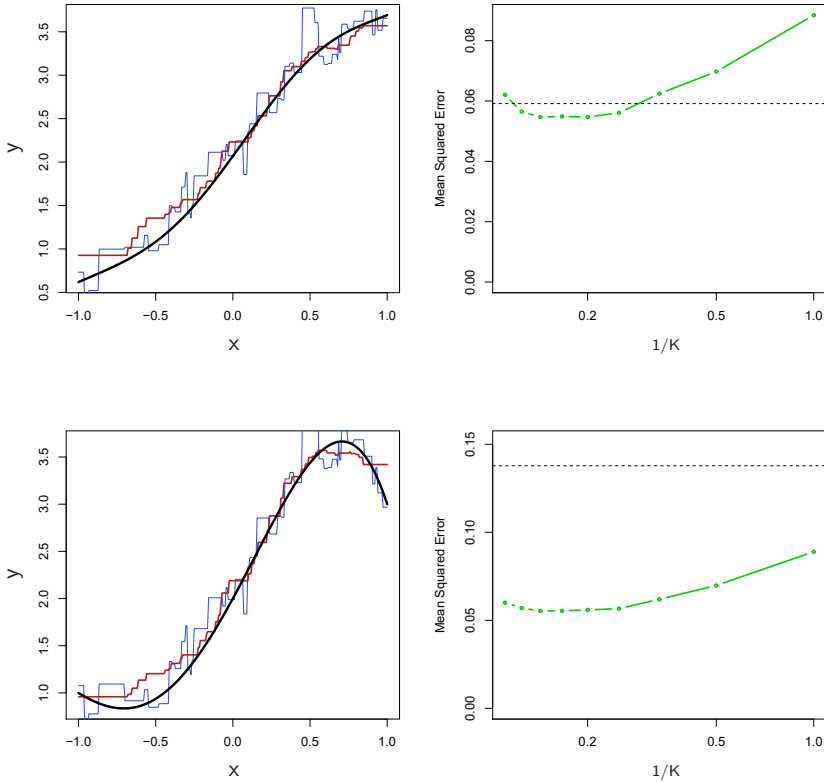


FIGURE 3.19. Top Left: In a setting with a slightly non-linear relationship between X and Y (solid black line), the KNN fits with $K = 1$ (blue) and $K = 9$ (red) are displayed. Top Right: For the slightly non-linear data, the test set MSE for least squares regression (horizontal black) and KNN with various values of $1/K$ (green) are displayed. Bottom Left and Bottom Right: As in the top panel, but with a strongly non-linear relationship between X and Y .

HÌNH 3.19. Trên trái: Với quan hệ hơi phi tuyến giữa X, Y (đường đen liền), biểu diễn KNN fit $K = 1$ (xanh) và $K = 9$ (đỏ). Trên phải: Test MSE của least squares regression (ngang đen) và KNN theo các giá trị $1/K$ (xanh lục). Hai panel dưới: Tương tự hàng trên nhưng quan hệ giữa X, Y phi tuyến mạnh.

khi quan hệ thực highly non-linear, KNN vẫn có thể kém linear regression. Các hình trên đều chỉ có $p = 1$ predictor; trong higher dimension, KNN thường hoạt động kém hơn.

Hình 3.20 xét cùng tình huống phi tuyến mạnh như hàng dưới Hình 3.19, nhưng thêm các noise predictor không associated với response. Khi $p = 1$ hoặc $p = 2$, KNN vượt linear regression. Với $p = 3$, kết quả lẫn lộn; khi $p \geq 4$, linear regression tốt hơn. Tăng dimension chỉ làm linear-regression test MSE giảm nhẹ, nhưng khiến KNN MSE tăng hơn mười lần.

Performance của KNN giảm khi dimension tăng vì sample size hữu hiệu bị thu nhỏ. Dataset có 50 training observation; với $p = 1$, lượng thông

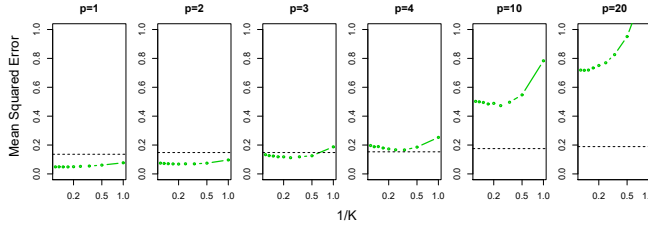


FIGURE 3.20. Test MSE for linear regression (black dashed lines) and KNN (green curves) as the number of variables p increases. The true function is non-linear in the first variable, as in the lower panel in Figure 3.19, and does not depend on the additional variables. The performance of linear regression deteriorates slowly in the presence of these additional noise variables, whereas KNN's performance degrades much more quickly as p increases.

and 3.19 illustrate settings with $p = 1$ predictor. But in higher dimensions,

HÌNH 3.20. Test MSE của linear regression (đường đen đứt) và KNN (đường xanh lục) khi số biến p tăng. Hàm thực phi tuyến theo biến thứ nhất, như panel dưới Hình 3.19, và không phụ thuộc các biến bổ sung. Noise variable làm linear-regression performance giảm chậm, còn KNN giảm nhanh hơn nhiều khi p tăng.

tin này đủ để ước lượng chính xác $f(X)$. Nhưng trải 50 observation trên $p = 20$ dimension dẫn đến hiện tượng một observation không có neighbor nào thực sự gần—gọi là *curse of dimensionality*. Khi p lớn, K observation gần một test observation x_0 nhất vẫn có thể ở rất xa x_0 trong không gian p chiều, dẫn đến prediction $f(x_0)$ rất kém và KNN fit kém. Theo nguyên tắc chung, parametric method có xu hướng vượt non-parametric approach khi số observation trên mỗi predictor nhỏ.

curse of dimensionality

Ngay cả khi dimension nhỏ, ta vẫn có thể chọn linear regression thay vì KNN vì interpretability. Nếu test MSE của KNN chỉ thấp hơn một chút, ta có thể chấp nhận mất ít prediction accuracy để đổi lấy mô hình đơn giản, mô tả bằng vài coefficient và có sẵn p -value.

3.6 Lab: Linear Regression



3.6.1 Importing Packages

Ở cấp đầu của notebook, ta import các library chuẩn thường dùng.

```
In [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
```

Các import mới

Trong lab này, ta sẽ giới thiệu các function và library mới. Chúng được import ở đây để nhấn mạnh những code object mới của lab. Đặt import gần đầu notebook giúp code dễ đọc: chỉ cần nhìn vài dòng đầu là biết các library được dùng.

```
In [2]: import statsmodels.api as sm
```

Chi tiết liên quan của các function sẽ được trình bày khi cần.

Ngoài import toàn bộ module, ta có thể chỉ import một vài item để giữ *namespace* gọn gàng. Ta sẽ dùng một số object cụ thể từ package *statsmodels*.

namespace
statsmodels

```
In [3]: from statsmodels.stats.outliers_influence \
import variance_inflation_factor as VIF
from statsmodels.stats.anova import anova_lm
```

Vì một import statement khá dài, ta chèn line break `\` để dễ đọc.

Ta cũng dùng một số function được viết riêng cho lab của sách trong package *ISLP*.

```
In [4]: from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                        summarize,
                        poly)
```

Kiểm tra Object và Namespace

Function *dir()* cung cấp danh sách object trong một namespace.

dir()

```
In [5]: dir()
```

```
Out[5]: ['In',
         'MS',
         '_',
         '___',
         '___',
         '__builtin__',
         '__builtins__',
         ...,
         'poly',
         'quit',
         'sm',
         'summarize']
```

Kết quả cho thấy mọi thứ Python tìm thấy ở top level. Một số object như `__builtins__` chứa reference đến built-in function như `print()`.

Mỗi Python object có namespace riêng, cũng truy cập bằng `dir()`; nó gồm attribute và method của object. Chẳng hạn, danh sách của một array có `'sum'`.

```
In [6]: A = np.array([3,5,11])
        dir(A)
```

```
Out[6]: ...
        'strides',
        'sum',
        'swapaxes',
        ...
```

Điều này chỉ ra object `A.sum` tồn tại; đây là method tính tổng array, có thể kiểm tra bằng `A.sum?`.

```
In [7]: A.sum()
```

```
Out[7]: 19
```

3.6.2 Simple Linear Regression

Trong mục này, ta dựng model matrix (còn gọi là design matrix) bằng transform `ModelSpec()` từ `ISLP.models`.

Ta dùng Boston housing dataset trong package `ISLP`. Dataset ghi `medv` (median house value) cho 506 khu dân cư quanh Boston. Ta xây regression model dự đoán `medv` bằng 13 predictor như `rmvar` (số phòng trung bình mỗi nhà), `age` (tỷ lệ đơn vị do chủ sở hữu cư trú được xây trước năm 1940) và `lstat` (phần trăm hộ gia đình có socioeconomic status thấp). Công việc dùng `statsmodels`, một Python package triển khai nhiều regression method thông dụng. Package `ISLP` cung cấp function nạp dữ liệu đơn giản `load_data()`:

```
load_data()
```

```
In [8]: Boston = load_data("Boston")
Boston.columns
```

```
Out[8]: Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
              'rad', 'tax', 'ptratio', 'black', 'lstat', 'medv'],
              dtype='object')
```

Gõ `Boston`? để xem thêm thông tin về dữ liệu.

Đầu tiên, ta dùng `sm.OLS()` để chỉ định simple linear regression model với response `medv` và predictor duy nhất `lstat`. Model matrix có thể dựng thủ công.

```
In [9]: X = pd.DataFrame({'intercept': np.ones(Boston.shape[0]),
                          'lstat': Boston['lstat']})
X[:4]
```

```
Out[9]:   intercept  lstat
0         1.0    4.98
1         1.0    9.14
2         1.0    4.03
3         1.0    2.94
```

Sau đó, ta lấy response và fit mô hình.

```
In [10]: y = Boston['medv']
model = sm.OLS(y, X)
results = model.fit()
```

Lưu ý `sm.OLS()` không fit mô hình mà chỉ đặc tả nó; `model.fit()` mới thực hiện fitting.

Function `summarize()` của `ISLP` tạo bảng gọn gồm parameter estimate, standard error, *t*-statistic và *p*-value. Nó nhận một argument, chẳng hạn object `results` do method `fit()` trả về.

```
In [11]: summarize(results)
```

```
Out[11]:      coef  std err      t  P>|t|
intercept  34.5538    0.563  61.415    0.0
lstat      -0.9500    0.039 -24.528    0.0
```

Dùng Transformation: Fit và Transform

Mô hình trên chỉ có một predictor nên việc dựng X rất trực tiếp. Trong thực tế, ta thường fit nhiều predictor được chọn từ array hoặc data frame, có thể biến đổi biến trước khi fit, chỉ định interaction, hoặc mở rộng một biến thành một tập biến như polynomial. Package *sklearn* gọi loại tác vụ này là một *transform*. Transform object được tạo với một số parameter, rồi dùng hai method chính: `fit()` và `transform()`.

Library *ISLP* cung cấp transform `ModelSpec()` (đã đổi tên thành `MS()` ở phần đầu) để đặc tả mô hình và dựng model matrix. `MS()` tạo transform object; cập method `fit()` và `transform()` dựng model matrix tương ứng.

Với simple regression dùng `lstat`, ta tạo transform bằng `design = MS(['lstat'])`. Method `fit()` nhận array gốc và có thể thực hiện các phép tính ban đầu do transform quy định, chẳng hạn mean và standard deviation để centering, scaling. Method `transform()` áp dụng transformation đã fit và tạo model matrix.

```
In [12]: design = MS(['lstat'])
design = design.fit(Boston)
X = design.transform(Boston)
X[:4]
```

```
Out[12]:  intercept  lstat
0         1.0    4.98
1         1.0    9.14
2         1.0    4.03
3         1.0    2.94
```

Ở trường hợp đơn giản này, `fit()` chỉ kiểm tra biến `'lstat'` có trong `Boston`; `transform()` dựng matrix gồm intercept và `lstat`.

Hai thao tác có thể gộp bằng method `fit_transform()`.

```
In [13]: design = MS(['lstat'])
X = design.fit_transform(Boston)
X[:4]
```

```
Out[13]:  intercept  lstat
0         1.0    4.98
1         1.0    9.14
2         1.0    4.03
3         1.0    2.94
```

Giống khi thực hiện riêng hai bước, object `design` bị thay đổi bởi thao tác `fit()`. Sức mạnh của pipeline này sẽ rõ hơn với mô hình phức tạp có interaction và transformation.

Object `results` có nhiều method cho inference. `summarize()` trình bày phần cốt lõi; method `summary()` cho bản tổng hợp đầy đủ hơn (output không hiển thị).

```
In [14]: results.summary()
```

Fitted coefficient cũng có thể lấy từ attribute `params`.

```
In [15]: results.params
```

```
Out[15]: intercept    34.553841
lstat              -0.950049
dtype: float64
```

Method `get_prediction()` tạo prediction, confidence interval và prediction interval của `medv` tại các giá trị `lstat`. Trước tiên, ta tạo data frame mới chứa những giá trị cần dự đoán, rồi dùng `design.transform()` dựng model matrix tương ứng.


```
In [16]: new_df = pd.DataFrame({'lstat': [5, 10, 15]})
newX = design.transform(new_df)
newX
```

```
Out[16]:   intercept   lstat
0         1.0         5
1         1.0        10
2         1.0        15
```

Tiếp theo, ta tính prediction tại `newX` và lấy attribute `predicted_mean`.

```
In [17]: new_predictions = results.get_prediction(newX);
new_predictions.predicted_mean
```

```
Out[17]: array([29.80359411, 25.05334734, 20.30310057])
```

Confidence interval của predicted value được tạo như sau.

```
In [18]: new_predictions.conf_int(alpha=0.05)
```

```
Out[18]: array([[29.00741194, 30.59977628],
               [24.47413202, 25.63256267],
               [19.73158815, 20.87461299]])
```

Để tính prediction interval, đặt `obs=True`:

```
In [19]: new_predictions.conf_int(obs=True, alpha=0.05)
```

```
Out[19]: array([[17.56567478, 42.04151344],
               [12.82762635, 37.27906833],
               [ 8.0777421 , 32.52845905]])
```

Với `lstat=10`, confidence interval 95% là (24.47, 25.63) và prediction interval 95% là (12.82, 37.28). Cả hai cùng có tâm tại predicted value 25,05 của `medv`, nhưng prediction interval rộng hơn đáng kể như kỳ vọng.

Tiếp theo, ta vẽ `medv` và `lstat` bằng `DataFrame.plot.scatter()`, rồi thêm regression line.

Định nghĩa Function

Package `ISLP` có sẵn function thêm đường vào plot, nhưng đây là dịp để ta tự định nghĩa function đầu tiên bằng `def`.

def

```
In [20]: def abline(ax, b, m):
          "Add a line with slope m and intercept b to ax"
          xlim = ax.get_xlim()
          ylim = [m * xlim[0] + b, m * xlim[1] + b]
          ax.plot(xlim, ylim)
```

Syntax là `def funcname(...)`. Function nhận `ax`, `b`, `m`, trong đó `ax` là axis object của plot có sẵn, `b` là intercept và `m` là slope. Ta có thể truyền thêm plotting option vào `ax.plot` bằng optional argument:

```
In [21]: def abline(ax, b, m, *args, **kwargs):
          "Add a line with slope m and intercept b to ax"
          xlim = ax.get_xlim()
          ylim = [m * xlim[0] + b, m * xlim[1] + b]
          ax.plot(xlim, ylim, *args, **kwargs)
```

`*args` cho phép số lượng tùy ý non-named argument; `**kwargs` cho phép số lượng tùy ý named argument như `linewidth=3`. Function chuyển nguyên các argument đó tới `ax.plot`. Bạn đọc muốn tìm hiểu thêm có thể xem mục defining functions tại docs.python.org/tutorial.

Ta dùng function mới để thêm regression line vào plot `medv` theo `lstat`.

```
In [22]: ax = Boston.plot.scatter('lstat', 'medv')
          abline(ax,
                  results.params[0],
                  results.params[1],
                  'r--',
                  linewidth=3)
```

Lời gọi cuối cùng tới `ax.plot()` tương đương `ax.plot(xlim, ylim, 'r--', linewidth=3)`. Argument `'r--'` tạo đường đỏ đứt, còn width bằng 3. Quan hệ giữa `lstat` và `medv` có dấu hiệu phi tuyến; ta sẽ khảo sát sau trong lab. Function có sẵn `ax.axline()` cũng thêm được đường, nhưng biết tự viết function giúp tạo display biểu cảm hơn.

Tiếp theo, ta xem diagnostic plot đã bàn trong Mục 3.3.3. Fitted value và residual là attribute của `results`; các influence measure được tính bằng method `get_influence()`. Vì không dùng component `fig` đầu tiên từ `subplots()`, ta chỉ nhận giá trị thứ hai vào `ax`.

```
In [23]: ax = subplots(figsize=(8,8))[1]

          ax.scatter(results.fittedvalues, results.resid)
          ax.set_xlabel('Fitted value')
          ax.set_ylabel('Residual')
          ax.axhline(0, c='k', ls='--');
```

Ta thêm đường ngang tại 0 bằng `ax.axhline()`, đặt màu đen `c='k'` và `ax.axhline()` linestyle đứt `ls='--'`. Residual plot (không hiển thị) cho một số bằng chứng phi tuyến. Leverage statistic với số predictor bất kỳ được lấy qua attribute `hat_matrix_diag` của kết quả `get_influence()`.

```
In [24]: infl = results.get_influence()
          ax = subplots(figsize=(8,8))[1]
          ax.scatter(np.arange(X.shape[0]), infl.hat_matrix_diag)
          ax.set_xlabel('Index')
          ax.set_ylabel('Leverage')
          np.argmax(infl.hat_matrix_diag)
```

Out [24]: 374

Function `np.argmax()` tìm index của phần tử lớn nhất trong array, tùy chọn theo một axis. Ở đây ta tối đa trên toàn array để xác định observation có leverage statistic lớn nhất.

3.6.3 Multiple Linear Regression

Để fit multiple linear regression bằng least squares, ta lại dùng transform `ModelSpec()` dựng model matrix và response. Argument của `ModelSpec()` có thể rất tổng quát; ở đây danh sách column name là đủ. Trước tiên, ta fit với `lstat` và `age`.

```
In [25]: X = MS(['lstat', 'age']).fit_transform(Boston)
model1 = sm.OLS(y, X)
results1 = model1.fit()
summarize(results1)
```

```
Out[25]:
```

	coef	std err	t	P> t
intercept	33.2228	0.731	45.458	0.000
lstat	-1.0321	0.048	-21.416	0.000
age	0.0345	0.012	2.826	0.005

Dòng đầu được nén thành biểu thức ngắn gọn mô tả việc dựng `X`.

Dataset `Boston` có 12 predictor; gỡ tất cả để regression sẽ công kênh. Ta dùng shorthand `.columns.drop()`:

```
In [26]: terms = Boston.columns.drop('medv')
terms
```

```
Out[26]: Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
               'rad', 'tax', 'ptratio', 'lstat'],
              dtype='object')
```

Giờ có thể fit mô hình với mọi biến trong `terms` bằng cùng model-matrix builder.

```
In [27]: X = MS(terms).fit_transform(Boston)
model = sm.OLS(y, X)
results = model.fit()
summarize(results)
```

```
Out[27]:
```

	coef	std err	t	P> t
intercept	41.6173	4.936	8.431	0.000
crim	-0.1214	0.033	-3.678	0.000
zn	0.0470	0.014	3.384	0.001
indus	0.0135	0.062	0.217	0.829
chas	2.8400	0.870	3.264	0.001
nox	-18.7580	3.851	-4.870	0.000
rm	3.6581	0.420	8.705	0.000
age	0.0036	0.013	0.271	0.787
dis	-1.4908	0.202	-7.394	0.000
rad	0.2894	0.067	4.325	0.000
tax	-0.0127	0.004	-3.337	0.001
ptratio	-0.9375	0.132	-7.091	0.000
lstat	-0.5520	0.051	-10.897	0.000

Nếu muốn regression với mọi biến trừ một biến, chẳng hạn bỏ `age` vì `p-value` cao, syntax sau thực hiện fit mà không có predictor đó (output không hiển thị).

```
In [28]: minus_age = Boston.columns.drop(['medv', 'age'])
Xma = MS(minus_age).fit_transform(Boston)
model1 = sm.OLS(y, Xma)
summarize(model1.fit())
```

3.6.4 Multivariate Goodness of Fit

Ta truy cập từng component của `results` bằng tên; `dir(results)` cho biết những gì có sẵn. Vì vậy, `results.rsquared` cho R^2 , còn `np.sqrt(results.scale)` cho RSE.

Variance inflation factor ở Mục 3.3.3 đôi khi hữu ích để đánh giá ảnh hưởng của collinearity trong model matrix. Ta sẽ tính VIF của multiple regression fit và đồng thời giới thiệu *list comprehension*.

list comprehension

List Comprehension

Ta thường cần biến đổi một chuỗi object cho tác vụ khác. Ở đây, ta tính VIF cho từng feature trong X và tạo data frame có index khớp column của X . List comprehension giúp thao tác này gọn hơn. Python còn hỗ trợ dictionary và generator comprehension, nhưng nằm ngoài phạm vi.

Ta dùng function `variance_inflation_factor()` để tính VIF cho từng biến trong model matrix.

```
In [29]: vals = [VIF(X, i)
               for i in range(1, X.shape[1])]
vif = pd.DataFrame({'vif':vals},
                   index=X.columns[1:])
vif
```

```
Out[29]:
```

	vif
crim	1.767
zn	2.298
indus	3.987
chas	1.071
nox	4.369
rm	1.913
age	3.088
dis	3.954
rad	7.445
tax	9.002
ptratio	1.797
lstat	2.871

Function `VIF()` nhận data frame hoặc array cùng column index. Ta gọi nó tại chỗ cho mọi column của X , bỏ column 0 là intercept vì không quan tâm. Trong trường hợp này, các VIF không quá đáng chú ý.

Object `vals` cũng có thể được dựng bằng for loop:

```
In [30]: vals = []
for i in range(1, X.values.shape[1]):
    vals.append(VIF(X.values, i))
```

List comprehension giúp thực hiện thao tác lặp như vậy trực tiếp hơn.

3.6.5 Interaction Terms

Đưa interaction term vào linear model bằng `ModelSpec()` rất dễ. Tuple `("lstat", "age")` yêu cầu model-matrix builder thêm interaction giữa `lstat` và `age`.

```
In [31]: X = MS(['lstat',
                'age',
                ('lstat', 'age')]).fit_transform(Boston)
model2 = sm.OLS(y, X)
summarize(model2.fit())
```

```
Out[31]:
```

	coef	std err	t	P> t
intercept	36.0885	1.470	24.553	0.000
lstat	-1.3921	0.167	-8.313	0.000
age	-0.0007	0.020	-0.036	0.971
lstat:age	0.0042	0.002	2.244	0.025

3.6.6 Non-linear Transformation của Predictor

Model-matrix builder có thể chứa term ngoài column name và interaction. Function `poly()` của `ISLP` yêu cầu thêm các column biểu diễn polynomial function của argument đầu tiên.

```
In [32]: X = MS([poly('lstat', degree=2), 'age']).fit_transform(Boston)
model3 = sm.OLS(y, X)
results3 = model3.fit()
summarize(results3)
```

```
Out[32]:
```

	coef	std err	t	P> t
intercept	17.7151	0.781	22.681	0.000
poly(lstat, degree=2)[0]	-179.2279	6.733	-26.620	0.000
poly(lstat, degree=2)[1]	72.9908	5.482	13.315	0.000
age	0.0703	0.011	6.471	0.000

p -value gần bằng 0 của quadratic term (hàng thứ ba) gợi ý rằng nó cải thiện mô hình.

Mặc định, `poly()` tạo basis matrix gồm các *orthogonal polynomial*, được thiết kế cho phép tính least squares ổn định.¹ Nếu thêm `raw=True`, basis matrix chỉ gồm `lstat` và `lstat**2`. Cả hai basis đều biểu diễn quadratic polynomial nên fitted value không đổi, chỉ polynomial coefficient thay đổi. Mặc định, column do `poly()` tạo không có intercept vì `MS()` tự thêm.

Ta dùng `anova_lm()` để định lượng thêm mức quadratic fit vượt linear fit.

```
In [33]: anova_lm(results1, results3)
```

```
Out[33]:
```

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	503.0	19168.13	0.0	NaN	NaN	NaN
1	502.0	14165.61	1.0	5002.52	177.28	7.47e-35

¹Thực chất, `poly()` là wrapper của workhorse function độc lập `Poly()` dùng để dựng model matrix.

`results1` là linear submodel chứa `lstat`, `age`; `results3` là mô hình lớn hơn có quadratic term của `lstat`. `anova_lm()` thực hiện hypothesis test so sánh hai mô hình. Null hypothesis nói quadratic term không cần thiết; alternative nói mô hình lớn tốt hơn. Ở đây $F = 177.28$ và p -value gần 0. F -statistic là bình phương t -statistic của quadratic term vì hai nested model chỉ khác một degree of freedom. Đây là bằng chứng rất rõ rằng quadratic polynomial của `lstat` cải thiện linear model, phù hợp dấu hiệu phi tuyến đã thấy giữa `medv` và `lstat`.

`anova_lm()` có thể nhận nhiều hơn hai nested model và so sánh từng cặp liên tiếp. Điều đó cũng giải thích các NaN ở hàng đầu: không có mô hình trước để so sánh.

```
In [34]: ax = subplots(figsize=(8,8))[1]
ax.scatter(results3.fittedvalues, results3.resid)
ax.set_xlabel('Fitted value')
ax.set_ylabel('Residual')
ax.axhline(0, c='k', ls='--')
```

Khi thêm quadratic term, residual hầu như không còn pattern rõ. Muốn fit polynomial bậc ba hoặc cao hơn, chỉ cần đổi argument `degree` của `poly()`.

3.6.7 Qualitative Predictors

Ở đây ta dùng dữ liệu `Carseats` trong package `ISLP`, dự đoán `Sales` (doanh số ghế trẻ em trên ô tô) tại 400 địa điểm bằng một số predictor.

```
In [35]: Carseats = load_data('Carseats')
Carseats.columns
```

```
Out[35]: Index(['Sales', 'CompPrice', 'Income', 'Advertising',
               'Population', 'Price', 'ShelveLoc', 'Age', 'Education',
               'Urban', 'US'],
              dtype='object')
```

Dữ liệu có qualitative predictor như `ShelveLoc`, chỉ chất lượng vị trí trưng bày ghế trong cửa hàng, với ba giá trị `Bad`, `Medium`, `Good`. Với qualitative variable như vậy, `ModelSpec()` tự tạo dummy variable, thường gọi là *one-hot encoding*. Các column của chúng có tổng bằng 1; để tránh collinearity với intercept, column đầu bị bỏ. Ở đây `ShelveLoc[Bad]` bị bỏ vì `Bad` là level đầu. Ta fit multiple regression model có một số interaction term.

one-hot
encoding

```
In [36]: allvars = list(Carseats.columns.drop('Sales'))
y = Carseats['Sales']
final = allvars + [('Income', 'Advertising'),
                  ('Price', 'Age')]
X = MS(final).fit_transform(Carseats)
model = sm.OLS(y, X)
summarize(model.fit())
```

```
Out[36]:
```

	coef	std err	t	P> t
intercept	6.5756	1.009	6.519	0.000
CompPrice	0.0929	0.004	22.567	0.000
Income	0.0109	0.003	4.183	0.000
Advertising	0.0702	0.023	3.107	0.002
Population	0.0002	0.000	0.433	0.665
Price	-0.1008	0.007	-13.549	0.000
ShelveLoc[Good]	4.8487	0.153	31.724	0.000
ShelveLoc[Medium]	1.9533	0.126	15.531	0.000
Age	-0.0579	0.016	-3.633	0.000
Education	-0.0209	0.020	-1.063	0.288
Urban[Yes]	0.1402	0.112	1.247	0.213
US[Yes]	-0.1576	0.149	-1.058	0.291
Income:Advertising	0.0008	0.000	2.698	0.007
Price:Age	0.0001	0.000	0.801	0.424

Ở dòng đầu, `allvars` được chuyển thành list để ta có thể thêm interaction term hai dòng sau. Model-matrix builder tạo `ShelveLoc[Good]` bằng 1 khi vị trí trưng bày tốt và 0 nếu không; `ShelveLoc[Medium]` bằng 1 khi vị trí trưng bày trung bình và 0 nếu không. Vị trí xấu tương ứng cả hai dummy variable đều bằng 0. Coefficient dương của `ShelveLoc[Good]` cho thấy vị trí tốt associated với sales cao hơn vị trí xấu. `ShelveLoc[Medium]` có coefficient dương nhỏ hơn: vị trí trưng bày trung bình cho sales cao hơn vị trí xấu nhưng thấp hơn vị trí tốt.

3.7 Exercises



Conceptual

1. Hãy mô tả các null hypothesis tương ứng với những p -value trong Bảng 3.4. Giải thích các kết luận có thể rút ra từ những p -value này. Phần giải thích phải được diễn đạt theo **sales**, **TV**, **radio** và **newspaper**, thay vì theo coefficient của linear model.
2. Giải thích cẩn thận sự khác nhau giữa KNN classifier và KNN regression.
3. Giả sử ta có dataset với năm predictor: $X_1 = \text{GPA}$, $X_2 = \text{IQ}$, $X_3 = \text{Level}$ (1 cho College, 0 cho High School), X_4 là interaction giữa GPA và IQ, còn X_5 là interaction giữa GPA và Level. Response là mức lương khởi điểm sau khi tốt nghiệp, tính bằng nghìn đô la. Dùng least squares fit mô hình và thu được

$$\hat{\beta}_0 = 50, \quad \hat{\beta}_1 = 20, \quad \hat{\beta}_2 = 0.07, \quad \hat{\beta}_3 = 35, \quad \hat{\beta}_4 = 0.01, \quad \hat{\beta}_5 = -10.$$

- (a) Đáp án nào đúng, và tại sao?
 - i. Với IQ và GPA cố định, trung bình học sinh tốt nghiệp High School kiếm được nhiều hơn người tốt nghiệp College.
 - ii. Với IQ và GPA cố định, trung bình người tốt nghiệp College kiếm được nhiều hơn học sinh tốt nghiệp High School.
 - iii. Với IQ và GPA cố định, học sinh tốt nghiệp High School kiếm được nhiều hơn người tốt nghiệp College nếu GPA đủ cao.
 - iv. Với IQ và GPA cố định, người tốt nghiệp College kiếm được nhiều hơn học sinh tốt nghiệp High School nếu GPA đủ cao.
 - (b) Dự đoán lương của một người tốt nghiệp College có IQ bằng 110 và GPA bằng 4.0.
 - (c) Đúng hay sai: Vì coefficient của interaction term GPA/IQ rất nhỏ nên có rất ít bằng chứng về interaction effect. Hãy giải thích.
4. Tôi thu thập dataset có $n = 100$ observation, gồm một predictor và một quantitative response. Sau đó, tôi fit linear regression model và một cubic regression riêng biệt,

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon.$$

- (a) Giả sử quan hệ thực giữa X, Y là tuyến tính, $Y = \beta_0 + \beta_1 X + \epsilon$. Xét training RSS của linear regression và cubic regression. Ta kỳ vọng một giá trị thấp hơn, kỳ vọng chúng bằng nhau, hay chưa đủ thông tin để kết luận? Hãy giải thích.
- (b) Trả lời (a) khi dùng test RSS thay cho training RSS.
- (c) Giả sử quan hệ thực giữa X, Y không tuyến tính nhưng ta không biết nó lệch tuyến tính bao xa. Xét training RSS của linear

regression và cubic regression. Ta kỳ vọng một giá trị thấp hơn, chúng bằng nhau, hay chưa đủ thông tin? Hãy giải thích.

(d) Trả lời (c) khi dùng test RSS thay cho training RSS.

5. Xét fitted value thu được từ linear regression không có intercept. Khi đó, fitted value thứ i là

$$\hat{y}_i = x_i \hat{\beta}, \quad \hat{\beta} = \frac{\sum_{i=1}^n x_i y_i}{\sum_{i=1}^n x_i^2}. \quad (3.1)$$

Chứng minh rằng có thể viết

$$\hat{y}_i = \sum_{i'=1}^n a_{i'} y_{i'}.$$

$a_{i'}$ là gì?

Lưu ý: Kết quả này được diễn giải rằng fitted value của linear regression là linear combination của response value.

6. Dùng (3.4) để lập luận rằng trong simple linear regression, least squares line luôn đi qua điểm $(\bar{x}, \bar{y})$.
7. Trong simple linear regression của Y trên X , sách khẳng định R^2 statistic (3.17) bằng bình phương correlation giữa X, Y trong (3.18). Hãy chứng minh. Để đơn giản, có thể giả sử $\bar{x} = \bar{y} = 0$.

Applied

8. Bài này dùng simple linear regression trên dataset **Auto**.
- Dùng function `sm.OLS()` thực hiện simple linear regression với **mpg** là response và **horsepower** là predictor. Dùng `summarize()` in kết quả và nhận xét output, chẳng hạn:
 - Predictor và response có quan hệ không?
 - Mối quan hệ mạnh đến đâu?
 - Quan hệ dương hay âm?
 - Predicted **mpg** ứng với **horsepower=98** là bao nhiêu? Confidence interval và prediction interval 95% tương ứng là gì?
 - Vẽ response và predictor trên một tập axis mới **ax**. Dùng method `ax.axline()` hoặc function `abline()` đã định nghĩa trong lab để biểu diễn least squares regression line.
 - Tạo một số diagnostic plot của least squares regression fit như mô tả trong lab. Nhận xét mọi vấn đề nhìn thấy ở fit.
9. Bài này dùng multiple linear regression trên dataset **Auto**.
- Tạo scatterplot matrix gồm mọi biến trong dataset.
 - Tính correlation matrix giữa các biến bằng method `DataFrame.corr()`.

- (c) Dùng `sm.OLS()` thực hiện multiple linear regression với `mpg` là response và mọi biến khác trừ `name` là predictor. Dùng `summarize()` in kết quả và nhận xét output, chẳng hạn:
- Các predictor và response có quan hệ không? Dùng `anova_lm()` từ `statsmodels` để trả lời.
 - Predictor nào có quan hệ statistically significant với response?
 - Coefficient của biến `year` gợi ý điều gì?
- (d) Tạo một số diagnostic plot của linear regression fit như trong lab. Nhận xét vấn đề nhìn thấy. Residual plot có gợi ý outlier đặc biệt lớn không? Leverage plot có chỉ ra observation có leverage cao bất thường không?
- (e) Fit một số model có interaction như trong lab. Interaction nào statistically significant không?
- (f) Thử vài transformation khác nhau của biến, như $\log(X)$, $\sqrt{X}$, X^2 . Nhận xét kết quả.
10. Trả lời bài này bằng dataset `Carseats`.
- Fit multiple regression model dự đoán `Sales` bằng `Price`, `Urban` và `US`.
 - Diễn giải từng coefficient trong mô hình. Hãy cẩn thận: một số biến là qualitative.
 - Viết mô hình dưới dạng equation, xử lý đúng qualitative variable.
 - Với predictor nào có thể bác bỏ null hypothesis $H_0 : \beta_j = 0$?
 - Dựa trên câu trước, fit mô hình nhỏ hơn chỉ dùng predictor có bằng chứng associated với outcome.
 - Mô hình ở (a) và (e) fit dữ liệu tốt đến đâu?
 - Dùng mô hình ở (e), tìm confidence interval 95% cho các coefficient.
 - Có bằng chứng về outlier hay high-leverage observation trong mô hình ở (e) không?
11. Ta khảo sát t -statistic cho null hypothesis $H_0 : \beta = 0$ trong simple linear regression không có intercept. Trước hết, tạo predictor `x` và response `y` như sau:
- ```
rng = np.random.default_rng(1)
x = rng.normal(size=100)
y = 2 * x + rng.normal(size=100)
```
- Thực hiện simple linear regression của `y` trên `x`, không có intercept. Báo cáo coefficient estimate  $\hat{\beta}$ , standard error,  $t$ -statistic và  $p$ -value của  $H_0 : \beta = 0$ ; nhận xét kết quả. Có thể regression không intercept bằng keyword argument `intercept=False` cho `ModelSpec()`.
  - Regression `x` trên `y` không intercept, rồi báo cáo coefficient estimate, standard error,  $t$ -statistic và  $p$ -value tương ứng với  $H_0 : \beta = 0$ . Nhận xét.

- (c) Kết quả ở (a) và (b) liên hệ thế nào?
- (d) Với regression  $Y$  trên  $X$  không intercept,  $t$ -statistic cho  $H_0 : \beta = 0$  là  $\hat{\beta}/\text{SE}(\hat{\beta})$ , trong đó  $\hat{\beta}$  được cho bởi (3.38) và

$$\text{SE}(\hat{\beta}) = \sqrt{\frac{\sum_{i=1}^n (y_i - x_i \hat{\beta})^2}{(n-1) \sum_{i'=1}^n x_{i'}^2}}.$$

Chứng minh bằng đại số và xác nhận bằng số trong Python rằng  $t$ -statistic có thể viết thành

$$\frac{\sqrt{n-1} \sum_{i=1}^n x_i y_i}{\sqrt{(\sum_{i=1}^n x_i^2)(\sum_{i'=1}^n y_{i'}^2) - (\sum_{i'=1}^n x_{i'} y_{i'})^2}}.$$

- (e) Từ (d), lập luận rằng  $t$ -statistic của regression  $\mathbf{y}$  trên  $\mathbf{x}$  giống  $t$ -statistic của regression  $\mathbf{x}$  trên  $\mathbf{y}$ .
- (f) Trong Python, chỉ ra rằng khi regression có intercept,  $t$ -statistic cho  $H_0 : \beta_1 = 0$  giống nhau giữa regression  $\mathbf{y}$  trên  $\mathbf{x}$  và regression  $\mathbf{x}$  trên  $\mathbf{y}$ .
12. Bài này xét simple linear regression không có intercept.
- (a) Nhớ rằng coefficient estimate  $\hat{\beta}$  của regression  $Y$  trên  $X$  không intercept được cho bởi (3.38). Trong điều kiện nào coefficient estimate của regression  $X$  trên  $Y$  bằng coefficient estimate của regression  $Y$  trên  $X$ ?
- (b) Tạo một ví dụ bằng Python với  $n = 100$  observation trong đó hai coefficient estimate khác nhau.
- (c) Tạo một ví dụ bằng Python với  $n = 100$  observation trong đó hai coefficient estimate bằng nhau.
13. Trong bài này, bạn sẽ tạo dữ liệu mô phỏng và fit simple linear regression model. Trước phần (a), hãy dùng default random number generator với seed bằng 1 để bảo đảm kết quả nhất quán.
- (a) Dùng method `normal()` của random number generator tạo vector  $\mathbf{x}$  gồm 100 observation từ phân phối  $N(0, 1)$ . Đây là feature  $X$ .
- (b) Dùng `normal()` tạo vector  $\mathbf{eps}$  gồm 100 observation từ phân phối  $N(0, 0.25)$ —normal distribution có mean 0 và variance 0,25.
- (c) Dùng  $\mathbf{x}$ ,  $\mathbf{eps}$  tạo vector  $\mathbf{y}$  theo mô hình

$$Y = -1 + 0.5X + \epsilon. \quad (3.2)$$

Vector  $\mathbf{y}$  dài bao nhiêu?  $\beta_0, \beta_1$  của linear model bằng bao nhiêu?

- (d) Tạo scatterplot biểu diễn quan hệ giữa  $\mathbf{x}$ ,  $\mathbf{y}$ . Nhận xét điều quan sát được.
- (e) Fit least squares linear model dự đoán  $\mathbf{y}$  bằng  $\mathbf{x}$ . Nhận xét mô hình.  $\hat{\beta}_0, \hat{\beta}_1$  so với  $\beta_0, \beta_1$  thế nào?

- (f) Vẽ least squares line trên scatterplot ở (d). Vẽ population regression line bằng màu khác. Dùng method `legend()` của axis tạo legend phù hợp.
- (g) Fit polynomial regression model dự đoán  $y$  bằng  $x$ ,  $x^2$ . Có bằng chứng quadratic term cải thiện model fit không? Giải thích.
- (h) Lặp (a)–(f) sau khi sửa data-generation process để dữ liệu ít noise hơn, vẫn giữ mô hình (3.39). Có thể giảm variance của normal distribution dùng tạo error term  $\epsilon$  ở (b). Mô tả kết quả.
- (i) Lặp (a)–(f) với nhiều noise hơn bằng cách tăng variance của error term, vẫn giữ (3.39). Mô tả kết quả.
- (j) Confidence interval của  $\beta_0, \beta_1$  trên dataset gốc, dataset nhiều noise và dataset ít noise là gì? Nhận xét.

14. Bài này tập trung vào collinearity.

- (a) Thực hiện các lệnh Python sau:

```
rng = np.random.default_rng(10)
x1 = rng.uniform(0, 1, size=100)
x2 = 0.5 * x1 + rng.normal(size=100) / 10
y = 2 + 2 * x1 + 0.3 * x2 + rng.normal(size=100)
```

Dòng cuối tạo linear model trong đó  $y$  là hàm của  $x1$ ,  $x2$ . Viết dạng linear model và xác định regression coefficient.

- (b) Correlation giữa  $x1$ ,  $x2$  bằng bao nhiêu? Tạo scatterplot biểu diễn quan hệ giữa chúng.
- (c) Dùng dữ liệu này fit least squares regression dự đoán  $y$  bằng  $x1$ ,  $x2$ . Mô tả kết quả.  $\hat{\beta}_0, \hat{\beta}_1, \hat{\beta}_2$  bằng bao nhiêu và liên hệ thế nào với giá trị thực? Có bác bỏ  $H_0 : \beta_1 = 0$  không? Còn  $H_0 : \beta_2 = 0$ ?
- (d) Fit least squares regression dự đoán  $y$  chỉ bằng  $x1$ . Nhận xét. Có bác bỏ  $H_0 : \beta_1 = 0$  không?
- (e) Fit least squares regression dự đoán  $y$  chỉ bằng  $x2$ . Nhận xét. Có bác bỏ  $H_0 : \beta_1 = 0$  không?
- (f) Kết quả ở (c)–(e) có mâu thuẫn nhau không? Giải thích.
- (g) Giả sử có thêm một observation bị đo sai. Dùng `np.concatenate()` thêm nó vào  $x1$ ,  $x2$ ,  $y$ :

```
x1 = np.concatenate([x1, [0.1]])
x2 = np.concatenate([x2, [0.8]])
y = np.concatenate([y, [6]])
```

Fit lại các linear model từ (c)–(e). Observation mới ảnh hưởng thế nào đến từng mô hình? Trong mỗi mô hình, nó là outlier, high-leverage point, cả hai, hay không phải? Giải thích.

15. Bài này dùng dataset `Boston` đã gặp trong lab. Ta dự đoán per-capita crime rate bằng các biến khác: crime rate là response và các biến còn lại là predictor.

- (a) Với từng predictor, fit simple linear regression model để dự đoán response. Mô tả kết quả. Mô hình nào có association statistically significant giữa predictor và response? Tạo plot hỗ trợ kết luận.
- (b) Fit multiple regression model dự đoán response bằng mọi predictor. Mô tả kết quả. Với predictor nào có thể bác bỏ  $H_0 : \beta_j = 0$ ?
- (c) So sánh kết quả (a) với (b). Tạo plot có univariate regression coefficient ở trục  $x$  và multiple regression coefficient ở trục  $y$ . Mỗi predictor là một điểm: coefficient simple linear regression trên trục  $x$ , coefficient estimate multiple linear regression trên trục  $y$ .
- (d) Có bằng chứng về association phi tuyến giữa predictor nào và response không? Với từng predictor  $X$ , fit mô hình

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon.$$



# Classification

Linear regression model được thảo luận trong Chương 3 giả định response variable  $Y$  là quantitative. Nhưng trong nhiều tình huống, response variable lại là qualitative. Chẳng hạn, màu mắt là qualitative. Các qualitative variable thường được gọi là categorical; chúng tôi sẽ dùng hai thuật ngữ này thay thế cho nhau. Trong chương này, ta nghiên cứu các cách tiếp cận để dự đoán qualitative response, một quá trình được gọi là *classification*. Việc dự đoán qualitative response cho một observation có thể được gọi là phân loại observation đó, vì nó bao gồm việc gán observation vào một category, hay class. Mặt khác, các phương pháp dùng cho classification thường trước hết dự đoán xác suất observation thuộc về từng category của qualitative variable, rồi dựa vào đó để thực hiện classification. Theo nghĩa này, chúng cũng hoạt động giống các regression method.

Có nhiều kỹ thuật classification, hay *classifier*, có thể được dùng để dự đoán qualitative response. Ta đã đề cập sơ lược một số kỹ thuật trong các Mục 2.1.5 và 2.2.3. Trong chương này, chúng tôi thảo luận một số classifier được dùng rộng rãi: *logistic regression*, *linear discriminant analysis*, *quadratic discriminant analysis*, *naive Bayes* và *K-nearest neighbors*. Phần trình bày logistic regression được dùng làm điểm xuất phát để thảo luận *generalized linear model*, đặc biệt là *Poisson regression*. Những classification method đòi hỏi tính toán chuyên sâu hơn sẽ được thảo luận ở các chương sau: gồm generalized additive model (Chương 7); tree, random forest và boosting (Chương 8); cùng support vector machine (Chương 9).

## 4.1 An Overview of Classification

Các bài toán classification xuất hiện rất thường xuyên, thậm chí có lẽ còn thường xuyên hơn bài toán regression. Một số ví dụ gồm:

classification

classifier

logistic  
regressionlinear  
discriminant  
analysisquadratic  
discriminant  
analysis

naive Bayes

K-nearest  
neighborsgeneralized  
linear modelPoisson  
regression

1. Một người đến phòng cấp cứu với một tập triệu chứng có thể được quy cho một trong ba tình trạng bệnh lý. Người đó mắc tình trạng nào trong ba tình trạng này?
2. Một dịch vụ ngân hàng trực tuyến phải xác định một giao dịch đang được thực hiện trên trang có gian lận hay không, dựa trên địa chỉ IP của người dùng, lịch sử giao dịch trước đây, v.v.
3. Dựa trên dữ liệu DNA sequence của một số bệnh nhân có và không mắc một căn bệnh nhất định, một nhà sinh học muốn xác định những DNA mutation nào là deleterious (gây bệnh) và những mutation nào không.

Cũng như trong bối cảnh regression, trong bối cảnh classification ta có một tập training observation  $(x_1, y_1), \dots, (x_n, y_n)$  để xây dựng classifier. Ta muốn classifier hoạt động tốt không chỉ trên training data mà cả trên những test observation không được dùng để train classifier.

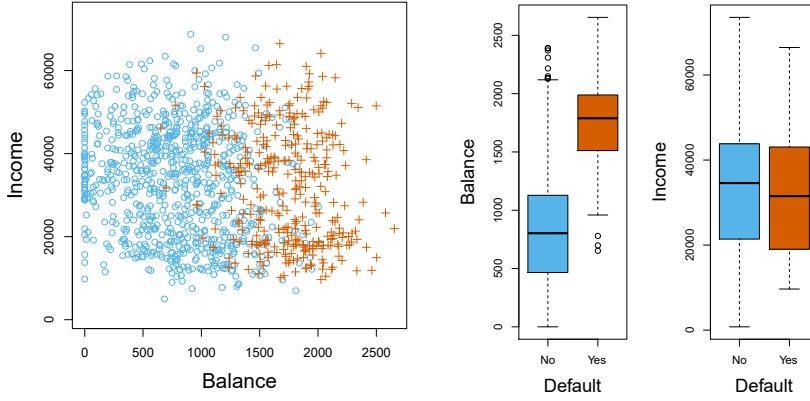
Trong chương này, chúng tôi minh họa khái niệm classification bằng dataset mô phỏng **Default**. Ta quan tâm đến việc dự đoán liệu một cá nhân có default khoản thanh toán thẻ tín dụng hay không, dựa trên annual income và monthly credit card balance. Dataset được biểu diễn trong Hình 4.1. Ở ô bên trái của Hình 4.1, chúng tôi vẽ annual income và monthly credit card balance cho một tập con gồm 10.000 cá nhân. Những người default trong một tháng nhất định được biểu diễn bằng màu cam, còn những người không default bằng màu xanh lam. (Default rate tổng thể khoảng 3%, vì vậy chúng tôi chỉ vẽ một phần những người không default.) Có vẻ những người default có xu hướng mang credit card balance cao hơn những người không default. Hai ô giữa và phải của Hình 4.1 biểu diễn hai cặp boxplot. Cặp thứ nhất cho thấy distribution của **balance** được chia theo binary variable **default**; cặp thứ hai là đồ thị tương tự cho **income**. Trong chương này, ta sẽ học cách xây dựng một model để dự đoán **default** ( $Y$ ) với bất kỳ giá trị nào của **balance** ( $X_1$ ) và **income** ( $X_2$ ). Vì  $Y$  không phải quantitative, simple linear regression model của Chương 3 không phải lựa chọn tốt; ta sẽ giải thích kỹ hơn điều này trong Mục 4.2.

Đáng lưu ý là Hình 4.1 thể hiện mối quan hệ rất rõ rệt giữa predictor **balance** và response **default**. Trong phần lớn ứng dụng thực tế, mối quan hệ giữa predictor và response sẽ không mạnh đến vậy. Tuy nhiên, để minh họa các classification procedure được thảo luận trong chương này, chúng tôi dùng một ví dụ trong đó mối quan hệ giữa predictor và response được nhấn mạnh hơn đôi chút.

## 4.2 Why Not Linear Regression?

Chúng tôi đã khẳng định rằng linear regression không phù hợp khi response là qualitative. Tại sao?

Giả sử ta đang cố dự đoán tình trạng bệnh lý của một bệnh nhân tại phòng cấp cứu dựa trên các triệu chứng của người đó. Trong ví dụ đơn giản này, có ba chẩn đoán khả dĩ: stroke, drug overdose và epileptic seizure. Ta



**HÌNH 4.1.** Dataset *Default*. Trái: Annual income và monthly credit card balance của một số cá nhân. Những người default khoản thanh toán thẻ tín dụng được biểu diễn bằng màu cam, còn những người không default bằng màu xanh lam. Giữa: Boxplot của *balance* theo trạng thái *default*. Phải: Boxplot của *income* theo trạng thái *default*.

có thể cân nhắc mã hóa các giá trị này thành một quantitative response variable  $Y$  như sau:

$$Y = \begin{cases} 1 & \text{nếu stroke;} \\ 2 & \text{nếu drug overdose;} \\ 3 & \text{nếu epileptic seizure.} \end{cases}$$

Với cách mã hóa này, có thể dùng least squares để fit một linear regression model nhằm dự đoán  $Y$  dựa trên một tập predictor  $X_1, \dots, X_p$ . Tiếc rằng cách mã hóa này áp đặt một thứ tự lên các outcome: đặt drug overdose ở giữa stroke và epileptic seizure, đồng thời buộc chênh lệch giữa stroke và drug overdose phải bằng chênh lệch giữa drug overdose và epileptic seizure. Trong thực tế, không có lý do cụ thể nào khiến điều này phải đúng. Chẳng hạn, ta có thể chọn một cách mã hóa hợp lý không kém:

$$Y = \begin{cases} 1 & \text{nếu epileptic seizure;} \\ 2 & \text{nếu stroke;} \\ 3 & \text{nếu drug overdose,} \end{cases}$$

nhưng cách này lại hàm ý một mối quan hệ hoàn toàn khác giữa ba tình trạng. Mỗi cách mã hóa sẽ tạo ra những linear model khác nhau về căn bản, cuối cùng dẫn đến những tập prediction khác nhau trên test observation.

Nếu các giá trị của response variable thực sự có một thứ tự tự nhiên, chẳng hạn mild, moderate và severe, đồng thời ta cho rằng khoảng cách giữa mild và moderate tương tự khoảng cách giữa moderate và severe, thì cách mã hóa 1, 2, 3 là hợp lý. Tiếc rằng nhìn chung không có cách tự nhiên nào để chuyển một qualitative response variable có hơn hai level thành một quantitative response sẵn sàng cho linear regression.

Với binary qualitative response (có hai level), tình hình tốt hơn. Chẳng hạn, có thể chỉ có hai khả năng đối với tình trạng bệnh lý của bệnh nhân:



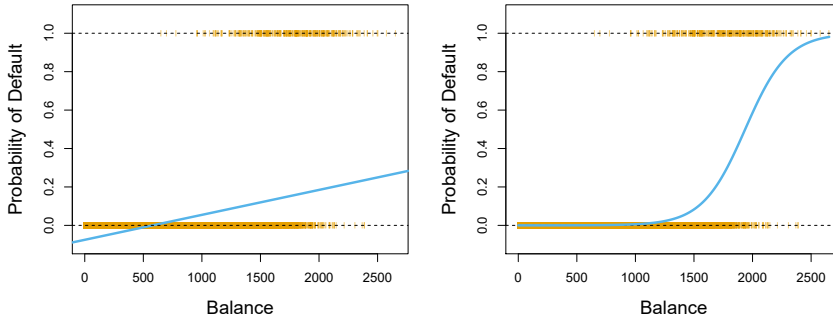
stroke và drug overdose. Khi đó, ta có thể dùng cách tiếp cận dummy variable từ Mục 3.3.1 để mã hóa response như sau:

$$Y = \begin{cases} 0 & \text{nếu stroke;} \\ 1 & \text{nếu drug overdose.} \end{cases}$$

Sau đó, ta có thể fit linear regression cho binary response này và dự đoán drug overdose nếu  $\hat{Y} > 0.5$ , còn không thì dự đoán stroke. Trong trường hợp binary, không khó để chứng minh rằng ngay cả khi đảo cách mã hóa trên, linear regression vẫn tạo ra cùng những prediction cuối cùng.

Đối với binary response được mã hóa 0/1 như trên, regression bằng least squares không hoàn toàn vô lý: có thể chứng minh rằng  $X\hat{\beta}$  thu được bằng linear regression thực ra là một ước lượng của  $\Pr(\text{drug overdose} \mid X)$  trong trường hợp đặc biệt này. Tuy nhiên, nếu dùng linear regression, một số ước lượng có thể nằm ngoài khoảng  $[0, 1]$  (xem Hình 4.2), khiến chúng khó được diễn giải như xác suất! Dẫu vậy, các prediction vẫn cung cấp một thứ tự và có thể được diễn giải như những ước lượng xác suất thô. Điều khá thú vị là classification thu được khi dùng linear regression để dự đoán binary response sẽ giống với classification của thủ tục linear discriminant analysis (LDA) được thảo luận trong Mục 4.4.

Tóm lại, có ít nhất hai lý do không nên thực hiện classification bằng regression method: (a) regression method không thể xử lý qualitative response có hơn hai class; (b) regression method sẽ không cung cấp các ước lượng có ý nghĩa cho  $\Pr(Y \mid X)$ , ngay cả khi chỉ có hai class. Vì vậy, nên dùng một classification method thực sự phù hợp với qualitative response value. Ở mục tiếp theo, chúng tôi trình bày logistic regression, phương pháp rất phù hợp cho binary qualitative response; trong các mục sau nữa, chúng tôi sẽ đề cập những classification method thích hợp khi qualitative response có từ hai class trở lên.



**HÌNH 4.2.** Classification bằng dữ liệu *Default*. Trái: Xác suất default ước lượng bằng linear regression. Một số xác suất ước lượng mang giá trị âm! Các vạch màu cam biểu diễn giá trị 0/1 được mã hóa cho *default* (No hoặc Yes). Phải: Xác suất default dự đoán bằng logistic regression. Mọi xác suất đều nằm giữa 0 và 1.

### 4.3 Logistic Regression

Xét lại dataset *Default*, trong đó response *default* thuộc một trong hai category, *Yes* hoặc *No*. Thay vì trực tiếp modeling response  $Y$  này, logistic regression modeling xác suất  $Y$  thuộc một category cụ thể.

Với dữ liệu *Default*, logistic regression modeling xác suất default. Chẳng hạn, xác suất default khi biết *balance* có thể được viết là

$$\Pr(\text{default} = \text{Yes} \mid \text{balance}).$$

Các giá trị của  $\Pr(\text{default} = \text{Yes} \mid \text{balance})$ , được viết tắt là  $p(\text{balance})$ , sẽ nằm trong khoảng từ 0 đến 1. Khi đó, với bất kỳ giá trị *balance* nào, ta đều có thể đưa ra prediction cho *default*. Chẳng hạn, có thể dự đoán  $\text{default} = \text{Yes}$  cho mọi cá nhân có  $p(\text{balance}) > 0.5$ . Hoặc nếu một công ty muốn thận trọng khi dự đoán những cá nhân có nguy cơ default, họ có thể chọn threshold thấp hơn, chẳng hạn  $p(\text{balance}) > 0.1$ .

#### 4.3.1 The Logistic Model

Ta nên modeling mối quan hệ giữa  $p(X) = \Pr(Y = 1 \mid X)$  và  $X$  như thế nào? (Để tiện trình bày, ta dùng cách mã hóa response tổng quát 0/1.) Trong Mục 4.2, ta đã cân nhắc dùng linear regression model để biểu diễn các xác suất này:

$$p(X) = \beta_0 + \beta_1 X. \quad (4.1)$$

Nếu dùng cách tiếp cận này để dự đoán  $\text{default} = \text{Yes}$  bằng *balance*, ta thu được model trong ô bên trái của Hình 4.2. Ở đây, vấn đề của cách tiếp cận này bộc lộ rõ: với *balance* gần bằng 0, ta dự đoán xác suất default âm; nếu dự đoán cho *balance* rất lớn, ta sẽ nhận giá trị lớn hơn 1. Những prediction này không hợp lý, vì xác suất default thực, bất kể credit card *balance* là bao nhiêu, hiển nhiên phải nằm giữa 0 và 1. Vấn đề này không chỉ xảy ra với dữ liệu credit default. Bất cứ khi nào fit một đường thẳng cho binary response được mã hóa là 0 hoặc 1, về nguyên tắc ta luôn có thể

dự đoán  $p(X) < 0$  với một số giá trị  $X$  và  $p(X) > 1$  với những giá trị khác (trừ khi range của  $X$  bị giới hạn).

Để tránh vấn đề này, ta phải modeling  $p(X)$  bằng một hàm cho output nằm giữa 0 và 1 với mọi giá trị  $X$ . Có nhiều hàm thỏa mãn mô tả đó. Trong logistic regression, ta dùng *logistic function*,

$$p(X) = \frac{e^{\beta_0 + \beta_1 X}}{1 + e^{\beta_0 + \beta_1 X}}. \quad (4.2)$$

Để fit model (4.2), ta dùng một phương pháp gọi là *maximum likelihood*, sẽ được thảo luận ở mục tiếp theo. Ở bên phải của Hình 4.2 minh họa logistic regression model được fit vào dữ liệu **Default**. Lưu ý rằng với balance thấp, giờ đây xác suất default được dự đoán gần bằng nhưng không bao giờ nhỏ hơn 0. Tương tự, với balance cao, ta dự đoán xác suất default gần bằng nhưng không bao giờ lớn hơn 1. Logistic function luôn tạo ra đường cong hình chữ S như vậy, nên với mọi giá trị  $X$ , ta đều nhận được prediction hợp lý. Ta cũng thấy logistic model nắm bắt range của xác suất tốt hơn linear regression model ở đồ thị bên trái. Average fitted probability trong cả hai trường hợp là 0.0333 (lấy trung bình trên training data), bằng đúng tỷ lệ người default tổng thể trong dataset.

Sau một vài biến đổi của (4.2), ta thu được

$$\frac{p(X)}{1 - p(X)} = e^{\beta_0 + \beta_1 X}. \quad (4.3)$$

Đại lượng  $p(X)/[1 - p(X)]$  được gọi là *odds* và có thể nhận mọi giá trị từ 0 đến  $\infty$ . Odds gần 0 và  $\infty$  lần lượt biểu thị xác suất default rất thấp và rất cao. Chẳng hạn, trung bình cứ 5 người có odds bằng 1/4 thì 1 người sẽ default, vì  $p(X) = 0.2$  hàm ý odds bằng  $0.2/(1 - 0.2) = 1/4$ . Tương tự, trung bình cứ 10 người có odds bằng 9 thì 9 người sẽ default, vì  $p(X) = 0.9$  hàm ý odds bằng  $0.9/(1 - 0.9) = 9$ . Trong horse-racing, theo truyền thống người ta dùng odds thay cho xác suất vì odds liên hệ tự nhiên hơn với betting strategy đúng.

Lấy logarithm hai vế của (4.3), ta có

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X. \quad (4.4)$$

Vế trái được gọi là *log odds* hoặc *logit*. Ta thấy logistic regression model (4.2) có logit tuyến tính theo  $X$ .

Nhớ lại từ Chương 3, trong linear regression model,  $\beta_1$  cho biết mức thay đổi trung bình của  $Y$  đi kèm mức tăng một đơn vị của  $X$ . Trái lại, trong logistic regression model, tăng  $X$  một đơn vị làm log odds thay đổi  $\beta_1$  (4.4). Tương đương, nó nhân odds với  $e^{\beta_1}$  (4.3). Tuy nhiên, vì mối quan hệ giữa  $p(X)$  và  $X$  trong (4.2) không phải đường thẳng,  $\beta_1$  không tương ứng với mức thay đổi của  $p(X)$  đi kèm mức tăng một đơn vị của  $X$ . Mức thay đổi của  $p(X)$  do  $X$  thay đổi một đơn vị phụ thuộc vào giá trị hiện tại của  $X$ . Nhưng bất kể  $X$  nhận giá trị nào, nếu  $\beta_1$  dương thì tăng  $X$  sẽ đi kèm tăng  $p(X)$ ; nếu  $\beta_1$  âm thì tăng  $X$  sẽ đi kèm giảm  $p(X)$ . Việc  $p(X)$  và  $X$  không có quan hệ đường thẳng, cũng như rate of change của  $p(X)$  trên mỗi đơn vị thay đổi của  $X$  phụ thuộc vào giá trị hiện tại của  $X$ , cũng có thể thấy trực tiếp ở ô bên phải của Hình 4.2.

**BẢNG 4.1.** Với dữ liệu *Default*, các *coefficient estimate* của *logistic regression model* dự đoán xác suất default bằng *balance*. *balance* tăng một đơn vị đi kèm log odds của default tăng 0.0055 đơn vị.

|           | Coefficient | Std. error | z-statistic | p-value  |
|-----------|-------------|------------|-------------|----------|
| Intercept | -10.6513    | 0.3612     | -29.5       | < 0.0001 |
| balance   | 0.0055      | 0.0002     | 24.9        | < 0.0001 |

### 4.3.2 Estimating the Regression Coefficients

Các coefficient  $\beta_0$  và  $\beta_1$  trong (4.2) chưa biết và phải được ước lượng dựa trên training data sẵn có. Trong Chương 3, ta dùng cách tiếp cận least squares để ước lượng các linear regression coefficient chưa biết. Dù có thể dùng (non-linear) least squares để fit model (4.4), phương pháp tổng quát hơn là maximum likelihood được ưu tiên vì có statistical property tốt hơn. Trực giác cơ bản của việc dùng maximum likelihood để fit logistic regression model như sau: ta tìm các ước lượng  $\beta_0$  và  $\beta_1$  sao cho xác suất default dự đoán  $\hat{p}(x_i)$  của từng cá nhân theo (4.2) khớp sát nhất có thể với trạng thái default quan sát được của cá nhân đó. Nói cách khác, ta cố tìm  $\hat{\beta}_0$  và  $\hat{\beta}_1$  sao cho thay các ước lượng này vào model của  $p(X)$  trong (4.2) sẽ cho một số gần 1 với mọi cá nhân đã default và một số gần 0 với mọi cá nhân không default. Trực giác này có thể được hình thức hóa bằng một phương trình toán học gọi là *likelihood function*:

likelihood  
function

$$\ell(\beta_0, \beta_1) = \prod_{i: y_i=1} p(x_i) \prod_{i': y_{i'}=0} (1 - p(x_{i'})). \quad (4.5)$$

Các ước lượng  $\hat{\beta}_0$  và  $\hat{\beta}_1$  được chọn để maximize likelihood function này.

Maximum likelihood là một cách tiếp cận rất tổng quát, được dùng để fit nhiều non-linear model mà ta khảo sát xuyên suốt cuốn sách. Trong bối cảnh linear regression, cách tiếp cận least squares thực ra là một trường hợp đặc biệt của maximum likelihood. Các chi tiết toán học của maximum likelihood nằm ngoài phạm vi cuốn sách này. Tuy nhiên, nhìn chung logistic regression và các model khác có thể dễ dàng được fit bằng statistical software như R, nên ta không cần bận tâm đến chi tiết của thủ tục maximum likelihood fitting.

Bảng 4.1 trình bày coefficient estimate và thông tin liên quan thu được khi fit logistic regression model trên dữ liệu *Default* để dự đoán xác suất *default = Yes* bằng *balance*. Ta thấy  $\hat{\beta}_1 = 0.0055$ ; điều này cho biết *balance* tăng đi kèm xác suất default tăng. Chính xác hơn, *balance* tăng một đơn vị đi kèm log odds của default tăng 0.0055 đơn vị.

Nhiều khía cạnh trong logistic regression output ở Bảng 4.1 tương tự linear regression output của Chương 3. Chẳng hạn, ta có thể đo độ chính xác của coefficient estimate bằng standard error. *z*-statistic trong Bảng 4.1 có vai trò giống *t*-statistic trong linear regression output, ví dụ Bảng 3.1. *z*-statistic ứng với  $\beta_1$  bằng  $\hat{\beta}_1 / \text{SE}(\hat{\beta}_1)$ , nên giá trị tuyệt đối lớn của *z*-statistic cung cấp bằng chứng chống lại null hypothesis  $H_0 : \beta_1 = 0$ . Null hypothesis này hàm ý  $p(X) = e^{\beta_0} / (1 + e^{\beta_0})$ ; nói cách khác, xác suất default không phụ thuộc vào *balance*. Vì *p*-value gắn với *balance* trong Bảng 4.1 cực nhỏ,

**BẢNG 4.2.** Với dữ liệu *Default*, các *coefficient estimate* của *logistic regression model* dự đoán xác suất default bằng *student status*. *Student status* được mã hóa thành *dummy variable*, nhận giá trị 1 với *student* và 0 với *non-student*, đồng thời được biểu diễn bằng biến *student[Yes]* trong bảng.

|              | Coefficient | Std. error | z-statistic | p-value  |
|--------------|-------------|------------|-------------|----------|
| Intercept    | -3.5041     | 0.0707     | -49.55      | < 0.0001 |
| student[Yes] | 0.4049      | 0.1150     | 3.52        | 0.0004   |

ta có thể bác bỏ  $H_0$ . Nói cách khác, ta kết luận thực sự có association giữa balance và xác suất default. Estimated intercept trong Bảng 4.1 thường không phải điều được quan tâm; mục đích chính của nó là điều chỉnh average fitted probability cho khớp với tỷ lệ các giá trị 1 trong dữ liệu (ở đây là default rate tổng thể).

### 4.3.3 Making Predictions

Sau khi các coefficient đã được ước lượng, ta có thể tính xác suất default với bất kỳ credit card balance nào. Chẳng hạn, dùng coefficient estimate trong Bảng 4.1, ta dự đoán xác suất default của một cá nhân có balance 1.000 đô la là

$$\hat{p}(X) = \frac{e^{\hat{\beta}_0 + \hat{\beta}_1 X}}{1 + e^{\hat{\beta}_0 + \hat{\beta}_1 X}} = \frac{e^{-10.6513 + 0.0055 \times 1,000}}{1 + e^{-10.6513 + 0.0055 \times 1,000}} = 0.00576,$$

thấp hơn 1%. Trái lại, xác suất default dự đoán cho một cá nhân có balance 2.000 đô la cao hơn nhiều, bằng 0.586 hay 58.6%.

Ta có thể dùng qualitative predictor với logistic regression model bằng cách tiếp cận dummy variable trong Mục 3.3.1. Chẳng hạn, dataset *Default* chứa qualitative variable *student*. Để fit một model dùng student status làm predictor variable, ta chỉ cần tạo dummy variable nhận giá trị 1 với student và 0 với non-student. Logistic regression model thu được khi dự đoán xác suất default từ student status được trình bày trong Bảng 4.2. Coefficient gắn với dummy variable là dương và *p*-value tương ứng có statistical significance. Điều này cho thấy student có xu hướng mang xác suất default cao hơn non-student:

$$\begin{aligned} \Pr(\text{default} = \text{Yes} \mid \text{student} = \text{Yes}) &= \frac{e^{-3.5041 + 0.4049 \times 1}}{1 + e^{-3.5041 + 0.4049 \times 1}} = 0.0431, \\ \Pr(\text{default} = \text{Yes} \mid \text{student} = \text{No}) &= \frac{e^{-3.5041 + 0.4049 \times 0}}{1 + e^{-3.5041 + 0.4049 \times 0}} = 0.0292. \end{aligned}$$

### 4.3.4 Multiple Logistic Regression

Bây giờ ta xét bài toán dự đoán binary response bằng nhiều predictor. Tương tự phép mở rộng từ simple sang multiple linear regression trong Chương 3, ta có thể tổng quát hóa (4.4) như sau:

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p, \quad (4.6)$$

**BẢNG 4.3.** Với dữ liệu *Default*, các coefficient estimate của logistic regression model dự đoán xác suất default bằng *balance*, *income* và *student status*. *Student status* được mã hóa thành dummy variable *student[Yes]*, nhận giá trị 1 với *student* và 0 với *non-student*. Khi fit model này, *income* được đo theo nghìn đô la.

|              | Coefficient | Std. error | z-statistic | p-value  |
|--------------|-------------|------------|-------------|----------|
| Intercept    | -10.8690    | 0.4923     | -22.08      | < 0.0001 |
| balance      | 0.0057      | 0.0002     | 24.74       | < 0.0001 |
| income       | 0.0030      | 0.0082     | 0.37        | 0.7115   |
| student[Yes] | -0.6468     | 0.2362     | -2.74       | 0.0062   |

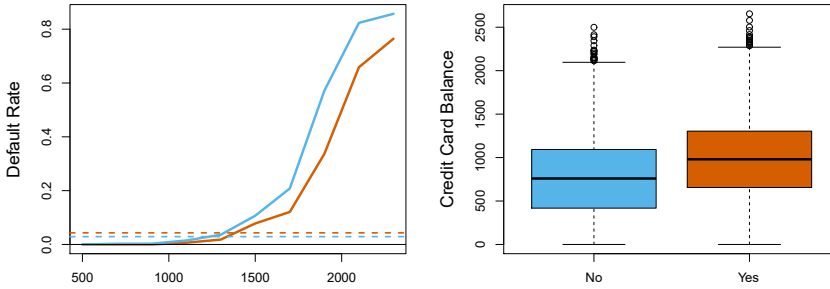
trong đó  $X = (X_1, \dots, X_p)$  gồm  $p$  predictor. Phương trình 4.6 có thể được viết lại thành

$$p(X) = \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}. \quad (4.7)$$

Cũng như trong Mục 4.3.2, ta dùng maximum likelihood để ước lượng  $\beta_0, \beta_1, \dots, \beta_p$ .

Bảng 4.3 trình bày coefficient estimate của logistic regression model dùng *balance*, *income* (theo nghìn đô la) và *student status* để dự đoán xác suất default. Ở đây có một kết quả đáng ngạc nhiên. Các  $p$ -value gắn với *balance* và dummy variable của *student status* đều rất nhỏ, cho thấy mỗi biến đều associated với xác suất default. Tuy nhiên, coefficient của dummy variable lại âm, cho biết *student* ít có khả năng default hơn *non-student*. Trái lại, coefficient của dummy variable trong Bảng 4.2 là dương. Làm sao *student status* có thể associated với sự gia tăng xác suất default trong Bảng 4.2, nhưng lại associated với sự suy giảm xác suất default trong Bảng 4.3? Ở bên trái của Hình 4.3 minh họa bằng đồ thị nghịch lý có vẻ tồn tại này. Các đường liền màu cam và xanh lam lần lượt biểu diễn average default rate của *student* và *non-student* theo credit card balance. Coefficient âm của *student* trong multiple logistic regression cho biết với một giá trị *balance* và *income* cố định, *student* ít có khả năng default hơn *non-student*. Thật vậy, từ ô bên trái của Hình 4.3, ta thấy default rate của *student* không vượt quá default rate của *non-student* ở mọi giá trị *balance*. Nhưng các đường đứt đoạn nằm ngang gần đáy đồ thị, biểu diễn default rate của *student* và *non-student* được lấy trung bình trên mọi giá trị *balance* và *income*, lại gợi ý tác động ngược lại: overall *student* default rate cao hơn *non-student* default rate. Do đó, coefficient của *student* trong single-variable logistic regression output ở Bảng 4.2 là dương.

Ở bên phải của Hình 4.3 giải thích sự khác biệt này. Hai biến *student* và *balance* correlated với nhau. *Student* có xu hướng mang mức nợ cao hơn, và điều này lại associated với xác suất default cao hơn. Nói cách khác, *student* có nhiều khả năng mang credit card balance lớn; như ta biết từ ô bên trái của Hình 4.3, *balance* lớn thường associated với default rate cao. Vì vậy, dù một *student* cụ thể có credit card balance nhất định thường mang xác suất default thấp hơn một *non-student* có cùng credit card balance, việc *student* nhìn chung có credit card balance cao hơn đồng nghĩa với việc xét tổng thể, *student* có xu hướng default với rate cao hơn *non-student*. Đây là khác biệt quan trọng đối với một công ty thẻ tín dụng đang xác định nên



**HÌNH 4.3.** *Confounding trong dữ liệu **Default**. Trái: Default rate của student (màu cam) và non-student (màu xanh lam). Các đường liền biểu diễn default rate theo **balance**, còn các đường đứt đoạn nằm ngang biểu diễn overall default rate. Phải: Boxplot của **balance** cho student (màu cam) và non-student (màu xanh lam).*

cung cấp tín dụng cho ai. Nếu không có thông tin về credit card balance, một student rủi ro hơn một non-student. Tuy nhiên, student đó lại ít rủi ro hơn một non-student có cùng credit card balance!

Ví dụ đơn giản này minh họa những nguy cơ và điểm tinh tế khi thực hiện regression chỉ với một predictor trong khi các predictor khác cũng có thể liên quan. Cũng như trong bối cảnh linear regression, kết quả thu được khi dùng một predictor có thể rất khác kết quả khi dùng nhiều predictor, đặc biệt khi các predictor correlated với nhau. Nhìn chung, hiện tượng trong Hình 4.3 được gọi là *confounding*.

confounding

Thay các regression coefficient estimate từ Bảng 4.3 vào (4.7), ta có thể đưa ra prediction. Chẳng hạn, một student có credit card balance 1.500 đô la và income 40.000 đô la có xác suất default ước lượng là

$$\hat{p}(X) = \frac{e^{-10.869+0.00574 \times 1,500+0.003 \times 40-0.6468 \times 1}}{1 + e^{-10.869+0.00574 \times 1,500+0.003 \times 40-0.6468 \times 1}} = 0.058. \quad (4.8)$$

Một non-student có cùng balance và income có xác suất default ước lượng là

$$\hat{p}(X) = \frac{e^{-10.869+0.00574 \times 1,500+0.003 \times 40-0.6468 \times 0}}{1 + e^{-10.869+0.00574 \times 1,500+0.003 \times 40-0.6468 \times 0}} = 0.105. \quad (4.9)$$

(Ở đây, ta nhân income coefficient estimate trong Bảng 4.3 với 40 thay vì 40.000, vì trong bảng đó model được fit với income đo theo đơn vị 1.000 đô la.)

#### 4.3.5 Multinomial Logistic Regression

Đôi khi ta muốn phân loại một response variable có hơn hai class. Chẳng hạn, trong Mục 4.2, ta có ba category tình trạng bệnh lý tại phòng cấp cứu: stroke, drug overdose và epileptic seizure. Tuy nhiên, cách tiếp cận logistic regression đã trình bày trong mục này chỉ cho phép response variable có  $K = 2$  class.

Hóa ra có thể mở rộng cách tiếp cận two-class logistic regression sang bối cảnh  $K > 2$  class. Phần mở rộng này đôi khi được gọi là *multinomial*

*logistic regression*. Để thực hiện, trước hết ta chọn một class làm baseline; không mất tính tổng quát, ta chọn class thứ  $K$  cho vai trò này. Sau đó, ta thay model (4.7) bằng model

multinomial  
logistic  
regression

$$\Pr(Y = k \mid X = x) = \frac{e^{\beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p}}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}} \quad (4.10)$$

với  $k = 1, \dots, K-1$ , và

$$\Pr(Y = K \mid X = x) = \frac{1}{1 + \sum_{l=1}^{K-1} e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}}. \quad (4.11)$$

Không khó để chứng minh rằng với  $k = 1, \dots, K-1$ ,

$$\log\left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)}\right) = \beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p. \quad (4.12)$$

Lưu ý rằng (4.12) khá giống (4.6). Phương trình 4.12 cho thấy một lần nữa, log odds giữa bất kỳ cặp class nào cũng tuyến tính theo các feature.

Hóa ra trong các Phương trình 4.10–4.12, quyết định dùng class thứ  $K$  làm baseline không quan trọng. Chẳng hạn, khi phân loại các lượt đến phòng cấp cứu thành stroke, drug overdose và epileptic seizure, giả sử ta fit hai multinomial logistic regression model: một model dùng stroke làm baseline, model kia dùng drug overdose làm baseline. Coefficient estimate sẽ khác nhau giữa hai fitted model do lựa chọn baseline khác nhau, nhưng fitted value (prediction), log odds giữa bất kỳ cặp class nào và các model output quan trọng khác vẫn giống nhau.

Dẫu vậy, phải cẩn thận khi diễn giải coefficient trong multinomial logistic regression model vì diễn giải đó gắn với lựa chọn baseline. Chẳng hạn, nếu đặt epileptic seizure làm baseline, ta có thể diễn giải  $\beta_{\text{stroke},0}$  là log odds của stroke so với epileptic seizure khi  $x_1 = \dots = x_p = 0$ . Hơn nữa,  $X_j$  tăng một đơn vị đi kèm log odds của stroke so với epileptic seizure tăng  $\beta_{\text{stroke},j}$ . Nói cách khác, nếu  $X_j$  tăng một đơn vị thì

$$\frac{\Pr(Y = \text{stroke} \mid X = x)}{\Pr(Y = \text{epileptic seizure} \mid X = x)}$$

tăng theo hệ số  $e^{\beta_{\text{stroke},j}}$ .

Bây giờ chúng tôi trình bày ngắn gọn một cách mã hóa khác cho multinomial logistic regression, gọi là *softmax coding*. Softmax coding tương đương cách mã hóa vừa mô tả theo nghĩa fitted value, log odds giữa bất kỳ cặp class nào và các model output quan trọng khác vẫn giống nhau, bất kể cách mã hóa. Tuy nhiên, softmax coding được dùng rộng rãi trong một số lĩnh vực của machine learning literature (và sẽ xuất hiện lại trong Chương 10), nên ta cần biết đến nó. Trong softmax coding, thay vì chọn một baseline class, ta xử lý đối xứng cả  $K$  class và giả định rằng với  $k = 1, \dots, K$ ,

softmax  
coding

$$\Pr(Y = k \mid X = x) = \frac{e^{\beta_{k0} + \beta_{k1}x_1 + \dots + \beta_{kp}x_p}}{\sum_{l=1}^K e^{\beta_{l0} + \beta_{l1}x_1 + \dots + \beta_{lp}x_p}}. \quad (4.13)$$



Vì vậy, thay vì ước lượng coefficient cho  $K - 1$  class, ta thực sự ước lượng coefficient cho cả  $K$  class. Từ (4.13), không khó để thấy log odds ratio giữa class thứ  $k$  và class thứ  $k'$  bằng

$$\begin{aligned} \log \left( \frac{\Pr(Y = k \mid X = x)}{\Pr(Y = k' \mid X = x)} \right) &= (\beta_{k0} - \beta_{k'0}) + (\beta_{k1} - \beta_{k'1})x_1 + \cdots \\ &\quad + (\beta_{kp} - \beta_{k'p})x_p. \end{aligned} \quad (4.14)$$

## 4.4 Generative Models for Classification

Logistic regression trực tiếp modeling  $\Pr(Y = k \mid X = x)$  bằng logistic function, được cho bởi (4.7) trong trường hợp có hai response class. Theo thuật ngữ thống kê, ta modeling conditional distribution của response  $Y$  khi biết predictor  $X$ . Bây giờ ta xét một cách tiếp cận khác, gián tiếp hơn, để ước lượng các xác suất này. Trong cách tiếp cận mới, ta modeling distribution của predictor  $X$  riêng biệt trong từng response class (tức với từng giá trị của  $Y$ ). Sau đó, ta dùng Bayes' theorem để đảo chiều chúng thành các ước lượng của  $\Pr(Y = k \mid X = x)$ . Khi distribution của  $X$  trong mỗi class được giả định là normal, model thu được có dạng rất giống logistic regression.

Tại sao cần một phương pháp khác khi đã có logistic regression? Có một số lý do:

- Khi hai class có sự phân tách đáng kể, parameter estimate của logistic regression model bắt đầu trở nên không ổn định. Các phương pháp được xét trong mục này không gặp vấn đề đó.
- Nếu distribution của predictor  $X$  trong mỗi class xấp xỉ normal và sample size nhỏ, các cách tiếp cận trong mục này có thể chính xác hơn logistic regression.
- Các phương pháp trong mục này có thể mở rộng tự nhiên sang trường hợp có hơn hai response class. (Khi có hơn hai response class, ta cũng có thể dùng multinomial logistic regression trong Mục 4.3.5.)

Giả sử ta muốn phân loại một observation vào một trong  $K$  class, với  $K \geq 2$ . Nói cách khác, qualitative response variable  $Y$  có thể nhận  $K$  giá trị riêng biệt và không có thứ tự. Gọi  $\pi_k$  là xác suất tổng thể, hay *prior probability*, để một observation được chọn ngẫu nhiên đến từ class thứ  $k$ . Gọi  $f_k(X) \equiv \Pr(X \mid Y = k)$ <sup>1</sup> là *density function* của  $X$  đối với một observation đến từ class thứ  $k$ . Nói cách khác,  $f_k(x)$  tương đối lớn nếu một observation trong class thứ  $k$  có khả năng cao mang  $X \approx x$ , và  $f_k(x)$  nhỏ nếu điều đó rất khó xảy ra. Khi đó, Bayes' theorem cho biết

$$\Pr(Y = k \mid X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^K \pi_l f_l(x)}. \quad (4.15)$$

Theo ký hiệu trước đây, ta viết tắt  $p_k(x) = \Pr(Y = k \mid X = x)$ ; đây là *posterior probability* để observation  $X = x$  thuộc class thứ  $k$ . Tức là xác suất observation thuộc class thứ  $k$  khi biết predictor value của observation đó.

Phương trình 4.15 gợi ý rằng thay vì trực tiếp tính posterior probability  $p_k(x)$  như trong Mục 4.3.1, ta chỉ cần thay các ước lượng của  $\pi_k$  và  $f_k(x)$  vào (4.15). Nhìn chung, nếu có random sample từ population thì việc ước lượng  $\pi_k$  rất dễ: ta chỉ cần tính tỷ lệ training observation thuộc class thứ  $k$ . Tuy nhiên, ước lượng density function  $f_k(x)$  khó hơn nhiều. Như sẽ thấy, để ước lượng  $f_k(x)$ , ta thường phải đưa ra một số simplifying assumption.

<sup>1</sup>Về mặt kỹ thuật, định nghĩa này chỉ đúng nếu  $X$  là qualitative random variable. Nếu  $X$  là quantitative,  $f_k(x) dx$  tương ứng với xác suất  $X$  rơi vào một vùng nhỏ  $dx$  quanh  $x$ .

prior  
probability  
density  
function

posterior  
probability

Ta biết từ Chương 2 rằng Bayes classifier, tức classifier gán observation  $x$  vào class có  $p_k(x)$  lớn nhất, có error rate thấp nhất có thể trong tất cả classifier. (Tất nhiên, điều này chỉ đúng nếu mọi term trong (4.15) đều được chỉ định đúng.) Vì vậy, nếu tìm được cách ước lượng  $f_k(x)$ , ta có thể thay nó vào (4.15) để xấp xỉ Bayes classifier.

Trong các mục tiếp theo, chúng tôi thảo luận ba classifier dùng những ước lượng khác nhau của  $f_k(x)$  trong (4.15) để xấp xỉ Bayes classifier: linear discriminant analysis, quadratic discriminant analysis và naive Bayes.

#### 4.4.1 Linear Discriminant Analysis for $p = 1$

Trước mắt, giả sử  $p = 1$ , tức ta chỉ có một predictor. Ta muốn thu được một ước lượng của  $f_k(x)$  để thay vào (4.15) nhằm ước lượng  $p_k(x)$ . Sau đó, ta sẽ phân loại observation vào class có  $p_k(x)$  lớn nhất. Để ước lượng  $f_k(x)$ , trước hết ta đưa ra một số giả định về dạng của nó.

Cụ thể, ta giả định  $f_k(x)$  là *normal* hay *Gaussian*. Trong bối cảnh một chiều, normal density có dạng normal  
Gaussian

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x - \mu_k)^2\right), \quad (4.16)$$

trong đó  $\mu_k$  và  $\sigma_k^2$  là mean parameter và variance parameter của class thứ  $k$ . Trước mắt, ta giả định thêm  $\sigma_1^2 = \dots = \sigma_K^2$ , tức cả  $K$  class dùng chung một variance term, được ký hiệu đơn giản là  $\sigma^2$ . Thay (4.16) vào (4.15), ta có

$$p_k(x) = \frac{\pi_k \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2\sigma^2}(x - \mu_k)^2\right]}{\sum_{l=1}^K \pi_l \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2\sigma^2}(x - \mu_l)^2\right]}. \quad (4.17)$$

(Lưu ý trong (4.17),  $\pi_k$  biểu diễn prior probability để một observation thuộc class thứ  $k$ , không nên nhầm với hằng số toán học  $\pi \approx 3.14159$ .) Bayes classifier<sup>2</sup> gán observation  $X = x$  vào class có (4.17) lớn nhất.

Lấy log của (4.17) rồi sắp xếp lại các term, không khó để chứng minh<sup>3</sup> rằng điều này tương đương với việc gán observation vào class có

$$\delta_k(x) = x \cdot \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k) \quad (4.18)$$

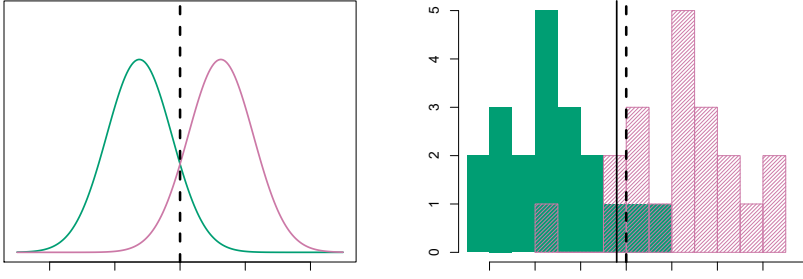
lớn nhất. Chẳng hạn, nếu  $K = 2$  và  $\pi_1 = \pi_2$ , Bayes classifier gán observation vào class 1 nếu  $2x(\mu_1 - \mu_2) > \mu_1^2 - \mu_2^2$ , và vào class 2 trong trường hợp còn lại. Bayes decision boundary là điểm tại đó  $\delta_1(x) = \delta_2(x)$ ; có thể chứng minh điểm này là

$$x = \frac{\mu_1^2 - \mu_2^2}{2(\mu_1 - \mu_2)} = \frac{\mu_1 + \mu_2}{2}. \quad (4.19)$$

Một ví dụ được biểu diễn trong ô trái của Hình 4.4. Hai normal density function  $f_1(x)$  và  $f_2(x)$  đại diện cho hai class riêng biệt. Mean và variance

<sup>2</sup>Nhớ rằng Bayes classifier gán observation vào class có  $p_k(x)$  lớn nhất. Điều này khác Bayes' theorem trong (4.15), vốn cho phép biến đổi conditional distribution.

<sup>3</sup>Xem Bài tập 2 ở cuối chương.



**HÌNH 4.4.** Trái: Hai one-dimensional normal density function. Đường đứt đoạn thẳng đứng biểu diễn Bayes decision boundary. Phải: 20 observation được rút từ mỗi class và biểu diễn bằng histogram. Bayes decision boundary vẫn là đường đứt đoạn thẳng đứng. Đường liền thẳng đứng biểu diễn LDA decision boundary được ước lượng từ training data.

parameter của hai density function là  $\mu_1 = -1.25$ ,  $\mu_2 = 1.25$  và  $\sigma_1^2 = \sigma_2^2 = 1$ . Hai density chồng lấn, nên khi biết  $X = x$ , vẫn có sự không chắc chắn về class của observation. Nếu giả định observation có khả năng như nhau đến từ mỗi class, tức  $\pi_1 = \pi_2 = 0.5$ , thì từ (4.19), ta thấy Bayes classifier gán observation vào class 1 nếu  $x < 0$ , và class 2 trong trường hợp còn lại. Lưu ý ở đây ta tính được Bayes classifier vì biết  $X$  được rút từ Gaussian distribution trong mỗi class và biết mọi parameter liên quan. Trong tình huống thực tế, ta không thể tính Bayes classifier.

Trong thực tế, ngay cả khi khá chắc chắn về giả định  $X$  được rút từ Gaussian distribution trong mỗi class, để áp dụng Bayes classifier ta vẫn phải ước lượng các parameter  $\mu_1, \dots, \mu_K$ ,  $\pi_1, \dots, \pi_K$  và  $\sigma^2$ . Phương pháp *linear discriminant analysis* (LDA) xấp xỉ Bayes classifier bằng cách thay các ước lượng của  $\pi_k$ ,  $\mu_k$  và  $\sigma^2$  vào (4.18). Cụ thể, các ước lượng sau được dùng:

linear  
discriminant  
analysis

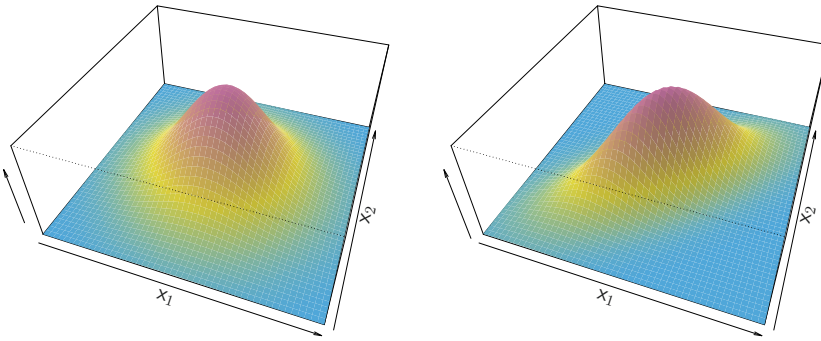
$$\begin{aligned}\hat{\mu}_k &= \frac{1}{n_k} \sum_{i:y_i=k} x_i, \\ \hat{\sigma}^2 &= \frac{1}{n-K} \sum_{k=1}^K \sum_{i:y_i=k} (x_i - \hat{\mu}_k)^2.\end{aligned}\quad (4.20)$$

Ở đây,  $n$  là tổng số training observation và  $n_k$  là số training observation trong class thứ  $k$ . Ước lượng  $\mu_k$  đơn giản là trung bình của mọi training observation từ class thứ  $k$ , còn  $\hat{\sigma}^2$  có thể được xem là weighted average của sample variance trong từng class. Đôi khi ta biết các class membership probability  $\pi_1, \dots, \pi_K$  và có thể dùng trực tiếp. Khi không có thông tin bổ sung, LDA ước lượng  $\pi_k$  bằng tỷ lệ training observation thuộc class thứ  $k$ :

$$\hat{\pi}_k = n_k/n. \quad (4.21)$$

LDA classifier thay các ước lượng trong (4.20) và (4.21) vào (4.18), rồi gán observation  $X = x$  vào class có

$$\hat{\delta}_k(x) = x \cdot \frac{\hat{\mu}_k}{\hat{\sigma}^2} - \frac{\hat{\mu}_k^2}{2\hat{\sigma}^2} + \log(\hat{\pi}_k) \quad (4.22)$$



**HÌNH 4.5.** Hai multivariate Gaussian density function với  $p = 2$ . Trái: Hai predictor không correlated. Phải: Hai variable có correlation bằng 0.7.

lớn nhất. Từ *linear* trong tên classifier bắt nguồn từ việc các *discriminant function*  $\hat{\delta}_k(x)$  trong (4.22) là linear function của  $x$  chứ không phải một hàm phức tạp hơn.

discriminant  
function

Ô bên phải của Hình 4.4 biểu diễn histogram của random sample gồm 20 observation từ mỗi class. Để triển khai LDA, trước hết ta ước lượng  $\pi_k$ ,  $\mu_k$  và  $\sigma^2$  bằng (4.20) và (4.21). Sau đó, ta tính decision boundary, biểu diễn bằng đường liền màu đen, thu được từ việc gán observation vào class có (4.22) lớn nhất. Mọi điểm bên trái đường này được gán vào class xanh lục, còn điểm bên phải được gán vào class tím. Vì  $n_1 = n_2 = 20$ , ta có  $\hat{\pi}_1 = \hat{\pi}_2$ . Do đó, decision boundary tương ứng với midpoint giữa sample mean của hai class,  $(\hat{\mu}_1 + \hat{\mu}_2)/2$ . Hình cho thấy LDA decision boundary nằm hơi lệch trái so với Bayes decision boundary tối ưu, vốn bằng  $(\mu_1 + \mu_2)/2 = 0$ . LDA classifier hoạt động tốt đến đâu trên dữ liệu này? Vì đây là simulated data, ta có thể sinh nhiều test observation để tính Bayes error rate và LDA test error rate. Hai giá trị lần lượt là 10.6% và 11.1%. Nói cách khác, error rate của LDA classifier chỉ cao hơn error rate nhỏ nhất có thể 0.5%! Điều này cho thấy LDA hoạt động khá tốt trên dataset này.

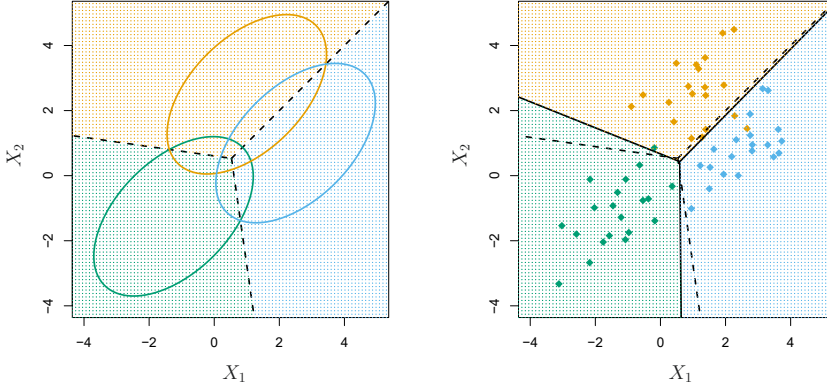
Nhắc lại, LDA classifier xuất phát từ giả định observation trong mỗi class đến từ normal distribution với class-specific mean và common variance  $\sigma^2$ , rồi thay các estimate của parameter này vào Bayes classifier. Trong Mục 4.4.3, ta sẽ xét một tập giả định ít nghiêm ngặt hơn bằng cách cho phép observation trong class thứ  $k$  có class-specific variance  $\sigma_k^2$ .

#### 4.4.2 Linear Discriminant Analysis for $p > 1$

Bây giờ ta mở rộng LDA classifier sang trường hợp có nhiều predictor. Ta giả định  $X = (X_1, X_2, \dots, X_p)$  được rút từ *multivariate Gaussian* (hay multivariate normal) distribution, với class-specific mean vector và common covariance matrix. Trước hết, ta ôn lại ngắn gọn distribution này.

multivariate  
Gaussian

Multivariate Gaussian distribution giả định mỗi predictor riêng lẻ tuân theo one-dimensional normal distribution như trong (4.16), cùng một mức



**HÌNH 4.6.** Ví dụ với ba class. Observation trong mỗi class được rút từ multivariate Gaussian distribution với  $p = 2$ , class-specific mean vector và common covariance matrix. Trái: Các ellipse chứa 95% probability của từng class. Đường đứt đoạn là Bayes decision boundary. Phải: 20 observation được sinh từ mỗi class; LDA decision boundary tương ứng là các đường liền màu đen. Bayes decision boundary vẫn được biểu diễn bằng đường đứt đoạn.

correlation giữa từng cặp predictor. Hai ví dụ về multivariate Gaussian distribution với  $p = 2$  được biểu diễn trong Hình 4.5. Độ cao của surface tại một điểm biểu diễn xác suất cả  $X_1$  và  $X_2$  cùng rơi vào một vùng nhỏ quanh điểm đó. Trong mỗi ô, nếu cắt surface dọc trục  $X_1$  hoặc  $X_2$ , cross-section thu được có dạng one-dimensional normal distribution. Ô trái của Hình 4.5 minh họa trường hợp  $\text{Var}(X_1) = \text{Var}(X_2)$  và  $\text{Cor}(X_1, X_2) = 0$ ; surface này có dạng chuông đặc trưng. Tuy nhiên, dạng chuông bị biến dạng nếu các predictor correlated hoặc có variance không bằng nhau, như ô phải của Hình 4.5. Khi đó, đáy chuông có dạng ellipse thay vì hình tròn. Để ký hiệu random variable  $p$  chiều  $X$  có multivariate Gaussian distribution, ta viết  $X \sim N(\mu, \Sigma)$ . Ở đây  $E(X) = \mu$  là mean của  $X$  (một vector có  $p$  component), còn  $\text{Cov}(X) = \Sigma$  là covariance matrix  $p \times p$  của  $X$ . Về hình thức, multivariate Gaussian density được định nghĩa là

$$f(x) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp \left[ -\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right]. \quad (4.23)$$

Trong trường hợp  $p > 1$  predictor, LDA classifier giả định observation trong class thứ  $k$  được rút từ multivariate Gaussian distribution  $N(\mu_k, \Sigma)$ , trong đó  $\mu_k$  là class-specific mean vector và  $\Sigma$  là covariance matrix chung cho cả  $K$  class. Thay density function  $f_k(X = x)$  của class thứ  $k$  vào (4.15) và thực hiện một ít đại số cho thấy Bayes classifier gán observation  $X = x$  vào class có

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k \quad (4.24)$$

lớn nhất. Đây là phiên bản vector/matrix của (4.18).

Một ví dụ được biểu diễn trong ô trái của Hình 4.6. Ba Gaussian class có kích thước bằng nhau được biểu diễn với class-specific mean vector và common covariance matrix. Ba ellipse đại diện cho các vùng chứa 95%

**BẢNG 4.4.** *Confusion matrix so sánh LDA prediction với trạng thái `default` thực của 10.000 training observation trong dataset `Default`. Các element trên diagonal là những cá nhân được dự đoán đúng trạng thái default; các element ngoài diagonal là những cá nhân bị misclassified. LDA dự đoán sai 23 người không default và 252 người default.*

|                          |       | True default status |     |       |
|--------------------------|-------|---------------------|-----|-------|
|                          |       | No                  | Yes | Total |
| Predicted default status | No    | 9644                | 252 | 9896  |
|                          | Yes   | 23                  | 81  | 104   |
|                          | Total | 9667                | 333 | 10000 |

probability của từng class. Các đường đứt đoạn là Bayes decision boundary, tức tập giá trị  $x$  sao cho  $\delta_k(x) = \delta_l(x)$ :

$$x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k = x^T \Sigma^{-1} \mu_l - \frac{1}{2} \mu_l^T \Sigma^{-1} \mu_l, \quad (4.25)$$

với  $k \neq l$ . (Term  $\log \pi_k$  trong (4.24) biến mất vì ba class có cùng số training observation, tức  $\pi_k$  như nhau với mỗi class.) Có ba đường Bayes decision boundary vì có ba cặp class trong ba class. Một boundary tách class 1 khỏi class 2, một boundary tách class 1 khỏi class 3 và một boundary tách class 2 khỏi class 3. Ba boundary này chia predictor space thành ba vùng. Bayes classifier phân loại observation theo vùng chứa nó.

Một lần nữa, ta cần ước lượng các parameter chưa biết  $\mu_1, \dots, \mu_K$ ,  $\pi_1, \dots, \pi_K$  và  $\Sigma$ ; các công thức tương tự trường hợp một chiều trong (4.20). Để gán observation mới  $X = x$ , LDA thay các ước lượng này vào (4.24) để thu được  $\hat{\delta}_k(x)$  và phân loại vào class có  $\hat{\delta}_k(x)$  lớn nhất. Lưu ý  $\delta_k(x)$  trong (4.24) là linear function của  $x$ ; tức LDA decision rule chỉ phụ thuộc vào  $x$  thông qua linear combination của các element. Như đã thảo luận, đây là lý do có từ *linear* trong LDA.

Ở bên phải của Hình 4.6 biểu diễn 20 observation được rút từ mỗi class, cùng các LDA decision boundary thu được bằng đường liền màu đen. Nhìn chung, LDA decision boundary khá gần Bayes decision boundary, vẫn được biểu diễn bằng đường đứt đoạn. Test error rate của Bayes classifier và LDA classifier lần lượt là 0.0746 và 0.0770. Điều này cho thấy LDA hoạt động tốt trên dữ liệu này.

Ta có thể thực hiện LDA trên dữ liệu `Default` để dự đoán một cá nhân có default hay không dựa trên credit card balance và student status.<sup>4</sup> LDA model được fit trên 10.000 training sample cho training error rate bằng 2.75%. Con số này có vẻ thấp, nhưng cần lưu ý hai điểm:

- Training error rate thường thấp hơn test error rate, là đại lượng thực sự được quan tâm. Nói cách khác, classifier này có thể hoạt động kém hơn khi dùng để dự đoán một tập cá nhân mới có default hay không, vì ta đã điều chỉnh parameter của model để hoạt động tốt trên training data.

<sup>4</sup>Người đọc cẩn thận sẽ nhận thấy student status là qualitative, nên giả định normality của LDA rõ ràng bị vi phạm trong ví dụ này! Tuy nhiên, LDA thường bền vững đáng kể trước model violation, như ví dụ cho thấy. Naive Bayes trong Mục 4.4.4 là một lựa chọn thay thế không giả định predictor có normal distribution.

Tỷ lệ parameter  $p$  trên sample  $n$  càng cao, ta càng kỳ vọng *overfitting* đóng vai trò lớn. Với dữ liệu này, đây không phải vấn đề đáng ngại vì  $p = 2$  và  $n = 10,000$ .

- Vì chỉ 3.33% cá nhân trong training sample default, một classifier đơn giản nhưng vô dụng luôn dự đoán một cá nhân không default, bất kể credit card balance và student status, sẽ có error rate 3.33%. Nói cách khác, *null classifier* tầm thường đạt error rate chỉ cao hơn LDA training set error rate đôi chút.

Trong thực tế, binary classifier như thế này có thể mắc hai loại error: gán nhầm một cá nhân default vào category no default, hoặc gán nhầm một cá nhân không default vào category default. Ta thường muốn xác định classifier đang mắc loại error nào. *Confusion matrix* trong Bảng 4.4 là cách thuận tiện để biểu diễn thông tin này. Matrix cho thấy LDA dự đoán tổng cộng 104 người sẽ default; trong số đó, 81 người thực sự default và 23 người không. Vì vậy, chỉ 23 trong số 9.667 cá nhân không default bị gán nhầm sai. Error rate này có vẻ rất thấp! Tuy nhiên, trong 333 cá nhân default, LDA bỏ sót 252 người, tức 75.7%. Vì thế, dù overall error rate thấp, error rate trong nhóm default rất cao. Từ góc nhìn của công ty thẻ tín dụng muốn nhận diện cá nhân rủi ro cao, error rate  $252/333 = 75.7\%$  trong nhóm default có thể hoàn toàn không chấp nhận được.

Class-specific performance cũng quan trọng trong y học và sinh học, nơi các thuật ngữ *sensitivity* và *specificity* đặc trưng hóa performance của classifier hay screening test. Ở đây, sensitivity là tỷ lệ true defaulter được nhận diện, bằng 24.3%. Specificity là tỷ lệ non-defaulter được nhận diện đúng, bằng  $1 - 23/9667 = 99.8\%$ .

Tại sao LDA phân loại những khách hàng default kém đến vậy, tức tại sao sensitivity thấp? Như đã thấy, LDA cổ xấp xỉ Bayes classifier, classifier có total error rate thấp nhất trong mọi classifier. Bayes classifier tạo ra tổng số observation bị misclassified nhỏ nhất có thể, bất kể error đến từ class nào. Một số misclassification phát sinh do gán nhầm khách hàng không default vào class default, số khác do gán nhầm khách hàng default vào class non-default. Trái lại, công ty thẻ tín dụng có thể đặc biệt muốn tránh phân loại sai một người sẽ default; còn phân loại sai một người không default, dù vẫn nên tránh, ít nghiêm trọng hơn. Bây giờ ta sẽ thấy có thể sửa đổi LDA để tạo classifier phù hợp hơn với nhu cầu của công ty.

Bayes classifier gán observation vào class có posterior probability  $p_k(X)$  lớn nhất. Trong trường hợp two-class, điều này tương đương gán observation vào class default nếu

$$\Pr(\text{default} = \text{Yes} \mid X = x) > 0.5. \quad (4.26)$$

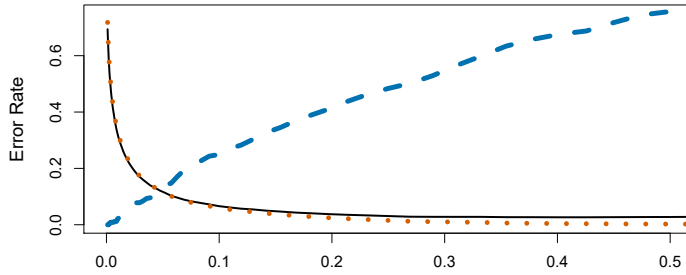
Vì vậy, Bayes classifier, và theo đó là LDA, dùng threshold 50% cho posterior probability của default để gán observation vào class default. Tuy nhiên, nếu quan tâm đến việc dự đoán sai trạng thái của những cá nhân sẽ default, ta có thể hạ threshold. Chẳng hạn, có thể gán mọi khách hàng có posterior probability của default lớn hơn 20% vào class default. Tức thay vì yêu cầu (4.26), ta dùng

$$\Pr(\text{default} = \text{Yes} \mid X = x) > 0.2. \quad (4.27)$$



**BẢNG 4.5.** Confusion matrix so sánh LDA prediction với trạng thái *default* thực của 10.000 training observation trong dataset *Default*, dùng threshold đã sửa đổi: dự đoán default cho mọi cá nhân có posterior default probability vượt 20%.

|                          |       | True default status |     |       |
|--------------------------|-------|---------------------|-----|-------|
|                          |       | No                  | Yes | Total |
| Predicted default status | No    | 9432                | 138 | 9570  |
|                          | Yes   | 235                 | 195 | 430   |
|                          | Total | 9667                | 333 | 10000 |



**HÌNH 4.7.** Với dataset *Default*, các error rate được biểu diễn theo threshold của posterior probability dùng để thực hiện phép gán. Đường liền màu đen biểu diễn overall error rate. Đường đứt đoạn màu xanh lam là tỷ lệ khách hàng default bị phân loại sai; đường chấm màu cam là tỷ lệ error trong nhóm khách hàng không default.

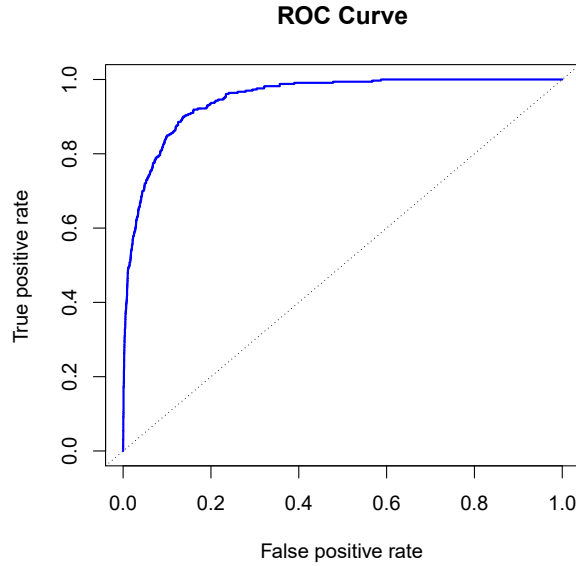
Các error rate thu được từ cách tiếp cận này được biểu diễn trong Bảng 4.5.

Bây giờ LDA dự đoán 430 cá nhân sẽ default. Trong 333 cá nhân default, LDA dự đoán đúng tất cả trừ 138 người, tức sai 41.4%. Đây là cải thiện rất lớn so với error rate 75.7% khi dùng threshold 50%. Tuy nhiên, sự cải thiện này có cái giá của nó: giờ đây 235 cá nhân không default bị phân loại sai. Vì vậy, overall error rate tăng nhẹ lên 3.73%. Nhưng một công ty thẻ tín dụng có thể xem mức tăng nhỏ của total error rate này là cái giá nhỏ để nhận diện chính xác hơn những người thực sự default.

Hình 4.7 minh họa trade-off khi thay đổi threshold của posterior probability của default. Nhiều error rate được biểu diễn theo threshold. Dùng threshold 0.5 như trong (4.26) sẽ giảm thiểu overall error rate, là đường liền màu đen. Điều này phù hợp kỳ vọng vì Bayes classifier dùng threshold 0.5 và có overall error rate thấp nhất. Nhưng khi dùng threshold 0.5, error rate trong nhóm default khá cao (đường đứt đoạn màu xanh lam). Khi giảm threshold, error rate trong nhóm default giảm đều, nhưng error rate trong nhóm không default tăng. Làm sao quyết định threshold nào tốt nhất? Quyết định này phải dựa trên domain knowledge, chẳng hạn thông tin chi tiết về chi phí gắn với default.

ROC curve là đồ thị phổ biến để đồng thời biểu diễn hai loại error trên mọi threshold có thể. Tên “ROC” có nguồn gốc lịch sử từ communications theory và là viết tắt của *receiver operating characteristics*. Hình 4.8 biểu diễn ROC curve của LDA classifier trên training data.

ROC curve



**HÌNH 4.8.** ROC curve của LDA classifier trên dữ liệu *Default*. Nó biểu diễn hai loại error khi thay đổi threshold của posterior probability của default; threshold thực tế không được thể hiện. True positive rate là sensitivity: tỷ lệ defaulter được nhận diện đúng với một threshold. False positive rate là  $1 - \text{specificity}$ : tỷ lệ non-defaulter bị phân loại sai thành defaulter với cùng threshold. ROC curve lý tưởng áp sát góc trên bên trái, biểu thị true positive rate cao và false positive rate thấp. Đường chấm biểu diễn classifier “không có thông tin”, là điều ta kỳ vọng nếu student status và credit card balance không associated với xác suất default.

Overall performance của classifier, được tóm tắt trên mọi threshold có thể, được cho bởi *area under the (ROC) curve* (AUC). ROC curve lý tưởng áp sát góc trên bên trái, nên AUC càng lớn thì classifier càng tốt. Với dữ liệu này, AUC bằng 0.95, gần mức tối đa 1.0 và được xem là rất tốt. Ta kỳ vọng một classifier không tốt hơn chance sẽ có AUC 0.5 (khi được đánh giá trên independent test set không dùng để train model). ROC curve hữu ích để so sánh các classifier khác nhau vì tính đến mọi threshold có thể. Hóa ra ROC curve của logistic regression model trong Mục 4.3.4 được fit trên dữ liệu này gần như không thể phân biệt với ROC curve của LDA model, nên không được biểu diễn ở đây.

area under  
the (ROC)  
curve

Như đã thấy, thay đổi classifier threshold làm thay đổi true positive rate và false positive rate. Chúng cũng được gọi là sensitivity và một trừ specificity của classifier. Vì có một hệ thuật ngữ gần như gây rối trong bối cảnh này, ta tóm tắt chúng. Bảng 4.6 biểu diễn các kết quả có thể khi áp dụng classifier (hay diagnostic test) cho population. Để kết nối với epidemiology literature, ta xem “+” là “disease” cần phát hiện và “−” là trạng thái “non-disease”. Để kết nối với classical hypothesis testing literature, ta xem “−” là null hypothesis và “+” là alternative (non-null) hypothesis. Trong dữ liệu *Default*, “+” chỉ cá nhân default và “−” chỉ người không default.

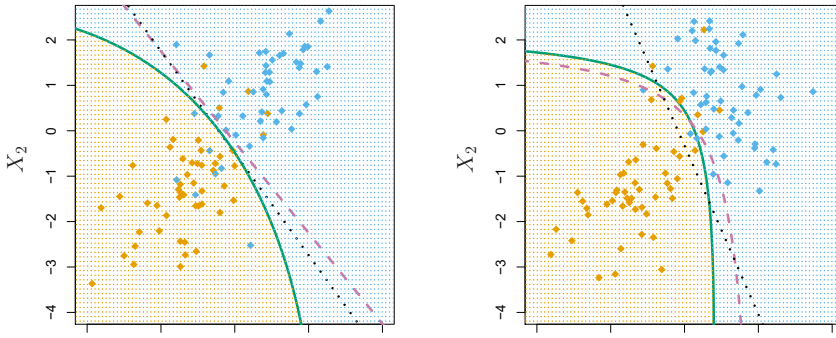
**BẢNG 4.6.** Các kết quả có thể khi áp dụng classifier hoặc diagnostic test cho population.

|                 |               | True class      |                 | Total |
|-----------------|---------------|-----------------|-----------------|-------|
|                 |               | – or Null       | + or Non-null   |       |
| Predicted class | – or Null     | True Neg. (TN)  | False Neg. (FN) | $N^*$ |
|                 | + or Non-null | False Pos. (FP) | True Pos. (TP)  | $P^*$ |
|                 | Total         | $N$             | $P$             |       |

**BẢNG 4.7.** Các performance measure quan trọng cho classification và diagnostic testing, được suy ra từ các đại lượng trong Bảng 4.6.

| Name             | Definition | Synonyms                                                |
|------------------|------------|---------------------------------------------------------|
| False Pos. rate  | $FP/N$     | Type I error, $1 - \text{Specificity}$                  |
| True Pos. rate   | $TP/P$     | $1 - \text{Type II error}$ , power, sensitivity, recall |
| Pos. Pred. value | $TP/P^*$   | Precision, $1 - \text{false discovery proportion}$      |
| Neg. Pred. value | $TN/N^*$   |                                                         |

Bảng 4.7 liệt kê nhiều performance measure phổ biến trong bối cảnh này. Denominator của false positive rate và true positive rate là population count thực trong từng class. Trái lại, denominator của positive predictive value và negative predictive value là tổng predicted count trong từng class.



**HÌNH 4.9.** Trái: Bayes decision boundary (đường đứt đoạn màu tím), LDA decision boundary (đường chấm màu đen) và QDA decision boundary (đường liền màu xanh lục) cho bài toán two-class với  $\Sigma_1 = \Sigma_2$ . Phần tô màu biểu diễn QDA decision rule. Vì Bayes decision boundary tuyến tính, LDA xấp xỉ nó chính xác hơn QDA. Phải: Các chỉ tiết giống ô trái, ngoại trừ  $\Sigma_1 \neq \Sigma_2$ . Vì Bayes decision boundary phi tuyến, QDA xấp xỉ nó chính xác hơn LDA.

#### 4.4.3 Quadratic Discriminant Analysis

Như đã thảo luận, LDA giả định observation trong mỗi class được rút từ multivariate Gaussian distribution với class-specific mean vector và một covariance matrix chung cho cả  $K$  class. Quadratic discriminant analysis (QDA) cung cấp một cách tiếp cận thay thế. Giống LDA, QDA classifier xuất phát từ giả định observation của mỗi class được rút từ Gaussian distribution, rồi thay các parameter estimate vào Bayes' theorem để thực hiện prediction. Tuy nhiên, khác với LDA, QDA giả định mỗi class có covariance matrix riêng. Tức nó giả định observation từ class thứ  $k$  có dạng  $X \sim N(\mu_k, \Sigma_k)$ , trong đó  $\Sigma_k$  là covariance matrix của class thứ  $k$ . Với giả định này, Bayes classifier gán observation  $X = x$  vào class có

Quadratic  
discriminant  
analysis

$$\begin{aligned} \delta_k(x) &= -\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) - \frac{1}{2} \log |\Sigma_k| + \log \pi_k \\ &= -\frac{1}{2} x^T \Sigma_k^{-1} x + x^T \Sigma_k^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma_k^{-1} \mu_k - \frac{1}{2} \log |\Sigma_k| + \log \pi_k \end{aligned} \quad (4.28)$$

lớn nhất. Vì vậy, QDA classifier thay các ước lượng của  $\Sigma_k$ ,  $\mu_k$  và  $\pi_k$  vào (4.28), rồi gán observation  $X = x$  vào class có đại lượng này lớn nhất. Khác với (4.24),  $x$  xuất hiện như một quadratic function trong (4.28). Đây là nguồn gốc tên gọi QDA.

Việc giả định  $K$  class có chung covariance matrix hay không quan trọng thế nào? Nói cách khác, khi nào nên chọn LDA thay cho QDA, hoặc ngược lại? Câu trả lời nằm ở bias-variance trade-off.

Khi có  $p$  predictor, việc ước lượng một covariance matrix đòi hỏi ước lượng  $p(p+1)/2$  parameter. QDA ước lượng một covariance matrix riêng cho mỗi class, tổng cộng  $Kp(p+1)/2$  parameter. Với 50 predictor, con số này là một bội số của 1.275, tức rất nhiều parameter. Trái lại, khi giả định  $K$  class dùng chung covariance matrix, LDA model trở thành tuyến tính theo  $x$ , nghĩa là có  $Kp$  linear coefficient cần ước lượng. Do đó, LDA là

classifier kém flexible hơn QDA rất nhiều và vì vậy có variance thấp hơn đáng kể. Điều này có thể giúp cải thiện prediction performance. Nhưng có một trade-off: nếu giả định các class dùng chung covariance matrix của LDA sai nghiêm trọng, LDA có thể chịu bias cao. Nói gần đúng, LDA thường là lựa chọn tốt hơn QDA khi có tương đối ít training observation và việc giảm variance là thiết yếu. Ngược lại, QDA được khuyến nghị nếu training set rất lớn, nên variance của classifier không phải mối lo chính, hoặc nếu giả định common covariance matrix cho  $K$  class rõ ràng không thể chấp nhận.

Hình 4.9 minh họa performance của LDA và QDA trong hai kịch bản. Ở ô trái, hai Gaussian class có cùng correlation 0.7 giữa  $X_1$  và  $X_2$ . Do đó, Bayes decision boundary tuyến tính và được LDA decision boundary xấp xỉ chính xác. QDA decision boundary kém hơn vì chịu variance cao hơn mà bias không giảm tương ứng. Trái lại, ô phải biểu diễn tình huống class màu cam có correlation 0.7 giữa hai variable, còn class màu xanh lam có correlation  $-0.7$ . Lúc này Bayes decision boundary là quadratic, nên QDA xấp xỉ boundary này chính xác hơn LDA.

#### 4.4.4 Naive Bayes

Trong các mục trước, ta dùng Bayes' theorem (4.15) để phát triển LDA và QDA classifier. Ở đây, ta dùng Bayes' theorem để tạo động lực cho *naive Bayes* classifier phổ biến.

naive Bayes

Nhớ rằng Bayes' theorem (4.15) cung cấp biểu thức cho posterior probability  $p_k(x) = \Pr(Y = k \mid X = x)$  theo  $\pi_1, \dots, \pi_K$  và  $f_1(x), \dots, f_K(x)$ . Để dùng (4.15) trong thực tế, ta cần các ước lượng của  $\pi_1, \dots, \pi_K$  và  $f_1(x), \dots, f_K(x)$ . Như đã thấy, việc ước lượng prior probability  $\pi_1, \dots, \pi_K$  thường khá trực tiếp: chẳng hạn, ta có thể ước lượng  $\hat{\pi}_k$  bằng tỷ lệ training observation thuộc class thứ  $k$ , với  $k = 1, \dots, K$ .

Tuy nhiên, việc ước lượng  $f_1(x), \dots, f_K(x)$  tinh tế hơn. Nhớ rằng  $f_k(x)$  là  $p$ -dimensional density function của observation trong class thứ  $k$ , với  $k = 1, \dots, K$ . Nhìn chung, ước lượng một  $p$ -dimensional density function là bài toán khó. Trong LDA, ta đưa ra một giả định rất mạnh giúp đơn giản hóa đáng kể nhiệm vụ:  $f_k$  là density function của multivariate normal random variable với class-specific mean  $\mu_k$  và shared covariance matrix  $\Sigma$ . Trái lại, trong QDA, ta giả định  $f_k$  là density function của multivariate normal random variable với class-specific mean  $\mu_k$  và class-specific covariance matrix  $\Sigma_k$ . Nhờ các giả định rất mạnh này, ta thay bài toán cực khó là ước lượng  $K$  density function  $p$  chiều bằng bài toán đơn giản hơn nhiều: ước lượng  $K$  mean vector  $p$  chiều và một covariance matrix  $p \times p$  (với LDA) hoặc  $K$  covariance matrix  $p \times p$  (với QDA).

Naive Bayes classifier dùng một hướng khác để ước lượng  $f_1(x), \dots, f_K(x)$ . Thay vì giả định các hàm này thuộc một family of distribution cụ thể, chẳng hạn multivariate normal, ta chỉ đưa ra một giả định:

*Trong class thứ  $k$ ,  $p$  predictor độc lập với nhau.*

Viết bằng toán học, giả định này có nghĩa rằng với  $k = 1, \dots, K$ ,

$$f_k(x) = f_{k1}(x_1) \times f_{k2}(x_2) \times \dots \times f_{kp}(x_p), \quad (4.29)$$

trong đó  $f_{kj}$  là density function của predictor thứ  $j$  trong các observation thuộc class thứ  $k$ .

Tại sao giả định này mạnh đến vậy? Về bản chất, việc ước lượng  $p$ -dimensional density function khó vì ta không chỉ phải xét *marginal distribution* của từng predictor, tức distribution riêng của từng predictor, mà còn phải xét *joint distribution* của các predictor, tức association giữa những predictor khác nhau. Trong multivariate normal distribution, association giữa các predictor được tóm tắt bởi các element ngoài diagonal của covariance matrix. Tuy nhiên, nhìn chung association này có thể rất khó đặc trưng hóa và cực kỳ khó ước lượng. Nhưng bằng cách giả định  $p$  covariate độc lập trong mỗi class, ta hoàn toàn loại bỏ nhu cầu xem xét association giữa  $p$  predictor, vì đơn giản ta đã giả định không có association giữa chúng!

marginal  
distribution  
joint  
distribution

Ta có thực sự tin giả định naive Bayes rằng  $p$  covariate độc lập trong mỗi class không? Trong phần lớn bối cảnh, câu trả lời là không. Nhưng dù modeling assumption này được đưa ra vì sự tiện lợi, nó thường cho kết quả khá tốt, đặc biệt trong những bối cảnh  $n$  không đủ lớn so với  $p$  để ta ước lượng hiệu quả joint distribution của predictor trong mỗi class. Thật vậy, vì ước lượng joint distribution đòi hỏi lượng dữ liệu khổng lồ, naive Bayes là lựa chọn tốt trong nhiều bối cảnh. Về bản chất, giả định naive Bayes đưa thêm một ít bias nhưng giảm variance, tạo ra một classifier hoạt động khá tốt trong thực tế nhờ bias-variance trade-off.

Sau khi đưa ra giả định naive Bayes, ta có thể thay (4.29) vào (4.15) để thu được biểu thức cho posterior probability,

$$\Pr(Y = k \mid X = x) = \frac{\pi_k \prod_{j=1}^p f_{kj}(x_j)}{\sum_{l=1}^K \pi_l \prod_{j=1}^p f_{lj}(x_j)}, \quad (4.30)$$

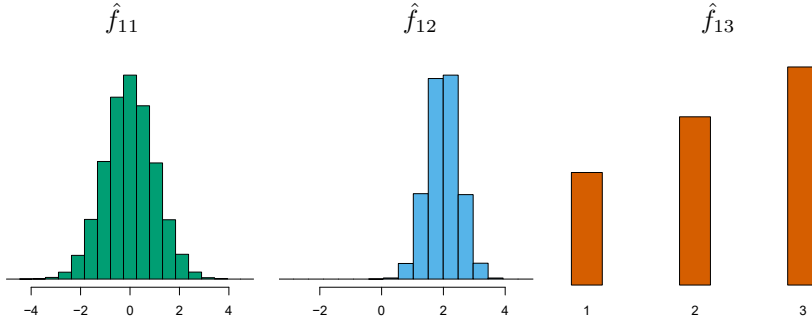
với  $k = 1, \dots, K$ .

Để ước lượng one-dimensional density function  $f_{kj}$  bằng training data  $x_{1j}, \dots, x_{nj}$ , ta có một vài lựa chọn.

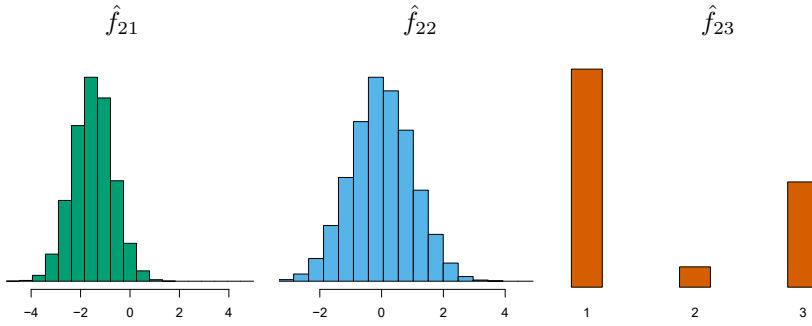
- Nếu  $X_j$  là quantitative, ta có thể giả định  $X_j \mid Y = k \sim N(\mu_{jk}, \sigma_{jk}^2)$ . Nói cách khác, trong mỗi class, predictor thứ  $j$  được rút từ univariate normal distribution. Điều này có vẻ hơi giống QDA, nhưng có một khác biệt then chốt: ở đây ta giả định các predictor độc lập; tương đương QDA với giả định bổ sung rằng class-specific covariance matrix là diagonal.
- Nếu  $X_j$  là quantitative, một lựa chọn khác là dùng non-parametric estimate cho  $f_{kj}$ . Một cách rất đơn giản là tạo histogram cho observation của predictor thứ  $j$  trong mỗi class. Sau đó, ta có thể ước lượng  $f_{kj}(x_j)$  bằng tỷ lệ training observation trong class thứ  $k$  thuộc cùng histogram bin với  $x_j$ . Hoặc ta có thể dùng *kernel density estimator*, về bản chất là phiên bản được làm mượt của histogram.
- Nếu  $X_j$  là qualitative, ta chỉ cần đếm tỷ lệ training observation của predictor thứ  $j$  ứng với từng class. Chẳng hạn, giả sử  $X_j \in \{1, 2, 3\}$  và

kernel density  
estimator

## Density estimates for class k=1



## Density estimates for class k=2



**HÌNH 4.10.** Trong toy example ở Mục 4.4.4, ta sinh dữ liệu với  $p = 3$  predictor và  $K = 2$  class. Hai predictor đầu là quantitative, predictor thứ ba là qualitative với ba level. Với mỗi class, estimated density của từng predictor được biểu diễn. Nếu prior probability của hai class bằng nhau thì observation  $x^* = (0.4, 1.5, 1)^T$  có posterior probability 94.4% thuộc class thứ nhất.

ta có 100 observation trong class thứ  $k$ . Giả sử predictor thứ  $j$  nhận các giá trị 1, 2 và 3 lần lượt trong 32, 55 và 13 observation. Khi đó, ta ước lượng  $f_{kj}$  bằng

$$\hat{f}_{kj}(x_j) = \begin{cases} 0.32 & \text{nếu } x_j = 1, \\ 0.55 & \text{nếu } x_j = 2, \\ 0.13 & \text{nếu } x_j = 3. \end{cases}$$

Bây giờ ta xét naive Bayes classifier trong một toy example với  $p = 3$  predictor và  $K = 2$  class. Hai predictor đầu là quantitative, predictor thứ ba là qualitative với ba level. Giả sử thêm rằng  $\hat{\pi}_1 = \hat{\pi}_2 = 0.5$ . Các estimated density function  $\hat{f}_{kj}$  với  $k = 1, 2$  và  $j = 1, 2, 3$  được biểu diễn trong Hình 4.10. Bây giờ giả sử ta muốn phân loại observation mới  $x^* = (0.4, 1.5, 1)^T$ .

Trong ví dụ này,  $\hat{f}_{11}(0.4) = 0.368$ ,  $\hat{f}_{12}(1.5) = 0.484$ ,  $\hat{f}_{13}(1) = 0.226$ ,  $\hat{f}_{21}(0.4) = 0.030$ ,  $\hat{f}_{22}(1.5) = 0.130$  và  $\hat{f}_{23}(1) = 0.616$ . Thay các estimate này vào (4.30) cho posterior probability estimate  $\Pr(Y = 1 \mid X = x^*) = 0.944$  và  $\Pr(Y = 2 \mid X = x^*) = 0.056$ .

**BẢNG 4.8.** So sánh naive Bayes prediction với trạng thái *default* thực của 10.000 training observation trong dataset *Default*, khi dự đoán default cho mọi observation có  $\Pr(Y = \text{default} \mid X = x) > 0.5$ .

|                             |       | True default status |     |       |
|-----------------------------|-------|---------------------|-----|-------|
|                             |       | No                  | Yes | Total |
| Predicted<br>default status | No    | 9621                | 244 | 9865  |
|                             | Yes   | 46                  | 89  | 135   |
|                             | Total | 9667                | 333 | 10000 |

**BẢNG 4.9.** So sánh naive Bayes prediction với trạng thái *default* thực của 10.000 training observation trong dataset *Default*, khi dự đoán default cho mọi observation có  $\Pr(Y = \text{default} \mid X = x) > 0.2$ .

|                             |       | True default status |     |       |
|-----------------------------|-------|---------------------|-----|-------|
|                             |       | No                  | Yes | Total |
| Predicted<br>default status | No    | 9339                | 130 | 9469  |
|                             | Yes   | 328                 | 203 | 531   |
|                             | Total | 9667                | 333 | 10000 |

Bảng 4.8 cung cấp confusion matrix thu được khi áp dụng naive Bayes classifier cho dataset *Default*, trong đó ta dự đoán default nếu posterior probability của default, tức  $\Pr(Y = \text{default} \mid X = x)$ , vượt 0.5. So với kết quả LDA trong Bảng 4.4, các phát hiện có cả mặt tốt lẫn mặt xấu. Dù LDA có overall error rate thấp hơn đôi chút, naive Bayes dự đoán đúng tỷ lệ true defaulter cao hơn. Trong implementation naive Bayes này, ta giả định mỗi quantitative predictor được rút từ Gaussian distribution và, tất nhiên, mỗi predictor độc lập trong từng class.

Cũng như với LDA, ta có thể dễ dàng điều chỉnh probability threshold để dự đoán default. Chẳng hạn, Bảng 4.9 cung cấp confusion matrix khi dự đoán default nếu  $\Pr(Y = \text{default} \mid X = x) > 0.2$ . Một lần nữa, so với LDA có cùng threshold trong Bảng 4.5, kết quả có cả mặt tốt lẫn mặt xấu. Naive Bayes có error rate cao hơn nhưng dự đoán đúng gần hai phần ba true default.

Trong ví dụ này, việc naive Bayes không vượt trội rõ ràng so với LDA không quá bất ngờ: dataset có  $n = 10,000$  và  $p = 2$ , nên mức giảm variance nhờ giả định naive Bayes chưa chắc đáng giá. Ta kỳ vọng naive Bayes mang lại lợi ích lớn hơn so với LDA hoặc QDA trong trường hợp  $p$  lớn hơn hoặc  $n$  nhỏ hơn, khi việc giảm variance đặc biệt quan trọng.



## 4.5 A Comparison of Classification Methods

### 4.5.1 An Analytical Comparison

Bây giờ ta tiến hành một phép so sánh về mặt phân tích (hay toán học) giữa LDA, QDA, naive Bayes và logistic regression. Ta xét các phương pháp này trong bối cảnh có  $K$  class, sao cho một observation được gán vào class làm cực đại  $\Pr(Y = k \mid X = x)$ . Tương đương, ta có thể chọn class  $K$  làm *baseline class* và gán một observation vào class làm cực đại

$$\log \left( \frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)} \right) \quad (4.31)$$

cho  $k = 1, \dots, K$ . Việc xem xét dạng cụ thể của (4.31) đối với từng method giúp ta hiểu rõ các điểm giống và khác nhau giữa chúng.

Trước hết, với LDA, ta có thể sử dụng định lý Bayes (4.15) cùng với giả định rằng các predictor trong mỗi class được rút ra từ một multivariate normal density (4.23), với mean riêng cho từng class và covariance matrix dùng chung, để chỉ ra rằng

$$\begin{aligned} \log \left( \frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)} \right) &= \log \left( \frac{\pi_k f_k(x)}{\pi_K f_K(x)} \right) \\ &= \log \left( \frac{\pi_k \exp[-\frac{1}{2}(x - \mu_k)^T \Sigma^{-1}(x - \mu_k)]}{\pi_K \exp[-\frac{1}{2}(x - \mu_K)^T \Sigma^{-1}(x - \mu_K)]} \right) \\ &= \log \left( \frac{\pi_k}{\pi_K} \right) - \frac{1}{2}(x - \mu_k)^T \Sigma^{-1}(x - \mu_k) \\ &\quad + \frac{1}{2}(x - \mu_K)^T \Sigma^{-1}(x - \mu_K) \\ &= \log \left( \frac{\pi_k}{\pi_K} \right) - \frac{1}{2}(\mu_k + \mu_K)^T \Sigma^{-1}(\mu_k - \mu_K) \\ &\quad + x^T \Sigma^{-1}(\mu_k - \mu_K) \\ &= a_k + \sum_{j=1}^p b_{kj} x_j, \end{aligned} \quad (4.32)$$

trong đó

$$a_k = \log \left( \frac{\pi_k}{\pi_K} \right) - \frac{1}{2}(\mu_k + \mu_K)^T \Sigma^{-1}(\mu_k - \mu_K),$$

và  $b_{kj}$  là component thứ  $j$  của  $\Sigma^{-1}(\mu_k - \mu_K)$ . Do đó LDA, giống logistic regression, giả định rằng log odds của các posterior probability là linear theo  $x$ .

Bằng các phép tính tương tự, trong bối cảnh QDA, (4.31) trở thành

$$\log \left( \frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)} \right) = a_k + \sum_{j=1}^p b_{kj} x_j + \sum_{j=1}^p \sum_{l=1}^p c_{kjl} x_j x_l, \quad (4.33)$$

trong đó  $a_k$ ,  $b_{kj}$  và  $c_{kjl}$  là các hàm của  $\pi_k$ ,  $\pi_K$ ,  $\mu_k$ ,  $\mu_K$ ,  $\Sigma_k$  và  $\Sigma_K$ . Một lần

nữa, đúng như tên gọi gợi ý, QDA giả định rằng log odds của các posterior probability là quadratic theo  $x$ .

Cuối cùng, ta xem xét (4.31) trong bối cảnh naive Bayes. Nhớ rằng ở đây,  $f_k(x)$  được model hóa thành tích của  $p$  hàm một chiều  $f_{kj}(x_j)$ , với  $j = 1, \dots, p$ . Do đó,

$$\begin{aligned}
 \log\left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)}\right) &= \log\left(\frac{\pi_k f_k(x)}{\pi_K f_K(x)}\right) \\
 &= \log\left(\frac{\pi_k \prod_{j=1}^p f_{kj}(x_j)}{\pi_K \prod_{j=1}^p f_{Kj}(x_j)}\right) \\
 &= \log\left(\frac{\pi_k}{\pi_K}\right) + \sum_{j=1}^p \log\left(\frac{f_{kj}(x_j)}{f_{Kj}(x_j)}\right) \quad (4.34) \\
 &= a_k + \sum_{j=1}^p g_{kj}(x_j),
 \end{aligned}$$

trong đó  $a_k = \log(\pi_k/\pi_K)$  và  $g_{kj}(x_j) = \log(f_{kj}(x_j)/f_{Kj}(x_j))$ . Vì vậy, vế phải của (4.34) có dạng một generalized additive model, chủ đề sẽ được bàn sâu hơn trong Chương 7.

Quan sát (4.32), (4.33) và (4.34) cho ta các nhận xét sau về LDA, QDA và naive Bayes:

- LDA là một trường hợp đặc biệt của QDA với  $c_{kjl} = 0$  cho mọi  $j = 1, \dots, p$ ,  $l = 1, \dots, p$  và  $k = 1, \dots, K$ . (Dĩ nhiên điều này không đáng ngạc nhiên, vì LDA đơn giản là một phiên bản bị ràng buộc của QDA với  $\Sigma_1 = \dots = \Sigma_K = \Sigma$ .)
- Bất kỳ classifier nào có linear decision boundary đều là một trường hợp đặc biệt của naive Bayes với  $g_{kj}(x_j) = b_{kj}x_j$ . Đặc biệt, điều này có nghĩa là LDA là một trường hợp đặc biệt của naive Bayes! Điều này hoàn toàn không hiển nhiên từ các mô tả về LDA và naive Bayes trước đó trong chương, bởi hai method đưa ra những giả định rất khác nhau: LDA giả định các feature có normal distribution với một within-class covariance matrix dùng chung, còn naive Bayes lại giả định các feature độc lập.
- Nếu ta model  $f_{kj}(x_j)$  trong naive Bayes classifier bằng một Gaussian distribution một chiều  $N(\mu_{kj}, \sigma_j^2)$ , thì ta thu được  $g_{kj}(x_j) = b_{kj}x_j$ , trong đó  $b_{kj} = (\mu_{kj} - \mu_{Kj})/\sigma_j^2$ . Trong trường hợp này, naive Bayes thực ra là một trường hợp đặc biệt của LDA với  $\Sigma$  bị ràng buộc là một diagonal matrix có phần tử đường chéo thứ  $j$  bằng  $\sigma_j^2$ .
- QDA và naive Bayes không method nào là trường hợp đặc biệt của method kia. Naive Bayes có thể tạo ra một fit linh hoạt hơn, vì ta có thể chọn tùy ý  $g_{kj}(x_j)$ . Tuy nhiên, nó bị giới hạn ở một *additive fit* thuần túy, theo nghĩa là trong (4.34), một hàm của  $x_j$  được cộng với một hàm của  $x_l$ , với  $j \neq l$ ; các hạng tử này không bao giờ được nhân với nhau. Trái lại, QDA bao gồm các hạng tử nhân dạng  $c_{kjl}x_jx_l$ . Do đó, QDA có tiềm năng chính xác hơn trong những bối cảnh mà interaction giữa các predictor đóng vai trò quan trọng khi phân biệt

các class.

Không method nào trong số này luôn vượt trội so với các method còn lại: trong bất kỳ bối cảnh nào, việc lựa chọn method sẽ phụ thuộc vào distribution thực của các predictor trong mỗi class trong số  $K$  class, cùng với các yếu tố khác như giá trị của  $n$  và  $p$ . Yếu tố sau liên hệ trực tiếp với bias-variance trade-off.

Logistic regression liên hệ với câu chuyện này như thế nào? Nhớ từ (4.12) rằng multinomial logistic regression có dạng

$$\log\left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)}\right) = \beta_{k0} + \sum_{j=1}^p \beta_{kj}x_j.$$

Dạng này giống hệt dạng linear của LDA trong (4.32): ở cả hai trường hợp,  $\log(\Pr(Y = k \mid X = x) / \Pr(Y = K \mid X = x))$  là một hàm linear của các predictor. Trong LDA, các coefficient của hàm linear này là các hàm của những estimate cho  $\pi_k$ ,  $\pi_K$ ,  $\mu_k$ ,  $\mu_K$  và  $\Sigma$ , thu được bằng cách giả định rằng  $X_1, \dots, X_p$  tuân theo normal distribution trong mỗi class. Trái lại, trong logistic regression, các coefficient được chọn để cực đại likelihood function (4.5). Vì vậy, ta kỳ vọng LDA sẽ vượt logistic regression khi giả định normality (xấp xỉ) đúng, và kỳ vọng logistic regression hoạt động tốt hơn khi giả định đó không đúng.

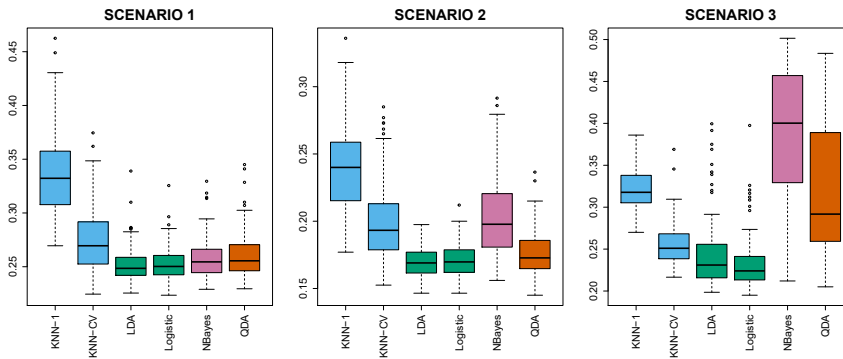
Ta kết thúc bằng một thảo luận ngắn về K-nearest neighbors (KNN), được giới thiệu ở Chương 2. Nhớ rằng KNN sử dụng một cách tiếp cận hoàn toàn khác với các classifier đã thấy trong chương này. Để đưa ra prediction cho một observation  $X = x$ , ta xác định các training observation gần  $x$  nhất. Sau đó,  $X$  được gán vào class có số lượng nhiều nhất trong số các observation này. Vì vậy KNN là một cách tiếp cận hoàn toàn non-parametric: không có giả định nào về hình dạng của decision boundary. Ta có các nhận xét sau về KNN:

- Vì KNN hoàn toàn non-parametric, ta có thể kỳ vọng phương pháp này vượt LDA và logistic regression khi decision boundary rất non-linear, với điều kiện  $n$  rất lớn và  $p$  nhỏ.
- Để classification chính xác, KNN đòi hỏi *rất nhiều* observation so với số predictor—tức là  $n$  lớn hơn  $p$  rất nhiều. Điều này liên quan đến việc KNN là non-parametric, do đó thường làm giảm bias nhưng phải trả giá bằng variance lớn.
- Trong những bối cảnh decision boundary là non-linear nhưng  $n$  chỉ ở mức vừa phải, hoặc  $p$  không thật sự nhỏ, QDA có thể được ưu tiên hơn KNN. Lý do là QDA có thể cung cấp một non-linear decision boundary trong khi vẫn tận dụng dạng parametric; vì vậy, so với KNN, nó cần sample size nhỏ hơn để classification chính xác.
- Không giống logistic regression, KNN không cho ta biết predictor nào quan trọng: ta không thu được một bảng coefficient như ở Bảng 4.3.

### 4.5.2 An Empirical Comparison

Bây giờ ta so sánh hiệu năng thực nghiệm (thực tiễn) của logistic regression, LDA, QDA, naive Bayes và KNN. Ta sinh dữ liệu từ sáu kịch bản khác nhau, mỗi kịch bản là một bài toán binary classification (hai class). Trong ba kịch bản, Bayes decision boundary là linear; trong các kịch bản còn lại, nó là non-linear. Với mỗi kịch bản, ta tạo 100 training dataset ngẫu nhiên. Trên từng training set, ta fit mỗi method vào dữ liệu và tính test error rate thu được trên một test set lớn. Kết quả cho các kịch bản linear được trình bày trong Hình 4.11, còn kết quả cho các kịch bản non-linear nằm trong Hình 4.12. Method KNN yêu cầu chọn  $K$ , tức số neighbor (không nên nhầm với số class ở các phần trước của chương này). Ta thực hiện KNN với hai giá trị của  $K$ :  $K = 1$ , và một giá trị  $K$  được chọn tự động bằng một phương pháp gọi là cross-validation, sẽ được thảo luận thêm trong Chương 5. Với naive Bayes, ta giả định các univariate Gaussian density cho feature trong mỗi class (và dĩ nhiên—vì đây là đặc trưng cốt lõi của naive Bayes—giả định các feature độc lập).

Trong cả sáu kịch bản, có  $p = 2$  quantitative predictor. Các kịch bản như sau:



**HÌNH 4.11.** Các boxplot của test error rate cho từng kịch bản linear được mô tả trong phần văn bản chính.

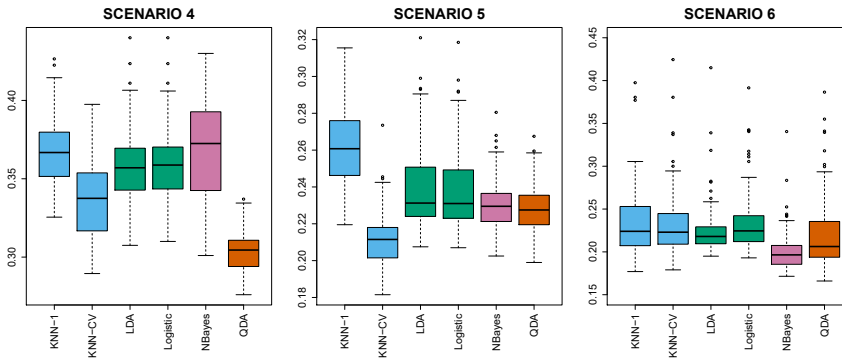
*Scenario 1:* Có 20 training observation trong mỗi class của hai class. Các observation trong mỗi class là những biến ngẫu nhiên normal không tương quan, với mean khác nhau giữa hai class. Panel bên trái của Hình 4.11 cho thấy LDA hoạt động tốt trong bối cảnh này, đúng như ta kỳ vọng vì đây chính là model mà LDA giả định. Logistic regression cũng hoạt động khá tốt, vì nó giả định linear decision boundary. KNN hoạt động kém vì phải trả giá về variance mà không được bù lại bằng mức giảm bias. QDA cũng kém hơn LDA, do nó fit một classifier linh hoạt hơn mức cần thiết. Naive Bayes hoạt động tốt hơn QDA một chút, vì giả định các predictor độc lập của naive Bayes là đúng.

*Scenario 2:* Các chỉ tiết giống Scenario 1, ngoại trừ việc trong mỗi class, hai predictor có correlation bằng  $-0.5$ . Panel giữa của Hình 4.11 cho thấy hiệu năng của hầu hết các method tương tự kịch bản trước. Ngoại lệ đáng chú ý là naive Bayes, hoạt động rất kém ở đây vì giả định các predictor độc lập của naive Bayes bị vi phạm.

*Scenario 3:* Giống như ở kịch bản trước, giữa các predictor trong mỗi

class tồn tại correlation âm đáng kể. Tuy nhiên, lần này ta sinh  $X_1$  và  $X_2$  từ t-distribution, với 50 observation cho mỗi class. T-distribution có hình dạng tương tự normal distribution, nhưng có xu hướng tạo ra nhiều điểm cực đoan hơn—tức nhiều điểm nằm xa mean hơn. Trong bối cảnh này, decision boundary vẫn linear, vì vậy phù hợp với framework của logistic regression. Thiết lập này vi phạm các giả định của LDA, vì các observation không được rút ra từ normal distribution. Panel bên phải của Hình 4.11 cho thấy logistic regression vượt LDA, dù cả hai method đều tốt hơn các cách tiếp cận còn lại. Đặc biệt, kết quả của QDA suy giảm đáng kể do non-normality. Naive Bayes hoạt động rất kém vì giả định independence bị vi phạm.

*Scenario 4:* Dữ liệu được sinh từ normal distribution, với correlation bằng 0.5 giữa các predictor trong class thứ nhất và correlation bằng  $-0.5$  giữa các predictor trong class thứ hai. Thiết lập này phù hợp với giả định của QDA và tạo ra các quadratic decision boundary.



**HÌNH 4.12.** Các boxplot của test error rate cho từng kịch bản non-linear được mô tả trong phần văn bản chính.

Panel bên trái của Hình 4.12 cho thấy QDA vượt tất cả các cách tiếp cận còn lại. Giả định các predictor độc lập của naive Bayes bị vi phạm, vì vậy naive Bayes hoạt động kém.

*Scenario 5:* Dữ liệu được sinh từ normal distribution với các predictor không tương quan. Sau đó, các response được lấy mẫu từ logistic function áp dụng lên một hàm non-linear phức tạp của các predictor. Panel giữa của Hình 4.12 cho thấy cả QDA lẫn naive Bayes đều cho kết quả tốt hơn đôi chút so với các linear method, trong khi method KNN-CV linh hoạt hơn nhiều cho kết quả tốt nhất. Nhưng KNN với  $K = 1$  lại cho kết quả kém nhất trong tất cả các method. Điều này nhấn mạnh rằng ngay cả khi dữ liệu biểu hiện một quan hệ non-linear phức tạp, một non-parametric method như KNN vẫn có thể cho kết quả kém nếu mức độ smoothness không được chọn đúng.

*Scenario 6:* Các observation được sinh từ normal distribution với một diagonal covariance matrix khác nhau cho mỗi class. Tuy nhiên, sample size rất nhỏ: chỉ  $n = 6$  trong mỗi class. Naive Bayes hoạt động rất tốt vì các giả định của nó được thỏa mãn. LDA và logistic regression hoạt động kém vì decision boundary thực là non-linear, do các covariance matrix không bằng nhau. QDA kém hơn naive Bayes một chút vì, với sample size rất nhỏ,

QDA phải chịu variance quá lớn khi estimate correlation giữa các predictor trong mỗi class. Hiệu năng của KNN cũng bị ảnh hưởng bởi sample size rất nhỏ.

Sáu ví dụ này minh họa rằng không một method nào sẽ vượt trội các method khác trong mọi tình huống. Khi các decision boundary thực là linear, LDA và logistic regression thường hoạt động tốt. Khi các boundary ở mức non-linear vừa phải, QDA hoặc naive Bayes có thể cho kết quả tốt hơn. Cuối cùng, với các decision boundary phức tạp hơn nhiều, một non-parametric approach như KNN có thể vượt trội. Nhưng mức độ smoothness của một non-parametric approach phải được chọn cẩn thận. Trong chương tiếp theo, ta sẽ xem xét một số cách để chọn đúng mức độ smoothness và, tổng quát hơn, chọn overall method tốt nhất.

Cuối cùng, nhớ từ Chương 3 rằng trong bối cảnh regression, ta có thể xử lý một quan hệ non-linear giữa các predictor và response bằng cách thực hiện regression sử dụng các phép biến đổi của predictor. Một cách tiếp cận tương tự cũng có thể được dùng trong bối cảnh classification. Chẳng hạn, ta có thể tạo một phiên bản logistic regression linh hoạt hơn bằng cách đưa  $X^2$ ,  $X^3$ , và thậm chí  $X^4$  vào làm predictor.

Việc này có thể cải thiện hoặc không cải thiện hiệu năng của logistic regression, tùy vào việc mức tăng variance do độ linh hoạt bổ sung có được bù lại bởi một mức giảm bias đủ lớn hay không. Ta cũng có thể làm tương tự với LDA. Nếu thêm vào LDA tất cả các hạng tử quadratic có thể có và các cross-product, dạng của model sẽ giống model QDA, mặc dù các parameter estimate sẽ khác nhau. Cách làm này cho phép ta di chuyển tới một vị trí nào đó giữa model LDA và model QDA.

## 4.6 Generalized Linear Models

Trong Chương 3, ta giả định response  $Y$  là quantitative, và xem xét việc dùng least squares linear regression để dự đoán  $Y$ . Cho đến thời điểm này trong chương hiện tại, ta lại giả định  $Y$  là qualitative. Tuy nhiên, đôi khi ta có thể gặp những tình huống mà  $Y$  không hoàn toàn là qualitative cũng không hoàn toàn là quantitative, và vì thế cả linear regression của Chương 3 lẫn các classification approach đã trình bày trong chương này đều không áp dụng trực tiếp được.

Để có một ví dụ cụ thể, ta xét dataset **Bikeshare**. Response là **bikers**, tức số người sử dụng một chương trình chia sẻ xe đạp trong mỗi giờ tại Washington, DC. Giá trị response này không phải qualitative cũng không phải quantitative theo nghĩa thông thường: thay vào đó, nó nhận các giá trị nguyên không âm, hay *counts*. Ta sẽ dự đoán **bikers** bằng các covariate **mnth** (tháng trong năm), **hr** (giờ trong ngày, từ 0 đến 23), **workingday** (indicator variable bằng 1 nếu ngày đó không phải cuối tuần cũng không phải ngày lễ), **temp** (nhiệt độ đã chuẩn hóa, theo độ Celsius), và **weathersit** (một qualitative variable nhận một trong bốn giá trị: trời quang; sương mù hoặc nhiều mây; mưa nhẹ hoặc tuyết nhẹ; mưa lớn hoặc tuyết lớn).

*counts*

Trong các phân tích tiếp theo, ta sẽ coi **mnth**, **hr**, và **weathersit** là các qualitative variable.

### 4.6.1 Linear Regression on the Bikeshare Data

Trước hết, ta xét việc dự đoán **bikers** bằng linear regression. Kết quả được trình bày trong Bảng 4.10.

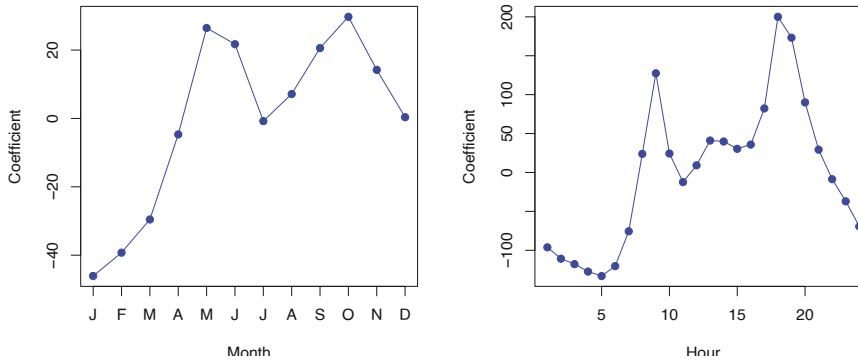
|                                     | Coefficient | Std. error | <i>t</i> -statistic | <i>p</i> -value |
|-------------------------------------|-------------|------------|---------------------|-----------------|
| Intercept                           | 73.60       | 5.13       | 14.34               | 0.00            |
| <b>workingday</b>                   | 1.27        | 1.78       | 0.71                | 0.48            |
| <b>temp</b>                         | 157.21      | 10.26      | 15.32               | 0.00            |
| <b>weathersit</b> [cloudy/misty]    | -12.89      | 1.96       | -6.56               | 0.00            |
| <b>weathersit</b> [light rain/snow] | -66.49      | 2.97       | -22.43              | 0.00            |
| <b>weathersit</b> [heavy rain/snow] | -109.75     | 76.67      | -1.43               | 0.15            |

**BẢNG 4.10.** Kết quả của một least squares linear model được fit để dự đoán **bikers** trong dữ liệu **Bikeshare**. Các predictor **mnth** và **hr** được lược khỏi bảng này do giới hạn không gian, và có thể xem trong Hình 4.13. Với qualitative variable **weathersit**, baseline level tương ứng với trời quang.

Chẳng hạn, ta thấy việc thời tiết chuyển từ trời quang sang nhiều mây dẫn đến trung bình ít hơn 12.89 người đi xe đạp mỗi giờ; tuy nhiên, nếu thời tiết xấu thêm thành mưa hoặc tuyết, thì con số này tiếp tục giảm thêm 53.60 người mỗi giờ. Hình 4.13 hiển thị các coefficient gắn với **mnth** và các coefficient gắn với **hr**.

Ta thấy mức sử dụng xe đạp cao nhất vào mùa xuân và mùa thu, và thấp nhất trong các tháng mùa đông. Hơn nữa, việc sử dụng xe đạp đạt mức lớn nhất quanh giờ cao điểm (9 AM và 6 PM), và thấp nhất vào ban đêm. Vì vậy, thoạt nhìn, việc fit một linear regression model cho dataset **Bikeshare** dường như cho những kết quả hợp lý và trực quan.

Nhưng khi xem xét kỹ hơn, một số vấn đề trở nên rõ ràng. Chẳng hạn,



**HÌNH 4.13.** Một least squares linear regression model được fit để dự đoán **bikers** trong dataset **Bikeshare**. Trái: Các coefficient gắn với tháng trong năm. Mức sử dụng xe đạp cao nhất vào mùa xuân và mùa thu, và thấp nhất vào mùa đông. Phải: Các coefficient gắn với giờ trong ngày. Mức sử dụng xe đạp cao nhất trong các khung giờ đi làm cao điểm, và thấp nhất vào ban đêm.

9.6% fitted value trong dataset **Bikeshare** là âm: tức là linear regression model dự đoán một số lượng người dùng âm trong 9.6% số giờ của dataset. Điều này khiến khả năng tạo ra các prediction có ý nghĩa từ dữ liệu trở nên đáng ngờ, đồng thời cũng làm nảy sinh lo ngại về độ chính xác của coefficient estimate, confidence interval và các output khác của regression model.

Ngoài ra, hợp lý khi nghi ngờ rằng khi expected value của **bikers** nhỏ thì variance của **bikers** cũng nên nhỏ. Chẳng hạn, lúc 2 AM trong một trận bão tuyết lớn vào tháng Mười Hai, ta kỳ vọng cực kỳ ít người sử dụng xe đạp, và hơn nữa số người dùng trong các điều kiện đó cũng chỉ dao động rất ít. Điều này đúng với dữ liệu: từ 1 AM đến 4 AM, trong các tháng Mười Hai, Một và Hai, khi trời mưa, trung bình có 5.05 người dùng với standard deviation bằng 3.73. Trái lại, từ 7 AM đến 10 AM, trong các tháng Tư, Năm và Sáu, khi trời quang, trung bình có 243.59 người dùng với standard deviation bằng 131.7. Quan hệ mean–variance được hiển thị ở panel bên trái của Hình 4.14. Đây là một vi phạm lớn đối với các giả định của linear model, vốn phát biểu rằng

$$Y = \sum_{j=1}^p X_j \beta_j + \epsilon,$$

trong đó  $\epsilon$  là một error term có mean bằng 0 và variance  $\sigma^2$  không đổi, không phụ thuộc vào các covariate. Vì vậy, heteroscedasticity của dữ liệu khiến tính phù hợp của linear regression model bị đặt dấu hỏi.

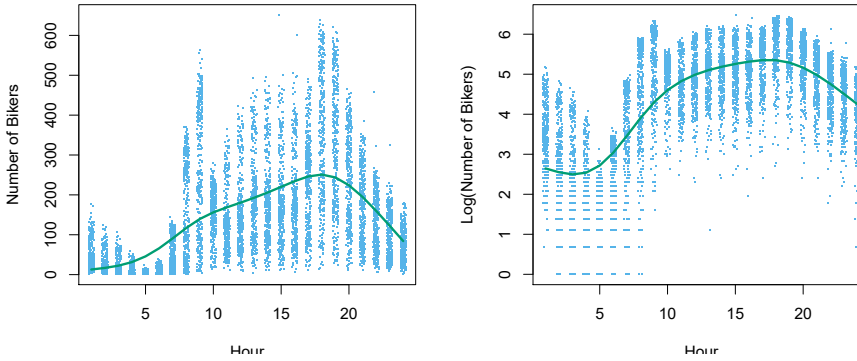
Cuối cùng, response **bikers** nhận giá trị nguyên. Nhưng trong linear model,

$$Y = \beta_0 + \sum_{j=1}^p X_j \beta_j + \epsilon,$$

trong đó  $\epsilon$  là một error term nhận giá trị liên tục. Điều này có nghĩa rằng trong linear model, response  $Y$  tất yếu nhận giá trị liên tục (quantitative). Do đó, bản chất integer-valued của response **bikers** cho thấy linear



regression model không hoàn toàn thỏa đáng với dataset này.



**HÌNH 4.14.** Trái: Trên dataset *Bikeshare*, số người đi xe đạp được hiển thị trên trục  $y$ , còn giờ trong ngày được hiển thị trên trục  $x$ . Jitter được áp dụng để dễ quan sát. Phần lớn thời gian, khi mean của số người đi xe đạp tăng thì variance của số người đi xe đạp cũng tăng. Một smoothing spline fit được vẽ bằng màu xanh lá. Phải: Log của số người đi xe đạp giờ được hiển thị trên trục  $y$ .

Một số vấn đề phát sinh khi fit linear regression model cho dữ liệu *Bikeshare* có thể được khắc phục bằng cách biến đổi response; chẳng hạn, ta có thể fit model

$$\log(Y) = \sum_{j=1}^p X_j \beta_j + \epsilon.$$

Việc biến đổi response tránh khả năng xuất hiện prediction âm, và khắc phục phần lớn heteroscedasticity trong dữ liệu chưa biến đổi, như được thể hiện ở panel bên phải của Hình 4.14. Tuy nhiên, đây vẫn chưa phải một giải pháp hoàn toàn thỏa đáng, vì prediction và inference được thực hiện theo log của response thay vì theo chính response. Điều này gây khó khăn khi diễn giải, chẳng hạn: “một mức tăng một đơn vị của  $X_j$  gắn với mức tăng  $\beta_j$  trong mean của log của  $Y$ ”. Hơn nữa, không thể áp dụng log transformation cho response trong những bối cảnh response có thể nhận giá trị 0. Vì vậy, dù việc fit linear model cho một phép biến đổi của response có thể là một cách tiếp cận phù hợp với một số count-valued dataset, nó thường vẫn để lại một số điểm chưa thỏa đáng. Trong mục tiếp theo, ta sẽ thấy Poisson regression model cung cấp một cách tiếp cận tự nhiên và thanh lịch hơn nhiều cho bài toán này.

#### 4.6.2 Poisson Regression on the Bikeshare Data

Để khắc phục những điểm bất cập của linear regression khi phân tích dataset *Bikeshare*, ta sẽ sử dụng một cách tiếp cận khác gọi là *Poisson regression*. Trước khi bàn về Poisson regression, trước hết ta phải giới thiệu *Poisson distribution*.

Giả sử random variable  $Y$  nhận các giá trị nguyên không âm, tức  $Y \in \{0, 1, 2, \dots\}$ . Nếu  $Y$  tuân theo Poisson distribution, thì

$$\Pr(Y = k) = \frac{e^{-\lambda} \lambda^k}{k!}, \quad k = 0, 1, 2, \dots \quad (4.35)$$

*Poisson regression*  
*Poisson*  
*distribution*

Ở đây,  $\lambda > 0$  là expected value của  $Y$ , tức  $E(Y)$ . Hóa ra  $\lambda$  cũng bằng variance của  $Y$ , tức  $\lambda = E(Y) = \text{Var}(Y)$ . Điều này có nghĩa là nếu  $Y$  tuân theo Poisson distribution, thì mean của  $Y$  càng lớn, variance của nó cũng càng lớn. (Trong (4.35), ký hiệu  $k!$ , đọc là “ $k$  factorial”, được định nghĩa là  $k! = k \times (k-1) \times (k-2) \times \dots \times 3 \times 2 \times 1$ .)

Poisson distribution thường được dùng để model các count; đây là một lựa chọn tự nhiên vì nhiều lý do, trong đó có việc count, giống Poisson distribution, nhận các giá trị nguyên không âm. Để thấy cách có thể dùng Poisson distribution trong thực tế, đặt  $Y$  là số người sử dụng chương trình chia sẻ xe đạp trong một giờ cụ thể của ngày, dưới một tập điều kiện thời tiết cụ thể và trong một tháng cụ thể của năm. Ta có thể model  $Y$  bằng một Poisson distribution với mean  $E(Y) = \lambda = 5$ . Khi đó probability không có người dùng nào trong giờ cụ thể này là  $\Pr(Y = 0) = e^{-5}5^0/0! = e^{-5} = 0.0067$  (với  $0! = 1$  theo quy ước). Probability có đúng một người dùng là  $\Pr(Y = 1) = e^{-5}5^1/1! = 5e^{-5} = 0.034$ , probability có hai người dùng là  $\Pr(Y = 2) = e^{-5}5^2/2! = 0.084$ , và cứ tiếp tục như vậy.

Dĩ nhiên, trong thực tế, ta kỳ vọng mean số người sử dụng chương trình chia sẻ xe đạp,  $\lambda = E(Y)$ , thay đổi theo giờ trong ngày, tháng trong năm, điều kiện thời tiết, v.v. Vì vậy, thay vì model số người đi xe đạp  $Y$  bằng một Poisson distribution với mean cố định như  $\lambda = 5$ , ta muốn cho phép mean thay đổi như một hàm của các covariate. Cụ thể, ta xét model sau cho mean  $\lambda = E(Y)$ , mà giờ ta viết là  $\lambda(X_1, \dots, X_p)$  để nhấn mạnh rằng nó là một hàm của các covariate  $X_1, \dots, X_p$ :

$$\log(\lambda(X_1, \dots, X_p)) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p, \quad (4.36)$$

hoặc tương đương

$$\lambda(X_1, \dots, X_p) = e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}. \quad (4.37)$$

Ở đây,  $\beta_0, \beta_1, \dots, \beta_p$  là các parameter cần estimate. Cùng nhau, (4.35) và (4.36) định nghĩa Poisson regression model. Lưu ý rằng trong (4.36), ta lấy log của  $\lambda(X_1, \dots, X_p)$  là linear theo  $X_1, \dots, X_p$ , thay vì để chính  $\lambda(X_1, \dots, X_p)$  linear theo  $X_1, \dots, X_p$ ; điều này đảm bảo  $\lambda(X_1, \dots, X_p)$  nhận giá trị không âm với mọi giá trị của các covariate.

Để estimate các coefficient  $\beta_0, \beta_1, \dots, \beta_p$ , ta dùng cùng maximum likelihood approach đã sử dụng cho logistic regression trong Mục 4.3.2. Cụ thể, với  $n$  observation độc lập từ Poisson regression model, likelihood có dạng

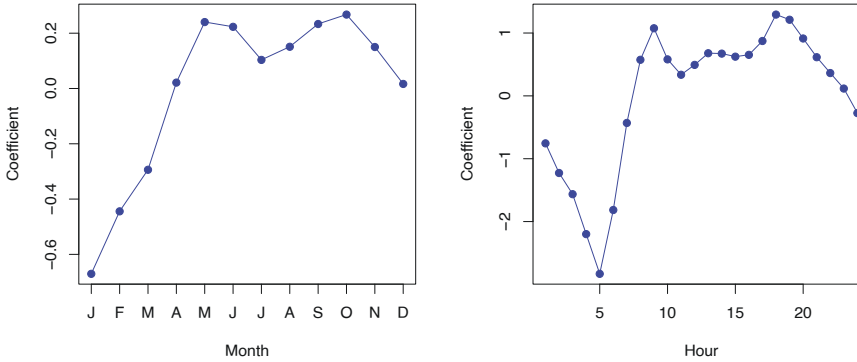
$$\ell(\beta_0, \beta_1, \dots, \beta_p) = \prod_{i=1}^n \frac{e^{-\lambda(x_i)} \lambda(x_i)^{y_i}}{y_i!}, \quad (4.38)$$

trong đó  $\lambda(x_i) = e^{\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}}$  theo (4.37). Ta estimate các coefficient làm cực đại likelihood  $\ell(\beta_0, \beta_1, \dots, \beta_p)$ , tức các coefficient khiến observed data có likelihood lớn nhất có thể.

Bây giờ ta fit một Poisson regression model cho dataset **Bikeshare**. Kết quả được trình bày trong Bảng 4.11 và Hình 4.15. Xét về mặt định tính, kết quả tương tự kết quả từ linear regression ở Mục 4.6.1. Ta một lần nữa thấy mức sử dụng xe đạp cao nhất vào mùa xuân và mùa thu và trong giờ cao điểm,

|                             | Coefficient | Std. error | z-statistic | p-value |
|-----------------------------|-------------|------------|-------------|---------|
| Intercept                   | 4.12        | 0.01       | 683.96      | 0.00    |
| workingday                  | 0.01        | 0.00       | 7.5         | 0.00    |
| temp                        | 0.79        | 0.01       | 68.43       | 0.00    |
| weathersit[cloudy/misty]    | -0.08       | 0.00       | -34.53      | 0.00    |
| weathersit[light rain/snow] | -0.58       | 0.00       | -141.91     | 0.00    |
| weathersit[heavy rain/snow] | -0.93       | 0.17       | -5.55       | 0.00    |

**BẢNG 4.11.** Kết quả của một Poisson regression model được fit để dự đoán **bikers** trong dữ liệu **Bikeshare**. Các predictor *mnth* và *hr* được lược khỏi bảng này do giới hạn không gian, và có thể xem trong Hình 4.15. Với qualitative variable *weathersit*, baseline tương ứng với trời quang.



**HÌNH 4.15.** Một Poisson regression model được fit để dự đoán **bikers** trong dataset **Bikeshare**. Trái: Các coefficient gắn với tháng trong năm. Mức sử dụng xe đạp cao nhất vào mùa xuân và mùa thu, và thấp nhất vào mùa đông. Phải: Các coefficient gắn với giờ trong ngày. Mức sử dụng xe đạp cao nhất trong các khung giờ đi làm cao điểm, và thấp nhất vào ban đêm.

và thấp nhất vào mùa đông và trong các giờ sáng sớm. Hơn nữa, mức sử dụng xe đạp tăng khi nhiệt độ tăng, và giảm khi thời tiết xấu đi. Điều thú vị là coefficient gắn với **workingday** có statistical significance trong Poisson regression model, nhưng không có statistical significance trong linear regression model.

Một số điểm khác biệt quan trọng giữa Poisson regression model và linear regression model như sau:

- **Interpretation:** Để diễn giải các coefficient trong Poisson regression model, ta phải chú ý kỹ đến (4.37), vốn phát biểu rằng khi  $X_j$  tăng một đơn vị thì  $E(Y) = \lambda$  thay đổi theo một hệ số  $\exp(\beta_j)$ . Chẳng hạn, việc thời tiết chuyển từ trời quang sang nhiều mây gắn với sự thay đổi mean mức sử dụng xe đạp theo hệ số  $\exp(-0.08) = 0.923$ , tức trung bình chỉ có 92.3% số người sử dụng xe đạp khi trời nhiều mây so với khi trời quang. Nếu thời tiết xấu thêm và bắt đầu mưa, thì mean mức sử dụng xe đạp tiếp tục thay đổi theo hệ số  $\exp(-0.5) = 0.607$ , tức trung bình chỉ có 60.7% số người sử dụng xe đạp khi trời mưa so với khi trời nhiều mây.

overdispersion

- *Mean–variance relationship*: Như đã đề cập trước đó, dưới Poisson model,  $\lambda = E(Y) = \text{Var}(Y)$ . Do đó, bằng cách model mức sử dụng xe đạp với Poisson regression, ta ngầm giả định rằng mean mức sử dụng xe đạp trong một giờ nhất định bằng variance của mức sử dụng xe đạp trong giờ đó. Trái lại, trong linear regression model, variance của mức sử dụng xe đạp luôn nhận một giá trị không đổi. Nhớ từ Hình 4.14 rằng trong dữ liệu **Bikeshare**, khi điều kiện đi xe đạp thuận lợi, cả mean lẫn variance của mức sử dụng xe đạp đều cao hơn nhiều so với khi điều kiện bất lợi. Vì vậy, Poisson regression model có thể xử lý quan hệ mean–variance quan sát được trong dữ liệu **Bikeshare** theo cách mà linear regression model không làm được.<sup>5</sup>
- *nonnegative fitted values*: Poisson regression model không tạo ra prediction âm. Lý do là bản thân Poisson model chỉ cho phép các giá trị không âm; xem (4.35). Trái lại, khi ta fit linear regression model cho dataset **Bikeshare**, gần 10% prediction là âm.

#### 4.6.3 Generalized Linear Models in Greater Generality

Đến đây ta đã thảo luận ba loại regression model: linear, logistic và Poisson. Các cách tiếp cận này có một số đặc điểm chung:

1. Mỗi cách tiếp cận dùng các predictor  $X_1, \dots, X_p$  để dự đoán response  $Y$ . Ta giả định rằng, conditional on  $X_1, \dots, X_p$ ,  $Y$  thuộc một family of distributions nhất định. Với linear regression, ta thường giả định  $Y$  tuân theo Gaussian hay normal distribution. Với logistic regression, ta giả định  $Y$  tuân theo Bernoulli distribution. Cuối cùng, với Poisson regression, ta giả định  $Y$  tuân theo Poisson distribution.
2. Mỗi cách tiếp cận model mean của  $Y$  như một hàm của các predictor. Trong linear regression, mean của  $Y$  có dạng

$$E(Y \mid X_1, \dots, X_p) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p, \quad (4.39)$$

tức là một hàm linear của các predictor. Với logistic regression, mean thay vào đó có dạng

$$\begin{aligned} E(Y \mid X_1, \dots, X_p) &= \Pr(Y = 1 \mid X_1, \dots, X_p) \\ &= \frac{e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}}, \end{aligned} \quad (4.40)$$

trong khi với Poisson regression, nó có dạng

$$E(Y \mid X_1, \dots, X_p) = \lambda(X_1, \dots, X_p) = e^{\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p}. \quad (4.41)$$

Các phương trình (4.39)–(4.41) có thể được biểu diễn bằng một *link function*,  $\eta$ , áp dụng một phép biến đổi lên  $E(Y \mid X_1, \dots, X_p)$  sao cho mean

<sup>5</sup>Trên thực tế, variance trong dữ liệu **Bikeshare** dường như cao hơn mean rất nhiều, một tình huống được gọi là *overdispersion*. Điều này khiến các Z-value trong Bảng 4.11 bị phóng đại. Một phân tích cẩn thận hơn nên tính đến overdispersion để thu được các Z-value chính xác hơn, và có nhiều method khác nhau để làm điều này. Tuy nhiên, chúng nằm ngoài phạm vi của cuốn sách này.

sau biến đổi là một hàm linear của các predictor. Cụ thể,

$$\eta(E(Y \mid X_1, \dots, X_p)) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p. \quad (4.42)$$

*link function*

Các link function cho linear, logistic và Poisson regression lần lượt là  $\eta(\mu) = \mu$ ,  $\eta(\mu) = \log(\mu/(1 - \mu))$ , và  $\eta(\mu) = \log(\mu)$ .

Gaussian, Bernoulli và Poisson distribution đều là thành viên của một class of distributions rộng hơn, được gọi là *exponential family*. Những thành viên nổi tiếng khác của family này gồm exponential distribution, Gamma distribution và negative binomial distribution. Tổng quát hơn, ta có thể thực hiện regression bằng cách model response  $Y$  như được sinh ra từ một thành viên cụ thể của exponential family, rồi biến đổi mean của response sao cho mean sau biến đổi là một hàm linear của các predictor thông qua (4.42). Bất kỳ regression approach nào tuân theo công thức rất tổng quát này đều được gọi là một *generalized linear model (GLM)*. Vì vậy, linear regression, logistic regression và Poisson regression là ba ví dụ về GLM. Các ví dụ khác không được trình bày ở đây gồm Gamma regression và negative binomial regression.

*exponential family*  
*exponential*  
*Gamma*  
*negative binomial*  
*generalized linear*  
*model*

## 4.7 Lab: Logistic Regression, LDA, QDA, and KNN

### 4.7.1 The Stock Market Data

Trong lab này, ta sẽ khảo sát dữ liệu **Smarket**, một phần của thư viện **ISLP**. Dataset này gồm percentage return của chỉ số chứng khoán S&P 500 trong 1,250 ngày, từ đầu năm 2001 đến cuối năm 2005. Với mỗi ngày, ta ghi lại percentage return của từng ngày trong năm ngày giao dịch trước đó, từ **Lag1** đến **Lag5**. Ta cũng ghi lại **Volume** (số cổ phiếu được giao dịch trong ngày trước đó, tính theo tỷ), **Today** (percentage return vào ngày đang xét) và **Direction** (thị trường đi Up hay Down trong ngày đó).

Ta bắt đầu bằng cách import các thư viện ở top level; đây đều là những import đã gặp trong các lab trước.

```
In [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
 summarize)
```

Ta cũng gom các import mới cần dùng cho lab này.

```
In [2]: from ISLP import confusion_table
from ISLP.models import contrast
from sklearn.discriminant_analysis import \
 (LinearDiscriminantAnalysis as LDA,
 QuadraticDiscriminantAnalysis as QDA)
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

Bây giờ ta đã sẵn sàng load dữ liệu **Smarket**.

```
In [3]: Smarket = load_data('Smarket')
Smarket
```

Lệnh này cho một listing dữ liệu đã bị rút gọn mà ta không hiển thị ở đây. Ta có thể xem tên các variable.

```
In [4]: Smarket.columns
```

```
Out[4]: Index(['Year', 'Lag1', 'Lag2', 'Lag3', 'Lag4', 'Lag5', 'Volume',
 'Today', 'Direction'],
 dtype='object')
```

Ta tính correlation matrix bằng method **corr()** cho data frame; method này tạo một matrix chứa toàn bộ pairwise correlation giữa các variable. (Ta không hiển thị output ở đây.) Thư viện **pandas** không báo correlation cho variable **Direction** vì đây là một qualitative variable.

`.corr()`

In [5]: `Smarket.corr()`

Đúng như kỳ vọng, correlation giữa các lagged return variable và return của ngày hôm nay đều gần bằng 0. Correlation đáng kể duy nhất là giữa **Year** và **Volume**. Khi plot dữ liệu, ta thấy **Volume** tăng theo thời gian. Nói cách khác, số cổ phiếu trung bình được giao dịch mỗi ngày tăng từ năm 2001 đến 2005.

In [6]: `Smarket.plot(y='Volume');`

### 4.7.2 Logistic Regression

Tiếp theo, ta fit một logistic regression model để dự đoán **Direction** bằng **Lag1** đến **Lag5** và **Volume**. Function `sm.GLM()` fit generalized linear model, một class of models bao gồm logistic regression. Ngoài ra, function `sm.Logit()` fit trực tiếp một logistic regression model. Syntax của `sm.GLM()` tương tự `sm.OLS()`, ngoại trừ việc ta phải truyền argument `family=sm.families.Binomial()` để cho `statsmodels` biết cần chạy logistic regression thay vì một generalized linear model loại khác.

*sm.GLM()  
generalized  
linear model*

In [7]:

```
allvars = Smarket.columns.drop(['Today', 'Direction', 'Year'])
design = MS(allvars)
X = design.fit_transform(Smarket)
y = Smarket.Direction == 'Up'
glm = sm.GLM(y,
 X,
 family=sm.families.Binomial())
results = glm.fit()
summarize(results)
```

Out[7]:

|           | coef    | std err | z      | P> z  |
|-----------|---------|---------|--------|-------|
| intercept | -0.1260 | 0.241   | -0.523 | 0.601 |
| Lag1      | -0.0731 | 0.050   | -1.457 | 0.145 |
| Lag2      | -0.0423 | 0.050   | -0.845 | 0.398 |
| Lag3      | 0.0111  | 0.050   | 0.222  | 0.824 |
| Lag4      | 0.0094  | 0.050   | 0.187  | 0.851 |
| Lag5      | 0.0103  | 0.050   | 0.208  | 0.835 |
| Volume    | 0.1354  | 0.158   | 0.855  | 0.392 |

$p$ -value nhỏ nhất ở đây gắn với **Lag1**. Coefficient âm của predictor này gợi ý rằng nếu thị trường có return dương vào hôm qua thì hôm nay khả năng thị trường tăng sẽ thấp hơn. Tuy nhiên, với giá trị 0.15,  $p$ -value vẫn tương đối lớn, vì vậy chưa có bằng chứng rõ ràng về một association thực sự giữa **Lag1** và **Direction**.

Ta dùng attribute `params` của `results` để truy cập chỉ các coefficient của fitted model này.

In [8]: `results.params`

Out[8]:

|           |           |
|-----------|-----------|
| intercept | -0.126000 |
| Lag1      | -0.073074 |
| Lag2      | -0.042301 |
| Lag3      | 0.011085  |
| Lag4      | 0.009359  |
| Lag5      | 0.010313  |

```
Volume 0.135441
dtype: float64
```

Tương tự, ta có thể dùng attribute `pvalues` để truy cập các  $p$ -value của coefficient (không hiển thị).

In [9]:

```
results.pvalues
```

Method `predict()` của `results` có thể được dùng để dự đoán probability rằng thị trường sẽ tăng, khi biết các giá trị của predictor. Method này trả về prediction trên probability scale. Nếu không cung cấp dataset cho function `predict()`, thì probability được tính cho training data đã dùng để fit logistic regression model. Tương tự linear regression, ta có thể truyền một optional argument `exog` phù hợp với một design matrix nếu muốn. Ở đây ta chỉ in mười probability đầu tiên.

In [10]:

```
probs = results.predict()
probs[:10]
```

Out[10]:

```
array([0.5070841, 0.4814679, 0.4811388, 0.5152223, 0.5107812,
 0.5069565, 0.4926509, 0.5092292, 0.5176135, 0.4888378])
```

Để tạo prediction về việc thị trường sẽ tăng hay giảm trong một ngày cụ thể, ta phải chuyển các predicted probability này thành class label, **Up** hoặc **Down**. Hai command sau tạo một vector class prediction dựa trên việc predicted probability của thị trường tăng lớn hơn hay nhỏ hơn 0.5.

In [11]:

```
labels = np.array(['Down']*1250)
labels[probs > 0.5] = "Up"
```

Function `confusion_table()` từ package ISLP tóm tắt các prediction này, cho biết bao nhiêu observation được classification đúng hoặc sai. Function của chúng ta, được adapted từ một function tương tự trong module `sklearn.metrics`, transpose matrix kết quả và thêm label cho hàng và cột. Function `confusion_table()` nhận argument thứ nhất là predicted label và argument thứ hai là true label.

`confusion_`  
`table()`

In [12]:

```
confusion_table(labels, Smarket.Direction)
```

Out[12]:

|           | Truth | Down | Up  |
|-----------|-------|------|-----|
| Predicted |       |      |     |
| Down      |       | 145  | 141 |
| Up        |       | 457  | 507 |

Các phần tử trên đường chéo của confusion matrix biểu thị prediction đúng, còn các phần tử ngoài đường chéo biểu thị prediction sai. Vì vậy model của ta dự đoán đúng rằng thị trường sẽ tăng trong 507 ngày và giảm trong 145 ngày, tổng cộng  $507 + 145 = 652$  prediction đúng. Function `np.mean()` có thể được dùng để tính tỷ lệ ngày mà prediction là đúng. Trong trường hợp này, logistic regression dự đoán đúng chuyển động của thị trường 52.2% số lần.



In [13]: `(507+145)/1250, np.mean(labels == Smarket.Direction)`

Out[13]: `(0.5216, 0.5216)`

Thoạt nhìn, có vẻ logistic regression model hoạt động tốt hơn random guessing một chút. Tuy nhiên, kết quả này gây hiểu nhầm vì ta train và test model trên cùng một tập gồm 1,250 observation. Nói cách khác,  $100 - 52.2 = 47.8\%$  là training error rate. Như đã thấy trước đây, training error rate thường quá lạc quan — nó có xu hướng đánh giá thấp test error rate. Để đánh giá tốt hơn accuracy của logistic regression model trong setting này, ta có thể fit model bằng một phần dữ liệu rồi xem nó dự đoán held-out data tốt đến đâu. Cách này sẽ cho một error rate thực tế hơn, theo nghĩa rằng trong thực tế ta quan tâm đến performance của model không phải trên dữ liệu đã dùng để fit model, mà trên những ngày tương lai mà chuyển động thị trường còn chưa biết.

Để triển khai strategy này, trước tiên ta tạo một Boolean vector tương ứng với các observation từ năm 2001 đến 2004. Sau đó dùng vector này để tạo một held-out dataset gồm các observation của năm 2005.

In [14]: 

```
train = (Smarket.Year < 2005)
Smarket_train = Smarket.loc[train]
Smarket_test = Smarket.loc[~train]
Smarket_test.shape
```

Out[14]: `(252, 9)`

Object `train` là một vector gồm 1,250 phần tử, tương ứng với các observation trong dataset. Các phần tử của vector tương ứng với observation xảy ra trước năm 2005 được đặt thành `True`, còn những phần tử tương ứng với observation trong năm 2005 được đặt thành `False`. Do đó `train` là một boolean array, vì các phần tử của nó là `True` và `False`. Boolean array có thể được dùng để lấy subset các row hoặc column của data frame bằng method `loc`.

Chẳng hạn, command `Smarket.loc[train]` sẽ chọn ra một submatrix của dataset thị trường chứng khoán chỉ tương ứng với các ngày trước năm 2005, vì đó là những ngày mà phần tử của `train` bằng `True`. Symbol `~` có thể được dùng để negate toàn bộ phần tử của một Boolean vector. Tức là, `~train` là một vector giống `train`, ngoại trừ các phần tử `True` trong `train` được đổi thành `False` trong `~train`, và ngược lại. Vì vậy, `Smarket.loc[~train]` cho một subset các row của data frame thị trường chứng khoán chỉ chứa những observation mà `train` bằng `False`. Output ở trên cho biết có 252 observation như vậy.

Bây giờ ta fit một logistic regression model chỉ bằng subset các observation tương ứng với các ngày trước năm 2005. Sau đó ta thu được predicted probability rằng thị trường chứng khoán sẽ tăng cho từng ngày trong test set — tức các ngày của năm 2005.

In [15]: 

```
X_train, X_test = X.loc[train], X.loc[~train]
y_train, y_test = y.loc[train], y.loc[~train]
glm_train = sm.GLM(y_train,
 X_train,
```



```
results = glm_train.fit()
probs = results.predict(exog=X_test)
labels = np.array(['Down']*252)
labels[probs > 0.5] = 'Up'
confusion_table(labels, L_test)
```

Out[19]:

|           | Truth | Down | Up  |
|-----------|-------|------|-----|
| Predicted |       |      |     |
| Down      |       | 35   | 35  |
| Up        |       | 76   | 106 |

Hãy đánh giá overall accuracy cũng như accuracy trong những ngày mà logistic regression dự đoán thị trường tăng.

In [20]:

```
(35+106)/252, 106/(106+76)
```

Out[20]:

```
(0.5595, 0.5824)
```

Bây giờ kết quả có vẻ tốt hơn một chút: 56% chuyển động hằng ngày được dự đoán đúng. Đáng chú ý là trong trường hợp này, một strategy đơn giản hơn rất nhiều — dự đoán thị trường sẽ tăng mỗi ngày — cũng đúng 56% số lần! Vì vậy, xét về overall error rate, method logistic regression không tốt hơn naive approach. Tuy nhiên, confusion matrix cho thấy vào những ngày logistic regression dự đoán thị trường tăng, nó đạt accuracy 58%. Điều này gợi ý một trading strategy khả dĩ: mua vào những ngày model dự đoán thị trường tăng và tránh giao dịch vào những ngày dự đoán giảm. Dĩ nhiên ta cần khảo sát cẩn thận hơn xem cải thiện nhỏ này là thực hay chỉ do random chance.

Giả sử ta muốn dự đoán return ứng với các giá trị cụ thể của **Lag1** và **Lag2**. Cụ thể, ta muốn dự đoán **Direction** vào một ngày mà **Lag1** và **Lag2** lần lượt bằng 1.2 và 1.1, và một ngày mà chúng bằng 1.5 và  $-0.8$ . Ta thực hiện việc này bằng function `predict()`.

In [21]:

```
newdata = pd.DataFrame({'Lag1': [1.2, 1.5],
 'Lag2': [1.1, -0.8]});
newX = model.transform(newdata)
results.predict(newX)
```

Out[21]:

```
0 0.4791
1 0.4961
dtype: float64
```

### 4.7.3 Linear Discriminant Analysis

Ta bắt đầu bằng cách thực hiện LDA trên dữ liệu **Smarket**, dùng function `LinearDiscriminantAnalysis()`, mà ta viết tắt là `LDA()`. Ta fit model chỉ bằng các observation trước năm 2005.

In [22]:

```
lda = LDA(store_covariance=True)
```

*Linear  
Discriminant  
Analysis()*

Vì LDA estimator tự động thêm intercept, ta nên loại column tương ứng với intercept trong cả **X\_train** lẫn **X\_test**. Ta cũng có thể dùng trực tiếp các label thay vì các Boolean vector **y\_train**.

```
In [23]: X_train, X_test = [M.drop(columns=['intercept'])
 for M in [X_train, X_test]]
 lda.fit(X_train, L_train)
```

```
Out[23]: LinearDiscriminantAnalysis(store_covariance=True)
```

Ở đây ta đã dùng list comprehension được giới thiệu trong Mục 3.6.4. Nhìn vào dòng đầu tiên phía trên, ta thấy về phải là một list có length bằng hai. Lý do là code `for M in [X_train, X_test]` iterate qua một list có length bằng hai. Dù ở đây ta loop qua một list, method list comprehension hoạt động khi loop qua bất kỳ iterable object nào. Sau đó ta áp dụng method `drop()` cho từng phần tử trong iteration và thu thập kết quả vào một list. Về trái bảo Python unpack list có length bằng hai này, gán các phần tử của nó cho variable `X_train` và `X_test`. Dĩ nhiên điều này overwrite các giá trị trước đó của `X_train` và `X_test`.

Sau khi fit model, ta có thể trích các mean trong hai class bằng attribute `means_`. Đây là average của từng predictor trong từng class, và được LDA dùng làm estimate của  $\mu_k$ . Các giá trị này gợi ý rằng return của hai ngày trước có xu hướng âm vào những ngày thị trường tăng, và return của những ngày trước có xu hướng dương vào những ngày thị trường giảm.

```
In [24]: lda.means_
```

```
Out[24]: array([[0.04, 0.03],
 [-0.04, -0.03]])
```

Estimated prior probability được lưu trong attribute `priors_`. Package `sklearn` thường dùng dấu `_` ở cuối như vậy để biểu thị một quantity được estimate khi dùng method `fit()`. Ta có thể chắc chắn entry nào tương ứng với label nào bằng cách xem attribute `classes_`.

```
In [25]: lda.classes_
```

```
Out[25]: array(['Down', 'Up'], dtype='<U4')
```

Output của LDA cho biết  $\hat{\pi}_{\text{Down}} = 0.492$  và  $\hat{\pi}_{\text{Up}} = 0.508$ .

```
In [26]: lda.priors_
```

```
Out[26]: array([0.492, 0.508])
```

Các linear discriminant vector có thể được tìm thấy trong attribute `scalings_`:

```
In [27]: lda.scalings_
```

```
Out[27]: array([[-0.642],
 [-0.513]])
```

Các giá trị này cho linear combination của `Lag1` và `Lag2` được dùng để

tạo LDA decision rule. Nói cách khác, đây là các multiplier của các phần tử của  $X = x$  trong (4.24). Nếu  $-0.64 \times \text{Lag1} - 0.51 \times \text{Lag2}$  lớn, LDA classifier sẽ dự đoán thị trường tăng; nếu nhỏ, LDA classifier sẽ dự đoán thị trường giảm.

```
In [28]: lda_pred = lda.predict(X_test)
```

Như ta đã thấy khi so sánh các classification method (Mục 4.5), prediction của LDA và logistic regression gần như giống hệt nhau.

```
In [29]: confusion_table(lda_pred, L_test)
```

```
Out[29]:
```

|           | Truth | Down | Up  |
|-----------|-------|------|-----|
| Predicted |       |      |     |
| Down      |       | 35   | 35  |
| Up        |       | 76   | 106 |

Ta cũng có thể estimate probability của mỗi class cho từng point trong một training set. Áp dụng threshold 50% cho posterior probability thuộc class một cho phép ta tái tạo các prediction chứa trong `lda_pred`.

```
In [30]: lda_prob = lda.predict_proba(X_test)
np.all(
 np.where(lda_prob[:,1] >= 0.5, 'Up', 'Down') == lda_pred
)
```

```
Out[30]: True
```

Ở trên, ta dùng function `np.where()` để tạo một array có giá trị 'Up' tại các index mà column thứ hai của `lda_prob` (estimated posterior probability của 'Up') lớn hơn 0.5. Với problem có nhiều hơn hai class, label được chọn là class có posterior probability lớn nhất:

*np.where()*

```
In [31]: np.all(
 [lda.classes_[i] for i in np.argmax(lda_prob, 1)] ==
 lda_pred
)
```

```
Out[31]: True
```

Nếu muốn dùng posterior probability threshold khác 50% để đưa ra prediction, ta có thể làm điều đó dễ dàng. Chẳng hạn, giả sử ta chỉ muốn dự đoán thị trường giảm nếu rất chắc chắn rằng thị trường thực sự sẽ giảm trong ngày đó — ví dụ nếu posterior probability ít nhất là 90%. Ta biết column đầu tiên của `lda_prob` tương ứng với label `Down` sau khi đã kiểm tra attribute `classes_`, vì vậy ta dùng column index 0 thay vì 1 như ở trên.

```
In [32]: np.sum(lda_prob[:,0] > 0.9)
```

```
Out[32]: 0
```

Không có ngày nào trong năm 2005 đạt threshold đó! Trên thực tế,

posterior probability lớn nhất cho việc thị trường giảm trong toàn bộ năm 2005 là 52.02%.

LDA classifier ở trên là classifier đầu tiên từ thư viện **sklearn**. Ta sẽ dùng thêm một số object khác từ thư viện này. Các object tuân theo một cấu trúc chung giúp đơn giản hóa những task như cross-validation, mà ta sẽ gặp trong Chương 5. Cụ thể, các method trước hết tạo một generic classifier mà chưa tham chiếu đến dữ liệu nào. Classifier này sau đó được fit vào dữ liệu bằng method `fit()`, và prediction luôn được tạo bằng method `predict()`. Pattern gồm việc đầu tiên instantiate classifier, sau đó fit nó, rồi tạo prediction là một design choice explicit của **sklearn**. Tính uniform này giúp ta có thể cleanly copy classifier để fit trên các dữ liệu khác nhau; ví dụ các training set khác nhau phát sinh trong cross-validation. Standard pattern này cũng cho phép hình thành workflow theo cách có thể dự đoán trước.

#### 4.7.4 Quadratic Discriminant Analysis

Bây giờ ta sẽ fit một QDA model cho dữ liệu `Smarket`. QDA được triển khai bằng `QuadraticDiscriminantAnalysis()` trong package `sklearn`, mà ta viết tắt là `QDA()`. Syntax rất giống `LDA()`.

*Quadratic  
Discriminant  
Analysis()*

```
In [33]: qda = QDA(store_covariance=True)
 qda.fit(X_train, L_train)
```

```
Out[33]: QuadraticDiscriminantAnalysis(store_covariance=True)
```

Function `QDA()` một lần nữa sẽ tính `means_` và `priors_`.

```
In [34]: qda.means_, qda.priors_
```

```
Out[34]: (array([[0.04279022, 0.03389409],
 [-0.03954635, -0.03132544]]),
 array([0.49198397, 0.50801603]))
```

Classifier `QDA()` sẽ estimate một covariance cho mỗi class. Dưới đây là estimated covariance của class thứ nhất:

```
In [35]: qda.covariance_[0]
```

```
Out[35]: array([[1.50662277, -0.03924806],
 [-0.03924806, 1.53559498]])
```

Output chứa các group mean. Tuy nhiên, nó không chứa các coefficient của linear discriminant, vì QDA classifier sử dụng một hàm quadratic của các predictor thay vì một hàm linear. Function `predict()` hoạt động hoàn toàn giống như với `LDA`.

```
In [36]: qda_pred = qda.predict(X_test)
 confusion_table(qda_pred, L_test)
```

```
Out[36]: Truth Down Up
Predicted
 Down 30 20
 Up 81 121
```

Điều thú vị là prediction của QDA chính xác gần 60% số lần, mặc dù dữ liệu năm 2005 không được dùng để fit model.

```
In [37]: np.mean(qda_pred == L_test)
```

```
Out[37]: 0.599
```

Mức accuracy này khá ấn tượng đối với stock market data, vốn nổi tiếng là rất khó model chính xác. Kết quả này gợi ý rằng dạng quadratic mà QDA giả định có thể nắm bắt relationship thực sự chính xác hơn các dạng linear mà `LDA` và logistic regression giả định. Tuy nhiên, ta khuyến nghị đánh giá performance của method này trên một test set lớn hơn trước khi đặt cược rằng approach này sẽ liên tục đánh bại thị trường!

### 4.7.5 Naive Bayes

Tiếp theo, ta fit một naive Bayes model cho dữ liệu **Smarket**. Syntax tương tự `LDA()` và `QDA()`. Theo mặc định, implementation `GaussianNB()` của naive Bayes classifier model mỗi quantitative feature bằng một Gaussian distribution. Tuy nhiên, cũng có thể dùng một kernel density method để estimate các distribution.

*GaussianNB()*

In [38]:

```
NB = GaussianNB()
NB.fit(X_train, L_train)
```

Out[38]:

GaussianNB()

Các class được lưu trong `classes_`.

In [39]:

```
NB.classes_
```

Out[39]:

```
array(['Down', 'Up'], dtype='<U4')
```

Class prior probability được lưu trong attribute `class_prior_`.

In [40]:

```
NB.class_prior_
```

Out[40]:

```
array([0.49, 0.51])
```

Các parameter của feature có thể được tìm thấy trong các attribute `theta_` và `var_`. Số row bằng số class, còn số column bằng số feature. Ta thấy bên dưới rằng mean của feature **Lag1** trong class **Down** là 0.043.

In [41]:

```
NB.theta_
```

Out[41]:

```
array([[0.043, 0.034],
 [-0.040, -0.031]])
```

Variance của nó là 1.503.

In [42]:

```
NB.var_
```

Out[42]:

```
array([[1.503, 1.532],
 [1.514, 1.487]])
```

Làm sao ta biết tên của các attribute này? Ta dùng `NB?` (hoặc `?NB`). Ta có thể dễ dàng verify phép tính mean:

In [43]:

```
X_train[L_train == 'Down'].mean()
```

Out[43]:

```
Lag1 0.042790
Lag2 0.033894
dtype: float64
```

Tương tự đối với variance:



```
In [44]: X_train[L_train == 'Down'].var(ddof=0)
```

```
Out[44]: Lag1 1.503554
Lag2 1.532467
dtype: float64
```

Function `GaussianNB()` tính variance bằng công thức  $1/n$ .<sup>6</sup> Vì `NB()` là một classifier trong thư viện `sklearn`, việc tạo prediction dùng cùng syntax như với `LDA()` và `QDA()` ở trên.

```
In [45]: nb_labels = NB.predict(X_test)
confusion_table(nb_labels, L_test)
```

```
Out[45]:
```

|           | Truth | Down | Up  |
|-----------|-------|------|-----|
| Predicted |       |      |     |
| Down      |       | 29   | 20  |
| Up        |       | 82   | 121 |

Naive Bayes hoạt động tốt trên các dữ liệu này, với prediction chính xác hơn 59% số lần. Kết quả này hơi kém QDA nhưng tốt hơn LDA nhiều.

Tương tự LDA, method `predict_proba()` estimate probability rằng mỗi observation thuộc về một class cụ thể.

```
In [46]: NB.predict_proba(X_test)[:5]
```

```
Out[46]: array([[0.4873, 0.5127],
[0.4762, 0.5238],
[0.4653, 0.5347],
[0.4748, 0.5252],
[0.4902, 0.5098]])
```

#### 4.7.6 *K-Nearest Neighbors*

Bây giờ ta sẽ thực hiện KNN bằng function `KNeighborsClassifier()`. Function này hoạt động tương tự các model-fitting function khác mà ta đã gặp cho đến đây.

Giống như với LDA và QDA, ta fit classifier bằng method `fit`. Prediction mới được tạo bằng method `predict` của object do `fit()` trả về.

*KNeighbors  
Classifier()*

```
In [47]: knn1 = KNeighborsClassifier(n_neighbors=1)
knn1.fit(X_train, L_train)
knn1_pred = knn1.predict(X_test)
confusion_table(knn1_pred, L_test)
```

```
Out[47]:
```

|           | Truth | Down | Up |
|-----------|-------|------|----|
| Predicted |       |      |    |
| Down      |       | 43   | 58 |
| Up        |       | 68   | 83 |

Kết quả với  $K = 1$  không tốt lắm, vì chỉ 50% observation được dự đoán

<sup>6</sup>Có hai công thức để tính sample variance của  $n$  observation  $x_1, \dots, x_n$ :  $\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$  và  $\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$ , trong đó  $\bar{x}$  là sample mean. Trong hầu hết trường hợp, sự khác biệt này không quan trọng.

đúng. Tất nhiên, có thể  $K = 1$  tạo ra một fit quá flexible đối với dữ liệu.

```
In [48]: (83+43)/252, np.mean(knn1_pred == L_test)
```

```
Out[48]: (0.5, 0.5)
```

Ta lặp lại analysis bên dưới với  $K = 3$ .

```
In [49]: knn3 = KNeighborsClassifier(n_neighbors=3)
knn3_pred = knn3.fit(X_train, L_train).predict(X_test)
np.mean(knn3_pred == L_test)
```

```
Out[49]: 0.532
```

Kết quả đã cải thiện đôi chút. Tuy nhiên, tăng  $K$  thêm nữa không đem lại cải thiện nào khác. Có vẻ rằng với dữ liệu này và train/test split này, QDA cho kết quả tốt nhất trong số các method mà ta đã khảo sát cho đến đây.

KNN không hoạt động tốt trên dữ liệu **Smarket**, nhưng nó thường có thể cho kết quả ấn tượng. Ví dụ, ta sẽ áp dụng KNN approach cho dataset **Caravan**, một phần của thư viện ISLP. Dataset này gồm 85 predictor đo các đặc trưng nhân khẩu học của 5,822 cá nhân. Response variable là **Purchase**, cho biết một cá nhân có mua một hợp đồng bảo hiểm caravan hay không. Trong dataset này, chỉ 6% số người mua bảo hiểm caravan.

```
In [50]: Caravan = load_data('Caravan')
Purchase = Caravan.Purchase
Purchase.value_counts()
```

```
Out[50]: No 5474
Yes 348
Name: Purchase, dtype: int64
```

Method `value_counts()` nhận một `pd.Series` hoặc `pd.DataFrame` và trả về một `pd.Series` với các count tương ứng cho từng unique element. Trong trường hợp này, **Purchase** chỉ có các giá trị **Yes** và **No**, và trả về số lượng của từng giá trị.

```
In [51]: 348 / 5822
```

```
Out[51]: 0.0598
```

Các feature của ta sẽ gồm mọi column ngoại trừ **Purchase**.

```
In [52]: feature_df = Caravan.drop(columns=['Purchase'])
```

Vì KNN classifier dự đoán class của một test observation bằng cách xác định các observation gần nó nhất, scale của các variable rất quan trọng. Bất kỳ variable nào có scale lớn sẽ có ảnh hưởng lớn hơn nhiều đến distance giữa các observation, và do đó đến KNN classifier, so với variable có scale nhỏ. Chẳng hạn, hãy tưởng tượng một dataset có hai variable là lương và tuổi (được đo lần lượt bằng đô la và năm). Đối với KNN, chênh lệch 1,000

USD về lương là cực lớn so với chênh lệch 50 năm về tuổi. Do đó lương sẽ chi phối kết quả KNN classification, còn tuổi gần như không có ảnh hưởng. Điều này trái với trực giác rằng chênh lệch 1,000 USD về lương khá nhỏ so với chênh lệch 50 năm về tuổi. Hơn nữa, tầm quan trọng của scale đối với KNN classifier dẫn đến một vấn đề khác: nếu ta đo lương bằng yên Nhật, hoặc đo tuổi bằng phút, thì kết quả classification sẽ rất khác so với khi hai variable này được đo bằng đô la và năm.

Một cách tốt để xử lý vấn đề này là standardize dữ liệu sao cho mọi variable đều có mean bằng 0 và standard deviation bằng 1. Khi đó mọi variable sẽ ở scale có thể so sánh được. Việc này được thực hiện bằng transformation `StandardScaler()`.

*standardize*

In [53]:

```
scaler = StandardScaler(with_mean=True,
 with_std=True,
 copy=True)
```

*Standard  
Scaler()*

Argument `with_mean` cho biết có trừ mean hay không, còn `with_std` cho biết có scale các column để có standard deviation bằng 1 hay không. Cuối cùng, argument `copy=True` cho biết ta luôn copy dữ liệu thay vì cố thực hiện calculation in place khi có thể.

Transformation này có thể được fit rồi áp dụng cho dữ liệu tùy ý. Ở dòng đầu bên dưới, các parameter cho scaling được tính và lưu trong `scaler`; dòng thứ hai thực sự dùng standardized feature set.

In [54]:

```
scaler.fit(feature_df)
X_std = scaler.transform(feature_df)
```

Bây giờ mọi column của `feature_std` bên dưới đều có standard deviation bằng 1 và mean bằng 0.

In [55]:

```
feature_std = pd.DataFrame(
 X_std,
 columns=feature_df.columns);
feature_std.std()
```

Out[55]:

```
MOSTYPE 1.000086
MAANTHUI 1.000086
MGEMOMV 1.000086
MGEMLEEF 1.000086
MOSHOOFD 1.000086
...
AZEILPL 1.000086
APLEZIER 1.000086
AFIETS 1.000086
AINBOED 1.000086
ABYSTAND 1.000086
Length: 85, dtype: float64
```

Lưu ý rằng standard deviation ở đây không hoàn toàn bằng 1; nguyên nhân một lần nữa là do một số procedure dùng convention  $1/n$  cho variance (trong trường hợp này là `scaler()`), trong khi những procedure khác dùng  $1/(n-1)$  (method `std()`). Xem footnote ở trang 183. Trong trường hợp này điều đó không quan trọng, miễn là mọi variable đều ở cùng scale.

*.std()*

Dùng function `train_test_split()`, bây giờ ta chia các observation thành một test set chứa 1000 observation và một training set chứa các

observation còn lại. Argument `random_state=0` bảo đảm rằng ta nhận được cùng một split mỗi lần chạy lại code.

`train_test_split()`

In [56]:

```
(X_train,
 X_test,
 y_train,
 y_test) = train_test_split(feature_std,
 Purchase,
 test_size=1000,
 random_state=0)
```

?`train_test_split` cho thấy các non-keyword argument có thể là list, array, pandas dataframe, v.v., tất cả đều có cùng length (`shape[0]`) và vì thế đều indexable. Trong trường hợp này chúng là dataframe `feature_std` và response variable `Purchase`. Ta fit một KNN model trên training data với  $K = 1$ , rồi đánh giá performance của nó trên test data.

`indexable`

In [57]:

```
knn1 = KNeighborsClassifier(n_neighbors=1)
knn1_pred = knn1.fit(X_train, y_train).predict(X_test)
np.mean(y_test != knn1_pred), np.mean(y_test != "No")
```

Out[57]:

```
(0.111, 0.067)
```

KNN error rate trên 1,000 test observation xấp xỉ 11%. Thoạt nhìn, kết quả này có vẻ khá tốt. Tuy nhiên, vì chỉ hơn 6% khách hàng mua bảo hiểm, ta có thể giảm error rate xuống gần 6% chỉ bằng cách luôn dự đoán No, bất kể giá trị predictor! Đây được gọi là null rate.

`null rate`

Giả sử có một chi phí không nhỏ khi cố bán bảo hiểm cho một cá nhân. Chẳng hạn, có thể nhân viên bán hàng phải đến gặp từng khách hàng tiềm năng. Nếu công ty cố bán bảo hiểm cho một tập khách hàng được chọn ngẫu nhiên thì tỷ lệ thành công chỉ khoảng 6%, có thể quá thấp so với chi phí bỏ ra. Thay vào đó, công ty muốn chỉ cố bán bảo hiểm cho những khách hàng có khả năng mua. Vì vậy overall error rate không phải điều ta quan tâm. Thay vào đó, điều đáng quan tâm là tỷ lệ những cá nhân được dự đoán đúng là sẽ mua bảo hiểm.

In [58]:

```
confusion_table(knn1_pred, y_test)
```

Out[58]:

|           | Truth | No  | Yes |
|-----------|-------|-----|-----|
| Predicted |       |     |     |
| No        |       | 880 | 58  |
| Yes       |       | 53  | 9   |

Hóa ra KNN với  $K = 1$  tốt hơn random guessing rất nhiều trong nhóm khách hàng được dự đoán sẽ mua bảo hiểm. Trong 62 khách hàng như vậy, có 9 người, tức 14.5%, thực sự mua bảo hiểm. Tỷ lệ này gấp đôi tỷ lệ thu được từ random guessing.

In [59]:

```
9/(53+9)
```

Out[59]:

```
0.145
```

### *Tuning Parameters*

Số neighbor trong KNN được gọi là một tuning parameter, cũng gọi là hyperparameter. Ta không biết trước nên dùng giá trị nào. Vì vậy, điều đáng quan tâm là xem classifier hoạt động thế nào trên test data khi ta thay đổi các parameter này. Có thể thực hiện việc đó bằng một for loop, được mô tả trong Mục 2.3.8. Ở đây ta dùng một for loop để xem accuracy của classifier trong nhóm được dự đoán sẽ mua bảo hiểm thay đổi thế nào khi số neighbor chạy từ 1 đến 5:

tuning  
parameter  
hyper-  
parameter

In [60]:

```
for K in range(1,6):
 knn = KNeighborsClassifier(n_neighbors=K)
 knn_pred = knn.fit(X_train, y_train).predict(X_test)
 C = confusion_table(knn_pred, y_test)
 templ = ('K={0:d}: # predicted to rent: {1:>2}, ' +
 ' # who did rent {2:d}, accuracy {3:.1%} ')
 pred = C.loc['Yes'].sum()
 did_rent = C.loc['Yes', 'Yes']
 print(templ.format(
 K,
 pred,
 did_rent,
 did_rent / pred))
```

Out[60]:

```
K=1: # predicted to rent: 62, # who did rent 9, accuracy 14.5%
K=2: # predicted to rent: 6, # who did rent 1, accuracy 16.7%
K=3: # predicted to rent: 20, # who did rent 3, accuracy 15.0%
K=4: # predicted to rent: 3, # who did rent 0, accuracy 0.0%
K=5: # predicted to rent: 7, # who did rent 1, accuracy 14.3%
```

Ta thấy có một mức variability nhất định — các con số với  $K = 4$  rất khác phần còn lại.

### Comparison to Logistic Regression

Để so sánh, ta cũng có thể fit một logistic regression model cho dữ liệu. Việc này cũng có thể làm bằng **sklearn**, mặc dù theo mặc định nó fit một thứ giống ridge regression version của logistic regression, sẽ được giới thiệu trong Chương 6. Có thể thay đổi điều này bằng cách đặt argument **C** phù hợp như bên dưới. Giá trị mặc định của nó là 1, nhưng khi đặt thành một số rất lớn, algorithm hội tụ về cùng solution như estimator logistic regression thông thường (unregularized) đã thảo luận ở trên.

Khác với package **statsmodels**, **sklearn** tập trung ít hơn vào inference và nhiều hơn vào classification. Do đó, các summary method thấy trong **statsmodels** và version đơn giản hóa của ta dùng **summarize** thường không có cho classifier trong **sklearn**.

In [61]:

```
logit = LogisticRegression(C=1e10, solver='liblinear')
logit.fit(X_train, y_train)
logit_pred = logit.predict_proba(X_test)
logit_labels = np.where(logit_pred[:,1] > 5, 'Yes', 'No')
confusion_table(logit_labels, y_test)
```

Out[61]:

|           | Truth | No  | Yes |
|-----------|-------|-----|-----|
| Predicted |       |     |     |
| No        |       | 933 | 67  |
| Yes       |       | 0   | 0   |

Ta dùng argument **solver='liblinear'** ở trên để tránh warning với default solver, warning này sẽ cho biết algorithm không converge.

Nếu dùng 0.5 làm predicted probability cut-off cho classifier thì ta gặp một vấn đề: không test observation nào được dự đoán sẽ mua insurance. Tuy nhiên, ta không bắt buộc phải dùng cut-off 0.5. Nếu thay vào đó ta dự đoán một khách hàng sẽ mua bất cứ khi nào predicted probability của việc mua vượt 0.25, ta thu được kết quả tốt hơn nhiều: ta dự đoán 29 người sẽ mua bảo hiểm và đúng với khoảng 31% trong số đó. Con số này tốt hơn random guessing gần năm lần!

In [62]:

```
logit_labels = np.where(logit_pred[:,1]>0.25, 'Yes', 'No')
confusion_table(logit_labels, y_test)
```

Out[62]:

|           | Truth | No  | Yes |
|-----------|-------|-----|-----|
| Predicted |       |     |     |
| No        |       | 913 | 58  |
| Yes       |       | 20  | 9   |

In [63]:

```
9/(20+9)
```

Out[63]:

```
0.310
```

### 4.7.7 Linear and Poisson Regression on the Bikeshare Data

Ở đây ta fit các linear regression model và Poisson regression model cho dữ liệu **Bikeshare**, như đã mô tả trong Mục 4.6. Response **bikers** đo số lượt thuê xe đạp mỗi giờ tại Washington, DC trong giai đoạn 2010–2012.

In [64]: `Bike = load_data('Bikeshare')`

Ta hãy xem nhanh kích thước và tên các variable trong dataframe này.

In [65]: `Bike.shape, Bike.columns`

Out[65]: `((8645, 15),  
Index(['season', 'mnth', 'day', 'hr', 'holiday', 'weekday',  
 'workingday', 'weathersit', 'temp', 'atemp', 'hum',  
 'windspeed', 'casual', 'registered', 'bikers'],  
 dtype='object'))`

#### Linear Regression

Ta bắt đầu bằng cách fit một linear regression model cho dữ liệu.

In [66]: `X = MS(['mnth',  
 'hr',  
 'workingday',  
 'temp',  
 'weathersit']).fit_transform(Bike)  
Y = Bike['bikers']  
M_lm = sm.OLS(Y, X).fit()  
summarize(M_lm)`

Out[66]:

|             | coef     | std err | t       | P> t  |
|-------------|----------|---------|---------|-------|
| intercept   | -68.6317 | 5.307   | -12.932 | 0.000 |
| mnth[Feb]   | 6.8452   | 4.287   | 1.597   | 0.110 |
| mnth[March] | 16.5514  | 4.301   | 3.848   | 0.000 |
| mnth[April] | 41.4249  | 4.972   | 8.331   | 0.000 |
| mnth[May]   | 72.5571  | 5.641   | 12.862  | 0.000 |
| mnth[June]  | 67.8187  | 6.544   | 10.364  | 0.000 |
| mnth[July]  | 45.3245  | 7.081   | 6.401   | 0.000 |
| mnth[Aug]   | 53.2430  | 6.640   | 8.019   | 0.000 |
| mnth[Sept]  | 66.6783  | 5.925   | 11.254  | 0.000 |
| mnth[Oct]   | 75.8343  | 4.950   | 15.319  | 0.000 |
| mnth[Nov]   | 60.3100  | 4.610   | 13.083  | 0.000 |
| mnth[Dec]   | 46.4577  | 4.271   | 10.878  | 0.000 |
| hr[1]       | -14.5793 | 5.699   | -2.558  | 0.011 |
| hr[2]       | -21.5791 | 5.733   | -3.764  | 0.000 |
| hr[3]       | -31.1408 | 5.778   | -5.389  | 0.000 |
| .....       | .....    | .....   | .....   | ..... |

Có 24 level của **hr** và tổng cộng 40 row, vì vậy ta đã truncate summary. Trong **M\_lm**, các level đầu tiên **hr[0]** và **mnth[Jan]** được xem là baseline value, nên không có coefficient estimate nào được báo cáo cho chúng: ngầm hiểu rằng coefficient estimate của chúng bằng 0, và mọi level khác được đo tương đối so với các baseline này. Ví dụ, coefficient 6.845 của **Feb** cho biết rằng, khi giữ mọi variable khác cố định, trung bình có khoảng 7 rider nhiều hơn vào tháng Hai so với tháng Một. Tương tự, có khoảng 16.5 rider nhiều hơn vào tháng Ba so với tháng Một.

Các kết quả thấy trong Mục 4.6.1 dùng một coding hơi khác cho các

variable `hr` và `mnth`, như sau:

```
In [67]: hr_encode = contrast('hr', 'sum')
mnth_encode = contrast('mnth', 'sum')
```

Ta refit lại:

```
In [68]: X2 = MS([mnth_encode,
 hr_encode,
 'workingday',
 'temp',
 'weathersit']).fit_transform(Bike)
M2_lm = sm.OLS(Y, X2).fit()
S2 = summarize(M2_lm)
S2
```

```
Out[68]:
```

|             | coef      | std err | t       | P> t  |
|-------------|-----------|---------|---------|-------|
| intercept   | 73.5974   | 5.132   | 14.340  | 0.000 |
| mnth[Jan]   | -46.0871  | 4.085   | -11.281 | 0.000 |
| mnth[Feb]   | -39.2419  | 3.539   | -11.088 | 0.000 |
| mnth[March] | -29.5357  | 3.155   | -9.361  | 0.000 |
| mnth[April] | -4.6622   | 2.741   | -1.701  | 0.089 |
| mnth[May]   | 26.4700   | 2.851   | 9.285   | 0.000 |
| mnth[June]  | 21.7317   | 3.465   | 6.272   | 0.000 |
| mnth[July]  | -0.7626   | 3.908   | -0.195  | 0.845 |
| mnth[Aug]   | 7.1560    | 3.535   | 2.024   | 0.043 |
| mnth[Sept]  | 20.5912   | 3.046   | 6.761   | 0.000 |
| mnth[Oct]   | 29.7472   | 2.700   | 11.019  | 0.000 |
| mnth[Nov]   | 14.2229   | 2.860   | 4.972   | 0.000 |
| hr[0]       | -96.1420  | 3.955   | -24.307 | 0.000 |
| hr[1]       | -110.7213 | 3.966   | -27.916 | 0.000 |
| hr[2]       | -117.7212 | 4.016   | -29.310 | 0.000 |
| .....       | .....     | .....   | .....   | ..... |

Hai cách coding khác nhau như thế nào? Trong `M2_lm`, coefficient estimate được báo cáo cho mọi level ngoại trừ level 23 của `hr` và level Dec của `mnth`. Quan trọng là trong `M2_lm`, coefficient estimate (không được báo cáo) cho level cuối của `mnth` không bằng 0; thay vào đó, nó bằng số âm của tổng các coefficient estimate cho mọi level còn lại. Tương tự, trong `M2_lm`, coefficient estimate cho level cuối của `hr` là số âm của tổng các coefficient estimate cho mọi level còn lại. Điều này có nghĩa các coefficient của `hr` và `mnth` trong `M2_lm` luôn có tổng bằng 0, và có thể được diễn giải là độ chênh so với mean level. Ví dụ, coefficient  $-46.087$  cho tháng Một cho biết rằng, khi giữ mọi variable khác cố định, thông thường có ít hơn 46 rider vào tháng Một so với mức trung bình hằng năm.

Cần nhận ra rằng lựa chọn coding thực sự không quan trọng, miễn là ta diễn giải model output đúng theo coding được sử dụng. Ví dụ, ta thấy prediction từ linear model là như nhau bất kể coding:

```
In [69]: np.sum(M_lm.fittedvalues - M2_lm.fittedvalues)**2
```

```
Out[69]: 1.53e-20
```

Tổng squared difference bằng 0. Ta cũng có thể thấy điều này bằng function `np.allclose()`:

```
np.allclose()
```



```
In [70]: np.allclose(M_lm.fittedvalues , M2_lm.fittedvalues)
```

```
Out[70]: True
```

Để tái tạo phía bên trái của Hình 4.13, trước hết ta phải lấy các coefficient estimate gắn với `mnth`. Các coefficient từ tháng Một đến tháng Mười Một có thể lấy trực tiếp từ object `M2_lm`. Coefficient của tháng Mười Hai phải được tính tường minh bằng số âm của tổng tất cả các tháng còn lại. Trước tiên ta extract mọi coefficient của month từ các coefficient của `M2_lm`.

```
In [71]: coef_month = S2[S2.index.str.contains('mnth')]['coef']
coef_month
```

```
Out[71]: mnth[Jan] -46.0871
mnth[Feb] -39.2419
mnth[March] -29.5357
mnth[April] -4.6622
mnth[May] 26.4700
mnth[June] 21.7317
mnth[July] -0.7626
mnth[Aug] 7.1560
mnth[Sept] 20.5912
mnth[Oct] 29.7472
mnth[Nov] 14.2229
Name: coef, dtype: float64
```

Tiếp theo, ta append Dec dưới dạng số âm của tổng mọi tháng còn lại.

```
In [72]: months = Bike['mnth'].dtype.categories
coef_month = pd.concat([
 coef_month,
 pd.Series([-coef_month.sum()],
 index=['mnth[Dec]'],
)
])
coef_month
```

```
Out[72]: mnth[Jan] -46.0871
mnth[Feb] -39.2419
mnth[March] -29.5357
mnth[April] -4.6622
mnth[May] 26.4700
mnth[June] 21.7317
mnth[July] -0.7626
mnth[Aug] 7.1560
mnth[Sept] 20.5912
mnth[Oct] 29.7472
mnth[Nov] 14.2229
mnth[Dec] 0.3705
Name: coef, dtype: float64
```

Cuối cùng, để plot gọn hơn, ta chỉ dùng chữ cái đầu tiên của mỗi tháng, tức entry thứ 6 của mỗi label trong index.

```
In [73]: fig_month, ax_month = subplots(figsize=(8,8))
x_month = np.arange(coef_month.shape[0])
ax_month.plot(x_month, coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize
```

```

=20)
ax_month.set_xlabel('Month', fontsize=20)
ax_month.set_ylabel('Coefficient', fontsize=20);

```

Việc tái tạo plot phía bên phải trong Hình 4.13 theo một process tương tự.

```

In [74]:
coef_hr = S2[S2.index.str.contains('hr')]['coef']
coef_hr = coef_hr.reindex(['hr[{}]' .format(h) for h in range(23)])
coef_hr = pd.concat([coef_hr,
 pd.Series([-coef_hr.sum()], index=['hr[23]'])
])

```

Bây giờ ta tạo hour plot.

```

In [75]:
fig_hr, ax_hr = subplots(figsize=(8,8))
x_hr = np.arange(coef_hr.shape[0])
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticks(x_hr[::2])
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);

```

### *Poisson Regression*

Bây giờ thay vào đó ta fit một Poisson regression model cho dữ liệu Bikeshare. Hầu như không có gì thay đổi, ngoại trừ việc giờ đây ta dùng function `sm.GLM()` với Poisson family được chỉ định:

```

In [76]:
M_pois = sm.GLM(Y, X2, family=sm.families.Poisson()).fit()

```

Ta có thể plot các coefficient gắn với `mnth` và `hr` để tái tạo Hình 4.15. Trước hết ta hoàn thiện các coefficient này như trước.

```

In [77]:
S_pois = summarize(M_pois)
coef_month = S_pois[S_pois.index.str.contains('mnth')]['coef']
coef_month = pd.concat([coef_month,
 pd.Series([-coef_month.sum()],
 index=['mnth[Dec]'])])
coef_hr = S_pois[S_pois.index.str.contains('hr')]['coef']
coef_hr = pd.concat([coef_hr,
 pd.Series([-coef_hr.sum()],
 index=['hr[23]'])])

```

Việc plot giống như trước.

```

In [78]:
fig_pois, (ax_month, ax_hr) = subplots(1, 2, figsize=(16,8))
ax_month.plot(x_month, coef_month, marker='o', ms=10)
ax_month.set_xticks(x_month)
ax_month.set_xticklabels([l[5] for l in coef_month.index], fontsize
 =20)
ax_month.set_xlabel('Month', fontsize=20)
ax_month.set_ylabel('Coefficient', fontsize=20)
ax_hr.plot(x_hr, coef_hr, marker='o', ms=10)
ax_hr.set_xticklabels(range(24)[::2], fontsize=20)
ax_hr.set_xlabel('Hour', fontsize=20)
ax_hr.set_ylabel('Coefficient', fontsize=20);

```

Ta so sánh fitted value của hai model. Fitted value được lưu trong attribute `fittedvalues` do method `fit()` trả về cho cả linear regression

fit lần Poisson fit. Linear predictor được lưu dưới attribute `lin_pred`.

In [79]:

```
fig, ax = subplots(figsize=(8, 8))
ax.scatter(M2_lm.fittedvalues ,
 M_pois.fittedvalues,
 s=20)
ax.set_xlabel('Linear Regression Fit', fontsize=20)
ax.set_ylabel('Poisson Regression Fit', fontsize=20)
ax.axline([0,0], c='black', linewidth=3,
 linestyle='--', slope=1);
```

Các prediction từ Poisson regression model có tương quan với các prediction từ linear model; tuy nhiên, prediction của Poisson model là non-negative. Vì vậy, Poisson regression prediction có xu hướng lớn hơn linear model prediction ở cả những mức ridership rất thấp lẫn rất cao.

Trong subsection này, ta fit Poisson regression model bằng function `sm.GLM()` với argument `family=sm.families.Poisson()`. Trước đó trong lab, ta dùng function `sm.GLM()` với `family=sm.families.Binomial()` để thực hiện logistic regression. Các lựa chọn khác cho argument `family` có thể được dùng để fit những loại GLM khác. Chẳng hạn, `family=sm.families.Gamma()` fit một Gamma regression model.

## 4.8 Exercises

### Conceptual

1. Dùng một chút đại số, hãy chứng minh rằng (4.2) tương đương với (4.3). Nói cách khác, biểu diễn logistic function và biểu diễn logit của logistic regression model là tương đương.
2. Trong phần nội dung trước, ta đã nói rằng việc classification một observation vào class mà (4.17) lớn nhất tương đương với việc classification observation đó vào class mà (4.18) lớn nhất. Hãy chứng minh điều này. Nói cách khác, dưới giả định rằng các observation trong class thứ  $k$  được rút ra từ distribution  $N(\mu_k, \sigma^2)$ , Bayes classifier gán một observation vào class mà discriminant function đạt cực đại.

3. Bài này liên quan đến QDA model, trong đó các observation thuộc mỗi class được rút ra từ một normal distribution với mean vector riêng cho class và covariance matrix riêng cho class. Ta xét trường hợp đơn giản khi  $p = 1$ ; tức là chỉ có một feature.

Giả sử ta có  $K$  class, và nếu một observation thuộc class thứ  $k$  thì  $X$  đến từ một one-dimensional normal distribution,  $X \sim N(\mu_k, \sigma_k^2)$ . Nhắc lại rằng density function của one-dimensional normal distribution được cho trong (4.16). Hãy chứng minh rằng trong trường hợp này, Bayes classifier không phải là linear. Hãy lập luận rằng trên thực tế nó là quadratic.

*Hint:* Với bài này, hãy đi theo các lập luận được trình bày trong Mục 4.4.1, nhưng không đưa ra giả định  $\sigma_1^2 = \dots = \sigma_K^2$ .

4. Khi số feature  $p$  lớn, performance của KNN và các local approach khác có xu hướng suy giảm; các approach này thực hiện prediction chỉ bằng những observation ở gần test observation cần được dự đoán. Hiện tượng này được gọi là *curse of dimensionality*, và nó liên hệ với thực tế rằng các non-parametric approach thường hoạt động kém khi  $p$  lớn. Bây giờ ta sẽ khảo sát lời nguyên này.

- (a) Giả sử ta có một tập observation, mỗi observation có measurement trên  $p = 1$  feature,  $X$ . Ta giả sử  $X$  được phân bố đều (uniformly) trên  $[0, 1]$ . Mỗi observation có một response value tương ứng. Giả sử ta muốn dự đoán response của một test observation chỉ bằng các observation nằm trong 10% range của  $X$  gần test observation đó nhất. Chẳng hạn, để dự đoán response cho một test observation với  $X = 0.6$ , ta sẽ dùng các observation trong range  $[0.55, 0.65]$ . Trung bình, ta sẽ dùng bao nhiêu phần của các observation sẵn có để thực hiện prediction?

- (b) Bây giờ giả sử ta có một tập observation, mỗi observation có measurement trên  $p = 2$  feature,  $X_1$  và  $X_2$ . Ta giả sử  $(X_1, X_2)$  được phân bố đều trên  $[0, 1] \times [0, 1]$ . Ta muốn dự đoán response của một test observation chỉ bằng các observation nằm trong 10% range của  $X_1$  và trong 10% range của  $X_2$  gần test observation đó nhất. Chẳng hạn, để dự đoán response cho một test observation với  $X_1 = 0.6$  và  $X_2 = 0.35$ , ta sẽ dùng các observation trong



range  $[0.55, 0.65]$  đối với  $X_1$  và trong range  $[0.3, 0.4]$  đối với  $X_2$ . Trung bình, ta sẽ dùng bao nhiêu phần của các observation sẵn có để thực hiện prediction?

- (c) Bây giờ giả sử ta có một tập observation trên  $p = 100$  feature. Một lần nữa, các observation được phân bố đều trên từng feature, và mỗi feature lại nhận giá trị từ 0 đến 1. Ta muốn dự đoán response của một test observation bằng các observation nằm trong 10% range của từng feature gần test observation đó nhất. Ta sẽ dùng bao nhiêu phần của các observation sẵn có để thực hiện prediction?
- (d) Dựa trên các câu trả lời ở (a)–(c), hãy lập luận rằng một nhược điểm của KNN khi  $p$  lớn là có rất ít training observation “gần” một test observation bất kỳ.
- (e) Bây giờ giả sử ta muốn thực hiện prediction cho một test observation bằng cách tạo một  $p$ -dimensional hypercube có tâm tại test observation và trung bình chứa 10% training observation. Với  $p = 1, 2$ , và 100, độ dài mỗi cạnh của hypercube là bao nhiêu? Hãy nhận xét về câu trả lời của bạn.  
*Note:* Hypercube là dạng tổng quát hóa của cube cho số chiều tùy ý. Khi  $p = 1$ , hypercube chỉ là một line segment; khi  $p = 2$  nó là một square; và khi  $p = 100$  nó là một 100-dimensional cube.

5. Bây giờ ta xem xét sự khác biệt giữa LDA và QDA.
  - (a) Nếu Bayes decision boundary là linear, ta kỳ vọng LDA hay QDA hoạt động tốt hơn trên training set? Trên test set?
  - (b) Nếu Bayes decision boundary là non-linear, ta kỳ vọng LDA hay QDA hoạt động tốt hơn trên training set? Trên test set?
  - (c) Nói chung, khi sample size  $n$  tăng, ta kỳ vọng test prediction accuracy của QDA so với LDA sẽ cải thiện, suy giảm, hay không đổi? Tại sao?
  - (d) Đúng hay Sai: Ngay cả khi Bayes decision boundary của một problem là linear, có lẽ ta vẫn đạt test error rate tốt hơn khi dùng QDA thay vì LDA vì QDA đủ flexible để model một linear decision boundary. Hãy giải thích câu trả lời.
6. Giả sử ta thu thập dữ liệu cho một nhóm sinh viên trong lớp thống kê với các variable  $X_1 =$  số giờ học,  $X_2 =$  undergrad GPA, và  $Y =$  nhận điểm A. Ta fit một logistic regression và thu được các estimated coefficient,  $\hat{\beta}_0 = -6$ ,  $\hat{\beta}_1 = 0.05$ ,  $\hat{\beta}_2 = 1$ .
  - (a) Hãy estimate probability để một sinh viên học 40 giờ và có undergrad GPA là 3.5 nhận được điểm A trong lớp.
  - (b) Sinh viên ở phần (a) cần học bao nhiêu giờ để có 50% chance nhận được điểm A trong lớp?
7. Giả sử ta muốn dự đoán liệu một stock nhất định có phát hành dividend trong năm nay (“Yes” hoặc “No”) dựa trên  $X$ , phần trăm profit của năm trước. Ta xem xét một số lượng lớn company và phát hiện rằng mean value của  $X$  đối với các company đã phát hành dividend là

$\bar{X} = 10$ , trong khi mean của các company không phát hành là  $\bar{X} = 0$ . Ngoài ra, variance của  $X$  đối với hai tập company này là  $\hat{\sigma}^2 = 36$ . Cuối cùng, 80% company đã phát hành dividend. Giả sử  $X$  tuân theo normal distribution, hãy dự đoán probability một company sẽ phát hành dividend trong năm nay khi phần trăm profit của năm trước là  $X = 4$ .

*Hint:* Nhắc lại rằng density function của một normal random variable là

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\mu)^2/(2\sigma^2)}.$$

Bạn sẽ cần dùng Bayes' theorem.

8. Giả sử ta lấy một dataset, chia thành training set và test set có kích thước bằng nhau, rồi thử hai classification procedure khác nhau. Đầu tiên ta dùng logistic regression và thu được error rate 20% trên training data và 30% trên test data. Tiếp theo ta dùng 1-nearest neighbors (tức  $K = 1$ ) và thu được average error rate (lấy trung bình trên cả test và training dataset) là 18%. Dựa trên các kết quả này, ta nên ưu tiên method nào để classification các observation mới? Tại sao?
9. Bài này liên quan đến odds.
  - (a) Trung bình, tỷ lệ bao nhiêu trong số những người có odds 0.37 về việc default khoản thanh toán credit card sẽ thực sự default?
  - (b) Giả sử một người có 16% chance default khoản thanh toán credit card. Odds để người đó default là bao nhiêu?
10. Equation 4.32 đã suy ra một biểu thức cho

$$\log\left(\frac{\Pr(Y = k \mid X = x)}{\Pr(Y = K \mid X = x)}\right)$$

trong setting  $p > 1$ , vì thế mean của class thứ  $k$ ,  $\mu_k$ , là một  $p$ -dimensional vector, còn shared covariance  $\Sigma$  là một  $p \times p$  matrix. Tuy nhiên, trong setting  $p = 1$ , (4.32) có dạng đơn giản hơn vì các mean  $\mu_1, \dots, \mu_K$  và variance  $\sigma^2$  là scalar. Trong setting đơn giản hơn này, hãy lặp lại phép tính trong (4.32), và đưa ra các biểu thức cho  $a_k$  và  $b_{kj}$  theo  $\pi_k$ ,  $\pi_K$ ,  $\mu_k$ ,  $\mu_K$ , và  $\sigma^2$ .

11. Hãy tính dạng chi tiết của  $a_k$ ,  $b_{kj}$ , và  $b_{kjl}$  trong (4.33). Câu trả lời của bạn nên chứa  $\pi_k$ ,  $\pi_K$ ,  $\mu_k$ ,  $\mu_K$ ,  $\Sigma_k$ , và  $\Sigma_K$ .
12. Giả sử bạn muốn classification một observation  $X \in \mathbb{R}$  thành *apples* và *oranges*. Bạn fit một logistic regression model và tìm được

$$\widehat{\Pr}(Y = \text{orange} \mid X = x) = \frac{\exp(\hat{\beta}_0 + \hat{\beta}_1 x)}{1 + \exp(\hat{\beta}_0 + \hat{\beta}_1 x)}.$$

Bạn của bạn fit một logistic regression model cho cùng dữ liệu bằng softmax formulation trong (4.13), và tìm được

$$\widehat{\Pr}(Y = \text{orange} \mid X = x) = \frac{\exp(\hat{\alpha}_{\text{orange}0} + \hat{\alpha}_{\text{orange}1}x)}{\exp(\hat{\alpha}_{\text{orange}0} + \hat{\alpha}_{\text{orange}1}x) + \exp(\hat{\alpha}_{\text{apple}0} + \hat{\alpha}_{\text{apple}1}x)}.$$

- (a) Log odds của orange so với apple trong model của bạn là gì?
- (b) Log odds của orange so với apple trong model của người bạn là gì?
- (c) Giả sử trong model của bạn,  $\hat{\beta}_0 = 2$  và  $\hat{\beta}_1 = -1$ . Các coefficient estimate trong model của người bạn là gì? Hãy cụ thể nhất có thể.
- (d) Bây giờ giả sử bạn và người bạn fit cùng hai model trên một dataset khác. Lần này, người bạn thu được các coefficient estimate  $\hat{\alpha}_{\text{orange0}} = 1.2$ ,  $\hat{\alpha}_{\text{orange1}} = -2$ ,  $\hat{\alpha}_{\text{orange0}} = 3$ ,  $\hat{\alpha}_{\text{orange1}} = 0.6$ . Các coefficient estimate trong model của bạn là gì?
- (e) Cuối cùng, giả sử bạn apply cả hai model ở (d) cho một dataset có 2,000 test observation. Bạn kỳ vọng predicted class label từ model của mình đồng ý với model của người bạn trong bao nhiêu phần thời gian? Hãy giải thích câu trả lời.

### Applied

13. Câu hỏi này cần được trả lời bằng dataset **Weekly**, là một phần của package **ISLP**. Dữ liệu này tương tự về bản chất với dữ liệu **Smarket** trong lab của chapter này, ngoại trừ việc nó chứa 1,089 weekly return trong 21 năm, từ đầu năm 1990 đến cuối năm 2010.
  - (a) Hãy tạo một số numerical và graphical summary của dữ liệu **Weekly**. Có pattern nào xuất hiện không?
  - (b) Dùng full dataset để thực hiện logistic regression với **Direction** làm response và năm lag variable cộng với **Volume** làm predictor. Dùng summary function để in kết quả. Có predictor nào có vẻ statistically significant không? Nếu có, predictor nào?
  - (c) Tính confusion matrix và overall fraction của các prediction đúng. Hãy giải thích confusion matrix cho biết điều gì về các loại mistake do logistic regression tạo ra.
  - (d) Bây giờ fit logistic regression model bằng training data period từ 1990 đến 2008, với **Lag2** là predictor duy nhất. Tính confusion matrix và overall fraction của các prediction đúng cho held-out data (tức dữ liệu từ 2009 và 2010).
  - (e) Lặp lại (d) bằng LDA.
  - (f) Lặp lại (d) bằng QDA.
  - (g) Lặp lại (d) bằng KNN với  $K = 1$ .
  - (h) Lặp lại (d) bằng naive Bayes.
  - (i) Method nào trong số này có vẻ cho kết quả tốt nhất trên dữ liệu này?
  - (j) Hãy thử nghiệm các combination khác nhau của predictor, bao gồm các transformation và interaction có thể có, cho từng method. Báo cáo các variable, method, và confusion matrix tương ứng có vẻ cho kết quả tốt nhất trên held-out data. Lưu ý rằng bạn cũng nên thử nghiệm các giá trị của  $K$  trong KNN

classifier.

14. Trong bài này, bạn sẽ phát triển một model để dự đoán liệu một car cho trước có gas mileage cao hay thấp dựa trên dataset **Auto**.

- (a) Tạo binary variable **mpg01**, nhận giá trị 1 nếu **mpg** có giá trị lớn hơn median của nó, và 0 nếu **mpg** có giá trị nhỏ hơn median. Bạn có thể tính median bằng method **median()** của dataframe. Lưu ý rằng có thể sẽ hữu ích khi thêm column **mpg01** vào dataframe bằng assignment. Giả sử bạn đã lưu dataframe dưới tên **Auto**, điều này có thể được thực hiện như sau:

```
Auto['mpg01'] = mpg01
```

- (b) Khám phá dữ liệu bằng graphical analysis để khảo sát association giữa **mpg01** và các feature khác. Feature nào trong số các feature còn lại có vẻ hữu ích nhất để dự đoán **mpg01**? Scatterplot và boxplot có thể là các tool hữu ích để trả lời câu hỏi này. Hãy mô tả các phát hiện của bạn.
- (c) Chia dữ liệu thành training set và test set.
- (d) Thực hiện LDA trên training data để dự đoán **mpg01** bằng các variable có vẻ association mạnh nhất với **mpg01** trong (b). Test error của model thu được là bao nhiêu?
- (e) Thực hiện QDA trên training data để dự đoán **mpg01** bằng các variable có vẻ association mạnh nhất với **mpg01** trong (b). Test error của model thu được là bao nhiêu?
- (f) Thực hiện logistic regression trên training data để dự đoán **mpg01** bằng các variable có vẻ association mạnh nhất với **mpg01** trong (b). Test error của model thu được là bao nhiêu?
- (g) Thực hiện naive Bayes trên training data để dự đoán **mpg01** bằng các variable có vẻ association mạnh nhất với **mpg01** trong (b). Test error của model thu được là bao nhiêu?
- (h) Thực hiện KNN trên training data với một số giá trị của  $K$  để dự đoán **mpg01**. Chỉ dùng các variable có vẻ association mạnh nhất với **mpg01** trong (b). Bạn thu được những test error nào? Giá trị  $K$  nào có vẻ hoạt động tốt nhất trên dataset này?
15. Bài này liên quan đến việc viết function.
- (a) Viết một function, **Power()**, in ra kết quả của việc nâng 2 lên lũy thừa 3. Nói cách khác, function của bạn phải tính  $2^3$  và in kết quả.  
*Hint:* Nhắc lại rằng  $x**a$  nâng  $x$  lên lũy thừa  $a$ . Dùng function **print()** để hiển thị kết quả.
- (b) Tạo một function mới, **Power2()**, cho phép bạn truyền vào hai số bất kỳ,  $x$  và  $a$ , rồi in giá trị  $x**a$ . Bạn có thể làm điều này bằng cách bắt đầu function bằng dòng

```
def Power2(x, a):
```

Bạn có thể gọi function bằng cách nhập, chẳng hạn,

```
Power2(3, 8)
```



trên command line. Lệnh này phải output giá trị của  $3^8$ , tức 6,561.

- (c) Dùng function `Power2()` vừa viết để tính  $10^3$ ,  $8^{17}$ , và  $13^{13}$ .
- (d) Bây giờ tạo một function mới, `Power3()`, thực sự *return* kết quả  $x**a$  dưới dạng một Python object, thay vì chỉ in nó ra screen. Tức là, nếu bạn lưu giá trị  $x**a$  trong một object tên `result` bên trong function, thì bạn chỉ cần return result bằng dòng sau:

```
return result
```

*return*

Lưu ý rằng dòng trên phải là dòng cuối cùng trong function, và nó phải được indent 4 space.

- (e) Bây giờ dùng function `Power3()` để tạo plot của  $f(x) = x^2$ . Trục  $x$  phải hiển thị range các integer từ 1 đến 10, và trục  $y$  phải hiển thị  $x^2$ . Label các axis một cách phù hợp, và dùng title phù hợp cho figure. Hãy cân nhắc hiển thị trục  $x$ , trục  $y$ , hoặc cả hai trên log-scale. Bạn có thể làm điều này bằng cách dùng method `ax.set_xscale()` và `ax.set_yscale()` của axes mà bạn đang plot lên.

*.set\_xscale()  
.set\_yscale()*

- (f) Tạo một function, `PlotPower()`, cho phép bạn tạo plot của  $x$  so với  $x**a$  cho một  $a$  cố định và một sequence các giá trị của  $x$ . Chẳng hạn, nếu bạn gọi

```
PlotPower(np.arange(1, 11), 3)
```

thì một plot phải được tạo với trục  $x$  nhận các giá trị 1, 2, ..., 10, và trục  $y$  nhận các giá trị  $1^3, 2^3, \dots, 10^3$ .

16. Dùng dataset `Boston`, fit các classification model để dự đoán liệu một suburb cho trước có crime rate cao hơn hay thấp hơn median. Hãy khám phá logistic regression, LDA, naive Bayes, và KNN model bằng nhiều subset khác nhau của predictor. Mô tả các phát hiện của bạn. *Hint:* Bạn sẽ phải tự tạo response variable bằng các variable có trong dataset `Boston`.

# 5

## Resampling Methods

*Resampling methods* là một tool không thể thiếu trong thống kê hiện đại. Các method này liên quan đến việc lặp đi lặp lại quá trình lấy sample từ một training set và fit lại model quan tâm trên từng sample để thu được thêm thông tin về fitted model. Chẳng hạn, để ước lượng variability của một linear regression fit, ta có thể liên tục lấy các sample khác nhau từ training data, fit linear regression trên từng sample mới, rồi xem mức độ khác nhau giữa các fit thu được. Cách làm như vậy có thể cung cấp những thông tin mà ta không thể có được nếu chỉ fit model một lần trên training sample ban đầu.

Các resampling approach có thể tốn nhiều computation, bởi chúng đòi hỏi cùng một statistical method phải được fit nhiều lần trên các subset khác nhau của training data. Tuy nhiên, nhờ những tiến bộ gần đây về computing power, yêu cầu tính toán của resampling methods nhìn chung không còn quá nặng nề. Trong chương này, ta thảo luận hai resampling method được dùng phổ biến nhất: *cross-validation* và *bootstrap*. Cả hai đều là những tool quan trọng trong ứng dụng thực tế của nhiều statistical learning procedure. Ví dụ, cross-validation có thể được dùng để ước lượng test error gắn với một statistical learning method nhằm đánh giá performance của method đó, hoặc để chọn mức flexibility thích hợp. Quá trình đánh giá performance của một model được gọi là *model assessment*, còn quá trình chọn mức flexibility phù hợp cho model được gọi là *model selection*. Bootstrap được dùng trong nhiều bối cảnh, phổ biến nhất là để cung cấp một thước đo về accuracy của một parameter estimate hoặc của một statistical learning method cho trước.

*model assessment*  
*model selection*

## 5.1 Cross-Validation

Trong Chapter 2, ta đã thảo luận sự khác biệt giữa test error rate và training error rate. Test error là average error phát sinh khi dùng một statistical learning method để dự đoán response trên một observation mới — tức một measurement không được dùng khi training method. Với một dataset cho trước, việc sử dụng một statistical learning method chỉ thực sự hợp lý nếu method đó tạo ra test error thấp. Test error có thể tính dễ dàng nếu ta có một test set được dành riêng. Đáng tiếc, trường hợp này thường không xảy ra. Ngược lại, training error có thể tính dễ dàng bằng cách áp dụng statistical learning method lên chính các observation đã được dùng để training. Nhưng như ta đã thấy ở Chapter 2, training error rate thường khác đáng kể so với test error rate, và đặc biệt là nó có thể đánh giá thấp test error một cách nghiêm trọng.

Khi không có một test set riêng đủ lớn để ước lượng trực tiếp test error rate, ta có thể dùng nhiều technique khác nhau để ước lượng đại lượng này từ training data hiện có. Một số method thực hiện điều chỉnh toán học đối với training error rate để ước lượng test error rate; các approach như vậy sẽ được bàn ở Chapter 6. Trong section này, thay vào đó ta xét một class các method ước lượng test error rate bằng cách giữ lại một subset các training observation khỏi quá trình fitting, rồi áp dụng statistical learning method lên các held-out observation đó.

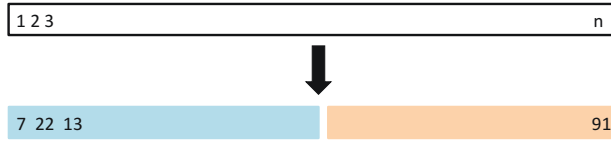
Trong Sections 5.1.1–5.1.4, để đơn giản ta giả sử mình quan tâm đến regression với một quantitative response. Trong Section 5.1.5, ta xét classification với qualitative response. Như sẽ thấy, các ý tưởng cốt lõi hầu như không thay đổi dù response là quantitative hay qualitative.

### 5.1.1 The Validation Set Approach

Giả sử ta muốn ước lượng test error gắn với việc fit một statistical learning method cụ thể trên một tập observation. *Validation set approach*, được minh họa trong Figure 5.1, là một strategy rất đơn giản cho nhiệm vụ này. Nó chia ngẫu nhiên tập observation sẵn có thành hai phần: một training set và một validation set, hay *hold-out set*. Model được fit trên training set, sau đó fitted model được dùng để dự đoán response cho các observation trong validation set. Validation set error rate thu được — thường được đánh giá bằng MSE khi response là quantitative — đóng vai trò như một estimate của test error rate.

Ta minh họa validation set approach trên dataset **Auto**. Nhắc lại từ Chapter 3 rằng dường như tồn tại một non-linear relationship giữa **mpg** và **horsepower**, và model dự đoán **mpg** bằng **horsepower** cùng **horsepower**<sup>2</sup> cho kết quả tốt hơn model chỉ có linear term. Một câu hỏi tự nhiên là liệu cubic fit hoặc higher-order fit có cho kết quả tốt hơn nữa hay không. Trong Chapter 3, ta trả lời câu hỏi này bằng cách xem các p-value gắn với cubic term và các higher-order polynomial term trong linear regression. Nhưng ta cũng có thể trả lời bằng validation method. Ta chia ngẫu nhiên 392 observation thành hai tập: training set gồm 196 data point và validation set gồm 196 observation còn lại. Các validation set error rate thu được khi fit nhiều regression model trên training sample rồi đánh giá performance

*validation set  
approach  
validation set  
hold-out set*



**HÌNH 5.1.** Minh họa dạng sơ đồ của validation set approach. Một tập gồm  $n$  observation được chia ngẫu nhiên thành training set (màu xanh, chứa các observation 7, 22 và 13, cùng những observation khác) và validation set (màu be, chứa observation 91, cùng những observation khác). Statistical learning method được fit trên training set, và performance của nó được đánh giá trên validation set.

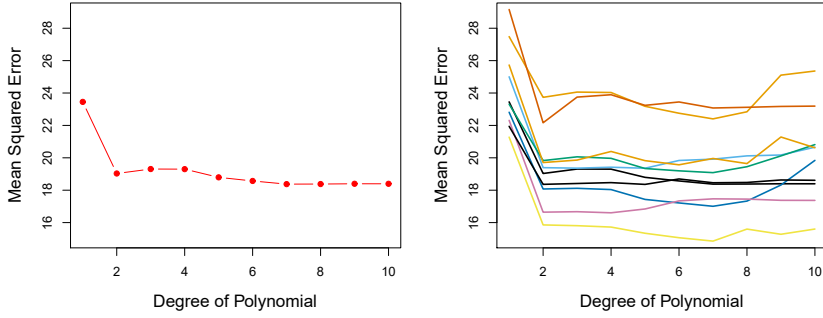
trên validation sample, dùng MSE làm thước đo validation set error, được trình bày ở panel trái của Figure 5.2. Validation set MSE của quadratic fit nhỏ hơn đáng kể so với linear fit. Tuy nhiên, validation set MSE của cubic fit thực ra hơi lớn hơn quadratic fit. Điều này cho thấy việc thêm cubic term vào regression không làm prediction tốt hơn so với chỉ dùng quadratic term.

Nhắc lại rằng để tạo panel trái của Figure 5.2, ta đã chia ngẫu nhiên dataset thành training set và validation set. Nếu lặp lại quá trình chia sample thành hai phần, ta sẽ thu được một estimate hơi khác cho test MSE. Để minh họa, panel phải của Figure 5.2 hiển thị mười validation set MSE curve khác nhau trên dataset Auto, được tạo từ mười cách chia ngẫu nhiên observation thành training set và validation set. Cả mười curve đều cho thấy model có quadratic term có validation set MSE nhỏ hơn rất nhiều so với model chỉ có linear term. Đồng thời, cả mười curve đều cho thấy việc thêm cubic hoặc higher-order polynomial term không mang lại nhiều lợi ích. Tuy nhiên, cần chú ý rằng mỗi curve trong số mười curve lại tạo ra test MSE estimate khác nhau cho từng regression model được xét, và các curve cũng không thống nhất model nào có validation set MSE nhỏ nhất. Dựa trên variability giữa các curve này, điều duy nhất ta có thể kết luận với độ tin cậy đáng kể là linear fit không phù hợp với dữ liệu này.

Validation set approach đơn giản về mặt ý tưởng và dễ implement, nhưng có hai nhược điểm tiềm ẩn:

1. Như panel phải của Figure 5.2 cho thấy, validation estimate của test error rate có thể có variability rất cao, tùy thuộc chính xác observation nào nằm trong training set và observation nào nằm trong validation set.
2. Trong validation approach, chỉ một subset các observation — những observation nằm trong training set thay vì validation set — được dùng để fit model. Vì statistical method thường hoạt động kém hơn khi được training trên ít observation hơn, điều này gợi ý rằng validation set error rate có xu hướng overestimate test error rate của model được fit trên toàn bộ dataset.

Trong các subsection tiếp theo, ta trình bày cross-validation, một refinement của validation set approach nhằm giải quyết hai vấn đề trên.



**HÌNH 5.2.** Validation set approach được dùng trên dataset *Auto* để ước lượng test error phát sinh khi dự đoán *mpg* bằng các polynomial function của *horsepower*. Left: Validation error estimate cho một lần chia duy nhất thành training và validation dataset. Right: Validation method được lặp lại mười lần, mỗi lần dùng một cách chia ngẫu nhiên khác nhau các observation thành training set và validation set. Hình này minh họa variability trong estimated test MSE do approach này tạo ra.

### 5.1.2 Leave-One-Out Cross-Validation

Leave-one-out cross-validation (LOOCV) có quan hệ rất gần với validation set approach của Section 5.1.1, nhưng nó cố gắng xử lý các nhược điểm của method đó. Giống validation set approach, LOOCV chia tập observation thành hai phần. Tuy nhiên, thay vì tạo hai subset có size tương đương, một observation  $(x_1, y_1)$  được dùng làm validation set, còn các observation còn lại  $\{(x_2, y_2), \dots, (x_n, y_n)\}$  tạo thành training set. Statistical learning method được fit trên  $n - 1$  training observation, sau đó prediction  $\hat{y}_1$  được tạo cho observation bị loại ra bằng giá trị  $x_1$ . Vì  $(x_1, y_1)$  không được dùng trong fitting process,  $\text{MSE}_1 = (y_1 - \hat{y}_1)^2$  cung cấp một estimate xấp xỉ unbiased cho test error. Nhưng dù  $\text{MSE}_1$  là unbiased đối với test error, nó vẫn là một estimate kém vì có variability rất cao do chỉ dựa trên một observation  $(x_1, y_1)$ .

leave-one-out  
cross-validation

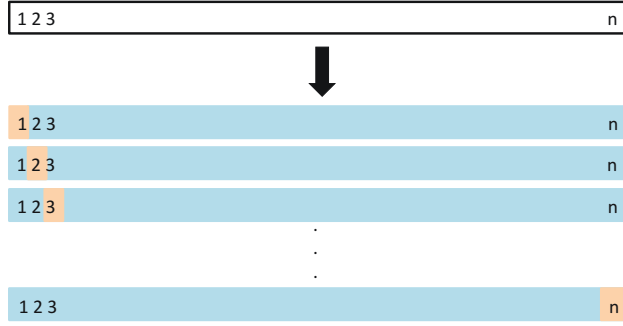
Ta có thể lặp lại procedure bằng cách chọn  $(x_2, y_2)$  làm validation data và training statistical learning procedure trên  $n - 1$  observation còn lại, tức

$$\{(x_1, y_1), (x_3, y_3), \dots, (x_n, y_n)\}.$$

Sau đó ta tính  $\text{MSE}_2 = (y_2 - \hat{y}_2)^2$ . Lặp approach này  $n$  lần tạo ra  $n$  squared error,  $\text{MSE}_1, \dots, \text{MSE}_n$ . LOOCV estimate cho test MSE là trung bình của  $n$  test error estimate này:

$$\text{CV}_{(n)} = \frac{1}{n} \sum_{i=1}^n \text{MSE}_i. \quad (5.1)$$

Sơ đồ LOOCV được minh họa trong Figure 5.3. LOOCV có một vài ưu điểm lớn so với validation set approach. Thứ nhất, bias của nó thấp hơn nhiều. Trong LOOCV, ta lặp lại việc fit statistical learning method bằng các training set chứa  $n - 1$  observation, gần bằng số observation trong toàn bộ dataset. Điều này khác với validation set approach, nơi training set thường chỉ bằng khoảng một nửa size của dataset gốc. Vì vậy, LOOCV thường



**HÌNH 5.3.** Minh họa dạng sơ đồ của LOOCV. Một tập gồm  $n$  data point được lặp đi lặp lại việc chia thành training set (màu xanh) chứa tất cả trừ một observation, và validation set chỉ chứa observation đó (màu be). Test error sau đó được ước lượng bằng cách lấy trung bình của  $n$  MSE thu được. Training set đầu tiên chứa tất cả trừ observation 1, training set thứ hai chứa tất cả trừ observation 2, và cứ tiếp tục như vậy.

không overestimate test error rate nhiều như validation set approach. Thứ hai, trái với validation approach vốn cho kết quả khác nhau mỗi khi chạy lại do randomness trong cách chia training/validation set, chạy LOOCV nhiều lần luôn tạo ra cùng một kết quả: việc chia training/validation set không có randomness.

Ta đã dùng LOOCV trên dataset **Auto** để ước lượng test set MSE khi fit linear regression model dự đoán **mpg** bằng polynomial function của **horsepower**. Kết quả được trình bày ở panel trái của Figure 5.4.

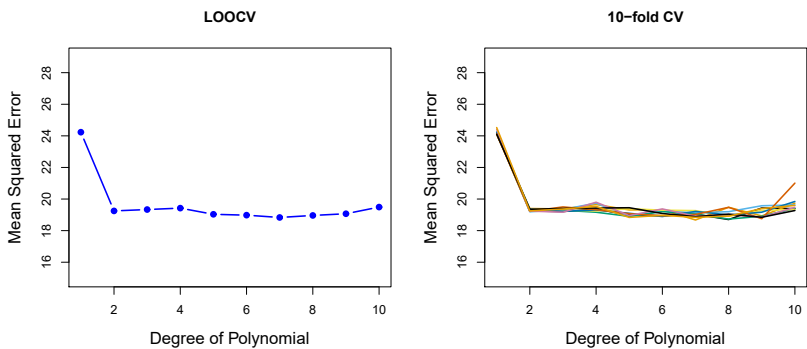
LOOCV có thể tốn kém khi implement vì model phải được fit  $n$  lần. Điều này có thể rất chậm nếu  $n$  lớn và mỗi individual model mất nhiều thời gian để fit. Với least squares linear hoặc polynomial regression, có một shortcut đáng kinh ngạc khiến chi phí của LOOCV bằng đúng chi phí của một lần fit model. Công thức sau đúng:

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \left( \frac{y_i - \hat{y}_i}{1 - h_i} \right)^2. \quad (5.2)$$

Ở đây  $\hat{y}_i$  là fitted value thứ  $i$  từ original least squares fit, còn  $h_i$  là leverage được định nghĩa trong (3.37) ở page 105.<sup>1</sup> Công thức này giống ordinary MSE, ngoại trừ residual thứ  $i$  được chia cho  $1 - h_i$ . Leverage nằm giữa  $1/n$  và 1, và phản ánh mức độ một observation ảnh hưởng đến chính fit của nó. Vì vậy residual của các high-leverage point được inflate trong công thức này đúng một lượng cần thiết để equality trên đúng.

LOOCV là một method rất tổng quát và có thể được dùng với hầu như mọi loại predictive modeling. Chẳng hạn, ta có thể dùng nó với logistic regression, linear discriminant analysis, hoặc bất kỳ method nào được thảo luận ở các chapter sau. Tuy nhiên, “magic formula” (5.2) không đúng trong trường hợp tổng quát; khi đó model phải được refit  $n$  lần.

<sup>1</sup>Trong multiple linear regression, leverage có dạng phức tạp hơn một chút so với (3.37), nhưng (5.2) vẫn đúng.



**HÌNH 5.4.** Cross-validation được dùng trên dataset *Auto* để ước lượng test error khi dự đoán *mpg* bằng *polynomial function* của *horsepower*. Left: LOOCV error curve. Right: 10-fold CV được chạy chín lần riêng biệt, mỗi lần với một cách chia ngẫu nhiên khác nhau của dữ liệu thành mười phần. Hình cho thấy chín CV error curve hơi khác nhau.

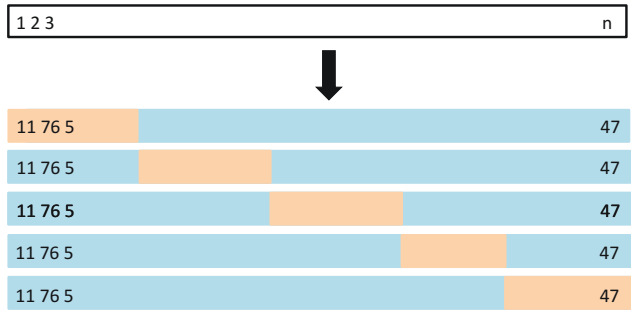
5.1.3 *k*-Fold Cross-Validation

Một alternative cho LOOCV là *k-fold CV*. Approach này chia ngẫu nhiên tập observation thành *k* group, hay *fold*, có size xấp xỉ bằng nhau. Fold đầu tiên được xem là validation set, còn method được fit trên *k* – 1 fold còn lại. Mean squared error  $MSE_1$  sau đó được tính trên các observation trong held-out fold. Procedure này được lặp *k* lần; mỗi lần, một group observation khác được dùng làm validation set. Quá trình này tạo ra *k* estimate của test error,  $MSE_1, MSE_2, \dots, MSE_k$ . *k*-fold CV estimate được tính bằng cách lấy trung bình:

*k*-fold CV

$$CV_{(k)} = \frac{1}{k} \sum_{i=1}^k MSE_i. \tag{5.3}$$

Figure 5.5 minh họa *k*-fold CV approach.

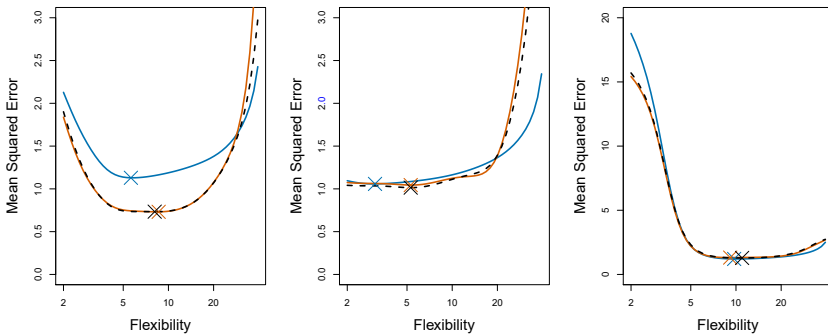


**HÌNH 5.5.** Minh họa dạng sơ đồ của 5-fold CV. Một tập gồm *n* observation được chia ngẫu nhiên thành năm group không overlap. Mỗi một phần năm lần lượt đóng vai trò validation set (màu be), còn phần còn lại là training set (màu xanh). Test error được ước lượng bằng cách lấy trung bình năm MSE estimate thu được.

Không khó để thấy LOOCV là một trường hợp đặc biệt của  $k$ -fold CV khi  $k = n$ . Trong thực tế, người ta thường dùng  $k$ -fold CV với  $k = 5$  hoặc  $k = 10$ . Lợi ích của  $k = 5$  hoặc  $k = 10$  so với  $k = n$  là gì? Lợi ích rõ nhất là computation. LOOCV đòi hỏi fit statistical learning method  $n$  lần, có thể rất tốn kém về tính toán (ngoại trừ linear model fit bằng least squares, khi có thể dùng formula (5.2)). Nhưng cross-validation là approach rất tổng quát, có thể áp dụng cho gần như mọi statistical learning method. Một số method có fitting procedure rất nặng, vì thế LOOCV có thể gây vấn đề tính toán, đặc biệt khi  $n$  cực lớn. Ngược lại, 10-fold CV chỉ cần fit learning procedure mười lần, khả thi hơn nhiều. Như sẽ thấy ở Section 5.1.4, 5-fold hoặc 10-fold CV còn có những lợi ích khác ngoài computation, liên quan đến bias-variance trade-off.

Panel phải của Figure 5.4 hiển thị chín 10-fold CV estimate khác nhau cho dataset `Auto`, mỗi estimate đến từ một cách chia ngẫu nhiên khác nhau các observation thành mười fold. Hình cho thấy có một mức variability nhất định trong CV estimate do variability của cách chia observation thành mười fold. Tuy vậy, variability này thường thấp hơn rất nhiều so với variability trong test error estimate tạo ra bởi validation set approach (panel phải của Figure 5.2).

Khi làm việc với real data, ta không biết true test MSE nên khó đánh giá accuracy của cross-validation estimate. Nhưng với simulated data, ta có thể tính true test MSE, và nhờ đó đánh giá accuracy của cross-validation result. Trong Figure 5.6, ta plot cross-validation estimate và true test error rate thu được khi áp dụng smoothing spline cho các simulated dataset trong Figures 2.9–2.11 của Chapter 2. True test MSE được vẽ màu xanh. Black dashed line và orange solid line lần lượt biểu diễn LOOCV estimate và 10-fold CV estimate. Trong cả ba plot, hai cross-validation estimate rất giống nhau. Ở panel phải của Figure 5.6, true test MSE và cross-validation curve gần như trùng nhau. Ở panel giữa, hai nhóm curve khá giống nhau tại mức flexibility thấp, còn CV curve overestimate test set MSE khi flexibility cao hơn. Ở panel trái, CV curve có shape tổng thể đúng nhưng underestimate true test MSE.



**HÌNH 5.6.** True và estimated test MSE cho các simulated dataset trong Figures 2.9 (left), 2.10 (center), và 2.11 (right). True test MSE được vẽ màu xanh, LOOCV estimate là black dashed line, còn 10-fold CV estimate là đường màu cam. Dấu cross chỉ minimum của từng MSE curve.

Khi thực hiện cross-validation, mục tiêu của ta có thể là xác định một



statistical learning procedure cho trước dự kiến sẽ hoạt động tốt đến đâu trên independent data; khi đó actual estimate của test MSE là điều ta quan tâm. Nhưng trong những trường hợp khác, ta chỉ quan tâm vị trí minimum trên estimated test MSE curve. Điều này xảy ra khi ta chạy cross-validation trên nhiều statistical learning method, hoặc trên cùng một method với nhiều mức flexibility khác nhau, để tìm method có test error thấp nhất. Với mục đích này, vị trí minimum của estimated test MSE curve là quan trọng, còn actual value của estimated test MSE thì không. Figure 5.6 cho thấy dù đôi khi underestimate true test MSE, tất cả CV curve vẫn xác định khá gần mức flexibility đúng — tức mức flexibility tương ứng với test MSE nhỏ nhất.

#### 5.1.4 Bias-Variance Trade-Off for $k$ -Fold Cross-Validation

Trong Section 5.1.3, ta đã nói rằng  $k$ -fold CV với  $k < n$  có lợi thế computation so với LOOCV. Nhưng nếu bỏ qua computation, một lợi thế ít hiển nhiên nhưng có thể quan trọng hơn của  $k$ -fold CV là nó thường cho test error rate estimate chính xác hơn LOOCV. Điều này liên quan đến bias-variance trade-off.

Section 5.1.1 đã đề cập rằng validation set approach có thể overestimate test error rate vì training set dùng để fit statistical learning method chỉ chứa một nửa observation của toàn dataset. Theo logic đó, LOOCV sẽ cho estimate gần như unbiased của test error, bởi mỗi training set chứa  $n - 1$  observation, gần bằng toàn bộ số observation trong dataset. Còn  $k$ -fold CV với, chẳng hạn,  $k = 5$  hoặc  $k = 10$  sẽ có bias ở mức trung gian vì mỗi training set chứa khoảng  $(k - 1)n/k$  observation — ít hơn LOOCV nhưng nhiều hơn đáng kể so với validation set approach. Vì thế, xét riêng khía cạnh giảm bias, LOOCV rõ ràng đáng được ưu tiên hơn  $k$ -fold CV.

Tuy nhiên, bias không phải nguồn lo ngại duy nhất của một estimating procedure; ta còn phải xét variance. Hóa ra LOOCV có variance cao hơn  $k$ -fold CV với  $k < n$ . Tại sao? Khi thực hiện LOOCV, ta thực chất đang lấy trung bình output của  $n$  fitted model, mỗi model được training trên một tập observation gần như giống hệt nhau; do đó output của chúng có positive correlation rất cao. Ngược lại, với  $k$ -fold CV khi  $k < n$ , ta lấy trung bình output của  $k$  fitted model có correlation thấp hơn phần nào, bởi overlap giữa các training set nhỏ hơn. Vì mean của nhiều quantity có correlation cao có variance lớn hơn mean của nhiều quantity không correlation cao đến vậy, test error estimate từ LOOCV thường có variance lớn hơn estimate từ  $k$ -fold CV.

Tóm lại, việc chọn  $k$  trong  $k$ -fold cross-validation kéo theo một bias-variance trade-off. Trong thực tế, dựa trên các cân nhắc này, người ta thường dùng  $k = 5$  hoặc  $k = 10$ , vì empirical evidence cho thấy các giá trị này tạo ra test error rate estimate không bị bias quá cao cũng không có variance quá lớn.

#### 5.1.5 Cross-Validation on Classification Problems

Cho đến đây trong chapter này, ta minh họa cross-validation trong regression setting với quantitative outcome  $Y$ , nên đã dùng MSE để định lượng test error. Nhưng cross-validation cũng rất hữu ích trong classification set-

ting khi  $Y$  là qualitative. Trong setting này, cross-validation hoạt động y như đã mô tả trước đó, chỉ khác ở chỗ thay vì MSE, ta dùng số observation bị misclassified để định lượng test error. Chẳng hạn, trong classification setting, LOOCV error rate có dạng

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^n \text{Err}_i, \quad (5.4)$$

trong đó  $\text{Err}_i = I(y_i \neq \hat{y}_i)$ .  $k$ -fold CV error rate và validation set error rate được định nghĩa tương tự.

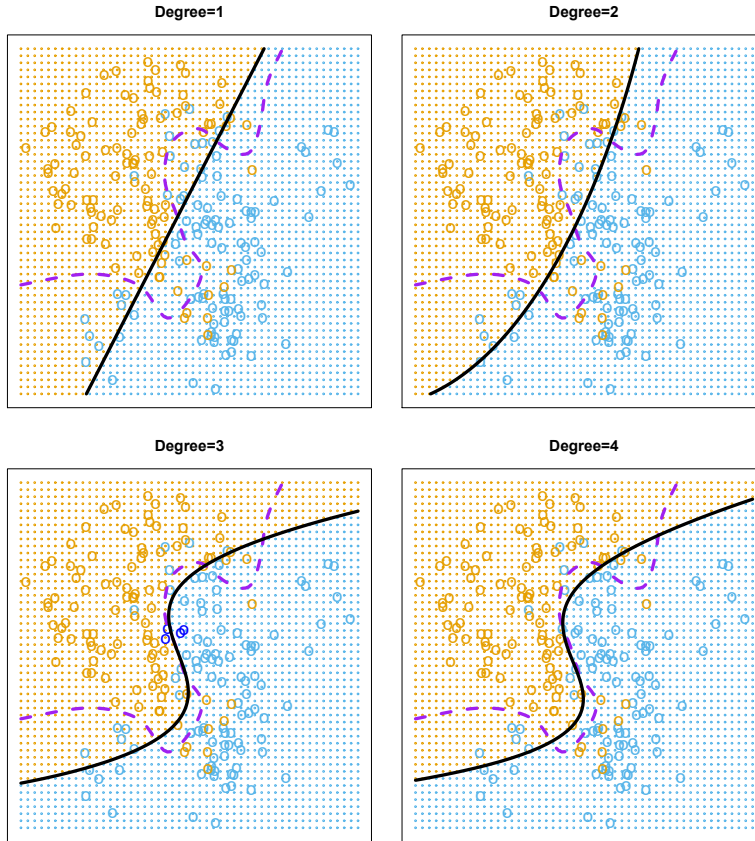
Ví dụ, ta fit nhiều logistic regression model trên two-dimensional classification data trong Figure 2.13. Ở panel trên-trái của Figure 5.7, black solid line biểu diễn estimated decision boundary khi fit standard logistic regression model trên dataset này. Vì đây là simulated data, ta có thể tính true test error rate, bằng 0.201 và do đó lớn hơn đáng kể Bayes error rate 0.133. Rõ ràng logistic regression không đủ flexibility để model Bayes decision boundary trong setting này. Ta có thể mở rộng logistic regression để có non-linear decision boundary bằng polynomial function của predictors, giống cách đã làm trong regression setting ở Section 3.3.2. Chẳng hạn, ta có thể fit quadratic logistic regression model:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \beta_3 X_2 + \beta_4 X_2^2. \quad (5.5)$$

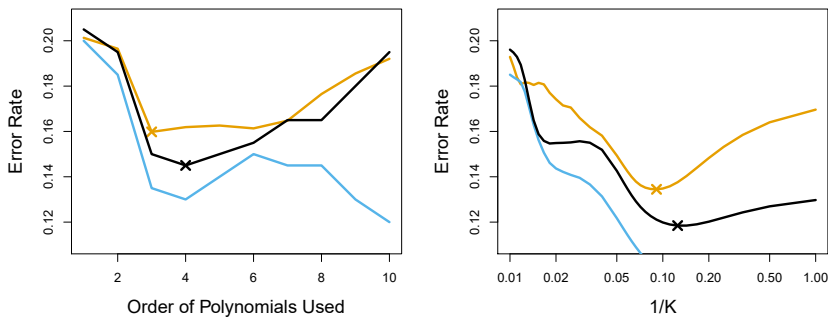
Panel trên-phải của Figure 5.7 cho thấy decision boundary thu được, giờ đã cong. Tuy nhiên, test error rate chỉ cải thiện nhẹ xuống 0.197. Một cải thiện lớn hơn nhiều xuất hiện ở panel dưới-trái, nơi ta fit logistic regression model có cubic polynomial của predictors: test error rate giảm xuống 0.160. Chuyển sang quartic polynomial (panel dưới-phải) khiến test error tăng nhẹ.

Trong thực tế với real data, Bayes decision boundary và test error rate đều không biết trước. Vậy ta chọn thế nào giữa bốn logistic regression model trong Figure 5.7? Ta có thể dùng cross-validation. Panel trái của Figure 5.8 hiển thị bằng màu đen 10-fold CV error rate khi fit mười logistic regression model, dùng polynomial function của predictors đến order 10. True test error được vẽ màu nâu, training error màu xanh. Như đã thấy, training error có xu hướng giảm khi flexibility của fit tăng. Trái lại, test error có U-shape đặc trưng. 10-fold CV error rate xấp xỉ khá tốt test error rate. Dù hơi underestimate error rate, nó đạt minimum khi dùng fourth-order polynomial, rất gần minimum của test curve xảy ra với third-order polynomial. Trên thực tế, fourth-order polynomial có lẽ vẫn cho test set performance tốt vì true test error rate gần như tương đương cho polynomial order 3, 4, 5 và 6.

Panel phải của Figure 5.8 hiển thị ba curve tương tự khi dùng KNN classification như một function của  $K$  (ở đây  $K$  là số neighbor của KNN classifier, không phải số CV fold). Một lần nữa, training error rate giảm khi method linh hoạt hơn, nên training error rate không thể dùng để chọn  $K$  tối ưu. Dù cross-validation error curve hơi underestimate test error rate, minimum của nó nằm rất gần best value của  $K$ .



**HÌNH 5.7.** Logistic regression fit trên two-dimensional classification data trong Figure 2.13. Bayes decision boundary được biểu diễn bằng purple dashed line. Estimated decision boundary từ linear, quadratic, cubic và quartic (degree 1–4) logistic regression được vẽ màu đen. Test error rate của bốn logistic regression fit lần lượt là 0.201, 0.197, 0.160 và 0.162, còn Bayes error rate là 0.133.



**HÌNH 5.8.** Test error (nâu), training error (xanh), và 10-fold CV error (đen) trên two-dimensional classification data trong Figure 5.7. Left: Logistic regression dùng polynomial function của predictors; polynomial order được hiển thị trên trục  $x$ . Right: KNN classifier với nhiều giá trị  $K$ , là số neighbor được dùng trong KNN classifier.

## 5.2 The Bootstrap

*Bootstrap* là một statistical tool cực kỳ mạnh và có phạm vi áp dụng rộng, có thể dùng để định lượng uncertainty gắn với một estimator hoặc statistical learning method cho trước. Chẳng hạn, bootstrap có thể được dùng để ước lượng standard error của các coefficient từ một linear regression fit. Riêng trong linear regression, điều này không đặc biệt hữu ích vì ở Chapter 3 ta đã thấy các statistical software chuẩn tự động output các standard error này. Tuy nhiên, sức mạnh của bootstrap nằm ở chỗ nó có thể áp dụng rất dễ dàng cho rất nhiều statistical learning method, bao gồm cả những method mà nếu không dùng bootstrap thì variability rất khó định lượng và software cũng không tự động output.

*bootstrap*

Trong section này, ta minh họa bootstrap bằng một toy example, nơi mục tiêu là xác định investment allocation tốt nhất dưới một model đơn giản. Trong Section 5.3, ta sẽ dùng bootstrap để đánh giá variability gắn với regression coefficient trong một linear model fit.

Giả sử ta muốn đầu tư một số tiền cố định vào hai financial asset tạo ra return lần lượt là  $X$  và  $Y$ , trong đó  $X$  và  $Y$  là random quantity. Ta đầu tư fraction  $\alpha$  của số tiền vào  $X$ , và phần còn lại  $1 - \alpha$  vào  $Y$ . Vì return của hai asset có variability, ta muốn chọn  $\alpha$  sao cho total risk, hay variance, của investment nhỏ nhất. Nói cách khác, ta muốn minimize  $\text{Var}(\alpha X + (1 - \alpha)Y)$ . Có thể chứng minh rằng giá trị minimize risk là

$$\alpha = \frac{\sigma_Y^2 - \sigma_{XY}}{\sigma_X^2 + \sigma_Y^2 - 2\sigma_{XY}}, \quad (5.6)$$

trong đó  $\sigma_X^2 = \text{Var}(X)$ ,  $\sigma_Y^2 = \text{Var}(Y)$ , và  $\sigma_{XY} = \text{Cov}(X, Y)$ .

Trong thực tế, các quantity  $\sigma_X^2$ ,  $\sigma_Y^2$ , và  $\sigma_{XY}$  là unknown. Ta có thể tính các estimate  $\hat{\sigma}_X^2$ ,  $\hat{\sigma}_Y^2$ , và  $\hat{\sigma}_{XY}$  từ một dataset chứa các measurement trong quá khứ của  $X$  và  $Y$ . Khi đó, giá trị  $\alpha$  làm minimize variance của investment có thể được estimate bằng

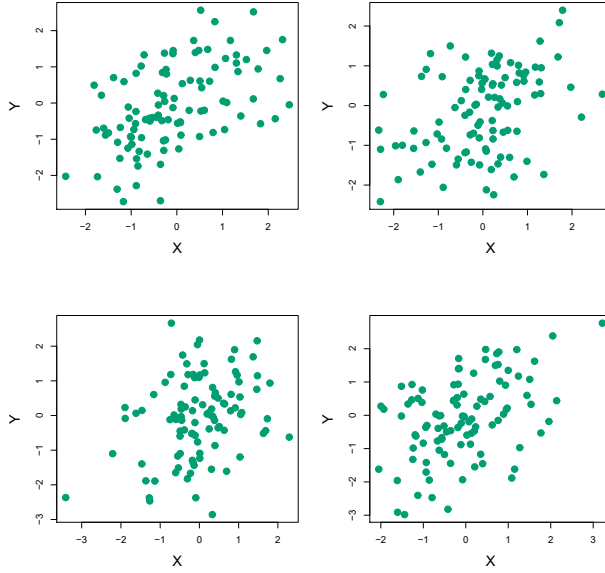
$$\hat{\alpha} = \frac{\hat{\sigma}_Y^2 - \hat{\sigma}_{XY}}{\hat{\sigma}_X^2 + \hat{\sigma}_Y^2 - 2\hat{\sigma}_{XY}}. \quad (5.7)$$

Figure 5.9 minh họa approach này trên simulated data. Trong mỗi panel, ta simulate 100 pair return cho hai investment  $X$  và  $Y$ . Ta dùng các return này để estimate  $\sigma_X^2$ ,  $\sigma_Y^2$ , và  $\sigma_{XY}$ , rồi thay vào (5.7) để thu được estimate của  $\alpha$ . Giá trị  $\hat{\alpha}$  từ bốn simulated dataset nằm trong khoảng 0.532 đến 0.657.

Ta tự nhiên muốn định lượng accuracy của estimate  $\alpha$ . Để estimate standard deviation của  $\hat{\alpha}$ , ta lặp lại quá trình simulate 100 paired observation của  $X$  và  $Y$  và estimate  $\alpha$  bằng (5.7) tổng cộng 1,000 lần. Qua đó ta thu được 1,000 estimate  $\hat{\alpha}_1, \hat{\alpha}_2, \dots, \hat{\alpha}_{1,000}$ . Panel trái của Figure 5.10 hiển thị histogram của các estimate này. Trong simulation, parameter được đặt là  $\sigma_X^2 = 1$ ,  $\sigma_Y^2 = 1.25$ , và  $\sigma_{XY} = 0.5$ , nên true value của  $\alpha$  là 0.6. Giá trị này được biểu diễn bằng solid vertical line trên histogram.

Mean của toàn bộ 1,000 estimate của  $\alpha$  là

$$\bar{\alpha} = \frac{1}{1000} \sum_{r=1}^{1000} \hat{\alpha}_r = 0.5996,$$



**HÌNH 5.9.** Mỗi panel hiển thị 100 simulated return cho investments  $X$  và  $Y$ . Từ trái sang phải, từ trên xuống dưới, các estimate  $\alpha$  thu được lần lượt là 0.576, 0.532, 0.657, và 0.651.

rất gần  $\alpha = 0.6$ , và standard deviation của các estimate là

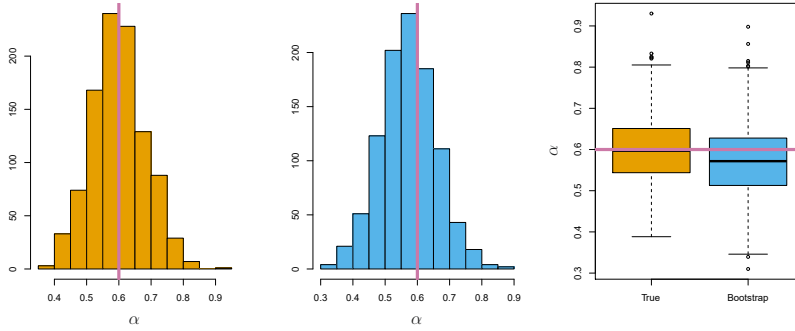
$$\sqrt{\frac{1}{1000-1} \sum_{r=1}^{1000} (\hat{\alpha}_r - \bar{\alpha})^2} = 0.083.$$

Điều này cho ta một hình dung rất tốt về accuracy của  $\hat{\alpha}$ :  $SE(\hat{\alpha}) \approx 0.083$ . Nói gần đúng, với một random sample từ population, ta kỳ vọng  $\hat{\alpha}$  khác  $\alpha$  khoảng 0.08, trung bình.

Tuy nhiên, trong thực tế procedure vừa dùng để estimate  $SE(\hat{\alpha})$  không thể áp dụng trực tiếp, vì với real data ta không thể tạo các sample mới từ original population. Bootstrap giải quyết vấn đề này bằng cách dùng computer để mô phỏng quá trình tạo sample set mới, nhờ đó estimate variability của  $\hat{\alpha}$  mà không cần thu thập thêm sample. Thay vì liên tục lấy các independent dataset từ population, ta tạo các dataset khác nhau bằng cách liên tục sample observation từ original dataset.

Approach này được minh họa trong Figure 5.11 trên một dataset đơn giản, ký hiệu  $Z$ , chỉ có  $n = 3$  observation. Ta chọn ngẫu nhiên  $n$  observation từ dataset để tạo bootstrap dataset  $Z^{*1}$ . Sampling được thực hiện *with replacement*, nghĩa là cùng một observation có thể xuất hiện nhiều hơn một lần trong bootstrap dataset. Trong ví dụ này,  $Z^{*1}$  chứa observation thứ ba hai lần, observation thứ nhất một lần, và không có observation thứ hai. Lưu ý rằng nếu một observation nằm trong  $Z^{*1}$  thì cả giá trị  $X$  và  $Y$  của observation đó đều được đưa vào. Từ  $Z^{*1}$  ta tạo bootstrap estimate mới của  $\alpha$ , ký hiệu  $\hat{\alpha}^{*1}$ . Procedure được lặp  $B$  lần với  $B$  đủ lớn, tạo ra  $B$  bootstrap dataset  $Z^{*1}, Z^{*2}, \dots, Z^{*B}$  và  $B$  estimate tương ứng  $\hat{\alpha}^{*1}, \hat{\alpha}^{*2}, \dots, \hat{\alpha}^{*B}$ .

*with replacement*



**HÌNH 5.10.** Left: Histogram của các estimate  $\alpha$  thu được bằng cách tạo 1,000 simulated dataset từ true population. Center: Histogram của các estimate  $\alpha$  thu được từ 1,000 bootstrap sample của một dataset duy nhất. Right: Các estimate  $\alpha$  ở panel trái và giữa được hiển thị dưới dạng boxplot. Trong mỗi panel, đường màu hồng chỉ true value của  $\alpha$ .

Ta có thể tính standard error của các bootstrap estimate bằng formula

$$SE_B(\hat{\alpha}) = \sqrt{\frac{1}{B-1} \sum_{r=1}^B \left( \hat{\alpha}^{*r} - \frac{1}{B} \sum_{r'=1}^B \hat{\alpha}^{*r'} \right)^2}. \quad (5.8)$$

Quantity này đóng vai trò estimate cho standard error của  $\hat{\alpha}$  được tính từ original dataset.

Bootstrap approach được minh họa ở panel giữa của Figure 5.10, hiển thị histogram của 1,000 bootstrap estimate của  $\alpha$ , mỗi estimate được tính từ một bootstrap dataset khác nhau. Panel này được dựng chỉ từ một dataset duy nhất, nên có thể tạo từ real data. Histogram trông rất giống panel trái, nơi hiển thị idealized histogram của các estimate  $\alpha$  thu được bằng cách generate 1,000 simulated dataset từ true population. Đặc biệt, bootstrap estimate  $SE(\hat{\alpha})$  từ (5.8) bằng 0.087, rất gần estimate 0.083 thu được từ 1,000 simulated dataset. Panel phải biểu diễn lại thông tin ở hai panel trái và giữa bằng boxplot. Một lần nữa, spread của hai boxplot rất giống nhau, cho thấy bootstrap có thể estimate variability gần với  $\hat{\alpha}$  một cách hiệu quả.

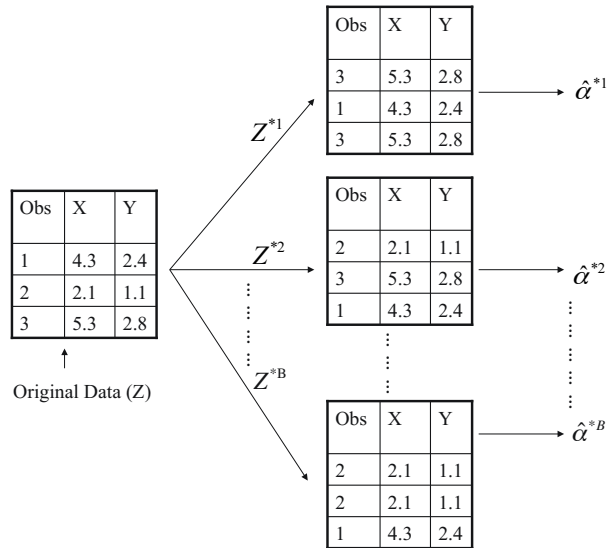
### 5.3 Lab: Cross-Validation and the Bootstrap

Trong lab này, ta khám phá các resampling technique đã được trình bày trong chapter. Một số command trong lab có thể mất một lúc để chạy trên computer của bạn.

Ta lại bắt đầu bằng cách đặt phần lớn các import ở top level.

In [1]:

```
import numpy as np
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
 summarize,
 poly)
from sklearn.model_selection import train_test_split
```



**HÌNH 5.11.** Minh họa bằng đồ thị của bootstrap approach trên một sample nhỏ gồm  $n = 3$  observation. Mỗi bootstrap dataset chứa  $n$  observation được sample with replacement từ original dataset. Mỗi bootstrap dataset được dùng để tạo một estimate của  $\alpha$ .

Lab này cần thêm một số import mới.

```
In [2]: from functools import partial
 from sklearn.model_selection import \
 (cross_validate,
 KFold,
 ShuffleSplit)
 from sklearn.base import clone
 from ISLP.models import sklearn_sm
```

### 5.3.1 The Validation Set Approach

Ta dùng validation set approach để estimate test error rate khi fit nhiều linear model khác nhau trên dataset Auto.

Ta dùng function `train_test_split()` để chia dữ liệu thành training set và validation set. Vì có 392 observation, ta chia thành hai tập bằng nhau size 196 bằng argument `test_size=196`. Nói chung nên đặt random seed khi thực hiện những operation có yếu tố randomness như thế này, để kết quả có thể được reproduce chính xác ở thời điểm sau. Ta đặt random seed của splitter bằng argument `random_state=0`.

`train_test_split()`

```
In [3]: Auto = load_data('Auto')
 Auto_train, Auto_valid = train_test_split(Auto,
 test_size=196,
 random_state=0)
```

Bây giờ ta có thể fit linear regression chỉ bằng các observation thuộc training set `Auto_train`.

```
In [4]: hp_mm = MS(['horsepower'])
X_train = hp_mm.fit_transform(Auto_train)
y_train = Auto_train['mpg']
model = sm.OLS(y_train, X_train)
results = model.fit()
```

Tiếp theo, ta dùng method `predict()` của `results`, đánh giá trên model matrix của model này được tạo từ validation dataset. Đồng thời ta tính validation MSE của model.

```
In [5]: X_valid = hp_mm.transform(Auto_valid)
y_valid = Auto_valid['mpg']
valid_pred = results.predict(X_valid)
np.mean((y_valid - valid_pred)**2)
```

```
Out[5]: 23.6166
```

Vì vậy estimate của validation MSE cho linear regression fit là 23.62.

Ta cũng có thể estimate validation error cho higher-degree polynomial regression. Trước hết ta định nghĩa function `evalMSE()` nhận vào một model string cùng training set và test set, rồi trả về MSE trên test set.

```
In [6]: def evalMSE(terms,
 response,
 train,
 test):

 mm = MS(terms)
 X_train = mm.fit_transform(train)
 y_train = train[response]

 X_test = mm.transform(test)
 y_test = test[response]

 results = sm.OLS(y_train, X_train).fit()
 test_pred = results.predict(X_test)

 return np.mean((y_test - test_pred)**2)
```

Hãy dùng function này để estimate validation MSE cho linear, quadratic và cubic fit. Ở đây ta dùng function `enumerate()`, function này cung cấp cả value lẫn index của object khi ta iterate qua một for loop.

*enumerate()*

```
In [7]: MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
 MSE[idx] = evalMSE([poly('horsepower', degree)],
 'mpg',
 Auto_train,
 Auto_valid)

MSE
```

```
Out[7]: array([23.62, 18.76, 18.80])
```

Ba error rate lần lượt là 23.62, 18.76 và 18.80. Nếu chọn một cách chia training/validation khác, ta kỳ vọng validation set error cũng sẽ khác phần nào.



```
In [8]: Auto_train, Auto_valid = train_test_split(Auto,
 test_size=196,
 random_state=3)

MSE = np.zeros(3)
for idx, degree in enumerate(range(1, 4)):
 MSE[idx] = evalMSE([poly('horsepower', degree)],
 'mpg',
 Auto_train,
 Auto_valid)

MSE
```

```
Out[8]: array([20.76, 16.95, 16.97])
```

Với cách chia này, validation set error rate của model có linear, quadratic và cubic term lần lượt là 20.76, 16.95 và 16.97. Kết quả phù hợp với những gì ta đã thấy trước đó: model dự đoán mpg bằng quadratic function của horsepower hoạt động tốt hơn model chỉ có linear function của horsepower, và không có evidence cho thấy cubic function cải thiện thêm.

### 5.3.2 Cross-Validation

Về lý thuyết, cross-validation estimate có thể được tính cho bất kỳ generalized linear model nào. Trong thực tế, cách đơn giản nhất để thực hiện cross-validation trong Python là dùng **sklearn**, thư viện có interface hay API khác với **statsmodels**, là code mà ta đã dùng để fit GLM.

Đây là một vấn đề data scientist thường gặp: “Tôi có một function làm task A, và cần đưa nó vào một thứ thực hiện task B, để có thể tính  $B(A(D))$ , với  $D$  là data.” Khi A và B không tự nhiên tương tác với nhau, ta cần một *wrapper*. Trong package ISLP, ta cung cấp wrapper **sklearn\_sm()** để dễ dàng dùng các cross-validation tool của **sklearn** với model fit bằng **statsmodels**.

Class **sklearn\_sm()** nhận argument đầu tiên là một model từ **statsmodels**. Nó có thể nhận thêm hai optional argument: **model\_str**, dùng để chỉ định formula, và **model\_args**, là dictionary chứa các additional argument dùng khi fit model. Ví dụ, để fit logistic regression model ta phải chỉ định family argument; giá trị được truyền dưới dạng sau:

*wrapper*  
*sklearn\_sm()*

```
model_args={'family':sm.families.Binomial()}
```

Dưới đây là wrapper đang hoạt động:

```
In [9]: hp_model = sklearn_sm(sm.OLS,
 MS(['horsepower']))
X, Y = Auto.drop(columns=['mpg'], Auto['mpg'])
cv_results = cross_validate(hp_model,
 X,
 Y,
 cv=Auto.shape[0])
cv_err = np.mean(cv_results['test_score'])
cv_err
```

```
Out[9]: 24.2315
```

Các argument của **cross\_validate()** gồm: một object có các method

`fit()`, `predict()` và `score()` phù hợp, một array feature  $X$ , và response  $Y$ . Ta còn truyền argument `cv`; nếu chỉ định một integer  $K$ , function sẽ thực hiện  $K$ -fold cross-validation. Ở đây ta truyền tổng số observation, nên thu được leave-one-out cross-validation (LOOCV). Function `cross_validate()` tạo ra một dictionary với nhiều component; ở đây ta chỉ cần cross-validated test score (MSE), estimate bằng 24.23.

`cross_validate()`

Ta có thể lặp procedure này cho các polynomial fit ngày càng phức tạp. Để tự động hóa, ta lại dùng for loop lần lượt fit polynomial regression degree 1 đến 5, tính cross-validation error tương ứng và lưu vào phần tử thứ  $i$  của vector `cv_error`. Variable `d` trong loop là degree của polynomial. Trước hết ta initialize vector; command này có thể mất vài giây.

In [10]:

```
cv_error = np.zeros(5)
H = np.array(Auto['horsepower'])
M = sklearn_sm(sm.OLS)
for i, d in enumerate(range(1,6)):
 X = np.power.outer(H, np.arange(d+1))
 M_CV = cross_validate(M,
 X,
 Y,
 cv=Auto.shape[0])
 cv_error[i] = np.mean(M_CV['test_score'])
cv_error
```

Out[10]:

```
array([24.2315, 19.2482, 19.3350, 19.4244, 19.0332])
```

Như Figure 5.4, estimated test MSE giảm mạnh từ linear fit sang quadratic fit, nhưng sau đó không có cải thiện rõ ràng khi dùng higher-degree polynomial.

Ở trên ta đã giới thiệu method `outer()` của function `np.power()`. Method `outer()` được áp dụng cho một operation có hai argument, như `add()`, `min()` hay `power()`. Nó nhận hai array và tạo một array lớn hơn, trong đó operation được áp dụng cho từng pair element của hai array.

`.outer()`  
`np.power()`

In [11]:

```
A = np.array([3, 5, 9])
B = np.array([2, 4])
np.add.outer(A, B)
```

Out[11]:

```
array([[5, 7],
 [7, 9],
 [11, 13]])
```

Trong ví dụ CV ở trên, ta dùng  $K = n$ , nhưng tất nhiên cũng có thể dùng  $K < n$ . Code rất giống phần trên và nhanh hơn đáng kể. Ở đây ta dùng `KFold()` để chia dữ liệu thành  $K = 10$  random group. Ta dùng `random_state` để đặt random seed và initialize vector `cv_error`, nơi lưu CV error ứng với polynomial fit degree 1 đến 5.

`KFold()`

In [12]:

```
cv_error = np.zeros(5)
cv = KFold(n_splits=10,
 shuffle=True,
 random_state=0) # use same splits for each degree
for i, d in enumerate(range(1,6)):
 X = np.power.outer(H, np.arange(d+1))
 M_CV = cross_validate(M,
 X,
```

```

 Y,
 cv=cv)
 cv_error[i] = np.mean(M_CV['test_score'])
cv_error

```

Out[12]: array([24.2077, 19.1853, 19.2763, 19.4785, 19.1372])

Computation time ngắn hơn nhiều so với LOOCV. Về nguyên tắc, với least squares linear model, LOOCV phải nhanh hơn  $K$ -fold CV nhờ formula (5.2); tuy nhiên generic function `cross_validate()` không sử dụng formula này. Ta vẫn thấy rất ít evidence rằng cubic hoặc higher-degree polynomial term làm test error thấp hơn quadratic fit.

Function `cross_validate()` linh hoạt và có thể nhận nhiều splitting mechanism khác nhau làm argument. Chẳng hạn, function `ShuffleSplit()` có thể implement validation set approach dễ dàng như  $K$ -fold cross-validation.

*ShuffleSplit()*

```

In [13]: validation = ShuffleSplit(n_splits=1,
 test_size=196,
 random_state=0)
 results = cross_validate(hp_model,
 Auto.drop(['mpg'], axis=1),
 Auto['mpg'],
 cv=validation);
 results['test_score']

```

Out[13]: array([23.6166])

Ta có thể estimate variability trong test error bằng cách chạy:

```

In [14]: validation = ShuffleSplit(n_splits=10,
 test_size=196,
 random_state=0)
 results = cross_validate(hp_model,
 Auto.drop(['mpg'], axis=1),
 Auto['mpg'],
 cv=validation)
 results['test_score'].mean(), results['test_score'].std()

```

Out[14]: (23.8022, 1.4218)

Lưu ý standard deviation này không phải estimate hợp lệ của sampling variability của mean test score hay của individual score, vì các training sample được chọn ngẫu nhiên overlap nhau và do đó tạo correlation. Tuy vậy, nó cho ta hình dung về Monte Carlo variation phát sinh khi chọn các random fold khác nhau.

### 5.3.3 The Bootstrap

Ta minh họa bootstrap bằng ví dụ đơn giản của Section 5.2, đồng thời qua một ví dụ estimate accuracy của linear regression model trên dataset `Auto`. *Estimating the Accuracy of a Statistic of Interest*

Một lợi thế lớn của bootstrap là nó có thể áp dụng trong gần như mọi tình huống mà không cần calculation toán học phức tạp. Dù Python có

những bootstrap implementation, việc dùng bootstrap để estimate standard error đơn giản đến mức ta tự viết function cho trường hợp data được lưu trong dataframe.

Ta bắt đầu bằng ví dụ đơn giản. Dataset **Portfolio** trong package ISLP được mô tả ở Section 5.2. Mục tiêu là estimate sampling variance của parameter  $\alpha$  trong formula (5.7). Ta tạo function `alpha_func()`, nhận input là dataframe `D` được giả định có các column `X`, `Y`, cùng vector `idx` chỉ các observation cần dùng để estimate  $\alpha$ . Function output estimate của  $\alpha$  từ các observation được chọn.

```
In [15]: Portfolio = load_data('Portfolio')
def alpha_func(D, idx):
 cov_ = np.cov(D[['X', 'Y']].loc[idx], rowvar=False)
 return ((cov_[1,1] - cov_[0,1]) /
 (cov_[0,0]+cov_[1,1]-2*cov_[0,1]))
```

Function này trả về estimate của  $\alpha$  bằng cách áp dụng minimum variance formula (5.7) lên các observation được index bởi `idx`. Chẳng hạn, command sau estimate  $\alpha$  bằng toàn bộ 100 observation.

```
In [16]: alpha_func(Portfolio, range(100))
```

```
Out[16]: 0.5758
```

Tiếp theo ta chọn ngẫu nhiên 100 observation từ `range(100)`, with replacement. Điều này tương đương tạo một bootstrap dataset mới rồi tính lại  $\hat{\alpha}$  trên dataset đó.

```
In [17]: rng = np.random.default_rng(0)
alpha_func(Portfolio,
 rng.choice(100,
 100,
 replace=True))
```

```
Out[17]: 0.6074
```

Process này có thể được generalized thành function đơn giản `boot_SE()` để tính bootstrap standard error cho arbitrary function chỉ nhận dataframe làm argument.

```
In [18]: def boot_SE(func,
 D,
 n=None,
 B=1000,
 seed=0):
 rng = np.random.default_rng(seed)
 first_, second_ = 0, 0
 n = n or D.shape[0]
 for _ in range(B):
 idx = rng.choice(D.index,
 n,
 replace=True)
 value = func(D, idx)
 first_ += value
 second_ += value**2
 return np.sqrt(second_ / B - (first_ / B)**2)
```

Lưu ý việc dùng `_` làm loop variable trong `for _ in range(B)`. Cách viết này thường được dùng khi value của counter không quan trọng và chỉ cần bảo đảm loop chạy đúng  $B$  lần.

Hãy dùng function này để đánh giá accuracy của estimate  $\alpha$  với  $B = 1,000$  bootstrap replication.

In [19]:

```
alpha_SE = boot_SE(alpha_func,
 Portfolio,
 B=1000,
 seed=0)

alpha_SE
```

Out[19]:

0.0912

Output cuối cho thấy bootstrap estimate của  $SE(\hat{\alpha})$  là 0.0912.

### *Estimating the Accuracy of a Linear Regression Model*

Bootstrap có thể dùng để đánh giá variability của coefficient estimate và prediction từ một statistical learning method. Ở đây ta dùng bootstrap để đánh giá variability của estimate  $\beta_0$  và  $\beta_1$ , tức intercept và slope của linear regression model dùng **horsepower** để dự đoán **mpg** trên dataset **Auto**. Ta sẽ so sánh estimate thu được từ bootstrap với estimate từ các formula cho  $SE(\hat{\beta}_0)$  và  $SE(\hat{\beta}_1)$  ở Section 3.1.2.

Để dùng function `boot_SE()`, ta phải viết một function nhận dataframe `D` và indices `idx` làm hai argument duy nhất. Nhưng ở đây ta muốn bootstrap một regression model cụ thể được chỉ định bởi model formula và data. Ta thực hiện điều đó qua vài bước đơn giản.

Trước hết ta viết generic function `boot_OLS()` để bootstrap regression model; function nhận một formula xác định regression tương ứng. Ta dùng function `clone()` để tạo copy của formula có thể được refit trên dataframe mới. Điều này có nghĩa các derived feature như những feature định nghĩa bằng `poly()` cũng sẽ được refit trên resampled dataframe.

*clone()*

In [20]:

```
def boot_OLS(model_matrix, response, D, idx):
 D_ = D.loc[idx]
 Y_ = D_[response]
 X_ = clone(model_matrix).fit_transform(D_)
 return sm.OLS(Y_, X_).fit().params
```

Đây vẫn chưa chính xác là thứ ta cần làm argument đầu tiên của `boot_SE()`. Hai argument đầu chỉ định model sẽ không thay đổi trong bootstrap process, nên ta muốn freeze chúng. Function `partial()` từ module `functools` làm đúng việc này: nó nhận một function làm argument rồi freeze một số argument của function đó, bắt đầu từ bên trái. Ta dùng nó để freeze hai model-formula argument đầu của `boot_OLS()`.

*partial()*

In [21]:

```
hp_func = partial(boot_OLS, MS(['horsepower']), 'mpg')
```

Gỡ `hp_func`? sẽ cho thấy function có hai argument `D` và `idx`: đây là version của `boot_OLS()` trong đó hai argument đầu đã được freeze, và vì thế rất phù hợp làm argument đầu cho `boot_SE()`.

Function `hp_func()` giờ có thể dùng để tạo bootstrap estimate cho intercept và slope term bằng cách sample ngẫu nhiên các observation with

replacement. Ta trước hết minh họa utility của nó trên 10 bootstrap sample.

```
In [22]: rng = np.random.default_rng(0)
np.array([hp_func(Auto,
 rng.choice(392,
 392,
 replace=True)) for _ in range(10)])
```

```
Out[22]: array([[39.8806, -0.1568],
 [38.733 , -0.147],
 [38.3173, -0.1444],
 [39.9145, -0.1578],
 [39.4335, -0.1507],
 [40.3663, -0.1591],
 [39.6233, -0.1545],
 [39.0581, -0.1495],
 [38.6669, -0.1452],
 [39.6428, -0.1556]])
```

Tiếp theo, ta dùng `boot_SE()` để tính standard error của 1,000 bootstrap estimate cho intercept và slope term.

```
In [23]: hp_se = boot_SE(hp_func,
 Auto,
 B=1000,
 seed=10)

hp_se
```

```
Out[23]: intercept 0.8488
horsepower 0.0074
dtype: float64
```

Điều này cho biết bootstrap estimate của  $SE(\hat{\beta}_0)$  là 0.85 và của  $SE(\hat{\beta}_1)$  là 0.0074. Như đã thảo luận trong Section 3.1.2, có thể dùng các standard formula để tính standard error cho regression coefficient trong linear model. Ta lấy chúng bằng function `summarize()`.

```
In [24]: hp_model.fit(Auto, Auto['mpg'])
model_se = summarize(hp_model.results_['std err'])
model_se
```

```
Out[24]: intercept 0.717
horsepower 0.006
Name: std err, dtype: float64
```

Standard error estimate cho  $\hat{\beta}_0$  và  $\hat{\beta}_1$  từ formula ở Section 3.1.2 lần lượt là 0.717 cho intercept và 0.006 cho slope. Khả thú vị là chúng khác phần nào với estimate từ bootstrap. Điều này có phải cho thấy bootstrap có vấn đề? Thực ra nó gợi ý điều ngược lại. Nhắc lại rằng các standard formula trong Equation 3.8 ở page 75 dựa trên một số assumption. Chẳng hạn, chúng phụ thuộc vào unknown parameter  $\sigma^2$ , noise variance, rồi ta estimate  $\sigma^2$  bằng RSS. Dù formula cho standard error không đòi linear model phải đúng, estimate của  $\sigma^2$  thì có. Figure 3.8 ở page 99 cho thấy data có non-linear relationship, nên residual từ linear fit sẽ bị inflate, kéo theo  $\hat{\sigma}^2$  cũng bị inflate. Thứ hai, standard formula giả định — hơi thiếu thực tế — rằng  $x_i$

là fixed và toàn bộ variability đến từ error  $\epsilon_i$ . Bootstrap không dựa trên các assumption này, vì vậy có khả năng nó cung cấp estimate chính xác hơn cho standard error của  $\hat{\beta}_0$  và  $\hat{\beta}_1$  so với kết quả từ `sm.OLS`.

Dưới đây ta tính bootstrap standard error estimate và standard linear regression estimate khi fit quadratic model. Vì model này fit data khá tốt (Figure 3.8), giờ đây bootstrap estimate và standard estimate của  $SE(\hat{\beta}_0)$ ,  $SE(\hat{\beta}_1)$ ,  $SE(\hat{\beta}_2)$  tương ứng tốt hơn.

```
In [25]: quad_model = MS([poly('horsepower', 2, raw=True)])
quad_func = partial(boot_OLS,
 quad_model,
 'mpg')
boot_SE(quad_func, Auto, B=1000)
```

```
Out[25]: intercept 2.067840
poly(horsepower, 2, raw=True)[0] 0.033019
poly(horsepower, 2, raw=True)[1] 0.000120
dtype: float64
```

Ta so sánh kết quả với standard error tính bằng `sm.OLS()`.

```
In [26]: M = sm.OLS(Auto['mpg'],
 quad_model.fit_transform(Auto))
summarize(M.fit())['std err']
```

```
Out[26]: intercept 1.800
poly(horsepower, 2, raw=True)[0] 0.031
poly(horsepower, 2, raw=True)[1] 0.000
Name: std err, dtype: float64
```

## 5.4 Exercises

### Conceptual

- Dùng các property thống kê cơ bản của variance, cùng single-variable calculus, hãy derive (5.6). Nói cách khác, chứng minh rằng  $\alpha$  trong (5.6) thực sự minimize  $\text{Var}(\alpha X + (1 - \alpha)Y)$ .
- Bây giờ ta sẽ derive probability rằng một observation cho trước thuộc một bootstrap sample. Giả sử ta lấy một bootstrap sample từ một tập gồm  $n$  observation.
  - Probability rằng bootstrap observation đầu tiên *không phải* observation thứ  $j$  của original sample là bao nhiêu? Hãy giải thích.
  - Probability rằng bootstrap observation thứ hai không phải observation thứ  $j$  của original sample là bao nhiêu?
  - Hãy lập luận rằng probability observation thứ  $j$  không nằm trong bootstrap sample là  $(1 - 1/n)^n$ .
  - Khi  $n = 5$ , probability observation thứ  $j$  nằm trong bootstrap sample là bao nhiêu?
  - Khi  $n = 100$ , probability observation thứ  $j$  nằm trong bootstrap sample là bao nhiêu?

- (f) Khi  $n = 10,000$ , probability observation thứ  $j$  nằm trong bootstrap sample là bao nhiêu?
- (g) Tạo một plot hiển thị, với mỗi integer  $n$  từ 1 đến 100,000, probability observation thứ  $j$  nằm trong bootstrap sample. Hãy nhận xét điều bạn quan sát được.
- (h) Bây giờ ta khảo sát bằng numerical experiment probability rằng một bootstrap sample size  $n = 100$  chứa observation thứ  $j$ , với  $j = 4$ . Trước hết tạo array **store** bằng function `np.empty()`, các value trong array sẽ được overwrite sau đó. Tiếp theo, liên tục tạo bootstrap sample, và mỗi lần ghi lại liệu observation thứ  $n$  có nằm trong bootstrap sample hay không.

`np.empty()`

```
rng = np.random.default_rng(10)
store = np.empty(10000)
for i in range(10000):
 store[i] = np.sum(rng.choice(100, replace=True) == 4)
 > 0
np.mean(store)
```

Hãy nhận xét kết quả thu được.

3. Bây giờ ta review  $k$ -fold cross-validation.
  - (a) Giải thích cách implement  $k$ -fold cross-validation.
  - (b) Ưu điểm và nhược điểm của  $k$ -fold cross-validation so với:
    - i. Validation set approach?
    - ii. LOOCV?
4. Giả sử ta dùng một statistical learning method để prediction response  $Y$  tại một giá trị cụ thể của predictor  $X$ . Hãy mô tả cẩn thận cách ta có thể estimate standard deviation của prediction đó.

### Applied

5. Trong Chapter 4, ta đã dùng logistic regression để dự đoán probability of default bằng **income** và **balance** trên dataset **Default**. Bây giờ ta estimate test error của logistic regression model này bằng validation set approach. Đừng quên đặt random seed trước khi bắt đầu analysis.
  - (a) Fit logistic regression model dùng **income** và **balance** để dự đoán **default**.
  - (b) Dùng validation set approach để estimate test error của model. Để làm điều này, bạn phải thực hiện các bước sau:
    - i. Chia sample thành training set và validation set.
    - ii. Fit multiple logistic regression model chỉ bằng các training observation.
    - iii. Tạo prediction về default status cho từng individual trong validation set bằng cách tính posterior probability of default cho individual đó, và classify individual vào default category nếu posterior probability lớn hơn 0.5.
    - iv. Tính validation set error, là fraction các observation trong validation set bị misclassified.



- (c) Lặp process trong (b) ba lần, dùng ba cách chia khác nhau các observation thành training set và validation set. Nhận xét kết quả thu được.
  - (d) Bây giờ xét logistic regression model dự đoán probability of default dùng **income**, **balance**, và một dummy variable cho **student**. Estimate test error cho model này bằng validation set approach. Nhận xét liệu việc thêm dummy variable cho **student** có làm giảm test error rate hay không.
6. Ta tiếp tục xét logistic regression model dự đoán probability of default bằng **income** và **balance** trên dataset **Default**. Cụ thể, bây giờ ta tính estimate standard error của logistic regression coefficient ứng với **income** và **balance** theo hai cách: (1) dùng bootstrap, và (2) dùng standard formula để tính standard error trong function **sm.GLM()**. Đừng quên đặt random seed trước khi bắt đầu analysis.
- (a) Dùng function **summarize()** và **sm.GLM()**, xác định estimated standard error của coefficient ứng với **income** và **balance** trong multiple logistic regression model dùng cả hai predictor.
  - (b) Viết function **boot\_fn()** nhận input là dataset **Default** cùng index của các observation, và output coefficient estimate cho **income** và **balance** trong multiple logistic regression model.
  - (c) Theo bootstrap example trong lab, dùng function **boot\_fn()** để estimate standard error của logistic regression coefficient cho **income** và **balance**.
  - (d) Nhận xét các estimated standard error thu được bằng function **sm.GLM()** và bằng bootstrap.
7. Trong Sections 5.1.2 và 5.1.3, ta thấy function **cross\_validate()** có thể dùng để tính LOOCV test error estimate. Một cách khác là chỉ dùng **sm.GLM()** và method **predict()** của fitted model trong một for loop. Bây giờ bạn sẽ dùng approach này để tính LOOCV error cho một simple logistic regression model trên dataset **Weekly**. Nhắc lại rằng trong classification problem, LOOCV error được cho bởi (5.4).
- (a) Fit logistic regression model dự đoán **Direction** bằng **Lag1** và **Lag2**.
  - (b) Fit logistic regression model dự đoán **Direction** bằng **Lag1** và **Lag2**, dùng tất cả observation trừ observation đầu tiên.
  - (c) Dùng model ở (b) để dự đoán direction của observation đầu tiên. Bạn có thể làm điều này bằng cách dự đoán observation đầu tiên sẽ **Up** nếu  $P(\text{Direction} = \text{"Up"} \mid \text{Lag1}, \text{Lag2}) > 0.5$ . Observation này có được classify đúng không?
  - (d) Viết for loop từ  $i = 1$  đến  $i = n$ , với  $n$  là số observation trong dataset, thực hiện từng bước sau:
    - i. Fit logistic regression model bằng tất cả observation trừ observation thứ  $i$  để dự đoán **Direction** bằng **Lag1** và **Lag2**.
    - ii. Tính posterior probability thị trường tăng cho observation thứ  $i$ .

- iii. Dùng posterior probability của observation thứ  $i$  để dự đoán thị trường có tăng hay không.
    - iv. Xác định prediction direction của observation thứ  $i$  có sai hay không. Nếu sai, ghi 1; nếu không, ghi 0.
  - (e) Lấy average của  $n$  số thu được ở (d)iv để có LOOCV estimate của test error. Nhận xét kết quả.
- 8. Bây giờ ta thực hiện cross-validation trên một simulated dataset.
  - (a) Generate simulated dataset như sau:
 

```
rng = np.random.default_rng(1)
x = rng.normal(size=100)
y = x - 2 * x**2 + rng.normal(size=100)
```

Trong dataset này,  $n$  và  $p$  bằng bao nhiêu? Viết model dùng để generate data dưới dạng equation.
  - (b) Tạo scatterplot của  $X$  so với  $Y$ . Nhận xét điều bạn thấy.
  - (c) Đặt random seed, rồi tính LOOCV error khi fit bốn model sau bằng least squares:
    - i.  $Y = \beta_0 + \beta_1 X + \epsilon$
    - ii.  $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$
    - iii.  $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \epsilon$
    - iv.  $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \beta_4 X^4 + \epsilon$ .

Lưu ý bạn có thể thấy hữu ích khi dùng function `data.frame()` để tạo một dataset duy nhất chứa cả  $X$  và  $Y$ .
  - (d) Lập (c) với một random seed khác và báo cáo kết quả. Kết quả có giống ở (c) không? Tại sao?
  - (e) Model nào trong (c) có LOOCV error nhỏ nhất? Đây có phải điều bạn kỳ vọng không? Hãy giải thích.
  - (f) Nhận xét statistical significance của coefficient estimate khi fit từng model trong (c) bằng least squares. Kết quả này có phù hợp với conclusion dựa trên cross-validation không?
- 9. Bây giờ ta xét Boston housing dataset từ ISLP library.
  - (a) Dựa trên dataset này, hãy đưa ra estimate cho population mean của `medv`. Gọi estimate này là  $\hat{\mu}$ .
  - (b) Đưa ra estimate standard error của  $\hat{\mu}$ . Hãy diễn giải kết quả.  
*Hint:* Ta có thể tính standard error của sample mean bằng sample standard deviation chia cho square root của số observation.
  - (c) Bây giờ estimate standard error của  $\hat{\mu}$  bằng bootstrap. Kết quả so với câu trả lời ở (b) thế nào?
  - (d) Dựa trên bootstrap estimate ở (c), đưa ra 95% confidence interval cho mean của `medv`. So sánh với kết quả thu được bằng `Boston['medv'].std()` và two standard error rule (3.9).  
*Hint:* Có thể xấp xỉ 95% confidence interval bằng formula  $[\hat{\mu} - 2SE(\hat{\mu}), \hat{\mu} + 2SE(\hat{\mu})]$ .

- (e) Dựa trên dataset này, đưa ra estimate  $\hat{\mu}_{med}$  cho median value của `medv` trong population.
- (f) Bây giờ ta muốn estimate standard error của  $\hat{\mu}_{med}$ . Không may, không có formula đơn giản để tính standard error của median. Thay vào đó, estimate standard error của median bằng bootstrap. Nhận xét kết quả.
- (g) Dựa trên dataset này, đưa ra estimate cho tenth percentile của `medv` trong Boston census tract. Gọi quantity này là  $\hat{\mu}_{0.1}$ . (Bạn có thể dùng function `np.percentile()`.) *np.percentile()*
- (h) Dùng bootstrap để estimate standard error của  $\hat{\mu}_{0.1}$ . Nhận xét kết quả.

# 6

## Linear Model Selection and Regularization

Trong regression setting, standard linear model

$$Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \varepsilon \quad (6.1)$$

được dùng rất phổ biến để mô tả relationship giữa một response  $Y$  và một tập variable  $X_1, X_2, \dots, X_p$ . Ở Chapter 3, ta đã thấy model này thường được fit bằng least squares.

Trong các chapter tiếp theo, ta sẽ xét một số approach mở rộng linear model framework. Ở Chapter 7, ta tổng quát hóa (6.1) để accommodate các relationship non-linear nhưng vẫn additive; còn ở Chapters 8 và 10, ta xét những non-linear model tổng quát hơn nữa. Tuy nhiên, linear model có những ưu điểm rõ rệt về inference và, trên các real-world problem, thường cạnh tranh đáng ngạc nhiên so với non-linear methods. Vì vậy, trước khi chuyển sang thế giới non-linear, chapter này thảo luận một số cách cải thiện simple linear model bằng cách thay plain least squares fitting bằng các fitting procedure khác.

Tại sao ta lại muốn dùng một fitting procedure khác thay cho least squares? Như sẽ thấy, các alternative fitting procedure có thể mang lại prediction accuracy tốt hơn và model interpretability cao hơn.

- **Prediction Accuracy:** Nếu true relationship giữa response và predictors xấp xỉ linear, các least squares estimate sẽ có bias thấp. Nếu  $n \gg p$  — tức  $n$ , số observation, lớn hơn nhiều so với  $p$ , số variable — thì các least squares estimate cũng thường có variance thấp và do đó hoạt động tốt trên test observation. Tuy nhiên, nếu  $n$  không lớn hơn  $p$  nhiều, least squares fit có thể biến động mạnh, dẫn đến overfitting và prediction kém trên future observation không được dùng để training model. Nếu  $p > n$ , thậm chí không còn một least squares coefficient estimate duy nhất: có vô số solution. Mỗi least squares solution đó cho error bằng zero trên training data, nhưng thường có test-set performance rất kém do variance cực cao.<sup>1</sup> Bằng cách con-

---

<sup>1</sup>Khi  $p \geq n$ , least squares solution có tổng bình phương coefficient nhỏ nhất đôi khi có thể

strain hoặc shrink các estimated coefficient, ta thường có thể giảm variance đáng kể với cái giá là bias chỉ tăng rất ít. Điều này có thể cải thiện mạnh accuracy khi dự đoán response cho các observation không được dùng trong model training.

- **Model Interpretability:** Trong nhiều multiple regression model, một phần hoặc nhiều variable thực tế không liên quan đến response. Việc đưa các irrelevant variable này vào làm model phức tạp không cần thiết. Nếu loại bỏ chúng — tức đặt các coefficient estimate tương ứng bằng zero — ta có thể thu được model dễ interpret hơn. Least squares hầu như không bao giờ tạo ra coefficient estimate chính xác bằng zero. Trong chapter này, ta sẽ gặp một số approach tự động thực hiện *feature selection* hay *variable selection*, tức loại các irrelevant variable khỏi multiple regression model.

*feature selection*  
*variable selection*

Có rất nhiều alternative, cả classical lẫn modern, cho việc dùng least squares để fit (6.1). Trong chapter này, ta thảo luận ba class method quan trọng.

- **Subset Selection.** Approach này xác định một subset trong số  $p$  predictors mà ta tin rằng có liên quan đến response. Sau đó ta fit model bằng least squares trên tập variable đã được rút gọn.
- **Shrinkage.** Approach này fit một model chứa toàn bộ  $p$  predictors, nhưng các estimated coefficient bị shrink về phía zero so với least squares estimate. Sự shrinkage này (cũng gọi là regularization) làm giảm variance. Tùy loại shrinkage, một số coefficient có thể được estimate chính xác bằng zero; vì vậy shrinkage method cũng có thể thực hiện variable selection.
- **Dimension Reduction.** Approach này project  $p$  predictors lên một  $M$ -dimensional subspace, với  $M < p$ . Việc này được thực hiện bằng cách tính  $M$  linear combination, hay projection, khác nhau của các variable. Sau đó  $M$  projection này được dùng làm predictors để fit linear regression model bằng least squares.

Trong các section sau, ta mô tả chi tiết từng approach cùng ưu và nhược điểm của chúng. Mặc dù chapter này trình bày các extension và modification của linear model cho regression ở Chapter 3, những concept tương tự cũng áp dụng cho các method khác, chẳng hạn classification model ở Chapter 4.

## 6.1 Subset Selection

Trong section này, ta xét một số method chọn subset của predictors, bao gồm best subset và stepwise model selection procedure.

### 6.1.1 Best Subset Selection

Để thực hiện *best subset selection*, ta fit một least squares regression riêng cho mọi combination có thể của  $p$  predictors. Cụ thể, ta fit toàn bộ  $p$  model chứa đúng một predictor, toàn bộ  $\binom{p}{2} = p(p-1)/2$  model chứa đúng hai predictors, và tiếp tục như vậy. Sau đó ta xem xét toàn bộ model thu được, với mục tiêu xác định model tốt nhất.

Việc chọn best model trong số  $2^p$  possibility được xét bởi best subset selection không hề đơn giản. Thường ta chia bài toán thành hai stage, như Algorithm 6.1.

#### Algorithm 6.1 Best subset selection

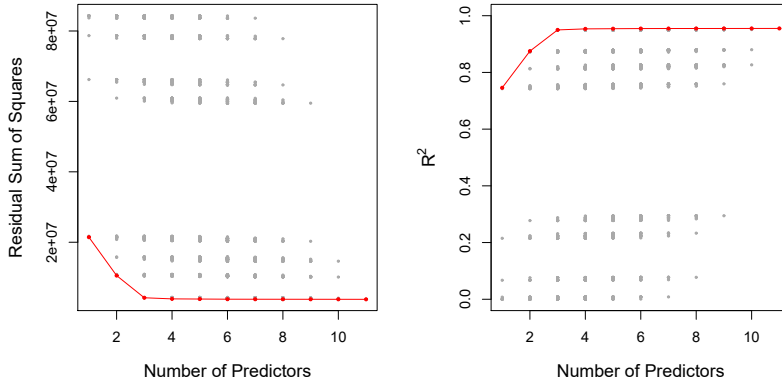
1. Gọi  $\mathcal{M}_0$  là null model, không chứa predictor nào. Model này đơn giản dự đoán sample mean cho mọi observation.
2. Với  $k = 1, 2, \dots, p$ :
  - (a) Fit toàn bộ  $\binom{p}{k}$  model chứa đúng  $k$  predictors.
  - (b) Chọn model tốt nhất trong  $\binom{p}{k}$  model này và gọi là  $\mathcal{M}_k$ . Ở đây, “best” nghĩa là có RSS nhỏ nhất, hoặc tương đương  $R^2$  lớn nhất.
3. Chọn một best model duy nhất trong  $\mathcal{M}_0, \dots, \mathcal{M}_p$  bằng prediction error trên validation set,  $C_p$  (AIC), BIC, adjusted  $R^2$ , hoặc cross-validation.

Trong Algorithm 6.1, Step 2 xác định best model (trên training data) cho mỗi subset size, nhờ đó rút bài toán từ  $2^p$  possible model xuống còn  $p+1$  possible model. Trong Figure 6.1, các model này tạo thành lower frontier màu đỏ.

Để chọn một best model duy nhất, ta chỉ còn phải chọn trong  $p+1$  option đó. Tuy nhiên việc này cần cẩn trọng, vì RSS của  $p+1$  model giảm đơn điệu còn  $R^2$  tăng đơn điệu khi số feature trong model tăng. Nếu dùng trực tiếp hai statistic này để chọn best model, ta sẽ luôn kết thúc với model chứa tất cả variable. Vấn đề là RSS thấp hoặc  $R^2$  cao chỉ ra training error thấp, trong khi điều ta muốn là test error thấp. Như Figures 2.9–2.11 ở Chapter 2 cho thấy, training error thường nhỏ hơn test error khá nhiều và training error thấp hoàn toàn không đảm bảo test error thấp. Vì vậy, ở Step 3 ta dùng error trên validation set,  $C_p$ , BIC, adjusted  $R^2$ , hoặc cross-validation để chọn giữa  $\mathcal{M}_0, \mathcal{M}_1, \dots, \mathcal{M}_p$ . Nếu dùng cross-validation, Step 2 được lặp riêng trên từng training fold rồi validation error được average để chọn giá trị  $k$  tốt nhất.

Sau khi chọn được  $k$ , model  $\mathcal{M}_k$  được fit trên toàn bộ training set và được dùng làm model cuối. Các approach để chọn  $k$  được thảo luận ở Section 6.1.3.

Figure 6.1 minh họa best subset selection trên dataset **Credit**, đã được



**HÌNH 6.1.** Với mỗi possible model chứa một subset của mười predictors trong dataset *Credit*, RSS và  $R^2$  được hiển thị. Red frontier nối best model cho từng số lượng predictors, theo RSS và  $R^2$ . Mặc dù dataset chỉ có mười predictors, trục  $x$  chạy từ 1 đến 11 vì một variable là categorical với ba value, dẫn đến việc tạo hai dummy variable.

thảo luận ở Chapter 3. Mỗi point là một least squares regression model fit bằng một subset khác nhau của 10 predictors. Variable **region** là qualitative variable có ba level nên được biểu diễn bằng hai dummy variable, và trong trường hợp này hai dummy variable được chọn riêng rẽ. Vì thế có tổng cộng 11 possible variable có thể xuất hiện trong model. Ta plot RSS và  $R^2$  của mỗi model theo số variable. Các red curve nối best model ở mỗi model size, dựa trên RSS hoặc  $R^2$ . Đúng như kỳ vọng, hai quantity này cải thiện khi số variable tăng; tuy nhiên từ model ba-variable trở đi, việc thêm predictor chỉ cải thiện RSS và  $R^2$  rất ít.

Dù ở đây ta trình bày best subset selection cho least squares regression, cùng ý tưởng cũng áp dụng cho các model khác, chẳng hạn logistic regression. Với logistic regression, thay vì sắp xếp model theo RSS ở Step 2 của Algorithm 6.1, ta dùng *deviance*, một measure đóng vai trò giống RSS cho một class model rộng hơn. Deviance bằng âm hai lần maximized log-likelihood; deviance càng nhỏ thì fit càng tốt.

*deviance*

Best subset selection đơn giản và hấp dẫn về mặt concept, nhưng có limitation về computation. Số possible model phải xét tăng rất nhanh theo  $p$ . Nói chung, có  $2^p$  model hình thành từ các subset của  $p$  predictors. Nếu  $p = 10$ , có khoảng 1,000 possible model; nếu  $p = 20$ , có hơn một triệu possibility. Do đó best subset selection trở nên computationally infeasible khi  $p$  lớn hơn khoảng 40, ngay cả với computer hiện đại rất nhanh. Có các computational shortcut — gọi là branch-and-bound technique — để loại bớt một số choice, nhưng chúng cũng có limitation khi  $p$  lớn và chỉ áp dụng cho least squares linear regression. Tiếp theo ta xét các alternative hiệu quả hơn về computation.

### 6.1.2 Stepwise Selection

Vì lý do computation, best subset selection không thể áp dụng khi  $p$  rất lớn. Khi  $p$  lớn, best subset selection còn có thể gặp statistical problem: search space càng lớn thì càng dễ tìm được model trông rất tốt trên training data nhưng không có predictive power trên future data. Vì vậy một search space khổng lồ có thể dẫn đến overfitting và variance cao của coefficient estimate. Do cả hai lý do này, stepwise method — vốn chỉ khám phá một tập model hạn chế hơn nhiều — là alternative hấp dẫn cho best subset selection.

#### Forward Stepwise Selection

*Forward stepwise selection* là một alternative hiệu quả về computation cho best subset selection. Nếu best subset selection xét tất cả  $2^p$  possible model chứa subset của  $p$  predictors, forward stepwise chỉ xét một tập model nhỏ hơn rất nhiều. Forward stepwise bắt đầu bằng model không có predictor, rồi thêm predictor vào model từng variable một cho đến khi toàn bộ predictors đều đã được đưa vào. Ở mỗi step, variable tạo ra improvement lớn nhất cho fit được thêm vào model. Procedure chính thức được cho trong Algorithm 6.2.

*forward stepwise  
selection*

#### Algorithm 6.2 Forward stepwise selection

1. Gọi  $\mathcal{M}_0$  là null model, không chứa predictor nào.
2. Với  $k = 0, \dots, p - 1$ :
  - (a) Xét toàn bộ  $p - k$  model tạo ra bằng cách thêm một predictor vào tập predictor của  $\mathcal{M}_k$ .
  - (b) Chọn best model trong số  $p - k$  model này và gọi là  $\mathcal{M}_{k+1}$ . “Best” nghĩa là có RSS nhỏ nhất hoặc  $R^2$  lớn nhất.
3. Chọn một best model trong  $\mathcal{M}_0, \dots, \mathcal{M}_p$  bằng prediction error trên validation set,  $C_p$  (AIC), BIC, adjusted  $R^2$ , hoặc cross-validation.

Khác với best subset selection phải fit  $2^p$  model, forward stepwise selection fit một null model và  $p - k$  model ở iteration thứ  $k$ , với  $k = 0, \dots, p - 1$ . Tổng cộng là

$$1 + \sum_{k=0}^{p-1} (p - k) = 1 + \frac{p(p+1)}{2}$$

model. Khác biệt là rất lớn: khi  $p = 20$ , best subset selection cần fit 1,048,576 model, còn forward stepwise chỉ cần 211 model.<sup>2</sup>

Ở Step 2(b) của Algorithm 6.2, ta phải xác định best model trong số  $p - k$  model được tạo bằng cách thêm một predictor vào  $\mathcal{M}_k$ . Ta có thể đơn giản chọn model có RSS thấp nhất hoặc  $R^2$  cao nhất. Nhưng ở Step 3, ta phải xác định best model trong một tập model có số variable khác nhau; bài toán này khó hơn và được thảo luận ở Section 6.1.3.

Lợi thế computation của forward stepwise selection so với best subset selection là rõ ràng. Dù forward stepwise thường hoạt động tốt trong thực

<sup>2</sup>Mặc dù forward stepwise selection xét  $p(p+1)/2 + 1$  model một cách trực tiếp, nó thực hiện guided search trong model space, nên effective model space được xét lớn hơn đáng kể con số này.



| # Variables | Best subset             | Forward stepwise        |
|-------------|-------------------------|-------------------------|
| One         | rating                  | rating                  |
| Two         | rating, income          | rating, income          |
| Three       | rating, income, student | rating, income, student |
| Four        | cards, income           | rating, income,         |
|             | student, limit          | student, limit          |

**BẢNG 6.1.** Bốn model đầu tiên được chọn bởi best subset selection và forward stepwise selection trên dataset *Credit*. Ba model đầu giống nhau, nhưng model thứ tư khác nhau.

tế, nó không được đảm bảo tìm ra best possible model trong tất cả  $2^p$  model. Ví dụ, giả sử dataset có  $p = 3$  predictors, best possible one-variable model chứa  $X_1$ , trong khi best possible two-variable model lại chứa  $X_2$  và  $X_3$ . Forward stepwise sẽ không thể chọn best two-variable model vì  $\mathcal{M}_1$  đã chứa  $X_1$ , nên  $\mathcal{M}_2$  bắt buộc cũng chứa  $X_1$  cùng một variable khác.

Table 6.1 minh họa phenomenon này trên dataset *Credit*. Cả best subset selection lẫn forward stepwise selection đều chọn **rating** cho best one-variable model, rồi thêm **income** và **student** vào two- và three-variable model. Tuy nhiên, ở four-variable model, best subset selection thay **rating** bằng **cards**, trong khi forward stepwise phải giữ **rating**. Figure 6.1 cho thấy trong ví dụ này three- và four-variable model không khác nhau nhiều về RSS, vì thế cả hai four-variable model đều có thể đủ tốt.

Forward stepwise selection có thể áp dụng ngay cả trong high-dimensional setting  $n < p$ ; tuy nhiên khi đó chỉ có thể xây dựng các submodel  $\mathcal{M}_0, \dots, \mathcal{M}_{n-1}$ , vì mỗi submodel được fit bằng least squares, vốn không có unique solution nếu số predictor trong model lớn hơn hoặc bằng  $n$ .

### Backward Stepwise Selection

Giống forward stepwise selection, *backward stepwise selection* là một alternative hiệu quả cho best subset selection. Tuy nhiên, thay vì bắt đầu từ null model, backward stepwise bắt đầu bằng full least squares model chứa toàn bộ  $p$  predictors, rồi lặp đi lặp lại việc loại predictor ít hữu ích nhất, từng variable một. Chi tiết ở Algorithm 6.3.

*backward stepwise  
selection*

#### Algorithm 6.3 Backward stepwise selection

1. Gọi  $\mathcal{M}_p$  là full model, chứa toàn bộ  $p$  predictors.
2. Với  $k = p, p - 1, \dots, 1$ :
  - (a) Xét toàn bộ  $k$  model chứa tất cả predictors trong  $\mathcal{M}_k$  trừ một predictor, nên mỗi model có  $k - 1$  predictors.
  - (b) Chọn best model trong số  $k$  model này và gọi là  $\mathcal{M}_{k-1}$ . “Best” nghĩa là RSS nhỏ nhất hoặc  $R^2$  lớn nhất.
3. Chọn một best model trong  $\mathcal{M}_0, \dots, \mathcal{M}_p$  bằng prediction error trên validation set,  $C_p$  (AIC), BIC, adjusted  $R^2$ , hoặc cross-validation.

Giống forward stepwise selection, backward selection chỉ search qua  $1 + p(p+1)/2$  model và vì thế có thể dùng khi  $p$  quá lớn để best subset selection

khả thi.<sup>3</sup> Nó cũng không được đảm bảo tạo ra best model trong tất cả subset của  $p$  predictors.

Backward selection yêu cầu số sample  $n$  lớn hơn số variable  $p$  để full model có thể được fit. Ngược lại, forward stepwise dùng được ngay cả khi  $n < p$ , vì vậy đây là subset method khả thi duy nhất khi  $p$  rất lớn.

### Hybrid Approaches

Best subset, forward stepwise và backward stepwise selection thường cho các model tương tự nhưng không hoàn toàn giống nhau. Một alternative khác là các hybrid version của forward và backward stepwise selection. Trong đó variable được thêm vào model sequentially như forward selection, nhưng sau mỗi lần thêm variable mới, method cũng có thể loại đi những variable không còn tạo improvement cho model fit. Approach này cố gắng mô phỏng best subset selection sát hơn trong khi vẫn giữ computational advantage của forward và backward stepwise selection.

#### 6.1.3 Choosing the Optimal Model

Best subset selection, forward selection và backward selection tạo ra một tập model, mỗi model chứa một subset của  $p$  predictors. Để dùng các method này, ta cần một cách xác định model nào là best. Như đã thảo luận ở Section 6.1.1, model chứa toàn bộ predictors luôn có RSS nhỏ nhất và  $R^2$  lớn nhất vì các quantity này gắn với training error. Nhưng điều ta muốn là model có test error thấp. Training error có thể là estimate rất kém của test error; vì thế RSS và  $R^2$  không phù hợp để chọn best model trong một collection model có số predictor khác nhau.

Để chọn best model theo test error, ta cần estimate test error. Có hai approach phổ biến:

1. Indirectly estimate test error bằng cách điều chỉnh training error để account cho bias do overfitting.
2. Directly estimate test error bằng validation set approach hoặc cross-validation approach, như đã thảo luận ở Chapter 5.

Ta xét cả hai approach dưới đây.

#### $C_p$ , AIC, BIC, and Adjusted $R^2$

Chapter 2 cho thấy training-set MSE nói chung underestimate test MSE (nhắc lại  $MSE = RSS/n$ ). Khi fit model trên training data bằng least squares, ta cụ thể chọn regression coefficient sao cho training RSS — chứ không phải test RSS — nhỏ nhất có thể. Đặc biệt, training error sẽ giảm khi thêm variable vào model, trong khi test error thì không nhất thiết giảm. Do đó training-set RSS và training-set  $R^2$  không thể dùng để chọn giữa các model có số variable khác nhau.

Tuy nhiên, có một số technique điều chỉnh training error theo model size. Các approach này cho phép chọn giữa các model có số variable khác nhau. Ta xét bốn approach:  $C_p$ , Akaike information criterion (AIC), Bayesian information criterion (BIC), và adjusted  $R^2$ . Figure 6.2 hiển thị  $C_p$ , BIC

<sup>3</sup>Giống forward stepwise selection, backward stepwise selection thực hiện guided search trong model space nên effective number of model được xét lớn hơn đáng kể  $1 + p(p+1)/2$ .

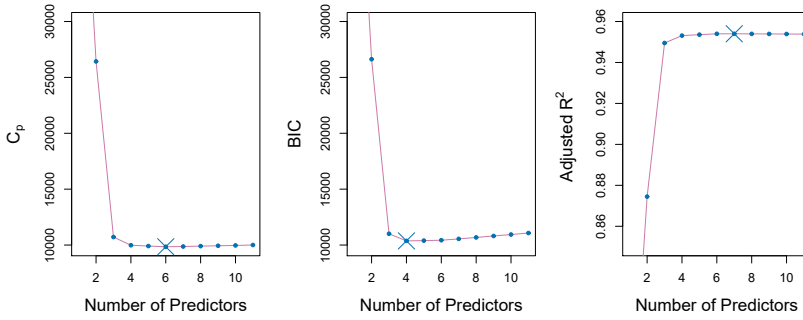
và adjusted  $R^2$  cho best model ở mỗi size do best subset selection tạo ra trên dataset **Credit**.

Với một fitted least squares model chứa  $d$  predictors,  $C_p$  estimate của test MSE được tính bởi

$$C_p = \frac{1}{n} (\text{RSS} + 2d\hat{\sigma}^2), \quad (6.2)$$

trong đó  $\hat{\sigma}^2$  là estimate của variance của error  $\varepsilon$  gắn với mỗi response measurement trong (6.1).<sup>4</sup> Thông thường  $\hat{\sigma}^2$  được estimate bằng full model chứa tất cả predictors. Về bản chất, statistic  $C_p$  cộng penalty  $2d\hat{\sigma}^2$  vào training RSS để điều chỉnh việc training error có xu hướng underestimate test error. Penalty tăng khi số predictor trong model tăng.

$C_p$   
Akaike information  
criterion  
Bayesian  
information  
criterion  
adjusted  $R^2$



**HÌNH 6.2.**  $C_p$ , BIC và adjusted  $R^2$  cho best model ở mỗi size trên dataset **Credit** (lower frontier trong Figure 6.1).  $C_p$  và BIC là estimate của test MSE. Ở middle plot, BIC estimate của test error tăng sau khi bốn variable được chọn. Hai plot còn lại khá flat sau khi bốn variable được đưa vào.

Mục đích của penalty là account cho decrease tương ứng trong training RSS khi thêm predictor. Dù nằm ngoài scope của sách, có thể chứng minh rằng nếu  $\hat{\sigma}^2$  là unbiased estimate của  $\sigma^2$  trong (6.2), thì  $C_p$  là unbiased estimate của test MSE. Vì vậy  $C_p$  có xu hướng nhỏ với model có test error thấp, và ta chọn model có  $C_p$  nhỏ nhất. Trong Figure 6.2,  $C_p$  chọn six-variable model chứa predictors **income**, **limit**, **rating**, **cards**, **age** và **student**.

AIC criterion được định nghĩa cho một class model rộng được fit bằng maximum likelihood. Với model (6.1) có Gaussian error, maximum likelihood và least squares là cùng một procedure. Trong trường hợp này,

$$\text{AIC} = \frac{1}{n} (\text{RSS} + 2d\hat{\sigma}^2),$$

trong đó để đơn giản ta đã bỏ các constant không liên quan.<sup>5</sup> Vì vậy, với

<sup>4</sup>Mallow's  $C_p$  đôi khi được định nghĩa là  $C'_p = \text{RSS}/\hat{\sigma}^2 + 2d - n$ . Định nghĩa này tương đương với định nghĩa ở đây theo nghĩa  $C_p = (\hat{\sigma}^2/n)(C'_p + n)$ , nên model có  $C_p$  nhỏ nhất cũng có  $C'_p$  nhỏ nhất.

<sup>5</sup>Có hai formula cho AIC trong least squares regression. Formula ở đây cần một expression cho  $\sigma^2$ , được estimate bằng full model chứa tất cả predictors. Formula thứ hai phù hợp khi  $\sigma^2$  unknown và ta không muốn estimate trực tiếp; khi đó có term  $\log(\text{RSS})$  thay cho RSS. Derivation chi tiết của hai formula nằm ngoài scope của sách.

least squares model,  $C_p$  và AIC tỷ lệ với nhau nên Figure 6.2 chỉ hiển thị  $C_p$ .

BIC được suy ra từ Bayesian point of view nhưng cuối cùng cũng có dạng tương tự  $C_p$  và AIC. Với least squares model chứa  $d$  predictors, bỏ qua các constant không liên quan,

$$\text{BIC} = \frac{1}{n} (\text{RSS} + \log(n)d\hat{\sigma}^2). \quad (6.3)$$

Giống  $C_p$ , BIC thường nhỏ với model có test error thấp, nên nhìn chung ta chọn model có BIC nhỏ nhất. BIC thay  $2d\hat{\sigma}^2$  trong  $C_p$  bằng  $\log(n)d\hat{\sigma}^2$ . Vì  $\log n > 2$  khi  $n > 7$ , BIC thường penalize model có nhiều variable mạnh hơn và do đó có xu hướng chọn model nhỏ hơn  $C_p$ .

Figure 6.2 cho thấy đúng phenomenon này trên dataset **Credit**: BIC chọn model chỉ gồm bốn predictors **income**, **limit**, **cards** và **student**. Trong ví dụ này các curve rất flat, nên accuracy của four-variable và six-variable model có vẻ không khác nhau nhiều.

Adjusted  $R^2$  là một approach phổ biến khác để chọn giữa các model có số variable khác nhau. Nhắc lại  $R^2 = 1 - \text{RSS}/\text{TSS}$ , với  $\text{TSS} = \sum_i (y_i - \bar{y})^2$ . Vì RSS luôn giảm khi thêm variable,  $R^2$  luôn tăng. Với least squares model có  $d$  variables, adjusted  $R^2$  được tính bởi

$$\text{Adjusted } R^2 = 1 - \frac{\text{RSS}/(n - d - 1)}{\text{TSS}/(n - 1)}. \quad (6.4)$$

Khác  $C_p$ , AIC và BIC, nơi value nhỏ gợi ý test error thấp, adjusted  $R^2$  càng lớn càng tốt. Maximizing adjusted  $R^2$  tương đương minimizing  $\text{RSS}/(n - d - 1)$ . Dù RSS luôn giảm khi tăng số variable, quantity này có thể tăng hoặc giảm vì  $d$  cũng xuất hiện trong denominator.

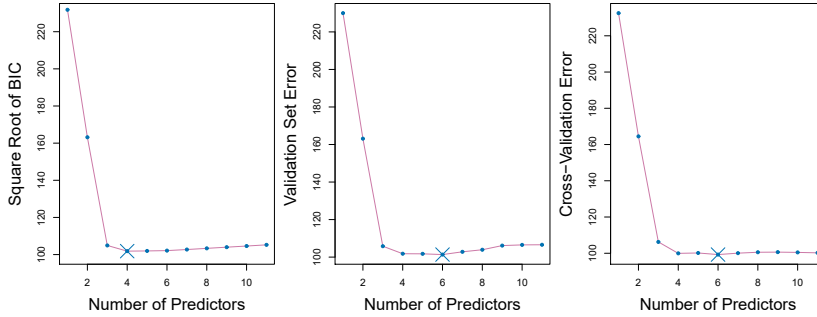
Intuition của adjusted  $R^2$  là: sau khi tất cả correct variable đã được đưa vào model, thêm noise variable chỉ làm RSS giảm rất ít. Nhưng vì noise variable làm  $d$  tăng, chúng có thể làm  $\text{RSS}/(n - d - 1)$  tăng và adjusted  $R^2$  giảm. Về lý thuyết, model có adjusted  $R^2$  lớn nhất sẽ chỉ chứa correct variable và không chứa noise variable. Khác ordinary  $R^2$ , adjusted  $R^2$  phải trả một “price” cho việc đưa unnecessary variable vào. Với dataset **Credit**, statistic này chọn seven-variable model, tức thêm **own** vào model được  $C_p$  và AIC chọn.

$C_p$ , AIC và BIC đều có theoretical justification chặt chẽ nhưng nằm ngoài scope của sách; chúng dựa trên asymptotic argument, tức scenario khi sample size  $n$  rất lớn. Dù phổ biến và khá intuitive, adjusted  $R^2$  không được motivate trong statistical theory tốt bằng AIC, BIC và  $C_p$ . Tất cả measure này đều dễ dùng và dễ compute. Ở đây ta trình bày formula cho linear model fit bằng least squares, nhưng AIC và BIC cũng có thể được định nghĩa cho các model tổng quát hơn.

### Validation and Cross-Validation

Một alternative là estimate test error trực tiếp bằng validation set và cross-validation method ở Chapter 5. Ta compute validation-set error hoặc cross-validation error cho từng model đang xét, rồi chọn model có estimated test error nhỏ nhất. Procedure này có advantage so với AIC, BIC,  $C_p$  và adjusted  $R^2$ : nó cho direct estimate của test error và đặt ít assumption

hơn lên true underlying model. Nó cũng áp dụng được cho nhiều model-selection task hơn, kể cả khi khó xác định model degrees of freedom (ví dụ số predictors) hoặc khó estimate error variance  $\sigma^2$ .



**HÌNH 6.3.** Với dataset *Credit*, ba quantity được hiển thị cho best model chứa  $d$  predictors, với  $d$  từ 1 đến 11. Overall best model theo từng quantity được đánh dấu bằng blue cross. Left: Square root của BIC. Center: Validation-set error. Right: Cross-validation error.

Khi dùng cross-validation, sequence  $\mathcal{M}_k$  trong Algorithms 6.1–6.3 được xác định riêng cho từng training fold, rồi validation error được average qua tất cả fold cho mỗi model size  $k$ . Điều này có nghĩa, chẳng hạn với best-subset regression,  $\mathcal{M}_k$  — best subset size  $k$  — có thể khác nhau giữa các fold. Sau khi chọn best size  $k$ , ta tìm best model có size đó trên full dataset.

Trước đây, cross-validation có thể quá tốn computation đối với problem có  $p$  và/hoặc  $n$  lớn, nên AIC, BIC,  $C_p$  và adjusted  $R^2$  hấp dẫn hơn để chọn trong một tập model. Với computer hiện nay, computation cho cross-validation hầu như không còn là vấn đề. Do đó cross-validation là approach rất hấp dẫn cho model selection.

Figure 6.3 hiển thị BIC, validation-set error và cross-validation error trên *Credit* theo  $d$ , cho best  $d$ -variable model. Validation error được tính bằng cách chọn ngẫu nhiên ba phần tư observation làm training set và phần còn lại làm validation set. Cross-validation error dùng  $k = 10$  folds. Trong trường hợp này, cả validation lẫn cross-validation đều chọn six-variable model. Tuy nhiên, cả ba approach đều gợi ý four-, five- và six-variable model gần như tương đương về test error.

Thực tế, các estimated test-error curve ở center và right panel của Figure 6.3 khá flat. Three-variable model rõ ràng có estimated test error thấp hơn two-variable model, nhưng model từ 3 đến 11 variable có error khá giống nhau. Nếu lặp validation set approach với một split khác, hoặc lặp cross-validation bằng các fold khác, precise model có estimated test error nhỏ nhất chắc chắn có thể thay đổi.

Trong setting như vậy, ta có thể dùng *one-standard-error rule*. Trước hết, tính standard error của estimated test MSE cho mỗi model size; sau đó chọn model nhỏ nhất có estimated test error nằm trong một standard error của lowest point trên curve. Lý do là nếu một nhóm model có vẻ tốt gần như ngang nhau, ta nên chọn model đơn giản nhất — tức model có ít predictors nhất. Ở đây, áp dụng one-standard-error rule cho validation-set

one-standard-error  
rule

hoặc cross-validation approach đều dẫn đến three-variable model.

## 6.2 Shrinkage Methods

Các subset selection method ở Section 6.1 dùng least squares để fit một linear model chỉ chứa một subset của predictors. Một alternative là fit model chứa toàn bộ  $p$  predictors bằng technique constrain hoặc regularize coefficient estimate, hay tương đương là shrink các coefficient estimate về zero. Ban đầu có thể chưa rõ tại sao constraint như vậy lại cải thiện fit, nhưng thực tế việc shrink coefficient estimate có thể làm variance giảm đáng kể. Hai technique nổi tiếng nhất để shrink regression coefficient về zero là ridge regression và lasso.

### 6.2.1 Ridge Regression

Nhắc lại từ Chapter 3 rằng least squares fitting procedure estimate  $\beta_0, \beta_1, \dots, \beta_p$  bằng các value minimize

$$\text{RSS} = \sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2.$$

Ridge regression rất giống least squares, chỉ khác coefficient được estimate bằng cách minimize một quantity hơi khác. Cụ thể, ridge regression coefficient estimate  $\hat{\beta}^R$  là các value minimize

$$\sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2 = \text{RSS} + \lambda \sum_{j=1}^p \beta_j^2, \quad (6.5)$$

trong đó  $\lambda \geq 0$  là một *tuning parameter* được xác định riêng. Equation (6.5) trade off hai criterion. Giống least squares, ridge regression tìm coefficient estimate fit data tốt bằng cách giữ RSS nhỏ. Nhưng term thứ hai,  $\lambda \sum_j \beta_j^2$ , gọi là *shrinkage penalty*, nhỏ khi  $\beta_1, \dots, \beta_p$  gần zero và vì thế có tác dụng shrink các estimate  $\beta_j$  về zero. Tuning parameter  $\lambda$  điều khiển relative impact của hai term.

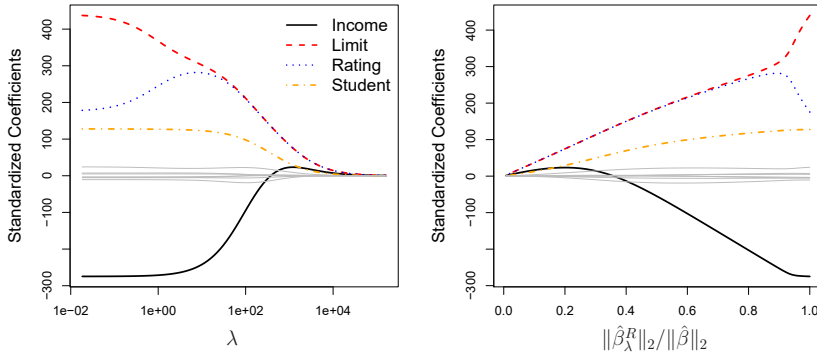
ridge regression  
tuning parameter  
shrinkage penalty

Khi  $\lambda = 0$ , penalty term không có effect và ridge regression cho đúng least squares estimate. Khi  $\lambda \rightarrow \infty$ , shrinkage penalty ngày càng chi phối và ridge coefficient estimate tiến về zero. Khác least squares chỉ tạo một bộ coefficient estimate, ridge regression tạo một bộ  $\hat{\beta}_\lambda^R$  khác nhau cho mỗi  $\lambda$ . Vì thế chọn một  $\lambda$  tốt là critical; Section 6.2.3 sẽ dùng cross-validation cho việc này.

Trong (6.5), shrinkage penalty áp dụng cho  $\beta_1, \dots, \beta_p$  nhưng không áp dụng cho intercept  $\beta_0$ . Ta muốn shrink estimated association của từng variable với response, nhưng không muốn shrink intercept, vốn chỉ đo mean response khi  $x_{i1} = \dots = x_{ip} = 0$ . Nếu các column của data matrix  $X$  đã được center để có mean zero trước khi ridge regression, thì estimated intercept là  $\hat{\beta}_0 = \bar{y} = \sum_i y_i/n$ .

### An Application to the Credit Data

Figure 6.4 hiển thị ridge regression coefficient estimate cho dataset **Credit**. Ở left panel, mỗi curve là coefficient estimate của một trong



**HÌNH 6.4.** Các standardized ridge regression coefficient trên dataset *Credit*, theo  $\lambda$  và  $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$ .

mười variable theo  $\lambda$ . Chẳng hạn black solid line là ridge estimate cho coefficient của **income**. Ở cực trái,  $\lambda$  gần zero nên coefficient gần như least squares estimate. Khi  $\lambda$  tăng, các ridge coefficient shrink về zero; với  $\lambda$  cực lớn, mọi estimate gần zero, tương ứng null model không predictor. Các variable **income**, **limit**, **rating** và **student** được vẽ bằng màu riêng vì có coefficient magnitude lớn nhất. Mặc dù aggregate coefficient thường giảm theo  $\lambda$ , một coefficient riêng lẻ như **rating** hoặc **income** đôi khi có thể tăng trong một đoạn khi  $\lambda$  tăng.

Right panel của Figure 6.4 hiển thị cùng các ridge coefficient nhưng trục  $x$  là  $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$ , trong đó  $\hat{\beta}$  là vector least squares coefficient estimate. Ký hiệu  $\|\beta\|_2$  là  $\ell_2$  norm (đọc là “ell 2”) của vector,

$$\|\beta\|_2 = \sqrt{\sum_{j=1}^p \beta_j^2},$$

và đo distance của  $\beta$  tới zero. Khi  $\lambda$  tăng,  $\ell_2$  norm của  $\hat{\beta}_\lambda^R$  luôn giảm, nên ratio này đi từ 1 khi  $\lambda = 0$  đến 0 khi  $\lambda = \infty$ . Vì vậy có thể xem trục  $x$  của right panel là mức độ các ridge coefficient đã bị shrink về zero.

$\ell_2$  norm

Standard least squares coefficient estimate là *scale equivariant*: nhân predictor  $X_j$  với constant  $c$  chỉ làm coefficient tương ứng scale bởi  $1/c$ , nên  $X_j \hat{\beta}_j$  không đổi. Ridge regression thì khác: coefficient estimate có thể thay đổi đáng kể khi scale một predictor. Chẳng hạn **income** có thể đo bằng dollars hoặc thousand dollars. Vì penalty trong (6.5) phụ thuộc trực tiếp vào tổng bình phương coefficient, thay đổi scale không chỉ đơn giản nhân ridge coefficient của **income** với 1,000. Giá trị  $X_j \hat{\beta}_{j,\lambda}^R$  phụ thuộc cả  $\lambda$  lẫn scale của predictor thứ  $j$ , thậm chí có thể phụ thuộc scale của các predictor khác. Vì vậy tốt nhất nên standardize predictors trước ridge regression:

$$\tilde{x}_{ij} = \frac{x_{ij}}{\sqrt{\frac{1}{n} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}}, \quad (6.6)$$

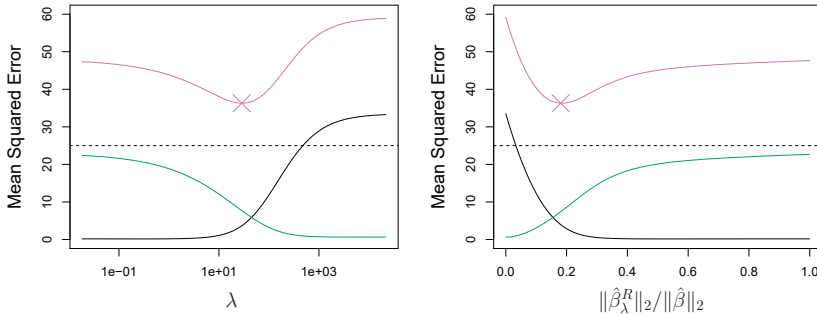
để tất cả predictor ở cùng scale. Denominator là estimated standard deviation của predictor thứ  $j$ , nên standardized predictor có standard deviation

scale equivariant

bằng one. Nhờ vậy final fit không phụ thuộc unit đo predictor. Figure 6.4 dùng standardized ridge coefficient estimate.

### Why Does Ridge Regression Improve Over Least Squares?

Advantage của ridge regression so với least squares bắt nguồn từ bias-variance trade-off. Khi  $\lambda$  tăng, flexibility của ridge fit giảm, variance giảm nhưng bias tăng. Figure 6.5 minh họa điều này trên simulated dataset có  $p = 45$  predictors và  $n = 50$  observations.



**HÌNH 6.5.** Squared bias (đen), variance (xanh lá), và test mean squared error (tím) của ridge regression prediction trên simulated dataset, theo  $\lambda$  và  $\|\hat{\beta}_\lambda^R\|_2 / \|\hat{\beta}\|_2$ . Horizontal dashed line là minimum possible MSE. Purple cross chỉ ridge regression model có MSE nhỏ nhất.

Green curve ở left panel của Figure 6.5 là variance của ridge prediction theo  $\lambda$ . Tại least squares solution ( $\lambda = 0$ ), variance cao nhưng bias bằng zero. Khi  $\lambda$  tăng, shrinkage làm variance giảm mạnh đổi lại bias tăng nhẹ. Test MSE (màu tím) liên hệ chặt với variance cộng squared bias. Với  $\lambda$  tăng từ 0 đến khoảng 10, variance giảm nhanh trong khi bias chỉ tăng rất ít, nên MSE giảm đáng kể. Sau đó variance giảm chậm hơn còn coefficient bị shrink quá mạnh, làm bias tăng nhanh. Minimum MSE đạt gần  $\lambda = 30$ . Đáng chú ý, do variance cao, MSE của least squares fit gần cao bằng MSE của null model; ở intermediate  $\lambda$ , MSE thấp hơn nhiều.

Right panel của Figure 6.5 vẽ cùng các curve theo ratio  $\ell_2$  norm của ridge coefficient so với least squares coefficient. Khi đi từ trái sang phải, fit flexible hơn, bias giảm và variance tăng.

Nói chung, khi relationship giữa response và predictors gần linear, least squares estimate có bias thấp nhưng có thể variance cao: một thay đổi nhỏ trong training data có thể làm coefficient estimate thay đổi lớn. Khi  $p$  gần bằng  $n$ , như Figure 6.5, least squares estimate cực kỳ variable; nếu  $p > n$ , least squares thậm chí không có unique solution. Ridge regression vẫn có thể hoạt động tốt bằng cách đổi một chút tăng bias lấy giảm mạnh variance. Vì thế ridge regression hữu ích nhất khi least squares estimate có variance cao.

Ridge regression cũng có computational advantage lớn so với best subset selection, vốn phải search qua  $2^p$  model. Ngay cả  $p$  vừa phải, search đó có thể infeasible. Với một fixed  $\lambda$ , ridge regression chỉ fit một model và có thể fit rất nhanh. Thực tế, có thể tính solution của (6.5) đồng thời cho toàn bộ  $\lambda$  với lượng computation gần như một least squares fit.



### 6.2.2 The Lasso

Ridge regression có một disadvantage rõ ràng. Khác best subset, forward stepwise và backward stepwise selection, ridge regression luôn giữ toàn bộ  $p$  predictors trong final model. Penalty  $\lambda \sum_j \beta_j^2$  shrink tất cả coefficient về zero nhưng không đặt coefficient nào chính xác bằng zero (trừ khi  $\lambda = \infty$ ). Điều này có thể không ảnh hưởng prediction accuracy nhưng gây khó cho model interpretation khi  $p$  lớn. Ví dụ trên **Credit**, các variable quan trọng nhất có vẻ là **income**, **limit**, **rating** và **student**; ta có thể muốn model chỉ gồm những predictor này, nhưng ridge regression vẫn luôn tạo model gồm cả mười predictors.

Lasso là alternative tương đối mới cho ridge regression và khắc phục drawback đó. Lasso coefficient  $\hat{\beta}_\lambda^L$  minimize

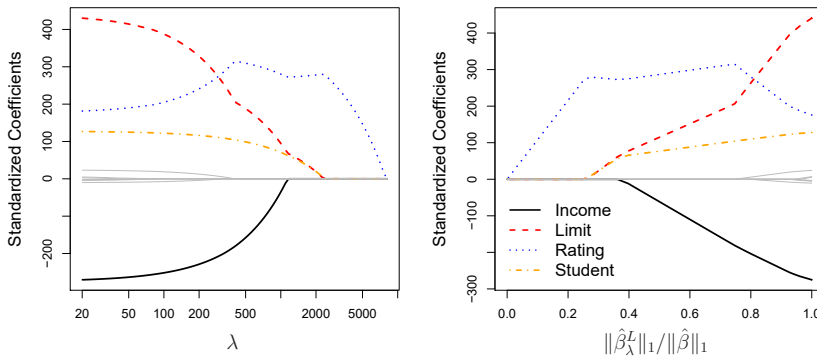
$$\sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| = \text{RSS} + \lambda \sum_{j=1}^p |\beta_j|. \quad (6.7)$$

So với (6.5), chỉ có  $\beta_j^2$  trong ridge penalty được thay bằng  $|\beta_j|$ . Theo terminology thống kê, lasso dùng  $\ell_1$  penalty thay vì  $\ell_2$  penalty;  $\ell_1$  norm là  $\|\beta\|_1 = \sum_j |\beta_j|$ .

lasso

Giống ridge regression, lasso shrink coefficient estimate về zero. Nhưng  $\ell_1$  penalty có thể ép một số coefficient estimate bằng đúng zero khi  $\lambda$  đủ lớn. Vì vậy, giống best subset selection, lasso thực hiện variable selection. Model do lasso tạo ra thường dễ interpret hơn ridge regression; ta nói lasso tạo *sparse model*, tức model chỉ gồm một subset variable. Chọn  $\lambda$  tốt vẫn critical và sẽ được giải quyết ở Section 6.2.3 bằng cross-validation.

sparse



**HÌNH 6.6.** Standardized lasso coefficient trên dataset **Credit**, theo  $\lambda$  và  $\|\hat{\beta}_\lambda^L\|_1 / \|\hat{\beta}\|_1$ .

Figure 6.6 cho coefficient path khi áp dụng lasso lên **Credit**. Khi  $\lambda = 0$ , lasso là least squares; khi  $\lambda$  đủ lớn, lasso là null model với mọi coefficient bằng zero. Ở giữa hai extreme này, lasso và ridge khác nhau đáng kể. Đi từ trái sang phải trong right panel, ban đầu lasso chỉ giữ predictor **rating**; sau đó **student** và **limit** gần như cùng lúc vào model, tiếp theo là **income**; cuối cùng các variable còn lại mới vào. Vì thế tùy  $\lambda$ , lasso có thể tạo model với bất kỳ số variable nào. Ridge regression thì luôn gồm tất cả variable, chỉ magnitude thay đổi theo  $\lambda$ .

*Another Formulation for Ridge Regression and the Lasso*

Có thể chứng minh lasso và ridge coefficient estimate lần lượt solve

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p |\beta_j| \leq s, \quad (6.8)$$

và

$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p \beta_j^2 \leq s. \quad (6.9)$$

Nói cách khác, với mỗi  $\lambda$  tồn tại một  $s$  sao cho (6.7) và (6.8) cho cùng lasso estimate; tương tự (6.5) và (6.9) cho cùng ridge estimate.

Khi  $p = 2$ , (6.8) nói rằng lasso estimate là point có RSS nhỏ nhất trong diamond  $|\beta_1| + |\beta_2| \leq s$ . Ridge estimate là point có RSS nhỏ nhất trong circle  $\beta_1^2 + \beta_2^2 \leq s$ . Có thể hiểu  $s$  như một budget: với lasso,  $\sum_j |\beta_j|$  không được vượt quá  $s$ ; với ridge,  $\sum_j \beta_j^2$  không được vượt quá  $s$ . Nếu  $s$  rất lớn và least squares solution nằm trong feasible region, constraint không có effect và solution chính là least squares. Nếu  $s$  nhỏ, coefficient bắt buộc phải nhỏ.

Các formulation (6.8) và (6.9) cho thấy connection gần giữa lasso, ridge regression và best subset selection. Xét problem

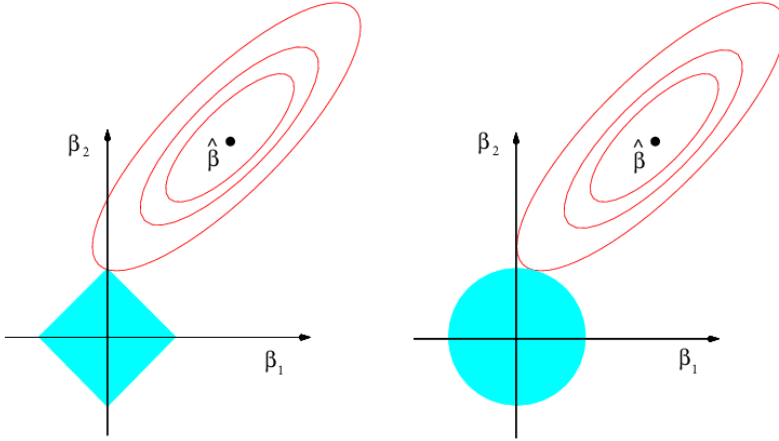
$$\underset{\beta}{\text{minimize}} \left\{ \sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p I(\beta_j \neq 0) \leq s. \quad (6.10)$$

Ở đây  $I(\beta_j \neq 0)$  là indicator variable, bằng 1 nếu  $\beta_j \neq 0$  và bằng 0 otherwise. (6.10) tìm coefficient estimate có RSS nhỏ nhất với không quá  $s$  coefficient nonzero, nên equivalent với best subset selection. Tuy nhiên khi  $p$  lớn, giải (6.10) computationally infeasible vì phải xét toàn bộ  $\binom{p}{s}$  model chứa  $s$  predictors. Có thể xem ridge và lasso là computationally feasible alternative, thay constraint khó trong (6.10) bằng constraint dễ solve hơn. Lasso gần best subset selection hơn vì với  $s$  đủ nhỏ nó thực hiện feature selection, còn ridge thì không.

*The Variable Selection Property of the Lasso*

Tại sao lasso, khác ridge regression, có thể tạo coefficient estimate chính xác bằng zero? Figure 6.7 giúp giải thích. Least squares solution được ký hiệu  $\hat{\beta}$ ; blue diamond và circle là lasso và ridge constraint region của (6.8) và (6.9). Nếu  $s$  đủ lớn, các region chứa  $\hat{\beta}$  và cả lasso lẫn ridge cho least squares estimate. Nhưng trong Figure 6.7, least squares estimate nằm ngoài cả diamond lẫn circle.

Mỗi ellipse centered tại  $\hat{\beta}$  là một contour: mọi point trên cùng ellipse có cùng RSS. Ellipse càng mở rộng khỏi least squares estimate thì RSS càng tăng. Theo (6.8) và (6.9), lasso hoặc ridge estimate là point đầu tiên nơi một ellipse chạm constraint region. Ridge có circular constraint không có sharp corner, nên intersection thường không nằm trên axis và coefficient thường đều nonzero. Lasso constraint lại có corner trên các axis, vì thế ellipse thường chạm region tại một axis; khi đó một coefficient bằng zero.



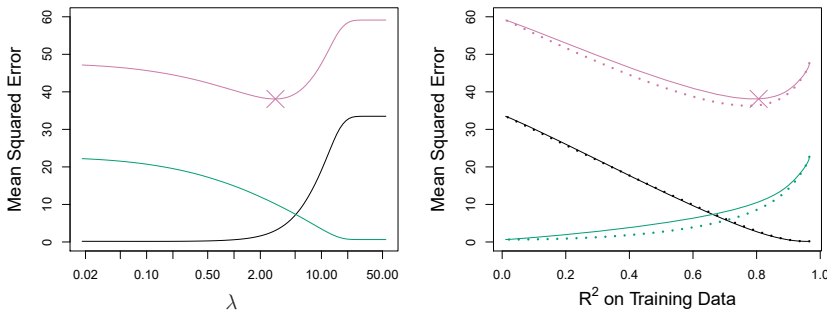
**HÌNH 6.7.** Contour của error và constraint function cho lasso (left) và ridge regression (right). Solid blue region là constraint  $|\beta_1| + |\beta_2| \leq s$  và  $\beta_1^2 + \beta_2^2 \leq s$ ; red ellipse là contour của RSS.

Trong higher dimension, nhiều coefficient có thể đồng thời bằng zero. Trong Figure 6.7, intersection xảy ra tại  $\beta_1 = 0$ , nên resulting model chỉ gồm  $\beta_2$ .

Với  $p = 3$ , ridge constraint region là sphere còn lasso là polyhedron. Với  $p > 3$ , ridge thành hypersphere còn lasso thành polytope. Key idea vẫn giữ nguyên: sharp corner của lasso region dẫn đến feature selection.

#### Comparing the Lasso and Ridge Regression

Lasso có advantage lớn về interpretability vì tạo model đơn giản hơn, chỉ gồm subset predictors. Nhưng method nào prediction accuracy tốt hơn? Figure 6.8 hiển thị variance, squared bias và test MSE của lasso trên cùng simulated data như Figure 6.5. Khi  $\lambda$  tăng, lasso cũng cho variance giảm và bias tăng, giống ridge về qualitative behavior.

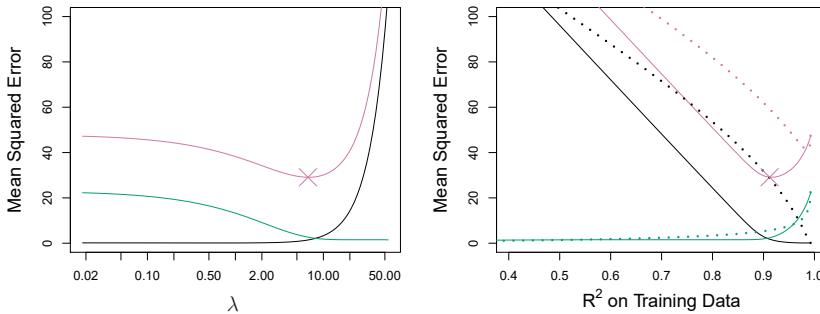


**HÌNH 6.8.** Left: Squared bias (đen), variance (xanh lá), và test MSE (tím) của lasso trên simulated dataset. Right: So sánh squared bias, variance và test MSE giữa lasso (solid) và ridge (dotted). Cả hai được plot theo training-data  $R^2$  như một common indexing. Cross ở hai plot chỉ lasso model có MSE nhỏ nhất.

Trong right panel, dotted line là ridge fit; cả hai method được plot theo training-data  $R^2$ , một cách hữu ích để compare model có regularization khác nhau. Trong ví dụ này, lasso và ridge có bias gần như identical, nhưng ridge có variance hơi thấp hơn nên minimum MSE cũng hơi nhỏ hơn.

Tuy nhiên, data của Figure 6.8 được generate sao cho cả 45 predictors đều liên quan đến response, tức không true coefficient nào bằng zero. Lasso implicitly assume một số coefficient thật sự bằng zero, nên không ngạc nhiên khi ridge có prediction error tốt hơn trong setting này. Figure 6.9 dùng setup tương tự nhưng response chỉ là function của 2 trong 45 predictors; khi đó lasso có xu hướng outperform ridge về bias, variance và MSE.

Hai ví dụ cho thấy không method nào universally dominate. Lasso có thể tốt hơn khi chỉ một số ít predictors có substantial coefficient, còn coefficient của predictor khác rất nhỏ hoặc zero. Ridge thường tốt hơn khi response phụ thuộc vào nhiều predictors với coefficient có magnitude tương đối giống nhau. Trong real data, ta không biết trước bao nhiêu predictor liên quan đến response; cross-validation có thể dùng để quyết định approach nào tốt hơn trên một dataset cụ thể. Cũng như ridge, khi least squares estimate có variance quá cao, lasso có thể giảm variance đổi lấy một increase nhỏ về bias và nhờ đó prediction chính xác hơn. Khác ridge, lasso thực hiện variable selection nên model dễ interpret hơn.



**HÌNH 6.9.** Left: Squared bias, variance và test MSE của lasso trên simulated data tương tự Figure 6.8 nhưng chỉ hai predictors liên quan đến response. Right: So sánh lasso (solid) và ridge (dotted) theo training-data  $R^2$ . Cross chỉ lasso model có MSE nhỏ nhất.

Có các algorithm rất efficient để fit cả ridge lẫn lasso; trong cả hai trường hợp, toàn bộ coefficient path có thể được compute với lượng work xấp xỉ một single least squares fit. Lab cuối chapter sẽ khảo sát điều này kỹ hơn.

### A Simple Special Case for Ridge Regression and the Lasso

Để có intuition rõ hơn về behavior của ridge và lasso, xét special case  $n = p$ , và  $X$  là diagonal matrix với 1 trên diagonal, 0 ở mọi off-diagonal entry. Để đơn giản thêm, giả sử regression không có intercept. Least squares problem trở thành tìm  $\beta_1, \dots, \beta_p$  minimize

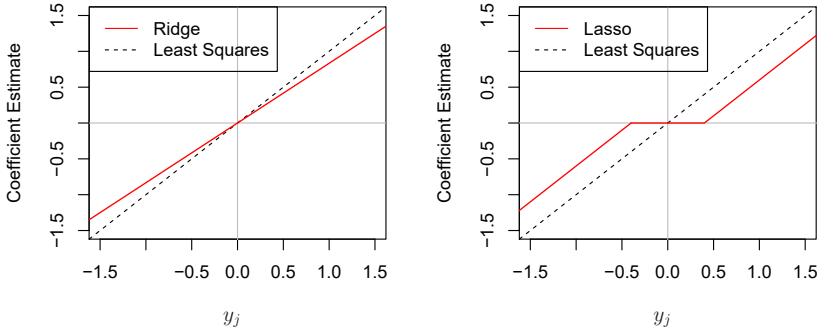
$$\sum_{j=1}^p (y_j - \beta_j)^2. \quad (6.11)$$

Solution là  $\hat{\beta}_j = y_j$ . Trong setting này, ridge regression minimize

$$\sum_{j=1}^p (y_j - \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2, \quad (6.12)$$

còn lasso minimize

$$\sum_{j=1}^p (y_j - \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j|. \quad (6.13)$$



**HÌNH 6.10.** Ridge regression và lasso coefficient estimate trong simple setting  $n = p$  với  $X$  là diagonal matrix có 1 trên diagonal. Left: Ridge estimate shrink proportionally về zero so với least squares estimate. Right: Lasso estimate được soft-threshold về zero.

Có thể chứng minh ridge estimate trong setting này là

$$\hat{\beta}_j^R = \frac{y_j}{1 + \lambda}, \quad (6.14)$$

còn lasso estimate là

$$\hat{\beta}_j^L = \begin{cases} y_j - \lambda/2, & y_j > \lambda/2, \\ y_j + \lambda/2, & y_j < -\lambda/2, \\ 0, & |y_j| \leq \lambda/2. \end{cases} \quad (6.15)$$

Figure 6.10 cho thấy hai loại shrinkage rất khác. Ridge shrink mọi least squares coefficient cùng một proportion. Lasso shrink mỗi coefficient về zero một constant amount  $\lambda/2$ ; coefficient có absolute value nhỏ hơn  $\lambda/2$  bị shrink hoàn toàn về zero. Kiểu shrinkage trong (6.15) gọi là *soft-thresholding*; đây là lý do lasso thực hiện feature selection.

*soft-thresholding*

Với data matrix  $X$  tổng quát hơn, picture phức tạp hơn Figure 6.10 nhưng main idea vẫn gần đúng: ridge shrink mọi dimension với proportion tương tự, còn lasso shrink mọi coefficient về zero với amount tương tự và các coefficient đủ nhỏ bị shrink hẳn về zero.

### Bayesian Interpretation of Ridge Regression and the Lasso

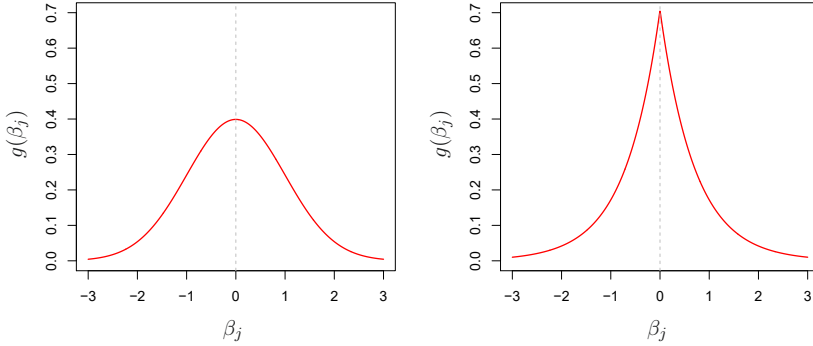
Ta có thể nhìn ridge regression và lasso qua Bayesian lens. Bayesian viewpoint cho regression giả sử coefficient vector  $\beta = (\beta_0, \beta_1, \dots, \beta_p)^T$  có

prior distribution  $p(\beta)$ . Likelihood của data viết là  $f(Y | X, \beta)$ , với  $X = (X_1, \dots, X_p)$ . Nhân prior với likelihood cho posterior distribution, bỏ constant tỷ lệ:

$$p(\beta | X, Y) \propto f(Y | X, \beta)p(\beta | X) = f(Y | X, \beta)p(\beta),$$

trong đó proportionality theo Bayes' theorem và equality cuối dựa trên assumption rằng  $X$  fixed.

posterior  
distribution



**HÌNH 6.11.** Left: Ridge regression là posterior mode của  $\beta$  dưới Gaussian prior. Right: Lasso là posterior mode của  $\beta$  dưới double-exponential prior.

Giả sử usual linear model  $Y = \beta_0 + X_1\beta_1 + \dots + X_p\beta_p + \varepsilon$ , với independent normally distributed error, và  $p(\beta) = \prod_{j=1}^p g(\beta_j)$  cho density  $g$ . Ridge và lasso xuất hiện tự nhiên từ hai special case:

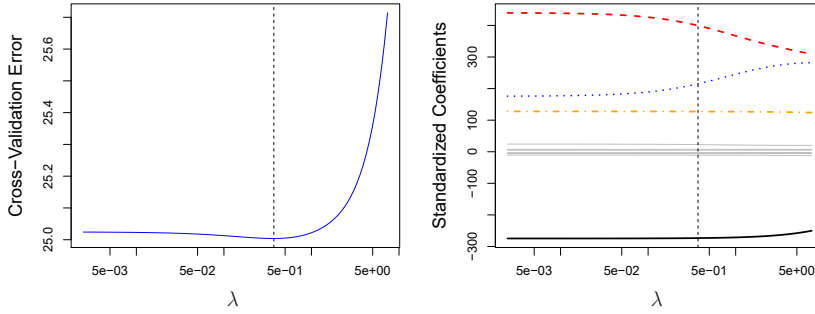
- Nếu  $g$  là Gaussian distribution mean zero với standard deviation là function của  $\lambda$ , posterior mode của  $\beta$  — most likely value của  $\beta$  given data — chính là ridge regression solution. Thực tế ridge solution cũng là posterior mean.
- Nếu  $g$  là double-exponential (Laplace) distribution mean zero với scale parameter là function của  $\lambda$ , posterior mode của  $\beta$  là lasso solution. Tuy nhiên lasso solution không phải posterior mean; posterior mean cũng không tạo sparse coefficient vector.

posterior mode

Gaussian và double-exponential prior được minh họa ở Figure 6.11. Từ Bayesian viewpoint, ridge và lasso theo trực tiếp từ usual linear model với normal error cộng một simple prior cho  $\beta$ . Lasso prior nhọn mạnh tại zero, còn Gaussian flatter và fatter ở zero. Vì vậy lasso kỳ vọng a priori nhiều coefficient bằng đúng zero, trong khi ridge giả sử coefficient phân bố ngẫu nhiên quanh zero.

### 6.2.3 Selecting the Tuning Parameter

Giống subset selection ở Section 6.1 cần một method xác định model nào tốt nhất, ridge và lasso cần method chọn tuning parameter  $\lambda$  trong (6.5), (6.7), hay tương đương constraint  $s$  trong (6.9), (6.8). Cross-validation cung cấp cách đơn giản: chọn một grid  $\lambda$ , compute cross-validation error



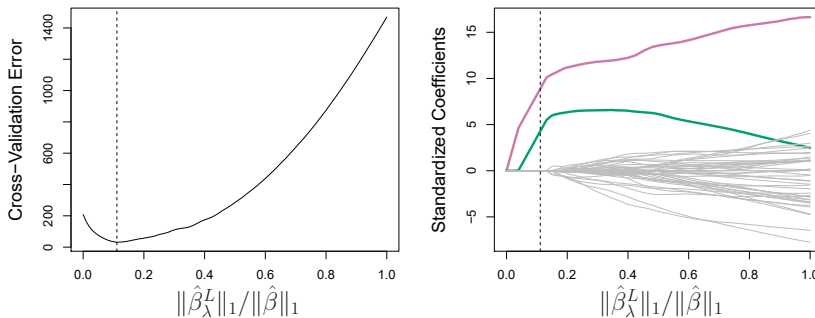
**HÌNH 6.12.** Left: Cross-validation error khi áp dụng ridge regression lên dataset *Credit* với nhiều  $\lambda$ . Right: Coefficient estimate theo  $\lambda$ . Vertical dashed line chỉ  $\lambda$  được cross-validation chọn.

cho từng value, chọn value có error nhỏ nhất, rồi refit model trên toàn bộ observation bằng  $\lambda$  đã chọn.

Figure 6.12 cho  $\lambda$  được chọn khi dùng leave-one-out cross-validation trên ridge fit của *Credit*. Dashed vertical line là selected  $\lambda$ . Value khá nhỏ, cho thấy optimal fit chỉ cần shrinkage nhẹ so với least squares. Dip của error curve cũng không rõ, nên một wide range  $\lambda$  cho error gần như nhau; trong case như vậy ta thậm chí có thể dùng least squares solution.

Figure 6.13 minh họa ten-fold cross-validation cho lasso trên sparse simulated data của Figure 6.9. Left panel là cross-validation error, right panel là coefficient estimate. Dashed vertical line là point error nhỏ nhất. Hai colored line ở right panel là hai predictors thật sự liên quan đến response; grey line là unrelated predictors, thường gọi lần lượt là *signal* và *noise* variable. Lasso không chỉ cho much larger coefficient cho hai signal predictors mà minimum cross-validation error còn ứng với coefficient set trong đó chỉ signal variable nonzero. Do đó cross-validation kết hợp lasso đã nhận diện đúng hai signal variable trong setting khó với  $p = 45$  và chỉ  $n = 50$  observations.

signal



**HÌNH 6.13.** Left: Ten-fold cross-validation MSE của lasso trên sparse simulated dataset trong Figure 6.9. Right: Lasso coefficient estimate tương ứng. Hai signal variable được vẽ màu, noise variable màu xám. Vertical dashed line chỉ lasso fit có cross-validation error nhỏ nhất.

Trong khi đó least squares solution ở cực phải của right panel Figure 6.13 lại assign coefficient estimate lớn cho chỉ một trong hai signal variable.

### 6.3 Dimension Reduction Methods

Các method đã bàn trong chapter này kiểm soát variance theo hai cách: dùng một subset của original variables, hoặc shrink coefficient của chúng về zero. Tất cả đều được định nghĩa trên original predictors  $X_1, X_2, \dots, X_p$ . Bây giờ ta xét một class approach transform predictors trước, rồi fit least squares model bằng transformed variables. Ta gọi các technique này là *dimension reduction methods*.

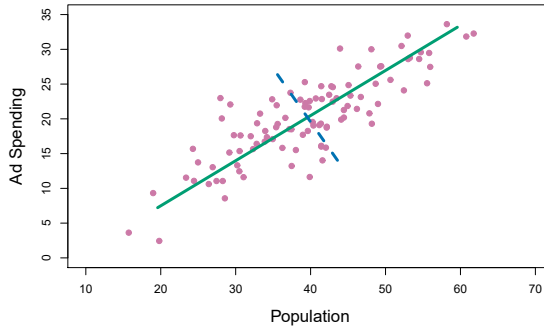
Gọi  $Z_1, Z_2, \dots, Z_M$  là  $M < p$  linear combination của  $p$  original predictors:

$$Z_m = \sum_{j=1}^p \phi_{jm} X_j, \quad (6.16)$$

cho một số constant  $\phi_{1m}, \phi_{2m}, \dots, \phi_{pm}$ ,  $m = 1, \dots, M$ . Sau đó ta fit linear regression model

$$y_i = \theta_0 + \sum_{m=1}^M \theta_m z_{im} + \varepsilon_i, \quad i = 1, \dots, n, \quad (6.17)$$

bằng least squares. Regression coefficient trong (6.17) là  $\theta_0, \theta_1, \dots, \theta_M$ . Nếu các  $\phi_{jm}$  được chọn khéo, dimension reduction approach thường có thể out-perform least squares regression trên original predictors.



**HÌNH 6.14.** Population size (*pop*) và ad spending (*ad*) của 100 city được hiển thị bằng purple circle. Green solid line là first principal component, blue dashed line là second principal component.

Tên gọi dimension reduction xuất phát từ việc problem estimate  $p + 1$  coefficient  $\beta_0, \beta_1, \dots, \beta_p$  được rút xuống còn  $M + 1$  coefficient  $\theta_0, \theta_1, \dots, \theta_M$ , với  $M < p$ . Tức dimension của problem giảm từ  $p + 1$  xuống  $M + 1$ .

Từ (6.16),

$$\sum_{m=1}^M \theta_m z_{im} = \sum_{m=1}^M \theta_m \sum_{j=1}^p \phi_{jm} x_{ij} = \sum_{j=1}^p \sum_{m=1}^M \theta_m \phi_{jm} x_{ij} = \sum_{j=1}^p \beta_j x_{ij},$$



trong đó

$$\beta_j = \sum_{m=1}^M \theta_m \phi_{jm}. \quad (6.18)$$

Vì vậy (6.17) có thể xem là special case của original linear regression model (6.1). Dimension reduction constrain estimated  $\beta_j$  vì giờ chúng phải có form (6.18). Constraint này có thể tạo bias, nhưng khi  $p$  lớn relative to  $n$ , chọn  $M \ll p$  có thể giảm variance của fitted coefficient đáng kể. Nếu  $M = p$  và mọi  $Z_m$  linearly independent, (6.18) không impose constraint nào; lúc đó không có dimension reduction và fit (6.17) tương đương least squares trên  $p$  original predictors.

Mọi dimension reduction method đều có hai step. Thứ nhất, tạo transformed predictors  $Z_1, \dots, Z_M$ . Thứ hai, fit model dùng  $M$  predictors này. Nhưng cách chọn  $Z_m$ , hay tương đương các  $\phi_{jm}$ , có thể khác nhau. Chapter này xét hai approach: principal components và partial least squares.

### 6.3.1 Principal Components Regression

*Principal components analysis* (PCA) là approach phổ biến để derive một low-dimensional feature set từ một tập variable lớn. PCA được thảo luận chi tiết hơn như một tool cho unsupervised learning ở Chapter 12. Ở đây ta mô tả cách dùng PCA như dimension reduction technique cho regression.

*principal  
components  
analysis*

#### *An Overview of Principal Components Analysis*

PCA là technique giảm dimension của data matrix  $X$  kích thước  $n \times p$ . *First principal component direction* là direction mà observations vary nhiều nhất. Chẳng hạn Figure 6.14 hiển thị population size (**pop**, đơn vị chục nghìn người) và ad spending (**ad**, nghìn dollars) của 100 city.<sup>6</sup> Green solid line là first principal component direction. Nhìn bằng mắt, đây là direction có greatest variability. Nếu project 100 observations lên line này (như left panel Figure 6.15), projected observations có variance lớn nhất có thể; project lên bất kỳ line nào khác sẽ cho variance nhỏ hơn. Project một point lên line đơn giản là tìm point trên line gần nó nhất.

First principal component trong Figure 6.14 có thể viết toán học là

$$Z_1 = 0.839(\text{pop} - \overline{\text{pop}}) + 0.544(\text{ad} - \overline{\text{ad}}). \quad (6.19)$$

Ở đây  $\phi_{11} = 0.839$  và  $\phi_{21} = 0.544$  là *principal component loadings*, định nghĩa direction nói trên. Trong (6.19),  $\overline{\text{pop}}$  và  $\overline{\text{ad}}$  là mean của các value tương ứng. Trong mọi linear combination của **pop** và **ad** thỏa  $\phi_{11}^2 + \phi_{21}^2 = 1$ , combination này có variance lớn nhất. Constraint norm bằng one là cần thiết; nếu không ta có thể tùy ý tăng  $\phi_{11}, \phi_{21}$  để làm variance phình ra. Hai loading trong (6.19) đều positive và có magnitude tương tự, nên  $Z_1$  gần như average của hai variables.

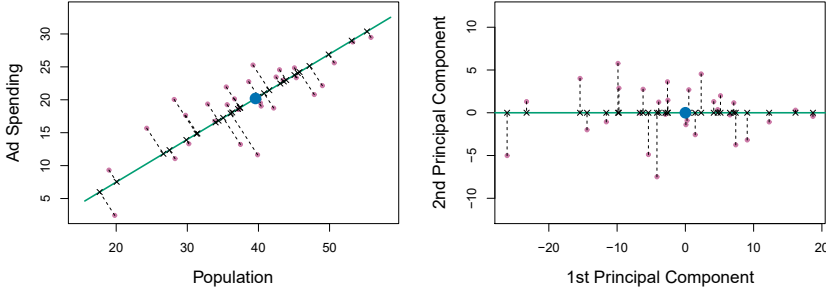
Vì  $n = 100$ , **pop**, **ad** và  $Z_1$  đều là vector length 100. Với observation thứ  $i$ ,

$$z_{i1} = 0.839(\text{pop}_i - \overline{\text{pop}}) + 0.544(\text{ad}_i - \overline{\text{ad}}). \quad (6.20)$$

Các value  $z_{11}, \dots, z_{n1}$  được gọi là *principal component scores* và xuất hiện ở right panel Figure 6.15.

<sup>6</sup>Dataset này khác dataset **Advertising** được thảo luận ở Chapter 3.

PCA còn có interpretation khác: first principal component vector định nghĩa line gần data nhất có thể. Trong Figure 6.14, first principal component line minimize tổng squared perpendicular distance từ mỗi point tới line. Các distance này được vẽ bằng dashed line segment ở left panel Figure 6.15; các cross là projection của point lên first principal component line. First principal component được chọn để projected observations gần original observations nhất.



**HÌNH 6.15.** Một subset của advertising data. Mean  $\mathbf{pop}$  và  $\mathbf{ad}$  được đánh dấu bằng blue circle. Left: First principal component direction màu xanh lá; đây vừa là dimension data vary nhiều nhất, vừa là line gần toàn bộ  $n$  observations nhất. Distance từ mỗi observation tới principal component được vẽ bằng black dashed segment. Blue dot là  $(\overline{\mathbf{pop}}, \overline{\mathbf{ad}})$ . Right: Left panel được rotate để first principal component direction trùng với trục  $x$ .

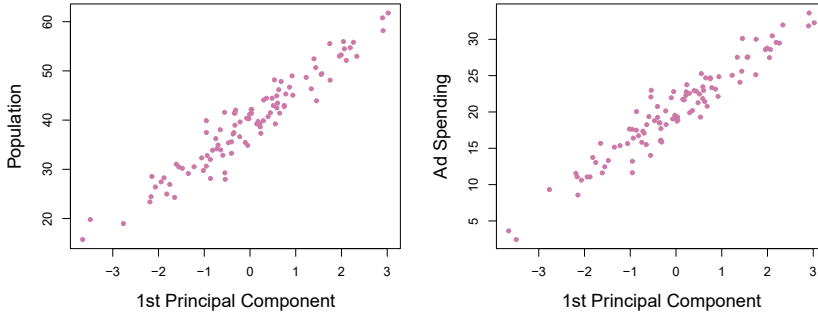
Ở right panel của Figure 6.15, left panel đã được rotate sao cho first principal component direction trùng trục  $x$ . Có thể chứng minh first principal component score của observation thứ  $i$ , (6.20), chính là distance theo direction  $x$  từ cross thứ  $i$  tới zero. Ví dụ, point ở bottom-left của left panel có score lớn âm  $z_{i1} = -26.1$ , còn point top-right có score lớn dương  $z_{i1} = 18.7$ . Các score này có thể tính trực tiếp bằng (6.20).

Ta có thể xem value của principal component  $Z_1$  như single-number summary của joint  $\mathbf{pop}$  và  $\mathbf{ad}$  budget cho mỗi location. Nếu  $z_{i1} < 0$ , city đó có xu hướng population size và ad spending dưới average; score dương gợi ý điều ngược lại. Một number duy nhất đại diện hai variable tốt đến đâu? Figure 6.14 cho thấy  $\mathbf{pop}$  và  $\mathbf{ad}$  gần linear, nên single-number summary có thể hoạt động tốt. Figure 6.16 plot  $z_{i1}$  với cả  $\mathbf{pop}$  và  $\mathbf{ad}$ ; relationship mạnh cho thấy first principal component giữ phần lớn information của hai predictors.<sup>7</sup>

Cho đến đây ta tập trung vào first principal component. Tổng quát có thể xây dựng tối đa  $p$  distinct principal components. Second principal component  $Z_2$  là linear combination của variables không correlated với  $Z_1$  và có variance lớn nhất dưới constraint đó. Second principal component direction là dashed blue line trong Figure 6.14. Condition zero correlation giữa  $Z_1$  và  $Z_2$  tương đương direction thứ hai phải perpendicular, hay *orthogonal*, với direction thứ nhất. Ở example này,

$$Z_2 = 0.544(\mathbf{pop} - \overline{\mathbf{pop}}) - 0.839(\mathbf{ad} - \overline{\mathbf{ad}}).$$

<sup>7</sup>Các principal component được tính sau khi standardize cả  $\mathbf{pop}$  và  $\mathbf{ad}$ , một common approach. Vì vậy x-axis của Figures 6.15 và 6.16 không cùng scale.



**HÌNH 6.16.** Plot first principal component score  $z_{i1}$  theo *pop* và *ad*. Các relationship đều mạnh.

Advertising data chỉ có hai predictors nên first two principal components chứa toàn bộ information của *pop* và *ad*; tuy nhiên theo construction, first component giữ nhiều information nhất. Right panel Figure 6.15 cho thấy variability của  $z_{i1}$  lớn hơn nhiều  $z_{i2}$ . Figure 6.17 cũng cho thấy relationship giữa second principal component với *pop*, *ad* rất yếu, gợi ý chỉ first component đã đủ để đại diện khá chính xác hai budget này.

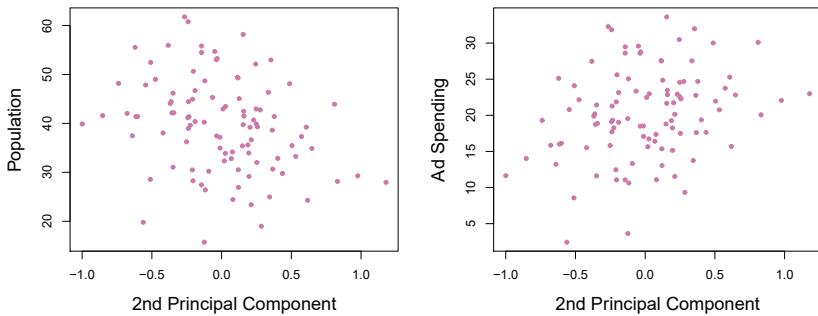
perpendicular  
orthogonal

Với two-dimensional data ta chỉ xây được tối đa hai principal component. Nếu có thêm predictors như population age, income level hay education, ta có thể xây thêm component; từng component kế tiếp maximize variance với constraint không correlated với các component trước.

### The Principal Components Regression Approach

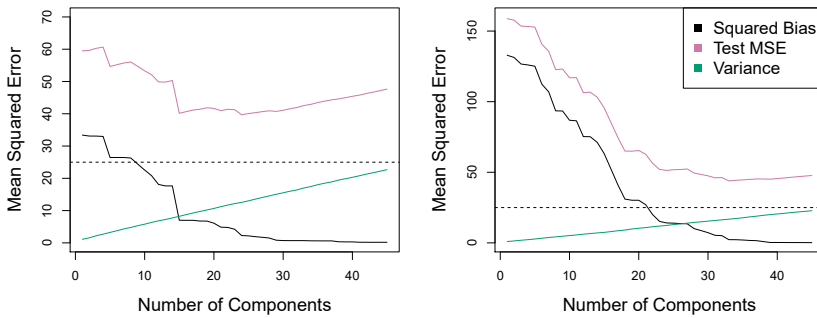
Principal components regression (PCR) xây first  $M$  principal components  $Z_1, \dots, Z_M$ , rồi dùng chúng làm predictors trong linear regression model fit bằng least squares. Key idea là thường chỉ một số ít principal component đã đủ explain phần lớn variability trong data và relationship với response. Nói cách khác, ta assume những direction mà  $X_1, \dots, X_p$  vary nhiều nhất cũng là những direction associated với  $Y$ . Assumption này không được guarantee, nhưng trong nhiều case là approximation đủ tốt.

principal  
components  
regression



**HÌNH 6.17.** Plot second principal component score  $z_{i2}$  theo *pop* và *ad*. Các relationship đều yếu.

and the relationships are shown



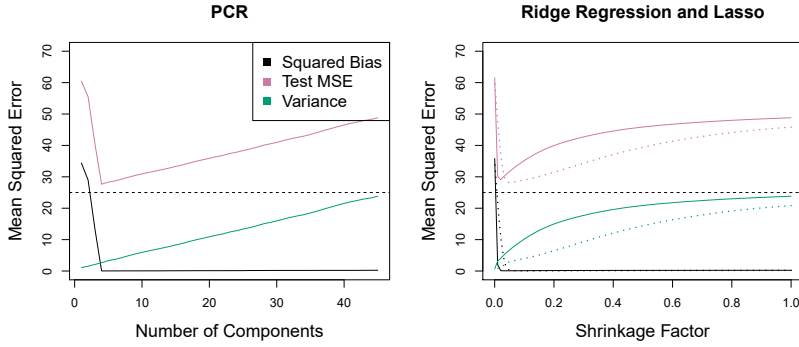
**HÌNH 6.18.** PCR được áp dụng lên hai simulated dataset. Horizontal dashed line ở mỗi panel là irreducible error. Left: Simulated data từ Figure 6.8. Right: Simulated data từ Figure 6.9.

Nếu assumption nền tảng của PCR đúng, fit least squares model trên  $Z_1, \dots, Z_M$  có thể tốt hơn fit trên  $X_1, \dots, X_p$ , vì phần lớn information liên quan response nằm trong  $M$  components và ta chỉ estimate  $M \ll p$  coefficient nên giảm overfitting. Trong advertising data, first principal component explain phần lớn variance của cả `pop` và `ad`; vì vậy PCR chỉ dùng component này để predict một response như `sales` có thể hoạt động khá tốt.

Figure 6.18 hiển thị PCR fit trên simulated dataset từ Figures 6.8 và 6.9. Cả hai có  $n = 50$  observations,  $p = 45$  predictors; ở dataset đầu response là function của tất cả predictors, còn dataset sau chỉ phụ thuộc hai predictors. Curve được plot theo  $M$ , số principal components dùng trong regression. Khi tăng  $M$ , bias giảm nhưng variance tăng, tạo U-shape quen thuộc cho MSE. Khi  $M = p = 45$ , PCR chính là least squares fit dùng toàn bộ original predictors. Figure cho thấy chọn  $M$  hợp lý có thể cải thiện đáng kể so với least squares, đặc biệt ở left panel. Tuy vậy, so với ridge và lasso trong Figures 6.5, 6.8 và 6.9, PCR không tốt bằng hai shrinkage method trong example này.

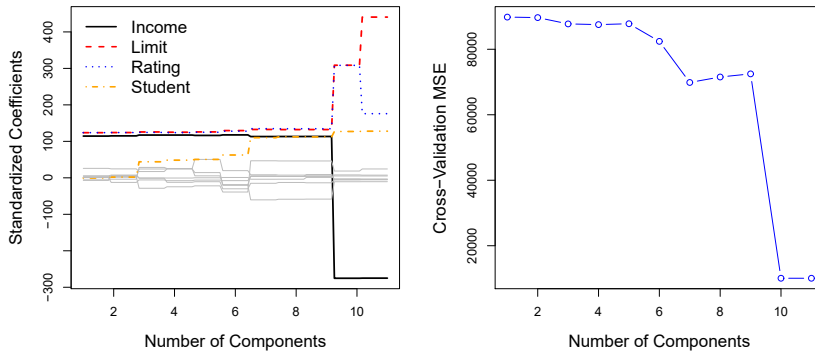
PCR tương đối kém trong Figure 6.18 vì data được generate sao cho cần nhiều principal component để model response đầy đủ. PCR sẽ làm tốt hơn khi chỉ first few principal components đã capture phần lớn variation của predictors và relationship với response. Left panel Figure 6.19 là simulated dataset thuận lợi hơn: response phụ thuộc exclusively vào first five principal components. Bias giảm nhanh về zero khi  $M$  tăng, MSE có minimum rõ tại  $M = 5$ . Right panel cho ridge và lasso trên cùng data; cả ba method đều cải thiện đáng kể so với least squares, nhưng PCR và ridge hơi outperform lasso.

Dù PCR cho cách đơn giản để regression với  $M < p$  predictors, nó không phải feature selection method, vì mỗi principal component là linear combination của toàn bộ  $p$  original features. Ví dụ  $Z_1$  trong (6.19) chứa cả `pop` và `ad`. Vì vậy PCR thường hoạt động tốt nhưng không tạo model chỉ dựa vào một small set original features. Theo nghĩa này, PCR gần ridge regression hơn lasso. Thực tế PCR và ridge regression rất closely related;



**HÌNH 6.19.** PCR, ridge regression và lasso được áp dụng lên simulated dataset nơi first five principal components của  $X$  chứa toàn bộ information về response  $Y$ . Horizontal dashed line là irreducible error  $\text{Var}(\varepsilon)$ . Left: PCR. Right: Lasso (solid) và ridge regression (dotted). Trục  $x$  là shrinkage factor, tức  $\ell_2$  norm của shrunken coefficient estimate chia cho  $\ell_2$  norm của least squares estimate.

thậm chí có thể xem ridge regression như continuous version của PCR.<sup>8</sup>



**HÌNH 6.20.** Left: PCR standardized coefficient estimate trên dataset *Credit* với các value  $M$  khác nhau. Right: Ten-fold cross-validation MSE của PCR theo  $M$ .

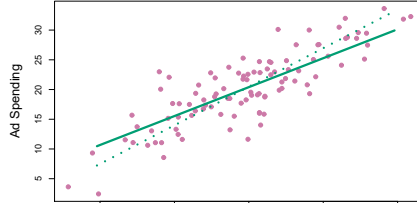
Trong PCR, số principal components  $M$  thường được chọn bằng cross-validation. Figure 6.20 hiển thị result trên dataset *Credit*; right panel là cross-validation error theo  $M$ . Lowest cross-validation error xảy ra tại  $M = 10$ , gần như không dimension reduction vì PCR với  $M = 11$  tương đương ordinary least squares.

Khi thực hiện PCR, thông thường nên standardize từng predictor bằng (6.6) trước khi tạo principal components, để mọi variable nằm trên cùng scale. Nếu không standardize, high-variance variable có xu hướng đóng vai trò lớn hơn trong principal component, và scale đo variable sẽ ảnh hưởng final PCR model. Tuy nhiên, nếu mọi variable đều đo cùng một unit, chẳng hạn kilograms hoặc inches, có thể lựa chọn không standardize.

<sup>8</sup>Xem thêm Section 3.5 của *The Elements of Statistical Learning* của Hastie, Tibshirani và Friedman.

### 6.3.2 Partial Least Squares

PCR approach vừa mô tả xác định các linear combination, hay direction, biểu diễn tốt nhất các predictor  $X_1, \dots, X_p$ . Các direction này được xác định theo cách *unsupervised*, vì response  $Y$  không được dùng để hỗ trợ xác định principal component directions. Nói cách khác, response không supervise việc nhận diện principal components. Vì vậy PCR có một nhược điểm: không có gì bảo đảm những direction giải thích predictors tốt nhất cũng là những direction tốt nhất để predict response.



**HÌNH 6.21.** Với *advertising data*, *first PLS direction* (solid line) và *first PCR direction* (dotted line) được hiển thị.

Các unsupervised method được bàn thêm trong Chapter 12. Bây giờ ta giới thiệu *partial least squares* (PLS), một supervised alternative cho PCR. Giống PCR, PLS là dimension reduction method: trước hết xác định một tập feature mới  $Z_1, \dots, Z_M$  là các linear combination của original features, rồi fit một linear model bằng least squares sử dụng  $M$  feature mới này. Nhưng khác PCR, PLS xác định các feature mới theo cách supervised—tức dùng response  $Y$  để tìm những feature vừa approximate original features tốt vừa liên quan tới response. Nói nôm na, PLS cố tìm các direction giúp explain cả response lẫn predictors.

Ta mô tả cách tính first PLS direction. Sau khi standardize  $p$  predictors, PLS tạo direction đầu tiên  $Z_1$  bằng cách đặt mỗi  $\phi_{j1}$  trong (6.16) bằng coefficient từ simple linear regression của  $Y$  theo  $X_j$ . Có thể chứng minh coefficient này tỷ lệ với correlation giữa  $Y$  và  $X_j$ . Vì vậy khi tính  $Z_1 = \sum_{j=1}^p \phi_{j1} X_j$ , PLS đặt weight lớn nhất lên các variable có relationship mạnh nhất với response.

Figure 6.21 minh họa PLS trên synthetic dataset, trong đó response là **Sales** ở 100 region và hai predictors là **Population Size** và **Advertising Spending**. Solid green line là first PLS direction, dotted line là first principal component direction. So với PCA, PLS chọn một direction có mức thay đổi theo dimension **ad** nhỏ hơn trên mỗi unit thay đổi theo dimension **pop**. Điều này gợi ý **pop** correlated với response mạnh hơn **ad**. PLS direction không fit predictors sát bằng PCA, nhưng explain response tốt hơn.

Để xác định second PLS direction, trước hết ta adjust từng variable theo  $Z_1$  bằng cách regress từng variable lên  $Z_1$  rồi lấy residual. Các residual này có thể được xem là phần information còn lại chưa được first PLS direction explain. Sau đó ta tính  $Z_2$  từ orthogonalized data này theo đúng cách đã dùng để tính  $Z_1$  từ original data.

Iterative approach này có thể lặp  $M$  lần để xác định nhiều PLS components  $Z_1, \dots, Z_M$ . Cuối cùng, ta dùng least squares để fit linear model predict  $Y$  bằng  $Z_1, \dots, Z_M$ , hoàn toàn tương tự PCR.

Giống PCR, số  $M$  partial least squares directions dùng trong PLS là một tuning parameter thường được chọn bằng cross-validation. Ta thường standardize cả predictors lẫn response trước khi thực hiện PLS.

PLS phổ biến trong chemometrics, nơi nhiều variable xuất phát từ digitized spectrometry signals. Trong thực tế nó thường không perform tốt hơn ridge regression hoặc PCR. Supervised dimension reduction của PLS có thể giảm bias, nhưng cũng có khả năng tăng variance, nên overall benefit của PLS so với PCR có thể gần như triệt tiêu.

## 6.4 Considerations in High Dimensions

### 6.4.1 High-Dimensional Data

Phần lớn statistical technique truyền thống cho regression và classification được thiết kế cho *low-dimensional setting*, trong đó  $n$ , số observations, lớn hơn nhiều so với  $p$ , số features. Một phần nguyên nhân là trong phần lớn lịch sử của statistics, đa số scientific problem cần statistics đều low-dimensional. Ví dụ, để xây model predict blood pressure của patient dựa trên age, sex và body mass index (BMI), ta có ba predictors, hay bốn nếu tính cả intercept, và có thể vài nghìn patient với đủ measurements. Khi đó  $n \gg p$ , nên problem là low-dimensional. (Ở đây *dimension* nói tới độ lớn của  $p$ .)

*low-dimensional*

Trong khoảng 20 năm qua, technology mới đã thay đổi cách data được collect trong finance, marketing, medicine và nhiều field khác. Hiện nay việc collect gần như unlimited number of feature measurements ( $p$  rất lớn) là bình thường. Trong khi  $p$  có thể cực lớn, số observations  $n$  thường bị giới hạn bởi cost, sample availability hoặc các consideration khác. Hai example:

1. Thay vì predict blood pressure chỉ từ age, sex và BMI, có thể thu thêm measurement cho nửa triệu single nucleotide polymorphisms (SNPs; các DNA mutation riêng lẻ tương đối phổ biến trong population) để đưa vào predictive model. Khi đó  $n \approx 200$  và  $p \approx 500,000$ .
2. Một marketing analyst muốn hiểu online shopping pattern có thể coi toàn bộ search term người dùng nhập vào search engine là features. Cách này đôi khi gọi là “bag-of-words” model. Researcher đó có thể chỉ có search history của vài trăm hoặc vài nghìn user đồng ý chia sẻ information. Với mỗi user, mỗi trong  $p$  search term được score present (0) hoặc absent (1), tạo nên một binary feature vector rất lớn. Khi đó  $n \approx 1,000$  còn  $p$  lớn hơn nhiều.

Dataset có nhiều features hơn observations thường được gọi là *high-dimensional*. Classical approach như least squares linear regression không phù hợp trong setting này. Nhiều issue của high-dimensional data đã xuất hiện trước đây trong sách vì chúng cũng có khi  $n > p$ , gồm bias-variance trade-off và danger of overfitting. Dù luôn quan trọng, các issue này đặc biệt nghiêm trọng khi số features rất lớn so với số observations.

*high-dimensional*

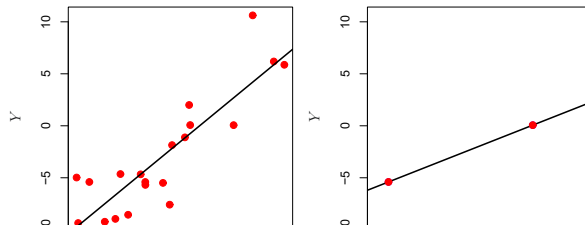
Ta định nghĩa high-dimensional setting là case số features  $p$  lớn hơn số observations  $n$ . Tuy nhiên, các consideration sắp bàn cũng áp dụng khi  $p$  chỉ hơi nhỏ hơn  $n$ , và tốt nhất luôn ghi nhớ chúng khi thực hiện supervised learning.

### 6.4.2 What Goes Wrong in High Dimensions?

Để minh họa vì sao cần extra care và specialized technique cho regression và classification khi  $p > n$ , ta xem điều gì xảy ra nếu áp dụng một statistical technique vốn không dành cho high-dimensional setting. Ta dùng least squares regression làm ví dụ, nhưng các concept tương tự áp dụng cho logistic regression, linear discriminant analysis và các classical statistical approach khác.

Khi số features  $p$  lớn bằng hoặc lớn hơn số observations  $n$ , least squares như Chapter 3 không thể—hay chính xác hơn là không nên—được thực hiện. Lý do rất đơn giản: bất kể features có thật sự relationship với response hay không, least squares sẽ tìm được một set coefficient estimates cho perfect fit trên data, với residual bằng zero.

Figure 6.22 minh họa với  $p = 1$  feature (cộng intercept) trong hai case: có 20 observations và chỉ có hai observations. Khi có 20 observations,  $n > p$  và least squares regression line không fit data hoàn hảo; thay vào đó nó approximate 20 observations tốt nhất có thể. Khi chỉ có hai observations, bất kể value của chúng là gì, regression line sẽ fit data chính xác. Điều này problematic vì perfect fit gần như chắc chắn dẫn tới overfitting. Dù có thể fit training data hoàn hảo trong high-dimensional setting, linear model thu được sẽ thường perform cực kém trên independent test set và vì thế không phải useful model. Thật vậy, line từ right panel Figure 6.22 sẽ perform rất kém nếu test set gồm observations ở left panel. Vấn đề là khi  $p > n$  hoặc  $p \approx n$ , ordinary least squares quá flexible và overfit data.

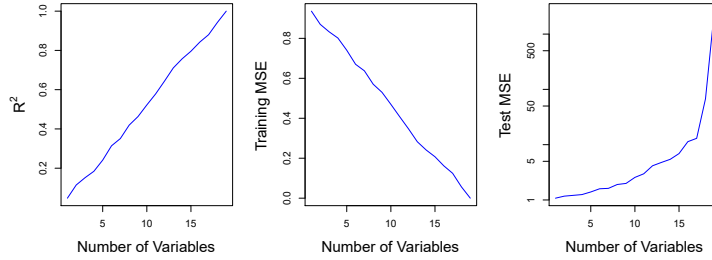


**HÌNH 6.22.** Left: Least squares regression trong low-dimensional setting. Right: Least squares regression với  $n = 2$  observations và hai parameter cần estimate (một intercept và một coefficient).

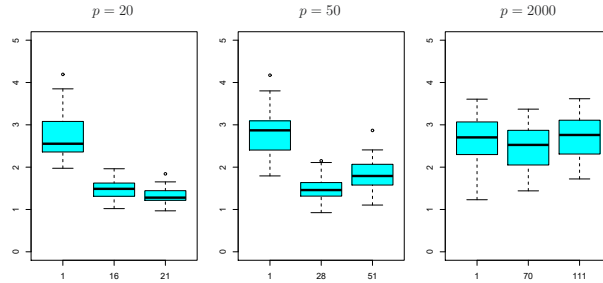
Figure 6.23 tiếp tục minh họa risk của việc bất cẩn dùng least squares khi  $p$  lớn. Data được simulate với  $n = 20$  observations, và regression dùng từ 1 đến 20 features, tất cả đều hoàn toàn unrelated với response. Như figure cho thấy, model  $R^2$  tăng lên 1 và training-set MSE tương ứng giảm về 0 khi số features tăng, dù các feature hoàn toàn không liên quan response. Ngược lại, MSE trên independent test set tăng cực mạnh khi thêm features vì additional predictors làm variance của coefficient estimates tăng rất lớn. Nhìn test-set MSE, model tốt nhất chỉ chứa nhiều nhất vài variables. Tuy nhiên, nếu chỉ nhìn  $R^2$  hoặc training-set MSE, ta có thể kết luận sai rằng model có nhiều variables nhất là tốt nhất. Điều này cho thấy cần extra care khi analyze dataset với nhiều variables và phải luôn evaluate model performance trên independent test set.

Trong Section 6.1.3 ta đã thấy nhiều approach adjust training-set RSS





**HÌNH 6.23.** Trong simulated example với  $n = 20$  training observations, các feature hoàn toàn unrelated với outcome lần lượt được thêm vào model. Left:  $R^2$  tăng lên 1 khi thêm nhiều feature. Center: Training-set MSE giảm về 0. Right: Test-set MSE tăng khi thêm feature.



**HÌNH 6.24.** Lasso được thực hiện với  $n = 100$  observations và ba value của  $p$ , số features. Trong  $p$  features có 20 feature thật sự associated với response. Boxplot cho test MSE với ba value khác nhau của tuning parameter  $\lambda$  trong (6.7). Để dễ interpretation, thay vì report  $\lambda$ , figure report degrees of freedom; với lasso đây chính là số estimated non-zero coefficients. Khi  $p = 20$ , test MSE thấp nhất đạt được với mức regularization nhỏ nhất. Khi  $p = 50$ , test MSE thấp nhất đạt được khi regularization đáng kể. Khi  $p = 2,000$ , lasso perform kém bất kể regularization, do chỉ 20 trong 2,000 features thật sự associated với outcome.

hoặc  $R^2$  để account cho số variables trong least squares model. Đáng tiếc,  $C_p$ , AIC và BIC không phù hợp trong high-dimensional setting vì estimate  $\hat{\sigma}^2$  trở nên problematic. Chẳng hạn formula  $\hat{\sigma}^2$  từ Chapter 3 cho estimate  $\hat{\sigma}^2 = 0$  trong setting này. Tương tự, adjusted  $R^2$  cũng gặp vấn đề vì ta có thể dễ dàng đạt adjusted  $R^2 = 1$ . Rõ ràng cần alternative approach phù hợp hơn với high-dimensional setting.

### 6.4.3 Regression in High Dimensions

Nhiều method trong chapter này nhằm fit least-squares model ít flexible hơn—như forward stepwise selection, ridge regression, lasso và principal components regression—đặc biệt hữu ích cho regression trong high-dimensional setting. Về bản chất, các approach này tránh overfitting bằng một fitting approach ít flexible hơn least squares.

Figure 6.24 minh họa performance của lasso trong một simulated example đơn giản. Có  $p = 20, 50$  hoặc 2,000 features, trong đó 20 feature thật sự associated với outcome. Lasso được fit trên  $n = 100$  training observations và mean squared error được evaluate trên independent test set. Khi số features tăng, test-set error tăng. Khi  $p = 20$ , lowest validation-set error đạt được khi  $\lambda$  trong (6.7) nhỏ; khi  $p$  lớn hơn, lowest validation-set error đạt được với  $\lambda$  lớn hơn.

Trong mỗi boxplot, thay vì report  $\lambda$ , degrees of freedom của resulting lasso solution được hiển thị; đây đơn giản là số non-zero coefficient estimates và là một measure của flexibility của lasso fit. Figure 6.24 làm nổi bật ba point quan trọng: (1) regularization hay shrinkage đóng vai trò then chốt trong high-dimensional problem, (2) appropriate tuning-parameter selection là crucial để predictive performance tốt, và (3) test error có xu hướng tăng khi dimensionality của problem (tức số features hoặc predictors) tăng, trừ khi additional features thật sự associated với response.

Point thứ ba là một key principle trong analysis của high-dimensional data, gọi là *curse of dimensionality*. Ta có thể nghĩ rằng tăng số features dùng để fit model sẽ làm quality của model tăng. Nhưng so sánh left và right panels Figure 6.24 cho thấy không nhất thiết vậy: trong example này test-set MSE gần gấp đôi khi  $p$  tăng từ 20 lên 2,000. Nói chung, thêm signal features thật sự associated với response sẽ cải thiện fitted model bằng cách giảm test-set error. Ngược lại, thêm noise features không thật sự associated với response làm model tệ đi và tăng test-set error.

Lý do là noise features tăng dimensionality của problem, làm risk of overfitting nghiêm trọng hơn—do noise feature có thể nhận non-zero coefficient chỉ vì chance association với response trên training set—mà không có upside về test-set error. Vì vậy technology mới cho phép collect measurements cho hàng nghìn hay hàng triệu features là con dao hai lưỡi: chúng có thể tạo predictive model tốt hơn nếu features thật sự relevant, nhưng sẽ làm result xấu đi nếu features không relevant. Ngay cả khi relevant, variance phát sinh khi fit coefficient của chúng có thể lớn hơn reduction in bias mà chúng đem lại.

*curse of  
dimensionality*

#### 6.4.4 Interpreting Results in High Dimensions

Khi thực hiện lasso, ridge regression hoặc regression procedure khác trong high-dimensional setting, phải rất thận trọng khi report result. Chapter 3 đã nói về multicollinearity, tức các variable trong regression có thể correlated với nhau. Trong high-dimensional setting, multicollinearity trở nên extreme: bất kỳ variable nào trong model cũng có thể viết như linear combination của toàn bộ variables còn lại. Về bản chất, điều này nghĩa là ta không bao giờ biết chính xác variable nào (nếu có) thật sự predictive of outcome, cũng không thể xác định duy nhất best coefficients cho regression. Tối đa, ta chỉ hy vọng assign large regression coefficients cho các variable correlated với những variable thật sự predictive.

Ví dụ, giả sử ta predict blood pressure từ nửa triệu SNPs, và forward stepwise selection cho thấy 17 SNPs tạo good predictive model trên training data. Sẽ sai nếu kết luận 17 SNP này predict blood pressure hiệu quả hơn các SNP không được chọn. Có khả năng tồn tại nhiều tập 17 SNP khác cũng predict tốt tương đương. Nếu obtain independent dataset rồi thực hiện forward stepwise selection, ta có thể nhận một model với tập SNP khác, thậm chí không overlap. Điều đó không làm giảm giá trị của model đã có—chẳng hạn nó có thể predict rất hiệu quả trên independent patient set và hữu ích clinically—nhưng ta không nên overstate result. Cần nói rõ rằng ta chỉ tìm ra một trong nhiều possible model để predict blood pressure và model còn phải được validate trên independent datasets.

Cũng phải đặc biệt cẩn thận khi report error và measure of model fit trong high-dimensional setting. Khi  $p > n$ , ta dễ dàng obtain một useless model có zero residuals. Vì vậy không bao giờ dùng sum of squared errors,  $p$ -values,  $R^2$  statistics hay traditional measure of model fit trên training data làm evidence cho good model fit trong high-dimensional setting. Như Figure 6.23, có thể dễ dàng có  $R^2 = 1$  khi  $p > n$ . Report điều đó có thể khiến người khác hiểu nhầm rằng đã obtain statistically valid, useful model, trong khi nó hoàn toàn không cung cấp evidence như vậy. Thay vào đó, cần report result trên independent test set hoặc cross-validation errors. Ví dụ MSE hoặc  $R^2$  trên independent test set là valid measure of model fit, còn MSE trên training set chắc chắn không phải.

### 6.5 Lab: Linear Models and Regularization Methods

Trong lab này ta implement nhiều technique đã bàn trong chapter. Trước hết import các library quen thuộc ở top level.

```
In [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
from statsmodels.api import OLS
import sklearn.model_selection as skm
import sklearn.linear_model as skl
from sklearn.preprocessing import StandardScaler
from ISLP import load_data
from ISLP.models import ModelSpec as MS
from functools import partial
```

Ta cũng gom các import mới cần cho lab này.

```
In [2]: from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
from ISLP.models import \
 (Stepwise,
 sklearn_selected,
 sklearn_selection_path)
!pip install l0bnb
from l0bnb import fit_path
```

Package `l0bnb` được install ngay trong lúc chạy. Lưu ý lệnh `!pip install` được escape; đây là một system command riêng.

### 6.5.1 Subset Selection Methods

Ở đây ta implement các method giảm số parameter trong model bằng cách restrict model vào một subset của input variables.

#### Forward Selection

Ta áp dụng forward-selection approach lên data `Hitters`. Mục tiêu là predict `Salary` của một baseball player dựa trên các statistic về performance ở previous year.

Trước hết, variable `Salary` bị missing ở một số player. Function `np.isnan()` có thể identify missing observations; nó trả về array cùng shape với input vector, với `True` cho missing element và `False` cho non-missing element. Method `sum()` sau đó đếm tổng số missing element.

`np.isnan()`  
`sum()`

```
In [3]: Hitters = load_data('Hitters')
np.isnan(Hitters['Salary']).sum()
```

Out[3]: 59

Ta thấy `Salary` missing cho 59 player. Method `dropna()` của data frame mặc định remove mọi row có missing value ở bất kỳ variable nào (xem `Hitters.dropna()`).

```
In [4]: Hitters = Hitters.dropna();
Hitters.shape
```

Out[4]: (263, 20)

Trước tiên ta chọn best model bằng forward selection dựa trên  $C_p$  (6.2). Score này không có sẵn như metric trong `sklearn`, vì vậy ta tự define function để compute và dùng nó làm scorer. Mặc định `sklearn` cố maximize score, nên scoring function của ta compute negative  $C_p$  statistic.

```
In [5]: def nCp(sigma2, estimator, X, Y):
 "Negative Cp statistic"
 n, p = X.shape
 Yhat = estimator.predict(X)
 RSS = np.sum((Y - Yhat)**2)
 return -(RSS + 2 * p * sigma2) / n
```

Ta cần estimate residual variance  $\sigma^2$ , là first argument của scoring func-

tion trên. Ta fit largest model, dùng toàn bộ variables, rồi estimate  $\sigma^2$  từ MSE của model đó.

```
In [6]: design = MS(Hitters.columns.drop('Salary')).fit(Hitters)
Y = np.array(Hitters['Salary'])
X = design.transform(Hitters)
sigma2 = OLS(Y,X).fit().scale
```

Function `sklearn_selected()` chỉ expect scorer có ba arguments—ba argument cuối trong definition `nCp()` ở trên. Ta dùng `partial()` đã gặp ở Section 5.3.3 để freeze first argument bằng estimate  $\sigma^2$ .

```
In [7]: neg_Cp = partial(nCp, sigma2)
```

Bây giờ có thể dùng `neg_Cp()` làm scorer cho model selection. Ngoài score, ta cần specify search strategy. Việc này được thực hiện qua object `Stepwise()` trong package `ISLP.models`. Method `Stepwise.first_peak()` chạy forward stepwise cho tới khi thêm variable nữa không còn improve evaluation score. Tương tự, `Stepwise.fixed_steps()` chạy một fixed number of steps trong stepwise search.

```
In [8]: strategy = Stepwise.first_peak(design,
 direction='forward',
 max_terms=len(design.terms))
```

Ta fit linear regression với `Salary` làm outcome bằng forward selection. Function `sklearn_selected()` trong `ISLP.models` nhận một model từ `statsmodels` cùng search strategy, rồi select model qua method `fit`. Nếu không specify scoring argument, score mặc định là MSE, nên cả 19 variables sẽ được chọn (output không hiển thị).

*sklearn\_selected()*

```
In [9]: hitters_MSE = sklearn_selected(OLS,
 strategy)
hitters_MSE.fit(Hitters, Y)
hitters_MSE.selected_state_
```

Dùng `neg_Cp` cho model nhỏ hơn, đúng như kỳ vọng, chỉ có 10 variables được chọn.

```
In [10]: hitters_Cp = sklearn_selected(OLS,
 strategy,
 scoring=neg_Cp)
hitters_Cp.fit(Hitters, Y)
hitters_Cp.selected_state_
```

```
Out[10]: ('Assists',
 'AtBat',
 'CAtBat',
 'CRBI',
 'CRuns',
 'CWalks',
 'Division',
 'Hits',
 'PutOuts',
 'Walks')
```

*Choosing Among Models Using the Validation Set Approach and Cross-Validation*

Thay cho  $C_p$ , ta có thể dùng cross-validation để select model trong forward selection. Ta cần một method lưu full path của các model được tìm trong forward selection và cho phép prediction với từng model. Estimator `sklearn_selection_path()` trong `ISLP.models` làm được việc này. Function `cross_val_predict()` tính cross-validated predictions cho từng model dọc path, từ đó ta evaluate cross-validated MSE trên path.

`sklearn_selection_`  
`path()`  
`cross_val_`  
`predict()`

Ta define một strategy fit toàn bộ forward-selection path. Dù `sklearn_selection_path()` có nhiều parameter choice, ở đây dùng defaults, chọn model tại mỗi step theo mức reduction in RSS lớn nhất.

In [11]:

```
strategy = Stepwise.fixed_steps(design,
 len(design.terms),
 direction='forward')
full_path = sklearn_selection_path(OLS, strategy)
```

Ta fit full forward-selection path trên `Hitters` và compute fitted values.

In [12]:

```
full_path.fit(Hitters, Y)
Yhat_in = full_path.predict(Hitters)
Yhat_in.shape
```

Out[12]:

(263, 20)

Kết quả là một array fitted values gồm 20 steps, kể cả fitted mean cho null model. Ta dùng nó để evaluate in-sample MSE. Như dự kiến, in-sample MSE improve ở mỗi step, nên phải dùng validation hoặc cross-validation để chọn số steps. Ta fix y-axis từ 50,000 đến 250,000 để so với cross-validation/validation-set MSE bên dưới, cũng như ridge regression, lasso và principal components regression.

In [13]:

```
mse_fig, ax = subplots(figsize=(8,8))
insample_mse = ((Yhat_in - Y[:,None])**2).mean(0)
n_steps = insample_mse.shape[0]
ax.plot(np.arange(n_steps),
 insample_mse,
 'k', # color black
 label='In-sample')
ax.set_ylabel('MSE',
 fontsize=20)
ax.set_xlabel('# steps of forward stepwise',
 fontsize=20)
ax.set_xticks(np.arange(n_steps)[::2])
ax.legend()
ax.set_ylim([50000,250000]);
```

Lưu ý expression `None` trong `Y[:,None]`. Nó thêm một axis (dimension) vào one-dimensional array `Y`, để array này có thể được recycle khi subtract khỏi two-dimensional `Yhat_in`.

Ta sẵn sàng dùng cross-validation để estimate test error dọc model path. Phải chỉ dùng training observations cho mọi aspect của model-fitting, gồm cả variable selection. Vì vậy model tốt nhất ở mỗi size phải được xác định chỉ bằng training observations trong từng training fold. Đây là point tinh tế nhưng quan trọng: nếu full dataset được dùng để select best subset tại mỗi

step, validation-set error và cross-validation error sẽ không phải accurate estimate của test error.

Ta compute cross-validated predicted values bằng 5-fold cross-validation.

In [14]:

```
K=5
kfold = skm.KFold(K,
 random_state=0,
 shuffle=True)
Yhat_cv = skm.cross_val_predict(full_path,
 Hitters,
 Y,
 cv=kfold)

Yhat_cv.shape
```

*skm.KFold()  
skm.cross\_val\_  
predict()*

Out[14]: (263, 20)

Prediction matrix `Yhat_cv` có cùng shape với `Yhat_in`; khác biệt là prediction ở mỗi row (tương ứng một sample index) được tạo từ model fit trên training fold không chứa row đó.

Tại mỗi model dọc path, ta compute MSE trong từng cross-validation fold, rồi average để lấy mean MSE và dùng individual values để tạo crude estimate của standard error of the mean.<sup>9</sup> Vì vậy ta cần biết test indices của từng cross-validation split. Có thể lấy bằng method `split()` của `kfold`. Do đã fix random state, mỗi lần split bất kỳ array nào có cùng số row như `Y`, ta recover cùng training/test indices; bên dưới chỉ ignore training indices.

In [15]:

```
cv_mse = []
for train_idx, test_idx in kfold.split(Y):
 errors = (Yhat_cv[test_idx] - Y[test_idx,None])**2
 cv_mse.append(errors.mean(0)) # column means
cv_mse = np.array(cv_mse).T
cv_mse.shape
```

Out[15]: (20, 5)

Bây giờ thêm cross-validation error estimate vào MSE plot, gồm mean error qua năm folds và estimate standard error of the mean.

In [16]:

```
ax.errorbar(np.arange(n_steps),
 cv_mse.mean(1),
 cv_mse.std(1) / np.sqrt(K),
 label='Cross-validated',
 c='r') # color red
ax.set_ylim([50000,250000])
ax.legend()
mse_fig
```

Để lặp lại bằng validation-set approach, ta chỉ đổi `cv` argument thành một validation set: một random split của data thành test và training. Ta chọn test size 20%, gần bằng size mỗi test set trong 5-fold cross-validation.

*skm.ShuffleSplit()*

<sup>9</sup>Estimate này crude vì năm error estimate dựa trên các overlapping training set và do đó không independent.

```
In [17]: validation = skm.ShuffleSplit(n_splits=1,
 test_size=0.2,
 random_state=0)
for train_idx, test_idx in validation.split(Y):
 full_path.fit(Hitters.iloc[train_idx],
 Y[train_idx])
 Yhat_val = full_path.predict(Hitters.iloc[test_idx])
 errors = (Yhat_val - Y[test_idx,None])**2
 validation_mse = errors.mean(0)
```

Giống case in-sample MSE, validation-set approach không cung cấp standard errors.

```
In [18]: ax.plot(np.arange(n_steps),
 validation_mse,
 'b--', # color blue, broken line
 label='Validation')
ax.set_xticks(np.arange(n_steps)[::2])
ax.set_ylim([50000,250000])
ax.legend()
mse_fig
```

### Best Subset Selection

Forward stepwise là greedy selection procedure: ở mỗi step nó augment current set bằng một additional variable. Bây giờ ta áp dụng best subset selection cho `Hitters`; với mỗi subset size, method search best set of predictors.

Ta dùng package `10bnb` để perform best subset selection. Thay vì constrain subset phải có một fixed size, package tạo path of solutions trong đó subset size đóng vai trò penalty thay vì constraint. Distinction này tinh tế, nhưng khác biệt trở nên quan trọng khi cross-validate.

```
In [19]: D = design.fit_transform(Hitters)
D = D.drop('intercept', axis=1)
X = np.asarray(D)
```

Ở đây first column tương ứng intercept đã được exclude vì `10bnb` sẽ fit intercept riêng. Ta tìm path bằng function `fit_path()`.

```
In [20]: path = fit_path(X,
 Y,
 max_nonzeros=X.shape[1])
```

Function `fit_path()` trả về list; mỗi value gồm fitted coefficients dưới key `B`, intercept dưới `B0`, cùng vài attribute khác liên quan particular path algorithm. Các detail đó nằm ngoài scope của sách.

```
In [21]: path[3]
```

```
Out[21]: {'B': array([0. , 3.254844, 0. , 0. , 0. ,
 0. , 0. , 0. , 0. ,
 0. , 0.677753, 0. , 0. , 0. ,
 0. , 0. , 0. , 0.]),
 'B0': -38.98216739555494,
 'lambda_0': 0.011416248027450194,
 'M': 0.5829861733382011,
 'Time_exceeded': False}
```



Trong example trên, ở fourth step của path có hai nonzero coefficients trong 'B', tương ứng value 0.114 của penalty parameter `lambda_0`. Có thể dùng sequence of fits này để prediction trên validation set như function của `lambda_0`, hoặc với nhiều work hơn bằng cross-validation.

### 6.5.2 Ridge Regression and the Lasso

Ta dùng package `sklearn.linear_model` (viết tắt `skl`) để fit ridge và lasso regularized linear model trên `Hitters`. Bắt đầu với model matrix `X` (không intercept) đã compute ở phần best subset regression.

#### Ridge Regression

Ta dùng function `skl.ElasticNet()` để fit cả ridge và lasso. Để fit path của ridge regression models, dùng `skl.ElasticNet.path()`, có thể fit ridge, lasso và hybrid mixture; ridge regression tương ứng `l1_ratio=0`. Tốt nhất nên standardize columns của `X` nếu variables đo ở units khác nhau. Vì `skl.ElasticNet()` không tự normalize, ta phải xử lý. Do standardize trước, để recover coefficient estimates trên original scale thì phải unstandardize coefficients. Parameter  $\lambda$  trong (6.5) và (6.7) được gọi là **alphas** trong `sklearn`; để nhất quán với chapter, bên dưới dùng tên **lambdas**.<sup>10</sup>

`skl.ElasticNet()`  
`skl.ElasticNet.`  
`path()`

In [22]:

```
Xs = X - X.mean(0)[None,:]
X_scale = X.std(0)
Xs = Xs / X_scale[None,:]
lambdas = 10*np.linspace(8, -2, 100) / Y.std()
soln_array = skl.ElasticNet.path(Xs,
 Y,
 l1_ratio=0.,
 alphas=lambdas)[1]

soln_array.shape
```

Out[22]: (19, 100)

Ở đây ta extract array of coefficients tương ứng solutions dọc regularization path. Mặc định `skl.ElasticNet.path` fit path trên automatically selected range of  $\lambda$ , trừ case `l1_ratio=0` tạo ridge regression.<sup>11</sup> Vì vậy ta explicit grid từ  $\lambda = 10^8$  đến  $10^{-2}$  được scale bởi standard deviation của  $y$ , gần như phủ toàn bộ range từ null model chỉ có intercept tới least-squares fit.

Mỗi value  $\lambda$  đi với một vector ridge regression coefficients, truy cập bằng một column của `soln_array`. Trong case này `soln_array` là matrix  $19 \times 100$ , với 19 rows (mỗi predictor một row) và 100 columns (mỗi  $\lambda$  một column).

Ta transpose matrix này và convert thành data frame để dễ view và plot.

In [23]:

```
soln_path = pd.DataFrame(soln_array.T,
 columns=D.columns,
 index=-np.log(lambdas))
soln_path.index.name = 'negative log(lambda)'
soln_path
```

<sup>10</sup>Tại thời điểm xuất bản, ridge fits như code chunk [22] phát sinh convergence warning không cần thiết; authors kỳ vọng warning này sẽ biến mất khi package trưởng thành hơn.

<sup>11</sup>Lý do khá technical: với mọi model trừ ridge, có thể tìm smallest  $\lambda$  sao cho mọi coefficient đều zero. Với ridge, value đó là  $\infty$ .

```
Out[23]:
 AtBat Hits HmRun Runs ...
negative
log(lambda)
-12.310855 0.000800 0.000889 0.000695 0.000851 ...
-12.078271 0.001010 0.001122 0.000878 0.001074 ...
-11.845686 0.001274 0.001416 0.001107 0.001355 ...
-11.613102 0.001608 0.001787 0.001397 0.001710 ...
-11.380518 0.002029 0.002255 0.001763 0.002158 ...
...

100 rows × 19 columns
```

Ta plot path để thấy coefficients vary theo  $\lambda$ . Để control legend location, trước hết set `legend=False` trong plot method rồi thêm legend bằng `legend()` của `ax`.

```
In [24]:
path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Standardized coefficients', fontsize=20)
ax.legend(loc='upper left');
```

(Ta dùng LaTeX formatting trong horizontal label để format Greek  $\lambda$ .) Ta expect coefficient estimates có  $\ell_2$  norm nhỏ hơn nhiều khi dùng  $\lambda$  lớn so với  $\lambda$  nhỏ. Nhớ rằng  $\ell_2$  norm là square root của sum of squared coefficient values. Ta display coefficients ở step 40, nơi  $\lambda = 25.535$ .

```
In [25]:
beta_hat = soln_path.loc[soln_path.index[39]]
lambdas[39], beta_hat
```

```
Out[25]:
(25.535,
 AtBat 5.433750
 Hits 6.223582
 HmRun 4.585498
 Runs 5.880855
 RBI 6.195921
 Walks 6.277975
 Years 5.299767
 ...)
```

Ta compute  $\ell_2$  norm của standardized coefficients.

```
In [26]:
np.linalg.norm(beta_hat)
```

```
Out[26]: 24.17
```

Ngược lại, dưới đây là  $\ell_2$  norm khi  $\lambda = 2.44e-01$ . Coefficient norm lớn hơn nhiều với value  $\lambda$  nhỏ hơn này.

```
In [27]:
beta_hat = soln_path.loc[soln_path.index[59]]
lambdas[59], np.linalg.norm(beta_hat)
```

```
Out[27]: (0.2437, 160.4237)
```

Ở trên ta normalize `X` trước rồi fit ridge model bằng `Xs`. Object `Pipeline()` trong `sklearn` cung cấp cách rõ ràng để tách feature normalization khỏi việc fit ridge model.

```
In [28]: ridge = skl.ElasticNet(alpha=lambdas[59], l1_ratio=0)
scaler = StandardScaler(with_mean=True, with_std=True)
pipe = Pipeline(steps=[('scaler', scaler), ('ridge', ridge)])
pipe.fit(X, Y)
```

Ta cho thấy nó cho cùng  $\ell_2$  norm như previous fit trên standardized data.

```
In [29]: np.linalg.norm(ridge.coef_)
```

```
Out[29]: 160.4237
```

Lưu ý operation `pipe.fit(X, Y)` đã thay đổi object `ridge`, đặc biệt thêm các attribute như `coef_` vốn chưa tồn tại trước đó.

### *Estimating Test Error of Ridge Regression*

Chọn trước một value  $\lambda$  cho ridge regression là khó, nếu không nói là impossible. Ta muốn dùng validation method hoặc cross-validation để select tuning parameter. Không bất ngờ khi `Pipeline()` có thể được dùng trong `skm.cross_validate()` với validation method (tức `validation`) hoặc  $k$ -fold cross-validation. Ta fix random state của splitter để result reproducible.

```
In [30]: validation = skm.ShuffleSplit(n_splits=1,
 test_size=0.5,
 random_state=0)

ridge.alpha = 0.01
results = skm.cross_validate(ridge,
 X,
 Y,
 scoring='neg_mean_squared_error',
 cv=validation)

- results['test_score']
```

```
Out[30]: array([134214.0])
```

Test MSE là  $1.342e+05$ . Nếu chỉ fit model có intercept, ta sẽ predict mỗi test observation bằng mean của training observations. Có thể lấy cùng result bằng ridge regression với  $\lambda$  rất lớn. Lưu ý  $1e10$  nghĩa là  $10^{10}$ .

```
In [31]: ridge.alpha = 1e10
results = skm.cross_validate(ridge,
 X,
 Y,
 scoring='neg_mean_squared_error',
 cv=validation)

- results['test_score']
```

```
Out[31]: array([231788.32])
```

Rõ ràng chọn  $\lambda = 0.01$  là arbitrary, nên ta dùng cross-validation hoặc validation-set approach để chọn tuning parameter  $\lambda$ . Object `GridSearchCV()` cho exhaustive grid search để chọn parameter như vậy. Trước hết ta dùng validation-set method để chọn  $\lambda$ .

*GridSearchCV()*

```
In [32]: param_grid = {'ridge__alpha': lambdas}
grid = skm.GridSearchCV(pipe,
 param_grid,
 cv=validation,
 scoring='neg_mean_squared_error')

grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

```
Out[32]: Pipeline(steps=[('scaler', StandardScaler()),
 ('ridge', ElasticNet(alpha=0.005899, l1_ratio=0))])
```

Alternative, có thể dùng 5-fold cross-validation.

```
In [33]: grid = skm.GridSearchCV(pipe,
 param_grid,
 cv=kfold,
 scoring='neg_mean_squared_error')

grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

Nhớ rằng ta setup object `kfold` cho 5-fold cross-validation ở page 271. Bây giờ plot cross-validated MSE như function của  $-\log(\lambda)$ , trong đó shrinkage giảm từ left sang right.

```
In [34]: ridge_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
 -grid.cv_results_['mean_test_score'],
 yerr=grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylim([50000, 250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated MSE', fontsize=20);
```

Có thể cross-validate các metric khác nhau để chọn parameter. Default metric cho `skl.ElasticNet()` là test  $R^2$ . Ta compare  $R^2$  với MSE cho cross-validation ở đây.

```
In [35]: grid_r2 = skm.GridSearchCV(pipe,
 param_grid,
 cv=kfold)

grid_r2.fit(X, Y)
```

Cuối cùng plot result cho cross-validated  $R^2$ .

```
In [36]: r2_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
 grid_r2.cv_results_['mean_test_score'],
 yerr=grid_r2.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated R^2', fontsize=20);
```

### Fast Cross-Validation for Solution Paths

Ridge, lasso và elastic net có thể được fit hiệu quả dọc sequence of  $\lambda$  values, tạo nên *solution path* hay *regularization path*. Vì vậy có specialized code để fit các path này và chọn suitable  $\lambda$  bằng cross-validation. Dù dùng identical splits, result sẽ không agree hoàn toàn với grid trên vì standard-

ization của mỗi feature trong `grid` được thực hiện riêng trên từng fold, còn trong `pipeCV` bên dưới chỉ thực hiện một lần. Tuy nhiên result tương tự vì normalization khá stable giữa các fold.

```
In [37]: ridgeCV = skl.ElasticNetCV(alphas=lambdas,
 l1_ratio=0,
 cv=kfold)
pipeCV = Pipeline(steps=[('scaler', scaler),
 ('ridge', ridgeCV)])
pipeCV.fit(X, Y)
```

Ta lại tạo plot cross-validation error để thấy nó tương tự khi dùng `skm.GridSearchCV`.

```
In [38]: tuned_ridge = pipeCV.named_steps['ridge']
ridgeCV_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
 tuned_ridge.mse_path_.mean(1),
 yerr=tuned_ridge.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_ridge.alpha_), c='k', ls='--')
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated MSE', fontsize=20);
```

Ta thấy value  $\lambda$  cho smallest cross-validation error là  $1.19e-02$ , có ở `tuned_ridge.alpha_`. Test MSE tương ứng là bao nhiêu?

```
In [39]: np.min(tuned_ridge.mse_path_.mean(1))
```

```
Out[39]: 115526.71
```

Đây là improvement thêm so với test MSE khi dùng  $\lambda = 4$ . Cuối cùng `tuned_ridge.coef_` chứa coefficients fit trên entire dataset tại value  $\lambda$  này.

```
In [40]: tuned_ridge.coef_
```

```
Out[40]: array([-222.80877051, 238.77246614, 3.21103754, -2.93050845,
 3.64888723, 108.90953869, -50.81896152, -105.15731984,
 122.00714801, 57.1859509 , 210.35170348, 118.05683748,
 -150.21959435, 30.36634231, -61.62459095, 77.73832472,
 40.07350744, -25.02151514, -13.68429544])
```

Đúng như expected, không coefficient nào bằng zero—ridge regression không perform variable selection.

### *Evaluating Test Error of Cross-Validated Ridge*

Chọn  $\lambda$  bằng cross-validation tạo ra một single regression estimator, tương tự fit linear regression model trong Chapter 3. Vì vậy việc estimate test error của nó là reasonable. Nhưng cross-validation đã “touch” toàn bộ data khi chọn  $\lambda$ , nên không còn data độc lập để estimate test error. Một compromise là split data ban đầu thành hai disjoint sets: training và test. Sau đó fit cross-validation-tuned ridge regression trên training set và evaluate trên test set. Có thể gọi đây là cross-validation nested trong validation-set approach. Không có lý do a priori để chia nửa data cho mỗi set; bên dưới dùng 75% training và 25% test, estimator là ridge regression tuned bằng 5-fold cross-validation.

```
In [41]: outer_valid = skm.ShuffleSplit(n_splits=1,
 test_size=0.25,
 random_state=1)

inner_cv = skm.KFold(n_splits=5,
 shuffle=True,
 random_state=2)

ridgeCV = skl.ElasticNetCV(alphas=lambdas,
 l1_ratio=0,
 cv=inner_cv)

pipeCV = Pipeline(steps=[('scaler', scaler),
 ('ridge', ridgeCV)]);
```

```
In [42]: results = skm.cross_validate(pipeCV,
 X,
 Y,
 cv=outer_valid,
 scoring='neg_mean_squared_error')

- results['test_score']
```

```
Out[42]: array([132393.84])
```

### *The Lasso*

Ta đã thấy ridge regression với lựa chọn  $\lambda$  hợp lý có thể outperform least squares và null model trên **Hitters**. Bây giờ hỏi liệu lasso có thể tạo model accurate hơn hoặc interpretable hơn ridge regression hay không. Để fit lasso, ta lại dùng `ElasticNetCV()`, nhưng lần này argument `l1_ratio=1`. Ngoài thay đổi đó, procedure giống ridge.

```
In [43]: lassoCV = skl.ElasticNetCV(n_alphas=100,
 l1_ratio=1,
 cv=kfold)

pipeCV = Pipeline(steps=[('scaler', scaler),
 ('lasso', lassoCV)])

pipeCV.fit(X, Y)
tuned_lasso = pipeCV.named_steps['lasso']
tuned_lasso.alpha_
```

```
Out[43]: 3.147
```

```
In [44]: lambdas , soln_array = skl.Lasso.path(Xs,
 Y,
 l1_ratio=1,
 n_alphas=100)[:2]

soln_path = pd.DataFrame(soln_array.T,
 columns=D.columns,
 index=-np.log(lambdas))
```

Từ coefficient plot của standardized coefficients có thể thấy tùy tuning parameter, một số coefficients sẽ exactly zero.

```
In [45]: path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.legend(loc='upper left')
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Standardized coefficients', fontsize=20);
```

Smallest cross-validated error thấp hơn test-set MSE của null model và

least squares, và rất gần test MSE 115526.71 của ridge regression (page 278) với  $\lambda$  chọn bằng cross-validation.

```
In [46]: np.min(tuned_lasso.mse_path_.mean(1))
```

```
Out[46]: 114690.73
```

Ta lại tạo plot cross-validation error.

```
In [47]: lassoCV_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(tuned_lasso.alphas_),
 tuned_lasso.mse_path_.mean(1),
 yerr=tuned_lasso.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_lasso.alpha_), c='k', ls='--')
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated MSE', fontsize=20);
```

Tuy nhiên lasso có advantage lớn so với ridge: resulting coefficient estimates sparse. Ở đây 6 trong 19 coefficient estimates exactly zero, nên lasso model với  $\lambda$  chọn bằng cross-validation chỉ chứa 13 variables.

```
In [48]: tuned_lasso.coef_
```

```
Out[48]: array([-210.01008773, 243.4550306 , 0. , 0. ,
 0. , 97.69397357, -41.52283116, -0. ,
 0. , 39.62298193, 205.75273856, 124.55456561,
 -126.29986768, 15.70262427, -59.50157967, 75.24590036,
 21.62698014, -12.04423675, -0.])
```

Giống ridge regression, ta có thể evaluate test error của cross-validated lasso bằng cách split test/training trước rồi chạy cross-validation internally trên training set. Việc này được để lại như exercise.

### 6.5.3 PCR and PLS Regression

#### Principal Components Regression

Principal components regression (PCR) có thể được thực hiện bằng PCA() từ module `sklearn.decomposition`. Ta áp dụng PCR cho `Hitters` để predict `Salary`; nhớ bảo đảm missing values đã được remove như Section 6.5.1. Ở đây dùng `LinearRegression()` để fit regression model. Function này fit intercept mặc định, khác `OLS()` ở Section 6.5.1.

```
In [49]: pca = PCA(n_components=2)
linreg = skl.LinearRegression()
pipe = Pipeline([('pca', pca),
 ('linreg', linreg)])
pipe.fit(X, Y)
pipe.named_steps['linreg'].coef_
```

PCA()  
LinearRegression()

```
Out[49]: array([0.09846131, 0.4758765])
```

Khi perform PCA, result phụ thuộc việc data có được standardize hay không. Như các example trước, có thể làm bằng cách thêm một step vào pipeline.

```
In [50]: pipe = Pipeline([('scaler', scaler),
 ('pca', pca),
 ('linreg', linreg)])
pipe.fit(X, Y)
pipe.named_steps['linreg'].coef_
```

```
Out[50]: array([106.36859204, -21.60350456])
```

Ta có thể dùng CV để chọn number of components bằng `skm.GridSearchCV`; ở đây vary parameter `n_components`.

```
In [51]: param_grid = {'pca__n_components': range(1, 20)}
grid = skm.GridSearchCV(pipe,
 param_grid,
 cv=kfold,
 scoring='neg_mean_squared_error')
grid.fit(X, Y)
```

Plot result giống các method trước.

```
In [52]: pcr_fig, ax = subplots(figsize=(8,8))
n_comp = param_grid['pca__n_components']
ax.errorbar(n_comp,
 -grid.cv_results_['mean_test_score'],
 grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylabel('Cross-validated MSE', fontsize=20)
ax.set_xlabel('# principal components', fontsize=20)
ax.set_xticks(n_comp[:2])
ax.set_ylim([50000, 250000]);
```

Smallest cross-validation error xảy ra khi dùng 17 components. Tuy nhiên plot cũng cho thấy cross-validation error gần như same khi chỉ có one component trong model. Điều này gợi ý model với rất ít components có thể đã đủ.

CV score được cung cấp cho mọi possible number of components từ 1 đến 19. Method `PCA()` phần này nếu cố fit intercept-only với `n_components=0`, nên ta compute thêm MSE cho null model trên chính các split này.

```
In [53]: Xn = np.zeros((X.shape[0], 1))
cv_null = skm.cross_validate(linreg,
 Xn,
 Y,
 cv=kfold,
 scoring='neg_mean_squared_error')
-cv_null['test_score'].mean()
```

```
Out[53]: 204139.31
```

Attribute `explained_variance_ratio_` của `PCA` object cho percentage of variance explained trong predictors và response khi dùng different numbers of components. Concept này được bàn kỹ hơn ở Section 12.2.

```
In [54]: pipe.named_steps['pca'].explained_variance_ratio_
```

```
Out[54]: array([0.3831424 , 0.21841076])
```



Có thể hiểu đây là lượng information về predictors được capture bởi  $M$  principal components. Ví dụ  $M = 1$  chỉ capture 38.31% variance, còn  $M = 2$  capture thêm 21.84%, tổng 60.15%. Đến  $M = 6$  tăng lên 88.63%. Sau đó increment tiếp tục giảm dần, cho tới  $M = p = 19$  thì capture đủ 100% variance.

### Partial Least Squares

Partial least squares (PLS) được implement bằng function `PLSRegression()`.

*PLSRegression()*

```
In [55]: pls = PLSRegression(n_components=2,
 scale=True)
pls.fit(X, Y)
```

Giống PCR, ta muốn dùng CV để chọn number of components.

```
In [56]: param_grid = {'n_components':range(1, 20)}
grid = skm.GridSearchCV(pls,
 param_grid,
 cv=kfold,
 scoring='neg_mean_squared_error')
grid.fit(X, Y)
```

Như các method khác, ta plot MSE.

```
In [57]: pls_fig, ax = subplots(figsize=(8,8))
n_comp = param_grid['n_components']
ax.errorbar(n_comp,
 -grid.cv_results_['mean_test_score'],
 grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylabel('Cross-validated MSE', fontsize=20)
ax.set_xlabel('# principal components', fontsize=20)
ax.set_xticks(n_comp[:2])
ax.set_ylim([50000,250000]);
```

CV error được minimize tại 12, dù difference với number nhỏ hơn nhiều như 2 hoặc 3 components là rất ít.

## 6.6 Exercises

### Conceptual

1. Ta thực hiện best subset, forward stepwise và backward stepwise selection trên cùng một dataset. Với mỗi approach, ta thu được  $p + 1$  models chứa  $0, 1, 2, \dots, p$  predictors. Hãy giải thích câu trả lời:
  - (a) Trong ba model có  $k$  predictors, model nào có training RSS nhỏ nhất?
  - (b) Trong ba model có  $k$  predictors, model nào có test RSS nhỏ nhất?
  - (c) True or False:
    - i. Predictors trong  $k$ -variable model do forward stepwise xác định là subset của predictors trong  $(k + 1)$ -variable model do forward stepwise selection xác định.
    - ii. Predictors trong  $k$ -variable model do backward stepwise xác định là subset của predictors trong  $(k + 1)$ -variable model

- do backward stepwise selection xác định.
- iii. Predictors trong  $k$ -variable model do backward stepwise xác định là subset của predictors trong  $(k + 1)$ -variable model do forward stepwise selection xác định.
  - iv. Predictors trong  $k$ -variable model do forward stepwise xác định là subset của predictors trong  $(k + 1)$ -variable model do backward stepwise selection xác định.
  - v. Predictors trong  $k$ -variable model do best subset xác định là subset của predictors trong  $(k + 1)$ -variable model do best subset selection xác định.
2. Với các phần (a)–(c), hãy chỉ ra lựa chọn nào trong i.–iv. là đúng và giải thích.
- (a) Lasso, so với least squares, là:
    - i. More flexible và do đó prediction accuracy sẽ tốt hơn khi increase in bias nhỏ hơn decrease in variance.
    - ii. More flexible và do đó prediction accuracy sẽ tốt hơn khi increase in variance nhỏ hơn decrease in bias.
    - iii. Less flexible và do đó prediction accuracy sẽ tốt hơn khi increase in bias nhỏ hơn decrease in variance.
    - iv. Less flexible và do đó prediction accuracy sẽ tốt hơn khi increase in variance nhỏ hơn decrease in bias.
  - (b) Lặp lại (a) cho ridge regression relative to least squares.
  - (c) Lặp lại (a) cho non-linear methods relative to least squares.
3. Giả sử ta estimate regression coefficients trong linear regression model bằng cách minimize

$$\sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

subject to

$$\sum_{j=1}^p |\beta_j| \leq s$$

cho một value  $s$  cụ thể. Với (a)–(e), chọn đáp án đúng trong i.–v. và giải thích.

- (a) Khi tăng  $s$  từ 0, training RSS sẽ:
  - i. Ban đầu tăng rồi cuối cùng giảm theo inverted U shape.
  - ii. Ban đầu giảm rồi cuối cùng tăng theo U shape.
  - iii. Steadily increase.
  - iv. Steadily decrease.
  - v. Remain constant.
- (b) Lặp lại (a) cho test RSS.
- (c) Lặp lại (a) cho variance.
- (d) Lặp lại (a) cho (squared) bias.

(e) Lặp lại (a) cho irreducible error.

4. Giả sử ta estimate regression coefficients trong linear regression model bằng cách minimize

$$\sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

cho một value  $\lambda$  cụ thể. Với (a)–(e), chọn đáp án đúng trong i.–v. và giải thích.

(a) Khi tăng  $\lambda$  từ 0, training RSS sẽ:

- i. Ban đầu tăng rồi cuối cùng giảm theo inverted U shape.
- ii. Ban đầu giảm rồi cuối cùng tăng theo U shape.
- iii. Steadily increase.
- iv. Steadily decrease.
- v. Remain constant.

(b) Lặp lại (a) cho test RSS.

(c) Lặp lại (a) cho variance.

(d) Lặp lại (a) cho (squared) bias.

(e) Lặp lại (a) cho irreducible error.



5. Ridge regression được biết là thường cho similar coefficient values đối với correlated variables, còn lasso có thể cho coefficient values khá different đối với correlated variables. Ta khảo sát property này trong một setting rất đơn giản.

Giả sử  $n = 2$ ,  $p = 2$ ,  $x_{11} = x_{12}$ ,  $x_{21} = x_{22}$ . Ngoài ra giả sử  $y_1 + y_2 = 0$ ,  $x_{11} + x_{21} = 0$  và  $x_{12} + x_{22} = 0$ , nên estimate intercept trong least squares, ridge regression hoặc lasso model bằng zero:  $\hat{\beta}_0 = 0$ .

(a) Viết ridge regression optimization problem trong setting này.

(b) Lập luận rằng trong setting này, ridge coefficient estimates thỏa  $\hat{\beta}_1 = \hat{\beta}_2$ .

(c) Viết lasso optimization problem trong setting này.

(d) Lập luận rằng trong setting này, lasso coefficients  $\hat{\beta}_1$  và  $\hat{\beta}_2$  không unique—tức có nhiều possible solution cho optimization problem trong (c). Mô tả các solution này.

6. Ta sẽ khảo sát thêm (6.12) và (6.13).

(a) Xét (6.12) với  $p = 1$ . Với một lựa chọn  $y_1$  và  $\lambda > 0$ , plot (6.12) như function của  $\beta_1$ . Plot phải confirm rằng (6.12) được giải bởi (6.14).

(b) Xét (6.13) với  $p = 1$ . Với một lựa chọn  $y_1$  và  $\lambda > 0$ , plot (6.13) như function của  $\beta_1$ . Plot phải confirm rằng (6.13) được giải bởi (6.15).



7. Ta sẽ derive Bayesian connection với lasso và ridge regression đã bàn trong Section 6.2.2.

- (a) Giả sử  $y_i = \beta_0 + \sum_{j=1}^p x_{ij}\beta_j + \varepsilon_i$ , trong đó  $\varepsilon_1, \dots, \varepsilon_n$  independent và identically distributed theo  $N(0, \sigma^2)$ . Viết likelihood cho data.
- (b) Assume prior sau cho  $\beta$ :  $\beta_1, \dots, \beta_p$  independent và identically distributed theo double-exponential distribution với mean 0 và common scale parameter  $b$ , tức

$$p(\beta) = \frac{1}{2b} \exp(-|\beta|/b).$$

Viết posterior cho  $\beta$  trong setting này.

- (c) Lập luận rằng lasso estimate là mode của  $\beta$  dưới posterior distribution này.
- (d) Bây giờ assume prior sau cho  $\beta$ :  $\beta_1, \dots, \beta_p$  independent và identically distributed theo normal distribution với mean zero và variance  $c$ . Viết posterior cho  $\beta$  trong setting này.
- (e) Lập luận rằng ridge regression estimate vừa là mode vừa là mean của  $\beta$  dưới posterior distribution này.

### Applied

8. Trong exercise này, ta generate simulated data rồi dùng data đó để perform forward và backward stepwise selection.

- (a) Tạo random number generator và dùng method `normal()` để generate predictor  $X$  length  $n = 100$ , cùng noise vector  $\varepsilon$  length  $n = 100$ .
- (b) Generate response vector  $Y$  length  $n = 100$  theo model

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \varepsilon,$$

trong đó  $\beta_0, \beta_1, \beta_2, \beta_3$  là constants do bạn chọn.

- (c) Dùng forward stepwise selection để select model chứa predictors  $X, X^2, \dots, X^{10}$ . Model thu được theo  $C_p$  là gì? Report coefficients của model đó.
- (d) Lặp lại (c) bằng backwards stepwise selection. Câu trả lời so với (c) thế nào?
- (e) Fit lasso model cho simulated data, vẫn dùng  $X, X^2, \dots, X^{10}$  làm predictors. Dùng cross-validation để select optimal  $\lambda$ . Tạo plot cross-validation error theo  $\lambda$ . Report resulting coefficient estimates và discuss results.
- (f) Generate response vector  $Y$  theo model

$$Y = \beta_0 + \beta_7 X^7 + \varepsilon,$$

rồi perform forward stepwise selection và lasso. Discuss results.

9. Trong exercise này, ta predict number of applications received bằng các variables khác trong dataset `College`.

- (a) Split dataset thành training set và test set.
- (b) Fit linear model bằng least squares trên training set và report test error.

- (c) Fit ridge regression model trên training set, với  $\lambda$  chọn bằng cross-validation. Report test error.
  - (d) Fit lasso model trên training set, với  $\lambda$  chọn bằng cross-validation. Report test error cùng number of non-zero coefficient estimates.
  - (e) Fit PCR model trên training set, với  $M$  chọn bằng cross-validation. Report test error cùng value  $M$  được cross-validation chọn.
  - (f) Fit PLS model trên training set, với  $M$  chọn bằng cross-validation. Report test error cùng value  $M$  được cross-validation chọn.
  - (g) Comment các result thu được. Ta có thể predict number of college applications received chính xác đến mức nào? Test error từ năm approach này có khác biệt nhiều không?
10. Ta đã thấy khi number of features dùng trong model tăng, training error nhất định giảm, nhưng test error có thể không giảm. Ta khảo sát điều này trong simulated dataset.
- (a) Generate dataset có  $p = 20$  features,  $n = 1,000$  observations, cùng quantitative response vector tạo theo model
 
$$Y = X\beta + \varepsilon,$$
 trong đó  $\beta$  có một số element exactly zero.
    - (b) Split dataset thành training set gồm 100 observations và test set gồm 900 observations.
    - (c) Perform best subset selection trên training set và plot training-set MSE associated với best model ở mỗi size.
    - (d) Plot test-set MSE associated với best model ở mỗi size.
    - (e) Test-set MSE đạt minimum ở model size nào? Comment result. Nếu minimum xảy ra với intercept-only model hoặc model chứa toàn bộ features, hãy điều chỉnh cách generate data trong (a) cho tới khi có scenario mà test-set MSE minimum ở intermediate model size.
    - (f) Model tại đó test-set MSE minimum so với true model dùng generate data như thế nào? Comment coefficient values.
    - (g) Tạo plot hiển thị  $\sqrt{\sum_{j=1}^p (\beta_j - \hat{\beta}_j^r)^2}$  cho một range  $r$ , trong đó  $\hat{\beta}_j^r$  là  $j$ th coefficient estimate của best model chứa  $r$  coefficients. Comment điều quan sát được. So sánh với test-MSE plot từ (d).
11. Ta thử predict per-capita crime rate trong dataset **Boston**.
- (a) Thử một số regression method trong chapter này như best subset selection, lasso, ridge regression và PCR. Present và discuss result cho các approach được xét.
  - (b) Propose một model (hoặc set of models) có vẻ perform tốt trên dataset này và justify answer. Hãy bảo đảm evaluate model performance bằng validation-set error, cross-validation hoặc reasonable alternative khác, thay vì training error.

- (c) Model được chọn có involve toàn bộ features trong dataset không? Vì sao?

# 7

## Moving Beyond Linearity

Cho đến đây, cuốn sách chủ yếu tập trung vào linear models. Linear models tương đối đơn giản để mô tả và triển khai, đồng thời có ưu thế về interpretation và inference. Tuy nhiên, standard linear regression có thể bị hạn chế đáng kể về predictive power, bởi linearity assumption hầu như luôn chỉ là một approximation và đôi khi là approximation kém. Ở Chapter 6, ta thấy có thể cải thiện least squares bằng ridge regression, lasso, principal components regression và các technique khác. Trong setting đó, improvement đến từ việc giảm complexity của linear model và do đó giảm variance của estimates. Nhưng ta vẫn đang dùng một linear model, nên mức cải thiện cũng có giới hạn. Trong chapter này, ta nói lỏng linearity assumption nhưng vẫn cố duy trì interpretability nhiều nhất có thể. Ta sẽ xét các extension rất đơn giản của linear models như polynomial regression và step functions, cùng các approach tinh vi hơn như splines, local regression và generalized additive models.

- **Polynomial regression** mở rộng linear model bằng cách thêm các predictor tạo ra từ lũy thừa của original predictor. Chẳng hạn, cubic regression dùng ba variable  $X$ ,  $X^2$ , và  $X^3$  làm predictors. Cách này cung cấp một phương pháp đơn giản để tạo non-linear fit.
- **Step functions** chia range của một variable thành  $K$  region riêng biệt để tạo một qualitative variable; kết quả là một piecewise constant function.
- **Regression splines** flexible hơn polynomials và step functions, và thực chất mở rộng cả hai. Range của  $X$  được chia thành  $K$  region; trong mỗi region ta fit một polynomial, nhưng các polynomial được constrain để nối trơn tại boundaries, hay *knots*. Nếu chia interval thành đủ nhiều region, fit có thể cực kỳ flexible.
- **Smoothing splines** tương tự regression splines nhưng xuất hiện từ một formulation khác: minimize residual sum of squares với một smoothness penalty.
- **Local regression** giống splines ở mục tiêu tạo flexible fit, nhưng

khác ở chỗ các region có thể overlap và chuyển tiếp rất smooth.

- **Generalized additive models** mở rộng các method trên để xử lý multiple predictors.

Trong Sections 7.1–7.6, ta trình bày nhiều approach để model relationship giữa response  $Y$  và một predictor  $X$  theo cách flexible. Ở Section 7.7, ta sẽ thấy những approach này có thể được tích hợp liền mạch để model response  $Y$  như một function của nhiều predictors  $X_1, \dots, X_p$ .



## 7.1 Polynomial Regression

Theo truyền thống, cách chuẩn để mở rộng linear regression sang setting mà relationship giữa predictors và response là non-linear là thay standard linear model

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

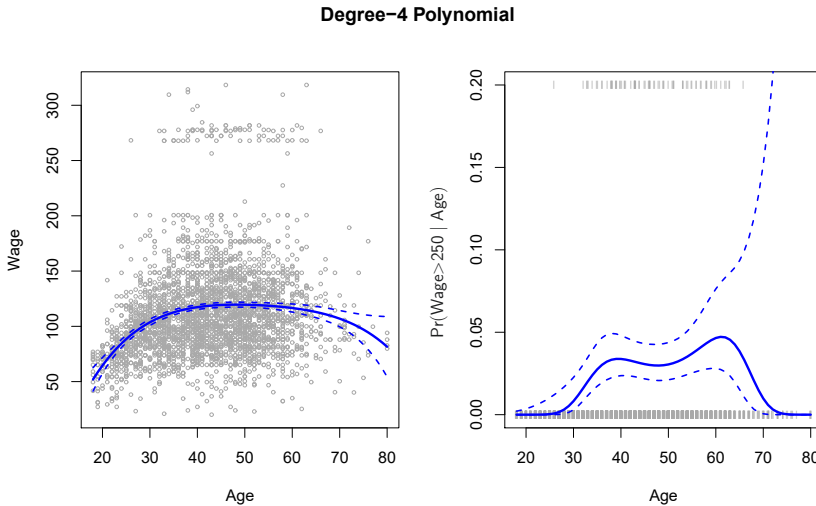
bằng một polynomial function

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \cdots + \beta_d x_i^d + \varepsilon_i. \quad (7.1)$$

Ở đây  $\varepsilon_i$  là error term. Approach này được gọi là *polynomial regression*; ta đã gặp một example ở Section 3.3.2. Với degree  $d$  đủ lớn, polynomial regression có thể tạo ra curve rất non-linear. Các coefficient trong (7.1) có thể estimate dễ dàng bằng least squares linear regression, vì đây chỉ là standard linear model với predictors  $x_i, x_i^2, x_i^3, \dots, x_i^d$ . Thông thường hiếm khi dùng  $d$  lớn hơn 3 hoặc 4, vì khi  $d$  lớn polynomial curve có thể quá flexible và mang shape rất kỳ lạ, đặc biệt gần boundary của variable  $X$ .

*polynomial  
regression*

Left-hand panel của Figure 7.1 plot **wage** theo **age** cho dataset **Wage**, chứa information về income và demographic của nam giới sống ở central Atlantic region của Hoa Kỳ. Solid blue curve là degree-4 polynomial fit bằng least squares. Dù đây vẫn là linear regression model như các model khác, individual coefficients không phải trọng tâm; thay vào đó, ta xem toàn bộ fitted function trên grid 63 value của **age** từ 18 đến 80 để hiểu relationship giữa **age** và **wage**.



**HÌNH 7.1.** Dataset *Wage*. Left: Solid blue curve là degree-4 polynomial của *wage* (nghìn đô la) theo *age*, fit bằng least squares. Dashed curves là estimated 95% confidence interval. Right: Binary event  $\text{wage} > 250$  được model bằng logistic regression, cũng với degree-4 polynomial. Fitted posterior probability rằng *wage* vượt \$250,000 được vẽ bằng blue curve cùng estimated 95% confidence interval.

Trong Figure 7.1, fit đi kèm một cặp dashed curve; đây là các curve ở mức  $2 \times$  standard error. Giả sử ta đã tính fit tại một value cụ thể của *age*,

$x_0$ :

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \hat{\beta}_2 x_0^2 + \hat{\beta}_3 x_0^3 + \hat{\beta}_4 x_0^4. \quad (7.2)$$

Ta muốn biết variance của fit, tức  $\text{Var}\{\hat{f}(x_0)\}$ . Least squares trả về variance estimate cho từng fitted coefficient  $\hat{\beta}_j$ , cùng covariance giữa từng pair coefficient estimate. Từ đó ta có thể tính estimated variance của  $\hat{f}(x_0)$ .<sup>1</sup> Estimated pointwise standard error của  $\hat{f}(x_0)$  là square root của variance này. Ta lặp calculation tại từng reference point  $x_0$ , rồi plot fitted curve cùng hai lần standard error ở hai phía. Với normally distributed error terms, khoảng  $\pm 2$  standard errors xấp xỉ 95% confidence interval.

Figure 7.1 gợi ý wages có thể đến từ hai population: high earners trên \$250,000 mỗi năm và low earners. Ta có thể biến **wage** thành binary variable bằng cách chia thành hai group này, rồi dùng logistic regression để predict binary response bằng polynomial functions của age. Cụ thể, ta fit

$$\Pr(y_i > 250 \mid x_i) = \frac{\exp(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d)}{1 + \exp(\beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d)}. \quad (7.3)$$

Kết quả nằm ở right-hand panel của Figure 7.1. Gray marks ở top và bottom cho biết ages của high earners và low earners. Solid blue curve biểu diễn fitted probability trở thành high earner theo age, cùng estimated 95% confidence interval. Confidence intervals khá rộng, nhất là phía bên phải. Mặc dù sample size của dataset lớn ( $n = 3,000$ ), chỉ có 79 high earners, dẫn tới variance cao của estimated coefficients và do đó confidence intervals rộng.

## 7.2 Step Functions

Dùng polynomial functions của features làm predictors trong linear model áp đặt một global structure lên non-linear function của  $X$ . Ta có thể tránh global structure này bằng *step functions*: chia range của  $X$  thành các bin rồi fit một constant khác nhau trong mỗi bin. Điều này tương đương chuyển một continuous variable thành ordered categorical variable.

Cụ thể hơn, ta tạo cutpoints  $c_1, c_2, \dots, c_K$  trong range của  $X$ , rồi tạo  $K + 1$  variable mới

$$\begin{aligned} C_0(X) &= I(X < c_1), \\ C_1(X) &= I(c_1 \leq X < c_2), \\ C_2(X) &= I(c_2 \leq X < c_3), \\ &\vdots \\ C_{K-1}(X) &= I(c_{K-1} \leq X < c_K), \\ C_K(X) &= I(c_K \leq X). \end{aligned} \quad (7.4)$$

Ở đây  $I(\cdot)$  là *indicator function*, trả về 1 nếu condition đúng và 0 nếu không. Chẳng hạn  $I(c_K \leq X) = 1$  nếu  $c_K \leq X$ . Những variable này đôi khi gọi là dummy variables. Với mọi value của  $X$ ,  $C_0(X) + C_1(X) + \cdots + C_K(X) = 1$ , vì  $X$  phải nằm đúng trong một trong  $K + 1$  intervals.

<sup>1</sup>Nếu  $\tilde{C}$  là covariance matrix  $5 \times 5$  của các  $\hat{\beta}_j$ , và  $\ell_0^T = (1, x_0, x_0^2, x_0^3, x_0^4)$ , thì  $\text{Var}[\hat{f}(x_0)] = \ell_0^T \tilde{C} \ell_0$ .

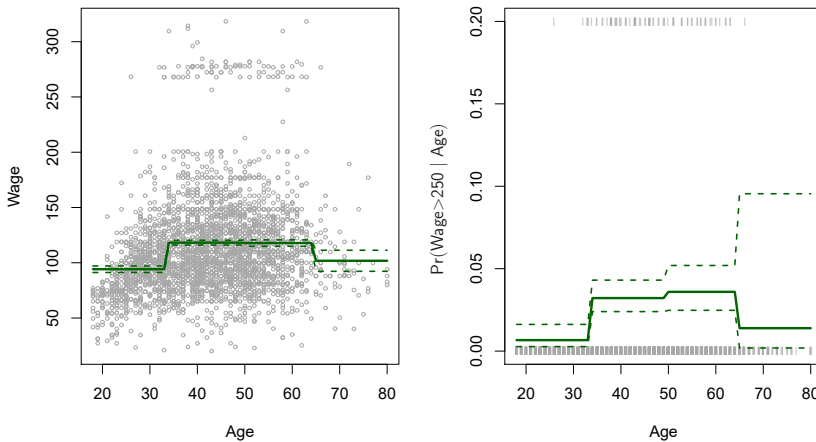
step function  
ordered categorical  
variable

indicator function

Sau đó ta dùng least squares để fit linear model với các predictors  $C_1(X), \dots, C_K(X)$ :

$$y_i = \beta_0 + \beta_1 C_1(x_i) + \beta_2 C_2(x_i) + \dots + \beta_K C_K(x_i) + \varepsilon_i. \quad (7.5)$$

<sup>2</sup> Với một value của  $X$ , nhiều nhất chỉ một trong  $C_1, \dots, C_K$  khác zero. Khi  $X < c_1$ , toàn bộ predictors trong (7.5) đều zero, nên  $\beta_0$  có thể được interpret là mean của  $Y$  cho  $X < c_1$ . Còn với  $c_j \leq X < c_{j+1}$ , model predict  $\beta_0 + \beta_j$ , nên  $\beta_j$  là average increase của response trong interval đó so với  $X < c_1$ .



**HÌNH 7.2.** Dataset *Wage*. Left: Solid curve là fitted value từ least squares regression của *wage* (nghìn đô la) dùng step functions của *age*; dashed curves là estimated 95% confidence interval. Right: Binary event  $\text{wage} > 250$  được model bằng logistic regression dùng step functions của *age*; fitted posterior probability rằng *wage* vượt \$250,000 được hiển thị cùng estimated 95% confidence interval.

Left-hand panel Figure 7.2 minh họa việc fit step functions vào *Wage*. Ta cũng fit logistic regression model

$$\Pr(y_i > 250 \mid x_i) = \frac{\exp\{\beta_0 + \beta_1 C_1(x_i) + \dots + \beta_K C_K(x_i)\}}{1 + \exp\{\beta_0 + \beta_1 C_1(x_i) + \dots + \beta_K C_K(x_i)\}} \quad (7.6)$$

để predict probability một individual là high earner dựa trên age. Right-hand panel Figure 7.2 hiển thị fitted posterior probabilities từ approach này.

Nếu predictors không có natural breakpoints, piecewise-constant functions có thể bỏ lỡ structure. Chẳng hạn ở left-hand panel Figure 7.2, bin đầu tiên rõ ràng không capture increasing trend của wage theo age. Dù vậy, step function approach rất phổ biến trong biostatistics, epidemiology và nhiều discipline khác; ví dụ người ta thường dùng age group 5 năm để định nghĩa bins.

<sup>2</sup>Ta loại  $C_0(X)$  khỏi predictors trong (7.5) vì nó redundant với intercept. Điều này tương tự việc chỉ cần hai dummy variable để code một qualitative variable có ba level khi model có intercept. Việc loại  $C_0(X)$  thay vì một  $C_k(X)$  khác là arbitrary. Hoặc có thể giữ  $C_0(X), \dots, C_K(X)$  và bỏ intercept.

## 7.3 Basis Functions

Polynomial và piecewise-constant regression models thực ra là special cases của *basis function* approach. Ý tưởng là có sẵn một family function hoặc transformation áp dụng cho variable  $X$ :  $b_1(X), b_2(X), \dots, b_K(X)$ . Thay vì fit linear model trực tiếp theo  $X$ , ta fit

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \beta_3 b_3(x_i) + \dots + \beta_K b_K(x_i) + \varepsilon_i. \quad (7.7)$$

Các basis function  $b_1(\cdot), \dots, b_K(\cdot)$  là fixed và known, tức được chọn trước. Với polynomial regression,  $b_j(x_i) = x_i^j$ ; với piecewise constant function,  $b_j(x_i) = I(c_j \leq x_i < c_{j+1})$ . Ta có thể xem (7.7) như standard linear model với predictors  $b_1(x_i), \dots, b_K(x_i)$ , nên dùng least squares để estimate unknown regression coefficients. Quan trọng là toàn bộ inference tool cho linear models ở Chapter 3, như standard errors của coefficient estimates và F-statistics cho overall significance của model, vẫn dùng được.

Cho đến đây ta dùng polynomial functions và piecewise constant functions làm basis functions, nhưng còn nhiều lựa chọn khác, như wavelets hay Fourier series. Section tiếp theo xét một lựa chọn rất phổ biến: regression splines.

## 7.4 Regression Splines

Bây giờ ta xét một class basis function flexible hơn, mở rộng polynomial regression và piecewise constant regression.

### 7.4.1 Piecewise Polynomials

Thay vì fit một high-degree polynomial trên toàn range của  $X$ , *piecewise polynomial regression* fit các low-degree polynomial riêng trên những region khác nhau của  $X$ . Chẳng hạn, piecewise cubic polynomial dùng cubic regression

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \varepsilon_i, \quad (7.8)$$

nhưng coefficients  $\beta_0, \beta_1, \beta_2, \beta_3$  khác nhau ở các phần khác nhau của range  $X$ . Các point nơi coefficients thay đổi được gọi là *knots*.

Ví dụ, piecewise cubic không có knot chỉ là standard cubic polynomial, tức (7.1) với  $d = 3$ . Với một knot tại  $c$ , model có dạng

$$y_i = \begin{cases} \beta_{01} + \beta_{11}x_i + \beta_{21}x_i^2 + \beta_{31}x_i^3 + \varepsilon_i, & x_i < c, \\ \beta_{02} + \beta_{12}x_i + \beta_{22}x_i^2 + \beta_{32}x_i^3 + \varepsilon_i, & x_i \geq c. \end{cases}$$

Nói cách khác, ta fit hai polynomial function khác nhau: một cho observations có  $x_i < c$ , và một cho observations có  $x_i \geq c$ . Mỗi function có thể fit bằng least squares trên simple functions của original predictor.

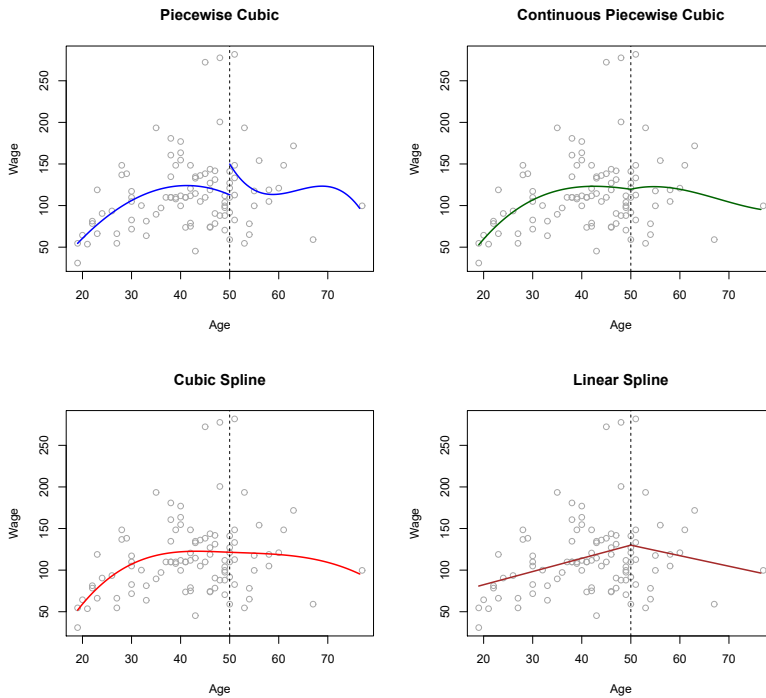
Dùng nhiều knot hơn tạo piecewise polynomial flexible hơn. Nếu đặt  $K$  knots trong range của  $X$ , ta sẽ fit  $K + 1$  cubic polynomial khác nhau. Không bắt buộc phải dùng cubic: có thể dùng piecewise linear function, và piecewise constant functions ở Section 7.2 chính là degree-0 piecewise polynomials.

Top-left panel Figure 7.3 fit một piecewise cubic vào subset của *Wage*, với knot duy nhất tại  $\text{age}=50$ . Function bị discontinuous và trông không hợp

*basis function*

*regression spline*

*piecewise  
polynomial  
regression  
knot*



**HÌNH 7.3.** Các piecewise polynomial khác nhau được fit vào một subset của *wage*, với một knot tại *age*=50. Top Left: Cubic polynomials không bị constrain. Top Right: Cubic polynomials được constrain continuous tại *age*=50. Bottom Left: Cubic polynomials được constrain continuous và có continuous first/second derivatives. Bottom Right: Linear spline được constrain continuous.

degrees of freedom

lý. Vì mỗi polynomial có bốn parameter, piecewise polynomial này dùng tổng cộng tám degrees of freedom.

### 7.4.2 Constraints and Splines

Top-left panel Figure 7.3 quá flexible. Để khắc phục, ta có thể fit piecewise polynomial dưới constraint rằng fitted curve phải continuous, tức không có jump tại age=50. Top-right panel cho fit này; nó tốt hơn nhưng V-shaped join vẫn không tự nhiên.

Ở lower-left panel, ta thêm hai constraint nữa: first và second derivatives của piecewise polynomials cũng continuous tại age=50. Như vậy function không chỉ continuous mà còn rất smooth. Mỗi constraint áp lên piecewise cubic polynomial thực chất giải phóng một degree of freedom bằng cách giảm complexity. Top-left dùng tám degrees of freedom; lower-left áp ba constraint (continuity, continuity của first derivative, continuity của second derivative), còn năm degrees of freedom. Curve này gọi là *cubic spline*. Nói chung, cubic spline với  $K$  knots dùng  $4 + K$  degrees of freedom.<sup>3</sup>

cubic spline  
derivative

Lower-right panel Figure 7.3 là *linear spline*, continuous tại age=50. Định nghĩa tổng quát: degree- $d$  spline là piecewise degree- $d$  polynomial, continuous ở các derivative đến degree  $d - 1$  tại mỗi knot. Vì vậy linear spline được tạo bằng cách fit một line trong mỗi region do knots định nghĩa, với continuity tại từng knot. Ta có thể thêm nhiều knots hơn và impose continuity tại mỗi knot.

linear spline

### 7.4.3 The Spline Basis Representation

Regression splines ở section trước có vẻ phức tạp: làm sao fit piecewise degree- $d$  polynomial dưới constraint rằng function (và có thể cả  $d - 1$  derivative đầu) continuous? Ta có thể dùng basis model (7.7). Một cubic spline với  $K$  knots có thể được model như

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_{K+3} b_{K+3}(x_i) + \varepsilon_i, \quad (7.9)$$

cho một lựa chọn thích hợp của basis functions  $b_1, \dots, b_{K+3}$ ; model này fit bằng least squares.

Có nhiều representation tương đương của cubic spline. Cách trực tiếp nhất là bắt đầu với basis của cubic polynomial,  $x, x^2, x^3$ , rồi thêm một *truncated power basis function* cho mỗi knot.

truncated power  
basis

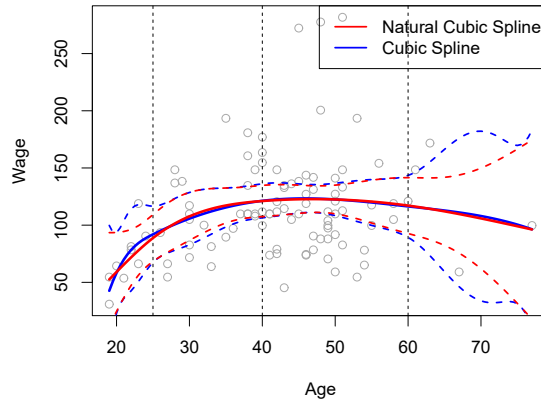
Truncated power basis function được định nghĩa

$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3, & x > \xi, \\ 0, & \text{otherwise.} \end{cases} \quad (7.10)$$

Ở đây  $\xi$  là knot. Có thể chứng minh rằng thêm term  $\beta_4 h(x, \xi)$  vào cubic polynomial model (7.8) chỉ tạo discontinuity ở third derivative tại  $\xi$ ; function vẫn continuous và có continuous first/second derivatives tại knot.

Để fit cubic spline vào dataset có  $K$  knots, ta thực hiện least squares regression với intercept và  $3 + K$  predictors:  $X, X^2, X^3, h(X, \xi_1), \dots, h(X, \xi_K)$ . Như vậy cần estimate tổng cộng  $K + 4$  regression coefficients, tức cubic spline có  $K + 4$  degrees of freedom.

<sup>3</sup>Cubic splines phổ biến vì mắt người thường không nhận ra discontinuity ở các knots.



**HÌNH 7.4.** Một cubic spline và một natural cubic spline với ba knots được fit vào subset của dataset *Wage*. Dashed lines đánh dấu knot locations.

Splines có thể có variance cao ở outer range của predictors, nơi  $X$  rất nhỏ hoặc rất lớn. Figure 7.4 cho fit trên *Wage* với ba knots; confidence bands ở boundary khá unstable. *Natural spline* là regression spline có thêm boundary constraints: function bắt buộc linear ở boundary region, tức nơi  $X$  nhỏ hơn smallest knot hoặc lớn hơn largest knot. Constraint này thường tạo estimates ổn định hơn ở boundaries. Figure 7.4 cũng hiển thị natural cubic spline bằng red line, với confidence intervals hẹp hơn.

*natural spline*

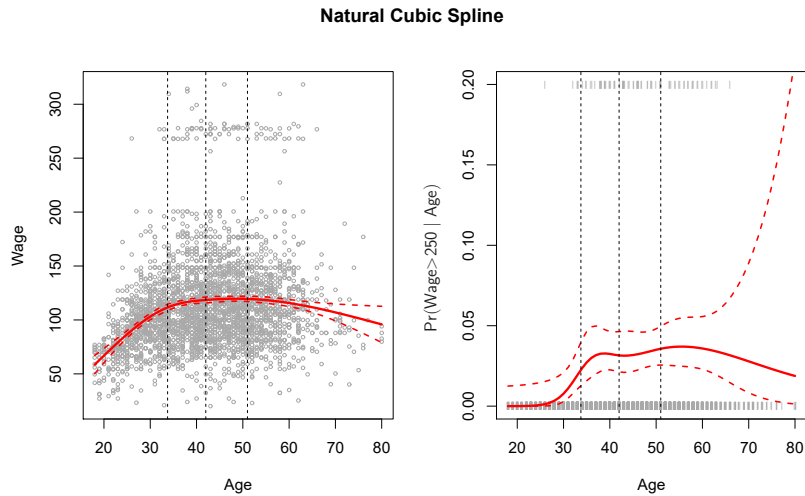
#### 7.4.4 Choosing the Number and Locations of the Knots

Khi fit spline, nên đặt knots ở đâu? Regression spline flexible nhất ở region có nhiều knot, vì polynomial coefficients có thể thay đổi nhanh. Có thể đặt nhiều knots ở nơi ta tin function thay đổi nhanh và ít knots ở nơi ổn định hơn. Tuy nhiên trong practice, thường đặt knots một cách uniform; một cách là chỉ định desired degrees of freedom rồi để software tự đặt số knot tương ứng tại uniform quantiles của data.

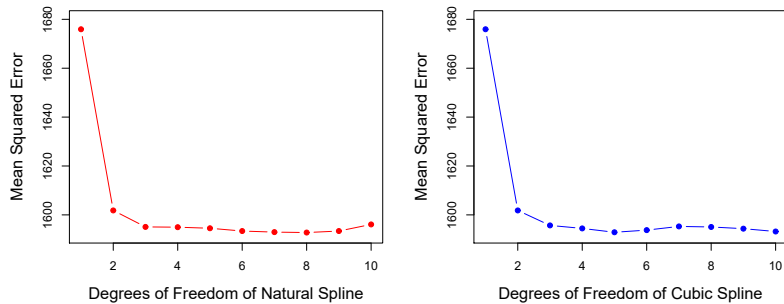
Figure 7.5 là example trên *Wage*. Tương tự Figure 7.4, ta fit natural cubic spline với ba knots, nhưng knot locations được chọn tự động là 25th, 50th và 75th percentiles của age. Điều này được request bằng bốn degrees of freedom. Lý do bốn degrees of freedom dẫn tới ba interior knots hơi technical.<sup>4</sup>

Ta nên dùng bao nhiêu knots, hay equivalently spline nên có bao nhiêu degrees of freedom? Có thể thử nhiều number of knots và nhìn curve, nhưng objective hơn là dùng cross-validation như Chapters 5 và 6. Ta bỏ ra một phần data (ví dụ 10%), fit spline với một number of knots nhất định trên phần còn lại, predict held-out portion, rồi lặp để mỗi observation được held out một lần và tính overall cross-validated RSS. Procedure được lặp cho các  $K$  khác nhau và chọn  $K$  có RSS nhỏ nhất.

<sup>4</sup>Thực tế có năm knots nếu tính cả hai boundary knots. Cubic spline với năm knots có chín degrees of freedom. Natural cubic spline có thêm hai natural constraints ở mỗi boundary để enforce linearity, nên còn  $9 - 4 = 5$  degrees of freedom. Vì một constant được absorbed vào intercept, ta đếm là bốn degrees of freedom.



**HÌNH 7.5.** Một natural cubic spline với bốn degrees of freedom được fit vào dataset *Wage*. Left: Spline fit *wage* (nghìn đô la) theo *age*. Right: Logistic regression model binary event *wage* > 250 theo *age*; fitted posterior probability rằng *wage* vượt \$250,000 được hiển thị. Dashed lines là knot locations.



**HÌNH 7.6.** Ten-fold cross-validated mean squared errors để chọn degrees of freedom khi fit splines vào dataset *Wage*; response là *wage*, predictor là *age*. Left: Natural cubic spline. Right: Cubic spline.

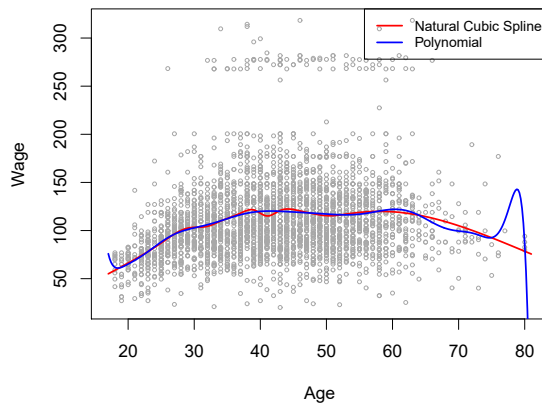


Figure 7.6 cho ten-fold cross-validated MSE của splines với nhiều degrees of freedom. Natural cubic spline và cubic spline cho result gần như identical, với evidence rõ ràng one-degree fit (linear regression) là không đủ. Hai curve flatten rất nhanh; khoảng ba degrees of freedom cho natural spline và bốn cho cubic spline có vẻ đủ.

Ở Section 7.7 ta sẽ fit additive spline models đồng thời trên nhiều variable. Việc này có thể đòi chọn degrees of freedom cho từng variable; trong case như vậy, ta thường pragmatic hơn và đặt cùng một fixed number, chẳng hạn bốn, cho mọi term.

#### 7.4.5 Comparison to Polynomial Regression

Figure 7.7 so sánh natural cubic spline với 15 degrees of freedom và degree-15 polynomial trên *Wage*. Extra flexibility của polynomial tạo undesirable behavior ở boundaries, trong khi natural cubic spline vẫn reasonable. Regression splines thường cho result tốt hơn polynomial regression vì polynomial muốn flexible phải tăng degree rất cao (ví dụ term  $X^{15}$ ), còn splines tăng flexibility bằng cách tăng number of knots nhưng giữ degree cố định. Cách này thường cho estimate ổn định hơn. Splines còn cho phép đặt nhiều knots hơn ở region nơi  $f$  thay đổi nhanh và ít hơn ở nơi  $f$  ổn định.



**HÌNH 7.7.** Trên dataset *Wage*, natural cubic spline với 15 degrees of freedom được so sánh với degree-15 polynomial. Polynomials có thể có behavior rất mạnh, đặc biệt ở tails.

## 7.5 Smoothing Splines

Ở section trước ta tạo regression splines bằng cách chỉ định một set knots, sinh sequence basis functions, rồi dùng least squares để estimate spline coefficients. Bây giờ ta giới thiệu một approach khác cũng tạo ra spline.

### 7.5.1 An Overview of Smoothing Splines

Khi fit một smooth curve vào data, điều ta muốn là tìm function  $g(x)$  fit observed data tốt, tức  $\text{RSS} = \sum_{i=1}^n (y_i - g(x_i))^2$  nhỏ. Nhưng nếu không impose constraint nào lên  $g$ , ta luôn có thể làm RSS bằng zero bằng cách

chọn  $g$  interpolate toàn bộ  $y_i$ . Function như vậy sẽ overfit nặng vì quá flexible. Ta cần một  $g$  vừa làm RSS nhỏ vừa smooth.

Một cách tự nhiên là tìm function  $g$  minimize

$$\sum_{i=1}^n \{y_i - g(x_i)\}^2 + \lambda \int g''(t)^2 dt, \quad (7.11)$$

trong đó  $\lambda \geq 0$  là tuning parameter. Function  $g$  minimize (7.11) được gọi là *smoothing spline*.

*smoothing spline  
loss function*

Equation (7.11) có “Loss+Penalty” formulation giống ridge regression và lasso ở Chapter 6. Term  $\sum_i (y_i - g(x_i))^2$  là loss function khuyến khích  $g$  fit data tốt, còn  $\lambda \int g''(t)^2 dt$  là penalty chống variability của  $g$ .  $g''(t)$  là second derivative; first derivative  $g'(t)$  đo slope, còn second derivative đo mức slope thay đổi. Vì vậy  $|g''(t)|$  lớn khi  $g$  rất wiggly quanh  $t$  và gần zero khi function smooth. Integral có thể hiểu như sum trên toàn range của  $t$ , nên  $\int g''(t)^2 dt$  đo tổng roughness của  $g$ . Nếu  $g$  smooth, quantity này nhỏ; nếu  $g$  jumpy và variable, quantity này lớn. Do đó penalty khuyến khích smoothness, và  $\lambda$  càng lớn thì  $g$  càng smooth.

Khi  $\lambda = 0$ , penalty không có effect và  $g$  sẽ rất jumpy, interpolate chính xác training observations. Khi  $\lambda \rightarrow \infty$ ,  $g$  hoàn toàn smooth: nó trở thành straight line fit training points tốt nhất, tức linear least squares line. Với intermediate  $\lambda$ ,  $g$  approximate training observations nhưng vẫn smooth. Như vậy  $\lambda$  điều khiển bias-variance trade-off của smoothing spline.

Function  $g(x)$  minimize (7.11) có special properties: nó là piecewise cubic polynomial với knots tại unique values  $x_1, \dots, x_n$ , có continuous first/second derivatives ở mỗi knot, và linear ngoài extreme knots. Nói cách khác, solution là natural cubic spline với knots tại  $x_1, \dots, x_n$ . Tuy nhiên nó không giống natural cubic spline nhận được nếu dùng basis-function approach ở Section 7.4.3 với knot tại mọi data point; đây là một shrunken version, trong đó  $\lambda$  điều khiển mức shrinkage.

### 7.5.2 Choosing the Smoothing Parameter $\lambda$

Smoothing spline là natural cubic spline có knot tại mọi unique  $x_i$ . Thoạt nhìn có vẻ nó sẽ có quá nhiều degrees of freedom, nhưng  $\lambda$  kiểm soát roughness và do đó *effective degrees of freedom*. Có thể chứng minh khi  $\lambda$  tăng từ 0 tới  $\infty$ , effective degrees of freedom, ký hiệu  $df_\lambda$ , giảm từ  $n$  xuống 2.

*effective degrees of  
freedom*

Tại sao lại nói effective degrees of freedom thay vì degrees of freedom? Thông thường degrees of freedom là số free parameters, như số coefficient trong polynomial hoặc cubic spline. Dù smoothing spline có  $n$  parameters và  $n$  nominal degrees of freedom, các parameter này bị constrain/shrink mạnh. Vì vậy  $df_\lambda$  là measure của flexibility: càng cao thì spline càng flexible, bias thấp hơn nhưng variance cao hơn.

Định nghĩa effective degrees of freedom hơi technical. Ta có thể viết

$$\hat{\mathbf{g}}_\lambda = S_\lambda \mathbf{y}, \quad (7.12)$$

trong đó  $\hat{\mathbf{g}}_\lambda$  là solution của (7.11) với một  $\lambda$  cụ thể, tức  $n$ -vector chứa fitted

values tại training points  $x_1, \dots, x_n$ . Matrix  $S_\lambda$  là  $n \times n$ . Khi đó

$$\text{df}_\lambda = \sum_{i=1}^n \{S_\lambda\}_{ii}, \quad (7.13)$$

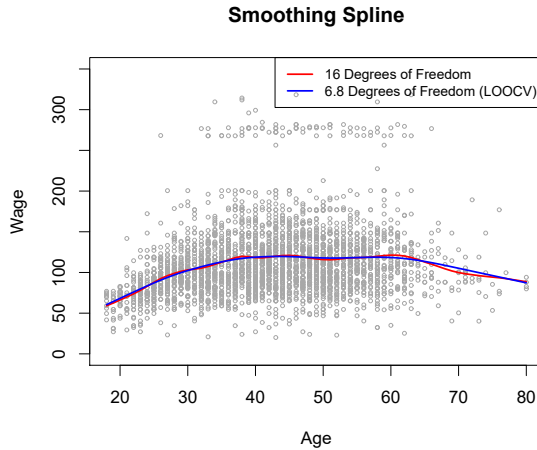
tức sum của diagonal elements của  $S_\lambda$ .

Khi fit smoothing spline, ta không cần chọn number/location of knots; knot đã nằm ở mọi training observation. Thay vào đó phải chọn  $\lambda$ . Cross-validation là một solution tự nhiên: tìm  $\lambda$  làm cross-validated RSS nhỏ nhất. Với smoothing splines, LOOCV error có thể compute rất efficiently, gần như cùng cost với một fit duy nhất:

$$\text{RSS}_{\text{CV}}(\lambda) = \sum_{i=1}^n \{y_i - \hat{g}_\lambda^{(-i)}(x_i)\}^2 = \sum_{i=1}^n \left\{ \frac{y_i - \hat{g}_\lambda(x_i)}{1 - \{S_\lambda\}_{ii}} \right\}^2.$$

Notation  $\hat{g}_\lambda^{(-i)}(x_i)$  là fitted value tại  $x_i$  khi fit dùng mọi training observation trừ observation thứ  $i$ ; còn  $\hat{g}_\lambda(x_i)$  dùng toàn bộ training observations. Formula này cho phép tính từng leave-one-out fit chỉ từ original fit  $\hat{g}_\lambda$ .<sup>5</sup> Ta đã gặp formula tương tự (5.2) cho least squares linear regression; nhờ đó LOOCV cũng có thể thực hiện nhanh cho regression splines và arbitrary basis functions fit bằng least squares.

Figure 7.8 cho smoothing-spline fits trên *Wage*. Red curve là fit khi pre-specify 16 effective degrees of freedom. Blue curve dùng  $\lambda$  chọn bằng LOOCV, tương ứng 6.8 effective degrees of freedom theo (7.13). Hai fit gần như không khác nhau, ngoài việc 16-df curve wiggly hơn một chút. Khi performance tương tự, fit 6.8 degrees of freedom preferable vì model đơn giản hơn nếu data không cung cấp evidence cho complexity thêm.



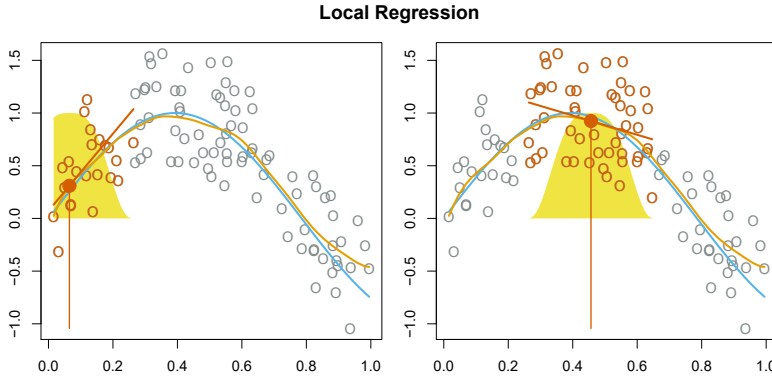
**HÌNH 7.8.** Smoothing-spline fits cho dataset *Wage*. Red curve dùng 16 effective degrees of freedom. Với blue curve,  $\lambda$  được chọn tự động bằng leave-one-out cross-validation, cho 6.8 effective degrees of freedom.

<sup>5</sup>Exact formulas để compute  $\hat{g}(x_i)$  và  $S_\lambda$  khá technical; tuy nhiên có efficient algorithms để compute các quantity này.

## 7.6 Local Regression

*Local regression* là approach khác để fit flexible non-linear functions: tại target point  $x_0$ , fit chỉ dùng nearby training observations. Figure 7.9 minh họa trên simulated data với một target point gần 0.4 và một point gần boundary 0.05. Blue line là true function  $f(x)$  dùng để generate data; light orange line là local-regression estimate  $\hat{f}(x)$ . Procedure được tóm tắt trong Algorithm 7.1.

local regression



**HÌNH 7.9.** Local regression trên simulated data. Blue curve là  $f(x)$  dùng để generate data, light orange curve là estimate  $\hat{f}(x)$ . Orange points là observations local quanh target  $x_0$  (orange vertical line). Yellow bell-shape biểu diễn weights, giảm về zero khi distance tới target tăng.  $\hat{f}(x_0)$  được obtain bằng weighted linear regression (orange line segment) rồi lấy fitted value tại  $x_0$  (orange solid point).

Ở Step 3 của Algorithm 7.1, weights  $K_{i0}$  khác nhau cho mỗi  $x_0$ . Do đó mỗi khi cần local-regression fit tại point mới, ta phải fit một weighted least squares regression mới bằng cách minimize (7.14) với set weights mới. Local regression đôi khi gọi là memory-based procedure vì, giống nearest-neighbors, ta cần toàn bộ training data mỗi lần compute prediction. Ta sẽ không đi sâu technical details ở đây.

Có nhiều choice khi perform local regression: weighting function  $K$  được define thế nào, và Step 3 fit linear, constant hay quadratic regression. Các choice đều có effect, nhưng quan trọng nhất là *span*  $s$ , proportion của points dùng để compute local regression tại  $x_0$ . Span đóng vai trò tương tự  $\lambda$  trong smoothing splines:  $s$  nhỏ tạo fit local và wiggly hơn;  $s$  rất lớn tạo global fit dùng gần như mọi training observations. Có thể chọn  $s$  bằng cross-validation hoặc specify trực tiếp. Figure 7.10 dùng  $s = 0.7$  và  $0.2$  trên Wage; đúng như kỳ vọng,  $s = 0.7$  smooth hơn.

Local regression có thể generalize theo nhiều cách. Với multiple features  $X_1, \dots, X_p$ , một generalization hữu ích là fit multiple linear regression global theo một số variable nhưng local theo variable khác, chẳng hạn time. Những *varying coefficient models* như vậy thích hợp để adapt model theo data mới thu thập. Local regression cũng mở rộng tự nhiên sang setting local theo pair  $X_1, X_2$ , bằng two-dimensional neighborhoods và bivariate linear regressions. Về lý thuyết có thể lên dimension cao hơn, nhưng lo-

cal regression thường perform kém khi  $p$  lớn hơn khoảng 3 hoặc 4 vì rất ít training observations ở gần  $x_0$ . Nearest-neighbors regression cũng gặp problem tương tự ở high dimensions.

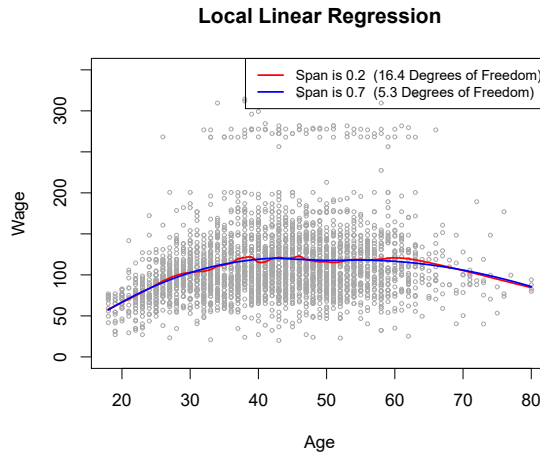
varying coefficient  
model

**Algorithm 7.1 Local Regression At  $X = x_0$**

1. Lấy fraction  $s = k/n$  training points có  $x_i$  gần  $x_0$  nhất.
2. Gán weight  $K_{i0} = K(x_i, x_0)$  cho mỗi point trong neighborhood, sao cho point xa  $x_0$  nhất có weight zero và point gần nhất có weight lớn nhất. Mọi point ngoài  $k$  nearest neighbors nhận weight zero.
3. Fit weighted least squares regression của  $y_i$  theo  $x_i$  bằng các weight trên, tìm  $\hat{\beta}_0, \hat{\beta}_1$  minimize

$$\sum_{i=1}^n K_{i0} (y_i - \beta_0 - \beta_1 x_i)^2. \quad (7.14)$$

4. Fitted value tại  $x_0$  là  $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$ .



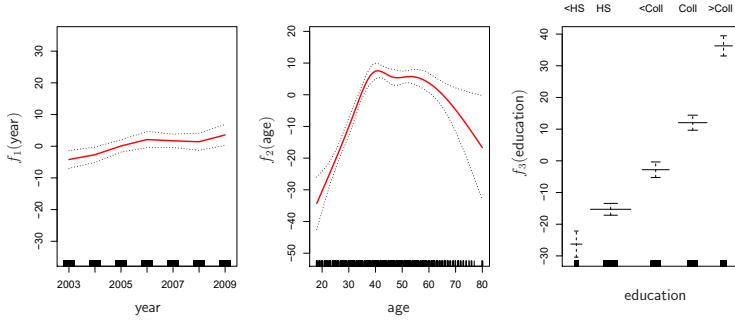
**HÌNH 7.10.** Local linear fits trên dataset *Wage*. Span chỉ fraction của data dùng để compute fit tại mỗi target point.

## 7.7 Generalized Additive Models

Sections 7.1–7.6 trình bày nhiều approach để flexibly predict response  $Y$  từ một predictor  $X$ , có thể xem là extensions của simple linear regression. Bây giờ ta xét việc flexibly predict  $Y$  từ nhiều predictors  $X_1, \dots, X_p$ , tức extension của multiple linear regression.

*Generalized additive models* (GAMs) cung cấp framework tổng quát để mở rộng standard linear model bằng cách cho phép non-linear function của từng variable nhưng vẫn giữ *additivity*. Giống linear models, GAMs dùng được với quantitative lẫn qualitative response. Trước hết ta xét quantitative response rồi đến qualitative response.

generalized additive  
model  
additivity



**HÌNH 7.11.** Với dataset *Wage*, relationship giữa từng feature và response *wage* trong fitted model (7.16). Mỗi panel hiển thị fitted function và pointwise standard errors. Hai function đầu là natural splines theo *year* và *age*, lần lượt 4 và 5 degrees of freedom. Function thứ ba là step function fit cho qualitative variable *education*.

### 7.7.1 GAMs for Regression Problems

Một cách tự nhiên để mở rộng multiple linear regression model

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} + \varepsilon_i$$

nhằm allow non-linear relationship giữa từng feature và response là thay mỗi linear component  $\beta_j x_{ij}$  bằng một smooth non-linear function  $f_j(x_{ij})$ :

$$\begin{aligned} y_i &= \beta_0 + \sum_{j=1}^p f_j(x_{ij}) + \varepsilon_i \\ &= \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \varepsilon_i. \end{aligned} \quad (7.15)$$

Đây là một GAM. Nó gọi là additive model vì ta estimate riêng  $f_j$  cho từng  $X_j$  rồi cộng contributions lại.

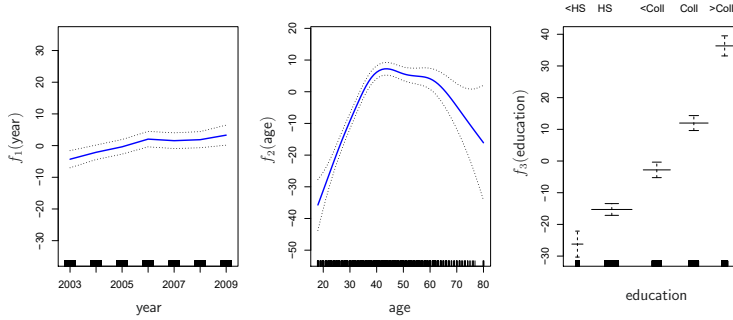
Các method fit single-variable function ở Sections 7.1–7.6 trở thành building blocks cho additive model. Ví dụ, xét model

$$\text{wage} = \beta_0 + f_1(\text{year}) + f_2(\text{age}) + f_3(\text{education}) + \varepsilon \quad (7.16)$$

trên *Wage*. *year* và *age* là quantitative, còn *education* là qualitative với năm level: <HS, HS, <Coll, Coll, >Coll. Ta fit hai function đầu bằng natural splines; function thứ ba bằng một constant riêng cho mỗi level, thông qua dummy-variable approach.

Figure 7.11 cho result của fitting (7.16) bằng least squares. Việc này đơn giản vì natural splines có thể construct bằng appropriate basis functions; toàn model chỉ là một regression lớn trên spline-basis variables và dummy variables trong một regression matrix.

Figure 7.11 dễ interpret. Left panel cho thấy khi giữ *age* và *education* fixed, *wage* tăng nhẹ theo *year*, có thể do inflation. Center panel cho thấy giữ *education* và *year* fixed, *wage* cao nhất ở intermediate ages và thấp ở rất trẻ/rất già. Right panel cho thấy giữ *year* và *age* fixed, *wage* tăng theo *education*; người học nhiều hơn có average salary cao hơn. Các finding đều intuitive.



**HÌNH 7.12.** Như Figure 7.11, nhưng  $f_1$  và  $f_2$  lần này là smoothing splines với lần lượt bốn và năm degrees of freedom.

Figure 7.12 cho ba plot tương tự, nhưng  $f_1$  và  $f_2$  là smoothing splines với lần lượt bốn và năm degrees of freedom. Fit GAM với smoothing spline không đơn giản như natural spline vì least squares không dùng trực tiếp được. Standard software như Python package `pygam` có thể fit GAM smoothing splines bằng approach gọi là *backfitting*: lặp lại việc update fit cho từng predictor trong khi giữ các predictor khác fixed. Mỗi lần update một function, ta chỉ apply fitting method cho variable đó lên một partial residual.<sup>6</sup>

*pygam  
backfitting*

Fitted functions trong Figures 7.11 và 7.12 khá giống nhau. Trong đa số setting, difference giữa GAM dùng smoothing splines và natural splines là nhỏ.

Ta không bắt buộc dùng splines làm building blocks của GAMs: có thể dùng local regression, polynomial regression hoặc bất kỳ combination nào của các approach trước đó. GAMs sẽ được investigate thêm trong lab cuối chapter.

### Pros and Cons of GAMs

Trước khi tiếp tục, ta tóm tắt advantage và limitation của GAM.

- GAMs cho phép fit non-linear  $f_j$  cho từng  $X_j$ , tự động model non-linear relationships mà standard linear regression bỏ lỡ; không cần thử thủ công nhiều transformation cho từng variable.
- Non-linear fits có thể tạo prediction chính xác hơn cho response  $Y$ .
- Vì model additive, ta có thể examine effect của từng  $X_j$  lên  $Y$  riêng rẽ trong khi giữ các variable khác fixed.
- Smoothness của  $f_j$  cho  $X_j$  có thể summarize bằng degrees of freedom.
- Limitation chính là GAM bị restrict additive, nên khi có nhiều variables có thể miss important interactions. Tuy vậy, như linear regression, có thể add interaction terms thủ công bằng predictors dạng  $X_j \times X_k$ , hoặc add low-dimensional interaction function  $f_{jk}(X_j, X_k)$  fit bằng two-dimensional smoothers như local regression hay two-

<sup>6</sup>Ví dụ partial residual cho  $X_3$  là  $r_i = y_i - f_1(x_{i1}) - f_2(x_{i2})$ . Nếu biết  $f_1, f_2$ , ta fit  $f_3$  bằng cách xem residual này như response trong non-linear regression theo  $X_3$ .

dimensional splines.

Với fully general models, cần flexible approaches hơn như random forests và boosting ở Chapter 8. GAMs là compromise hữu ích giữa linear và fully nonparametric models.

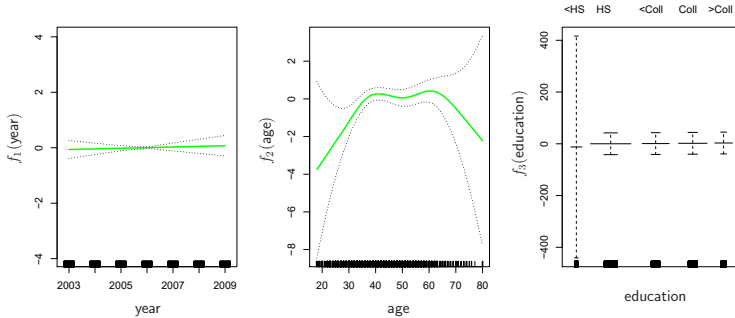
### 7.7.2 GAMs for Classification Problems

GAMs cũng dùng được khi  $Y$  qualitative. Để đơn giản, giả sử  $Y \in \{0, 1\}$  và  $p(X) = \Pr(Y = 1 | X)$ . Logistic regression model (4.6) là

$$\log \left\{ \frac{p(X)}{1 - p(X)} \right\} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p. \quad (7.17)$$

Left side là log odds của  $P(Y = 1 | X)$  so với  $P(Y = 0 | X)$ ; (7.17) biểu diễn quantity này như linear function của predictors. Extension tự nhiên cho non-linear relationships là

$$\log \left\{ \frac{p(X)}{1 - p(X)} \right\} = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p). \quad (7.18)$$



**HÌNH 7.13.** Với dataset *Wage*, logistic regression GAM (7.19) được fit cho binary response  $I(\text{wage} > 250)$ . Mỗi panel hiển thị fitted function và pointwise standard errors. Function thứ nhất linear theo *year*; function thứ hai là smoothing spline với năm degrees of freedom theo *age*; function thứ ba là step function cho *education*. Standard errors rất rộng ở level <HS.

Equation (7.18) là logistic regression GAM và có cùng pros/cons như case quantitative response. Ta fit GAM trên *Wage* để predict probability income vượt \$250,000 mỗi năm:

$$\log \left\{ \frac{p(X)}{1 - p(X)} \right\} = \beta_0 + \beta_1 \times \text{year} + f_2(\text{age}) + f_3(\text{education}), \quad (7.19)$$

trong đó

$$p(X) = \Pr(\text{wage} > 250 | \text{year}, \text{age}, \text{education}).$$

Một lần nữa  $f_2$  dùng smoothing spline năm degrees of freedom, còn  $f_3$  dùng step function qua dummy variables cho levels của education.

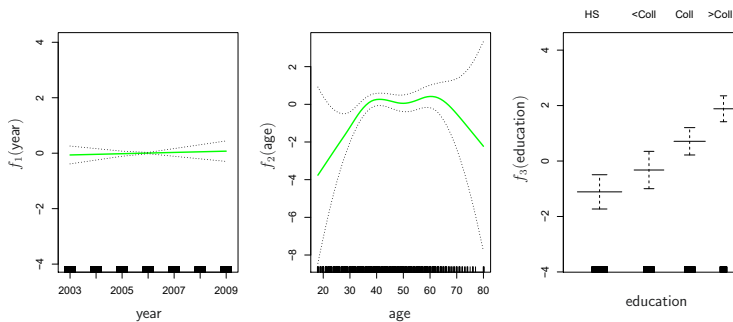
Fit thu được ở Figure 7.13. Last panel đáng ngờ vì confidence intervals rất rộng ở level <HS. Thực tế category này không có response bằng



one: không individual nào có less than high-school education kiếm hơn \$250,000/năm. Vì vậy ta refit GAM sau khi exclude category này. Result ở Figure 7.14. Như Figures 7.11 và 7.12, ba panels có vertical scales tương tự để so sánh relative contribution của variables. Ta thấy age và education có effect lớn hơn year lên probability trở thành high earner.

## 7.8 Lab: Non-Linear Modeling

Trong lab này, ta demonstrate một số nonlinear model đã bàn trong chapter. Dataset **Wage** được dùng xuyên suốt, và ta sẽ thấy nhiều complex non-linear fitting procedure có thể implement khá dễ trong Python.



**HÌNH 7.14.** Cùng model như Figure 7.13 được fit lại sau khi exclude observations có *education*=<HS. Bây giờ ta thấy education cao hơn có xu hướng associated với salary cao hơn.

Như thường lệ, ta bắt đầu với các standard imports.

In [1]:

```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (summarize,
 poly,
 ModelSpec as MS)
from statsmodels.stats.anova import anova_lm
```

Ta cũng gom các import mới cần cho lab; nhiều item được phát triển riêng cho package ISLP.

In [2]:

```
from pygam import (s as s_gam,
 l as l_gam,
 f as f_gam,
 LinearGAM,
 LogisticGAM)
from ISLP.transforms import (BSpline,
 NaturalSpline)
from ISLP.models import bs, ns
from ISLP.pygam import (approx_lam,
 degrees_of_freedom,
 plot as plot_gam,
 anova as anova_gam)
```

### 7.8.1 Polynomial Regression and Step Functions

Ta bắt đầu bằng cách reproduce Figure 7.1. Trước hết load data.

```
In [3]: Wage = load_data('Wage')
y = Wage['wage']
age = Wage['age']
```

Trong phần lớn lab, response là `Wage['wage']` đã lưu vào `y`. Như Section 3.6.6, ta dùng `poly()` để tạo model matrix fit fourth-degree polynomial theo `age`.

```
In [4]: poly_age = MS([poly('age', degree=4)]).fit(Wage)
M = sm.OLS(y, poly_age.transform(Wage)).fit()
summarize(M)
```

```
Out[4]:
```

|                         | coef      | std err | t       | P> t  |
|-------------------------|-----------|---------|---------|-------|
| intercept               | 111.7036  | 0.729   | 153.283 | 0.000 |
| poly(age, degree=4) [0] | 447.0679  | 39.915  | 11.201  | 0.000 |
| poly(age, degree=4) [1] | -478.3158 | 39.915  | -11.983 | 0.000 |
| poly(age, degree=4) [2] | 125.5217  | 39.915  | 3.145   | 0.002 |
| poly(age, degree=4) [3] | -77.9112  | 39.915  | -1.952  | 0.051 |

Polynomial này được construct bằng helper function `poly()`, tạo transformer đặc biệt `Poly()` (theo terminology của `sklearn` cho feature transformations, như `PCA()` ở Section 6.5.3) để evaluate polynomial dễ dàng trên new data points. `poly()` thiết lập transformation; `Poly()` là workhorse thực sự compute transformation. Xem thêm discussion về transformations ở page 118.

transformer  
helper

Trong code trên, first line chạy method `fit()` trên dataframe `Wage`. Việc này recompute và lưu dưới dạng attributes mọi parameter mà `Poly()` cần từ training data; các parameter này sau đó được dùng cho mọi evaluation của `transform()`. Chẳng hạn chúng được dùng ở second line và trong plotting function phía dưới.

Bây giờ tạo grid của age values để predict.

```
In [5]: age_grid = np.linspace(age.min(),
 age.max(),
 100)
age_df = pd.DataFrame({'age': age_grid})
```

Ta muốn plot data và thêm fourth-degree polynomial fit. Vì sẽ làm nhiều plot tương tự, ta viết một function tạo các ingredient và plot. Function nhận model specification (ở đây là basis do transform xác định) và grid age values; nó tạo fitted curve cùng 95% confidence bands. Argument `basis` giúp dùng lại function với nhiều transform khác, chẳng hạn splines.

```
In [6]: def plot_wage_fit(age_df,
 basis,
 title):
 X = basis.transform(Wage)
 Xnew = basis.transform(age_df)
 M = sm.OLS(y, X).fit()
 preds = M.get_prediction(Xnew)
 bands = preds.conf_int(alpha=0.05)
 fig, ax = subplots(figsize=(8,8))
```

```

ax.scatter(age,
 y,
 facecolor='gray',
 alpha=0.5)
for val, ls in zip([preds.predicted_mean,
 bands[:,0],
 bands[:,1]],
 ['b', 'r--', 'r--']):
 ax.plot(age_df.values, val, ls, linewidth=3)
ax.set_title(title, fontsize=20)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
return ax

```

Argument `alpha` trong `ax.scatter()` thêm transparency cho points, giúp visualize density. Chú ý `zip()` trong for loop (Section 2.3.8): ta có ba line với color/line type khác nhau, và `zip()` bundle chúng thành iterators trong loop. <sup>7</sup>

*iterator*

Ta plot fourth-degree polynomial fit bằng function vừa tạo.

In [7]:

```

plot_wage_fit(age_df,
 poly_age,
 'Degree-4 Polynomial');

```

Với polynomial regression ta phải chọn degree. Đôi khi ta đơn giản chọn degree 2 hoặc 3 để có nonlinear fit, nhưng có thể systematic hơn bằng hypothesis tests. Ta fit series model từ linear (degree 1) đến degree 5, rồi tìm simplest model đủ explain relationship giữa wage và age. Function `anova_lm()` thực hiện sequence ANOVA tests. Analysis of variance (ANOVA) kiểm định null hypothesis rằng model  $M_1$  đủ explain data, so với alternative rằng cần model phức tạp hơn  $M_2$ . Determination dựa trên F-test, và  $M_1, M_2$  phải nested: predictor space của  $M_1$  là subspace của predictor space của  $M_2$ .

*analysis of variance*

In [8]:

```

models = [MS([poly('age', degree=d)])
 for d in range(1, 6)]
Xs = [model.fit_transform(Wage) for model in models]
anova_lm(*[sm.OLS(y, X_).fit()
 for X_ in Xs])

```

Out[8]:

|   | df_resid | ssr       | df_diff | ss_diff    | F       | Pr(>F)    |
|---|----------|-----------|---------|------------|---------|-----------|
| 0 | 2998.0   | 5.022e+06 | 0.0     | NaN        | NaN     | NaN       |
| 1 | 2997.0   | 4.793e+06 | 1.0     | 228786.010 | 143.593 | 2.364e-32 |
| 2 | 2996.0   | 4.778e+06 | 1.0     | 15755.694  | 9.889   | 1.679e-03 |
| 3 | 2995.0   | 4.772e+06 | 1.0     | 6070.152   | 3.810   | 5.105e-02 |
| 4 | 2994.0   | 4.770e+06 | 1.0     | 1282.563   | 0.805   | 3.697e-01 |

Chú ý dấu `*` trong line `anova_lm()` ở trên. Function này nhận variable number of non-keyword arguments, ở đây là fitted models. Khi các model được cung cấp dưới dạng list, list phải prefix bằng `*`. P-value so sánh linear `models[0]` với quadratic `models[1]` gần như zero, cho thấy linear fit không đủ.<sup>8</sup> Tương tự, p-value so sánh quadratic và cubic là 0.0017, nên quadratic cũng không đủ. P-value so sánh cubic và degree-four khoảng 5%,

<sup>7</sup>Theo cách nói của Python, “iterator” là object có finite number of values và có thể iterate, như trong loop.

<sup>8</sup>Index bắt đầu từ zero hơi dễ gây nhầm trong example polynomial degree, vì `models[1]` là quadratic chứ không phải linear.

còn degree-five không cần thiết vì p-value 0.37. Vì vậy cubic hoặc quartic polynomial đều là reasonable fit; lower-order hoặc higher-order model không được justify.

Thay vì `anova_lm()`, trong case này ta có thể lấy p-values ngắn gọn hơn nhờ `poly()` tạo orthogonal polynomials.

In [9]:

```
summarize(M)
```

Out[9]:

|                         | coef      | std err | t       | P> t  |
|-------------------------|-----------|---------|---------|-------|
| intercept               | 111.7036  | 0.729   | 153.283 | 0.000 |
| poly(age, degree=4) [0] | 447.0679  | 39.915  | 11.201  | 0.000 |
| poly(age, degree=4) [1] | -478.3158 | 39.915  | -11.983 | 0.000 |
| poly(age, degree=4) [2] | 125.5217  | 39.915  | 3.145   | 0.002 |
| poly(age, degree=4) [3] | -77.9112  | 39.915  | -1.952  | 0.051 |

P-values giống nhau, và square của t-statistics bằng F-statistics từ `anova_lm()`; ví dụ:

In [10]:

```
(-11.983)**2
```

Out[10]:

```
143.59228
```

ANOVA method vẫn hoạt động dù ta không dùng orthogonal polynomials, miễn models nested. Ví dụ, ta compare ba model đều có linear term cho `education` và polynomial theo `age` với degree khác nhau:

In [11]:

```
models = [MS(['education', poly('age', degree=d)])
 for d in range(1, 4)]
XEs = [model.fit_transform(Wage)
 for model in models]
anova_lm([sm.OLS(y, X_).fit() for X_ in XEs])
```

Out[11]:

|   | df_resid | ssr       | df_diff | ss_diff    | F       | Pr(>F)    |
|---|----------|-----------|---------|------------|---------|-----------|
| 0 | 2997.0   | 3.902e+06 | 0.0     | NaN        | NaN     | NaN       |
| 1 | 2996.0   | 3.759e+06 | 1.0     | 142862.701 | 113.992 | 3.838e-26 |
| 2 | 2995.0   | 3.754e+06 | 1.0     | 5926.207   | 4.729   | 2.974e-02 |

Một alternative cho hypothesis tests và ANOVA là chọn polynomial degree bằng cross-validation, như Chapter 5.

Tiếp theo ta predict liệu một individual có earning trên \$250,000 mỗi năm hay không. Cách làm gần như trước: tạo response vector phù hợp, rồi dùng `sm.GLM()` với binomial family để fit polynomial logistic regression model.

In [12]:

```
X = poly_age.transform(Wage)
high_earn = Wage['high_earn'] = y > 250 # shorthand
glm = sm.GLM(y > 250,
 X,
 family=sm.families.Binomial())
B = glm.fit()
summarize(B)
```

Out[12]:

|                         | coef     | std err | z       | P> z  |
|-------------------------|----------|---------|---------|-------|
| intercept               | -4.3012  | 0.345   | -12.457 | 0.000 |
| poly(age, degree=4) [0] | 71.9642  | 26.133  | 2.754   | 0.006 |
| poly(age, degree=4) [1] | -85.7729 | 35.929  | -2.387  | 0.017 |

```
poly(age, degree=4)[2] 34.1626 19.697 1.734 0.083
poly(age, degree=4)[3] -47.4008 24.105 -1.966 0.049
```

Một lần nữa ta predict bằng method `get_prediction()`.

```
In [13]: newX = poly_age.transform(age_df)
 preds = B.get_prediction(newX)
 bands = preds.conf_int(alpha=0.05)
```

Bây giờ plot estimated relationship.

```
In [14]: fig, ax = subplots(figsize=(8,8))
 rng = np.random.default_rng(0)
 ax.scatter(age +
 0.2 * rng.uniform(size=y.shape[0]),
 np.where(high_earn, 0.198, 0.002),
 fc='gray',
 marker='|')
 for val, ls in zip([preds.predicted_mean,
 bands[:,0],
 bands[:,1]],
 ['b', 'r--', 'r--']):
 ax.plot(age_df.values, val, ls, linewidth=3)
 ax.set_title('Degree-4 Polynomial', fontsize=20)
 ax.set_xlabel('Age', fontsize=20)
 ax.set_ylim([0,0.2])
 ax.set_ylabel('P(Wage > 250)', fontsize=20);
```

Ta vẽ age values của observations có wage trên 250 thành gray marks ở top, còn dưới 250 ở bottom. Một lượng nhỏ noise được thêm để jitter age values, tránh observations cùng age che nhau; plot kiểu này thường gọi là *rug plot*.

Để fit step function như Section 7.2, ta dùng `pd.qcut()` discretize age theo quantiles, rồi `pd.get_dummies()` tạo columns của model matrix cho categorical variable. Function này include mọi column của categorical thay vì drop một level như usual approach.

```
In [15]: cut_age = pd.qcut(age, 4)
 summarize(sm.OLS(y, pd.get_dummies(cut_age)).fit())
```

```
Out[15]:
```

|                 | coef     | std err | t      | P> t |
|-----------------|----------|---------|--------|------|
| (17.999, 33.75] | 94.1584  | 1.478   | 63.692 | 0.0  |
| (33.75, 42.0]   | 116.6608 | 1.470   | 79.385 | 0.0  |
| (42.0, 51.0]    | 119.1887 | 1.416   | 84.147 | 0.0  |
| (51.0, 80.0]    | 116.5717 | 1.559   | 74.751 | 0.0  |

Ở đây `pd.qcut()` tự động chọn cutpoints theo quantiles 25%, 50%, 75%, tạo bốn regions. Ta cũng có thể specify quantiles trực tiếp thay vì argument 4. Với cuts không dựa trên quantiles, dùng `pd.cut()`. Cả `pd.qcut()` và `pd.cut()` trả về ordered categorical variable; regression model sau đó tạo dummy variables để dùng trong regression. Vì age là variable duy nhất trong model, \$94,158.40 là average salary của group dưới 33.75 tuổi, còn các coefficients khác là average salary ở các age group tương ứng. Prediction và plot có thể tạo tương tự polynomial fit.

*rug plot*

`pd.qcut()`  
`pd.get_dummies()`

`pd.cut()`

### 7.8.2 Splines

Để fit regression splines, ta dùng các transform từ package ISLP. Các spline evaluation functions thực sự nằm trong package `scipy.interpolate`; package ISLP chỉ wrap chúng thành các transform tương tự `Poly()` và `PCA()`.

Ở Section 7.4, ta đã thấy regression splines có thể được fit bằng cách construct một matrix basis functions phù hợp. Function `BSpline()` tạo toàn bộ matrix basis functions cho spline với set of knots được chỉ định. Mặc định các B-splines được tạo là cubic; để đổi degree, dùng argument `degree`.

*BSpline()*

In [16]:

```
bs_ = BSpline(internal_knots=[25,40,60], intercept=True).fit(age)
bs_age = bs_.transform(age)
bs_age.shape
```

Out[16]:

(3000, 7)

Kết quả là một matrix bảy columns, đúng như kỳ vọng với cubic-spline basis có 3 interior knots. Ta có thể tạo cùng matrix này bằng object `bs()`, thuận tiện cho việc đưa basis vào model-matrix builder (tương tự quan hệ giữa `poly()` và workhorse `Poly()`) đã mô tả trong Section 7.8.1.

Bây giờ ta fit cubic spline model cho data `Wage`.

In [17]:

```
bs_age = MS([bs('age', internal_knots=[25,40,60])])
Xbs = bs_age.fit_transform(Wage)
M = sm.OLS(y, Xbs).fit()
summarize(M)
```

Out[17]:

|                                         | coef   | std err | ... |
|-----------------------------------------|--------|---------|-----|
| intercept                               | 60.494 | 9.460   | ... |
| bs(age, internal_knots=[25, 40, 60])[0] | 3.980  | 12.538  | ... |
| bs(age, internal_knots=[25, 40, 60])[1] | 44.631 | 9.626   | ... |
| bs(age, internal_knots=[25, 40, 60])[2] | 62.839 | 10.755  | ... |
| bs(age, internal_knots=[25, 40, 60])[3] | 55.991 | 10.706  | ... |
| bs(age, internal_knots=[25, 40, 60])[4] | 50.688 | 14.402  | ... |
| bs(age, internal_knots=[25, 40, 60])[5] | 16.606 | 19.126  | ... |

Column names hơi cồng kềnh, khiến printed summary ở trên phải truncate. Ta có thể đặt tên ngay khi construct bằng argument `name` như sau.

In [18]:

```
bs_age = MS([bs('age',
 internal_knots=[25,40,60],
 name='bs(age)')])
Xbs = bs_age.fit_transform(Wage)
M = sm.OLS(y, Xbs).fit()
summarize(M)
```

Out[18]:

|                   | coef   | std err | t     | P> t  |
|-------------------|--------|---------|-------|-------|
| intercept         | 60.494 | 9.460   | 6.394 | 0.000 |
| bs(age, knots)[0] | 3.981  | 12.538  | 0.317 | 0.751 |
| bs(age, knots)[1] | 44.631 | 9.626   | 4.636 | 0.000 |
| bs(age, knots)[2] | 62.839 | 10.755  | 5.843 | 0.000 |
| bs(age, knots)[3] | 55.991 | 10.706  | 5.230 | 0.000 |
| bs(age, knots)[4] | 50.688 | 14.402  | 3.520 | 0.000 |
| bs(age, knots)[5] | 16.606 | 19.126  | 0.868 | 0.385 |

Chú ý có 6 spline coefficients chứ không phải 7. Nguyên nhân là mặc định

`bs()` giả định `intercept=False`, vì model thường đã có overall intercept. Do đó nó generate spline basis với các knots đã cho rồi discard một basis function để account cho intercept.

Ta cũng có thể dùng option `df` (degrees of freedom) để chỉ định complexity của spline. Ở trên, 3 knots tạo spline basis có 6 columns hay 6 degrees of freedom. Nếu chỉ định `df=6` thay vì actual knots, `bs()` sẽ tạo spline với 3 knots đặt tại uniform quantiles của training data. Có thể xem các knots được chọn trực tiếp bằng `BSpline()`:

```
In [19]: BSpline(df=6).fit(age).internal_knots_
```

```
Out[19]: array([33.75, 42.0, 51.0])
```

Khi yêu cầu sáu degrees of freedom, transform chọn knots tại ages 33.75, 42.0 và 51.0, tương ứng các percentiles 25th, 50th và 75th của `age`.

Với B-splines, ta không bắt buộc phải dùng cubic polynomials (tức `degree=3`). Chẳng hạn, `degree=0` tạo piecewise constant functions như example dùng `pd.qcut()` ở trên.

```
In [20]: bs_age0 = MS([bs('age',
 df=3,
 degree=0)]).fit(Wage)
Xbs0 = bs_age0.transform(Wage)
summarize(sm.OLS(y, Xbs0).fit())
```

```
Out[20]:
```

|                            | coef   | std err | t      | P> t |
|----------------------------|--------|---------|--------|------|
| intercept                  | 94.158 | 1.478   | 63.687 | 0.0  |
| bs(age, df=3, degree=0)[0] | 22.349 | 2.152   | 10.388 | 0.0  |
| bs(age, df=3, degree=0)[1] | 24.808 | 2.044   | 12.137 | 0.0  |
| bs(age, df=3, degree=0)[2] | 22.781 | 2.087   | 10.917 | 0.0  |

Fit này nên được compare với cell [15], nơi `qcut()` tạo bốn bins bằng cách cắt tại 25%, 50% và 75% quantiles của `age`. Vì ở đây ta chỉ định `df=3` cho degree-zero splines, cũng sẽ có knots tại đúng ba quantiles đó. Dù coefficients trông khác nhau, đây chỉ là hệ quả của coding khác nhau. Chẳng hạn first coefficient giống hệt ở cả hai case và chính là mean response trong first bin. Với second coefficient,  $94.158 + 22.349 = 116.507 \approx 116.611$ , trong đó giá trị sau là mean ở second bin trong cell [15]. Ở đây intercept được code bằng một column of ones, nên second, third và fourth coefficients là increments cho các bins tương ứng. Tổng không hoàn toàn giống nhau vì `qcut()` dùng `≤`, còn `bs()` dùng `<` khi quyết định bin membership.

Để fit natural spline, ta dùng transform `NaturalSpline()` với helper tương ứng `ns()`. Ở đây ta fit natural spline với năm degrees of freedom (không tính intercept) và plot kết quả.

*NaturalSpline()*

```
In [21]: ns_age = MS([ns('age', df=5)]).fit(Wage)
M_ns = sm.OLS(y, ns_age.transform(Wage)).fit()
summarize(M_ns)
```

```
Out[21]:
```

|                  | coef   | std err | t      | P> t  |
|------------------|--------|---------|--------|-------|
| intercept        | 60.475 | 4.708   | 12.844 | 0.000 |
| ns(age, df=5)[0] | 61.527 | 4.709   | 13.065 | 0.000 |
| ns(age, df=5)[1] | 55.691 | 5.717   | 9.741  | 0.000 |
| ns(age, df=5)[2] | 46.818 | 4.948   | 9.463  | 0.000 |

```
ns(age, df=5)[3] 83.204 11.918 6.982 0.000
ns(age, df=5)[4] 6.877 9.484 0.725 0.468
```

Bây giờ plot natural spline bằng plotting function đã viết trước đó.

In [22]:

```
plot_wage_fit(age_df,
 ns_age,
 'Natural spline, df=5');
```

### 7.8.3 Smoothing Splines and GAMs

Smoothing spline là một special case của GAM với squared-error loss và một feature. Để fit GAMs trong Python, ta dùng package `pygam`, có thể install bằng `pip install pygam`. Estimator `LinearGAM()` dùng squared-error loss. GAM được specified bằng cách associate mỗi column của model matrix với một smoothing operation cụ thể: `s` cho smoothing spline, `l` cho linear, và `f` cho factor hay categorical variables. Argument 0 truyền vào `s` bên dưới cho biết smoother sẽ apply vào first column của feature matrix. Ở đây ta truyền matrix chỉ có một column là `X_age`. Argument `lam` là penalty parameter  $\lambda$  như Section 7.5.2.

`pygam`  
`LinearGAM()`  
In [23]:

```
X_age = np.asarray(age).reshape((-1,1))
gam = LinearGAM(s_gam(0, lam=0.6))
gam.fit(X_age, y)
```

Out[23]:

```
LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
 max_iter=100, scale=None, terms=s(0) + intercept, tol=0.0001,
 verbose=False)
```

Library `pygam` thường kỳ vọng một matrix of features, nên ta reshape `age` thành matrix (two-dimensional array) thay vì vector (one-dimensional array). Giá trị `-1` trong `reshape()` bảo numpy suy ra size của dimension đó từ các entry còn lại của shape tuple.

Hãy xem fit thay đổi thế nào theo smoothing parameter `lam`. Function `np.logspace()` giống `np.linspace()` nhưng spacing đều trên log-scale. Dưới đây ta thay đổi `lam` từ  $10^{-2}$  đến  $10^6$ .

`np.logspace()`

In [24]:

```
fig, ax = subplots(figsize=(8,8))
ax.scatter(age, y, facecolor='gray', alpha=0.5)
for lam in np.logspace(-2, 6, 5):
 gam = LinearGAM(s_gam(0, lam=lam)).fit(X_age, y)
 ax.plot(age_grid,
 gam.predict(age_grid),
 label='{:.1e}'.format(lam),
 linewidth=3)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='$\\lambda$');
```

Package `pygam` có thể search optimal smoothing parameter.

In [25]:

```
gam_opt = gam.gridsearch(X_age, y)
ax.plot(age_grid,
 gam_opt.predict(age_grid),
 label='Grid search',
 linewidth=4)
ax.legend()
```



fig

Một alternative là fix degrees of freedom của smoothing spline bằng function trong package ISLP `pygam`. Bên dưới ta tìm một  $\lambda$  cho roughly four degrees of freedom. Lưu ý các degrees of freedom này bao gồm unpenalized intercept và linear term của smoothing spline, vì thế luôn có ít nhất hai degrees of freedom.

```
In [26]: age_term = gam.terms[0]
lam_4 = approx_lam(X_age, age_term, 4)
age_term.lam = lam_4
degrees_of_freedom(X_age, age_term)
```

```
Out[26]: 4.000000100004728
```

Ta thay đổi degrees of freedom trên plot tương tự phía trên. Ta chọn degrees of freedom bằng desired degrees of freedom cộng một để account cho fact rằng các smoothing splines này luôn có intercept term. Vì vậy giá trị  $df=1$  tương ứng linear fit.

```
In [27]: fig, ax = subplots(figsize=(8,8))
ax.scatter(X_age,
 y,
 facecolor='gray',
 alpha=0.3)
for df in [1,3,4,8,15]:
 lam = approx_lam(X_age, age_term, df+1)
 age_term.lam = lam
 gam.fit(X_age, y)
 ax.plot(age_grid,
 gam.predict(age_grid),
 label='{d}'.format(df),
 linewidth=4)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='Degrees of freedom');
```

### Additive Models with Several Terms

Điểm mạnh của generalized additive models là khả năng fit multivariate regression models flexible hơn linear models. Ta sẽ minh họa hai approach: một approach manual hơn dùng natural splines và piecewise constant functions, và approach thứ hai dùng package `pygam` cùng smoothing splines.

Trước hết ta fit GAM bằng tay để predict `wage` từ natural spline functions của `year` và `age`, đồng thời treat `education` như qualitative predictor, như trong (7.16). Vì đây đơn giản là một linear regression model lớn dùng basis functions thích hợp, ta có thể fit trực tiếp bằng `sm.OLS()`.

Ta build model matrix thủ công hơn ở đây, vì muốn truy cập riêng từng piece khi construct partial dependence plots.

```
In [28]: ns_age = NaturalSpline(df=4).fit(age)
ns_year = NaturalSpline(df=5).fit(Wage['year'])
Xs = [ns_age.transform(age),
 ns_year.transform(Wage['year']),
 pd.get_dummies(Wage['education']).values]
X_bh = np.hstack(Xs)
gam_bh = sm.OLS(y, X_bh).fit()
```

Ở đây `NaturalSpline()` là workhorse đứng sau helper `ns()`. Ta dùng toàn bộ columns của indicator matrix cho categorical variable `education`, khiến intercept trở nên redundant. Cuối cùng, ta stack ba component matrices theo chiều ngang để tạo model matrix `X_bh`.

Bây giờ ta construct partial dependence plots cho từng term trong rudimentary GAM này. Có thể làm thủ công nếu có grids cho `age` và `year`: predict bằng các new `X` matrices trong đó chỉ thay đổi một feature tại một thời điểm, còn lại fix.

In [29]:

```
age_grid = np.linspace(age.min(),
 age.max(),
 100)
X_age_bh = X_bh.copy()[1:100]
X_age_bh[:, :] = X_bh[:, :].mean(0)[None, :]
X_age_bh[:, :4] = ns_age.transform(age_grid)
preds = gam_bh.get_prediction(X_age_bh)
bounds_age = preds.conf_int(alpha=0.05)
partial_age = preds.predicted_mean
center = partial_age.mean()
partial_age -= center
bounds_age -= center
fig, ax = subplots(figsize=(8,8))
ax.plot(age_grid, partial_age, 'b', linewidth=3)
ax.plot(age_grid, bounds_age[:,0], 'r--', linewidth=3)
ax.plot(age_grid, bounds_age[:,1], 'r--', linewidth=3)
ax.set_xlabel('Age')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of age on wage', fontsize=20);
```

Chi tiết các bước vừa làm như sau. Ý tưởng là tạo một new prediction matrix, trong đó mọi column không thuộc `age` đều constant và set bằng training-data means, còn bốn columns của `age` được fill bằng natural spline basis evaluated tại 100 values trong `age_grid`.

1. Tạo grid length 100 theo `age`, và matrix `X_age_bh` có 100 rows cùng số columns như `X_bh`.
2. Replace mọi row của matrix bằng column means của original matrix.
3. Replace riêng bốn columns đầu, đại diện cho `age`, bằng natural spline basis tính tại các values trong `age_grid`.

Các bước còn lại đã quen thuộc.

Ta cũng xem effect của `year` lên `wage`; process giống hệt.

In [30]:

```
year_grid = np.linspace(2003, 2009, 100)
year_grid = np.linspace(Wage['year'].min(),
 Wage['year'].max(),
 100)
X_year_bh = X_bh.copy()[1:100]
X_year_bh[:, :] = X_bh[:, :].mean(0)[None, :]
X_year_bh[:, :4:9] = ns_year.transform(year_grid)
preds = gam_bh.get_prediction(X_year_bh)
bounds_year = preds.conf_int(alpha=0.05)
partial_year = preds.predicted_mean
center = partial_year.mean()
partial_year -= center
bounds_year -= center
fig, ax = subplots(figsize=(8,8))
ax.plot(year_grid, partial_year, 'b', linewidth=3)
ax.plot(year_grid, bounds_year[:,0], 'r--', linewidth=3)
ax.plot(year_grid, bounds_year[:,1], 'r--', linewidth=3)
```

```
ax.set_xlabel('Year')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of year on wage', fontsize=20);
```

Tiếp theo ta fit model (7.16) bằng smoothing splines thay vì natural splines. Mọi term trong (7.16) được fit simultaneously, nên mỗi term tính đến các term khác khi explain response. Package `pygam` chỉ làm việc với matrices, vì vậy categorical series `education` phải được convert sang array representation qua attribute `cat.codes`. Vì `year` chỉ có 7 unique values, ta chỉ dùng bảy basis functions cho term này.

In [31]:

```
gam_full = LinearGAM(s_gam(0) +
 s_gam(1, n_splines=7) +
 f_gam(2, lam=0))
Xgam = np.column_stack([age,
 Wage['year'],
 Wage['education'].cat.codes])
gam_full = gam_full.fit(Xgam, y)
```

Hai terms `s_gam()` tạo smoothing spline fits và dùng default  $\lambda$  (`lam=0.6`), một lựa chọn khá arbitrary. Với categorical term `education`, specified bằng `f_gam()`, ta đặt `lam=0` để tránh shrinkage. Ta tạo partial dependence plot theo `age` để xem effect của các lựa chọn này.

Các values cho plot được generate bởi package `pygam`. Package `ISLP.pygam` cung cấp function `plot_gam()` cho partial-dependence plots, giúp việc này dễ hơn example natural spline vừa làm.

*plot\_gam()*

In [32]:

```
fig, ax = subplots(figsize=(8,8))
plot_gam(gam_full, 0, ax=ax)
ax.set_xlabel('Age')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of age on wage - default lam=0.6',
 fontsize=20);
```

Function khá wiggly. Tự nhiên hơn là specify `df` thay vì `lam`. Ta refit GAM dùng four degrees of freedom cho cả `age` và `year`. Nhớ rằng cộng một bên dưới là để account cho intercept của smoothing spline.

In [33]:

```
age_term = gam_full.terms[0]
age_term.lam = approx_lam(Xgam, age_term, df=4+1)
year_term = gam_full.terms[1]
year_term.lam = approx_lam(Xgam, year_term, df=4+1)
gam_full = gam_full.fit(Xgam, y)
```

Chú ý: update `age_term.lam` ở trên cũng update term tương ứng trong `gam_full`; cụ thể là `gam_full.terms[0]`. Tương tự cho `year_term.lam`.

Lặp lại plot cho `age` cho thấy curve smooth hơn nhiều. Ta cũng plot `year`.

In [34]:

```
fig, ax = subplots(figsize=(8,8))
plot_gam(gam_full,
 1,
 ax=ax)
ax.set_xlabel('Year')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of year on wage', fontsize=20)
```

Cuối cùng plot `education`, một categorical variable. Partial dependence plot có dạng khác và phù hợp hơn với set of fitted constants cho từng level.

```
In [35]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_full, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of wage on education',
 fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```

### ANOVA Tests for Additive Models

Trong mọi model, function của `year` trông gần như linear. Ta có thể thực hiện series ANOVA tests để xác định model nào tốt nhất trong ba model: GAM bỏ `year` ( $M_1$ ), GAM dùng linear function của `year` ( $M_2$ ), hoặc GAM dùng spline function của `year` ( $M_3$ ).

```
In [36]: gam_0 = LinearGAM(age_term + f_gam(2, lam=0))
gam_0.fit(Xgam, y)
gam_linear = LinearGAM(age_term +
 l_gam(1, lam=0) +
 f_gam(2, lam=0))
gam_linear.fit(Xgam, y)
```

```
Out[36]: LinearGAM(callbacks=[Deviance(), Diffs()], fit_intercept=True,
 max_iter=100, scale=None, terms=s(0) + l(1) + f(2) + intercept,
 tol=0.0001, verbose=False)
```

Chú ý việc dùng `age_term` trong expressions trên. Ta làm vậy vì trước đó đã set `lam` trong term này để đạt four degrees of freedom.

Để trực tiếp assess effect của `year`, ta chạy ANOVA trên ba fitted models.

```
In [37]: anova_gam(gam_0, gam_linear, gam_full)
```

```
Out[37]:
```

|   | deviance    | df       | deviance_diff | df_diff | F      | pvalue |
|---|-------------|----------|---------------|---------|--------|--------|
| 0 | 3714362.366 | 2991.004 | NaN           | NaN     | NaN    | NaN    |
| 1 | 3696745.823 | 2990.005 | 17616.543     | 0.999   | 14.265 | 0.002  |
| 2 | 3693142.930 | 2987.007 | 3602.894      | 2.998   | 0.972  | 0.436  |

Có compelling evidence rằng GAM với linear function theo `year` tốt hơn GAM bỏ `year` hoàn toàn ( $p\text{-value} = 0.002$ ). Tuy nhiên, không có evidence rằng cần non-linear function của `year` ( $p\text{-value} = 0.435$ ). Nói cách khác, từ ANOVA này  $M_2$  được preferred.

Ta lặp process tương tự cho `age`. Kết quả cho evidence rất rõ rằng cần một non-linear term cho `age`.

```
In [38]: gam_0 = LinearGAM(year_term +
 f_gam(2, lam=0))
gam_linear = LinearGAM(l_gam(0, lam=0) +
 year_term +
 f_gam(2, lam=0))
gam_0.fit(Xgam, y)
gam_linear.fit(Xgam, y)
anova_gam(gam_0, gam_linear, gam_full)
```

```
Out[38]:
```

|   | deviance    | df       | deviance_diff | df_diff | F       | pvalue |
|---|-------------|----------|---------------|---------|---------|--------|
| 0 | 3975443.045 | 2991.001 | NaN           | NaN     | NaN     | NaN    |
| 1 | 3850246.908 | 2990.001 | 125196.137    | 1.000   | 101.270 | 0.000  |
| 2 | 3693142.930 | 2987.007 | 157103.978    | 2.993   | 42.448  | 0.000  |

GAM fit có method `summary()` khá verbose; sách không reproduce output ở đây.

```
In [39]: gam_full.summary()
```

Ta có thể predict từ `gam` objects giống `lm` objects, dùng method `predict()` của class `gam`. Ở đây ta predict trên training set.

```
In [40]: Yhat = gam_full.predict(Xgam)
```

Để fit logistic regression GAM, ta dùng `LogisticGAM()` từ `pygam`.

*LogisticGAM()*

```
In [41]: gam_logit = LogisticGAM(age_term +
 l_gam(1, lam=0) +
 f_gam(2, lam=0))
gam_logit.fit(Xgam, high_earn)
```

```
Out[41]: LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
 fit_intercept=True, max_iter=100,
 terms=s(0) + l(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

```
In [42]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of wage on education',
 fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories, fontsize=8);
```

Model trông khá flat, với error bars đặc biệt lớn ở first category. Hãy xem data kỹ hơn.

```
In [43]: pd.crosstab(Wage['high_earn'], Wage['education'])
```

Không có high earners ở first category của `education`, nên model sẽ rất khó fit. Ta sẽ fit logistic regression GAM sau khi exclude tất cả observations thuộc category này; kết quả hợp lý hơn.

Ta có thể subset model matrix, nhưng như vậy column tương ứng vẫn còn trong `Xgam`. Dù có thể suy ra column nào ứng với feature này, để reproducible ta reform model matrix trên subset nhỏ hơn.

```
In [44]: only_hs = Wage['education'] == '1. < HS Grad'
Wage_ = Wage.loc[~only_hs]
Xgam_ = np.column_stack([Wage_['age'],
 Wage_['year'],
 Wage_['education'].cat.codes-1])
high_earn_ = Wage_['high_earn']
```

Ở second-to-last line bên trên, ta subtract one khỏi category codes vì một bug trong `pygam`. Việc này chỉ relabel education values nên không ảnh

hưởng fit.

Bây giờ fit model.

```
In [45]: gam_logit_ = LogisticGAM(age_term +
 year_term +
 f_gam(2, lam=0))
gam_logit_.fit(Xgam_, high_earn_)
```

```
Out[45]: LogisticGAM(callbacks=[Deviance(), Diffs(), Accuracy()],
 fit_intercept=True, max_iter=100,
 terms=s(0) + s(1) + f(2) + intercept, tol=0.0001, verbose=False)
```

Hãy xem effect của education, year và age lên high-earner status sau khi đã remove các observations nói trên.

```
In [46]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 2)
ax.set_xlabel('Education')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of high earner status on education',
 fontsize=20);
ax.set_xticklabels(Wage['education'].cat.categories[1:],
 fontsize=8);
```

```
In [47]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 1)
ax.set_xlabel('Year')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of high earner status on year',
 fontsize=20);
```

```
In [48]: fig, ax = subplots(figsize=(8, 8))
ax = plot_gam(gam_logit_, 0)
ax.set_xlabel('Age')
ax.set_ylabel('Effect on wage')
ax.set_title('Partial dependence of high earner status on age',
 fontsize=20);
```

### 7.8.4 Local Regression

Ta minh họa local regression bằng function `lowess()`. Function này nằm trong module `sm.nonparametric`. Một số implementation của GAM cho phép terms là local regression operators; `pygam` thì không.

Ở đây ta fit local linear regression models với spans 0.2 và 0.5; tức mỗi neighborhood gồm 20% hoặc 50% observations. Như kỳ vọng, span 0.5 cho curve smooth hơn 0.2.

```
In [49]: lowess = sm.nonparametric.lowess
fig, ax = subplots(figsize=(8,8))
ax.scatter(age, y, facecolor='gray', alpha=0.5)
for span in [0.2, 0.5]:
 fitted = lowess(y,
 age,
 frac=span,
 xvals=age_grid)
 ax.plot(age_grid,
 fitted,
```

*lowess()*

```

label='{:.1f}'.format(span),
linewidth=4)
ax.set_xlabel('Age', fontsize=20)
ax.set_ylabel('Wage', fontsize=20);
ax.legend(title='span', fontsize=15);

```

## 7.9 Exercises

### Conceptual



1. Trong chapter này đã nói rằng cubic regression spline với một knot tại  $\xi$  có thể được tạo bằng basis  $x, x^2, x^3, (x-\xi)_+^3$ , trong đó  $(x-\xi)_+^3 = (x-\xi)^3$  nếu  $x > \xi$  và bằng 0 nếu không. Ta sẽ chứng minh một function dạng

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)_+^3$$

thực sự là cubic regression spline, bất kể values của  $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$ .

- (a) Tìm cubic polynomial

$$f_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3$$

sao cho  $f(x) = f_1(x)$  với mọi  $x \leq \xi$ . Biểu diễn  $a_1, b_1, c_1, d_1$  theo  $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$ .

- (b) Tìm cubic polynomial

$$f_2(x) = a_2 + b_2 x + c_2 x^2 + d_2 x^3$$

sao cho  $f(x) = f_2(x)$  với mọi  $x > \xi$ . Biểu diễn  $a_2, b_2, c_2, d_2$  theo  $\beta_0, \beta_1, \beta_2, \beta_3, \beta_4$ . Như vậy ta đã establish rằng  $f(x)$  là piecewise polynomial.

- (c) Chứng minh  $f_1(\xi) = f_2(\xi)$ ; tức  $f(x)$  continuous tại  $\xi$ .

- (d) Chứng minh  $f'_1(\xi) = f'_2(\xi)$ ; tức  $f'(x)$  continuous tại  $\xi$ .

- (e) Chứng minh  $f''_1(\xi) = f''_2(\xi)$ ; tức  $f''(x)$  continuous tại  $\xi$ .

Do đó  $f(x)$  thực sự là cubic spline.

*Hint:* Parts (d) và (e) cần kiến thức single-variable calculus. Nhắc lại, với cubic polynomial

$$f_1(x) = a_1 + b_1 x + c_1 x^2 + d_1 x^3,$$

first derivative là

$$f'_1(x) = b_1 + 2c_1 x + 3d_1 x^2,$$

và second derivative là

$$f''_1(x) = 2c_1 + 6d_1 x.$$

2. Giả sử một curve  $\hat{g}$  được compute để smoothly fit một set gồm  $n$  points bằng formula

$$\hat{g} = \arg \min_g \left\{ \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int [g^{(m)}(x)]^2 dx \right\},$$

trong đó  $g^{(m)}$  là  $m$ th derivative của  $g$  (và  $g^{(0)} = g$ ). Hãy sketch example của  $\hat{g}$  trong các scenario sau.

- (a)  $\lambda = \infty, m = 0$ .  
 (b)  $\lambda = \infty, m = 1$ .  
 (c)  $\lambda = \infty, m = 2$ .  
 (d)  $\lambda = \infty, m = 3$ .  
 (e)  $\lambda = 0, m = 3$ .
3. Giả sử ta fit một curve với basis functions  $b_1(X) = X$ ,  $b_2(X) = (X - 1)^2 I(X \geq 1)$ , trong đó  $I(X \geq 1)$  bằng 1 nếu  $X \geq 1$  và 0 nếu không. Ta fit linear regression model

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon,$$

và thu được coefficient estimates  $\hat{\beta}_0 = 1$ ,  $\hat{\beta}_1 = 1$ ,  $\hat{\beta}_2 = -2$ . Hãy sketch estimated curve từ  $X = -2$  đến  $X = 2$ , ghi rõ intercepts, slopes và các thông tin liên quan khác.

4. Giả sử ta fit một curve với basis functions

$$b_1(X) = I(0 \leq X \leq 2) - (X - 1)I(1 \leq X \leq 2),$$

$$b_2(X) = (X - 3)I(3 \leq X \leq 4) + I(4 < X \leq 5).$$

Ta fit linear regression model

$$Y = \beta_0 + \beta_1 b_1(X) + \beta_2 b_2(X) + \epsilon,$$

và thu được  $\hat{\beta}_0 = 1$ ,  $\hat{\beta}_1 = 1$ ,  $\hat{\beta}_2 = 3$ . Hãy sketch estimated curve từ  $X = -2$  đến  $X = 6$ , ghi rõ intercepts, slopes và các thông tin liên quan khác.

5. Xét hai curves  $\hat{g}_1$  và  $\hat{g}_2$  được định nghĩa bởi

$$\hat{g}_1 = \arg \min_g \left\{ \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int [g^{(3)}(x)]^2 dx \right\},$$

$$\hat{g}_2 = \arg \min_g \left\{ \sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int [g^{(4)}(x)]^2 dx \right\},$$

trong đó  $g^{(m)}$  là  $m$ th derivative của  $g$ .

- (a) Khi  $\lambda \rightarrow \infty$ ,  $\hat{g}_1$  hay  $\hat{g}_2$  có training RSS nhỏ hơn?  
 (b) Khi  $\lambda \rightarrow \infty$ ,  $\hat{g}_1$  hay  $\hat{g}_2$  có test RSS nhỏ hơn?  
 (c) Với  $\lambda = 0$ ,  $\hat{g}_1$  hay  $\hat{g}_2$  có training và test RSS nhỏ hơn?

### Applied

6. Trong exercise này, hãy phân tích thêm data set **Wage** đã dùng xuyên suốt chapter.
- (a) Thực hiện polynomial regression để predict **wage** bằng **age**. Dùng cross-validation chọn optimal degree  $d$  của polynomial. Degree nào được chọn, và kết quả này so với hypothesis testing dùng ANOVA ra sao? Vẽ plot của polynomial fit thu được trên data.



- (b) Fit step function để predict **wage** bằng **age**, và dùng cross-validation chọn optimal number of cuts. Vẽ plot của fit thu được.
7. Data set **Wage** chứa nhiều feature khác chưa được explore trong chapter, như marital status (**maritl**), job class (**jobclass**) và các variable khác. Hãy explore relationship giữa một số predictor này với **wage**, rồi dùng non-linear fitting techniques để fit flexible models. Tạo plots của kết quả và viết summary về findings.
  8. Fit một số non-linear models đã khảo sát trong chapter cho data set **Auto**. Có evidence về non-linear relationships trong data set này hay không? Tạo các informative plots để justify câu trả lời.
  9. Câu hỏi này dùng variables **dis** (weighted mean của distances tới năm Boston employment centers) và **nox** (nitrogen oxides concentration theo parts per 10 million) từ data **Boston**. Ta treat **dis** là predictor và **nox** là response.
    - (a) Dùng function **poly()** từ module **ISLP.models** để fit cubic polynomial regression predict **nox** từ **dis**. Report regression output và plot data cùng polynomial fit.
    - (b) Plot polynomial fits cho một range các polynomial degrees (chẳng hạn 1 đến 10), và report associated residual sum of squares.
    - (c) Dùng cross-validation hoặc approach khác để chọn optimal degree cho polynomial, rồi giải thích kết quả.
    - (d) Dùng function **bs()** từ module **ISLP.models** để fit regression spline predict **nox** từ **dis**. Report output cho fit với four degrees of freedom. Bạn chọn knots thế nào? Plot fit thu được.
    - (e) Tiếp theo fit regression spline cho một range degrees of freedom, plot các fits và report RSS tương ứng. Mô tả results.
    - (f) Dùng cross-validation hoặc approach khác để chọn best degrees of freedom cho regression spline trên data này. Mô tả results.
  10. Câu hỏi này liên quan đến data set **College**.
    - (a) Chia data thành training set và test set. Dùng out-of-state tuition làm response và các variables còn lại làm predictors, thực hiện forward stepwise selection trên training set để identify một satisfactory model chỉ dùng subset predictors.
    - (b) Fit GAM trên training data, với out-of-state tuition là response và các features chọn ở step trước là predictors. Plot results và giải thích findings.
    - (c) Evaluate model thu được trên test set và giải thích results.
    - (d) Với variable nào, nếu có, có evidence về non-linear relationship với response?
  11. Trong Section 7.7 có nói GAMs thường được fit bằng backfitting. Ý tưởng của backfitting thực ra khá đơn giản. Ta sẽ explore backfitting trong context multiple linear regression.  
Giả sử ta muốn thực hiện multiple linear regression nhưng không có

software cho việc đó; chỉ có software cho simple linear regression. Ta dùng iterative approach sau: repeatedly giữ fixed mọi coefficient estimate trừ một estimate tại current value, và chỉ update coefficient đó bằng simple linear regression. Process tiếp tục đến convergence, tức đến khi coefficient estimates ngừng thay đổi. Hãy thử trên toy example.

- (a) Generate response  $Y$  và hai predictors  $X_1, X_2$  với  $n = 100$ .
- (b) Viết function `simple_reg()` nhận hai arguments `outcome` và `feature`, fit simple linear regression model với `outcome` và `feature` này, rồi return estimated intercept và slope.
- (c) Initialize `beta1` bằng value tùy ý; choice không quan trọng.
- (d) Giữ `beta1` fixed, dùng `simple_reg()` fit model

$$Y - \text{beta1} \cdot X_1 = \beta_0 + \beta_2 X_2 + \epsilon.$$

Lưu values thu được thành `beta0` và `beta2`.

- (e) Giữ `beta2` fixed, fit model

$$Y - \text{beta2} \cdot X_2 = \beta_0 + \beta_1 X_1 + \epsilon.$$

Lưu kết quả thành `beta0` và `beta1`, overwrite previous values.

- (f) Viết for loop lặp (c) và (d) 1,000 lần. Report estimates của `beta0`, `beta1`, `beta2` ở mỗi iteration. Tạo plot hiển thị từng value của ba estimates này.
  - (g) Compare answer trong (e) với results khi đơn giản dùng multiple linear regression để predict  $Y$  bằng  $X_1$  và  $X_2$ . Dùng method `axline()` overlay các multiple linear regression coefficient estimates đó lên plot ở (e).
  - (h) Trên data set này, cần bao nhiêu backfitting iterations để có một “good” approximation cho multiple regression coefficient estimates?
12. Problem này tiếp tục exercise trước. Trong toy example với  $p = 100$ , hãy chứng minh có thể approximate multiple linear regression coefficient estimates bằng cách repeatedly thực hiện simple linear regression trong backfitting procedure. Cần bao nhiêu backfitting iterations để đạt “good” approximation cho multiple regression coefficient estimates? Tạo plot để justify câu trả lời.

# 8

## Tree-Based Methods

Trong chapter này, ta mô tả tree-based methods cho regression và classification. Các method này stratify hoặc segment predictor space thành một số region đơn giản. Để predict cho một observation, ta thường dùng mean hoặc mode response value của training observations nằm trong region chứa observation đó. Vì set of splitting rules dùng để segment predictor space có thể được summarize bằng một tree, các approach kiểu này được gọi là *decision tree methods*.

*decision tree*

Tree-based methods đơn giản và hữu ích cho interpretation. Tuy nhiên, xét về prediction accuracy, chúng thường không cạnh tranh được với các supervised learning approaches tốt nhất như những approach ở Chapters 6 và 7. Vì vậy chapter này còn giới thiệu bagging, random forests, boosting và Bayesian additive regression trees. Mỗi approach tạo nhiều trees rồi combine chúng thành một consensus prediction duy nhất. Ta sẽ thấy việc combine số lượng lớn trees thường cải thiện prediction accuracy rất mạnh, đổi lại là một phần interpretability.

### 8.1 The Basics of Decision Trees

Decision trees có thể dùng cho cả regression lẫn classification problems. Trước hết ta xét regression, rồi chuyển sang classification.

#### 8.1.1 Regression Trees

Để motivate regression trees, ta bắt đầu bằng một simple example.

*regression tree*

##### *Predicting Baseball Players' Salaries Using Regression Trees*

Ta dùng data set **Hitters** để predict **Salary** của một baseball player dựa trên **Years** (số năm chơi ở major leagues) và **Hits** (số hits ở năm trước). Trước hết remove observations thiếu **Salary**, rồi log-transform **Salary** để distribution có dạng bell-shape điển hình hơn. Nhớ rằng **Salary** được đo theo thousands of dollars.

Figure 8.1 cho regression tree fit vào data. Tree gồm series splitting rules bắt đầu từ top. Top split gán observations có **Years**<4.5 vào left branch.<sup>1</sup>

---

<sup>1</sup>Cả **Years** và **Hits** đều là integers trong data này; function dùng để fit tree label splits tại



**HÌNH 8.1.** Với data *Hitters*, regression tree dùng để predict log salary của một baseball player dựa trên số năm anh ta chơi ở major leagues và số hits trong năm trước. Tại một internal node, label dạng  $X_j < t_k$  chỉ left-hand branch của split; right-hand branch tương ứng  $X_j \geq t_k$ . Chẳng hạn top split tạo hai branch lớn: left branch là *Years*<4.5, right branch là *Years*>=4.5. Tree có hai internal nodes và ba terminal nodes, hay leaves. Số trong mỗi leaf là mean response của observations rơi vào leaf đó.

Predicted salary cho những players này bằng mean response của training observations có *Years*<4.5. Mean log salary là 5.107, nên prediction là  $e^{5.107}$  thousand dollars, tức \$165,174. Players có *Years*>=4.5 đi sang right branch, rồi group này tiếp tục được subdivide theo *Hits*. Nhìn chung tree stratify/segment players thành ba regions của predictor space: players chơi bốn năm hoặc ít hơn; players chơi ít nhất năm năm và có dưới 118 hits ở năm trước; và players chơi ít nhất năm năm với ít nhất 118 hits ở năm trước. Ba regions là

$$R_1 = \{X \mid \text{Years} < 4.5\}, \quad R_2 = \{X \mid \text{Years} \geq 4.5, \text{Hits} < 117.5\},$$

$$R_3 = \{X \mid \text{Years} \geq 4.5, \text{Hits} \geq 117.5\}.$$

Figure 8.2 minh họa các regions theo *Years* và *Hits*.

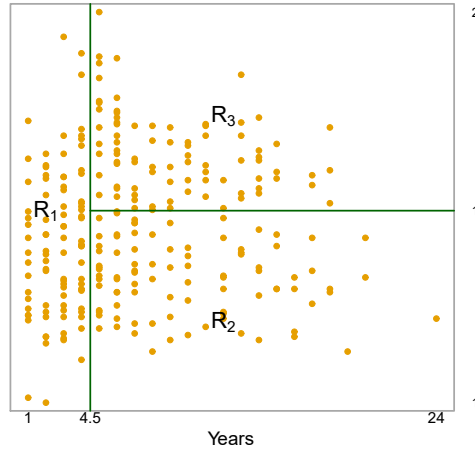
Predicted salaries cho ba group này lần lượt là  $\$1,000 \times e^{5.107} = \$165,174$ ,  $\$1,000 \times e^{5.999} = \$402,834$ , và  $\$1,000 \times e^{6.740} = \$845,346$ .

Theo analogy với tree, regions  $R_1, R_2, R_3$  được gọi là *terminal nodes* hay *leaves*. Decision trees thường được vẽ upside down: leaves ở bottom. Các points nơi predictor space bị split gọi là *internal nodes*. Trong Figure 8.1, hai internal nodes được chỉ bằng text *Years*<4.5 và *Hits*<117.5. Các segments nối nodes gọi là *branches*.

Có thể interpret tree ở Figure 8.1 như sau: *Years* là factor quan trọng nhất quyết định *Salary*; players ít experience nhận salary thấp hơn players nhiều experience. Với players ít experience, số hits năm trước dường như đóng vai trò nhỏ. Nhưng với players đã ở major leagues ít nhất năm năm, số hits năm trước ảnh hưởng đến salary, và players có nhiều hits hơn thường nhận salary cao hơn. Regression tree ở Figure 8.1 có lẽ oversimplify true

terminal node  
leaf  
internal node  
branch

midpoint giữa hai adjacent values.



**HÌNH 8.2.** Three-region partition của data set *Hitters* từ regression tree trong Figure 8.1.

relationship giữa *Hits*, *Years*, và *Salary*, nhưng có advantages so với các regression models khác ở Chapters 3 và 6: dễ interpret và có graphical representation rõ ràng.

### Prediction via Stratification of the Feature Space

Bây giờ xét process build regression tree. Roughly speaking có hai steps.

1. Chia predictor space—set of possible values của  $X_1, X_2, \dots, X_p$ —thành  $J$  distinct, non-overlapping regions  $R_1, R_2, \dots, R_J$ .
2. Với mọi observation rơi vào region  $R_j$ , dùng cùng prediction: mean của response values cho training observations trong  $R_j$ .

Ví dụ, nếu Step 1 tạo hai regions  $R_1, R_2$ , mean response ở first region là 10 và ở second region là 20, thì với observation  $X = x$ , ta predict 10 nếu  $x \in R_1$  và 20 nếu  $x \in R_2$ .

Làm thế nào construct  $R_1, \dots, R_J$ ? Về lý thuyết các regions có thể có shape bất kỳ, nhưng để đơn giản và dễ interpret, ta divide predictor space thành high-dimensional rectangles hay boxes. Goal là tìm boxes  $R_1, \dots, R_J$  minimize RSS

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2, \quad (8.1)$$

trong đó  $\hat{y}_{R_j}$  là mean response của training observations trong  $j$ th box. Không khả thi computationally khi xét mọi possible partition của feature space thành  $J$  boxes. Vì vậy ta dùng top-down, greedy approach gọi là *recursive binary splitting*. Approach là top-down vì bắt đầu ở top of tree, nơi mọi observations cùng thuộc một region, rồi successively split predictor space; mỗi split tạo hai branch mới. Nó là greedy vì ở mỗi step ta chọn best split ngay tại step đó, thay vì look ahead để chọn split có thể dẫn đến tree tốt hơn ở future step.

Để perform recursive binary splitting, đầu tiên chọn predictor  $X_j$  và cutpoint  $s$  sao cho split predictor space thành  $\{X \mid X_j < s\}$  và  $\{X \mid X_j \geq$

*recursive binary  
splitting*

$s\}$  tạo reduction RSS lớn nhất. Ta xét mọi predictors  $X_1, \dots, X_p$  và mọi possible cutpoint  $s$ , rồi chọn pair cho lowest RSS. Cụ thể, với  $j, s$  bất kỳ, define

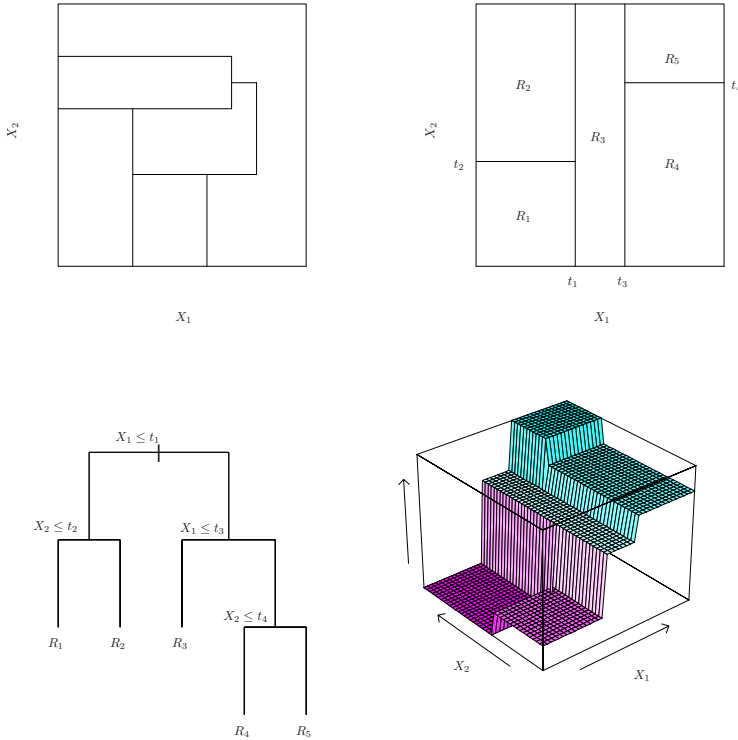
$$R_1(j, s) = \{X \mid X_j < s\} \quad \text{and} \quad R_2(j, s) = \{X \mid X_j \geq s\}, \quad (8.2)$$

và tìm  $j, s$  minimize

$$\sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2, \quad (8.3)$$

trong đó  $\hat{y}_{R_1}$  và  $\hat{y}_{R_2}$  là mean responses trong hai regions. Việc tìm  $j, s$  minimize (8.3) thực hiện khá nhanh, nhất là khi number of features  $p$  không quá lớn.

Tiếp theo repeat process, tìm best predictor và cutpoint để split data thêm sao cho minimize RSS trong resulting regions. Nhưng lần này chỉ split một trong hai regions đã có, tạo ba regions. Sau đó lại chọn một trong ba regions để split. Process tiếp tục đến stopping criterion; ví dụ, đến khi không region nào còn hơn năm observations.



**HÌNH 8.3.** Top Left: Một partition của two-dimensional feature space không thể sinh ra từ recursive binary splitting. Top Right: Output của recursive binary splitting trong một two-dimensional example. Bottom Left: Tree tương ứng với partition ở top right. Bottom Right: Perspective plot của prediction surface tương ứng tree đó.

Khi regions  $R_1, \dots, R_J$  đã được tạo, response cho một test observation

được predict bằng mean của training observations trong region chứa test observation đó. Figure 8.3 minh họa example năm regions.

### Tree Pruning

Process trên có thể predict tốt trên training set nhưng dễ overfit, dẫn đến poor test-set performance vì resulting tree quá complex. Một smaller tree với ít splits hơn, tức ít regions hơn, có thể giảm variance và tăng interpretability với cost là một chút bias. Có thể dùng grow tree khi reduction RSS từ mỗi split không vượt một high threshold, nhưng strategy này quá short-sighted: một split ban đầu có vẻ worthless có thể dẫn tới split rất tốt về sau.

Strategy tốt hơn là grow một very large tree  $T_0$ , rồi prune lại để lấy subtree. Goal trực giác là chọn subtree có lowest test error. Với một subtree, có thể estimate test error bằng cross-validation hoặc validation set. Tuy nhiên, compute CV error cho mọi possible subtree quá cumbersome vì số subtree rất lớn. Ta cần một cách chọn small set of subtrees để xét.

prune  
subtree

*Cost complexity pruning*, còn gọi là *weakest link pruning*, làm đúng việc này. Thay vì mọi subtree, xét sequence of trees indexed bởi nonnegative tuning parameter  $\alpha$ . Với mỗi  $\alpha$ , có subtree  $T \subset T_0$  sao cho

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \quad (8.4)$$

là nhỏ nhất. Ở đây  $|T|$  là number of terminal nodes,  $R_m$  là rectangle tương ứng với  $m$ th terminal node, và  $\hat{y}_{R_m}$  là predicted response của  $R_m$ , tức mean của training observations trong region đó. Parameter  $\alpha$  control trade-off giữa complexity của subtree và fit với training data. Khi  $\alpha = 0$ , subtree chính là  $T_0$  vì (8.4) chỉ measure training error. Khi  $\alpha$  tăng, tree có nhiều terminal nodes bị penalty, nên smaller subtree có xu hướng minimize criterion. Equation 8.4 gọi nhớ lasso (6.7) ở Chapter 6, nơi formulation tương tự control complexity của linear model.

cost complexity  
pruning  
weakest link  
pruning

Khi tăng  $\alpha$  từ zero, branches bị prune theo nested, predictable fashion, nên sequence of subtrees theo  $\alpha$  dễ obtain. Ta chọn  $\alpha$  bằng validation set hoặc cross-validation, rồi quay lại full data set và lấy subtree tương ứng. Process được summarize trong Algorithm 8.1.

Figures 8.4 và 8.5 cho results của fitting và pruning regression tree trên **Hitters** dùng chín features. Trước hết data được random split thành 132 training observations và 131 test observations. Sau đó grow large regression tree trên training data và vary  $\alpha$  trong (8.4) để tạo subtrees với different numbers of terminal nodes. Cuối cùng dùng six-fold cross-validation để estimate CV MSE theo  $\alpha$ .

#### Algorithm 8.1 Building a Regression Tree

1. Dùng recursive binary splitting để grow large tree trên training data, chỉ dừng khi mỗi terminal node có ít hơn một minimum number of observations.
2. Apply cost complexity pruning lên large tree để obtain sequence of best subtrees theo  $\alpha$ .
3. Dùng  $K$ -fold cross-validation để chọn  $\alpha$ . Chia training observations

thành  $K$  folds. Với mỗi  $k = 1, \dots, K$ :

- (a) Repeat Steps 1 và 2 trên tất cả training data trừ  $k$ th fold.
- (b) Evaluate mean squared prediction error trên left-out  $k$ th fold theo  $\alpha$ .

Average results cho mỗi  $\alpha$  và chọn  $\alpha$  minimize average error.

4. Return subtree từ Step 2 tương ứng với chosen  $\alpha$ .

Ta dùng six-fold CV vì 132 chia hết cho six. Unpruned regression tree được show ở Figure 8.4. Green curve trong Figure 8.5 là CV error theo number of leaves,<sup>2</sup> còn orange curve là test error; standard error bars cũng được show. Training error curve màu black làm reference. CV error là approximation hợp lý của test error: CV minimum tại three-node tree, còn test error cũng giảm mạnh ở three-node tree dù minimum thật nằm ở ten-node tree. Pruned tree với ba terminal nodes chính là Figure 8.1.

### 8.1.2 Classification Trees

Classification tree rất giống regression tree, trừ việc predict qualitative response thay vì quantitative response. Với regression tree, prediction cho observation là mean response của training observations cùng terminal node. Với classification tree, ta predict observation thuộc class xuất hiện nhiều nhất trong training observations của region đó. Khi interpret classification tree, ngoài class prediction ở mỗi terminal-node region ta thường quan tâm class proportions trong training observations của region.

Growing classification tree cũng gần như regression tree và vẫn dùng recursive binary splitting. Nhưng RSS không thể làm criterion cho binary splits. Một alternative tự nhiên là classification error rate. Vì observation ở region được assign vào most commonly occurring class, classification error rate là fraction training observations trong region không thuộc most common class:

$$E = 1 - \max_k (\hat{p}_{mk}). \quad (8.5)$$

Ở đây  $\hat{p}_{mk}$  là proportion training observations trong  $m$ th region thuộc  $k$ th class. Tuy nhiên classification error không đủ sensitive để grow tree, nên thực tế thường dùng hai measures khác.

Gini index được định nghĩa là

$$G = \sum_{k=1}^K \hat{p}_{mk}(1 - \hat{p}_{mk}), \quad (8.6)$$

một measure của total variance trên  $K$  classes. Gini index nhỏ nếu mọi  $\hat{p}_{mk}$  gần zero hoặc one; vì vậy nó là measure của *node purity*—small value nghĩa là node chủ yếu chứa observations từ một class.

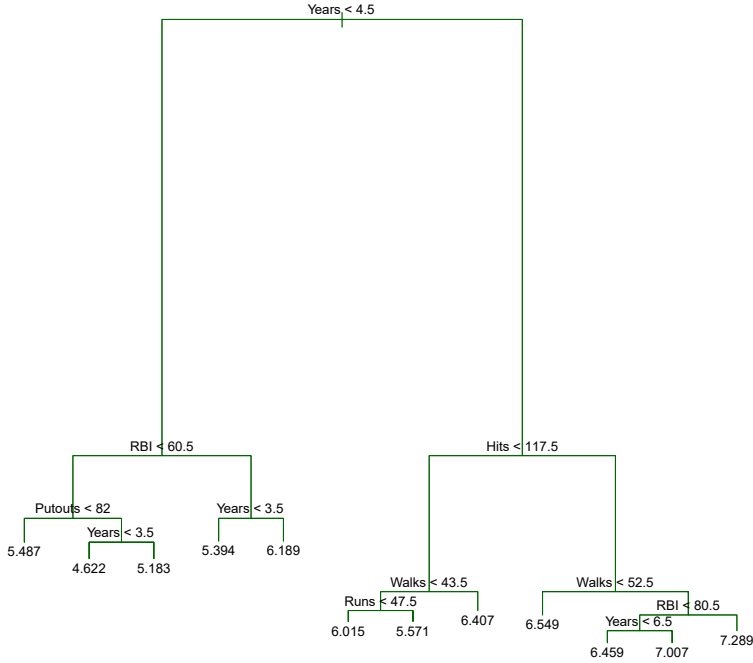
<sup>2</sup>Dù CV error được compute theo  $\alpha$ , tiện hơn khi display theo  $|T|$ , number of leaves; dựa trên relation giữa  $\alpha$  và  $|T|$  trong original tree grow từ toàn bộ training data.

classification tree

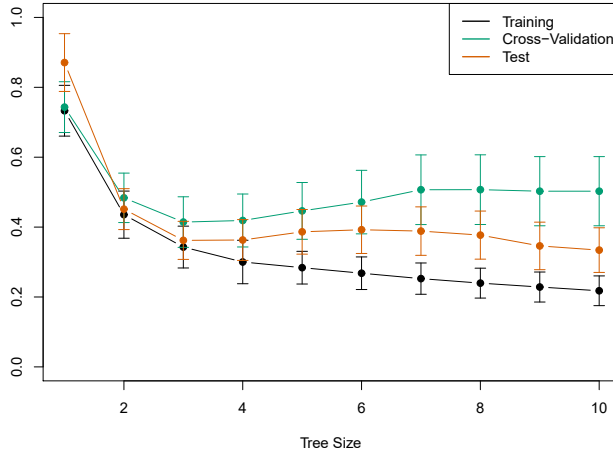
classification error  
rate

Gini index





**HÌNH 8.4.** Regression tree analysis cho data *Hitters*. Unpruned tree sinh ra từ top-down greedy splitting trên training data.



**HÌNH 8.5.** Regression tree analysis cho data *Hitters*. Training, cross-validation và test MSE được show theo number of terminal nodes trong pruned tree. Standard error bands được display. Minimum cross-validation error xảy ra ở tree size bằng three.

Một alternative cho Gini index là entropy,

$$D = - \sum_{k=1}^K \hat{p}_{mk} \log \hat{p}_{mk}. \quad (8.7)$$

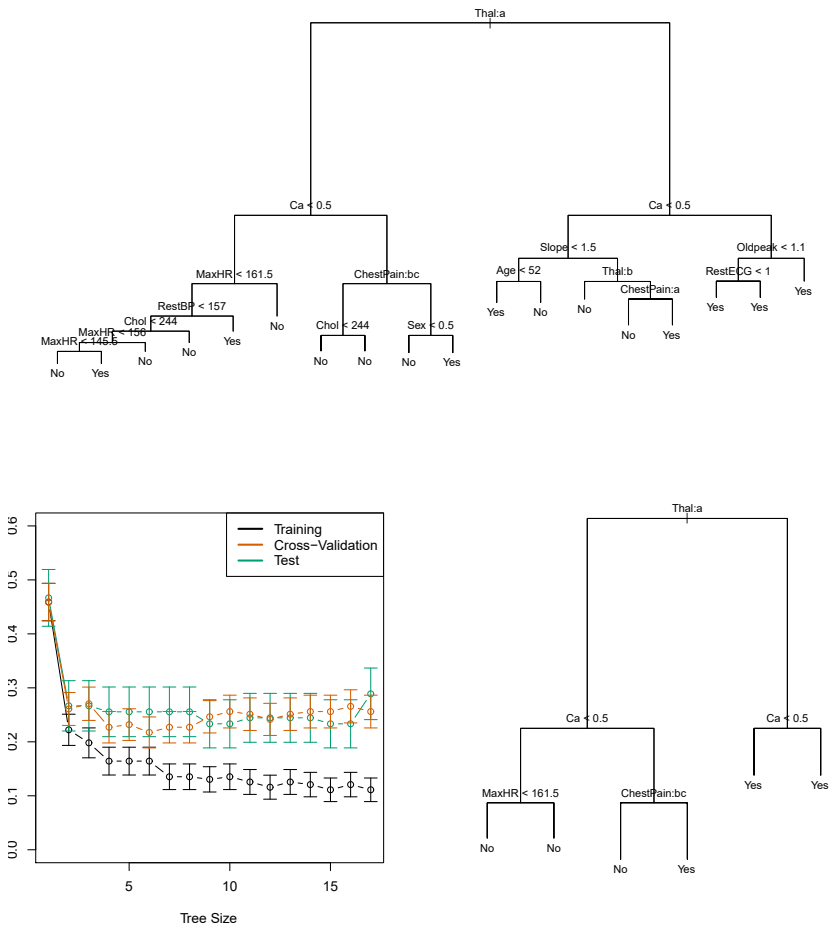
entropy

Vì  $0 \leq \hat{p}_{mk} \leq 1$ , nên  $0 \leq -\hat{p}_{mk} \log \hat{p}_{mk}$ . Entropy gần zero nếu các  $\hat{p}_{mk}$  đều gần zero hoặc one. Vì thế, giống Gini index, entropy nhỏ khi  $m$ th node pure. Trên thực tế Gini index và entropy khá similar numerically.

Khi build classification tree, Gini index hoặc entropy thường được dùng để evaluate quality của split vì sensitive với node purity hơn classification error rate. Khi prune tree có thể dùng cả ba, nhưng nếu goal là prediction accuracy của final pruned tree thì classification error rate preferable.

Figure 8.6 minh họa trên data set **Heart**. Data có binary outcome **HD** cho 303 patients đến khám vì chest pain. Outcome **Yes** nghĩa là có heart disease theo angiographic test, **No** nghĩa là không. Có 13 predictors gồm **Age**, **Sex**, **Chol** và các heart/lung function measurements khác. Cross-validation cho tree với six terminal nodes.

Đến đây ta giả định predictors continuous, nhưng decision trees vẫn construct được với qualitative predictors. Trong **Heart**, một số predictors như **Sex**, **Thal** (Thallium stress test), và **ChestPain** là qualitative.



**HÌNH 8.6.** Data **Heart**. Top: Unpruned tree. Bottom Left: Cross-validation error, training error và test error cho different sizes của pruned tree. Bottom Right: Pruned tree tương ứng minimum cross-validation error.

Do đó, split trên qualitative variable tương đương assign một số qualitative values vào one branch và remaining values vào other branch. Trong Figure 8.6, top internal node split **Thal**. Text **Thal:a** nghĩa là left branch chứa observations với first value của **Thal** (normal), còn right branch chứa remaining observations (fixed hoặc reversible defects). Text **ChestPain:bc** ở hai splits phía dưới bên trái nghĩa là left branch chứa second và third values của **ChestPain**; possible values là typical angina, atypical angina, non-anginal pain và asymptomatic.

Figure 8.6 có characteristic hơi surprising: một số splits tạo hai terminal nodes với same predicted value. Ví dụ split **RestECG<1** gần bottom-right của unpruned tree. Bất kể **RestECG**, response **Yes** được predict. Split vẫn được thực hiện vì tăng node purity: toàn bộ 9 observations ở right-hand leaf là **Yes**, trong khi chỉ 7/11 observations ở left-hand leaf là **Yes**.

Tại sao node purity quan trọng? Nếu test observation nằm trong right-hand leaf đó, ta khá chắc response là **Yes**; nếu nằm trong left-hand leaf, response có lẽ là **Yes** nhưng uncertainty lớn hơn. Dù split **RestECG<1** không giảm classification error, nó cải thiện Gini index và entropy, vốn sensitive hơn với node purity.

### 8.1.3 Trees Versus Linear Models

Regression và classification trees có flavor rất khác các classical approaches trong Chapters 3 và 4. Linear regression assume model

$$f(X) = \beta_0 + \sum_{j=1}^p X_j \beta_j, \quad (8.8)$$

trong khi regression trees assume

$$f(X) = \sum_{m=1}^M c_m \cdot \mathbf{1}(X \in R_m), \quad (8.9)$$

trong đó  $R_1, \dots, R_M$  là partition của feature space như Figure 8.3.

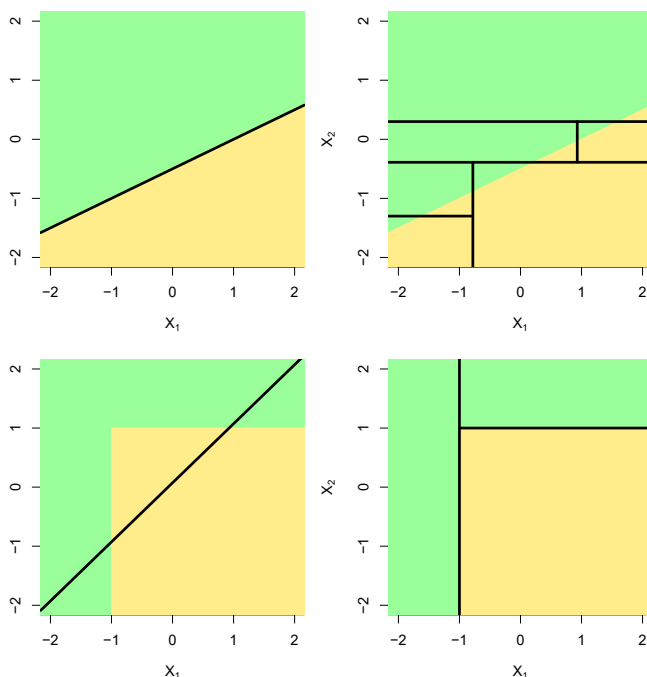
Model nào tốt hơn phụ thuộc problem. Nếu relationship giữa features và response được approximate tốt bởi linear model (8.8), linear regression thường work well và outperform regression tree vì tree không exploit linear structure. Nếu relationship highly non-linear và complex như (8.9), decision trees có thể outperform classical approaches. Figure 8.7 minh họa. Relative performance có thể assess bằng estimated test error qua cross-validation hoặc validation set approach (Chapter 5).

Ngoài test error còn các considerations khác khi chọn statistical learning method; chẳng hạn trong một số setting, prediction bằng tree được prefer vì interpretability và visualization.

### 8.1.4 Advantages and Disadvantages of Trees

Decision trees cho regression và classification có một số advantages so với classical approaches ở Chapters 3 và 4:

- ▲ Trees rất dễ explain cho người khác; thậm chí dễ explain hơn linear regression.



**HÌNH 8.7.** Top Row: Two-dimensional classification example với true decision boundary linear, thể hiện bằng shaded regions. Classical approach assume linear boundary (left) outperform decision tree với splits parallel to axes (right). Bottom Row: True decision boundary non-linear. Linear model không capture được boundary (left), còn decision tree thành công (right).

- ▲ Một số người cho rằng decision trees gần với human decision-making hơn regression/classification approaches ở các chapter trước.
- ▲ Trees có thể display graphically và dễ interpret ngay cả với non-expert, nhất là khi tree nhỏ.
- ▲ Trees handle qualitative predictors dễ dàng mà không cần create dummy variables.
- ▼ Tiếc là trees thường không đạt predictive accuracy ngang một số regression/classification approaches khác trong sách.
- ▼ Trees cũng có thể rất non-robust: small change trong data có thể gây large change trong final estimated tree.

Tuy nhiên, bằng cách aggregate nhiều decision trees qua bagging, random forests và boosting, predictive performance có thể cải thiện đáng kể. Các concepts này được giới thiệu ở section tiếp theo.

## 8.2 Bagging, Random Forests, Boosting, and Bayesian Additive Regression Trees

Ensemble method là approach combine nhiều simple “building block” models để obtain một single model có thể rất powerful. Các simple building blocks đôi khi được gọi là *weak learners*, vì từng model riêng lẻ chỉ có thể cho mediocre predictions.

*ensemble  
weak learners*

Ta sẽ discuss bagging, random forests, boosting và Bayesian additive regression trees. Đây là ensemble methods mà building block đơn giản là regression tree hoặc classification tree.

### 8.2.1 Bagging

Bootstrap, giới thiệu ở Chapter 5, là một idea cực kỳ powerful. Nó được dùng trong nhiều setting khi standard deviation của quantity cần quan tâm khó hoặc thậm chí không thể compute trực tiếp. Ở đây bootstrap được dùng theo context hoàn toàn khác: cải thiện statistical learning methods như decision trees.

Decision trees ở Section 8.1 có high variance. Nếu random split training data thành hai parts rồi fit decision tree trên mỗi half, results có thể rất khác nhau. Ngược lại, procedure có low variance cho similar results khi repeatedly apply lên distinct data sets; linear regression thường có low variance nếu ratio  $n/p$  moderately large. *Bootstrap aggregation*, hay *bagging*, là general-purpose procedure để reduce variance của statistical learning method; nó đặc biệt useful và commonly used với decision trees.

*bagging*

Nhớ rằng với  $n$  independent observations  $Z_1, \dots, Z_n$ , mỗi observation có variance  $\sigma^2$ , variance của mean  $\bar{Z}$  bằng  $\sigma^2/n$ . Nói cách khác, averaging observations làm giảm variance. Vì thế một natural way để giảm variance và tăng test-set accuracy là lấy nhiều training sets từ population, build separate prediction model trên mỗi training set, rồi average predictions. Nếu có  $B$  separate training sets, ta có thể compute  $\hat{f}_1(x), \dots, \hat{f}_B(x)$  và lấy

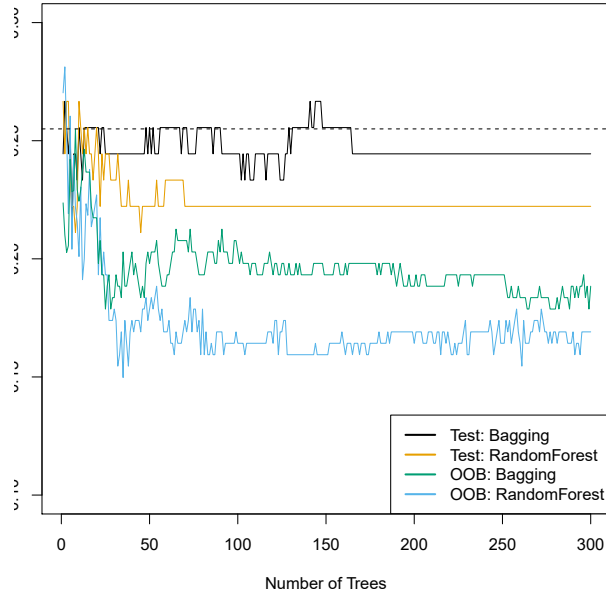
$$\hat{f}_{\text{avg}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x).$$

Dĩ nhiên điều này không practical vì thường chỉ có một training set. Thay vào đó, ta bootstrap bằng repeated samples từ single training data set. Generate  $B$  bootstrapped training data sets; train method trên  $b$ th bootstrapped set để có  $\hat{f}_b^*(x)$ , rồi average:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b^*(x).$$

Đây chính là bagging.

Bagging có thể improve predictions cho nhiều regression methods, nhưng đặc biệt useful với decision trees. Với regression trees, construct  $B$  regression trees từ  $B$  bootstrapped training sets rồi average predictions. Các trees này được grow deep và không prune, nên mỗi individual tree có high variance nhưng low bias; averaging  $B$  trees làm giảm variance. Bagging đã



**HÌNH 8.8.** Bagging và random forest results cho data *Heart*. Test error (black và orange) được show theo  $B$ , number of bootstrapped training sets. Random forests dùng  $m = \sqrt{p}$ . Dashed line là test error từ một single classification tree. Green và blue traces là OOB error, tình cờ thấp hơn đáng kể trong example này.

cho improvements ấn tượng bằng cách combine hàng trăm hoặc hàng nghìn trees thành một procedure duy nhất.

Đến đây bagging được mô tả cho regression, với quantitative outcome  $Y$ . Với classification, response  $Y$  qualitative, cách đơn giản nhất là ghi lại class predicted bởi mỗi trong  $B$  trees cho một test observation rồi lấy *majority vote*: overall prediction là class xuất hiện nhiều nhất trong  $B$  predictions.

Figure 8.8 show results từ bagging trees trên data *Heart*. Test error rate được show theo  $B$ , number of trees built từ bootstrapped training data. Bagging test error hơi thấp hơn single-tree test error trong case này. Number of trees  $B$  không phải critical parameter: dùng  $B$  rất lớn không gây overfitting. Trong practice, chọn  $B$  đủ lớn để error đã settle down;  $B = 100$  đủ cho good performance trong example.

### Out-of-Bag Error Estimation

Có một cách rất straightforward để estimate test error của bagged model mà không cần cross-validation hay validation set. Key của bagging là trees repeated fit trên bootstrapped subsets. Trung bình mỗi bagged tree dùng khoảng two-thirds observations.<sup>3</sup> One-third còn lại không dùng để fit tree được gọi là *out-of-bag (OOB) observations*.

Ta có thể predict response cho  $i$ th observation bằng từng tree trong đó observation đó là OOB, tạo khoảng  $B/3$  predictions. Để có single prediction, average predicted responses nếu là regression hoặc take majority vote nếu là classification. Làm vậy cho mọi  $n$  observations rồi compute overall

<sup>3</sup>Điều này liên quan tới Exercise 2 của Chapter 5.

majority vote

out-of-bag

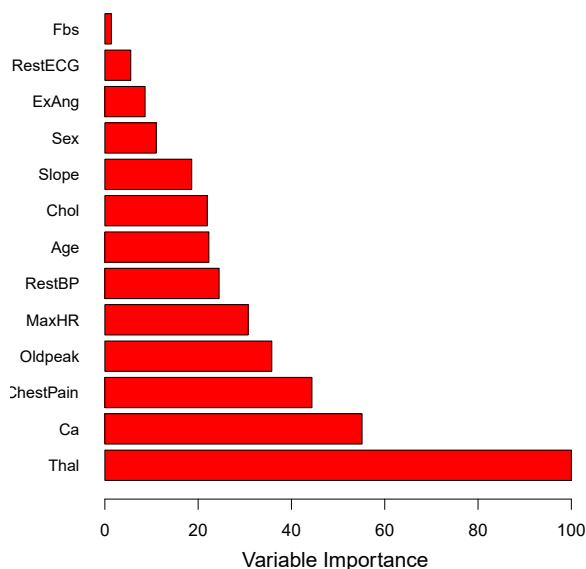
OOB MSE hoặc classification error. OOB error là valid estimate của test error vì response của mỗi observation chỉ được predict bởi trees không dùng observation đó khi fit.

Figure 8.8 display OOB error trên data **Heart**. Với  $B$  đủ lớn, OOB error gần như equivalent với leave-one-out cross-validation error. OOB estimation đặc biệt convenient khi bagging trên large data sets nơi cross-validation computationally onerous.

### Variable Importance Measures

Bagging thường tăng accuracy so với single tree, nhưng model result khó interpret. Một advantage của decision tree là diagram attractive và dễ hiểu như Figure 8.1. Khi bag rất nhiều trees, không thể represent whole procedure bằng một tree duy nhất, và không còn obvious variable nào important. Vì vậy bagging improve prediction accuracy với cost là interpretability.

Dù collection of bagged trees khó interpret hơn single tree, ta vẫn có overall summary của importance cho mỗi predictor qua RSS (bagging regression trees) hoặc Gini index (bagging classification trees). Với regression trees, record total amount RSS trong (8.1) giảm do splits trên một predictor, average trên  $B$  trees; large value chỉ important predictor. Với classification trees, cộng total amount Gini index (8.6) giảm do splits trên predictor đó, rồi average trên  $B$  trees.



**HÌNH 8.9.** Variable importance plot cho data **Heart**. Variable importance được compute bằng mean decrease in Gini index và biểu diễn relative to maximum.

Graphical representation của variable importances trong data **Heart** được show ở Figure 8.9. Ta thấy mean decrease in Gini index cho mỗi variable, relative to largest. Các variables có largest mean decrease in Gini index là Thal, Ca, và ChestPain.

variable importance

### 8.2.2 Random Forests

Random forests cải thiện bagged trees bằng một small tweak nhằm decorrelate các trees. Giống bagging, ta build nhiều decision trees trên bootstrapped training samples. Tuy nhiên, khi build các decision trees này, mỗi lần xét một split trong tree, một random sample gồm  $m$  predictors được chọn làm split candidates từ full set gồm  $p$  predictors. Split chỉ được phép dùng một trong  $m$  predictors đó. Một fresh sample gồm  $m$  predictors được lấy tại mỗi split, và thông thường chọn  $m \approx \sqrt{p}$ —tức number of predictors được xét tại mỗi split xấp xỉ square root của tổng number of predictors (4 trong số 13 đối với data **Heart**).

*random forest*

Nói cách khác, khi build random forest, tại mỗi split algorithm thậm chí không được phép xét phần lớn available predictors. Điều này nghe có vẻ lạ nhưng có rationale rất clever. Giả sử data set có một predictor rất strong cùng một số predictors moderately strong. Trong collection of bagged trees, phần lớn hoặc tất cả trees sẽ dùng strong predictor này ở top split. Kết quả là các bagged trees trông rất giống nhau và predictions từ chúng sẽ highly correlated.

Averaging nhiều highly correlated quantities không giảm variance nhiều như averaging nhiều uncorrelated quantities. Do đó trong setting này, bagging có thể không giảm variance đáng kể so với single tree. Random forests giải quyết vấn đề bằng cách buộc mỗi split chỉ xét một subset predictors. Trung bình  $(p-m)/p$  splits thậm chí không xét strong predictor, nên các predictors khác có thêm cơ hội được chọn. Có thể xem process này là decorrelating the trees, làm average của resulting trees ít variable hơn và reliable hơn.

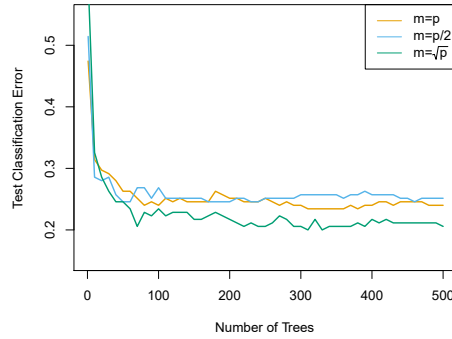
Main difference giữa bagging và random forests là predictor subset size  $m$ . Nếu random forest dùng  $m = p$  thì chính là bagging. Trên data **Heart**, random forests với  $m = \sqrt{p}$  giảm cả test error lẫn OOB error so với bagging (Figure 8.8).

Small  $m$  thường hữu ích khi có nhiều correlated predictors. Random forests được apply lên high-dimensional biological data set gồm expression measurements của 4,718 genes trên tissue samples từ 349 patients. Con người có khoảng 20,000 genes, và từng gene có levels of activity hay expression khác nhau trong particular cells, tissues và biological conditions. Mỗi patient sample trong data này có qualitative label với 15 levels: normal hoặc 1 trong 14 cancer types. Goal là predict cancer type bằng random forests dựa trên 500 genes có largest variance trong training set.

Ta random divide observations thành training và test set, rồi apply random forests lên training set với ba values khác nhau của number of splitting variables  $m$ . Results ở Figure 8.10. Error rate của single tree là 45.7%, null rate là 75.4%.<sup>4</sup> Ta thấy 400 trees là đủ cho good performance và choice  $m = \sqrt{p}$  cải thiện test error nhẹ so với bagging ( $m = p$ ). Giống bagging, random forests không overfit khi tăng  $B$ , nên trong practice ta chọn  $B$  đủ lớn để error rate đã settle down.

<sup>4</sup>Null rate có được khi đơn giản classify mọi observation vào dominant class overall, trong case này là normal class.





**HÌNH 8.10.** Results từ random forests cho 15-class gene expression data set với  $p = 500$  predictors. Test error được display theo number of trees. Mỗi colored line ứng với một value khác nhau của  $m$ , number of predictors available for splitting tại mỗi interior tree node. Random forests ( $m < p$ ) cho cải thiện nhẹ so với bagging ( $m = p$ ). Một single classification tree có error rate 45.7%.

### 8.2.3 Boosting

Ta tiếp tục với boosting, một approach khác để improve predictions từ decision tree. Giống bagging, boosting là general approach có thể apply cho nhiều statistical learning methods trong regression hoặc classification. Ở đây ta giới hạn discussion vào decision trees.

boosting

Bagging tạo multiple copies của original training data set bằng bootstrap, fit một separate decision tree trên mỗi copy, rồi combine tất cả trees thành single predictive model. Mỗi tree được build trên bootstrap data set độc lập với các trees khác. Boosting tương tự nhưng trees được grow sequentially: mỗi tree được grow dùng information từ previously grown trees. Boosting không dùng bootstrap sampling; mỗi tree được fit trên modified version của original data set.

Trước hết xét regression. Giống bagging, boosting combine một large number of decision trees  $\hat{f}^1, \dots, \hat{f}^B$ . Procedure được mô tả trong Algorithm 8.2. Khác với fit một single large decision tree—fit data rất hard và có thể overfit—boosting học chậm. Với current model, ta fit decision tree lên residuals của model, tức dùng current residuals thay vì outcome  $Y$  làm response. Sau đó add new tree vào fitted function để update residuals.

Mỗi tree có thể khá small, chỉ vài terminal nodes, controlled bởi parameter  $d$ . Bằng cách fit small trees vào residuals, ta từ từ improve  $\hat{f}$  ở nơi nó chưa perform tốt. Shrinkage parameter  $\lambda$  làm process còn chậm hơn, cho phép nhiều trees có shapes khác nhau cùng attack residuals. Nhìn chung statistical learning approaches học chậm thường perform tốt. Trong boosting, trái với bagging, construction của mỗi tree phụ thuộc strongly vào trees đã được grow trước đó. Boosting classification trees tương tự nhưng phức tạp hơn một chút; details được omitted.

#### Algorithm 8.2 Boosting for Regression Trees

1. Set  $\hat{f}(x) = 0$  và  $r_i = y_i$  cho mọi  $i$  trong training set.
2. Với  $b = 1, 2, \dots, B$ , repeat:

- (a) Fit một tree  $\hat{f}^b$  với  $d$  splits ( $d + 1$  terminal nodes) vào training data  $(X, r)$ .
- (b) Update  $\hat{f}$  bằng cách add một shrunken version của new tree:

$$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \quad (8.10)$$

- (c) Update residuals,

$$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \quad (8.11)$$

- 3. Output boosted model,

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x). \quad (8.12)$$

Boosting có ba tuning parameters:

1. Number of trees  $B$ . Không giống bagging và random forests, boosting có thể overfit nếu  $B$  quá lớn, dù overfitting thường diễn ra chậm hoặc thậm chí không xảy ra. Ta dùng cross-validation để select  $B$ .
2. Shrinkage parameter  $\lambda$ , một small positive number controlling rate at which boosting learns. Typical values là 0.01 hoặc 0.001, và choice phù hợp phụ thuộc problem. Very small  $\lambda$  có thể yêu cầu very large  $B$  để đạt good performance.
3. Number  $d$  of splits trong mỗi tree, controlling complexity của boosted ensemble. Thường  $d = 1$  works well; mỗi tree khi đó là một *stump*, gồm single split. Trong case này boosted ensemble fit một additive model vì mỗi term chỉ involve một variable. Tổng quát hơn  $d$  là *interaction depth*, controlling interaction order của boosted model, vì  $d$  splits involve nhiều nhất  $d$  variables.

*stump; interaction  
depth*

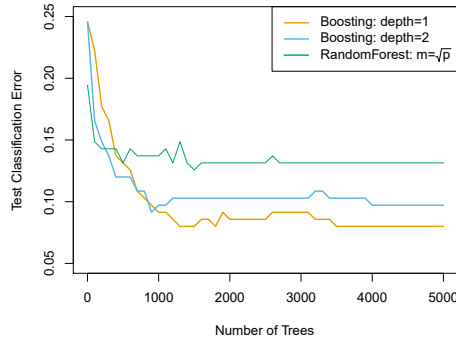
Trong Figure 8.11, boosting được apply lên 15-class cancer gene expression data set để develop classifier phân biệt normal class với 14 cancer classes. Test error được display theo total number of trees và interaction depth  $d$ . Simple stumps với depth one perform tốt nếu include đủ nhiều. Model này outperform depth-two model, và cả hai outperform random forest. Điều này highlight một difference giữa boosting và random forests: vì growth của particular tree trong boosting có xét tới các trees đã grow, smaller trees thường sufficient. Smaller trees cũng giúp interpretability; chẳng hạn stumps dẫn tới additive model.

#### 8.2.4 Bayesian Additive Regression Trees

Cuối cùng ta discuss Bayesian additive regression trees (BART), một ensemble method khác dùng decision trees làm building blocks. Để đơn giản, ta trình bày BART cho regression thay vì classification.

*Bayesian additive  
regression trees*

Bagging và random forests predict bằng average regression trees, mỗi tree được build từ random sample of data và/hoặc predictors; each tree built separately. Boosting dùng weighted sum of trees, mỗi tree fit vào residual của current fit, nên mỗi new tree cố capture signal chưa được current set of



**HÌNH 8.11.** Results từ boosting và random forests trên 15-class gene expression data set để predict cancer versus normal. Test error được display theo number of trees. Với hai boosted models,  $\lambda = 0.01$ . Depth-1 trees hơi outperform depth-2 trees và cả hai outperform random forest, dù standard errors khoảng 0.02 nên none of these differences significant. Test error rate của single tree là 24%.

trees account for. BART liên quan cả hai: mỗi tree constructed randomly như bagging/random forests và cũng cố capture signal chưa được current model account for như boosting. Main novelty của BART là cách generate new trees.

Ký hiệu  $K$  là number of regression trees và  $B$  là number of iterations BART chạy.  $\hat{f}_k^b(x)$  là prediction tại  $x$  của  $k$ th regression tree trong  $b$ th iteration. Cuối mỗi iteration,  $K$  trees được sum:

$$\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x), \quad b = 1, \dots, B.$$

Trong first iteration, all trees initialized với single root node và

$$\hat{f}_k^1(x) = \frac{1}{nK} \sum_{i=1}^n y_i,$$

mean response divided by total number of trees. Vì vậy  $\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = n^{-1} \sum_i y_i$ .

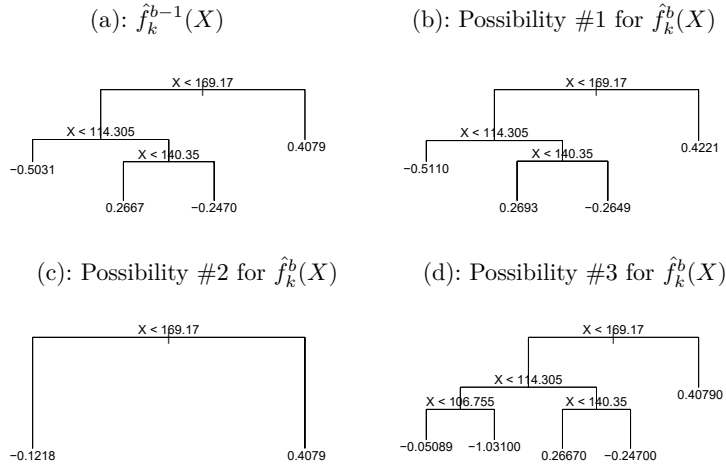
Ở subsequent iterations, BART update từng tree trong  $K$  trees, one at a time. Trong  $b$ th iteration, để update  $k$ th tree, ta subtract từ mỗi response value predictions của all trees trừ  $k$ th tree, tạo partial residual

$$r_i = y_i - \sum_{k' < k} \hat{f}_{k'}^b(x_i) - \sum_{k' > k} \hat{f}_{k'}^{b-1}(x_i), \quad i = 1, \dots, n.$$

Thay vì fit fresh tree vào partial residual này, BART randomly choose một perturbation của tree từ previous iteration  $\hat{f}_k^{b-1}$  từ set of possible perturbations, favoring perturbations improve fit. Perturbation có hai components:

1. Có thể change tree structure bằng adding hoặc pruning branches.
2. Có thể change prediction trong mỗi terminal node.

Figure 8.12 minh họa possible perturbations.



**HÌNH 8.12.** Schematic của perturbed trees từ BART algorithm. (a):  $k$ th tree ở  $(b-1)$ st iteration,  $\hat{f}_k^{b-1}(X)$ . Panels (b)–(d) show ba trong nhiều possibilities cho  $\hat{f}_k^b(X)$  given form của  $\hat{f}_k^{b-1}(X)$ . (b): Có thể giữ same tree structure nhưng terminal-node predictions khác. (c): Có thể prune  $\hat{f}_k^{b-1}(X)$ . (d): Hoặc  $\hat{f}_k^b(X)$  có nhiều terminal nodes hơn.

Output của BART là collection of prediction models

$$\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x), \quad b = 1, 2, \dots, B.$$

### Algorithm 8.3 Bayesian Additive Regression Trees

1. Let  $\hat{f}_1^1(x) = \hat{f}_2^1(x) = \dots = \hat{f}_K^1(x) = \frac{1}{nK} \sum_{i=1}^n y_i$ .

2. Compute  $\hat{f}^1(x) = \sum_{k=1}^K \hat{f}_k^1(x) = \frac{1}{n} \sum_{i=1}^n y_i$ .

3. Với  $b = 2, \dots, B$ :

(a) Với  $k = 1, 2, \dots, K$ :

i. Với  $i = 1, \dots, n$ , compute current partial residual

$$r_i = y_i - \sum_{k' < k} \hat{f}_{k'}^b(x_i) - \sum_{k' > k} \hat{f}_{k'}^{b-1}(x_i).$$

ii. Fit new tree  $\hat{f}_k^b(x)$  vào  $r_i$  bằng randomly perturbing  $k$ th tree từ previous iteration  $\hat{f}_k^{b-1}(x)$ . Perturbations improve fit được favored.

(b) Compute  $\hat{f}^b(x) = \sum_{k=1}^K \hat{f}_k^b(x)$ .

4. Compute mean sau  $L$  burn-in samples,

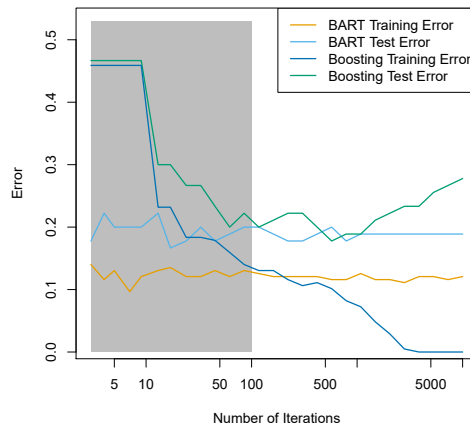
$$\hat{f}(x) = \frac{1}{B-L} \sum_{b=L+1}^B \hat{f}^b(x).$$

Ta thường discard vài prediction models đầu vì models ở early iterations—gọi là *burn-in period*—thường chưa give good results. Gọi  $L$  là number of burn-in iterations, chẳng hạn  $L = 200$ . Single prediction được lấy bằng average after burn-in. Nhưng cũng có thể compute quantities khác average: percentiles của  $\hat{f}^{L+1}(x), \dots, \hat{f}^B(x)$  provide measure of uncertainty trong final prediction.

*burn-in*

Key element của BART là trong Step 3(a)ii ta không fit fresh tree vào current partial residual. Thay vào đó, cố improve fit bằng slightly modifying tree ở previous iteration (Figure 8.12). Roughly speaking, điều này guards against overfitting vì giới hạn mức độ fit data “hard” ở mỗi iteration. Individual trees cũng thường small; limiting tree size giúp tránh overfitting.

Figure 8.13 show BART trên data **Heart** với  $K = 200$  trees, khi number of iterations tăng đến 10,000. Trong initial iterations, test và training errors jump around. Sau burn-in period, error rates settle down. Chỉ có small difference giữa training và test error, cho thấy tree perturbation process phần lớn avoid overfitting.



**HÌNH 8.13.** BART và boosting results cho data **Heart**. Cả training và test errors đều được display. Sau burn-in period 100 iterations (shown in gray), BART error rates settle down. Boosting bắt đầu overfit sau vài hundred iterations.

Training và test errors cho boosting cũng ở Figure 8.13. Test error của boosting approaches BART nhưng sau đó tăng khi number of iterations tăng. Training error của boosting giảm khi iterations tăng, cho thấy boosting đã overfit data.

Dù details ngoài scope, BART có thể được xem như Bayesian approach để fit ensemble of trees: mỗi lần randomly perturb tree để fit residuals, ta thực chất draw một new tree từ posterior distribution. Bayesian connection này là motivation cho tên BART. Algorithm 8.3 cũng có thể xem như một Markov chain Monte Carlo algorithm để fit BART model.

*Markov chain  
Monte Carlo*

Khi apply BART, phải select number of trees  $K$ , number of iterations  $B$ , và number of burn-in iterations  $L$ . Thường chọn large  $B$  và  $K$ , moderate  $L$ ; chẳng hạn  $K = 200$ ,  $B = 1,000$ ,  $L = 100$  là reasonable choice. BART có impressive out-of-box performance—perform well với minimal tuning.

### 8.2.5 Summary of Tree Ensemble Methods

Trees là attractive weak learners cho ensemble method nhờ flexibility và khả năng handle predictors of mixed types (qualitative lẫn quantitative). Ta đã thấy bốn approaches để fit ensemble of trees: bagging, random forests, boosting, và BART.

- Trong bagging, trees grow independently trên random samples of observations. Vì thế trees khá similar; bagging có thể mắc vào local optima và fail to thoroughly explore model space.
- Trong random forests, trees cũng grow independently trên random samples of observations. Nhưng mỗi split dùng random subset of features, decorrelating trees và dẫn tới more thorough exploration of model space so với bagging.
- Trong boosting, chỉ dùng original data, không draw random samples. Trees grow successively theo “slow” learning approach: mỗi new tree fit vào signal còn lại từ earlier trees rồi được shrunk trước khi dùng.
- Trong BART, cũng chỉ dùng original data và grow trees successively. Tuy nhiên, mỗi tree được perturbed để avoid local minima và explore model space thoroughly hơn.

## 8.3 Lab: Tree-Based Methods

Ta import một số usual libraries ở top level.

```
In [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
from statsmodels.datasets import get_rdataset
import sklearn.model_selection as skm
from ISLP import load_data, confusion_table
from ISLP.models import ModelSpec as MS
```

Ta cũng collect các new imports cần cho lab này.

```
In [2]: from sklearn.tree import (DecisionTreeClassifier as DTC,
 DecisionTreeRegressor as DTR,
 plot_tree,
 export_text)
from sklearn.metrics import (accuracy_score,
 log_loss)
from sklearn.ensemble import \
 (RandomForestRegressor as RF,
 GradientBoostingRegressor as GBR)
from ISLP.bart import BART
```

### 8.3.1 Fitting Classification Trees

Trước hết ta dùng classification trees để analyze data set **Carseats**. Trong data này, **Sales** là continuous variable, nên ta recode thành binary variable. Function **where()** tạo variable **High**, bằng **Yes** nếu **Sales**>8 và **No** otherwise.

*where()*

```
In [3]: Carseats = load_data('Carseats')
 High = np.where(Carseats.Sales > 8,
 "Yes",
 "No")
```

Ta dùng `DecisionTreeClassifier()` để fit classification tree predict High bằng tất cả variables trừ Sales. Trước đó phải form model matrix như khi fit regression models.

DTC

```
In [4]: model = MS(Carseats.columns.drop('Sales'), intercept=False)
 D = model.fit_transform(Carseats)
 feature_names = list(D.columns)
 X = np.asarray(D)
```

Ta convert D từ data frame thành array X, cần cho một số analysis bên dưới. `feature_names` sẽ được dùng annotate plots. Classifier có nhiều options như `max_depth` (tree sâu đến đâu), `min_samples_split` (minimum observations để node eligible for splitting), và `criterion` (Gini hay cross-entropy). Ta cũng set `random_state` để reproducibility; ties trong split criterion được break randomly.

```
In [5]: clf = DTC(criterion='entropy',
 max_depth=3,
 random_state=0)
 clf.fit(X, High)
```

```
Out[5]: DecisionTreeClassifier(criterion='entropy', max_depth=3)
```

Trong discussion về qualitative features ở Section 3.3, một linear regression model có thể represent feature như vậy bằng matrix of dummy variables (one-hot-encoding) trong model matrix. Với decision tree có cách natural hơn: mỗi split partition levels thành hai groups. Tuy nhiên sklearn implementation không tận dụng approach này; nó treats one-hot-encoded levels như separate variables.

```
In [6]: accuracy_score(High, clf.predict(X))
```

```
Out[6]: 0.7275
```

Với classification trees, deviance có thể access bằng `log_loss()`. Theo notation của chapter, quantity liên quan là

$$-2 \sum_m \sum_k n_{mk} \log \hat{p}_{mk},$$

trong đó  $n_{mk}$  là number of observations ở  $m$ th terminal node thuộc  $k$ th class.

`log_loss()`

```
In [7]: resid_dev = np.sum(log_loss(High, clf.predict_proba(X)))
 resid_dev
```

```
Out[7]: 0.4711
```

Quantity này closely related to entropy trong (8.7). Small deviance indi-

cates tree fit training data tốt. Một attractive property của trees là display graphically. Ở đây dùng `plot_tree()` để display tree structure (không show trong sách).

In [8]:

```
ax = subplots(figsize=(12,12))[1]
plot_tree(clf,
 feature_names=feature_names,
 ax=ax);
```

Important indicator nhất của Sales dường như là **ShelveLoc**. Ta xem text representation bằng `export_text()`, nó display split criterion cho mỗi branch và overall prediction cho leaf nodes. Với `show_weights=True`, cũng thấy number of observations trong leaf có values **Yes** và **No**.

`export_text()`

In [9]:

```
print(export_text(clf,
 feature_names=feature_names,
 show_weights=True))
```

Out[9]:

```
|--- ShelveLoc[Good] <= 0.50
| |--- Price <= 92.50
| | |--- Income <= 57.00
| | | |--- weights: [7.00, 3.00] class: No
| | | |--- Income > 57.00
| | | |--- weights: [7.00, 29.00] class: Yes
| | |--- Price > 92.50
| | |--- Advertising <= 13.50
| | | |--- weights: [183.00, 41.00] class: No
| | | |--- Advertising > 13.50
| | | |--- weights: [20.00, 25.00] class: Yes
|--- ShelveLoc[Good] > 0.50
| |--- Price <= 135.00
| | |--- US[Yes] <= 0.50
| | | |--- weights: [6.00, 11.00] class: Yes
| | | |--- US[Yes] > 0.50
| | | |--- weights: [2.00, 49.00] class: Yes
| | |--- Price > 135.00
| | |--- Income <= 46.00
| | | |--- weights: [6.00, 0.00] class: No
| | | |--- Income > 46.00
| | | |--- weights: [5.00, 6.00] class: Yes
```

Để properly evaluate classification-tree performance, phải estimate test error thay vì chỉ compute training error. Ta split observations thành training và test set, build tree trên training set rồi evaluate trên test data. Pattern giống Chapter 6, chỉ thay linear models bằng decision trees; validation code gần như identical. Approach này predict correct 68.5% locations trong test set.

In [10]:

```
validation = skm.ShuffleSplit(n_splits=1,
 test_size=200,
 random_state=0)

results = skm.cross_validate(clf,
 D,
 High,
 cv=validation)

results['test_score']
```

Out[10]:

```
array([0.685])
```



Tiếp theo xét pruning có improve classification performance không. Ta split data thành training/test set, use cross-validation prune tree trên training set, rồi evaluate pruned tree trên test set.

```
In [11]: (X_train,
 X_test,
 High_train,
 High_test) = skm.train_test_split(X,
 High,
 test_size=0.5,
 random_state=0)
```

Refit full tree trên training set; không set `max_depth` vì sẽ learn qua cross-validation.

```
In [12]: clf = DTC(criterion='entropy', random_state=0)
 clf.fit(X_train, High_train)
 accuracy_score(High_test, clf.predict(X_test))
```

Out[12]: 0.735

Method `cost_complexity_pruning_path()` của `clf` extract cost-complexity values.

```
In [13]: ccp_path = clf.cost_complexity_pruning_path(X_train, High_train)
 kfold = skm.KFold(10,
 random_state=1,
 shuffle=True)
```

*cost-complexity  
path*

Set of impurities và  $\alpha$  values này cho phép extract optimal one bằng cross-validation.

```
In [14]: grid = skm.GridSearchCV(clf,
 {'ccp_alpha': ccp_path.ccp_alphas},
 refit=True,
 cv=kfold,
 scoring='accuracy')
 grid.fit(X_train, High_train)
 grid.best_score_
```

Out[14]: 0.685

Ta xem pruned tree.

```
In [15]: ax = subplots(figsize=(12, 12))[1]
 best_ = grid.best_estimator_
 plot_tree(best_,
 feature_names=feature_names,
 ax=ax);
```

Tree khá bushy. Có thể count leaves hoặc query `best_`.

```
In [16]: best_.tree_.n_leaves
```

Out[16]: 30

Tree với 30 terminal nodes cho lowest cross-validation error rate, accuracy

68.5%. Để đánh giá trên test data, lại apply `predict()`.

In [17]:

```
print(accuracy_score(High_test,
 best_.predict(X_test)))
confusion = confusion_table(best_.predict(X_test),
 High_test)

confusion
```

Out[17]:

0.72

| Truth     | No  | Yes |
|-----------|-----|-----|
| Predicted |     |     |
| No        | 108 | 61  |
| Yes       | 10  | 21  |

Bây giờ 72.0% test observations classified correctly, slightly worse than full tree (35 leaves). Cross-validation không giúp nhiều ở đây: chỉ prune 5 leaves với cost là error hơi worse. Results sẽ change nếu đổi random seeds; dù cross-validation là unbiased approach cho model selection, nó vẫn có variance.

### 8.3.2 Fitting Regression Trees

Ta fit regression tree lên data Boston; steps tương tự classification trees.

In [18]:

```
Boston = load_data("Boston")
model = MS(Boston.columns.drop('medv'), intercept=False)
D = model.fit_transform(Boston)
feature_names = list(D.columns)
X = np.asarray(D)
```

Đầu tiên split data thành training/test sets, dùng 30% data cho test set.

In [19]:

```
(X_train,
 X_test,
 y_train,
 y_test) = skm.train_test_split(X,
 Boston['medv'],
 test_size=0.3,
 random_state=0)
```

Sau khi form training/test data sets, fit regression tree.

In [20]:

```
reg = DTR(max_depth=3)
reg.fit(X_train, y_train)
ax = subplots(figsize=(12,12))[1]
plot_tree(reg,
 feature_names=feature_names,
 ax=ax);
```

Variable `lstat` measures percentage of individuals có lower socioeconomic status. Tree indicates lower `lstat` corresponds to more expensive houses. Tree predicts median house price \$12,042 cho small homes (`rm`<6.8) trong suburbs nơi residents có low socioeconomic status (`lstat`>14.4) và crime rate moderate (`crim`>5.8).

Ta dùng cross-validation để xem pruning có improve performance không.

```
In [21]: ccp_path = reg.cost_complexity_pruning_path(X_train, y_train)
kfold = skm.KFold(5,
 shuffle=True,
 random_state=10)
grid = skm.GridSearchCV(reg,
 {'ccp_alpha': ccp_path.ccp_alphas},
 refit=True,
 cv=kfold,
 scoring='neg_mean_squared_error')
G = grid.fit(X_train, y_train)
```

Theo cross-validation results, dùng pruned tree để predict test set.

```
In [22]: best_ = grid.best_estimator_
np.mean((y_test - best_.predict(X_test))**2)
```

Out[22]: 28.07

Test-set MSE là 28.07. Square root của MSE khoảng 5.30, nghĩa là test predictions cách true median home value khoảng \$5,300. Ta plot best tree để xem interpretability.

```
In [23]: ax = subplots(figsize=(12,12))[1]
plot_tree(G.best_estimator_,
 feature_names=feature_names,
 ax=ax);
```

### 8.3.3 Bagging and Random Forests

Ta apply bagging và random forests lên data Boston bằng **RandomForestRegressor()** từ package **sklearn.ensemble**. Bagging là special case của random forest với  $m = p$ , nên function này perform được cả hai. Bắt đầu với bagging.

*random forest  
regressor*

```
In [24]: bag_boston = RF(max_features=X_train.shape[1], random_state=0)
bag_boston.fit(X_train, y_train)
```

Out[24]: RandomForestRegressor(max\_features=12, random\_state=0)

Argument **max\_features** chỉ rằng all 12 predictors được xét tại each split—tức perform bagging. Performance trên test set:

```
In [25]: ax = subplots(figsize=(8,8))[1]
y_hat_bag = bag_boston.predict(X_test)
ax.scatter(y_hat_bag, y_test)
np.mean((y_test - y_hat_bag)**2)
```

Out[25]: 14.63

Test-set MSE của bagged regression tree là 14.63, khoảng half value từ optimally-pruned single tree. Có thể đổi number of trees từ default 100 bằng **n\_estimators**.

In [26]:

```
bag_boston = RF(max_features=X_train.shape[1],
 n_estimators=500,
 random_state=0).fit(X_train, y_train)
y_hat_bag = bag_boston.predict(X_test)
np.mean((y_test - y_hat_bag)**2)
```

Out[26]: 14.61

Không có much change. Bagging và random forests không overfit do increasing number of trees, nhưng có thể underfit nếu number quá small. Random forest grow tương tự, chỉ dùng smaller `max_features`. By default, `RandomForestRegressor()` dùng  $p$  variables khi build regression forest (bagging), còn `RandomForestClassifier()` dùng  $\sqrt{p}$  variables cho classification trees. Ở đây dùng `max_features=6`.

In [27]:

```
RF_boston = RF(max_features=6,
 random_state=0).fit(X_train, y_train)
y_hat_RF = RF_boston.predict(X_test)
np.mean((y_test - y_hat_RF)**2)
```

Out[27]: 20.04

Test-set MSE là 20.04; random forests hơi worse hơn bagging trong case này. Extract `feature_importances_` để xem importance của mỗi variable.

In [28]:

```
feature_imp = pd.DataFrame(
 {'importance': RF_boston.feature_importances_},
 index=feature_names)
feature_imp.sort_values(by='importance', ascending=False)
```

Out[28]:

|         | importance |
|---------|------------|
| lstat   | 0.368683   |
| rm      | 0.333842   |
| ptratio | 0.057306   |
| indus   | 0.053303   |
| crim    | 0.052426   |
| dis     | 0.042493   |
| nox     | 0.034410   |
| age     | 0.024327   |
| tax     | 0.022368   |
| rad     | 0.005048   |
| zn      | 0.003238   |
| chas    | 0.002557   |

Đây là relative measure của total decrease in node impurity từ splits trên variable đó, averaged over all trees (Figure 8.9 dùng measure này cho model trên **Heart**). Results cho thấy wealth level của community (`lstat`) và house size (`rm`) là by far two most important variables.

### 8.3.4 Boosting

Ta dùng `GradientBoostingRegressor()` từ `sklearn.ensemble` để fit boosted regression trees cho data Boston. Classification sẽ dùng `GradientBoostingClassifier()`. Argument `n_estimators=5000` nghĩa là 5000 trees; `max_depth=3` limits depth mỗi tree. `learning_rate` là  $\lambda$

trong boosting description.

*gradient boosting*

In [29]:

```
boost_boston = GBR(n_estimators=5000,
 learning_rate=0.001,
 max_depth=3,
 random_state=0)
boost_boston.fit(X_train, y_train)
```

Training error decreases có thể xem qua attribute `train_score_`. Để xem test error decreases, dùng `staged_predict()` lấy predicted values along path.

In [30]:

```
test_error = np.zeros_like(boost_boston.train_score_)
for idx, y_ in enumerate(boost_boston.staged_predict(X_test)):
 test_error[idx] = np.mean((y_test - y_)**2)

plot_idx = np.arange(boost_boston.train_score_.shape[0])
ax = subplots(figsize=(8,8))[1]
ax.plot(plot_idx,
 boost_boston.train_score_,
 'b',
 label='Training')
ax.plot(plot_idx,
 test_error,
 'r',
 label='Test')
ax.legend();
```

Dùng boosted model để predict `medv` trên test set:

In [31]:

```
y_hat_boost = boost_boston.predict(X_test);
np.mean((y_test - y_hat_boost)**2)
```

Out[31]: 14.48

Test MSE 14.48, similar bagging. Có thể boosting với shrinkage parameter  $\lambda$  khác trong (8.10). Source dùng default value 0.001 ở trên và sau đó lấy  $\lambda = 0.2$ .

In [32]:

```
boost_boston = GBR(n_estimators=5000,
 learning_rate=0.2,
 max_depth=3,
 random_state=0)
boost_boston.fit(X_train,
 y_train)
y_hat_boost = boost_boston.predict(X_test);
np.mean((y_test - y_hat_boost)**2)
```

Out[32]: 14.50

Trong case này,  $\lambda = 0.2$  dẫn tới almost same test MSE như  $\lambda = 0.001$ .

### 8.3.5 Bayesian Additive Regression Trees

Section này demonstrate Python implementation của BART trong package `ISLP.bart`, fit model cho Boston housing data. Estimator `BART()` designed cho quantitative outcomes, dù có implementations khác cho logistic và probit models với categorical outcomes.

*BART()*

```
In [33]: bart_boston = BART(random_state=0, burnin=5, ndraw=15)
 bart_boston.fit(X_train, y_train)
```

```
Out[33]: BART(burnin=5, ndraw=15, random_state=0)
```

Với split test/training này, test error của BART similar random forest.

```
In [34]: yhat_test = bart_boston.predict(X_test.astype(np.float32))
 np.mean((y_test - yhat_test)**2)
```

```
Out[34]: 20.92
```

Ta check số lần mỗi variable xuất hiện trong collection of trees, summary tương tự variable importance plot cho boosting và random forests.

```
In [35]: var_inclusion = pd.Series(bart_boston.variable_inclusion_.mean(0),
 index=D.columns)
 var_inclusion
```

```
Out[35]: crim 25.333333
 zn 27.000000
 indus 21.266667
 chas 20.466667
 nox 25.400000
 rm 32.400000
 age 26.133333
 dis 25.666667
 rad 24.666667
 tax 23.933333
 ptratio 25.000000
 lstat 31.866667
 dtype: float64
```

## 8.4 Exercises

### Conceptual

1. Vẽ một example do bạn tự nghĩ ra về partition của two-dimensional feature space có thể tạo bởi recursive binary splitting. Example phải có ít nhất six regions. Vẽ decision tree tương ứng với partition này. Label đầy đủ mọi aspects của figures, gồm regions  $R_1, R_2, \dots$ , cutpoints  $t_1, t_2, \dots$ , v.v.  
*Hint:* Result nên trông tương tự Figures 8.1 và 8.2.
2. Section 8.2.3 nói rằng boosting dùng depth-one trees (hay stumps) dẫn tới additive model, tức model dạng

$$f(X) = \sum_{j=1}^p f_j(X_j).$$

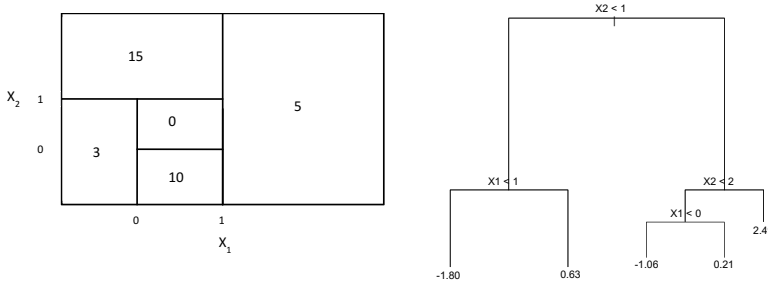
Explain why. Có thể bắt đầu từ (8.12) trong Algorithm 8.2.

3. Xét Gini index, classification error và entropy trong simple classification setting với two classes. Tạo một single plot display mỗi quantity

theo  $\hat{p}_{m1}$ . Trục  $x$  display  $\hat{p}_{m1}$  từ 0 đến 1; trục  $y$  display Gini index, classification error, và entropy.

*Hint:* Với two classes,  $\hat{p}_{m1} = 1 - \hat{p}_{m2}$ . Có thể draw by hand, nhưng source lưu ý sẽ dễ hơn nhiều nếu làm trong R.

4. Câu hỏi này liên quan tới plots trong Figure 8.14.



**HÌNH 8.14.** Left: Partition của predictor space tương ứng Exercise 4(a). Right: Tree tương ứng Exercise 4(b).

- Sketch tree tương ứng partition của predictor space ở left-hand panel của Figure 8.14. Các numbers trong boxes chỉ mean của  $Y$  trong mỗi region.
  - Tạo diagram tương tự left-hand panel của Figure 8.14, dùng tree ở right-hand panel của same figure. Divide predictor space thành correct regions và indicate mean cho mỗi region.
5. Giả sử tạo ten bootstrapped samples từ data set có red và green classes. Apply classification tree cho mỗi bootstrap sample và, với một specific value của  $X$ , thu được 10 estimates của  $P(\text{Class is Red} | X)$ :

0.1, 0.15, 0.2, 0.2, 0.55, 0.6, 0.6, 0.65, 0.7, 0.75.

Có two common ways combine results thành single class prediction: majority vote approach và classify dựa trên average probability. Trong example này, final classification dưới mỗi approach là gì?

6. Provide detailed explanation của algorithm dùng để fit regression tree.

### Applied

- Trong Section 8.3.3, random forests được apply cho data **Boston** với `max_features=6`, `n_estimators=100` và `n_estimators=500`. Tạo plot display test error từ random forests trên data set này với comprehensive range hơn của `max_features` và `n_estimators`. Có thể model plot theo Figure 8.10. Describe results.
- Trong lab, classification tree được apply lên data **Carseats** sau khi convert **Sales** thành qualitative response. Bây giờ predict **Sales** bằng regression trees và related approaches, treating response là quantitative variable.
  - Split data set thành training set và test set.

- (b) Fit regression tree trên training set. Plot tree và interpret results. Test MSE là bao nhiêu?
  - (c) Dùng cross-validation để determine optimal level của tree complexity. Pruning có improve test MSE không?
  - (d) Dùng bagging để analyze data. Test MSE là bao nhiêu? Dùng `feature_importance_` values để determine variables important nhất.
  - (e) Dùng random forests. Test MSE là bao nhiêu? Dùng `feature_importance_` values để determine variables important nhất. Describe effect của  $m$ , number of variables considered at each split, lên error rate.
  - (f) Analyze data bằng BART và report results.
9. Problem này dùng data set `OJ`, thuộc package `ISLP`.
- (a) Create training set gồm random sample 800 observations và test set gồm remaining observations.
  - (b) Fit tree trên training data, response là `Purchase`, predictors là other variables. Training error rate là bao nhiêu?
  - (c) Plot tree và interpret results. Tree có bao nhiêu terminal nodes?
  - (d) Dùng function `export_tree()` để produce text summary của fitted tree. Chọn một terminal node và interpret displayed information.
  - (e) Predict response trên test data và produce confusion matrix comparing test labels với predicted labels. Test error rate là bao nhiêu?
  - (f) Dùng cross-validation trên training set để determine optimal tree size.
  - (g) Produce plot với tree size trên x-axis và cross-validated classification error rate trên y-axis.
  - (h) Tree size nào ứng với lowest cross-validated classification error rate?
  - (i) Produce pruned tree tương ứng optimal tree size từ cross-validation. Nếu cross-validation không select pruned tree, tạo pruned tree với five terminal nodes.
  - (j) Compare training error rates giữa pruned và unpruned trees. Cái nào higher?
  - (k) Compare test error rates giữa pruned và unpruned trees. Cái nào higher?
10. Ta dùng boosting để predict `Salary` trong data set `Hitters`.
- (a) Remove observations thiếu salary information, rồi log-transform salaries.
  - (b) Create training set gồm first 200 observations và test set gồm remaining observations.
  - (c) Perform boosting trên training set với 1,000 trees cho một range



- values của shrinkage parameter  $\lambda$ . Produce plot với shrinkage values trên x-axis và corresponding training-set MSE trên y-axis.
- (d) Produce plot với shrinkage values trên x-axis và corresponding test-set MSE trên y-axis.
  - (e) Compare test MSE của boosting với test MSE từ two regression approaches ở Chapters 3 và 6.
  - (f) Variables nào appear to be most important predictors trong boosted model?
  - (g) Apply bagging lên training set. Test-set MSE là bao nhiêu?
11. Câu này dùng data set **Caravan**.
- (a) Create training set gồm first 1,000 observations và test set gồm remaining observations.
  - (b) Fit boosting model trên training set với **Purchase** là response và other variables là predictors. Dùng 1,000 trees và shrinkage value 0.01. Predictors nào appear to be most important?
  - (c) Dùng boosting model predict response trên test data. Predict rằng một person sẽ make a purchase nếu estimated probability of purchase  $>20\%$ . Form confusion matrix. Fraction của people predicted to make a purchase mà thực tế make one là bao nhiêu? Compare với results từ KNN hoặc logistic regression trên data set này.
12. Apply boosting, bagging, random forests, và BART lên data set tự chọn. Fit models trên training set và evaluate performance trên test set. Results accurate thế nào so với simple methods như linear hoặc logistic regression? Approach nào yield best performance?

# 9

## Support Vector Machines

Chapter này discuss support vector machine (SVM), một classification approach developed trong computer science community vào 1990s và ngày càng popular. SVMs perform well trong nhiều settings và thường được xem là một trong những best “out of the box” classifiers.

Support vector machine là generalization của simple, intuitive classifier gọi là maximal margin classifier, giới thiệu ở Section 9.1. Dù elegant và simple, classifier này không apply được cho most data sets vì yêu cầu classes separable bởi linear boundary. Section 9.2 giới thiệu support vector classifier, extension của maximal margin classifier apply được rộng hơn. Section 9.3 giới thiệu support vector machine, further extension để accommodate non-linear class boundaries. SVMs intended cho binary classification với two classes; Section 9.4 discuss extensions khi có more than two classes. Section 9.5 discuss close connections giữa support vector machines và statistical methods khác như logistic regression.

People often loosely gọi maximal margin classifier, support vector classifier, và support vector machine đều là “support vector machines”. Để tránh confusion, chapter này distinguish carefully ba notions đó.

### 9.1 Maximal Margin Classifier

Trong section này, ta define hyperplane và introduce concept của optimal separating hyperplane.

#### 9.1.1 What Is a Hyperplane?

Trong  $p$ -dimensional space, một *hyperplane* là flat affine subspace có dimension  $p - 1$ .<sup>1</sup> Ví dụ trong two dimensions, hyperplane là flat one-dimensional subspace—một line. Trong three dimensions, hyperplane là flat two-dimensional subspace—một plane. Với  $p > 3$ , khó visualize những notion của  $(p - 1)$ -dimensional flat subspace vẫn áp dụng.

Mathematical definition khá simple. Trong two dimensions, hyperplane được define bởi

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0, \quad (9.1)$$

---

<sup>1</sup>Từ *affine* chỉ rằng subspace không nhất thiết đi qua origin.

cho parameters  $\beta_0, \beta_1, \beta_2$ . Nói (9.1) “defines” hyperplane nghĩa là mọi  $X = (X_1, X_2)^T$  satisfying (9.1) là point on hyperplane. Đây đơn giản là equation of a line.

Extend tới  $p$  dimensions:

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0. \quad (9.2)$$

Nếu  $X = (X_1, X_2, \dots, X_p)^T$  trong  $p$ -dimensional space satisfies (9.2), thì  $X$  nằm trên hyperplane.

Nếu  $X$  không satisfy (9.2) mà

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p > 0, \quad (9.3)$$

thì  $X$  nằm về một side của hyperplane. Nếu

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p < 0, \quad (9.4)$$

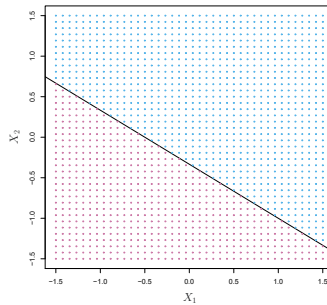
thì  $X$  ở side còn lại. Hyperplane divide  $p$ -dimensional space thành two halves; side của point được determine bằng sign của left-hand side (9.2). Figure 9.1 show two-dimensional example.

### 9.1.2 Classification Using a Separating Hyperplane

Giả sử có  $n \times p$  data matrix  $X$  gồm  $n$  training observations trong  $p$ -dimensional space,

$$x_1 = \begin{pmatrix} x_{11} \\ \vdots \\ x_{1p} \end{pmatrix}, \dots, x_n = \begin{pmatrix} x_{n1} \\ \vdots \\ x_{np} \end{pmatrix}, \quad (9.5)$$

và observations thuộc two classes:  $y_1, \dots, y_n \in \{-1, 1\}$ , với  $-1$  là một class và  $1$  class còn lại. Ta cũng có test observation  $x^* = (x_1^*, \dots, x_p^*)^T$ . Goal là develop classifier từ training data để classify test observation bằng feature measurements. Ta đã thấy LDA và logistic regression trong Chapter 4, classification trees, bagging, boosting trong Chapter 8; bây giờ xét approach dựa trên separating hyperplane.



**HÌNH 9.1.** Hyperplane  $1 + 2X_1 + 3X_2 = 0$ . Blue region là set of points với  $1 + 2X_1 + 3X_2 > 0$ , purple region là set of points với  $1 + 2X_1 + 3X_2 < 0$ .

Giả sử có thể construct hyperplane perfectly separates training observations theo class labels. Left-hand panel của Figure 9.2 show ba such

separating hyperplanes. Label blue class  $y_i = 1$ , purple class  $y_i = -1$ . Separating hyperplane có property

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} > 0 \quad \text{if } y_i = 1, \quad (9.6)$$

và

$$\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip} < 0 \quad \text{if } y_i = -1. \quad (9.7)$$

Equivalently,

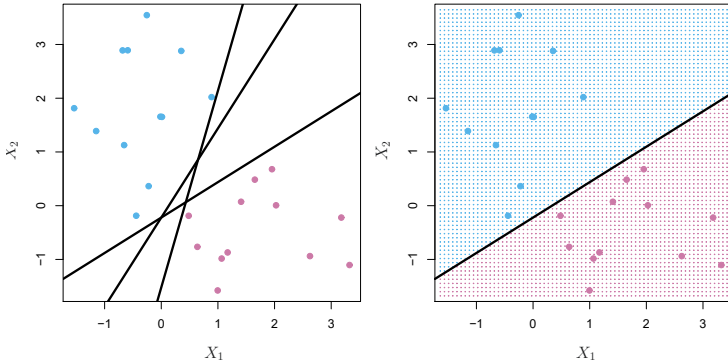
$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) > 0 \quad (9.8)$$

cho mọi  $i = 1, \dots, n$ .

Nếu separating hyperplane exists, natural classifier là assign test observation theo side của hyperplane. Right-hand panel Figure 9.2 minh họa. Ta classify  $x^*$  theo sign của

$$f(x^*) = \beta_0 + \beta_1 x_1^* + \beta_2 x_2^* + \cdots + \beta_p x_p^*.$$

Nếu positive assign class 1; nếu negative assign class  $-1$ . Magnitude của  $f(x^*)$  cũng useful: far from zero nghĩa là  $x^*$  far from hyperplane và confidence cao hơn; near zero nghĩa là observation near hyperplane và class assignment less certain.



**HÌNH 9.2.** Left: Two classes of observations, blue và purple, mỗi observation measured trên two variables. Ba separating hyperplanes trong nhiều possibilities được show bằng black. Right: Một separating hyperplane in black. Blue/purple grid chỉ decision rule: test observation trong blue region assign blue class, trong purple region assign purple class.

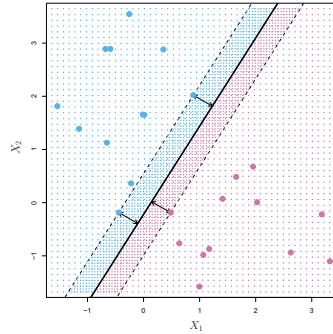
Không surprising, classifier dựa trên separating hyperplane dẫn tới linear decision boundary.

### 9.1.3 The Maximal Margin Classifier

Nếu data perfectly separable bởi hyperplane, thường tồn tại infinite number of separating hyperplanes. Một given hyperplane có thể shift slightly up/down hoặc rotate mà chưa touch observations. Left panel Figure 9.2 show three possibilities. Để build classifier, cần reasonable way chọn một hyperplane.

Natural choice là *maximal margin hyperplane*, còn gọi *optimal separating hyperplane*: separating hyperplane farthest from training observations. Với mỗi separating hyperplane, compute perpendicular distance từ từng training observation; smallest distance là minimal distance đến hyperplane, gọi là *margin*. Maximal margin hyperplane là one với largest margin. Classifier dựa trên side của maximal margin hyperplane gọi là *maximal margin classifier*. Ta hope large training margin cũng dẫn đến large test margin và correct classification; dù often successful, maximal margin classifier có thể overfit khi  $p$  large.

*maximal margin  
hyperplane;  
optimal separating  
hyperplane;  
margin; maximal  
margin classifier*



**HÌNH 9.3.** Two classes shown blue và purple. Maximal margin hyperplane là solid line. Margin là distance từ solid line tới một trong hai dashed lines. Hai blue points và một purple point nằm trên dashed lines là support vectors; arrows chỉ distance từ các points này tới hyperplane. Purple/blue grid indicate classifier decision rule.

Nếu  $\beta_0, \beta_1, \dots, \beta_p$  là coefficients của maximal margin hyperplane, classifier classifies  $x^*$  theo sign của  $f(x^*) = \beta_0 + \beta_1 x_1^* + \dots + \beta_p x_p^*$ .

Figure 9.3 show maximal margin hyperplane trên data Figure 9.2. So với right panel Figure 9.2, maximal margin hyperplane có greater minimal distance tới observations, tức larger margin. Có thể xem nó là mid-line của widest “slab” có thể insert giữa two classes.

Ba training observations trong Figure 9.3 equidistant từ maximal margin hyperplane và nằm trên dashed margin lines. Chúng gọi là *support vectors*: vectors trong  $p$ -dimensional space và “support” maximal margin hyperplane vì nếu move slightly thì hyperplane cũng move. Hyperplane depends directly on support vectors, không on other observations; movement của other observations không affect hyperplane miễn chúng không cross margin boundary. Property này sẽ reappear với support vector classifier và support vector machines.

*support vector*

### 9.1.4 Construction of the Maximal Margin Classifier

Với  $n$  training observations  $x_1, \dots, x_n \in \mathbb{R}^p$  và class labels  $y_1, \dots, y_n \in \{-1, 1\}$ , maximal margin hyperplane là solution của optimization problem

$$\text{maximize}_{\beta_0, \beta_1, \dots, \beta_p, M} \quad M, \quad (9.9)$$

$$\text{subject to} \quad \sum_{j=1}^p \beta_j^2 = 1, \quad (9.10)$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M, \quad \forall i = 1, \dots, n. \quad (9.11)$$

Problem (9.9)–(9.11) simpler than it looks. Constraint (9.11) guarantee mỗi observation on correct side của hyperplane nếu  $M > 0$ . Để merely on correct side chỉ cần product  $> 0$ ; (9.11) yêu cầu thêm cushion.

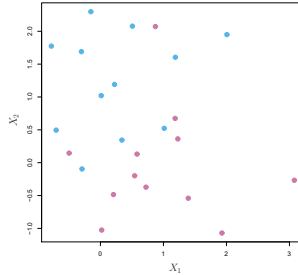
Constraint (9.10) không thật sự constrain hyperplane: nếu  $\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} = 0$  defines hyperplane thì  $k(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) = 0$  cũng defines same hyperplane cho any  $k \neq 0$ . Nhưng (9.10) gives meaning to (9.11): perpendicular distance từ  $i$ th observation tới hyperplane bằng

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}).$$

Do đó (9.10), (9.11) ensure every observation on correct side và at least distance  $M$  from hyperplane.  $M$  chính là margin, và optimization chooses coefficients maximize  $M$ . Details giải optimization nằm ngoài scope.

### 9.1.5 The Non-separable Case

Maximal margin classifier là natural nếu separating hyperplane exists. Nhưng trong nhiều cases không có separating hyperplane, nên không có maximal margin classifier; optimization (9.9)–(9.11) không có solution với  $M > 0$ . Figure 9.4 show example. Ta không thể exactly separate classes, nhưng có thể extend separating hyperplane để almost separate classes bằng so-called soft margin. Generalization này gọi là support vector classifier.



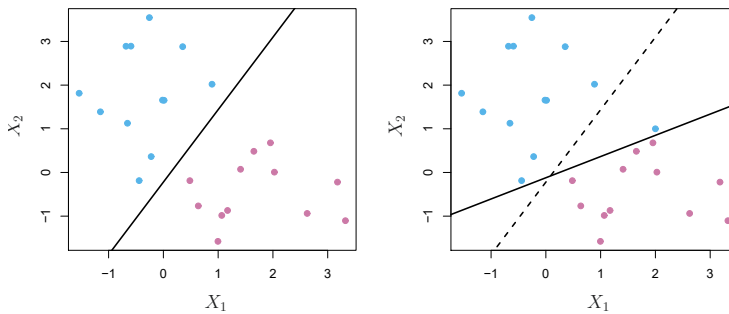
**HÌNH 9.4.** Two classes shown blue và purple. Classes không separable bởi hyperplane, nên maximal margin classifier không thể dùng.

## 9.2 Support Vector Classifiers

### 9.2.1 Overview of the Support Vector Classifier

Figure 9.4 cho thấy observations từ two classes không nhất thiết separable bởi hyperplane. Ngay cả khi separating hyperplane exists, classifier dựa trên perfect separation đôi khi không desirable: nó necessarily perfectly classify all training observations và vì thế sensitive to individual observations. Figure 9.5 show example: thêm single observation ở right panel làm maximal margin hyperplane change dramatically. New hyperplane có tiny margin; moreover high sensitivity to one observation suggests overfitting.

Trong case này, ta có thể prefer classifier dựa trên hyperplane không perfectly separate two classes để đổi lấy robustness với individual observations và better classification của most training observations. Nói cách khác, có thể worthwhile misclassify vài training observations để classify remaining observations tốt hơn.



**HÌNH 9.5.** Left: Two classes shown blue và purple cùng maximal margin hyperplane. Right: Add một blue observation, gây dramatic shift trong maximal margin hyperplane shown solid. Dashed line là maximal margin hyperplane trước khi thêm point.

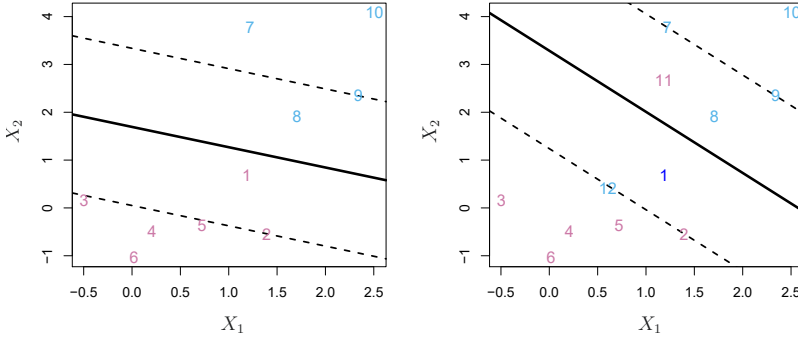
Support vector classifier, đôi khi gọi soft margin classifier, làm đúng điều này. Thay vì seek largest margin sao cho every observation vừa ở correct side của hyperplane vừa correct side của margin, ta allow some observations ở incorrect side của margin hoặc thậm chí incorrect side của hyperplane. Margin gọi là soft vì có thể bị violated bởi training observations. Left panel Figure 9.6: most observations correct side của margin, small subset wrong side.

Observation có thể không chỉ wrong side of margin mà còn wrong side of hyperplane. Khi không có separating hyperplane, điều này inevitable. Observations wrong side of hyperplane là training observations misclassified bởi support vector classifier. Right panel Figure 9.6 illustrate scenario.

support vector  
classifier; soft  
margin classifier

### 9.2.2 Details of the Support Vector Classifier

Support vector classifier classifies test observation theo side của hyperplane. Hyperplane được choose để correctly separate most training observations thành two classes nhưng may misclassify a few.



**HÌNH 9.6.** Left: Support vector classifier fit vào small data set. Hyperplane là solid line, margins là dashed lines. Purple observations: 3, 4, 5, 6 ở correct side of margin; 2 on margin; 1 wrong side of margin. Blue observations: 7, 10 correct side; 9 on margin; 8 wrong side of margin. Không observation nào wrong side of hyperplane. Right: Same panel với additional points 11 và 12; cả hai wrong side of hyperplane và margin.

Hyperplane là solution của optimization problem

$$\text{maximize}_{\beta_0, \beta_1, \dots, \beta_p, \varepsilon_1, \dots, \varepsilon_n, M} M, \quad (9.12)$$

$$\text{subject to} \quad \sum_{j=1}^p \beta_j^2 = 1, \quad (9.13)$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M(1 - \varepsilon_i), \quad (9.14)$$

$$\varepsilon_i \geq 0, \quad \sum_{i=1}^n \varepsilon_i \leq C. \quad (9.15)$$

Ở đây  $C$  là nonnegative tuning parameter. Như (9.11),  $M$  là width của margin, cần maximize.  $\varepsilon_1, \dots, \varepsilon_n$  là *slack variables* cho phép individual observations nằm wrong side of margin hoặc hyperplane. Sau khi solve (9.12)–(9.15), classify test observation  $x^*$  như trước theo sign của  $f(x^*) = \beta_0 + \beta_1 x_1^* + \dots + \beta_p x_p^*$ .

*slack variable*

Insight vào problem đến từ slack variables. Nếu  $\varepsilon_i = 0$ ,  $i$ th observation ở correct side of margin như Section 9.1.4. Nếu  $\varepsilon_i > 0$ , observation wrong side of margin và được nói là *violated the margin*. Nếu  $\varepsilon_i > 1$ , nó ở wrong side of hyperplane.

Tuning parameter  $C$  bounds sum của  $\varepsilon_i$ 's, nên determines number và severity của violations to margin (và hyperplane) tolerated. Có thể xem  $C$  là budget cho amount margin có thể violated bởi  $n$  observations. Nếu  $C = 0$ , không có violation budget, nên  $\varepsilon_1 = \dots = \varepsilon_n = 0$  và (9.12)–(9.15) trở lại maximal margin optimization (9.9)–(9.11), nếu classes separable. Với  $C > 0$ , không quá  $C$  observations có thể wrong side of hyperplane vì mỗi such observation có  $\varepsilon_i > 1$  và sum không vượt  $C$ .

Khi budget  $C$  tăng, ta tolerate margin violations nhiều hơn nên margin widen. Khi  $C$  giảm, tolerance giảm và margin narrows. Figure 9.7 show example. Trong practice,  $C$  treated như tuning parameter, thường chọn bằng cross-validation.  $C$  controls bias-variance trade-off: small  $C$  seeks

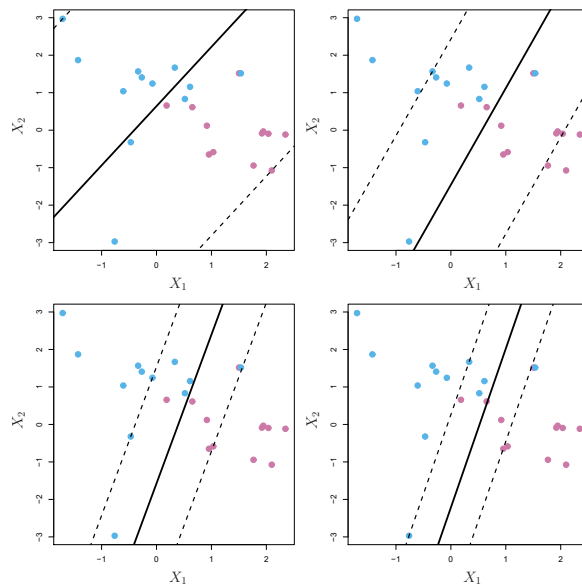


narrow margins rarely violated, classifier fit data hard, potentially low bias/high variance; larger  $C$  gives wider margin with more violations, fitting less hard, potentially higher bias/lower variance.

Optimization (9.12)–(9.15) có interesting property: chỉ observations nằm on margin hoặc violate margin affect hyperplane và resulting classifier. Observation strictly on correct side of margin không affect classifier; move observation đó sẽ không change classifier miễn nó vẫn ở correct side. Observations directly on margin hoặc wrong side of margin gọi là support vectors, và chúng affect support vector classifier.

Điều này consistent với role của  $C$  trong bias-variance trade-off. Khi  $C$  large, margin wide, many observations violate margin, nên có many support vectors; nhiều observations determine hyperplane, classifier low variance nhưng potentially high bias. Top-left Figure 9.7 illustrate. Khi  $C$  small, fewer support vectors; classifier low bias/high variance. Bottom-right panel Figure 9.7 có only eight support vectors.

Decision rule chỉ dựa trên potentially small subset training observations—support vectors—làm classifier robust với observations far away from hyperplane.

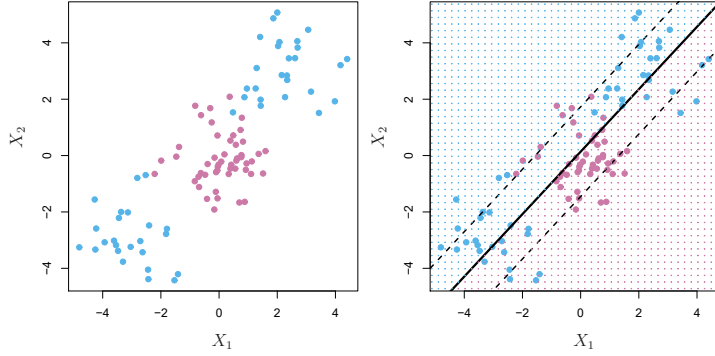


**HÌNH 9.7.** Support vector classifier fit bằng four values khác nhau của tuning parameter  $C$  trong (9.12)–(9.15). Largest  $C$  ở top-left; smaller values ở top-right, bottom-left, bottom-right. Khi  $C$  large, tolerance cho observations wrong side of margin cao, nên margin large. Khi  $C$  decreases, tolerance decreases và margin narrows.

Property này khác một số classification methods ở preceding chapters, như linear discriminant analysis. LDA classification rule depends on mean của all observations within each class và within-class covariance matrix computed using all observations. Ngược lại, logistic regression, unlike LDA, có very low sensitivity với observations far from decision boundary. Section 9.5 sẽ cho thấy support vector classifier và logistic regression closely related.

### 9.3 Support Vector Machines

Trước hết, ta thảo luận một cơ chế tổng quát để chuyển một linear classifier thành classifier tạo ra non-linear decision boundaries. Sau đó ta giới thiệu *support vector machine*, phương pháp thực hiện việc này một cách tự động.



**HÌNH 9.8.** Left: Các observations thuộc two classes, với một non-linear boundary giữa chúng. Right: Support vector classifier tìm một linear boundary và vì thế perform rất kém.

#### 9.3.1 Classification with Non-Linear Decision Boundaries

Support vector classifier là một approach tự nhiên cho classification trong two-class setting nếu boundary giữa two classes là linear. Tuy nhiên trong practice, ta đôi khi gặp non-linear class boundaries. Ví dụ, xét data ở left-hand panel của Figure 9.8. Rõ ràng support vector classifier, hay bất kỳ linear classifier nào, sẽ perform kém ở đây. Thật vậy, support vector classifier ở right-hand panel của Figure 9.8 gần như vô dụng.

Trong Chapter 7 ta gặp một situation tương tự. Ở đó, performance của linear regression có thể giảm khi relationship giữa predictors và outcome là non-linear. Khi ấy, ta mở rộng feature space bằng các functions của predictors, chẳng hạn quadratic và cubic terms, để xử lý non-linearity. Với support vector classifier, ta cũng có thể xử lý khả năng class boundary là non-linear bằng cách mở rộng feature space bằng quadratic, cubic, và thậm chí higher-order polynomial functions của predictors. Chẳng hạn, thay vì fit support vector classifier với  $p$  features

$$X_1, X_2, \dots, X_p,$$

ta có thể fit với  $2p$  features

$$X_1, X_1^2, X_2, X_2^2, \dots, X_p, X_p^2.$$

Khi đó, (9.12)–(9.15) trở thành

$$\begin{aligned}
 & \underset{\beta_0, \beta_{11}, \beta_{12}, \dots, \beta_{p1}, \beta_{p2}, \varepsilon_1, \dots, \varepsilon_n, M}{\text{maximize}} && M \\
 & \text{subject to} && y_i \left( \beta_0 + \sum_{j=1}^p \beta_{j1} x_{ij} + \sum_{j=1}^p \beta_{j2} x_{ij}^2 \right) \geq M(1 - \varepsilon_i), \\
 & && \sum_{i=1}^n \varepsilon_i \leq C, \quad \varepsilon_i \geq 0, \quad \sum_{j=1}^p \sum_{k=1}^2 \beta_{jk}^2 = 1.
 \end{aligned} \tag{9.16}$$

Vì sao cách này dẫn đến non-linear decision boundary? Trong enlarged feature space, decision boundary từ (9.16) thực ra là linear. Nhưng trong original feature space, decision boundary có dạng  $q(x) = 0$ , trong đó  $q$  là quadratic polynomial, và các solutions của nó nói chung là non-linear. Ta còn có thể mở rộng feature space bằng higher-order polynomial terms hoặc interaction terms dạng  $X_j X_{j'}$  với  $j \neq j'$ . Hoặc có thể xét các functions khác của predictors thay cho polynomials. Có rất nhiều cách để enlarge feature space; nếu không cẩn thận, number of features có thể trở nên khổng lồ và computations trở nên không khả thi. Support vector machine, được trình bày tiếp theo, cho phép enlarge feature space của support vector classifier theo cách dẫn đến computations hiệu quả.

### 9.3.2 The Support Vector Machine

*Support vector machine* (SVM) là extension của support vector classifier, tạo ra bằng cách enlarge feature space theo một cách cụ thể, sử dụng *kernels*. Details của extension này khá complex và nằm ngoài scope của sách; tuy nhiên main idea đã xuất hiện ở Section 9.3.1: ta muốn enlarge feature space để accommodate một non-linear boundary giữa classes. Kernel approach đơn giản là một computationally efficient way để thực hiện ý tưởng đó.

*support vector  
machine; kernel*

Ta chưa thảo luận chính xác cách compute support vector classifier vì details hơi technical. Tuy nhiên, solution của problem (9.12)–(9.15) chỉ involve *inner products* của observations, thay vì observations themselves. Inner product của hai  $r$ -vectors  $a$  và  $b$  được định nghĩa là  $\langle a, b \rangle = \sum_{i=1}^r a_i b_i$ . Do đó inner product của two observations  $x_i, x_{i'}$  là

$$\langle x_i, x_{i'} \rangle = \sum_{j=1}^p x_{ij} x_{i'j}. \tag{9.17}$$

Có thể chứng minh rằng:

- Linear support vector classifier có thể được biểu diễn dưới dạng

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle, \tag{9.18}$$

trong đó có  $n$  parameters  $\alpha_i$ ,  $i = 1, \dots, n$ , một parameter cho mỗi training observation.

- Để estimate  $\alpha_1, \dots, \alpha_n$  và  $\beta_0$ , ta chỉ cần  $\binom{n}{2}$  inner products  $\langle x_i, x_{i'} \rangle$  giữa mọi pairs of training observations. Ký hiệu  $\binom{n}{2} = n(n-1)/2$  là number of pairs trong một set gồm  $n$  items.

Trong (9.18), để evaluate  $f(x)$  ta cần compute inner product giữa new point  $x$  và từng training point  $x_i$ . Tuy nhiên  $\alpha_i$  chỉ nonzero đối với support vectors trong solution: nếu một training observation không phải support vector thì  $\alpha_i = 0$ . Nếu  $\mathcal{S}$  là collection of indices của support points, ta có thể rewrite (9.18) thành

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle, \quad (9.19)$$

thường có ít terms hơn nhiều so với (9.18).<sup>2</sup>

Tóm lại, để represent linear classifier  $f(x)$  và compute coefficients của nó, tất cả những gì ta cần là inner products.

Bây giờ, giả sử mỗi khi inner product (9.17) xuất hiện trong representation (9.18), hoặc trong calculation của solution cho support vector classifier, ta thay nó bằng một generalization của inner product dạng

$$K(x_i, x_{i'}), \quad (9.20)$$

trong đó  $K$  là một function gọi là *kernel*. Kernel là function định lượng similarity giữa two observations.

Ví dụ, ta có thể đơn giản chọn

$$K(x_i, x_{i'}) = \sum_{j=1}^p x_{ij} x_{i'j}, \quad (9.21)$$

điều này đưa ta trở lại support vector classifier. Equation 9.21 gọi là *linear kernel* vì support vector classifier linear theo features; linear kernel về cơ bản quantifies similarity của a pair of observations bằng Pearson (standard) correlation.

*linear kernel*

Ta cũng có thể chọn một form khác cho (9.20). Chẳng hạn thay mỗi  $\sum_{j=1}^p x_{ij} x_{i'j}$  bằng

$$K(x_i, x_{i'}) = \left( 1 + \sum_{j=1}^p x_{ij} x_{i'j} \right)^d. \quad (9.22)$$

Đây là *polynomial kernel* degree  $d$ , với  $d$  là positive integer. Dùng kernel này với  $d > 1$  thay cho standard linear kernel (9.21) trong support vector classifier algorithm tạo ra decision boundary flexible hơn nhiều. Về bản chất, điều này tương đương fit support vector classifier trong higher-dimensional space chứa degree- $d$  polynomials thay vì original feature space.

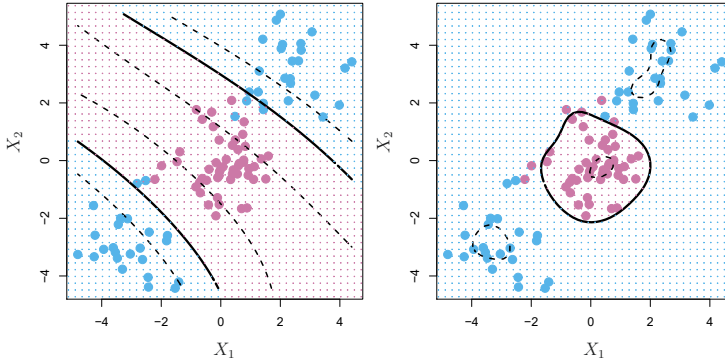
*polynomial kernel*

Khi support vector classifier được combine với một non-linear kernel như (9.22), resulting classifier gọi là support vector machine. Khi đó (non-linear) function có dạng

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i K(x, x_i). \quad (9.23)$$

Left-hand panel của Figure 9.9 cho example SVM với polynomial kernel trên non-linear data của Figure 9.8; fit cải thiện rõ rệt so với linear support

<sup>2</sup>Bằng cách expand từng inner product trong (9.19), dễ thấy  $f(x)$  là linear function của coordinates của  $x$ . Việc này cũng thiết lập correspondence giữa  $\alpha_i$  và original parameters  $\beta_j$ .



**HÌNH 9.9.** Left: SVM với polynomial kernel degree 3 được apply lên non-linear data từ Figure 9.8, cho decision rule phù hợp hơn nhiều. Right: SVM với radial kernel được apply. Trong example này, cả hai kernels đều capture được decision boundary.

vector classifier. Khi  $d = 1$ , SVM reduce về support vector classifier đã thấy trước đó.

Polynomial kernel trong (9.22) chỉ là một example; còn nhiều non-linear kernels khác. Một popular choice là *radial kernel*, có dạng

$$K(x_i, x_{i'}) = \exp \left( -\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right), \quad (9.24)$$

trong đó  $\gamma$  là positive constant. Right-hand panel của Figure 9.9 cho example SVM với radial kernel trên data này; nó cũng separate two classes tốt.

*radial kernel*

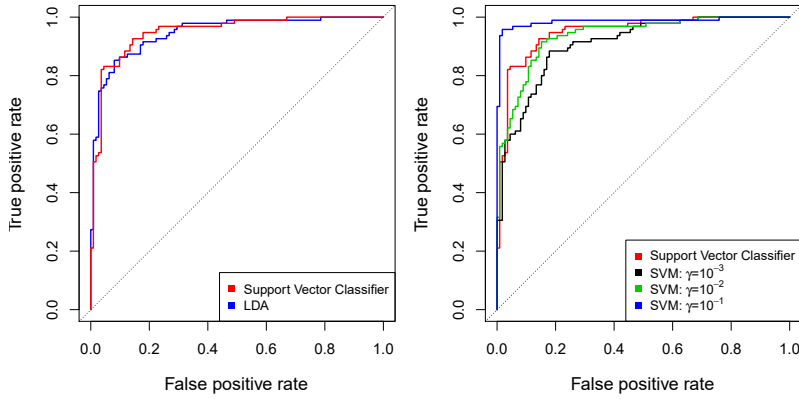
Radial kernel (9.24) hoạt động thế nào? Nếu test observation  $x^* = (x_1^*, \dots, x_p^*)^T$  ở xa training observation  $x_i$  theo Euclidean distance, thì  $\sum_j (x_j^* - x_{ij})^2$  large và  $K(x^*, x_i)$  tiny. Vì thế trong (9.23),  $x_i$  hầu như không đóng vai trò trong  $f(x^*)$ . Predicted class label của  $x^*$  dựa trên sign của  $f(x^*)$ , nên training observations far from  $x^*$  essentially không ảnh hưởng class label. Radial kernel do đó có behavior rất local: chỉ nearby training observations có ảnh hưởng đáng kể.

Advantage của kernel so với explicit enlargement của feature space như (9.16) là computational. Với kernels, chỉ cần compute  $K(x_i, x_{i'})$  cho tất cả  $\binom{n}{2}$  distinct pairs, mà không phải explicitly work trong enlarged feature space. Điều này important vì trong nhiều SVM applications, enlarged feature space quá large để calculations khả thi. Với một số kernels như radial kernel (9.24), feature space implicit và infinite-dimensional, nên explicit computation trong đó thậm chí không thể thực hiện.

### 9.3.3 An Application to the Heart Disease Data

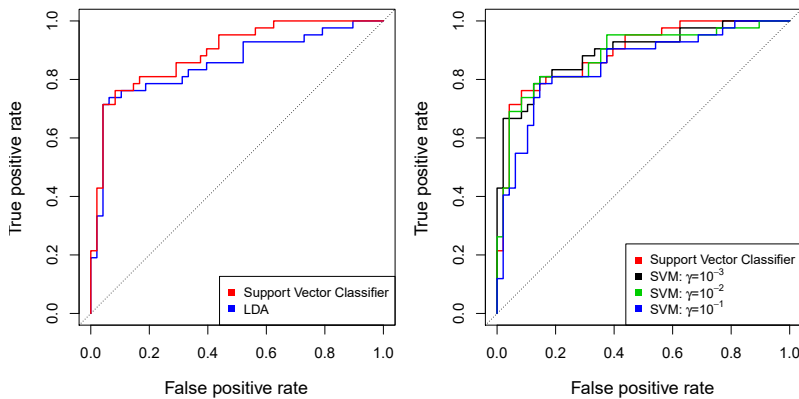
Trong Chapter 8 ta apply decision trees và related methods cho data **Heart**. Goal là dùng 13 predictors như **Age**, **Sex**, và **Chol** để predict một individual có heart disease hay không. Ta bây giờ investigate SVM so với LDA trên data này. Sau khi remove 6 missing observations, data gồm 297 subjects,

random split thành 207 training và 90 test observations.



**HÌNH 9.10.** ROC curves cho training set của data *Heart*. Left: Support vector classifier và LDA được compare. Right: Support vector classifier được compare với SVM dùng radial basis kernel với  $\gamma = 10^{-3}, 10^{-2}, 10^{-1}$ .

Trước hết ta fit LDA và support vector classifier lên training data. Support vector classifier tương đương SVM dùng polynomial kernel degree  $d = 1$ . Left-hand panel của Figure 9.10 display ROC curves (Section 4.4.2) cho training-set predictions của cả LDA và support vector classifier. Cả hai classifiers compute scores dạng  $\hat{f}(X) = \hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p$ . Với cutoff  $t$ , observations được classify thành heart disease hoặc no heart disease tùy  $\hat{f}(X) < t$  hay  $\hat{f}(X) \geq t$ . ROC curve được tạo bằng cách form predictions và compute false positive/true positive rates cho range of  $t$ . Optimal classifier sẽ hug top-left corner của ROC plot. Trong instance này cả LDA và support vector classifier perform tốt, với suggestion rằng support vector classifier có thể slightly superior.



**HÌNH 9.11.** ROC curves cho test set của data *Heart*. Left: Support vector classifier và LDA được compare. Right: Support vector classifier được compare với SVM dùng radial basis kernel với  $\gamma = 10^{-3}, 10^{-2}, 10^{-1}$ .

Right-hand panel của Figure 9.10 display ROC curves cho SVMs với

radial kernel và several values của  $\gamma$ . Khi  $\gamma$  tăng và fit trở nên more non-linear, ROC curves improve;  $\gamma = 10^{-1}$  gần như perfect trên training data. Nhưng training error rates có thể misleading về performance trên new test data. Figure 9.11 cho ROC curves trên 90 test observations. Left panel cho support vector classifier có small advantage over LDA, dù differences không statistically significant. Right panel cho thấy SVM với  $\gamma = 10^{-1}$ , tốt nhất trên training data, lại cho estimates tệ nhất trên test data. Đây là một ví dụ nữa rằng method flexible hơn có thể lower training error nhưng không necessarily improve test performance. SVMs với  $\gamma = 10^{-2}$  và  $10^{-3}$  perform tương đương support vector classifier, và cả ba outperform SVM với  $\gamma = 10^{-1}$ .

## 9.4 SVMs with More than Two Classes

Cho đến nay discussion chỉ xét binary classification, tức two-class setting. Để extend SVMs tới arbitrary number of classes, separating-hyperplane concept không mở rộng một cách hoàn toàn natural. Có nhiều proposals cho  $K$ -class case; hai popular approaches nhất là *one-versus-one* và *one-versus-all*.

### 9.4.1 One-Versus-One Classification

Giả sử có  $K > 2$  classes. One-versus-one, hay all-pairs approach, constructs  $\binom{K}{2}$  SVMs, mỗi SVM compares một pair of classes. Ví dụ một SVM compare  $k$ th class coded +1 với  $k'$ th class coded -1. Ta classify test observation bằng từng classifier và tally number of times observation được assign vào mỗi trong  $K$  classes. Final classification assign observation vào class mà nó được assign most frequently trong  $\binom{K}{2}$  pairwise classifications.

*one-versus-one*

### 9.4.2 One-Versus-All Classification

One-versus-all, còn gọi one-versus-rest, là alternative cho  $K > 2$ . Ta fit  $K$  SVMs, mỗi lần compare một trong  $K$  classes với remaining  $K - 1$  classes. Gọi  $\beta_{0k}, \dots, \beta_{pk}$  là parameters từ SVM comparing  $k$ th class (coded +1) với others (coded -1), và  $x^*$  là test observation. Ta assign observation vào class có  $\beta_{0k} + \beta_{1k}x_1^* + \dots + \beta_{pk}x_p^*$  lớn nhất, vì điều này biểu thị confidence cao rằng observation thuộc  $k$ th class hơn các classes khác.

*one-versus-all;  
one-versus-rest*

## 9.5 Relationship to Logistic Regression

Khi SVMs được introduce vào mid-1990s, chúng gây chú ý mạnh trong statistical và machine learning communities. Một phần do good performance và marketing, một phần vì underlying approach dường như vừa novel vừa mysterious. Ý tưởng tìm hyperplane separate data tốt nhất trong khi allow một số violations trông rất khác classical classification approaches như logistic regression và linear discriminant analysis. Việc dùng kernel để expand feature space nhằm accommodate non-linear class boundaries cũng có vẻ là unique, valuable characteristic.

Tuy nhiên, sau đó người ta nhận ra deep connections giữa SVMs và các

classical statistical methods. Criterion (9.12)–(9.15) cho support vector classifier  $f(X) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$  có thể rewrite thành

$$\underset{\beta_0, \beta_1, \dots, \beta_p}{\text{minimize}} \left\{ \sum_{i=1}^n \max[0, 1 - y_i f(x_i)] + \lambda \sum_{j=1}^p \beta_j^2 \right\}, \quad (9.25)$$

trong đó  $\lambda$  là nonnegative tuning parameter. Khi  $\lambda$  large,  $\beta_1, \dots, \beta_p$  small, nhiều margin violations được tolerated và classifier low-variance/high-bias. Khi  $\lambda$  small, ít margin violations occur, tương ứng high-variance/low-bias classifier. Vì vậy small  $\lambda$  trong (9.25) corresponds to small  $C$  trong (9.15). Term  $\lambda \sum_j \beta_j^2$  là ridge penalty từ Section 6.2.1 và đóng vai trò tương tự trong bias-variance trade-off.

Equation (9.25) có dạng “Loss + Penalty” đã gặp nhiều lần:

$$\underset{\beta_0, \beta_1, \dots, \beta_p}{\text{minimize}} \{L(X, y, \beta) + \lambda P(\beta)\}. \quad (9.26)$$

Ở đây  $L(X, y, \beta)$  là loss function quantifying mức độ model parametrized bởi  $\beta$  fits data  $(X, y)$ , còn  $P(\beta)$  là penalty function trên parameter vector, controlled bởi nonnegative tuning parameter  $\lambda$ .

Ví dụ ridge regression và lasso đều có form này với

$$L(X, y, \beta) = \sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2,$$

và  $P(\beta) = \sum_j \beta_j^2$  cho ridge regression,  $P(\beta) = \sum_j |\beta_j|$  cho lasso. Trong (9.25), loss function thay vào là

$$L(X, y, \beta) = \sum_{i=1}^n \max[0, 1 - y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip})].$$

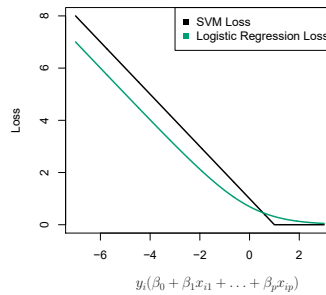
Loss này gọi là *hinge loss*, được depict trong Figure 9.12, và closely related với loss function dùng trong logistic regression.

Một characteristic thú vị của support vector classifier là chỉ support vectors đóng vai trò trong fitted classifier; observations ở correct side của margin không ảnh hưởng. Lý do là loss trong Figure 9.12 exactly zero nếu  $y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq 1$ , tương ứng observations ở correct side of margin.<sup>3</sup> Ngược lại, logistic-regression loss không exactly zero ở đâu, nhưng rất small cho observations far from decision boundary. Vì loss functions similar, logistic regression và support vector classifier thường cho very similar results. Khi classes well separated, SVMs có xu hướng behave better; trong more overlapping regimes, logistic regression thường được preferred.

Khi support vector classifier và SVM mới được introduce, tuning parameter  $C$  trong (9.15) từng bị xem là unimportant “nuisance” parameter có thể set default, ví dụ 1. Nhưng “Loss + Penalty” formulation (9.25) cho thấy không phải vậy. Choice của tuning parameter rất important và determine mức underfit/overfit của model, như Figure 9.7 minh họa.

<sup>3</sup>Trong hinge-loss + penalty representation này, margin corresponds to value one, và width của margin được determine bởi  $\sqrt{\sum_j \beta_j^2}$ .





**HÌNH 9.12.** SVM và logistic regression loss functions được compare theo  $y_i(\beta_0 + \beta_1x_{i1} + \dots + \beta_px_{ip})$ . Khi quantity này lớn hơn 1, SVM loss bằng zero vì observation ở correct side of margin. Overall, two loss functions có behavior khá similar.

Ta đã thấy support vector classifier closely related to logistic regression và các preexisting statistical methods. SVM có unique ở việc dùng kernels để enlarge feature space cho non-linear class boundaries không? Câu trả lời là không. Ta cũng có thể perform logistic regression hoặc nhiều classification methods khác bằng non-linear kernels; điều này closely related với các non-linear approaches ở Chapter 7. Tuy nhiên vì historical reasons, non-linear kernels widespread hơn nhiều trong context của SVMs.

Dù chưa address ở đây, có một extension của SVM cho regression (quantitative response), gọi là *support vector regression*. Least squares regression seek coefficients sao cho sum of squared residuals nhỏ nhất. Support vector regression thay vào đó minimizes một different loss, trong đó chỉ residuals có absolute value lớn hơn một positive constant mới contribute. Đây là extension của margin concept từ support vector classifiers sang regression setting.

*support vector  
regression*

## 9.6 Lab: Support Vector Machines

Trong lab này, ta dùng library `sklearn.svm` để demonstrate support vector classifier và support vector machine. Trước hết import một số usual libraries.

```
In [1]: import numpy as np
from matplotlib.pyplot import subplots, cm
import sklearn.model_selection as skm
from ISLP import load_data, confusion_table
```

Ta cũng collect các new imports cần cho lab.

```
In [2]: from sklearn.svm import SVC
from ISLP.svm import plot as plot_svm
from sklearn.metrics import RocCurveDisplay
```

Ta sẽ dùng `RocCurveDisplay.from_estimator()` để produce several ROC plots, với shorthand `roc_curve`.

*RocCurveDisplay*

```
In [3]: roc_curve = RocCurveDisplay.from_estimator # shorthand
```

### 9.6.1 Support Vector Classifier

Ta dùng `SVC()` từ `sklearn` để fit support vector classifier cho một value cụ thể của parameter `C`. Argument `C` cho phép specify cost của margin violation. Khi cost nhỏ, margins wide và nhiều support vectors nằm trên hoặc violate margin. Khi `C` lớn, margins narrow và có ít support vectors trên hoặc violating margin.

Ta demonstrate `SVC()` trên two-dimensional example để plot resulting decision boundary. Bắt đầu bằng generating observations thuộc two classes và kiểm tra classes có linearly separable hay không.

```
In [4]: rng = np.random.default_rng(1)
X = rng.standard_normal((50, 2))
y = np.array([-1]*25+[1]*25)
X[y==1] += 1
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0],
 X[:,1],
 c=y,
 cmap=cm.coolwarm);
```

Chúng không linearly separable. Ta fit classifier.

```
In [5]: svm_linear = SVC(C=10, kernel='linear')
svm_linear.fit(X, y)
```

```
Out[5]: SVC(C=10, kernel='linear')
```

Support vector classifier với two features có thể visualize bằng cách plot values của decision function. Package `ISLP` có function cho việc này, inspired by a similar example trong `sklearn docs`.

decision function

```
In [6]: fig, ax = subplots(figsize=(8,8))
plot_svm(X,
 y,
 svm_linear,
 ax=ax)
```

Decision boundary là linear vì ta dùng `kernel='linear'`. Support vectors được mark bằng dấu +, remaining observations là circles. Nếu dùng smaller value của cost parameter thì sao?

```
In [7]: svm_linear_small = SVC(C=0.1, kernel='linear')
svm_linear_small.fit(X, y)
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
 y,
 svm_linear_small,
 ax=ax)
```

Với smaller cost parameter, ta obtain larger number of support vectors vì margin wider. Với linear kernels, coefficients của linear decision boundary có thể extract như sau.

```
In [8]: svm_linear.coef_
```

Out[8]: `array([[1.173 , 0.7734]])`

Vì support vector machine là estimator trong sklearn, ta dùng usual machinery để tune nó.

```
In [9]: kfold = skm.KFold(5,
 random_state=0,
 shuffle=True)
grid = skm.GridSearchCV(svm_linear,
 {'C': [0.001, 0.01, 0.1, 1, 5, 10, 100]},
 refit=True,
 cv=kfold,
 scoring='accuracy')
grid.fit(X, y)
grid.best_params_
```

Out[9]: `{'C': 1}`

Ta có thể access cross-validation errors cho mỗi model qua `grid.cv_results_`. Vì object này print nhiều detail, ta chỉ extract accuracy results.

```
In [10]: grid.cv_results_[('mean_test_score')]
```

Out[10]: `array([0.46, 0.46, 0.72, 0.74, 0.74, 0.74, 0.74])`

$C = 1$  cho highest cross-validation accuracy 0.74, dù same accuracy cũng xuất hiện ở several values của  $C$ . Classifier `grid.best_estimator_` có thể predict class label cho test observations. Ta generate một test data set.

```
In [11]: X_test = rng.standard_normal((20, 2))
y_test = np.array([-1]*10+[1]*10)
X_test[y_test==1] += 1
```

Bây giờ predict class labels của test observations bằng best model selected by cross-validation.

```
In [12]: best_ = grid.best_estimator_
y_test_hat = best_.predict(X_test)
confusion_table(y_test_hat, y_test)
```

```
Out[12]: Truth -1 1
Predicted
-1 8 4
1 2 6
```

Với value này của  $C$ , 70% test observations được correctly classified. Nếu dùng  $C = 0.001$  thì sao?

```
In [13]: svm_ = SVC(C=0.001,
 kernel='linear').fit(X, y)
y_test_hat = svm_.predict(X_test)
confusion_table(y_test_hat, y_test)
```

```
Out[13]: Truth -1 1
Predicted
```

```
-1 2 0
1 8 10
```

Trong case này, 60% test observations được correctly classified.

Tiếp theo xét situation mà two classes linearly separable. Khi đó có thể tìm optimal separating hyperplane bằng `SVC()`. Ta further separate two classes trong simulated data.

```
In [14]: X[y==1] += 1.9;
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=y, cmap=cm.coolwarm);
```

Bây giờ observations chỉ vừa đủ linearly separable.

```
In [15]: svm_ = SVC(C=1e5, kernel='linear').fit(X, y)
y_hat = svm_.predict(X)
confusion_table(y_hat, y)
```

```
Out[15]: Truth -1 1
Predicted
-1 25 0
1 0 25
```

Ta fit support vector classifier và plot resulting hyperplane bằng very large  $C$  để không misclassify training observations.

```
In [16]: fig, ax = subplots(figsize=(8,8))
plot_svm(X,
 y,
 svm_,
 ax=ax)
```

Không có training errors và chỉ three support vectors được dùng. Large  $C$  cũng có nghĩa three support points này nằm trên margin và define nó. Classifier phụ thuộc vào chỉ three data points có thể không stable trên test data; ta thử smaller  $C$ .

```
In [17]: svm_ = SVC(C=0.1, kernel='linear').fit(X, y)
y_hat = svm_.predict(X)
confusion_table(y_hat, y)
```

```
Out[17]: Truth -1 1
Predicted
-1 25 0
1 0 25
```

Với  $C = 0.1$ , vẫn không misclassify training observations, nhưng margin rộng hơn nhiều và dùng twelve support vectors. Chúng jointly define orientation của decision boundary; vì có nhiều points hơn nên boundary stable hơn. Model này có thể perform tốt hơn trên test data so với  $C = 10^5$ .

```
In [18]: fig, ax = subplots(figsize=(8,8))
plot_svm(X,
 y,
 svm_,
 ax=ax)
```

### 9.6.2 Support Vector Machine

Để fit SVM với non-linear kernel, ta vẫn dùng `SVC()` nhưng chọn khác parameter `kernel`. Polynomial kernel dùng `kernel="poly"`; radial kernel dùng `kernel="rbf"`. Với polynomial kernel, argument `degree` specifies  $d$  trong (9.22); với radial basis kernel, `gamma` specifies  $\gamma$  trong (9.24).

Ta generate data có non-linear class boundary.

```
In [19]: X = rng.standard_normal((200, 2))
X[:100] += 2
X[100:150] -= 2
y = np.array([1]*150+[2]*50)
```

Plot data cho thấy class boundary thực sự non-linear.

```
In [20]: fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0],
 X[:,1],
 c=y,
 cmap=cm.coolwarm)
```

```
Out[20]: <matplotlib.collections.PathCollection at 0x7faa9ba52eb0>
```

Data được randomly split thành training và testing groups. Ta fit training data bằng `SVC()` với radial kernel và  $\gamma = 1$ .

```
In [21]: (X_train,
X_test,
y_train,
y_test) = skm.train_test_split(X,
 y,
 test_size=0.5,
 random_state=0)
svm_rbf = SVC(kernel="rbf", gamma=1, C=1)
svm_rbf.fit(X_train, y_train)
```

Plot cho thấy resulting SVM có decidedly non-linear boundary.

```
In [22]: fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
 y_train,
 svm_rbf,
 ax=ax)
```

Có khá nhiều training errors trong fit này. Nếu tăng  $C$ , ta có thể reduce training errors, nhưng price là decision boundary irregular hơn và có risk overfitting.

```
In [23]: svm_rbf = SVC(kernel="rbf", gamma=1, C=1e5)
svm_rbf.fit(X_train, y_train)
fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
 y_train,
 svm_rbf,
 ax=ax)
```

Ta dùng `skm.GridSearchCV()` để select best  $\gamma$  và  $C$  cho radial-kernel SVM.

In [24]:

```
kfold = skm.KFold(5,
 random_state=0,
 shuffle=True)
grid = skm.GridSearchCV(svm_rbf,
 {'C': [0.1, 1, 10, 100, 1000],
 'gamma': [0.5, 1, 2, 3, 4]},
 refit=True,
 cv=kfold,
 scoring='accuracy');
grid.fit(X_train, y_train)
grid.best_params_
```

Out[24]:

```
{'C': 100, 'gamma': 1}
```

Output report best parameters là  $C = 100$ ,  $\gamma = 1$ . Source text ngay sau output ghi rằng best choice dưới five-fold CV là  $C = 1$ ,  $\gamma = 0.5$ , và several other values cũng đạt same value; fidelity lock giữ cả output và prose đúng như nguồn.

In [25]:

```
best_svm = grid.best_estimator_
fig, ax = subplots(figsize=(8,8))
plot_svm(X_train,
 y_train,
 best_svm,
 ax=ax)
y_hat_test = best_svm.predict(X_test)
confusion_table(y_hat_test, y_test)
```

Out[25]:

|           |    |    |
|-----------|----|----|
| Truth     | 1  | 2  |
| Predicted |    |    |
| 1         | 69 | 6  |
| 2         | 6  | 19 |

Với các parameters này, 12% test observations bị misclassified.

### 9.6.3 ROC Curves

SVMs và support vector classifiers output class labels cho mỗi observation, nhưng ta cũng có thể obtain fitted values—numerical scores dùng để derive class labels. Với support vector classifier, fitted value cho  $X = (X_1, \dots, X_p)^T$  có form  $\hat{\beta}_0 + \hat{\beta}_1 X_1 + \dots + \hat{\beta}_p X_p$ . Với non-linear-kernel SVM, equation là (9.23). Sign của fitted value determine side của decision boundary. Nếu fitted value exceeds zero thì observation được assign vào one class; nếu below zero thì vào other class. Changing threshold tạo bias toward one class hay class kia; xét range of thresholds cho ingredients của ROC plot. Values này access bằng method `decision_function()` của fitted SVM estimator.

*decision\_function*

`RocCurveDisplay.from_estimator()` (shorthand `roc_curve()`) produce ROC curve. Nó nhận fitted estimator, model matrix  $X$  và labels  $y$ . Argument `name` dùng trong legend, `color` cho line color, và results được plot trên axis object `ax`.

In [26]:

```
fig, ax = subplots(figsize=(8,8))
roc_curve(best_svm,
 X_train,
 y_train,
```

```
name='Training',
color='r',
ax=ax);
```

Trong example này, SVM có vẻ provide accurate predictions. Tăng  $\gamma$  tạo more flexible fit và có thể improve training accuracy hơn nữa.

```
In [27]: svm_flex = SVC(kernel="rbf",
 gamma=50,
 C=1)
svm_flex.fit(X_train, y_train)
fig, ax = subplots(figsize=(8,8))
roc_curve(svm_flex,
 X_train,
 y_train,
 name='Training $\gamma=50$',
 color='r',
 ax=ax);
```

Nhưng các ROC curves trên đều từ training data. Ta quan tâm hơn prediction accuracy trên test data. Khi compute ROC curves trên test data, model với  $\gamma = 0.5$  dường như provide most accurate results.

```
In [28]: roc_curve(svm_flex,
 X_test,
 y_test,
 name='Test $\gamma=50$',
 color='b',
 ax=ax)
fig;
```

Xem tuned SVM của ta.

```
In [29]: fig, ax = subplots(figsize=(8,8))
for (X_, y_, c, name) in zip(
 (X_train, X_test),
 (y_train, y_test),
 ('r', 'b'),
 ('CV tuned on training',
 'CV tuned on test')):
 roc_curve(best_svm,
 X_,
 y_,
 name=name,
 ax=ax,
 color=c)
```

#### 9.6.4 SVM with Multiple Classes

Nếu response là factor có more than two levels, SVC() perform multi-class classification bằng one-versus-one khi `decision_function_shape='ovo'` hoặc one-versus-rest<sup>4</sup> khi `decision_function_shape='ovr'`. Ta briefly explore setting này bằng cách generate third class.

```
In [30]: rng = np.random.default_rng(123)
X = np.vstack([X, rng.standard_normal((50, 2))])
y = np.hstack([y, [0]*50])
X[y==0,1] += 2
```

<sup>4</sup>One-versus-rest còn gọi là one-versus-all.

```
fig, ax = subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=y, cmap=cm.coolwarm);
```

Ta fit SVM vào data.

In [31]:

```
svm_rbf_3 = SVC(kernel="rbf",
 C=10,
 gamma=1,
 decision_function_shape='ovo');
svm_rbf_3.fit(X, y)
fig, ax = subplots(figsize=(8,8))
plot_svm(X,
 y,
 svm_rbf_3,
 scatter_cmap=cm.tab10,
 ax=ax)
```

SVR

Library `sklearn.svm` cũng có thể perform support vector regression với numerical response bằng estimator `SupportVectorRegression()`.

### 9.6.5 Application to Gene Expression Data

Ta examine data set `Khan`, gồm tissue samples tương ứng four distinct types of small round blue cell tumors. Với mỗi tissue sample có gene expression measurements. Data gồm training data `xtrain`, `ytrain` và testing data `xtest`, `ytest`.

Xem dimensions của data.

In [32]:

```
Khan = load_data('Khan')
Khan['xtrain'].shape, Khan['xtest'].shape
```

Out[32]:

```
((63, 2308), (20, 2308))
```

Data set gồm expression measurements cho 2,308 genes. Training và test sets có 63 và 20 observations. Ta dùng support vector approach để predict cancer subtype từ gene expression. Vì number of features rất lớn relative to observations, linear kernel là appropriate; additional flexibility của polynomial/radial kernel là unnecessary.

In [33]:

```
khan_linear = SVC(kernel='linear', C=10)
khan_linear.fit(Khan['xtrain'], Khan['ytrain'])
confusion_table(khan_linear.predict(Khan['xtrain']),
 Khan['ytrain'])
```

Out[33]:

| Truth     | 1 | 2  | 3  | 4  |
|-----------|---|----|----|----|
| Predicted |   |    |    |    |
| 1         | 8 | 0  | 0  | 0  |
| 2         | 0 | 23 | 0  | 0  |
| 3         | 0 | 0  | 12 | 0  |
| 4         | 0 | 0  | 0  | 20 |

Không có training errors. Điều này không surprising vì large number of variables relative to observations khiến việc tìm hyperplanes fully separate classes khá easy.

Ta quan tâm hơn performance trên test observations.



In [34]:

```
confusion_table(khan_linear.predict(Khan['xtest']),
 Khan['ytest'])
```

Out[34]:

```
Truth
Predicted
1 3 0 0 0
2 0 6 2 0
3 0 0 4 0
4 0 0 0 5
```

Dùng  $C = 10$  tạo two test-set errors trên data này.

## 9.7 Exercises

### Conceptual

- Problem này liên quan đến hyperplanes trong two dimensions.
  - Sketch hyperplane  $1 + 3X_1 - X_2 = 0$ . Indicate set of points thỏa  $1 + 3X_1 - X_2 > 0$ , và set thỏa  $1 + 3X_1 - X_2 < 0$ .
  - Trên same plot, sketch hyperplane  $-2 + X_1 + 2X_2 = 0$ . Indicate set of points thỏa  $-2 + X_1 + 2X_2 > 0$ , và set thỏa  $-2 + X_1 + 2X_2 < 0$ .
- Ta đã thấy trong  $p = 2$  dimensions, linear decision boundary có form  $\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$ . Bây giờ investigate một non-linear decision boundary.
  - Sketch curve
 
$$(1 + X_1)^2 + (2 - X_2)^2 = 4.$$
  - Trên sketch, indicate set of points thỏa
 
$$(1 + X_1)^2 + (2 - X_2)^2 > 4,$$
 và set thỏa
 
$$(1 + X_1)^2 + (2 - X_2)^2 \leq 4.$$
  - Giả sử classifier assign observation vào blue class nếu  $(1 + X_1)^2 + (2 - X_2)^2 > 4$ , và red class otherwise. Observations  $(0, 0)$ ,  $(-1, 1)$ ,  $(2, 2)$ ,  $(3, 8)$  được classify vào class nào?
  - Argue rằng decision boundary ở (c) không linear theo  $X_1, X_2$ , nhưng linear theo  $X_1, X_1^2, X_2, X_2^2$ .
- Ta explore maximal margin classifier trên toy data set.
  - Có  $n = 7$  observations trong  $p = 2$  dimensions, mỗi observation có associated class label:

| Obs. | $X_1$ | $X_2$ | Y    |
|------|-------|-------|------|
| 1    | 3     | 4     | Red  |
| 2    | 2     | 2     | Red  |
| 3    | 4     | 4     | Red  |
| 4    | 1     | 4     | Red  |
| 5    | 2     | 1     | Blue |
| 6    | 4     | 3     | Blue |
| 7    | 4     | 1     | Blue |

Sketch observations.

- (b) Sketch optimal separating hyperplane và provide equation của hyperplane này (form (9.1)).
- (c) Describe classification rule cho maximal margin classifier, ví dụ “Classify to Red if  $\beta_0 + \beta_1 X_1 + \beta_2 X_2 > 0$ , and classify to Blue otherwise.” Provide values của  $\beta_0, \beta_1, \beta_2$ .
- (d) Trên sketch, indicate margin cho maximal margin hyperplane.
- (e) Indicate support vectors cho maximal margin classifier.
- (f) Argue rằng slight movement của seventh observation sẽ không affect maximal margin hyperplane.
- (g) Sketch một hyperplane không phải optimal separating hyperplane, và provide equation.
- (h) Draw thêm một observation để two classes không còn separable by a hyperplane.

### Applied

4. Generate simulated two-class data set với 100 observations và two features có visible nhưng non-linear separation giữa classes. Show rằng trong setting này, SVM với polynomial kernel (degree greater than 1) hoặc radial kernel outperform support vector classifier trên training data. Technique nào perform best trên test data? Make plots và report training/test error rates để support assertions.
5. Ta đã thấy SVM với non-linear kernel có thể perform classification bằng non-linear decision boundary. Bây giờ cho thấy logistic regression dùng non-linear transformations của features cũng có thể tạo non-linear decision boundary.
  - (a) Generate data set với  $n = 500$ ,  $p = 2$ , observations thuộc two classes với quadratic decision boundary. Ví dụ:

```
rng = np.random.default_rng(5)
x1 = rng.uniform(size=500) - 0.5
x2 = rng.uniform(size=500) - 0.5
y = x1**2 - x2**2 > 0
```

- (b) Plot observations, colored theo class labels; display  $X_1$  trên x-axis và  $X_2$  trên y-axis.
- (c) Fit logistic regression model dùng  $X_1, X_2$  làm predictors.
- (d) Apply model lên training data để obtain predicted class label cho mỗi observation. Plot observations colored theo predicted labels. Decision boundary nên linear.
- (e) Fit logistic regression model dùng non-linear functions của  $X_1, X_2$  làm predictors (ví dụ  $X_1^2$ ,  $X_1 \times X_2$ ,  $\log(X_2)$ , v.v.).
- (f) Apply model lên training data để obtain predicted class labels và plot. Decision boundary phải obviously non-linear; nếu chưa, repeat (a)–(e) cho đến khi có example phù hợp.
- (g) Fit support vector classifier với  $X_1, X_2$  làm predictors. Obtain

- class prediction cho mỗi training observation và plot colored theo predicted class labels.
- (h) Fit SVM dùng non-linear kernel. Obtain class prediction cho mỗi training observation và plot colored theo predicted class labels.
  - (i) Comment on results.
6. Cuối Section 9.6.1 có claim rằng khi data chỉ barely linearly separable, support vector classifier với small  $C$  misclassify vài training observations có thể perform better trên test data so với huge  $C$  không misclassify training observations. Investigate claim này.
- (a) Generate two-class data với  $p = 2$  sao cho classes chỉ barely linearly separable.
  - (b) Compute cross-validation error rates cho support vector classifiers với range of  $C$  values. Với mỗi  $C$ , bao nhiêu training observations bị misclassified, và relation với cross-validation errors là gì?
  - (c) Generate appropriate test data set và compute test errors cho từng  $C$ .  $C$  nào cho fewest test errors? Compare với  $C$  values cho fewest training errors và fewest cross-validation errors.
  - (d) Discuss results.
7. Dùng support vector approaches để predict một car có high hay low gas mileage dựa trên data set **Auto**.
- (a) Create binary variable bằng 1 cho cars có gas mileage above median và 0 cho below median.
  - (b) Fit support vector classifier với various  $C$  values để predict high-/low mileage. Report cross-validation errors và comment. Cần fit classifier without gas mileage variable để results sensible.
  - (c) Repeat (b) bằng SVMs với radial và polynomial basis kernels, thử different values của **gamma**, **degree**, và  $C$ . Comment on results.
  - (d) Make plots để support assertions trong (b), (c).  
*Hint:* Trong lab, `plot_svm()` dùng cho fitted SVMs. Khi  $p > 2$ , có thể dùng keyword argument **features** để create plots displaying pairs of variables at a time.
8. Problem này dùng data set **OJ** trong package **ISLP**.
- (a) Create training set gồm random sample 800 observations và test set gồm remaining observations.
  - (b) Fit support vector classifier trên training data với  $C = 0.01$ , response là **Purchase**, other variables là predictors. Có bao nhiêu support points?
  - (c) Training và test error rates là bao nhiêu?
  - (d) Dùng cross-validation để select optimal  $C$ . Xét values từ 0.01 đến 10.
  - (e) Compute training và test error rates với new  $C$ .
  - (f) Repeat (b)–(e) bằng SVM với radial kernel. Dùng default **gamma**.

- (g) Repeat (b)–(e) bằng SVM với polynomial kernel, set `degree=2`.
- (h) Overall, approach nào cho best results?

# 10

## Deep Learning

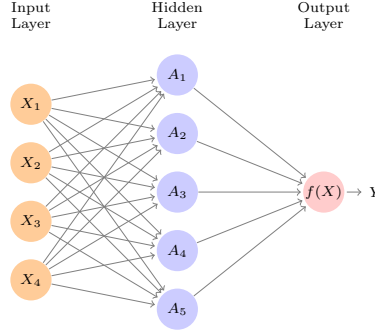
Chapter này trình bày topic quan trọng *deep learning*. Tại thời điểm sách được viết (2020), deep learning là một very active area of research trong machine learning và artificial intelligence communities. Cornerstone của deep learning là *neural network*.

*deep learning;*  
*neural network*

Neural networks trở nên nổi tiếng vào late 1980s. Khi đó có nhiều excitement và một mức hype đáng kể, và approach này là động lực cho các meetings Neural Information Processing Systems (NeurIPS, trước đây NIPS) được tổ chức hằng năm, thường ở những locations đặc biệt như ski resorts. Sau đó là giai đoạn synthesis: properties của neural networks được machine learners, mathematicians và statisticians analyze; algorithms được improve và methodology dần ổn định. Rồi SVMs, boosting, và random forests xuất hiện, khiến neural networks phần nào mất ưu thế. Một lý do là neural networks cần khá nhiều tinkering, còn các newer methods automatic hơn. Trên nhiều problems, newer methods cũng outperform poorly-trained neural networks. Đây là status quo trong first decade của new millennium.

Dù vậy, một core group những neural-network enthusiasts vẫn push technology trên ever-larger computing architectures và data sets. Sau 2010, neural networks resurfaced dưới tên *deep learning*, với new architectures, additional bells and whistles, cùng nhiều success stories trong image/video classification, speech và text modeling. Nhiều researchers tin rằng main reason cho thành công này là availability của ever-larger training datasets, nhờ wide-scale digitization trong science và industry.

Trong chapter này, ta discuss basics của neural networks và deep learning, rồi đi vào specializations cho specific problems, như convolutional neural networks (CNNs) cho image classification và recurrent neural networks (RNNs) cho time series cùng other sequences. Ta cũng demonstrate các models bằng Python package `torch` cùng một số helper packages. Material trong chapter này slightly more challenging so với phần khác của sách.



**HÌNH 10.1.** Neural network với single hidden layer. Hidden layer computes activations  $A_k = h_k(X)$  là nonlinear transformations của linear combinations của inputs  $X_1, X_2, \dots, X_p$ . Vì thế  $A_k$  không directly observed. Functions  $h_k(\cdot)$  không fixed in advance mà được learned trong training. Output layer là linear model dùng activations  $A_k$  làm inputs, tạo function  $f(X)$ .

## 10.1 Single Layer Neural Networks

Một neural network nhận input gồm  $p$  variables  $X = (X_1, X_2, \dots, X_p)$  và build nonlinear function  $f(X)$  để predict response  $Y$ . Ở các chapters trước ta đã build nonlinear prediction models bằng trees, boosting và generalized additive models. Điểm khác biệt của neural networks là particular structure của model. Figure 10.1 show một feed-forward neural network để model quantitative response với  $p = 4$  predictors. Trong neural-network terminology, four features  $X_1, \dots, X_4$  tạo nên units trong *input layer*. Arrows cho thấy mỗi input feeds into mỗi trong  $K$  *hidden units* (ta chọn  $K$ ; ở đây  $K = 5$ ).

feed-forward NN;  
input layer; hidden  
units

Neural network model có form

$$\begin{aligned} f(X) &= \beta_0 + \sum_{k=1}^K \beta_k h_k(X) \\ &= \beta_0 + \sum_{k=1}^K \beta_k g \left( w_{k0} + \sum_{j=1}^p w_{kj} X_j \right). \end{aligned} \quad (10.1)$$

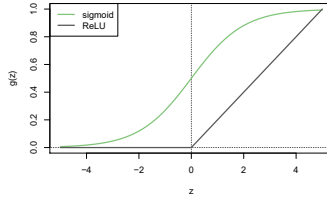
Nó được build qua two steps. First,  $K$  activations  $A_k$ ,  $k = 1, \dots, K$ , trong hidden layer được compute từ input features:

$$A_k = h_k(X) = g \left( w_{k0} + \sum_{j=1}^p w_{kj} X_j \right). \quad (10.2)$$

activations

Ở đây  $g(z)$  là nonlinear *activation function* specified in advance. Có thể xem mỗi  $A_k$  như một transformation khác nhau  $h_k(X)$  của original features, tương tự basis functions ở Chapter 7. Các  $K$  activations từ hidden layer sau đó feed vào output layer:

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k A_k, \quad (10.3)$$



**HÌNH 10.2.** Activation functions. Piecewise-linear ReLU function popular vì efficiency và computability. Trong figure, nó được scale down một factor five để dễ compare.

đây là linear regression model theo  $K = 5$  activations. Tất cả parameters  $\beta_0, \dots, \beta_K$  và  $w_{10}, \dots, w_{Kp}$  phải estimate từ data.

activation function

Trong early neural networks, sigmoid activation function được favored:

$$g(z) = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}}, \quad (10.4)$$

chính là function dùng trong logistic regression để convert linear function thành probabilities giữa zero và one (Figure 10.2). Preferred choice trong modern neural networks là ReLU (rectified linear unit):

$$g(z) = (z)_+ = \begin{cases} 0, & z < 0, \\ z, & \text{otherwise.} \end{cases} \quad (10.5)$$

ReLU activation có thể compute và store efficiently hơn sigmoid. Dù threshold ở zero, vì nó được apply lên linear function (10.2), constant term  $w_{k0}$  shifts inflection point này.

sigmoid; ReLU;  
rectified linear unit

Nói bằng lời, model trong Figure 10.1 derives five new features bằng cách compute five different linear combinations của  $X$ , rồi pass mỗi combination qua activation function  $g(\cdot)$ . Final model linear theo các derived variables này.

Tên *neural network* ban đầu xuất phát từ cách nghĩ hidden units analogous to neurons trong brain—với sigmoid activation, values  $A_k = h_k(X)$  gần one được xem là *firing*, còn gần zero là *silent*.

Nonlinearity trong activation function  $g(\cdot)$  là essential; nếu không có nó, model  $f(X)$  trong (10.1) collapse thành simple linear model theo  $X_1, \dots, X_p$ . Nonlinear activation cũng cho phép model capture complex nonlinearities và interaction effects. Xét simple example với  $p = 2$  inputs  $X = (X_1, X_2)$  và  $K = 2$  hidden units  $h_1(X), h_2(X)$ , dùng  $g(z) = z^2$ . Chọn

$$\begin{aligned} \beta_0 &= 0, & \beta_1 &= \frac{1}{4}, & \beta_2 &= -\frac{1}{4}, \\ w_{10} &= 0, & w_{11} &= 1, & w_{12} &= 1, \\ w_{20} &= 0, & w_{21} &= 1, & w_{22} &= -1. \end{aligned} \quad (10.6)$$

Từ (10.2),

$$\begin{aligned} h_1(X) &= (0 + X_1 + X_2)^2, \\ h_2(X) &= (0 + X_1 - X_2)^2. \end{aligned} \quad (10.7)$$

Thế (10.7) vào (10.1):

$$\begin{aligned} f(X) &= 0 + \frac{1}{4}(0 + X_1 + X_2)^2 - \frac{1}{4}(0 + X_1 - X_2)^2 \\ &= \frac{1}{4} [(X_1 + X_2)^2 - (X_1 - X_2)^2] \\ &= X_1 X_2. \end{aligned} \quad (10.8)$$

Vì vậy sum của two nonlinear transformations của linear functions có thể tạo ra interaction. Trong practice ta không dùng quadratic  $g(z)$  vì khi đó luôn obtain second-degree polynomial trong original coordinates. Sigmoid và ReLU không có limitation này.

Fitting neural network requires estimating unknown parameters trong (10.1). Với quantitative response, squared-error loss thường được dùng, nên parameters được chọn để minimize

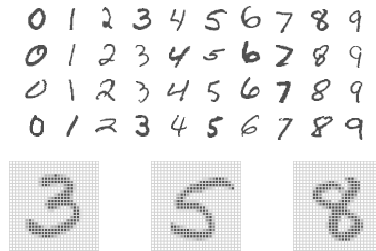
$$\sum_{i=1}^n (y_i - f(x_i))^2. \quad (10.9)$$

Details của minimization được trình bày ở Section 10.7.

## 10.2 Multilayer Neural Networks

Modern neural networks thường có more than one hidden layer và often many units per layer. Về theory, single hidden layer với large number of units có thể approximate most functions. Tuy nhiên learning task để discover good solution trở nên much easier với multiple layers, mỗi layer có modest size.

Ta illustrate một large dense network trên famous publicly available MNIST handwritten digit dataset.<sup>1</sup> Figure 10.3 cho examples. Goal là classify images vào correct digit class 0–9. Mỗi image có  $p = 28 \times 28 = 784$  pixels, mỗi pixel là eight-bit grayscale value từ 0 đến 255 biểu diễn mức dark trong tiny square.



**HÌNH 10.3.** Examples của handwritten digits từ MNIST corpus. Mỗi grayscale image có  $28 \times 28$  pixels; mỗi pixel là eight-bit number (0–255) biểu diễn mức dark. First 3, 5, và 8 được enlarge để show 784 individual pixel values.

Các pixels được store trong input vector  $X$  (ví dụ theo column order). Output là class label, represented bởi vector  $Y = (Y_0, Y_1, \dots, Y_9)$  gồm 10

<sup>1</sup>LeCun, Cortes, and Burges (2010), “The MNIST database of handwritten digits”, available at <http://yann.lecun.com/exdb/mnist>.



dummy variables, với one ở position corresponding to label và zeros elsewhere. Trong machine learning community, cách này gọi là *one-hot encoding*. Có 60,000 training images và 10,000 test images.<sup>2</sup>

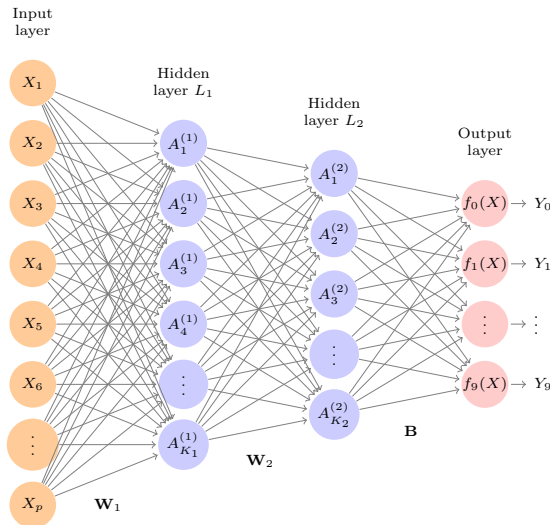
*one-hot encoding*

Về historical note, digit-recognition problems là catalyst thúc đẩy neural-network technology vào late 1980s ở AT&T Bell Laboratories và elsewhere. Pattern-recognition tasks kiểu này relatively simple cho humans vì visual system chiếm large fraction của brain và good recognition là evolutionary force for survival. Với machines, tasks này không simple; phải mất hơn 30 years để refine neural-network architectures đến human-level performance.

Figure 10.4 show multilayer network architecture perform tốt cho digit-classification task. Nó khác Figure 10.1 theo several ways:

- Có two hidden layers  $L_1$  (256 units) và  $L_2$  (128 units) thay vì one. Later ta sẽ thấy network với seven hidden layers.
- Có ten output variables thay vì one. Ở đây ten variables represent một single qualitative variable nên strongly dependent; chúng được index theo digit classes 0–9 thay vì 1–10. Tổng quát hơn, trong *multi-task learning*, một network có thể simultaneously predict different responses; tất cả responses đều influence formation của hidden layers.
- Loss function dùng để train network được tailored cho multiclass classification task.

*multi-task learning*



**HÌNH 10.4.** Neural-network diagram với two hidden layers và multiple outputs, suitable cho MNIST handwritten-digit problem. Input layer có  $p = 784$  units, hidden layers  $K_1 = 256$  và  $K_2 = 128$  units, output layer có 10 units. Cùng intercepts (gọi là biases trong deep-learning community), network có 235,146 parameters (gọi là weights).

<sup>2</sup>Trong analog-to-digital conversion, chỉ một phần của written numeral có thể nằm trong square corresponding to a particular pixel.

First hidden layer như (10.2):

$$A_k^{(1)} = h_k^{(1)}(X) = g \left( w_{k0}^{(1)} + \sum_{j=1}^p w_{kj}^{(1)} X_j \right), \quad (10.10)$$

cho  $k = 1, \dots, K_1$ . Second hidden layer treats activations  $A_k^{(1)}$  của first hidden layer làm inputs và compute

$$A_\ell^{(2)} = h_\ell^{(2)}(X) = g \left( w_{\ell 0}^{(2)} + \sum_{k=1}^{K_1} w_{\ell k}^{(2)} A_k^{(1)} \right), \quad (10.11)$$

cho  $\ell = 1, \dots, K_2$ .

Mỗi activation  $A_\ell^{(2)} = h_\ell^{(2)}(X)$  là function của input vector  $X$ : dù explicitly là function của  $A_k^{(1)}$  từ layer  $L_1$ , các  $A_k^{(1)}$  lại là functions của  $X$ . Với additional hidden layers cũng tương tự. Qua chain of transformations, network có thể build fairly complex transformations của  $X$  rồi feed chúng như features vào output layer.

Superscript notation như  $h_\ell^{(2)}(X)$  và  $w_{\ell j}^{(2)}$  trong (10.10), (10.11) cho biết activation/weight thuộc layer nào. Notation  $W_1$  trong Figure 10.4 represent entire matrix of weights feed từ input layer tới first hidden layer  $L_1$ . Matrix này có  $785 \times 256 = 200,960$  elements; 785 chứ không 784 vì phải include intercept hay *bias* term.<sup>3</sup>

*weights; bias*

Mỗi element  $A_k^{(1)}$  feeds vào second hidden layer  $L_2$  qua matrix  $W_2$  dimension  $257 \times 128 = 32,896$ .

Ở output layer ta có ten responses thay vì one. First compute ten different linear models tương tự (10.1):

$$\begin{aligned} Z_m &= \beta_{m0} + \sum_{\ell=1}^{K_2} \beta_{m\ell} h_\ell^{(2)}(X) \\ &= \beta_{m0} + \sum_{\ell=1}^{K_2} \beta_{m\ell} A_\ell^{(2)}, \end{aligned} \quad (10.12)$$

cho  $m = 0, 1, \dots, 9$ . Matrix  $B$  stores all  $129 \times 10 = 1,290$  weights này.

Nếu đây là separate quantitative responses, ta chỉ cần set  $f_m(X) = Z_m$ . Nhưng ta muốn estimates represent class probabilities  $f_m(X) = \Pr(Y = m \mid X)$  như multinomial logistic regression ở Section 4.3.5. Vì vậy dùng special *softmax* activation function (xem (4.13)):

$$f_m(X) = \Pr(Y = m \mid X) = \frac{e^{Z_m}}{\sum_{\ell=0}^9 e^{Z_\ell}}, \quad (10.13)$$

cho  $m = 0, 1, \dots, 9$ . Nhờ vậy 10 numbers non-negative và sum to one. Mặc dù goal là classifier, model estimate probability cho mỗi class rồi assign image vào class có highest probability.

*softmax*

<sup>3</sup>Cách dùng “weights” cho coefficients và “bias” cho intercepts  $w_{k0}$  trong (10.2) phổ biến trong machine learning; “bias” ở đây không nên nhầm với bias trong “bias-variance”.

Để train network với qualitative response, ta tìm coefficient estimates minimize negative multinomial log-likelihood

$$-\sum_{i=1}^n \sum_{m=0}^9 y_{im} \log(f_m(x_i)), \quad (10.14)$$

còn gọi là *cross-entropy*. Đây là generalization của criterion (4.5) cho two-class logistic regression. Details về minimization ở Section 10.7. Nếu response quantitative, ta instead minimize squared-error loss như (10.9).

*cross-entropy*

Table 10.1 compare test performance của neural network với two simple models từ Chapter 4 có linear decision boundaries: multinomial logistic regression và linear discriminant analysis. Improvement rất dramatic: network với dropout regularization đạt test error rate dưới 2% trên 10,000 test images. Dropout regularization được discuss ở Section 10.7.3; Section 10.9.2 trong lab trình bày code fit model này, chạy chỉ hơn two minutes trên laptop.

| Method                                  | Test Error |
|-----------------------------------------|------------|
| Neural Network + Ridge Regularization   | 2.3%       |
| Neural Network + Dropout Regularization | 1.8%       |
| Multinomial Logistic Regression         | 7.2%       |
| Linear Discriminant Analysis            | 12.7%      |

**BẢNG 10.1.** Test error rate trên MNIST data cho neural networks với two forms of regularization, cùng multinomial logistic regression và linear discriminant analysis. Trong example này, extra complexity của neural network tạo marked improvement về test error.

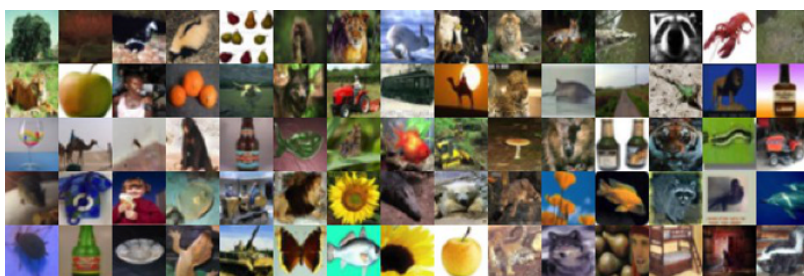
Cộng number of coefficients trong  $W_1$ ,  $W_2$  và  $B$ , ta có 235,146, hơn 33 lần number  $785 \times 9 = 7,065$  cần cho multinomial logistic regression. Training set có 60,000 images; dù nghe large, number of coefficients trong neural network gần four times number of training observations. Để avoid overfitting cần regularization. Example này dùng two forms: ridge regularization, similar to ridge regression Chapter 6, và dropout regularization. Cả hai được discuss ở Section 10.7.

*dropout*

## 10.3 Convolutional Neural Networks

Neural networks trở lại mạnh mẽ vào khoảng năm 2010 nhờ những thành công lớn trong image classification. Cũng vào thời điểm đó, các databases khổng lồ gồm labeled images với số lượng classes ngày càng tăng được tích lũy. Figure 10.5 cho thấy 75 images lấy từ database CIFAR100.<sup>4</sup> Database này gồm 60,000 images được gán nhãn theo 20 superclasses (ví dụ aquatic mammals), với năm classes trong mỗi superclass (beaver, dolphin, otter, seal, whale). Mỗi image có resolution  $32 \times 32$  pixels, với ba eight-bit numbers ở mỗi pixel biểu diễn red, green và blue. Các numbers của mỗi image được tổ chức thành một three-dimensional array gọi là *feature map*. Hai axes đầu tiên là spatial (đều có dimension 32), còn axis thứ ba là *channel axis*,<sup>5</sup> biểu diễn ba colors. Có một designated training set gồm 50,000 images và một test set gồm 10,000 images.

*feature map;*  
*channel*



**HÌNH 10.5.** Một sample images từ database CIFAR100: collection các natural images từ đời sống thường ngày, đại diện cho 100 classes khác nhau.

Một special family của *convolutional neural networks* (CNNs) đã được phát triển để classify các images kiểu này và đạt thành công nổi bật trên rất nhiều problems. Ở một mức độ nào đó, CNNs mô phỏng cách con người classify images: nhận diện những features hoặc patterns cụ thể xuất hiện ở bất kỳ vị trí nào trong image và phân biệt object classes. Trong section này, ta overview ngắn gọn cách chúng hoạt động.

*convolutional*  
*neural networks*

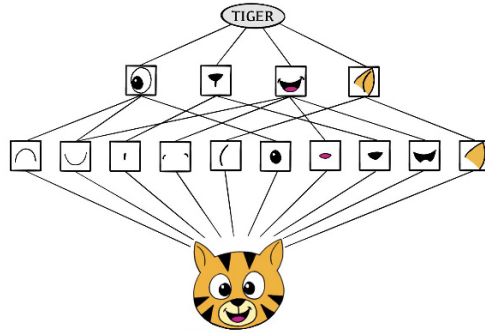
Figure 10.6 minh họa idea của một convolutional neural network trên cartoon image của tiger.<sup>6</sup> Network trước hết nhận diện low-level features trong input image, như small edges, patches of color, v.v. Những low-level features này sau đó được combine thành higher-level features, chẳng hạn parts of ears, eyes, v.v. Cuối cùng, sự presence hoặc absence của các higher-level features này đóng góp vào probability của từng output class.

Convolutional neural network xây hierarchy này bằng cách combine hai loại specialized hidden layers: *convolution layers* và *pooling layers*. Convolution layers tìm các instances của small patterns trong image, còn pooling layers downsample chúng để chọn một prominent subset. Để đạt state-of-the-art results, các contemporary neural-network architectures dùng nhiều

<sup>4</sup>Xem Chapter 3 của Krizhevsky (2009), “Learning multiple layers of features from tiny images”, tại <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.

<sup>5</sup>Thuật ngữ *channel* được lấy từ signal-processing literature. Mỗi channel là một source of information riêng biệt.

<sup>6</sup>Cảm ơn Elena Tuzhilina đã tạo diagram và <https://www.cartooning4kids.com/> đã cho phép sử dụng cartoon tiger.



**HÌNH 10.6.** Schematic minh họa cách một convolutional neural network classify image của một con tiger. Network nhận image làm input và nhận diện local features. Sau đó nó combine các local features để tạo compound features, trong example này gồm eyes và ears. Các compound features được dùng để output label “tiger”.

convolution và pooling layers. Tiếp theo ta mô tả hai loại layer này.

### 10.3.1 Convolution Layers

Một convolution layer gồm nhiều *convolution filters*; mỗi filter là một template dùng để xác định một particular local feature có hiện diện trong image hay không. Convolution filter dựa trên một operation rất đơn giản, gọi là *convolution*, về cơ bản là repeatedly multiply matrix elements rồi add results.

Để hiểu convolution filter hoạt động thế nào, xét một image  $4 \times 3$  rất đơn giản:

$$\text{Original Image} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \\ j & k & l \end{bmatrix}.$$

convolution layer;  
convolution filter

Bây giờ xét một filter  $2 \times 2$ :

$$\text{Convolution Filter} = \begin{bmatrix} \alpha & \beta \\ \gamma & \delta \end{bmatrix}.$$

Khi convolve image với filter, ta thu được<sup>7</sup>

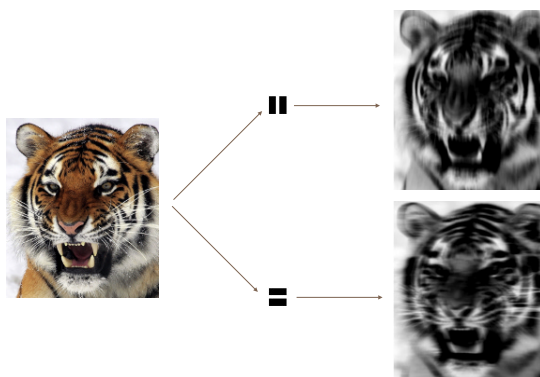
$$\text{Convolved Image} = \begin{bmatrix} a\alpha + b\beta + d\gamma + e\delta & b\alpha + c\beta + e\gamma + f\delta \\ d\alpha + e\beta + g\gamma + h\delta & e\alpha + f\beta + h\gamma + i\delta \\ g\alpha + h\beta + j\gamma + k\delta & h\alpha + i\beta + k\gamma + l\delta \end{bmatrix}.$$

Chẳng hạn, top-left element được tạo bằng cách multiply từng element trong  $2 \times 2$  filter với corresponding element trong top-left  $2 \times 2$  portion của image rồi add results. Các elements khác được tạo tương tự: convolution filter được apply lên mọi  $2 \times 2$  submatrix của original image để tạo convolved

<sup>7</sup>Convolved image nhỏ hơn original image vì dimension của nó bằng number of  $2 \times 2$  submatrices trong original image.  $2 \times 2$  ở đây chính là dimension của convolution filter. Nếu muốn convolved image có cùng dimension với original image, có thể dùng *padding*.

image. Nếu một  $2 \times 2$  submatrix của original image resemble convolution filter thì nó sẽ nhận large value trong convolved image; nếu không, value sẽ nhỏ. Vì vậy convolved image highlight các regions của original image resemble filter. Ta dùng  $2 \times 2$  chỉ làm example; nói chung convolution filters là các small  $\ell_1 \times \ell_2$  arrays, với  $\ell_1$  và  $\ell_2$  là small positive integers và không nhất thiết bằng nhau.

Figure 10.7 minh họa việc apply hai convolution filters lên image tiger kích thước  $192 \times 179$ .<sup>8</sup> Mỗi convolution filter là image  $15 \times 15$  chủ yếu gồm zeros (black), với một narrow strip of ones (white) oriented theo vertical hoặc horizontal direction. Khi mỗi filter được convolve với tiger image, các areas resemble filter (có horizontal/vertical stripes hoặc edges) nhận large values, còn areas không resemble feature nhận small values. Convolved images được hiển thị ở bên phải. Horizontal-stripe filter picks out horizontal stripes và edges, còn vertical-stripe filter picks out vertical stripes và edges.



**HÌNH 10.7.** Convolution filters tìm local features trong image, như edges và small shapes. Bắt đầu với tiger image bên trái và apply hai small convolution filters ở giữa. Convolved images highlight những vùng trong original image có details tương tự filters. Cụ thể, top convolved image highlight vertical stripes của tiger, còn bottom convolved image highlight horizontal stripes. Có thể xem original image như input layer trong convolutional neural network, còn convolved images là units trong first hidden layer.

Figure 10.7 dùng một large image và hai large filters chỉ để illustration. Với CIFAR100, mỗi image có  $32 \times 32$  color pixels và ta dùng  $3 \times 3$  convolution filters.

Trong một convolution layer, ta dùng cả một bank of filters để pick out nhiều edges và shapes với orientations khác nhau trong image. Dùng predefined filters theo cách này là standard practice trong image processing. Ngược lại, với CNNs, filters được learned cho specific classification task. Có thể xem filter weights như parameters đi từ input layer tới hidden layer, với một hidden unit cho mỗi pixel trong convolved image. Thực tế đúng là như vậy, nhưng parameters có structure và constraints rất mạnh (xem Exercise 4). Chúng operate trên localized patches của input image (nên có nhiều structural zeros), và cùng weights trong một filter được reuse cho tất

<sup>8</sup>Tiger image dùng trong Figures 10.7–10.9 lấy từ public-domain image resource <https://www.needpix.com/>.

cả patches có thể có trong image (nên weights bị constrained).<sup>9</sup>

Một số details bổ sung:

- Vì input image là color, nó có three channels biểu diễn bởi một three-dimensional feature map. Mỗi channel là một two-dimensional  $32 \times 32$  feature map—một cho red, một cho green, một cho blue. Một convolution filter cũng có three channels, một cho mỗi color, mỗi channel dimension  $3 \times 3$ , có thể có filter weights khác nhau. Results của three convolutions được summed để tạo một two-dimensional output feature map. Tại point này, color information đã được dùng và không pass trực tiếp tới các subsequent layers, ngoài vai trò của nó trong convolution.
- Nếu dùng  $K$  convolution filters khác nhau ở first hidden layer, ta thu được  $K$  two-dimensional output feature maps; cùng nhau chúng được xem như một single three-dimensional feature map. Mỗi output feature map là một separate channel of information, nên bây giờ có  $K$  channels thay vì three color channels của original input feature map. Three-dimensional feature map này giống activations trong hidden layer của simple neural network, nhưng được organized và produced theo spatial structure.
- Thông thường ta apply ReLU activation function (10.5) lên convolved image. Bước này đôi khi được xem như một separate layer trong CNN và gọi là *detector layer*.

*detector layer*

### 10.3.2 Pooling Layers

Một *pooling layer* cung cấp cách condense một large image thành smaller summary image. Có nhiều cách pooling; *max pooling* summarize mỗi non-overlapping  $2 \times 2$  block of pixels bằng maximum value trong block. Cách này giảm image size đi factor two ở mỗi direction và đồng thời tạo một mức location invariance: miễn là một trong bốn pixels trong block có large value thì cả block sẽ register là large value trong reduced image.

*pooling*

Một simple example của max pooling:

$$\text{Max pool} \begin{bmatrix} 1 & 2 & 5 & 3 \\ 3 & 0 & 1 & 2 \\ 2 & 1 & 3 & 4 \\ 1 & 1 & 2 & 0 \end{bmatrix} \longrightarrow \begin{bmatrix} 3 & 5 \\ 2 & 4 \end{bmatrix}.$$

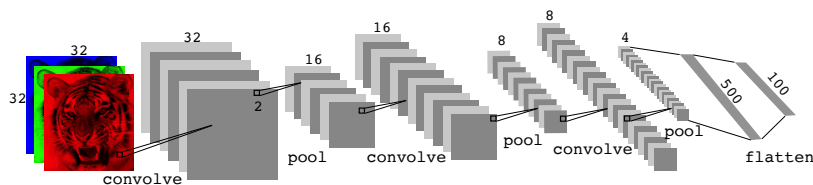
### 10.3.3 Architecture of a Convolutional Neural Network

Cho đến đây ta đã define một convolution layer: mỗi filter tạo một new two-dimensional feature map. Number of convolution filters trong một convolution layer analogous to number of units ở một hidden layer của fully-connected neural network như Section 10.2. Number này cũng define number of channels trong resulting three-dimensional feature map. Ta cũng đã mô tả pooling layer, làm giảm first two dimensions của mỗi three-

<sup>9</sup>Trong những năm đầu của neural networks, điều này từng được gọi là *weight sharing*.

dimensional feature map. Deep CNNs có nhiều layers kiểu này. Figure 10.8 cho một typical architecture cho CNN trong CIFAR100 image classification task.

Ở input layer, ta thấy three-dimensional feature map của color image, trong đó channel axis biểu diễn mỗi color bằng một  $32 \times 32$  two-dimensional feature map of pixels. Mỗi convolution filter tạo một new channel ở first hidden layer, mỗi channel là  $32 \times 32$  feature map (sau khi padding ở edges). Sau first round of convolutions, ta có một new “image”: feature map với nhiều channels hơn hẳn three color input channels (six trong figure, vì ta dùng six convolution filters).



**HÌNH 10.8.** Architecture của deep CNN cho CIFAR100 classification task. Convolution layers xen kẽ với  $2 \times 2$  max-pool layers, làm giảm size đi factor 2 theo cả hai dimensions.

Tiếp theo là max-pool layer, làm giảm size của feature map trong mỗi channel đi factor four: factor two ở mỗi dimension. Sequence convolve-then-pool này được repeat cho next two layers. Một số details:

- Mỗi subsequent convolve layer tương tự first one. Nó nhận three-dimensional feature map từ previous layer làm input và treat như một single multi-channel image. Mỗi learned convolution filter có cùng number of channels như feature map này.
- Vì channel feature maps giảm size sau mỗi pool layer, ta thường increase number of filters trong next convolve layer để compensate.
- Đôi khi ta repeat several convolve layers trước một pool layer. Cách này effectively tăng dimension của filter.

Các operations này được repeat cho đến khi pooling giảm mỗi channel feature map xuống chỉ còn vài pixels ở mỗi dimension. Lúc đó three-dimensional feature maps được *flattened*—pixels được treated như separate units—rồi fed vào một hoặc nhiều fully-connected layers trước khi đến output layer, là softmax activation cho 100 classes (như (10.13)).

Có nhiều tuning parameters phải select khi construct network kiểu này, ngoài number, nature và sizes của each layer. Dropout learning có thể dùng ở mỗi layer, cùng lasso hoặc ridge regularization (Section 10.7). Details của việc construct CNN có thể trông daunting; may mắn là software rất tốt đã có sẵn, kèm extensive examples và vignettes hướng dẫn sensible parameter choices. Với official CIFAR100 test set, best accuracy tại thời điểm sách được viết chỉ trên 75%, và performance chắc chắn sẽ còn improve.



### 10.3.4 Data Augmentation

Một important trick khác trong image modeling là *data augmentation*. Về cơ bản, mỗi training image được replicate nhiều lần, mỗi replicate được randomly distorted theo một cách natural sao cho human recognition không bị ảnh hưởng. Figure 10.9 cho một số examples. Typical distortions gồm zoom, horizontal/vertical shift, shear, small rotations và, trong example này, horizontal flips.

*data augmentation*



**HÌNH 10.9.** *Data augmentation. Original image (leftmost) được distort theo các natural ways để tạo different images cùng class label. Các distortions này không đánh lừa humans và hoạt động như một form of regularization khi fit CNN.*

Nhìn trực tiếp, đây là cách enlarge training set đáng kể bằng somewhat different examples, qua đó protect against overfitting. Thực ra có thể xem nó như một form of regularization: ta build một cloud of images quanh mỗi original image, tất cả có cùng label. Kiểu “fattening” data này tương tự về spirit với ridge regularization.

Section 10.7.2 sẽ cho thấy stochastic gradient descent algorithms để fit deep learning models repeatedly process randomly selected batches, ví dụ 128 training images một lần. Data augmentation phối hợp rất tự nhiên với process này vì ta có thể distort each image trong batch on the fly, không cần store tất cả new images.

### 10.3.5 Results Using a Pretrained Classifier

Ở đây ta dùng một industry-level pretrained classifier để predict class của một số new images. Classifier **resnet50** là một convolutional neural network được trained bằng dataset **imagenet**, gồm millions of images thuộc số lượng categories ngày càng tăng.<sup>10</sup> Figure 10.10 demonstrate performance của **resnet50** trên six photographs (private collection của một author).<sup>11</sup> CNN classify hawk trong second image khá tốt. Nếu zoom out như third image, nó bị confused và chọn fountain thay vì hawk. Trong final image, “jacamar” là tropical bird từ South và Central America có coloring tương tự South African Cape Weaver. Section 10.9.4 trình bày thêm details.

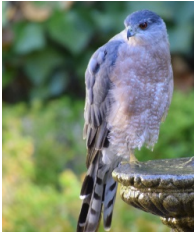
Phần lớn work khi fit CNN là learn convolution filters ở hidden layers; đây là coefficients của CNN. Với models fit trên massive corpora như **imagenet** có nhiều classes, output của các filters này có thể serve as features cho general natural-image classification problems. Ta có thể reuse các

<sup>10</sup>Thông tin thêm về **resnet50**: He, Zhang, Ren, and Sun (2015), “Deep residual learning for image recognition”, <https://arxiv.org/abs/1512.03385>. Chi tiết về **imagenet**: Russakovsky, Deng, et al. (2015), “ImageNet Large Scale Visual Recognition Challenge”, *International Journal of Computer Vision*.

<sup>11</sup>Các **resnet** results này có thể thay đổi theo thời gian vì publicly-trained model được update định kỳ.

pretrained hidden layers cho new problems với much smaller training sets (process gọi là *weight freezing*), rồi chỉ train last few layers của network, nên cần much less data.

*weight freezing*

|                                                                                   |                                                                                   |                                                                                    |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
|  |  |  |
| flamingo                                                                          | Cooper's hawk                                                                     | Cooper's hawk                                                                      |
| flamingo 0.83                                                                     | kite 0.60                                                                         | fountain 0.35                                                                      |
| spoonbill 0.17                                                                    | great grey owl 0.09                                                               | nail 0.12                                                                          |
| white stork 0.00                                                                  | robin 0.06                                                                        | hook 0.07                                                                          |
| Lhasa Apso                                                                        | cat                                                                               | Cape weaver                                                                        |
| Tibetan terrier 0.56                                                              | Old English sheepdog 0.82                                                         | jacamar 0.28                                                                       |
| Lhasa 0.32                                                                        | Shih-Tzu 0.04                                                                     | macaw 0.12                                                                         |
| cocker spaniel 0.03                                                               | Persian cat 0.04                                                                  | robin 0.12                                                                         |

**HÌNH 10.10.** Classification của six photographs bằng *resnet50* CNN trained trên *imagenet* corpus. Table dưới images hiển thị true (intended) label ở top của mỗi panel và top three choices của classifier (trong 100 classes). Numbers là estimated probabilities cho mỗi choice. (A kite is a raptor, but not a hawk.)

Các vignettes và book<sup>12</sup> đi kèm package `keras` cung cấp thêm details về những applications như vậy.

## 10.4 Document Classification

Trong section này ta giới thiệu một loại example mới có important applications trong industry và science: predicting attributes of documents. Documents có thể là articles trong medical journals, Reuters news feeds, emails, tweets, v.v. Example của ta là IMDb (Internet Movie Database) ratings—short documents trong đó viewers viết critiques of movies.<sup>13</sup> Response ở đây là sentiment của review, positive hoặc negative.

Đây là beginning của một negative review khá amusing; vì đoạn này là raw document data nên giữ nguyên tiếng Anh:

This has to be one of the worst films of the 1990s. When my friends & I were watching this film (being the target audience it was aimed at)

<sup>12</sup>*Deep Learning with R* của F. Chollet và J.J. Allaire, 2018, Manning Publications.  
<sup>13</sup>Xem Maas et al. (2011), “Learning word vectors for sentiment analysis”, trong *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pp. 142–150.

we just sat & watched the first half an hour with our jaws touching the floor at how bad it really was. The rest of the time, everyone else in the theater just started talking to each other, leaving or generally crying into their popcorn ...

Mỗi review có thể có length khác nhau, chứa slang hoặc non-words, spelling errors, v.v. Ta cần một cách để *featurize* document—modern parlance cho việc define một set of predictors.

*featurize*

Featurization đơn giản và phổ biến nhất là *bag-of-words model*. Ta score mỗi document theo presence/absence của từng word trong language dictionary—ở đây là English dictionary. Nếu dictionary có  $M$  words, thì với mỗi document ta tạo binary feature vector length  $M$ , ghi 1 cho mỗi word present và 0 nếu absent. Feature vector này có thể rất wide, nên ta giới hạn dictionary ở 10,000 words xuất hiện frequent nhất trong training corpus gồm 25,000 reviews. Có tools tiện lợi để làm việc này automatically.

*bag-of-words*

Đây là beginning của một positive review sau khi được redacted theo cách đó; tiếp tục giữ nguyên vì đây là data:

<START> this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you could just imagine being there robert <UNK> is an amazing actor and now the same being director <UNK> father came from the same scottish island as myself so i loved ...

Ta thấy nhiều words đã bị omitted và một số unknown words (UNK) được mark. Sau reduction này, binary feature vector có length 10,000, chủ yếu là zeros và chỉ rải rác ones ở positions tương ứng với words có trong document.

Ta có training set và test set, mỗi set 25,000 examples, đều balanced theo sentiment. Resulting training feature matrix  $X$  có dimension  $25,000 \times 10,000$ , nhưng chỉ 1.3% binary entries là non-zero. Ta gọi matrix như vậy là *sparse*, vì hầu hết values giống nhau (zero ở đây); nó có thể được store efficiently dưới *sparse matrix format*.<sup>14</sup> Có nhiều cách account for document length; ở đây ta chỉ score một word là in/out document, nhưng chẳng hạn có thể record relative frequency của words.

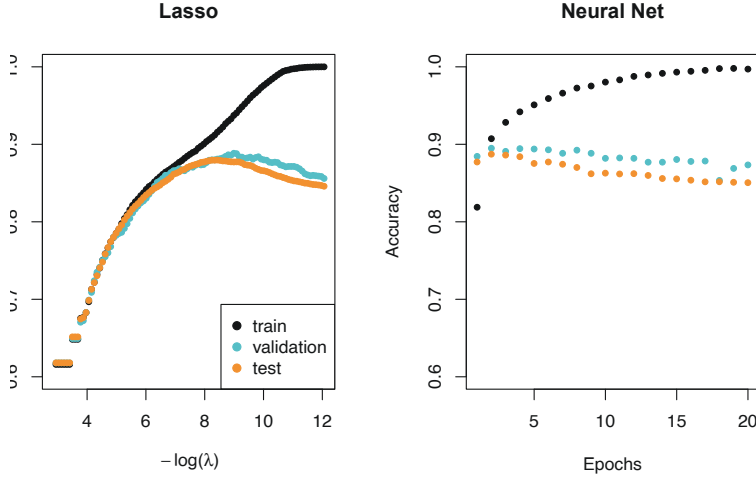
*sparse matrix format*

Ta tách một validation set size 2,000 khỏi 25,000 training observations (để model tuning), rồi fit two model sequences:

- một lasso logistic regression dùng package `glmnet`;
- một two-class neural network với two hidden layers, mỗi layer có 16 ReLU units.

Cả hai methods tạo ra sequence of solutions. Lasso sequence được indexed bởi regularization parameter  $\lambda$ . Neural-net sequence được indexed bởi number of gradient-descent iterations dùng trong fitting, measured bằng training epochs hay passes through training set (Section 10.7). Training accuracy trong Figure 10.11 (black points) tăng monotonically trong cả hai cases. Ta có thể dùng validation error để pick một good solution từ mỗi sequence (blue points), rồi dùng solution đó để make predictions trên test data.

<sup>14</sup>Thay vì store toàn bộ matrix, ta có thể chỉ store locations và values của nonzero entries. Trong case này, vì all nonzero entries đều bằng 1 nên chỉ locations cần được store.



**HÌNH 10.11.** Accuracy của lasso và two-hidden-layer neural network trên IMDB data. Với lasso, x-axis hiển thị  $-\log(\lambda)$ ; với neural network, x-axis hiển thị epochs (number of times fitting algorithm passes through training set). Cả hai đều có tendency to overfit và đạt approximately same test accuracy.

Một two-class neural network tương đương một nonlinear logistic regression model. Từ (10.12) và (10.13),

$$\begin{aligned} \log \left\{ \frac{\Pr(Y = 1 | X)}{\Pr(Y = 0 | X)} \right\} &= Z_1 - Z_0 \\ &= (\beta_{10} - \beta_{00}) + \sum_{\ell=1}^{K_2} (\beta_{1\ell} - \beta_{0\ell}) A_{\ell}^{(2)}. \end{aligned} \quad (10.15)$$

Điều này cho thấy redundancy trong softmax function: với  $K$  classes ta thực sự chỉ cần estimate  $K - 1$  sets of coefficients (xem Section 4.3.5). Trong Figure 10.11 ta show accuracy (fraction correct) thay vì classification error (fraction incorrect), vì former popular hơn trong machine learning community. Cả hai models đạt test-set accuracy khoảng 88%.

Bag-of-words model summarize document bằng words present và ignore context. Có ít nhất two popular ways để account for context:

- *Bag-of-n-grams model.* Ví dụ, bag of 2-grams record consecutive co-occurrence của mọi distinct pair of words. “Blissfully long” có thể là positive phrase trong movie review, còn “blissfully short” là negative.
- Treat document như một sequence, account for all words trong context của những words đi trước và theo sau.

Trong next section ta explore models cho sequences of data, có applications trong weather forecasting, speech recognition, language translation và time-series prediction, cùng nhiều lĩnh vực khác. Ta cũng tiếp tục IMDB example ở đó.

## 10.5 Recurrent Neural Networks

Nhiều data sources có bản chất sequential và cần special treatment khi build predictive models. Examples gồm:

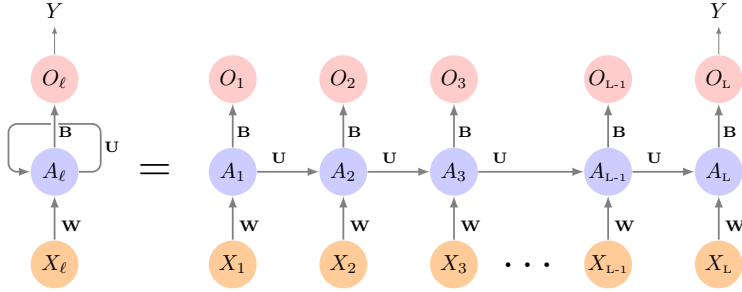
- Documents như book/movie reviews, newspaper articles và tweets. Sequence và relative positions của words trong document capture narrative, theme và tone, và có thể khai thác trong topic classification, sentiment analysis và language translation.
- Time series của temperature, rainfall, wind speed, air quality, v.v. Ta có thể muốn forecast weather vài ngày ahead hoặc climate vài decades ahead.
- Financial time series, nơi ta track market indices, trading volumes, stock/bond prices và exchange rates. Prediction thường khó, nhưng như sẽ thấy, một số indices có thể được predict với reasonable accuracy.
- Recorded speech, musical recordings và other sound recordings. Ta có thể muốn tạo text transcription của speech, hoặc language translation; cũng có thể assess quality của music hay assign attributes.
- Handwriting, như doctor's notes và handwritten digits như zip codes. Mục tiêu là turn handwriting into digital text hoặc read digits (optical character recognition).

Trong một *recurrent neural network* (RNN), input object  $X$  là một sequence. Xét corpus of documents như collection IMDb movie reviews. Mỗi document có thể represent như sequence gồm  $L$  words,  $X = \{X_1, X_2, \dots, X_L\}$ , trong đó mỗi  $X_\ell$  represents một word. Order của words và closeness của certain words trong sentence convey semantic meaning. RNNs được design để accommodate và exploit sequential nature của input objects, tương tự cách CNNs exploit spatial structure của image inputs. Output  $Y$  cũng có thể là sequence (như language translation), nhưng thường là scalar, ví dụ binary sentiment label của movie review.

*recurrent neural network*

Figure 10.12 minh họa structure của very basic RNN với input sequence  $X = \{X_1, X_2, \dots, X_L\}$ , simple output  $Y$  và hidden-layer sequence  $\{A_\ell\}_1^L = \{A_1, A_2, \dots, A_L\}$ . Mỗi  $X_\ell$  là vector; trong document example,  $X_\ell$  có thể represent one-hot encoding cho  $\ell$ th word dựa trên language dictionary của corpus (xem top panel Figure 10.13). Khi sequence được process từng vector  $X_\ell$ , network update activations  $A_\ell$  trong hidden layer, nhận input là  $X_\ell$  và activation vector  $A_{\ell-1}$  từ previous step. Mỗi  $A_\ell$  feeds vào output layer và tạo prediction  $O_\ell$  cho  $Y$ ;  $O_L$ , prediction cuối, là most relevant.

Cụ thể, giả sử mỗi vector  $X_\ell$  trong input sequence có  $p$  components  $X_\ell^T = (X_{\ell 1}, X_{\ell 2}, \dots, X_{\ell p})$ , và hidden layer có  $K$  units  $A_\ell^T = (A_{\ell 1}, A_{\ell 2}, \dots, A_{\ell K})$ . Như Figure 10.4, collection gồm  $K \times (p + 1)$  shared weights  $w_{kj}$  cho input layer được represented bởi matrix  $W$ ; tương tự  $U$  là  $K \times K$  matrix của weights  $u_{ks}$  cho hidden-to-hidden layers, và  $B$  là  $K + 1$  vector của weights



**HÌNH 10.12.** Schematic của một simple recurrent neural network. Input là sequence of vectors  $\{X_\ell\}_1^L$ , còn target ở đây là single response. Network process input sequence  $X$  sequentially; mỗi  $X_\ell$  feeds vào hidden layer, layer này cũng nhận activation vector  $A_{\ell-1}$  từ previous element và tạo current activation vector  $A_\ell$ . Cùng collections of weights  $W$ ,  $U$  và  $B$  được dùng khi process từng element. Output layer tạo sequence predictions  $O_\ell$  từ current activation  $A_\ell$ , nhưng thường chỉ final one,  $O_L$ , là relevant. Bên trái dấu bằng là concise representation của network; bên phải là version được unroll rõ hơn.

$\beta_k$  cho output layer. Khi đó

$$A_{\ell k} = g \left( w_{k0} + \sum_{j=1}^p w_{kj} X_{\ell j} + \sum_{s=1}^K u_{ks} A_{\ell-1, s} \right), \quad (10.16)$$

và output  $O_\ell$  được compute bởi

$$O_\ell = \beta_0 + \sum_{k=1}^K \beta_k A_{\ell k}, \quad (10.17)$$

cho quantitative response, hoặc thêm sigmoid activation function với binary response chẳng hạn. Ở đây  $g(\cdot)$  là activation function như ReLU.

Lưu ý cùng weights  $W$ ,  $U$  và  $B$  được dùng khi process mọi element trong sequence; tức chúng không depend on  $\ell$ . Đây là một form of *weight sharing* trong RNNs, tương tự việc reuse filters trong convolutional neural networks (Section 10.3.1). Khi đi từ beginning tới end, activations  $A_\ell$  accumulate history của những gì đã thấy, để learned context có thể dùng cho prediction.

*weight sharing*

Với regression problems, loss function cho observation  $(X, Y)$  là

$$(Y - O_L)^2, \quad (10.18)$$

chỉ reference final output  $O_L = \beta_0 + \sum_{k=1}^K \beta_k A_{Lk}$ ; vì vậy  $O_1, O_2, \dots, O_{L-1}$  không được dùng. Khi fit model, mỗi element  $X_\ell$  của input sequence  $X$  contributes tới  $O_L$  qua chain (10.16), nên indirectly contributes vào learning shared parameters  $W$ ,  $U$  và  $B$  thông qua loss (10.18). Với  $n$  input

sequence/response pairs  $(x_i, y_i)$ , parameters được tìm bằng cách minimize

$$\sum_{i=1}^n (y_i - o_{iL})^2 = \sum_{i=1}^n \left[ y_i - \left\{ \beta_0 + \sum_{k=1}^K \beta_k g \left( w_{k0} + \sum_{j=1}^p w_{kj} x_{iLj} + \sum_{s=1}^K u_{ks} a_{i,L-1,s} \right) \right\} \right]^2 \quad (10.19)$$

Ở đây lowercase letters dùng cho observed  $y_i$  và vector sequences  $x_i = \{x_{i1}, x_{i2}, \dots, x_{iL}\}$ ,<sup>15</sup> cũng như derived activations.

Vì intermediate outputs  $O_\ell$  không được dùng, ta có thể hỏi tại sao chúng tồn tại. Trước hết, chúng gần như “free”, vì dùng same output weights  $B$  cần để produce  $O_L$ , đồng thời provide evolving prediction cho output. Hơn nữa, với một số learning tasks response cũng là sequence, nên output sequence  $\{O_1, O_2, \dots, O_L\}$  thực sự cần thiết.

Khi dùng ở full strength, RNNs có thể khá complex. Ta minh họa bằng two simple applications. First, tiếp tục IMDb sentiment analysis từ previous section bằng cách process words trong reviews sequentially. Second, minh họa use trong financial time series forecasting problem.

### 10.5.1 Sequential Models for Document Classification

Ta quay lại classification task với IMDb reviews. Section 10.4 dùng bag-of-words model; ở đây plan là dùng sequence of words trong document để predict label cho toàn document.

Tuy nhiên có dimensionality problem: mỗi word được represented bởi one-hot-encoded vector (dummy variable) với 10,000 elements—một cho mỗi word trong dictionary. Một approach phổ biến là represent mỗi word trong lower-dimensional *embedding space*. Thay vì binary vector với 9,999 zeros và một single one, mỗi word được represented bởi set gồm  $m$  real numbers, thường không có số nào bằng zero.  $m$  là embedding dimension và có thể chỉ ở low hundreds hoặc nhỏ hơn. Trong case này ta cần matrix  $E$  dimension  $m \times 10,000$ , mỗi column indexed bởi một trong 10,000 words; values trong column là  $m$  coordinates của word trong embedding space.

*embedding*

Figure 10.13 minh họa idea này (dictionary 16 thay vì 10,000 words, và  $m = 5$ ).  $E$  đến từ đâu? Nếu có large corpus of labeled documents, neural network có thể learn  $E$  như một part of optimization. Khi đó  $E$  được gọi là *embedding layer*, và specialized  $E$  được learned cho task đang xét. Nếu không, ta có thể insert một precomputed matrix  $E$  vào embedding layer, process gọi là weight freezing. Hai pretrained embeddings được dùng rộng rãi là *word2vec* và *GloVe*.<sup>16</sup> Chúng được built từ very large corpus of documents bằng variant of principal components analysis (Section 12.2). Idea là positions của words trong embedding space preserve semantic meaning; ví dụ synonyms nên nằm gần nhau.

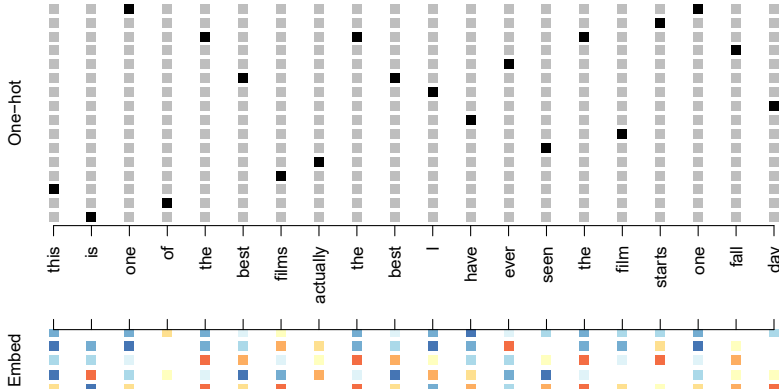
*embedding layer;  
weight freezing;  
word2vec; GloVe*

Mỗi document giờ được represented như sequence of  $m$ -vectors tương ứng sequence of words. Next step là limit mỗi document vào last  $L$  words.

<sup>15</sup>Đây là một sequence of vectors; mỗi element  $x_{i\ell}$  là một  $p$ -vector.

<sup>16</sup>word2vec được mô tả trong Mikolov, Chen, Corrado, and Dean (2013), tại <https://code.google.com/archive/p/word2vec>. GloVe được mô tả trong Pennington, Socher, and Manning (2014), tại <https://nlp.stanford.edu/projects/glove>.





**HÌNH 10.13.** *Depiction của sequence 20 words đại diện một single document: one-hot encoded bằng dictionary 16 words (top panel) và embedded trong  $m$ -dimensional space với  $m = 5$  (bottom panel).*

Documents shorter than  $L$  được padded bằng zeros ở front. Vì vậy mỗi document trở thành series gồm  $L$  vectors  $X = \{X_1, X_2, \dots, X_L\}$ , mỗi  $X_\ell$  có  $m$  components.

Ta dùng RNN structure trong Figure 10.12. Training corpus gồm  $n$  separate series (documents) length  $L$ , mỗi series được process sequentially left-to-right. Trong process này, parallel series of hidden activation vectors  $A_\ell$ ,  $\ell = 1, \dots, L$ , được tạo theo (10.16) cho mỗi document.  $A_\ell$  feeds vào output layer tạo evolving prediction  $O_\ell$ . Final value  $O_L$  được dùng để predict response: sentiment của review.

Đây là simple RNN với relatively few parameters. Nếu có  $K$  hidden units, common weight matrix  $W$  có  $K \times (m + 1)$  parameters, matrix  $U$  có  $K \times K$ , và  $B$  có  $2(K + 1)$  parameters cho two-class logistic regression như (10.15). Chúng được reuse khi process sequence  $X = \{X_\ell\}_1^L$  left-to-right, tương tự reuse một single convolution filter cho mỗi patch trong image (Section 10.3.1). Nếu embedding layer  $E$  được learned, nó add thêm  $m \times D$  parameters ( $D = 10,000$  ở đây), và đây là cost lớn nhất.

Ta fit RNN vừa mô tả cho IMDb data. Model có embedding matrix  $E$  với  $m = 32$  (learned during training, không phải precomputed), followed by single recurrent layer với  $K = 32$  hidden units. Model được trained với dropout regularization trên 25,000 reviews trong designated training set và chỉ đạt disappointing 76% accuracy trên IMDb test data. Network dùng pretrained GloVe embedding matrix  $E$  còn slightly worse.

Để dễ exposition ta chỉ trình bày very simple RNN. Versions phức tạp hơn dùng *long term and short term memory* (LSTM). Hai tracks của hidden-layer activations được maintained, nên khi activation  $A_\ell$  được compute, nó nhận input từ hidden units both further back in time và closer in time—một so-called *LSTM RNN*. Với long sequences, cách này khắc phục problem early signals bị washed out khi propagated qua chain tới final activation vector  $A_L$ .

Khi refit model bằng LSTM architecture cho hidden layer, performance improve lên 87% trên IMDb test data, comparable với 88% của bag-of-words model trong Section 10.4. Section 10.9.6 cung cấp fitting details. Dù

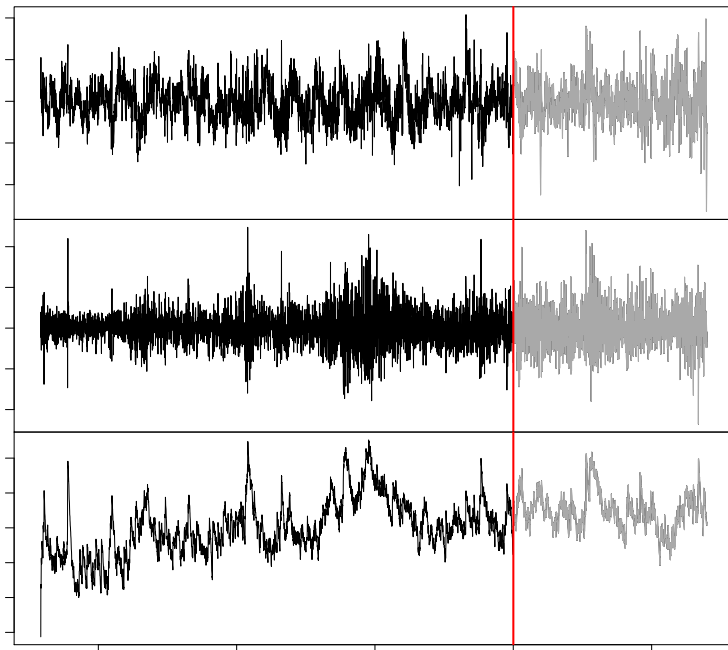


thêm LSTM complexity, RNN này vẫn chỉ ở mức “entry level”. Ta có thể improve đôi chút bằng cách change model size, regularization và add hidden layers, nhưng LSTM models train lâu nên exploration nhiều architectures và parameter optimization khá tedious.

RNNs cung cấp rich framework cho sequence modeling và tiếp tục evolve. Đã có nhiều advances về architecture, data augmentation và learning algorithms. Tại thời điểm sách được viết (early 2020), leading RNN configurations report accuracy trên 95% cho IMDB data; details nằm ngoài scope của sách.<sup>17</sup>

### 10.5.2 Time Series Forecasting

Figure 10.14 cho historical trading statistics từ New York Stock Exchange: three daily time series trong period December 3, 1962 đến December 31, 1986.<sup>18</sup>



**HÌNH 10.14.** Historical trading statistics từ New York Stock Exchange. Daily values của normalized log trading volume, DJIA return và log volatility trong 24-year period 1962–1986. Ta muốn predict trading volume của một ngày dựa trên history của all earlier days. Bên trái red bar (January 2, 1980) là training data, bên phải là test data.

Ba series là:

- *Log trading volume*: fraction của all outstanding shares được traded trong ngày, relative to 100-day moving average của past turnover,

<sup>17</sup>Một IMDB leaderboard có tại <https://paperswithcode.com/sota/sentiment-analysis-on-imdb>.

<sup>18</sup>Data được assembled bởi LeBaron and Weigend (1998), *IEEE Transactions on Neural Networks*, 9(1): 213–220.

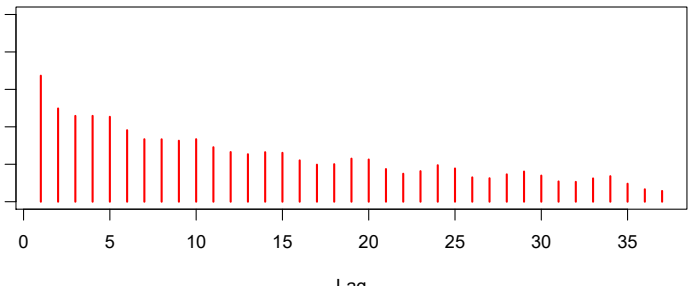
trên log scale.

- *Dow Jones return*: difference giữa log Dow Jones Industrial Index trên consecutive trading days.
- *Log volatility*: based on absolute values của daily price movements.

Predicting stock prices notoriously hard, nhưng predicting trading volume dựa trên recent history lại manageable hơn và useful cho planning trading strategies.

Một observation gồm measurements  $(v_t, r_t, z_t)$  ở day  $t$ , tương ứng `log_volume`, `DJ_return` và `log_volatility`. Có tổng cộng  $T = 6,051$  triples, mỗi series được plot trong Figure 10.14. Một feature nổi bật là day-to-day observations không independent. Series có *autocorrelation*: values gần nhau theo time có tendency similar. Điều này khác các datasets trước, nơi observations thường được assumed independent.

autocorrelation



**HÌNH 10.15.** Autocorrelation function cho `log_volume`. Nearby values correlated khá strongly, với correlations trên 0.2 tới khoảng lag 20 days.

Cụ thể, xét pairs  $(v_t, v_{t-\ell})$  cách nhau lag  $\ell$  days. Lấy all such pairs trong  $v_t$  series và compute correlation coefficient cho ta autocorrelation ở lag  $\ell$ . Figure 10.15 show autocorrelation function tới lag 37, với considerable correlation. Một characteristic thú vị khác là response variable  $v_t$ —`log_volume`—cũng là predictor: ta dùng past values của `log_volume` để predict future values.

lag

### RNN forecaster

Ta muốn predict  $v_t$  từ past values  $v_{t-1}, v_{t-2}, \dots$  và đồng thời dùng past values của  $r_{t-1}, r_{t-2}, \dots$  và  $z_{t-1}, z_{t-2}, \dots$ . Dù combined data là long series 6,051 trading days, structure của problem khác document-classification example:

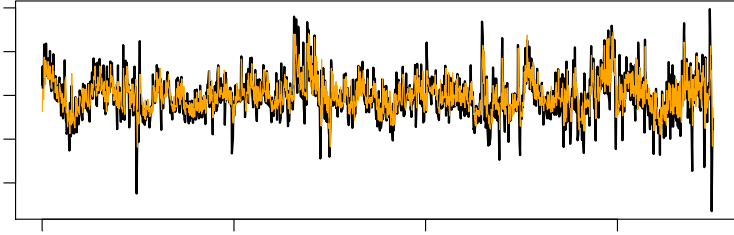
- chỉ có one series of data, không phải 25,000;
- có entire series of targets  $v_t$ , và inputs include past values của series này.

Để represent problem theo Figure 10.12, ta extract nhiều short mini-series input sequences  $X = \{X_1, X_2, \dots, X_L\}$  với predefined length  $L$  (gọi

là lag trong context này) và corresponding target  $Y$ :

$$X_1 = \begin{pmatrix} v_{t-L} \\ r_{t-L} \\ z_{t-L} \end{pmatrix}, \quad X_2 = \begin{pmatrix} v_{t-L+1} \\ r_{t-L+1} \\ z_{t-L+1} \end{pmatrix}, \quad \dots, \quad X_L = \begin{pmatrix} v_{t-1} \\ r_{t-1} \\ z_{t-1} \end{pmatrix}, \quad Y = v_t. \quad (10.20)$$

Target  $Y$  là `log_volume` value  $v_t$  tại single timepoint  $t$ ; input sequence  $X$  là series of 3-vectors  $\{X_\ell\}_1^L$ , mỗi vector gồm `log_volume`, `DJ_return`, `log_volatility` từ day  $t-L$  đến  $t-1$ . Mỗi  $t = L+1, \dots, T$  tạo một separate  $(X, Y)$  pair. Với NYSE data, ta dùng past five trading days để predict next day, nên  $L = 5$ . Vì  $T = 6,051$ , ta tạo được 6,046 pairs. Rõ ràng  $L$  là parameter cần chọn cẩn thận, có thể dùng validation data.



**HÌNH 10.16.** RNN forecast của `log_volume` trên NYSE test data. Black lines là true volumes, orange lines superimposed là forecasts. Forecasted series account for 42% variance của `log_volume`.

Ta fit model với  $K = 12$  hidden units bằng 4,281 training sequences derived từ data trước January 2, 1980 (Figure 10.14), rồi forecast 1,770 values của `log_volume` sau date này. Test  $R^2 = 0.42$ . Details ở Section 10.9.6. Là một *straw man*,<sup>19</sup> dùng yesterday's `log_volume` để predict today chỉ đạt  $R^2 = 0.18$ . Figure 10.16 show forecast results; observed daily values cho 1980–1986 bằng black, predictions bằng orange, và correspondence khá tốt. Khi forecast test period, ta phải dùng chính past test data để form input sequences  $X$ ; điều này không phải cheating vì ta luôn dùng past data để predict future.

#### Autoregression

RNN vừa fit có nhiều điểm chung với traditional *autoregression* (AR) linear model. Trước hết xét response sequence  $v_t$  một mình và construct response vector  $y$  cùng predictor matrix  $M$ :

$$y = \begin{bmatrix} v_{L+1} \\ v_{L+2} \\ v_{L+3} \\ \vdots \\ v_T \end{bmatrix}, \quad M = \begin{bmatrix} 1 & v_L & v_{L-1} & \cdots & v_1 \\ 1 & v_{L+1} & v_L & \cdots & v_2 \\ 1 & v_{L+2} & v_{L+1} & \cdots & v_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & v_{T-1} & v_{T-2} & \cdots & v_{T-L} \end{bmatrix}. \quad (10.21)$$

$M$  và  $y$  đều có  $T-L$  rows, one per observation. Predictors cho response  $v_t$  ở day  $t$  là previous  $L$  values của same series.

*autoregression*

<sup>19</sup>Straw man ở đây là simple và sensible prediction dùng làm baseline comparison.

Regressing  $y$  on  $M$  tương đương fit

$$\hat{v}_t = \hat{\beta}_0 + \hat{\beta}_1 v_{t-1} + \hat{\beta}_2 v_{t-2} + \cdots + \hat{\beta}_L v_{t-L}, \quad (10.22)$$

gọi là order- $L$  autoregressive model, hay  $AR(L)$ . Với NYSE data ta có thể include lagged versions của `DJ_return` và `log_volatility`,  $r_t$  và  $z_t$ , trong predictor matrix  $M$ , tạo  $3L + 1$  columns. AR model với  $L = 5$  đạt test  $R^2 = 0.41$ , slightly inferior to RNN's 0.42.

RNN và AR models rất similar: cùng response  $Y$  và input sequences  $X$  length  $L = 5$ , dimension  $p = 3$ . RNN process sequence left-to-right với same weights  $W$  (input layer), trong khi AR model simply treat all  $L$  sequence elements equally như vector of  $L \times p$  predictors—process gọi là *flattening*. RNN còn có hidden activations  $A_\ell$  truyền information dọc sequence và add nonlinearity. Từ (10.19), với  $K = 12$  hidden units, RNN có  $13 + 12(1 + 3 + 12) = 205$  parameters, so với 16 của  $AR(5)$ .

*flattening*

Một obvious extension của AR model là dùng set of lagged predictors làm input vector cho ordinary feedforward neural network (10.1), thêm flexibility. Cách này đạt test  $R^2 = 0.42$ , slightly better linear AR và same as RNN.

Tất cả models có thể improve nếu include variable `day_of_week` tương ứng day  $t$  của target  $v_t$  (learn được từ calendar dates); trading volume thường higher vào Mondays và Fridays. Với five trading days, variable này one-hot encode thành five binary variables. AR và RNN đều improve lên  $R^2 = 0.46$ , còn nonlinear AR lên  $R^2 = 0.47$ .

Ta dùng simplest RNN version trong examples. Experiments bổ sung với LSTM extension chỉ yield small improvements, thường tới khoảng 1% trong  $R^2$ . Fitting details của cả three models ở Section 10.9.6.

### 10.5.3 Summary of RNNs

Ta đã illustrate RNNs bằng two simple use cases và mới chỉ scratched the surface. Có nhiều variations và enhancements. Một approach không discuss ở trên dùng one-dimensional convolutional neural network, treat sequence of vectors (ví dụ words represented trong embedding space) như image. Convolution filter slides dọc sequence theo one dimension, có thể learn particular phrases hoặc short subsequences relevant cho task.

RNN cũng có thể có additional hidden layers. Với two hidden layers chẳng hạn, sequence  $A_\ell$  được treat như input sequence cho next hidden layer theo natural way.

RNN trong example scan document beginning-to-end; *bidirectional RNNs* thay vào đó scan sequences theo both directions. Trong language translation, target cũng là sequence of words ở một language khác input. Cả input sequence và target sequence được represented bằng structure tương tự Figure 10.12 và share hidden units. Trong so-called *Seq2Seq learning*, hidden units được xem như capture semantic meaning của sentences. Một số breakthroughs lớn trong language modeling và translation đến từ recent improvements của RNNs.

*bidirectional;  
Seq2Seq*

Algorithms để fit RNNs có thể complex và computationally costly. Fortunately, good software che bớt complexities và làm specification/fitting khá painless. Nhiều models ta dùng hằng ngày (như Google Translate) dùng state-of-the-art architectures phát triển bởi teams of highly skilled

engineers và được trained bằng massive computational và data resources.

## 10.6 When to Use Deep Learning

Performance của deep learning trong chapter này rất impressive: digit classification được xử lý rất tốt và deep CNNs đã revolutionize image classification. Ta thường xuyên thấy new success stories, nhiều trong số đó liên quan image classification như machine diagnosis of mammograms/digital X-rays, ophthalmology scans, MRI annotations, v.v. RNNs cũng thành công trong speech/language translation, forecasting và document modeling. Câu hỏi tự nhiên là: có nên bỏ các older tools và dùng deep learning cho mọi problem with data? Để trả lời, ta revisit **Hitters** dataset từ Chapter 6.

Đây là regression problem: predict **Salary** của baseball player năm 1987 từ performance statistics năm 1986. Sau khi remove players missing response, còn 263 players và 19 variables. Ta randomly split thành training set 176 players (two thirds) và test set 87 players (one third), rồi dùng three methods:

- linear model fit training data và predict test data; model có 20 parameters;
- same linear model với lasso regularization; tuning parameter selected bằng 10-fold cross-validation trên training data, chọn model có 12 variables với nonzero coefficients;
- neural network với one hidden layer gồm 64 ReLU units; model có 1,345 parameters.<sup>20</sup>

| Model             | # Parameters | Mean Abs. Error | Test Set $R^2$ |
|-------------------|--------------|-----------------|----------------|
| Linear Regression | 20           | 254.7           | 0.56           |
| Lasso             | 12           | 252.3           | 0.51           |
| Neural Network    | 1345         | 257.4           | 0.54           |

**BẢNG 10.2.** Prediction results trên **Hitters** test data cho linear models fit bằng ordinary least squares và lasso, so với neural network fit bằng stochastic gradient descent với dropout regularization.

Table 10.2 compare results. Performance khá similar cho all three models. Ta report mean absolute error trên test data và test  $R^2$ ; tất cả đều respectable (xem Exercise 5). Ta đã spend khá nhiều time fiddling với configuration parameters của neural network để đạt results này. Có thể nếu spend thêm time và chọn form/amount of regularization vừa đúng, neural network có thể match hoặc outperform linear regression và lasso. Nhưng linear models work well với great ease, và much easier to present/understand hơn neural-network black box. Lasso select 12 trong 19 variables. Trong case như vậy, tốt hơn nên theo *Occam's razor*: khi nhiều methods có roughly equivalent performance, choose simplest.

*Occam's razor*

Sau thêm exploration với lasso, ta identify even simpler model với four variables. Ta refit linear model với four variables trên training data (so-

<sup>20</sup>Model được fit bằng stochastic gradient descent với batch size 32 trong 1,000 epochs và 10% dropout regularization. Test-error performance flatten rồi bắt đầu slowly increase sau 1,000 epochs. Fitting details ở Section 10.7.

called *relaxed lasso*) và đạt test mean absolute error 224.8, overall winner. Ta có thể muốn present summary table từ fit này để xem coefficients và p-values, nhưng vì model được selected trên training data nên sẽ có selection bias. Thay vào đó, authors refit model trên test data, vốn không dùng trong selection. Table 10.3 cho results.

|           | Coefficient | Std. error | t-statistic | p-value  |
|-----------|-------------|------------|-------------|----------|
| Intercept | -226.67     | 86.26      | -2.63       | 0.0103   |
| Hits      | 3.06        | 1.02       | 3.00        | 0.0036   |
| Walks     | 0.181       | 2.04       | 0.09        | 0.9294   |
| CRuns     | 0.859       | 0.12       | 7.09        | < 0.0001 |
| PutOuts   | 0.465       | 0.13       | 3.60        | 0.0005   |

**BẢNG 10.3.** *Least squares coefficient estimates associated với regression của Salary lên four variables selected by lasso trên Hitters data. Model này đạt best performance trên test data với mean absolute error 224.8. Results ở đây được obtained từ regression trên test data, vốn không dùng khi fitting lasso model.*

Ta có nhiều powerful tools: neural networks, random forests/boosting, support vector machines, generalized additive models, v.v., cùng linear models và simple variants. Khi gặp new data modeling/prediction problems, rất tempting luôn chọn trendy new methods. Chúng thường rất impressive, đặc biệt khi datasets extremely large và support fitting high-dimensional nonlinear models. Nhưng nếu simpler tools cho performance tương đương, chúng likely easier to fit/understand và potentially less fragile. Wherever possible, nên thử simple models rồi choose dựa trên performance/complexity tradeoff.

Thông thường deep learning attractive khi training sample size extremely large và model interpretability không phải high priority.

## 10.7 Fitting a Neural Network

Fitting neural networks tương đối complex; ở đây chỉ overview ngắn, ideas generalize tới much more complex networks. Readers thấy material challenging có thể skip. Good software trong lab cuối chapter cho phép fit neural-network models tương đối automated mà không phải worry về technical details của fitting procedure.

Bắt đầu với simple network Figure 10.1. Trong model (10.1), parameters là  $\beta = (\beta_0, \beta_1, \dots, \beta_K)$  cùng  $w_k = (w_{k0}, w_{k1}, \dots, w_{kp})$ ,  $k = 1, \dots, K$ . Với observations  $(x_i, y_i)$ ,  $i = 1, \dots, n$ , có thể fit bằng nonlinear least squares:

$$\underset{\{w_k\}_1^K, \beta}{\text{minimize}} \frac{1}{2} \sum_{i=1}^n (y_i - f(x_i))^2, \quad (10.23)$$

trong đó

$$f(x_i) = \beta_0 + \sum_{k=1}^K \beta_k g \left( w_{k0} + \sum_{j=1}^p w_{kj} x_{ij} \right). \quad (10.24)$$

Objective (10.23) nhìn simple, nhưng nested arrangement của parameters và symmetry của hidden units khiến minimization không straightforward. Problem nonconvex theo parameters, nên có multiple solutions. Figure

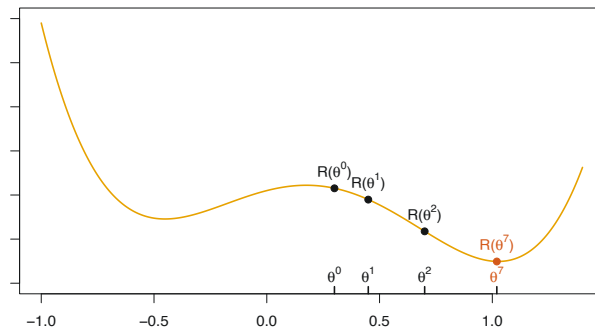
10.17 cho simple nonconvex function của one variable  $\theta$  với two solutions: one local minimum và one global minimum. Đây còn là simplest neural network; more complex networks làm vấn đề compounded. Hai general strategies thường được dùng để overcome issues và protect from overfitting:

- *Slow Learning*: fit model bằng iterative gradient descent tương đối slow, rồi stop fitting process khi detect overfitting;
- *Regularization*: impose penalties lên parameters, thường lasso hoặc ridge như Section 6.2.

Represent tất cả parameters thành long vector  $\theta$ . Objective (10.23) có thể viết

*local minimum;  
global minimum;  
gradient descent*

$$R(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2. \quad (10.25)$$



**HÌNH 10.17.** Illustration của gradient descent cho one-dimensional  $\theta$ . Objective  $R(\theta)$  không convex và có two minima: one tại  $\theta = -0.46$  (local), one tại  $\theta = 1.02$  (global). Bắt đầu từ  $\theta^0$  (typically randomly chosen), mỗi step đi downhill—against gradient—cho đến khi không thể xuống thêm. Ở đây gradient descent reach global minimum trong 7 steps.

Idea của gradient descent rất simple:

1. Start với guess  $\theta^0$  cho all parameters và set  $t = 0$ .
2. Iterate cho đến khi objective (10.25) không giảm nữa:
  - (a) find vector  $\delta$  phản ánh small change trong  $\theta$ , sao cho  $\theta^{t+1} = \theta^t + \delta$  reduces objective, tức  $R(\theta^{t+1}) < R(\theta^t)$ ;
  - (b) set  $t \leftarrow t + 1$ .

Có thể visualize việc đứng trên mountainous terrain và muốn đi xuống bottom qua series of steps. Miễn mỗi step downhill, cuối cùng ta tới một bottom. Example này lucky vì starting guess  $\theta^0$  dẫn tới global minimum; nói chung chỉ có thể hope đạt một good local minimum.

### 10.7.1 Backpropagation

Làm sao tìm direction để move  $\theta$  nhằm decrease  $R(\theta)$  trong (10.25)? Gradient của  $R(\theta)$  tại current value  $\theta = \theta^m$  là vector partial derivatives:

$$\nabla R(\theta^m) = \left. \frac{\partial R(\theta)}{\partial \theta} \right|_{\theta=\theta^m}. \quad (10.26)$$

Subscript  $\theta = \theta^m$  nghĩa là sau khi compute vector derivatives, ta evaluate tại current guess  $\theta^m$ . Vector này cho direction trong  $\theta$ -space mà  $R(\theta)$  increases most rapidly. Gradient descent move một chút theo opposite direction:

$$\theta^{m+1} \leftarrow \theta^m - \rho \nabla R(\theta^m). \quad (10.27)$$

gradient

learning rate

Với learning rate  $\rho$  đủ small, step này decrease objective:  $R(\theta^{m+1}) \leq R(\theta^m)$ . Nếu gradient vector bằng zero, có thể ta đã reach a minimum.

Calculation (10.26) phức tạp đến đâu? Ở đây nó khá simple và vẫn simple với much more complex networks nhờ chain rule of differentiation. Vì  $R(\theta) = \sum_{i=1}^n R_i(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_\theta(x_i))^2$  là sum, gradient cũng là sum over  $n$  observations; chỉ cần examine one term:

$$R_i(\theta) = \frac{1}{2} \left[ y_i - \beta_0 - \sum_{k=1}^K \beta_k g \left( w_{k0} + \sum_{j=1}^p w_{kj} x_{ij} \right) \right]^2. \quad (10.28)$$

chain rule

Để simplify expressions, đặt  $z_{ik} = w_{k0} + \sum_{j=1}^p w_{kj} x_{ij}$ .

Derivative theo  $\beta_k$ :

$$\frac{\partial R_i(\theta)}{\partial \beta_k} = \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial \beta_k} = -(y_i - f_\theta(x_i)) g(z_{ik}). \quad (10.29)$$

Derivative theo  $w_{kj}$ :

$$\begin{aligned} \frac{\partial R_i(\theta)}{\partial w_{kj}} &= \frac{\partial R_i(\theta)}{\partial f_\theta(x_i)} \cdot \frac{\partial f_\theta(x_i)}{\partial g(z_{ik})} \cdot \frac{\partial g(z_{ik})}{\partial z_{ik}} \cdot \frac{\partial z_{ik}}{\partial w_{kj}} \\ &= -(y_i - f_\theta(x_i)) \beta_k g'(z_{ik}) x_{ij}. \end{aligned} \quad (10.30)$$

Cả hai expressions chứa residual  $y_i - f_\theta(x_i)$ . Trong (10.29), một fraction của residual được attributed tới mỗi hidden unit theo  $g(z_{ik})$ ; trong (10.30), similar attribution đi tới input  $j$  qua hidden unit  $k$ . Differentiation vì thế assign fraction của residual tới từng parameter thông qua chain rule—process gọi là *backpropagation*. Calculations straightforward nhưng cần careful bookkeeping.

backpropagation

### 10.7.2 Regularization and Stochastic Gradient Descent

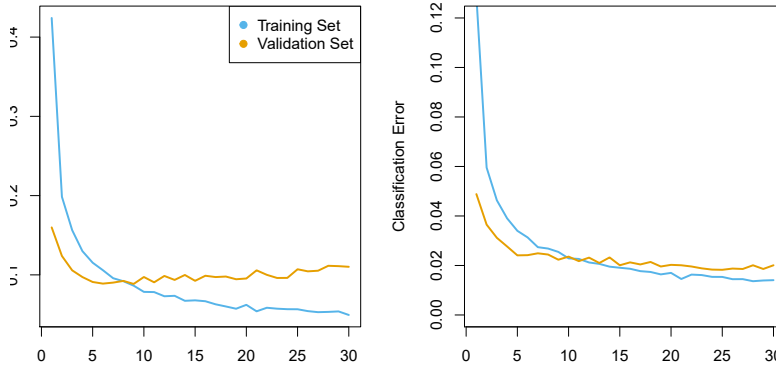
Gradient descent thường cần many steps để reach local minimum. Có nhiều approaches để accelerate process. Khi  $n$  large, thay vì sum (10.29)–(10.30) over all  $n$  observations, ta sample một small fraction hoặc *minibatch* mỗi lần compute gradient step. Process này gọi là *stochastic gradient descent* (SGD), state of the art cho learning deep neural networks. Good software



*minibatch;  
stochastic gradient  
descent*

giúp set up/fitting deep learning models và hide hầu hết technicalities khỏi user.

Quay lại multilayer network Figure 10.4 dùng cho digit recognition. Network có hơn 235,000 weights, khoảng four times number of training examples; regularization là essential.



**HÌNH 10.18.** Evolution của training và validation errors cho MNIST neural network Figure 10.4 theo training epochs. Objective refers to log-likelihood (10.14).

First row Table 10.1 dùng ridge regularization trên weights. Điều này được thực hiện bằng augment objective (10.14) với penalty term:

$$R(\theta; \lambda) = - \sum_{i=1}^n \sum_{m=0}^9 y_{im} \log(f_m(x_i)) + \lambda \sum_j \theta_j^2. \quad (10.31)$$

Parameter  $\lambda$  thường preset small hoặc find bằng validation-set approach. Ta cũng có thể dùng different  $\lambda$  cho groups of weights ở different layers; trong example này  $W_1$  và  $W_2$  bị penalized, còn relatively few weights  $B$  của output layer không penalized. Lasso regularization cũng popular như additional form hoặc alternative to ridge.

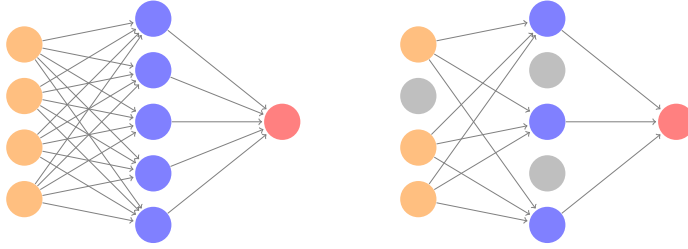
Figure 10.18 show metrics evolve trong training MNIST network. SGD tự nhiên enforce một form of approximately quadratic regularization.<sup>21</sup> Mini-batch size ở đây là 128 observations per gradient update. Horizontal-axis term *epochs* counts number of times equivalent of full training set được processed. Với network này, 20% của 60,000 training observations được dùng làm validation set để determine when training should stop; thực tế 48,000 observations dùng train, nên có khoảng  $48,000/128 \approx 375$  minibatch gradient updates per epoch. Validation objective bắt đầu increase vào khoảng 30 epochs, vì vậy *early stopping* cũng có thể act như additional regularization.

*epochs; early  
stopping*

### 10.7.3 Dropout Learning

Second row Table 10.1 labeled *dropout*. Đây là relatively new và efficient form of regularization, similar in some respects to ridge. Inspired by random

<sup>21</sup>Property này và các properties khác của SGD cho deep learning là subject of much research trong machine learning literature tại thời điểm sách được viết.



**HÌNH 10.19.** *Dropout Learning.* Left: fully connected network. Right: network với dropout trong input và hidden layer. Grey nodes được selected randomly và ignored trong một instance of training.

forests (Section 8.2), idea là randomly remove fraction  $\phi$  của units trong một layer khi fit model. Figure 10.19 minh họa. Việc remove được thực hiện separately mỗi khi process một training observation. Surviving units stand in for missing ones và weights được scale up factor  $1/(1-\phi)$  để compensate. Điều này prevent nodes from becoming over-specialized và có thể xem như regularization. Trong practice dropout được thực hiện bằng randomly set activations của “dropped out” units về zero, vẫn giữ architecture intact.

dropout

#### 10.7.4 Network Tuning

Network Figure 10.4 được xem là relatively straightforward nhưng vẫn cần nhiều choices ảnh hưởng performance:

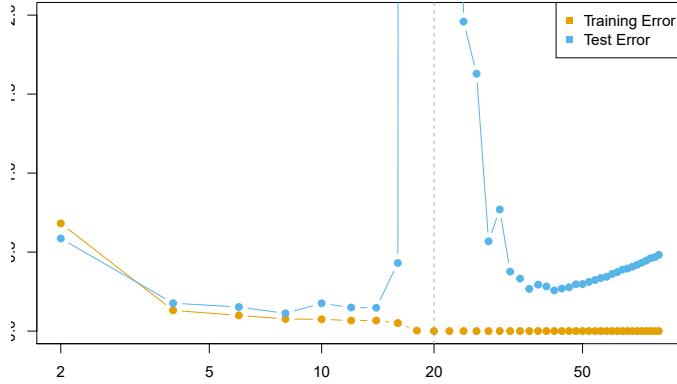
- number of hidden layers và units per layer. Modern thinking cho rằng units per hidden layer có thể large và overfitting controlled bằng regularization;
- regularization tuning parameters, gồm dropout rate  $\phi$  và strength  $\lambda$  của lasso/ridge, thường set separately ở each layer;
- details của stochastic gradient descent: batch size, number of epochs và, nếu dùng, details của data augmentation (Section 10.3.4).

Những choices này có thể matter. Khi prepare MNIST example, authors đạt respectable 1.8% misclassification error sau some trial and error. Finer tuning/training của similar network có thể xuống dưới 1%, nhưng tinkering tedious và có thể cause overfitting nếu careless.

### 10.8 Interpolation and Double Descent

Trong suốt sách, ta repeatedly discuss bias-variance trade-off, first presented ở Section 2.2.2. Trade-off này chỉ ra statistical learning methods thường perform best về test-set error tại intermediate model complexity. Nếu plot “flexibility” trên x-axis và error trên y-axis, ta thường expect test error U-shaped trong khi training error decreases monotonically. Examples điển hình có ở Figure 2.9 (right panel) và Figure 2.17. Một implication là thường không nên *interpolate* training data—tức achieve zero training error—vì thường dẫn tới very high test error.

interpolate



**HÌNH 10.20.** Double descent phenomenon, illustrated bằng error plots cho one-dimensional natural-spline example. Horizontal axis là number of spline basis functions trên log scale. Training error hit zero khi degrees of freedom bằng sample size  $n = 20$ , “interpolation threshold”, rồi remain zero. Test error increases dramatically tại threshold nhưng sau đó descends again tới reasonable value trước khi cuối cùng increase trở lại.

Tuy vậy, trong certain settings, statistical learning method interpolate training data vẫn có thể perform well—hoặc ít nhất better than một slightly less complex model chưa hẳn interpolate. Phenomenon này gọi là *double descent*, shown trong Figure 10.20. Tên “double descent” vì test error có U-shape trước interpolation threshold rồi descend thêm lần nữa (ít nhất trong một khoảng) khi increasingly flexible model được fit.

Set-up cho Figure 10.20: simulate  $n = 20$  observations từ

$$Y = \sin(X) + \varepsilon,$$

trong đó  $X \sim U[-5, 5]$  và  $\varepsilon \sim N(0, \sigma^2)$  với  $\sigma = 0.3$ . Sau đó fit natural spline như Section 7.4 với  $d$  degrees of freedom.<sup>22</sup>

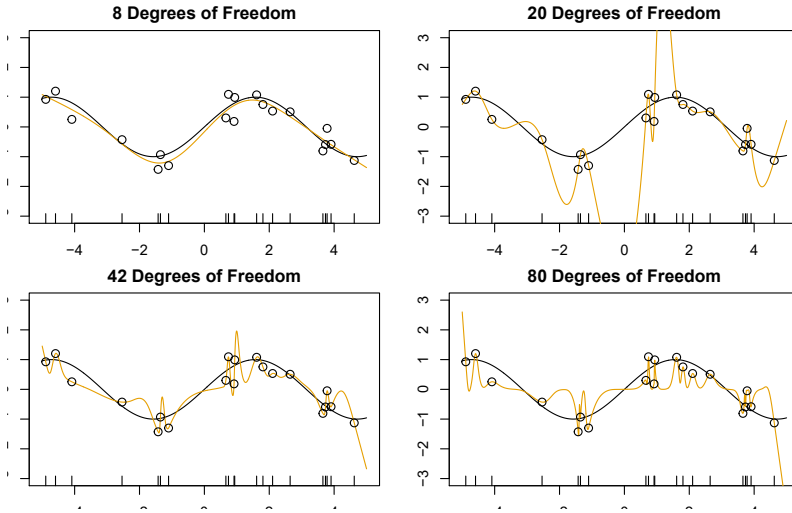
Recall rằng fit natural spline với  $d$  degrees of freedom tương đương fit least-squares regression của response lên set of  $d$  basis functions. Upper-left panel Figure 10.21 show data, true function  $f(X)$  và  $\hat{f}_8(X)$ , fitted natural spline với  $d = 8$ .

Tiếp theo fit natural spline với  $d = 20$ . Vì  $n = 20$ , ta có  $n = d$  và zero training error: training data được interpolated. Top-right panel Figure 10.21 cho thấy  $\hat{f}_{20}(X)$  có wild excursions nên test error sẽ large.

Tiếp tục tăng  $d$ . Khi  $d > 20$ , least-squares regression của  $Y$  lên  $d$  basis functions không unique: có infinitely many coefficient estimates đạt zero error. Để choose, ta lấy solution có smallest sum of squared coefficients  $\sum_{j=1}^d \hat{\beta}_j^2$ , gọi là *minimum-norm solution*.

Hai lower panels Figure 10.21 show minimum-norm natural-spline fits với  $d = 42$  và  $d = 80$ . Remarkably,  $\hat{f}_{42}(X)$  much less wild than  $\hat{f}_{20}(X)$  dù dùng more degrees of freedom;  $\hat{f}_{80}(X)$  cũng không khác nhiều. Lý do: với  $n = d = 20$  chỉ có one way để interpolate bằng 20 basis functions, và way đó tạo extreme fitted function. Nhưng với  $d = 42$  hoặc 80 có infinitely

<sup>22</sup>Điều này imply choice of  $d$  knots, ở đây chosen tại  $d$  equ-probability quantiles của training data. Khi  $d > n$ , quantiles được found by interpolation.



**HÌNH 10.21.** Fitted functions  $\hat{f}_d(X)$  (orange), true function  $f(X)$  (black), và 20 observed training data points. Mỗi panel dùng different  $d$  (degrees of freedom). Với  $d \geq 20$ , orange curves đều interpolate training points nên training error bằng zero.

many interpolating fits; smoothest one, tức minimum-norm solution, much less wild.

Figure 10.20 display training/test error của  $\hat{f}_d(X)$  qua range of  $d$ . Training error drop to zero từ  $d = 20$  trở đi, tức interpolation threshold. Test error U-shaped với  $d \leq 20$ , becomes extremely large quanh  $d = 20$ , rồi có second descent region khi  $d > 20$ . Trong example này signal-to-noise ratio  $\text{Var}(f(X))/\sigma^2 = 5.9$ , khá high, nên estimate interpolate data mà không wander quá xa giữa observed points likely perform well.

Figures 10.20–10.21 minh họa double descent trong simple one-dimensional natural-spline setting, nhưng phenomenon cũng có thể arise trong deep learning. Khi fit neural networks với huge number of parameters, đôi khi ta obtain good results với zero training error, đặc biệt ở high signal-to-noise problems như natural image recognition và language translation. Techniques fit neural networks, including SGD, naturally tend to select “smooth” interpolating model có good test performance trong những problems kiểu này.

Một số points cần emphasize:

- Double descent không contradict bias-variance trade-off. Right-hand double-descent curve trong Figure 10.20 xuất hiện vì x-axis là number of spline basis functions, không properly capture true “flexibility” của models interpolate training data. Nói cách khác, minimum-norm natural spline với  $d = 42$  có lower variance than spline với  $d = 20$ .
- Most statistical learning methods trong sách không exhibit double descent. Regularization approaches thường không interpolate training data, nên double descent không occur; đây không phải drawback. Trong examples này, nếu natural splines được fit bằng ridge regression với appropriately-chosen penalty thay vì least squares, ta sẽ không

thấy double descent và thực tế test error còn better.

- Chapter 9 cho thấy maximal margin classifiers và SVMs có zero training error vẫn thường achieve very good test error, partly vì chúng seek smooth minimum-norm solutions. Điều này tương tự minimum-norm natural spline có good results với zero training error.
- Double-descent phenomenon được machine learning community dùng để explain practice dùng over-parametrized neural network (many layers, many hidden units) rồi fit tới zero training error. Tuy vậy, zero error không phải lúc nào optimal; advisability depend on signal-to-noise ratio. Có thể dùng ridge regularization để avoid overfitting neural network như (10.31); với suitable  $\lambda$ , model sẽ không interpolate training data nên không thấy double descent, nhưng vẫn có thể đạt very good test performance. Early stopping trong SGD cũng có thể act như regularization, prevent interpolation mà vẫn cho good test results.

Tóm lại, dù double descent đôi khi occur trong neural networks, thường ta không muốn rely on behavior này. Quan trọng hơn, bias-variance trade-off luôn hold; chỉ là test error as a function of “flexibility” có thể không exhibit U-shape nếu notion of flexibility trên x-axis được parametrized theo một cách không phù hợp.

## 10.9 Lab: Deep Learning

Trong section này, ta demonstrate cách fit các examples đã thảo luận trong text. Ta dùng Python package `torch`, cùng với `pytorch_lightning`, package cung cấp các utilities để đơn giản hóa việc fitting và evaluating models. Code này có thể chạy rất nhanh trên một số special processors, chẳng hạn chip M1 mới của Apple. Package có structure tốt, flexible và sẽ khá quen thuộc với Python users. Một companion hữu ích là site [pytorch.org/tutorials](https://pytorch.org/tutorials). Phần lớn code của ta được adapted từ đó, cũng như từ documentation của `pytorch_lightning`.<sup>23</sup>

Ta bắt đầu với một số standard imports đã gặp trước đây.

In [1]:

```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
from sklearn.linear_model import \
 (LinearRegression,
 LogisticRegression,
 Lasso)
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import KFold
from sklearn.pipeline import Pipeline
from ISLP import load_data
from ISLP.models import ModelSpec as MS
from sklearn.model_selection import \
 (train_test_split,
 GridSearchCV)
```

### *Torch-Specific Imports*

Có một số imports dành cho `torch`. (Chúng không đi kèm với ISLP, nên phải được install riêng.) Trước hết ta import main library và các essential tools dùng để specify sequentially-structured networks.

In [2]:

```
import torch
from torch import nn
from torch.optim import RMSprop
from torch.utils.data import TensorDataset
```

Có một số helper packages khác cho `torch`. Chẳng hạn, package `torchmetrics` có utilities để compute nhiều metrics nhằm evaluate performance khi fitting một model. Package `torchinfo` cung cấp summary hữu ích về các layers của model. Ta dùng function `read_image()` khi loading test images trong Section 10.9.4.

`torchmetrics;`  
`torchinfo;`  
`read_image()`In [3]:

```
from torchmetrics import (MeanAbsoluteError,
 R2Score)
from torchinfo import summary
from torchvision.io import read_image
```

Package `pytorch_lightning` là một interface ở somewhat higher level cho `torch`, giúp simplify việc specification và fitting models bằng cách giảm lượng boilerplate code cần thiết so với dùng `torch` trực tiếp.

<sup>23</sup>Các URL chính xác tại thời điểm viết là <https://pytorch.org/tutorials/beginner/basics/intro.html> và <https://pytorch-lightning.readthedocs.io/en/latest/>.

```
In [4]: from pytorch_lightning import Trainer
 from pytorch_lightning.loggers import CSVLogger
```

Để reproduce results, ta dùng `seed_everything()`. Ta cũng instruct `torch` dùng deterministic algorithms khi có thể.

*seed\_everything()*

```
In [5]: from pytorch_lightning.utilities.seed import seed_everything
 seed_everything(0, workers=True)
 torch.use_deterministic_algorithms(True, warn_only=True)
```

Ta sẽ dùng một số datasets được ship cùng `torchvision` cho examples: một pretrained network cho image classification, cùng một số transforms dùng cho preprocessing.

*torchvision*

```
In [6]: from torchvision.datasets import MNIST, CIFAR100
 from torchvision.models import (resnet50,
 ResNet50_Weights)
 from torchvision.transforms import (Resize,
 Normalize,
 CenterCrop,
 ToTensor)
```

Ta đã cung cấp một vài utilities trong ISLP dành riêng cho lab này. `SimpleDataModule` và `SimpleModule` là các simple versions của objects dùng trong `pytorch_lightning`, high-level module để fit `torch` models. Dù module này cho phép những advanced uses như computing trên graphical processing units (GPUs) và parallel data processing, ta sẽ không tập trung vào chúng trong lab này. `ErrorTracker` xử lý collections của targets và predictions trên từng mini-batch trong validation hoặc test stage, cho phép compute metric trên toàn bộ validation hoặc test data set.

```
In [7]: from ISLP.torch import (SimpleDataModule,
 SimpleModule,
 ErrorTracker,
 rec_num_workers)
```

Ngoài ra ta còn include một số helper functions để load IMDb database, cùng một lookup map integers tới particular keys trong database. Ta đã include một slightly modified copy của preprocessed IMDb data từ `keras`, một package riêng để fitting deep learning models. Điều này tiết kiệm đáng kể preprocessing và cho phép ta tập trung vào specification và fitting models.

*keras*

```
In [8]: from ISLP.torch.imdb import (load_lookup,
 load_tensor,
 load_sparse,
 load_sequential)
```

Cuối cùng, ta giới thiệu một số utility imports không liên quan trực tiếp tới `torch`. Function `glob()` từ module `glob` dùng để tìm tất cả files matching wildcard characters; ta sẽ dùng nó trong example apply model ResNet50 lên một số images của chính mình. Module `json` sẽ được dùng để load một JSON file nhằm lookup classes để identify labels của pictures trong example ResNet50.

*glob(); json*

In [9]:

```
from glob import glob
import json
```

### 10.9.1 Single Layer Network on Hitters Data

Ta bắt đầu bằng cách fit các models trong Section 10.6 trên `Hitters` data.

In [10]:

```
Hitters = load_data('Hitters').dropna()
n = Hitters.shape[0]
```

Ta sẽ fit hai linear models (least squares và lasso) và compare performance của chúng với neural network. Trong comparison này, ta dùng mean absolute error trên một validation dataset,

$$\text{MAE}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

Ta setup model matrix và response.

In [11]:

```
model = MS(Hitters.columns.drop('Salary'), intercept=False)
X = model.fit_transform(Hitters).to_numpy()
Y = Hitters['Salary'].to_numpy()
```

Method `to_numpy()` ở trên convert pandas data frames hoặc series thành numpy arrays. Ta làm vậy vì sẽ cần dùng `sklearn` để fit lasso model, và nó yêu cầu conversion này. Ta cũng dùng một linear regression method từ `sklearn`, thay vì method trong Chapter 3 từ `statsmodels`, để facilitate comparisons.

`to_numpy()`

Bây giờ ta split data thành test và training, fixing random state mà `sklearn` dùng để thực hiện split.

In [12]:

```
(X_train,
 X_test,
 Y_train,
 Y_test) = train_test_split(X,
 Y,
 test_size=1/3,
 random_state=1)
```

### Linear Models

Ta fit linear model và evaluate test error trực tiếp.

In [13]:

```
hit_lm = LinearRegression().fit(X_train, Y_train)
Yhat_test = hit_lm.predict(X_test)
np.abs(Yhat_test - Y_test).mean()
```

Out[13]:

259.7153

Tiếp theo ta fit lasso bằng `sklearn`. Ở đây ta dùng mean absolute error để select và evaluate model, thay vì mean squared error. Specialized solver dùng trong Section 6.5.2 chỉ dùng mean squared error. Vì vậy ở đây, với thêm một chút work, ta tạo cross-validation grid và thực hiện cross-validation trực tiếp.



Ta encode một pipeline với hai steps: trước hết normalize features bằng transform `StandardScaler()`, sau đó fit lasso mà không normalize thêm.

In [14]:

```
scaler = StandardScaler(with_mean=True, with_std=True)
lasso = Lasso(warm_start=True, max_iter=30000)
standard_lasso = Pipeline(steps=[('scaler', scaler),
 ('lasso', lasso)])
```

Ta cần tạo một grid các values của  $\lambda$ . Theo common practice, ta chọn grid gồm 100 values của  $\lambda$ , uniform trên log scale từ `lam_max` xuống  $0.01 \times \text{lam\_max}$ . Ở đây `lam_max` là smallest value của  $\lambda$  với all-zero solution. Value này bằng largest absolute inner-product giữa bất kỳ predictor nào và centered response.<sup>24</sup>

In [15]:

```
X_s = scaler.fit_transform(X_train)
n = X_s.shape[0]
lam_max = np.fabs(X_s.T.dot(Y_train - Y_train.mean())).max() / n
param_grid = {'alpha': np.exp(np.linspace(0, np.log(0.01), 100))
 * lam_max}
```

Lưu ý ta phải transform data trước vì scale của variables ảnh hưởng tới choice của  $\lambda$ . Bây giờ ta perform cross-validation bằng sequence các  $\lambda$  values này.

In [16]:

```
cv = KFold(10,
 shuffle=True,
 random_state=1)
grid = GridSearchCV(lasso,
 param_grid,
 cv=cv,
 scoring='neg_mean_absolute_error')
grid.fit(X_train, Y_train);
```

Ta extract lasso model có best cross-validated mean absolute error, rồi evaluate performance của nó trên `X_test` và `Y_test`, vốn không được dùng trong cross-validation.

In [17]:

```
trained_lasso = grid.best_estimator_
Yhat_test = trained_lasso.predict(X_test)
np.fabs(Yhat_test - Y_test).mean()
```

Out[17]: 257.2382

Kết quả này tương tự những gì ta nhận được với linear model fit bằng least squares. Tuy nhiên results có thể vary nhiều với different train/test splits; reader được khuyến khích thử different seed trong code block 12 và rerun các code blocks tiếp theo tới point này.

### *Specifying a Network: Classes and Inheritance*

Để fit neural network, trước hết ta setup một model structure mô tả network. Việc này yêu cầu define các new classes specific cho model muốn fit. Thông thường điều này được thực hiện trong `pytorch` bằng cách subclass một generic representation của network; đây cũng là approach ta dùng. Dù example này simple, ta sẽ đi qua các steps khá chi tiết vì chúng sẽ hữu

<sup>24</sup>Derivation của result này nằm ngoài scope của book.

ích cho các examples phức tạp hơn phía sau.

In [18]:

```
class HittersModel(nn.Module):

 def __init__(self, input_size):
 super(HittersModel, self).__init__()
 self.flatten = nn.Flatten()
 self.sequential = nn.Sequential(
 nn.Linear(input_size, 50),
 nn.ReLU(),
 nn.Dropout(0.4),
 nn.Linear(50, 1))

 def forward(self, x):
 x = self.flatten(x)
 return torch.flatten(self.sequential(x))
```

Statement `class` identifies code chunk như declaration cho class `HittersModel`, class này inherits từ base class `nn.Module`. Base class này xuất hiện khắp `torch` và represent mappings trong neural networks.

Indented dưới statement `class` là methods của class: ở đây là `__init__` và `forward`. Method `__init__` được gọi khi một instance của class được create như trong cell dưới. Trong methods, `self` luôn refer tới một instance của class. Trong `__init__`, ta attach hai objects vào `self` như attributes: `flatten` và `sequential`. Chúng được dùng trong `forward` để describe map mà module implements.

Có thêm một line trong method `__init__`: call tới `super()`. Function này cho phép subclasses (ví dụ `HittersModel`) access methods của class mà chúng inherit. Chẳng hạn class `nn.Module` có method `__init__` riêng, khác với method `HittersModel.__init__()` ta vừa viết. Dùng `super()` cho phép call method của base class. Với `torch` models, ta luôn thực hiện call `super()` này vì nó cần thiết để model được `torch` interpret đúng.

Object `nn.Module` có nhiều methods hơn chỉ `__init__` và `forward`. Các methods này trực tiếp accessible cho `HittersModel` instances nhờ inheritance. Một method sẽ gặp ngay sau đây là `eval()`, dùng để disable dropout khi ta muốn evaluate model trên test data.

In [19]:

```
hit_model = HittersModel(X.shape[1])
```

Object `self.sequential` là composition của bốn maps. Map đầu tiên map 19 features của `Hitters` sang 50 dimensions, introducing  $50 \times 19 + 50$  parameters cho weights và intercept của map (thường gọi là bias). Layer này sau đó được map tới một ReLU layer, tiếp theo là 40% dropout layer, và cuối cùng là một linear map xuống 1 dimension, lại có bias. Tổng số trainable parameters vì vậy là  $50 \times 19 + 50 + 50 + 1 = 1051$ .

Package `torchinfo` cung cấp function `summary()` summarize information này một cách gọn gàng. Ta specify size của input và xem size của mỗi tensor khi đi qua các layers của network.

In [20]:

```
summary(hit_model,
 input_size=X_train.shape,
 col_names=['input_size',
 'output_size',
 'num_params'])
```

Out[20]:

```

=====
Layer (type:depth-idx) Input Shape Output Shape Param #
=====
HittersModel [175, 19] [175] --
 Flatten: 1-1 [175, 19] [175, 19] --
 Sequential: 1-2 [175, 19] [175, 1] --
 Linear: 2-1 [175, 19] [175, 50] 1,000
 ReLU: 2-2 [175, 50] [175, 50] --
 Dropout: 2-3 [175, 50] [175, 50] --
 Linear: 2-4 [175, 50] [175, 1] 51
=====
Total params: 1,051
Trainable params: 1,051

```

Ta đã truncate nhẹ phần cuối của output ở đây và ở các uses sau. Bây giờ ta cần transform training data sang form accessible cho **torch**. Basic datatype trong **torch** là tensor, rất giống **ndarray** ở các chapters đầu. Cũng lưu ý **torch** thường làm việc với 32-bit (single precision) thay vì 64-bit (double precision) floating point numbers. Vì vậy ta convert data sang **np.float32** trước khi forming tensor. Các *X* và *Y* tensors sau đó được arranged vào một **Dataset** recognized bởi **torch** bằng **TensorDataset()**.

*Dataset;*  
*TensorDataset()*

In [21]:

```

X_train_t = torch.tensor(X_train.astype(np.float32))
Y_train_t = torch.tensor(Y_train.astype(np.float32))
hit_train = TensorDataset(X_train_t, Y_train_t)

```

Ta làm tương tự cho test data.

In [22]:

```

X_test_t = torch.tensor(X_test.astype(np.float32))
Y_test_t = torch.tensor(Y_test.astype(np.float32))
hit_test = TensorDataset(X_test_t, Y_test_t)

```

Cuối cùng dataset này được pass vào một **DataLoader()** mà sau cùng sẽ pass data vào network. Dù có vẻ khá nhiều overhead, structure này hữu ích cho tasks phức tạp hơn nơi data có thể nằm trên different machines hoặc phải pass tới GPU. Ta cung cấp helper function **SimpleDataModule()** trong ISLP để task này dễ hơn cho standard usage. Một argument của nó là **num\_workers**, chỉ số processes sẽ dùng để loading data. Với small data như **Hitters**, điều này ít ảnh hưởng, nhưng có advantage cho examples **MNIST** và **CIFAR100** bên dưới. Package **torch** sẽ inspect process đang chạy và determine maximum number of workers.<sup>25</sup> Ta đã include function **rec\_num\_workers()** để compute value này nhằm biết số workers hợp lý (ở đây maximum là 16).

In [23]:

```

max_num_workers = rec_num_workers()

```

General training setup trong **pytorch\_lightning** gồm training, validation và test data. Mỗi loại được represented bởi different data loaders. Trong mỗi epoch, ta run training step để learn model và validation step để track error. Test data thường được dùng ở cuối training để evaluate model.

Trong case này, vì chỉ split thành test và training, ta dùng test data làm validation data với argument **validation=hit\_test**. Argument **validation** có thể là float giữa 0 và 1, integer, hoặc **Dataset**. Nếu là float (tương ứng integer), nó được interpret như percentage (tương ứng

<sup>25</sup>Điều này phụ thuộc vào computing hardware và number of cores khả dụng.

number) của training observations dùng cho validation. Nếu là `Dataset`, nó được pass trực tiếp tới data loader.

In [24]:

```
hit_dm = SimpleDataModule(hit_train,
 hit_test,
 batch_size=32,
 num_workers=min(4, max_num_workers),
 validation=hit_test)
```

Tiếp theo ta phải cung cấp một `pytorch_lightning` module control các steps được thực hiện trong training process. Ta cung cấp methods cho `SimpleModule()` đơn giản record value của loss function và bất kỳ additional metrics nào ở cuối mỗi epoch. Các operations này được controlled bởi methods `SimpleModule.[training/test/validation]_step()`, dù ta không modify chúng trong examples.

In [25]:

```
hit_module = SimpleModule.regression(hit_model,
 metrics={'mae': MeanAbsoluteError()})
```

Bằng cách dùng method `SimpleModule.regression()`, ta indicate sẽ dùng squared-error loss như trong (10.23). Ta cũng yêu cầu track mean absolute error trong metrics được logged.

Ta log results qua `CSVLogger()`, trong case này lưu results vào CSV file trong directory `logs/hitters`. Sau khi fitting hoàn tất, điều này cho phép load results thành `pd.DataFrame()` và visualize ở dưới. Có nhiều cách log results trong `pytorch_lightning`, nhưng ta không cover chi tiết.

In [26]:

```
hit_logger = CSVLogger('logs', name='hitters')
```

Cuối cùng ta sẵn sàng train model và log results. Ta dùng object `Trainer()` từ `pytorch_lightning`. Argument `datamodule=hit_dm` tells trainer cách produce training/validation/test logs, còn first argument `hit_module` specifies network architecture cũng như training/validation/test steps. Argument `callbacks` cho phép nhiều tasks được thực hiện ở various points khi training model. Ở đây callback `ErrorTracker()` cho phép compute validation error trong training và cuối cùng test error. Ta fit model trong 50 epochs.

In [27]:

```
hit_trainer = Trainer(deterministic=True,
 max_epochs=50,
 log_every_n_steps=5,
 logger=hit_logger,
 callbacks=[ErrorTracker()])
hit_trainer.fit(hit_module, datamodule=hit_dm)
```

Ở mỗi step của SGD, algorithm randomly selects 32 training observations để compute gradient. Nhớ từ Section 10.7 rằng một epoch là number of SGD steps cần để process  $n$  observations. Vì training set có  $n = 175$ , và ta specify `batch_size` là 32 khi construct `hit_dm`, một epoch là  $175/32 = 5.5$  SGD steps.

Sau khi fit model, ta evaluate performance trên test data bằng method `test()` của trainer.

In [28]: `hit_trainer.test(hit_module, datamodule=hit_dm)`

Out[28]: `[{'test_loss': 104098.5469, 'test_mae': 229.5012}]`

Results của fit đã được logged vào CSV file. Ta có thể tìm results specific cho run này trong attribute `experiment.metrics_file_path` của logger. Mỗi lần model được fit, logger output results vào một new subdirectory trong directory `logs/hitters`.

Bây giờ ta tạo plot của MAE (mean absolute error) như function của number of epochs. Trước hết retrieve logged summaries.

```
hit_results = pd.read_csv(hit_logger.experiment.metrics_file_path)
```

Vì sẽ produce các plots tương tự ở later examples, ta viết một simple generic function để produce plot này.

```
In [29]: def summary_plot(results,
 ax,
 col='loss',
 valid_legend='Validation',
 training_legend='Training',
 ylabel='Loss',
 fontsize=20):
 for (column,
 color,
 label) in zip([f'train_{col}_epoch',
 f'valid_{col}'],
 ['black',
 'red'],
 [training_legend,
 valid_legend]):
 results.plot(x='epoch',
 y=column,
 label=label,
 marker='o',
 color=color,
 ax=ax)
 ax.set_xlabel('Epoch')
 ax.set_ylabel(ylabel)
 return ax
```

Ta setup axes rồi dùng function vừa viết để produce MAE plot.

```
In [30]: fig, ax = subplots(1, 1, figsize=(6, 6))
ax = summary_plot(hit_results,
 ax,
 col='mae',
 ylabel='MAE',
 valid_legend='Validation (=Test)')
ax.set_ylim([0, 400])
ax.set_xticks(np.linspace(0, 50, 11).astype(int));
```

Ta có thể predict trực tiếp từ final model và evaluate performance trên test data. Trước prediction, ta call method `eval()` của `hit_model`. Điều này tells `torch` effectively consider model đã fitted để có thể predict trên new data. Với model này, thay đổi lớn nhất là dropout layers được turn off, tức không có weights bị randomly dropped khi predicting new data.

```
In [31]: hit_model.eval()
 preds = hit_module(X_test_t)
 torch.abs(Y_test_t - preds).mean()
```

```
Out[31]: tensor(229.5012, grad_fn=<MeanBackward0>)
```

### Cleanup

Khi setup data module, ta đã initiate một số worker processes sẽ tiếp tục running. Ta delete mọi references tới `torch` objects để đảm bảo các processes này bị killed.

```
In [32]: del(Hitters,
 hit_model, hit_dm,
 hit_logger,
 hit_test, hit_train,
 X, Y,
 X_test, X_train,
 Y_test, Y_train,
 X_test_t, Y_test_t,
 hit_trainer, hit_module)
```

### 10.9.2 Multilayer Network on the MNIST Digit Data

Package `torchvision` đi kèm một số example datasets, gồm MNIST digit data. Step đầu là retrieve training và test data sets; function `MNIST()` trong `torchvision.datasets` được cung cấp cho purpose này. Data sẽ được downloaded ở lần đầu function được execute và stored trong directory `data/MNIST`.

`MNIST()`

```
In [33]: (mnist_train,
 mnist_test) = [MNIST(root='data',
 train=train,
 download=True,
 transform=ToTensor())
 for train in [True, False]]
 mnist_train
```

```
Out[33]: Dataset MNIST
 Number of datapoints: 60000
 Root location: data
 Split: Train
 StandardTransform
 Transform: ToTensor()
```

Có 60,000 images trong training data và 10,000 trong test data. Images là  $28 \times 28$  và stored như matrix of pixels. Ta cần transform mỗi image thành vector.

Neural networks somewhat sensitive với scale của inputs, tương tự ridge và lasso regularization bị affected bởi scaling. Ở đây inputs là eight-bit grayscale values từ 0 đến 255, nên ta rescale sang unit interval.<sup>26</sup> Transformation này, cùng một số reordering của axes, được thực hiện bởi transform `ToTensor()` từ package `torchvision.transforms`.

<sup>26</sup>Eight bits nghĩa là  $2^8 = 256$ . Vì convention bắt đầu từ 0, possible values nằm từ 0 đến 255.

Như trong `Hitters` example, ta form data module từ training và test datasets, setting aside 20% training images cho validation.

```
In [34]: mnist_dm = SimpleDataModule(mnist_train,
 mnist_test,
 validation=0.2,
 num_workers=max_num_workers,
 batch_size=256)
```

Ta xem data sẽ được fed vào network. Ta loop qua vài chunks đầu của test dataset, breaking sau 2 batches:

```
In [35]: for idx, (X_, Y_) in enumerate(mnist_dm.train_dataloader()):
 print('X: ', X_.shape)
 print('Y: ', Y_.shape)
 if idx >= 1:
 break
```

```
Out[35]: X: torch.Size([256, 1, 28, 28])
 Y: torch.Size([256])
 X: torch.Size([256, 1, 28, 28])
 Y: torch.Size([256])
```

Ta thấy  $X$  của mỗi batch gồm 256 images size  $1 \times 28 \times 28$ . Số 1 biểu thị single channel (greyscale). Với RGB images như CIFAR100 bên dưới, số 1 trong size sẽ được replace bằng 3 cho three RGB channels.

Bây giờ ta sẵn sàng specify neural network.

```
In [36]: class MNISTModel(nn.Module):
 def __init__(self):
 super(MNISTModel, self).__init__()
 self.layer1 = nn.Sequential(
 nn.Flatten(),
 nn.Linear(28*28, 256),
 nn.ReLU(),
 nn.Dropout(0.4))
 self.layer2 = nn.Sequential(
 nn.Linear(256, 128),
 nn.ReLU(),
 nn.Dropout(0.3))
 self._forward = nn.Sequential(
 self.layer1,
 self.layer2,
 nn.Linear(128, 10))
 def forward(self, x):
 return self._forward(x)
```

Trong first layer, mỗi  $1 \times 28 \times 28$  image được flatten, rồi mapped tới 256 dimensions nơi ta apply ReLU activation với 40% dropout. Second layer map output của first layer xuống 128 dimensions, apply ReLU activation với 30% dropout. Cuối cùng 128 dimensions được mapped xuống 10, number of classes trong MNIST data.

```
In [37]: mnist_model = MNISTModel()
```

Ta check model produce output đúng expected size dựa trên existing batch  $X_$  ở trên.

```
In [38]: mnist_model(X_).size()
```

```
Out[38]: torch.Size([256, 10])
```

Ta xem summary của model. Thay vì `input_size`, ta có thể pass một tensor có correct shape. Ở đây ta pass final batched `X_` ở trên.

```
In [39]: summary(mnist_model,
 input_data=X_,
 col_names=['input_size',
 'output_size',
 'num_params'])
```

```
Out[39]: =====
Layer (type:depth-idx) Input Shape Output Shape Param #
=====
MNISTModel [256, 1, 28, 28] [256, 10] --
 Sequential: 1-1 [256, 1, 28, 28] [256, 10] --
 Sequential: 2-1 [256, 1, 28, 28] [256, 256] --
 Flatten: 3-1 [256, 1, 28, 28] [256, 784] --
 Linear: 3-2 [256, 784] [256, 256] 200,960
 ReLU: 3-3 [256, 256] [256, 256] --
 Dropout: 3-4 [256, 256] [256, 256] --
 Sequential: 2-2 [256, 256] [256, 128] --
 Linear: 3-5 [256, 256] [256, 128] 32,896
 ReLU: 3-6 [256, 128] [256, 128] --
 Dropout: 3-7 [256, 128] [256, 128] --
 Linear: 2-3 [256, 128] [256, 10] 1,290
=====
Total params: 235,146
Trainable params: 235,146
```

Sau khi setup cả model và data module, fitting model gần như identical với `Hitters` example. Khác với regression model, ở đây ta dùng method `SimpleModule.classification()`, dùng cross-entropy loss function thay vì mean squared error.

```
In [40]: mnist_module = SimpleModule.classification(mnist_model)
 mnist_logger = CSVLogger('logs', name='MNIST')
```

Bây giờ ta sẵn sàng. Final step là supply training data và fit model.

```
In [41]: mnist_trainer = Trainer(deterministic=True,
 max_epochs=30,
 logger=mnist_logger,
 callbacks=[ErrorTracker()])
 mnist_trainer.fit(mnist_module,
 datamodule=mnist_dm)
```

Ta đã suppress output ở đây; output là progress report về fitting model, grouped by epoch. Điều này rất useful vì fitting trên large datasets có thể mất thời gian. Fitting model này mất 245 seconds trên MacBook Pro với Apple M1 Pro chip 10 cores và 16 GB RAM. Ta specify validation split 20%, nên training thực tế dùng 80% của 60,000 observations trong training set. Đây là alternative cho việc supply validation data trực tiếp như với `Hitters`. SGD dùng batches 256 observations khi computing gradient; tính ra một epoch tương ứng 188 gradient steps.



`SimpleModule.classification()` include accuracy metric mặc định. Có thể add other classification metrics từ `torchmetrics`. Ta dùng function `summary_plot()` để display accuracy qua epochs.

```
In [42]: mnist_results = pd.read_csv(mnist_logger.experiment.
 metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(mnist_results,
 ax,
 col='accuracy',
 ylabel='Accuracy')
ax.set_ylim([0.5, 1])
ax.set_ylabel('Accuracy')
ax.set_xticks(np.linspace(0, 30, 7).astype(int));
```

Một lần nữa ta evaluate accuracy bằng method `test()` của trainer. Model này đạt 97% accuracy trên test data.

```
In [43]: mnist_trainer.test(mnist_module,
 datamodule=mnist_dm)
```

```
Out[43]: [{'test_loss': 0.1471, 'test_accuracy': 0.9681}]
```

Table 10.1 cũng report error rates từ LDA (Chapter 4) và multiclass logistic regression. Với LDA, reader được refer tới Section 4.7.3. Dù có thể dùng function `LogisticRegression()` của `sklearn` để fit multiclass logistic regression, ở đây ta đã setup để fit model này bằng `torch`. Ta chỉ có input layer và output layer, bỏ hidden layers.

```
In [44]: class MNIST_MLR(nn.Module):
 def __init__(self):
 super(MNIST_MLR, self).__init__()
 self.linear = nn.Sequential(nn.Flatten(),
 nn.Linear(784, 10))

 def forward(self, x):
 return self.linear(x)

mlr_model = MNIST_MLR()
mlr_module = SimpleModule.classification(mlr_model)
mlr_logger = CSVLogger('logs', name='MNIST_MLR')
```

```
In [45]: mlr_trainer = Trainer(deterministic=True,
 max_epochs=30,
 callbacks=[ErrorTracker()])
mlr_trainer.fit(mlr_module, datamodule=mnist_dm)
```

Ta fit model như trước rồi compute test results.

```
In [46]: mlr_trainer.test(mlr_module,
 datamodule=mnist_dm)
```

```
Out[46]: [{'test_loss': 0.3187, 'test_accuracy': 0.9241}]
```

Accuracy vượt 90% ngay cả với model khá simple này. Như trong `Hitters` example, ta delete một số objects đã create ở trên.

In [47]:

```
del(mnist_test,
 mnist_train,
 mnist_model,
 mnist_dm,
 mnist_trainer,
 mnist_module,
 mnist_results,
 mlr_model,
 mlr_module,
 mlr_trainer)
```

### 10.9.3 Convolutional Neural Networks

Trong section này ta fit một CNN cho CIFAR100 data, available trong package `torchvision`. Data được arranged tương tự MNIST data.

In [48]:

```
(cifar_train,
cifar_test) = [CIFAR100(root="data",
 train=train,
 download=True)
 for train in [True, False]]
```

In [49]:

```
transform = ToTensor()
cifar_train_X = torch.stack([transform(x) for x in
 cifar_train.data])
cifar_test_X = torch.stack([transform(x) for x in
 cifar_test.data])
cifar_train = TensorDataset(cifar_train_X,
 torch.tensor(cifar_train.targets))
cifar_test = TensorDataset(cifar_test_X,
 torch.tensor(cifar_test.targets))
```

CIFAR100 dataset gồm 50,000 training images, mỗi image represented bởi three-dimensional tensor: mỗi three-color image được represented như set of three channels, mỗi channel gồm  $32 \times 32$  eight-bit pixels. Ta standardize như đã làm với digits nhưng giữ array structure. Việc này được accomplished bằng transform `ToTensor()`.

Creating data module tương tự MNIST example.

In [50]:

```
cifar_dm = SimpleDataModule(cifar_train,
 cifar_test,
 validation=0.2,
 num_workers=max_num_workers,
 batch_size=128)
```

Ta lại xem shape của typical batches trong data loaders.

In [51]:

```
for idx, (X_, Y_) in enumerate(cifar_dm.train_dataloader()):
 print('X: ', X_.shape)
 print('Y: ', Y_.shape)
 if idx >= 1:
 break
```

Out[51]:

```
X: torch.Size([128, 3, 32, 32])
Y: torch.Size([128])
X: torch.Size([128, 3, 32, 32])
Y: torch.Size([128])
```

Trước khi bắt đầu, ta xem một số training images; similar code đã produce Figure 10.5 trên page 406. Example dưới cũng illustrate rằng `TensorDataset` objects có thể indexed bằng integers—ta chọn random images từ training data bằng cách indexing `cifar_train`. Để display đúng, ta phải reorder dimensions bằng call tới `np.transpose()`.

In [52]:

```
fig, axes = subplots(5, 5, figsize=(10,10))
rng = np.random.default_rng(4)
indices = rng.choice(np.arange(len(cifar_train)), 25,
 replace=False).reshape((5,5))

for i in range(5):
 for j in range(5):
 idx = indices[i,j]
 axes[i,j].imshow(np.transpose(cifar_train[idx][0],
 [1,2,0]),
 interpolation=None)
 axes[i,j].set_xticks([])
 axes[i,j].set_yticks([])
```

Ở đây method `imshow()` recognizes từ shape của argument rằng đó là 3-dimensional array, với last dimension indexing three RGB color channels.

`.imshow()`

Ta specify một moderately-sized CNN cho demonstration, similar về structure với Figure 10.8. Ta dùng several layers, mỗi layer gồm convolution, ReLU và max-pooling steps. Trước hết define một module cho một layer như vậy. Như trong previous examples, ta overwrite methods `__init__()` và `forward()` của `nn.Module`. User-defined module này giờ có thể được dùng giống `nn.Linear()` hoặc `nn.Dropout()`.

In [53]:

```
class BuildingBlock(nn.Module):

 def __init__(self,
 in_channels,
 out_channels):

 super(BuildingBlock, self).__init__()
 self.conv = nn.Conv2d(in_channels=in_channels,
 out_channels=out_channels,
 kernel_size=(3,3),
 padding='same')
 self.activation = nn.ReLU()
 self.pool = nn.MaxPool2d(kernel_size=(2,2))

 def forward(self, x):
 return self.pool(self.activation(self.conv(x)))
```

Lưu ý argument `padding="same"` cho `nn.Conv2d()` đảm bảo output channels có same dimension như input channels. Có 32 channels trong first hidden layer, so với three channels trong input layer. Ta dùng  $3 \times 3$  convolution filter cho mỗi channel ở tất cả layers. Mỗi convolution được followed by max-pooling layer trên  $2 \times 2$  blocks.

Khi forming deep learning model cho CIFAR100 data, ta dùng several `BuildingBlock()` modules sequentially. Simple example này illustrate power của `torch`: users có thể define modules riêng rồi combine chúng trong other modules. Cuối cùng mọi thứ được fit bởi generic trainer.

In [54]:

```
class CIFARModel(nn.Module):

 def __init__(self):
 super(CIFARModel, self).__init__()
 sizes = [(3,32),
 (32,64),
 (64,128),
 (128,256)]
 self.conv = nn.Sequential(*[BuildingBlock(in_, out_)
 for in_, out_ in sizes])

 self.output = nn.Sequential(nn.Dropout(0.5),
 nn.Linear(2*2*256, 512),
 nn.ReLU(),
 nn.Linear(512, 100))

 def forward(self, x):
 val = self.conv(x)
 val = torch.flatten(val, start_dim=1)
 return self.output(val)
```

Ta build model và xem summary. (Ta đã create examples của  $X_1$  trước đó.)

In [55]:

```
cifar_model = CIFARModel()
summary(cifar_model,
 input_data=X_,
 col_names=['input_size',
 'output_size',
 'num_params'])
```

Out[55]:

```
=====
Layer (type:depth-idx) Input Shape Output Shape Param #
=====
CIFARModel [128, 3, 32, 32] [128, 100] --
 Sequential: 1-1 [128, 3, 32, 32] [128, 256, 2, 2] --
 BuildingBlock: 2-1 [128, 3, 32, 32] [128, 32, 16, 16] --
 Conv2d: 3-1 [128, 3, 32, 32] [128, 32, 32, 32] 896
 ReLU: 3-2 [128, 32, 32, 32] [128, 32, 32, 32] --
 MaxPool2d: 3-3 [128, 32, 32, 32] [128, 32, 16, 16] --
 BuildingBlock: 2-2 [128, 32, 16, 16] [128, 64, 8, 8] --
 Conv2d: 3-4 [128, 32, 16, 16] [128, 64, 16, 16] 18,496
 ReLU: 3-5 [128, 64, 16, 16] [128, 64, 16, 16] --
 MaxPool2d: 3-6 [128, 64, 16, 16] [128, 64, 8, 8] --
 BuildingBlock: 2-3 [128, 64, 8, 8] [128, 128, 4, 4] --
 Conv2d: 3-7 [128, 64, 8, 8] [128, 128, 8, 8] 73,856
 ReLU: 3-8 [128, 128, 8, 8] [128, 128, 8, 8] --
 MaxPool2d: 3-9 [128, 128, 8, 8] [128, 128, 4, 4] --
 BuildingBlock: 2-4 [128, 128, 4, 4] [128, 256, 2, 2] --
 Conv2d: 3-10 [128, 128, 4, 4] [128, 256, 4, 4] 295,168
 ReLU: 3-11 [128, 256, 4, 4] [128, 256, 4, 4] --
 MaxPool2d: 3-12 [128, 256, 4, 4] [128, 256, 2, 2] --
 Sequential: 1-2 [128, 1024] [128, 100] --
 Dropout: 2-5 [128, 1024] [128, 1024] --
 Linear: 2-6 [128, 1024] [128, 512] 524,800
=====
ReLU: 2-7 [128, 512] [128, 512] --
Linear: 2-8 [128, 512] [128, 100] 51,300
=====
Total params: 964,516
Trainable params: 964,516
```

Total number of trainable parameters là 964,516. Nhìn parameter sizes, ta thấy channels halve ở cả two dimensions sau mỗi max-pooling opera-

tion. Sau last one, ta có layer với 256 channels dimension  $2 \times 2$ . Chúng được flatten thành dense layer size 1,024; tức mỗi matrix  $2 \times 2$  được turn thành 4-vector rồi placed side-by-side trong một layer. Sau đó là dropout regularization layer, một dense layer khác size 512, và cuối cùng output layer.

Tới đây ta dùng default optimizer trong `SimpleModule()`. Với data này, experiments cho thấy smaller learning rate perform tốt hơn default 0.01. Ta dùng custom optimizer với learning rate 0.001. Ngoài ra logging và training follow pattern tương tự previous examples. Optimizer nhận argument `params` cho biết parameters nào involved trong SGD.

Ta đã thấy entries của module's parameters là tensors. Khi passing parameters tới optimizer, ta làm nhiều hơn việc chỉ pass arrays; một phần structure của graph được encoded trong tensors themselves.

```
In [56]: cifar_optimizer = RMSprop(cifar_model.parameters(), lr=0.001)
cifar_module = SimpleModule.classification(cifar_model,
 optimizer=cifar_optimizer)
cifar_logger = CSVLogger('logs', name='CIFAR100')
```

```
In [57]: cifar_trainer = Trainer(deterministic=True,
 max_epochs=30,
 logger=cifar_logger,
 callbacks=[ErrorTracker()])
cifar_trainer.fit(cifar_module,
 datamodule=cifar_dm)
```

Model này mất 10 minutes hoặc hơn để run và đạt khoảng 42% accuracy trên test data. Dù không quá tệ với 100-class data (random classifier chỉ đạt 1%), searching the web cho thấy results khoảng 75%. Thông thường cần nhiều architecture carpentry, fiddling với regularization và time để đạt results như vậy.

Ta xem validation và training accuracy qua epochs.

```
In [58]: log_path = cifar_logger.experiment.metrics_file_path
cifar_results = pd.read_csv(log_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(cifar_results,
 ax,
 col='accuracy',
 ylabel='Accuracy')
ax.set_xticks(np.linspace(0, 10, 6).astype(int))
ax.set_ylabel('Accuracy')
ax.set_ylim([0, 1]);
```

Cuối cùng evaluate model trên test data.

```
In [59]: cifar_trainer.test(cifar_module,
 datamodule=cifar_dm)
```

```
Out[59]: [{'test_loss': 2.4238 'test_accuracy': 0.4206}]
```

### Hardware Acceleration

Khi deep learning trở nên ubiquitous trong machine learning, hardware manufacturers đã produce special libraries có thể speed up gradient-descent

steps.

Chẳng hạn Mac OS devices với M1 chip có thể enable Metal programming framework, giúp speed up torch computations. Ta trình bày example cách dùng acceleration này.

Main changes nằm ở call `Trainer()` cũng như metrics sẽ được evaluated trên data. Các metrics phải được told nơi data sẽ located ở evaluation time. Điều này accomplished bằng call method `to()` của metrics.

In [60]:

```
try:
 for name, metric in cifar_module.metrics.items():
 cifar_module.metrics[name] = metric.to('mps')
 cifar_trainer_mps = Trainer(accelerator='mps',
 deterministic=True,
 max_epochs=30)
 cifar_trainer_mps.fit(cifar_module,
 datamodule=cifar_dm)
 cifar_trainer_mps.test(cifar_module,
 datamodule=cifar_dm)
except:
 pass
```

Cách này yield approximately two- or three-fold acceleration cho mỗi epoch. Code block được protect bằng clauses `try:` và `except:`; nếu works thì ta có speedup, nếu fails thì không có gì xảy ra.

### 10.9.4 Using Pretrained CNN Models

Bây giờ ta cho thấy cách dùng CNN pretrained trên imagenet database để classify natural images, đồng thời demonstrate cách produce Figure 10.10. Ta copy six JPEG images từ digital photo album vào directory `book_images`. Các images available từ data section của [www.statlearning.com](http://www.statlearning.com), ISLP book website. Download `book_images.zip`; khi click, nó create directory `book_images`.

Pretrained network ta dùng gọi là `resnet50`; specification details có thể found on the web. Ta read images và convert sang array format expected bởi torch software để match specifications trong `resnet50`. Conversion gồm resize, crop, rồi predefined standardization cho mỗi of three channels. Ta read và preprocess images.

In [61]:

```
resize = Resize((232,232))
crop = CenterCrop(224)
normalize = Normalize([0.485,0.456,0.406],
 [0.229,0.224,0.225])
imgfiles = sorted([f for f in glob('book_images/*')])
imgs = torch.stack([torch.div(crop(resize(read_image(f))), 255)
 for f in imgfiles])
imgs = normalize(imgs)
imgs.size()
```

Out[61]:

```
torch.Size([6, 3, 224, 224])
```

Ta setup trained network với weights đã read trong code block 6. Model có 50 layers, với khá nhiều complexity.

In [62]:

```
resnet_model = resnet50(weights=ResNet50_Weights.DEFAULT)
summary(resnet_model,
```

```
input_data=imgs,
col_names=['input_size',
 'output_size',
 'num_params'])
```

Ta set mode thành `eval()` để đảm bảo model ready để predict trên new data.

```
In [63]: resnet_model.eval()
```

Inspecting output ở trên, ta thấy khi setup `resnet_model`, authors define một Bottleneck, tương tự module `BuildingBlock` của ta. Bây giờ feed six images qua fitted network.

```
In [64]: img_preds = resnet_model(imgs)
```

Ta xem predicted probabilities cho top 3 choices của mỗi image. Trước hết compute probabilities bằng cách apply softmax lên logits trong `img_preds`. Lưu ý phải call method `detach()` trên tensor `img_preds` để convert nó thành familiar `ndarray`.

```
In [65]: img_probs = np.exp(np.asarray(img_preds.detach()))
img_probs /= img_probs.sum(1)[:,None]
```

Để thấy class labels, ta phải download index file associated với imagenet.<sup>27</sup>

```
In [66]: labs = json.load(open('imagenet_class_index.json'))
class_labels = pd.DataFrame([(int(k), v[1]) for k, v in
 labs.items()],
 columns=['idx', 'label'])
class_labels = class_labels.set_index('idx')
class_labels = class_labels.sort_index()
```

Bây giờ construct data frame cho mỗi image file với labels có three highest probabilities estimated bởi model.

```
In [67]: for i, imgfile in enumerate(imgfiles):
img_df = class_labels.copy()
img_df['prob'] = img_probs[i]
img_df = img_df.sort_values(by='prob', ascending=False)[:3]
print(f'Image: {imgfile}')
print(img_df.reset_index().drop(columns=['idx']))
```

```
Out[67]: Image: book_images/Cape_Weaver.jpg
 label prob
0 jacamar 0.287283
1 bee_eater 0.046768
2 bulbul 0.037507
Image: book_images/Flamingo.jpg
 label prob
0 flamingo 0.591761
1 spoonbill 0.012386
2 American_egret 0.002105
Image: book_images/Hawk_Fountain.jpg
 label prob
```

<sup>27</sup>File này available từ book website và [https://s3.amazonaws.com/deep-learning-models/image-models/imagenet\\_class\\_index.json](https://s3.amazonaws.com/deep-learning-models/image-models/imagenet_class_index.json).

```

0 great_grey_owl 0.287959
1 kite 0.039478
2 fountain 0.029384
Image: book_images/Hawk_cropped.jpg
 label prob
0 kite 0.301830
1 jay 0.121674
2 magpie 0.015513
Image: book_images/Lhasa_Apso.jpg
 label prob
0 Lhasa 0.151143
1 Shih-Tzu 0.129850
2 Tibetan_terrier 0.102358
Image: book_images/Sleeping_Cat.jpg
 label prob
0 tabby 0.173627
1 tiger_cat 0.110414
2 doormat 0.093447

```

Ta thấy model khá confident với `Flamingo.jpg`, nhưng ít hơn với các images còn lại. Ta kết thúc section bằng usual cleanup.

In [68]:

```

del(cifar_test,
 cifar_train,
 cifar_dm,
 cifar_module,
 cifar_logger,
 cifar_optimizer,
 cifar_trainer)

```

### 10.9.5 IMDB Document Classification

Bây giờ ta implement models cho sentiment classification (Section 10.4) trên IMDB dataset. Như đã mentioned trước code block 8, ta dùng pre-processed version của IMDB dataset từ package `keras`. Vì `keras` dùng `tensorflow`, một tensor và deep learning library khác, ta đã convert data để suitable cho `torch`. Code dùng để convert từ `keras` available trong module `ISLP.torch._make_imdb`. Nó requires một số `keras` packages để run. Data này dùng dictionary size 10,000.

Ta stored three different representations của review data cho lab này:

- `load_tensor()`, sparse tensor version usable bởi `torch`;
- `load_sparse()`, sparse matrix version usable bởi `sklearn`, vì ta sẽ compare với lasso fit;
- `load_sequential()`, padded version của original sequence representation, limited tới last 500 words của mỗi review.

In [69]:

```

(imdb_seq_train,
 imdb_seq_test) = load_sequential(root='data/IMDB')
padded_sample = np.asarray(imdb_seq_train.tensors[0][0])
sample_review = padded_sample[padded_sample > 0][:12]
sample_review[:12]

```

Out[69]:

```

array([1, 14, 22, 16, 43, 530, 973, 1622, 1385,
 65, 458, 4468], dtype=int32)

```

Datasets `imdb_seq_train` và `imdb_seq_test` đều là instances của class



**TensorDataset.** Tensors dùng để construct chúng nằm trong attribute **tensors**, với first tensor là features  $X$  và second là outcome  $Y$ . Ta lấy first row của features và store thành **padded\_sample**. Trong preprocessing dùng để form data, sequences được padded bằng 0s ở beginning nếu không đủ dài; vì vậy remove padding bằng cách restrict tới entries **padded\_sample** > 0. Sau đó provide first 12 words của sample review.

Ta tìm các words này trong lookup dictionary từ module **ISLP.torch.imdb**.

```
In [70]: lookup = load_lookup(root='data/IMDB')
 ' '.join(lookup[i] for i in sample_review)
```

```
Out[70]: "<START> this film was just brilliant casting location scenery
 story direction everyone's"
```

Với first model, ta create binary feature cho mỗi of 10,000 possible words trong dataset, với entry 1 ở vị trí  $i, j$  nếu word  $j$  appears trong review  $i$ . Vì most reviews khá short, feature matrix như vậy có hơn 98% zeros. Data được access bằng **load\_tensor()** từ ISLP library.

```
In [71]: max_num_workers=10
 (imdb_train,
 imdb_test) = load_tensor(root='data/IMDB')
 imdb_dm = SimpleDataModule(imdb_train,
 imdb_test,
 validation=2000,
 num_workers=min(6, max_num_workers),
 batch_size=512)
```

Ta dùng two-layer model cho first model.

```
In [72]: class IMDBModel(nn.Module):

 def __init__(self, input_size):
 super(IMDBModel, self).__init__()
 self.dense1 = nn.Linear(input_size, 16)
 self.activation = nn.ReLU()
 self.dense2 = nn.Linear(16, 16)
 self.output = nn.Linear(16, 1)

 def forward(self, x):
 val = x
 for _map in [self.dense1,
 self.activation,
 self.dense2,
 self.activation,
 self.output]:
 val = _map(val)
 return torch.flatten(val)
```

Ta instantiate model và xem summary (không shown).

```
In [73]: imdb_model = IMDBModel(imdb_test.tensors[0].size()[1])
 summary(imdb_model,
 input_size=imdb_test.tensors[0].size(),
 col_names=['input_size',
 'output_size',
 'num_params'])
```

Ta lại dùng smaller learning rate cho data này, nên pass optimizer tới

`SimpleModule`. Vì reviews được classified thành positive hoặc negative sentiment, ta dùng `SimpleModule.binary_classification()`.<sup>28</sup>

```
In [74]: imdb_optimizer = RMSprop(imdb_model.parameters(), lr=0.001)
imdb_module = SimpleModule.binary_classification(
 imdb_model,
 optimizer=imdb_optimizer)
```

Sau khi load datasets vào data module và create `SimpleModule`, remaining steps đã familiar.

```
In [75]: imdb_logger = CSVLogger('logs', name='IMDB')
imdb_trainer = Trainer(deterministic=True,
 max_epochs=30,
 logger=imdb_logger,
 callbacks=[ErrorTracker()])
imdb_trainer.fit(imdb_module,
 datamodule=imdb_dm)
```

Evaluating test error yield roughly 86% accuracy.

```
In [76]: test_results = imdb_trainer.test(imdb_module, datamodule=imdb_dm)
test_results
```

```
Out[76]: [{'test_loss': 1.0863, 'test_accuracy': 0.8550}]
```

### Comparison to Lasso

Ta fit lasso logistic regression model bằng `LogisticRegression()` từ `sklearn`. Vì `sklearn` không recognize sparse tensors của `torch`, ta dùng sparse matrix mà `sklearn` recognize.

```
In [77]: ((X_train, Y_train),
 (X_valid, Y_valid),
 (X_test, Y_test)) = load_sparse(validation=2000,
 random_state=0,
 root='data/IMDB')
```

Tương tự Section 10.9.1, ta construct series 50 values cho lasso regularization parameter  $\lambda$ .

```
In [78]: lam_max = np.abs(X_train.T * (Y_train - Y_train.mean())).max()
lam_val = lam_max * np.exp(np.linspace(np.log(1),
 np.log(1e-4), 50))
```

Với `LogisticRegression()`, regularization parameter  $C$  được specified như inverse của  $\lambda$ . Có several solvers cho logistic regression; ở đây dùng `liblinear`, hoạt động tốt với sparse input format.

```
In [79]: logit = LogisticRegression(penalty='l1',
 C=1/lam_max,
 solver='liblinear',
 warm_start=True,
 fit_intercept=True)
```

<sup>28</sup>Việc dùng `binary_classification()` thay vì `classification()` xuất phát từ một subtlety trong cách `torchmetrics.Accuracy()` hoạt động, cũng như data type của targets.

Path gồm 50 values mất khoảng 40 seconds để run.

In [80]:

```
coefs = []
intercepts = []

for l in lam_val:
 logit.C = 1/l
 logit.fit(X_train, Y_train)
 coefs.append(logit.coef_.copy())
 intercepts.append(logit.intercept_)
```

Coefficients và intercepts có một extraneous dimension có thể removed bằng function `np.squeeze()`.

In [81]:

```
coefs = np.squeeze(coefs)
intercepts = np.squeeze(intercepts)
```

Ta tạo plot để compare neural network results với lasso.

In [82]:

```
%%capture
fig, axes = subplots(1, 2, figsize=(16, 8), sharey=True)
for ((X_, Y_),
 data_,
 color) in zip([(X_train, Y_train),
 (X_valid, Y_valid),
 (X_test, Y_test)],
 ['Training', 'Validation', 'Test'],
 ['black', 'red', 'blue']):
 linpred_ = X_ * coefs.T + intercepts[None,:]
 label_ = np.array(linpred_ > 0)
 accuracy_ = np.array([np.mean(Y_ == 1) for l in label_.T])
 axes[0].plot(-np.log(lam_val / X_train.shape[0]),
 accuracy_,
 '--',
 color=color,
 markersize=13,
 linewidth=2,
 label=data_)

axes[0].legend()
axes[0].set_xlabel(r'$-\log(\lambda)$', fontsize=20)
axes[0].set_ylabel('Accuracy', fontsize=20)
```

Lưu ý use của `%%capture`, suppress display của partially completed figure. Điều này useful khi making complex figure vì steps có thể spread qua two hoặc more cells. Ta add plot của lasso accuracy và display composed figure bằng cách simply entering name ở end of cell.

*%%capture*

In [83]:

```
imdb_results = pd.read_csv(imdb_logger.experiment.metrics_file_path)
summary_plot(imdb_results,
 axes[1],
 col='accuracy',
 ylabel='Accuracy')
axes[1].set_xticks(np.linspace(0, 30, 7).astype(int))
axes[1].set_ylabel('Accuracy', fontsize=20)
axes[1].set_xlabel('Epoch', fontsize=20)
axes[1].set_ylim([0.5, 1]);
axes[1].axhline(test_results[0]['test_accuracy'],
 color='blue',
 linestyle='--',
 linewidth=3)

fig
```

Từ graphs ta thấy accuracy của lasso logistic regression peaks ở khoảng 0.88, tương tự neural network. Một lần nữa, kết thúc bằng cleanup.

In [84]:

```
del(imdb_model,
 imdb_trainer,
 imdb_logger,
 imdb_dm,
 imdb_train,
 imdb_test)
```

### 10.9.6 Recurrent Neural Networks

Trong lab này ta fit các models illustrated trong Section 10.5.

#### Sequential Models for Document Classification

Ở đây ta fit một simple LSTM RNN cho sentiment prediction trên IMDB movie-review data như discussed trong Section 10.5.1. Với RNN, ta dùng sequence of words trong document, taking order into account. Ta đã load preprocessed data ở beginning của Section 10.9.5. Một script detailing preprocessing có trong ISLP library. Đáng chú ý, vì hơn 90% documents có fewer than 500 words, ta set document length bằng 500. Với longer documents, dùng last 500 words; với shorter documents, pad front bằng blanks.

In [85]:

```
imdb_seq_dm = SimpleDataModule(imdb_seq_train,
 imdb_seq_test,
 validation=2000,
 batch_size=300,
 num_workers=min(6, max_num_workers)
)
```

First layer của RNN là embedding layer size 32, sẽ được learned trong training. Layer này one-hot encode mỗi document thành matrix dimension  $500 \times 10,003$ , rồi map 10,003 dimensions này xuống 32.<sup>29</sup> Vì mỗi word represented bởi integer, điều này effectively achieved bằng creation of embedding matrix size  $10,003 \times 32$ ; mỗi of 500 integers trong document sau đó mapped tới appropriate 32 real numbers bằng indexing appropriate rows của matrix.

Second layer là LSTM với 32 units, còn output layer là single logit cho binary classification task. Trong last line của method `forward()` dưới đây, ta lấy last 32-dimensional output của LSTM rồi map nó tới response.

In [86]:

```
class LSTMModel(nn.Module):
 def __init__(self, input_size):
 super(LSTMModel, self).__init__()
 self.embedding = nn.Embedding(input_size, 32)
 self.lstm = nn.LSTM(input_size=32,
 hidden_size=32,
 batch_first=True)
 self.dense = nn.Linear(32, 1)
 def forward(self, x):
 val, (h_n, c_n) = self.lstm(self.embedding(x))
 return torch.flatten(self.dense(val[:,-1]))
```

Ta instantiate và xem summary của model, dùng first 10 documents trong

<sup>29</sup>Extra 3 dimensions correspond tới commonly occurring non-word entries trong reviews.

corpus.

```
In [87]: lstm_model = LSTMModel(X_test.shape[-1])
summary(lstm_model,
 input_data=imdb_seq_train.tensors[0][:10],
 col_names=['input_size',
 'output_size',
 'num_params'])
```

```
Out[87]: =====
Layer (type:depth-idx) Input Shape Output Shape Param #
=====
LSTMModel [10, 500] [10] --

Out[87]: Embedding: 1-1 [10, 500] [10, 500, 32] 320,096
 LSTM: 1-2 [10, 500, 32] [10, 500, 32] 8,448
 Linear: 1-3 [10, 32] [10, 1] 33
 =====
Total params: 328,577
Trainable params: 328,577
```

Value 10,003 bị suppressed trong summary, nhưng ta thấy nó qua parameter count vì  $10,003 \times 32 = 320,096$ .

```
In [88]: lstm_module = SimpleModule.binary_classification(lstm_model)
lstm_logger = CSVLogger('logs', name='IMDB_LSTM')
```

```
In [89]: lstm_trainer = Trainer(deterministic=True,
 max_epochs=20,
 logger=lstm_logger,
 callbacks=[ErrorTracker()])
lstm_trainer.fit(lstm_module,
 datamodule=imdb_seq_dm)
```

Phần còn lại tương tự other networks đã fit. Ta track test performance trong quá trình fitting và thấy model attain 85% accuracy.

```
In [90]: lstm_trainer.test(lstm_module, datamodule=imdb_seq_dm)
```

```
Out[90]: [{'test_loss': 0.8178, 'test_accuracy': 0.8476}]
```

Ta một lần nữa show learning progress, followed by cleanup.

```
In [91]: lstm_results = pd.read_csv(lstm_logger.experiment.metrics_file_path)
fig, ax = subplots(1, 1, figsize=(6, 6))
summary_plot(lstm_results,
 ax,
 col='accuracy',
 ylabel='Accuracy')
ax.set_xticks(np.linspace(0, 20, 5).astype(int))
ax.set_ylabel('Accuracy')
ax.set_ylim([0.5, 1])
```

```
In [92]: del(lstm_model,
 lstm_trainer,
 lstm_logger,
 imdb_seq_dm,
```

```
imdb_seq_train,
imdb_seq_test)
```

### Time Series Prediction

Ta show cách fit models trong Section 10.5.2 cho time series prediction. Trước hết load và standardize data.

```
In [93]: NYSE = load_data('NYSE')
cols = ['DJ_return', 'log_volume', 'log_volatility']
X = pd.DataFrame(StandardScaler(
 with_mean=True,
 with_std=True).fit_transform(NYSE[cols]),
 columns=NYSE[cols].columns,
 index=NYSE.index)
```

Tiếp theo setup lagged versions của data, dropping rows có missing values bằng method `dropna()`.

```
In [94]: for lag in range(1, 6):
 for col in cols:
 newcol = np.zeros(X.shape[0]) * np.nan
 newcol[lag:] = X[col].values[:-lag]
 X.insert(len(X.columns), "{0}_{1}".format(col, lag), newcol)
X.insert(len(X.columns), 'train', NYSE['train'])
X = X.dropna()
```

Cuối cùng extract response, training indicator, và drop current day's DJ\_return và log\_volatility để predict chỉ từ previous day's data.

```
In [95]: Y, train = X['log_volume'], X['train']
X = X.drop(columns=['train'] + cols)
X.columns
```

```
Out[95]: Index(['DJ_return_1', 'log_volume_1', 'log_volatility_1',
 'DJ_return_2', 'log_volume_2', 'log_volatility_2',
 'DJ_return_3', 'log_volume_3', 'log_volatility_3',
 'DJ_return_4', 'log_volume_4', 'log_volatility_4',
 'DJ_return_5', 'log_volume_5', 'log_volatility_5'],
 dtype='object')
```

Trước hết fit simple linear model và compute  $R^2$  trên test data bằng method `score()`.

```
In [96]: M = LinearRegression()
M.fit(X[train], Y[train])
M.score(X[-train], Y[-train])
```

```
Out[96]: 0.4129
```

Ta refit model này, including factor variable `day_of_week`. Với categorical series trong pandas, ta có thể form indicators bằng method `get_dummies()`.

```
In [97]: X_day = pd.merge(X,
 pd.get_dummies(NYSE['day_of_week']),
 on='date')
```

Lưu ý không cần reinstantiate linear regression model vì method `fit()` accept design matrix và response trực tiếp.

```
In [98]: M.fit(X_day[train], Y[train])
 M.score(X_day[-train], Y[-train])
```

```
Out[98]: 0.4595
```

Model này đạt  $R^2$  khoảng 46%.

Để fit RNN, ta phải reshape data vì nó expect 5 lagged versions của mỗi feature như indicated bởi `input_shape` argument tới layer `nn.RNN()` bên dưới. Trước hết ensure columns của data frame sao cho reshaped matrix có variables lagged đúng. Ta dùng method `reindex()`.

Với input shape (5,3), mỗi row represent một lagged version của three variables. Layer `nn.RNN()` cũng expect first row của mỗi observation là earliest in time, nên phải reverse current order. Vì vậy ta loop over `range(5,0,-1)`, một example dùng `slice()` để index iterable objects. General notation là `start:end:step`.

```
In [99]: ordered_cols = []
 for lag in range(5,0,-1):
 for col in cols:
 ordered_cols.append('{0}_{1}'.format(col, lag))
 X = X.reindex(columns=ordered_cols)
 X.columns
```

```
Out[99]: Index(['DJ_return_5', 'log_volume_5', 'log_volatility_5',
 'DJ_return_4', 'log_volume_4', 'log_volatility_4',
 'DJ_return_3', 'log_volume_3', 'log_volatility_3',
 'DJ_return_2', 'log_volume_2', 'log_volatility_2',
 'DJ_return_1', 'log_volume_1', 'log_volatility_1'],
 dtype='object')
```

Bây giờ reshape data.

```
In [100]: X_rnn = X.to_numpy().reshape((-1,5,3))
 X_rnn.shape
```

```
Out[100]: (6046, 5, 3)
```

Khi specify first size bằng -1, `numpy.reshape()` deduces size dựa trên remaining arguments.

Bây giờ proceed với RNN dùng 12 hidden units và 10% dropout. Sau khi pass qua RNN, ta extract final time point như `val[:,-1]` trong `forward()` bên dưới. Nó được pass qua 10% dropout rồi flattened qua linear layer.

```
In [101]: class NYSEModel(nn.Module):
 def __init__(self):
 super(NYSEModel, self).__init__()
 self.rnn = nn.RNN(3,
 12,
 batch_first=True)
 self.dense = nn.Linear(12, 1)
 self.dropout = nn.Dropout(0.1)
 def forward(self, x):
 val, h_n = self.rnn(x)
```

```

 val = self.dense(self.dropout(val[:,-1]))
 return torch.flatten(val)
nyse_model = NYSEModel()

```

Ta fit model tương tự previous networks. Ta supply fit function với test data như validation data, để khi monitor progress và plot history function có thể thấy progress trên test data. Dĩ nhiên không nên dùng điều này làm basis cho early stopping vì khi đó test performance sẽ biased. Ta form training dataset tương tự Hitters example.

```

In [102]: datasets = []
for mask in [train, ~train]:
 X_rnn_t = torch.tensor(X_rnn[mask].astype(np.float32))
 Y_t = torch.tensor(Y[mask].astype(np.float32))
 datasets.append(TensorDataset(X_rnn_t, Y_t))
nyse_train, nyse_test = datasets

```

Theo usual pattern, ta inspect summary.

```

In [103]: summary(nyse_model,
 input_data=X_rnn_t,
 col_names=['input_size',
 'output_size',
 'num_params'])

```

```

Out[103]: =====
Layer (type:depth-idx) Input Shape Output Shape Param #
=====
NYSEModel [1770, 5, 3] [1770] --
 RNN: 1-1 [1770, 5, 3] [1770, 5, 12] 204
 Dropout: 1-2 [1770, 12] [1770, 12] --
 Linear: 1-3 [1770, 12] [1770, 1] 13
=====
Total params: 217
Trainable params: 217

```

Ta lại put two datasets vào data module, với batch size 64.

```

In [104]: nyse_dm = SimpleDataModule(nyse_train,
 nyse_test,
 num_workers=min(4, max_num_workers),
 validation=nyse_test,
 batch_size=64)

```

Ta run some data qua model để chắc chắn sizes match đúng.

```

In [105]: for idx, (x, y) in enumerate(nyse_dm.train_dataloader()):
 out = nyse_model(x)
 print(y.size(), out.size())
 if idx >= 2:
 break

```

```

Out[105]: torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])
torch.Size([64]) torch.Size([64])

```

Ta follow previous example để setup trainer cho regression problem, requesting  $R^2$  metric được compute ở mỗi epoch.



```
In [106]: nyse_optimizer = RMSprop(nyse_model.parameters(),
 lr=0.001)
nyse_module = SimpleModule.regression(nyse_model,
 optimizer=nyse_optimizer,
 metrics={'r2': R2Score()})
```

Fitting model giờ đã familiar. Results trên test data rất similar với linear AR model.

```
In [107]: nyse_trainer = Trainer(deterministic=True,
 max_epochs=200,
 callbacks=[ErrorTracker()])
nyse_trainer.fit(nyse_module,
 datamodule=nyse_dm)
nyse_trainer.test(nyse_module,
 datamodule=nyse_dm)
```

```
Out[107]: [{'test_loss': 0.6141, 'test_r2': 0.4172}]
```

Ta cũng có thể fit model without layer `nn.RNN()` bằng cách chỉ dùng `nn.Flatten()` layer. Đây sẽ là nonlinear AR model. Nếu ngoài ra exclude hidden layer, nó sẽ equivalent với earlier linear AR model.

Thay vào đó ta fit nonlinear AR model bằng feature set `X_day`, gồm các `day_of_week` indicators. Để làm vậy, trước hết phải create test và training datasets và corresponding data module. Việc này có vẻ hơi burdensome nhưng là part of general pipeline cho `torch`.

```
In [108]: datasets = []
for mask in [train, ~train]:
 X_day_t = torch.tensor(
 np.asarray(X_day[mask]).astype(np.float32))
 Y_t = torch.tensor(np.asarray(Y[mask]).astype(np.float32))
 datasets.append(TensorDataset(X_day_t, Y_t))
day_train, day_test = datasets
```

Creating data module follows familiar pattern.

```
In [109]: day_dm = SimpleDataModule(day_train,
 day_test,
 num_workers=min(4, max_num_workers),
 validation=day_test,
 batch_size=64)
```

Ta build `NonLinearARModel()` nhận 20 features làm input và hidden layer 32 units. Remaining steps familiar.

```
In [110]: class NonLinearARModel(nn.Module):
 def __init__(self):
 super(NonLinearARModel, self).__init__()
 self._forward = nn.Sequential(nn.Flatten(),
 nn.Linear(20, 32),
 nn.ReLU(),
 nn.Dropout(0.5),
 nn.Linear(32, 1))

 def forward(self, x):
 return torch.flatten(self._forward(x))
```



Chúng ta tiếp tục với các bước training quen thuộc, fit model và evaluate test error. Có thể thấy test  $R^2$  cải thiện nhẹ so với linear AR model cũng bao gồm `day_of_week`.

```
In [112]: nl_trainer = Trainer(deterministic=True,
 max_epochs=20,
 callbacks=[ErrorTracker()])
nl_trainer.fit(nl_module, datamodule=day_dm)
nl_trainer.test(nl_module, datamodule=day_dm)
```

```
Out[112]: [{'test_loss': 0.5625, 'test_r2': 0.4662}]
```

## 10.10 Exercises

### Conceptual

- Xét một neural network có two hidden layers:  $p = 4$  input units, 2 units ở first hidden layer, 3 units ở second hidden layer, và một output duy nhất.
  - Vẽ sơ đồ network, tương tự Figures 10.1 hoặc 10.4.
  - Viết một biểu thức cho  $f(X)$ , giả sử dùng ReLU activation functions. Hãy viết cụ thể nhất có thể.
  - Bây giờ hãy thay một số giá trị cho các coefficients và tính giá trị của  $f(X)$ .
  - Có tất cả bao nhiêu parameters?
- Xét softmax function trong (10.13) (xem thêm (4.13) ở page 145) dùng để model multinomial probabilities.
  - Trong (10.13), hãy chứng minh rằng nếu cộng cùng một constant  $c$  vào mỗi  $z_\ell$ , thì probability không thay đổi.
  - Trong (4.13), hãy chứng minh rằng nếu cộng các constants  $c_j$ ,  $j = 0, 1, \dots, p$ , vào các coefficients tương ứng của mỗi class, thì predictions tại bất kỳ new point  $x$  nào cũng không thay đổi.

Điều này cho thấy softmax function là over-parametrized. Tuy nhiên, regularization và SGD thường constrain các solutions nên điều này không gây ra vấn đề.
- Chứng minh rằng negative multinomial log-likelihood (10.14) tương đương với negative log của likelihood expression (4.5) khi có  $M = 2$  classes.
- Xét một CNN nhận input là các grayscale images  $32 \times 32$  và có một convolution layer với ba convolution filters  $5 \times 5$  (không dùng boundary padding).
  - Vẽ phác thảo input và first hidden layer, tương tự Figure 10.8.
  - Model này có bao nhiêu parameters?
  - Giải thích vì sao model này có thể được xem như một ordinary feed-forward neural network với từng pixel làm input, nhưng có

các constraints trên weights trong hidden units. Các constraints đó là gì?

- (d) Nếu không có các constraints đó, ordinary feed-forward neural network trong (c) sẽ có bao nhiêu weights?
5. Trong Table 10.2 ở page 426, ordering của ba methods theo mean absolute error khác với ordering theo test-set  $R^2$ . Điều này có thể xảy ra như thế nào?

### Applied

6. Xét simple function  $R(\beta) = \sin(\beta) + \beta/10$ .
  - (a) Vẽ graph của function này trên range  $\beta \in [-6, 6]$ .
  - (b) Derivative của function này là gì?
  - (c) Với  $\beta_0 = 2.3$ , chạy gradient descent để tìm một local minimum của  $R(\beta)$  với learning rate  $\rho = 0.1$ . Hãy hiển thị từng  $\beta_0, \beta_1, \dots$  trên plot, cùng với đáp án cuối cùng.
  - (d) Lặp lại với  $\beta_0 = 1.4$ .
7. Fit một neural network cho `Default` data. Dùng một hidden layer với 10 units và dropout regularization. Tham khảo Labs 10.9.1–10.9.2 để được hướng dẫn. So sánh classification performance của model với linear logistic regression.
8. Từ bộ sưu tập ảnh cá nhân của bạn, chọn 10 images về động vật (chẳng hạn dogs, cats, birds, farm animals, v.v.). Nếu subject không chiếm một phần hợp lý của image thì crop image. Sau đó dùng một pretrained image-classification CNN như trong Lab 10.9.4 để predict class của từng image, và báo cáo probabilities của năm predicted classes có probability cao nhất cho mỗi image.
9. Fit một lag-5 autoregressive model cho `NYSE` data, như mô tả trong phần lý thuyết và Lab 10.9.6. Refit model với một 12-level factor biểu diễn month. Factor này có cải thiện performance của model không?
10. Trong Section 10.9.6, chúng ta đã fit một linear AR model cho `NYSE` data bằng function `LinearRegression()`. Tuy nhiên, chúng ta cũng đề cập rằng có thể “flatten” các short sequences được tạo cho RNN model để fit một linear AR model. Hãy dùng cách thứ hai này để fit một linear AR model cho `NYSE` data. So sánh test  $R^2$  của linear AR model này với linear AR model đã fit trong lab. Ưu điểm và nhược điểm của mỗi approach là gì?
11. Lặp lại exercise trước, nhưng lần này fit một nonlinear AR model bằng cách “flatten” các short sequences được tạo cho RNN model.
12. Xét RNN fit cho `NYSE` data trong Section 10.9.6. Modify code để cho phép include variable `day_of_week`, rồi fit RNN. Tính test  $R^2$ .
13. Lặp lại analysis của Lab 10.9.5 trên IMDb data bằng một neural network có structure tương tự. Chúng ta đã dùng 16 hidden units ở mỗi trong two hidden layers. Hãy explore effect của việc tăng con số này

lên 32 và 64 units per layer, cả khi có và không có 30% dropout regularization.

# 11

## Survival Analysis and Censored Data

*survival analysis;  
censored data*

Trong chapter này, chúng ta sẽ xem xét hai chủ đề quan trọng là *survival analysis* và *censored data*. Chúng xuất hiện khi outcome variable có một dạng đặc biệt: thời gian cho đến khi một event xảy ra.

Ví dụ, giả sử chúng ta tiến hành một medical study kéo dài 5 năm, trong đó các patients được điều trị cancer. Ta muốn fit một model để predict survival time của patient, sử dụng các features như baseline health measurements hoặc loại treatment. Thoạt nhìn, đây có vẻ là một regression problem giống như những bài toán đã xét ở Chapter 3. Tuy nhiên, có một complication quan trọng: hy vọng rằng một số, hoặc nhiều, patients vẫn sống cho đến khi study kết thúc. Survival time của một patient như vậy được gọi là *censored*: ta biết nó ít nhất là 5 năm, nhưng không biết true value. Ta không muốn loại bỏ subset những patients còn sống này, vì việc họ sống được ít nhất 5 năm vẫn là thông tin có giá trị. Tuy nhiên, chưa rõ làm thế nào để sử dụng thông tin đó bằng các techniques đã trình bày cho đến thời điểm này trong sách.

Mặc dù cụm từ “survival analysis” thường gợi đến một medical study, các applications của survival analysis vượt xa phạm vi medicine. Chẳng hạn, một company có thể muốn model *churn*, tức quá trình customers hủy subscription đối với một service. Company có thể thu thập data về customers trong một khoảng thời gian để model time to cancellation của mỗi customer như một function của demographics hoặc các predictors khác. Tuy nhiên, có lẽ không phải mọi customer đều hủy subscription trước khi khoảng thời gian quan sát kết thúc; với những customers như vậy, time to cancellation bị censored.

Thực ra, survival analysis còn có liên quan ngay cả trong các application areas không gắn trực tiếp với time. Ví dụ, giả sử ta muốn model weight của một người như một function của một số covariates, dựa trên một dataset gồm measurements của rất nhiều người. Nếu chiếc scale dùng để cân không thể report các weights vượt quá một ngưỡng nhất định, thì mọi weight vượt ngưỡng đó đều bị censored. Các survival-analysis methods được trình bày trong chapter này có thể được dùng để analyze dataset như vậy.

Survival analysis là một topic đã được nghiên cứu rất kỹ trong statistics, do tầm quan trọng then chốt của nó trong nhiều applications cả trong và ngoài medicine. Tuy vậy, chủ đề này nhận được tương đối ít sự chú ý trong machine learning community.

## 11.1 Survival and Censoring Times

Với mỗi individual, giả sử tồn tại một true survival time  $T$ , đồng thời tồn tại một true censoring time  $C$ . Survival time còn được gọi là failure time hoặc event time. Survival time biểu diễn thời điểm event of interest xảy ra, chẳng hạn thời điểm patient chết hoặc customer hủy subscription. Ngược lại, censoring time là thời điểm censoring xảy ra, ví dụ thời điểm patient rời study hoặc thời điểm study kết thúc.

Ta quan sát hoặc survival time  $T$ , hoặc censoring time  $C$ . Cụ thể, ta quan sát random variable

$$Y = \min(T, C). \quad (11.1)$$

Nói cách khác, nếu event xảy ra trước censoring (tức  $T < C$ ) thì ta quan sát true survival time  $T$ ; còn nếu censoring xảy ra trước event ( $T > C$ ) thì ta quan sát censoring time. Ta cũng quan sát một status indicator,

$$\delta = \begin{cases} 1, & T \leq C, \\ 0, & T > C. \end{cases}$$

Do đó,  $\delta = 1$  nếu ta quan sát true survival time, và  $\delta = 0$  nếu giá trị quan sát được là censoring time.

Bây giờ, giả sử ta quan sát  $n$  pairs  $(Y, \delta)$ , được ký hiệu là  $(y_1, \delta_1), \dots, (y_n, \delta_n)$ . Figure 11.1 minh họa một medical study giả định với  $n = 4$  patients được follow-up trong 365 days. Với patients 1 và 3, ta quan sát time to event (chẳng hạn death hoặc disease relapse), nên  $T = t_i$ . Patient 2 vẫn sống khi study kết thúc, còn patient 4 rời study, hay “lost to follow-up”; với hai patients này ta quan sát  $C = c_i$ . Vì vậy,  $y_1 = t_1$ ,  $y_3 = t_3$ ,  $y_2 = c_2$ ,  $y_4 = c_4$ ,  $\delta_1 = \delta_3 = 1$ , và  $\delta_2 = \delta_4 = 0$ .

## 11.2 A Closer Look at Censoring

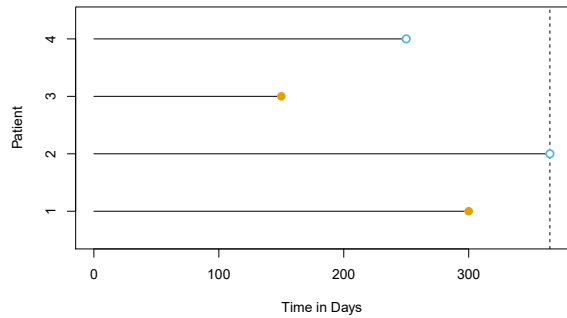
Để analyze survival data, ta cần đưa ra một số assumptions về nguyên nhân censoring xảy ra. Chẳng hạn, giả sử một số patients rời một cancer study sớm vì họ đang rất sick. Một analysis không tính đến lý do patients rời study có khả năng sẽ overestimate true average survival time. Tương tự, giả sử các males đang rất sick có khả năng rời study cao hơn các females đang rất sick. Khi đó, comparison giữa male và female survival times có thể sai lệch và gợi ý rằng males sống lâu hơn females.

Nói chung, ta cần giả sử censoring mechanism là *independent*: conditional on the features, event time  $T$  independent of censoring time  $C$ . Hai examples ở trên đều vi phạm assumption về independent censoring. Thông thường, chỉ từ data thì không thể xác định censoring mechanism có independent hay không. Thay vào đó, cần xem xét cẩn thận data-collection process để đánh giá xem independent censoring có phải là một assumption hợp lý hay không. Trong phần còn lại của chapter, ta sẽ giả sử censoring mechanism là independent.<sup>1</sup>

<sup>1</sup>Assumption về independent censoring có thể được nối lỏng phần nào bằng notion of non-informative censoring; tuy nhiên, định nghĩa của non-informative censoring quá technical so với phạm vi của cuốn sách này.

*survival time;*  
*censoring time;*  
*failure time; event*  
*time*

*independent*  
*censoring*



**HÌNH 11.1.** Minh họa censored survival data. Với patients 1 và 3, event được quan sát. Patient 2 vẫn sống khi study kết thúc. Patient 4 rời khỏi study.

Trong chapter này, chúng ta tập trung vào *right censoring*, xảy ra khi  $T \geq Y$ , tức true event time  $T$  ít nhất bằng observed time  $Y$ . Tên “right censoring” xuất phát từ việc time thường được biểu diễn từ trái sang phải, như trong Figure 11.1. Tuy nhiên, còn có các dạng censoring khác. Với *left censoring*, true event time  $T$  nhỏ hơn hoặc bằng observed time  $Y$ . Ví dụ, trong một study về pregnancy duration, giả sử ta khảo sát patients 250 days sau conception, khi một số người đã sinh con. Với những patients đó, ta chỉ biết pregnancy duration nhỏ hơn 250 days. Tổng quát hơn, *interval censoring* là setting trong đó ta không biết exact event time, nhưng biết nó nằm trong một interval nào đó; chẳng hạn khi patients chỉ được kiểm tra mỗi tuần để xác định event đã xảy ra hay chưa. Left censoring và interval censoring có thể được xử lý bằng các variants của những ideas trình bày trong chapter này, nhưng từ đây trở đi ta tập trung cụ thể vào right censoring.

right censoring; left  
censoring; interval  
censoring

### 11.3 The Kaplan–Meier Survival Curve

*Survival curve*, hay *survival function*, được định nghĩa bởi

$$S(t) = \Pr(T > t). \quad (11.2)$$

Đây là một decreasing function định lượng probability of surviving past time  $t$ . Ví dụ, giả sử một company quan tâm đến việc model customer churn. Gọi  $T$  là thời điểm một customer hủy subscription. Khi đó  $S(t)$  là probability rằng customer sẽ hủy sau time  $t$ . Giá trị  $S(t)$  càng lớn thì customer càng ít có khả năng hủy trước time  $t$ .

survival curve;  
survival function

Trong section này, ta xem xét bài toán estimate survival curve. Phần trình bày được motivate bởi **BrainCancer** dataset, chứa survival times của patients có primary brain tumors và được treatment bằng stereotactic radiation methods.<sup>2</sup> Các predictors gồm **gtv** (gross tumor volume, tính bằng cubic centimeters); **sex** (male hoặc female); **diagnosis** (meningioma, LG glioma, HG glioma, hoặc other); **loc** (vị trí tumor: infratentorial hoặc supratentorial); **ki** (Karnofsky index); và **stereo** (stereotactic method: stereotactic radiosurgery hoặc fractionated stereotactic radiotherapy, lần

<sup>2</sup>Dataset này được mô tả trong Selingerová et al. (2016), “Survival of patients with primary brain tumors: Comparison of two statistical approaches”, PLoS One, 11(2):e0148733.



lượt viết tắt là SRS và SRT). Chỉ 53 trong số 88 patients vẫn sống ở cuối study.

Bây giờ xét bài toán estimate  $S(20) = \Pr(T > 20)$ , tức probability rằng một patient sống ít nhất  $t = 20$  months. Ta có thể bị cám dỗ đơn giản lấy proportion các patients chắc chắn sống quá 20 months, tức proportion có  $Y > 20$ . Con số này là 48/88, xấp xỉ 55%. Tuy nhiên, cách làm này chưa đúng vì  $Y$  và  $T$  là hai quantities khác nhau. Cụ thể, trong 40 patients không có  $Y > 20$ , có 17 patients thực ra bị censored; cách tính vừa rồi ngầm giả sử  $T < 20$  với tất cả 17 patients đó, trong khi ta không hề biết điều này có đúng hay không.

Một alternative để estimate  $S(20)$  là lấy proportion patients có  $Y > 20$  trong số 71 patients chưa bị censored trước time  $t = 20$ . Khi đó ta được 48/71, xấp xỉ 68%. Tuy nhiên, cách này cũng chưa đúng vì nó hoàn toàn bỏ qua những patients bị censored trước  $t = 20$ , dù time at which they are censored có thể cung cấp thông tin. Chẳng hạn, một patient bị censored tại  $t = 19.9$  nhiều khả năng vẫn sẽ sống qua  $t = 20$  nếu không bị censored.

Ta đã thấy việc estimate  $S(t)$  trở nên phức tạp do censoring. Để giải quyết vấn đề, đặt  $d_1 < d_2 < \dots < d_K$  là  $K$  unique death times trong số các non-censored patients, và đặt  $q_k$  là số patients chết tại time  $d_k$ . Với  $k = 1, \dots, K$ , đặt  $r_k$  là số patients còn sống và vẫn ở trong study ngay trước  $d_k$ ; đây là các patients *at risk*. Tập các patients at risk tại một time được gọi là *risk set*. Theo law of total probability,<sup>3</sup>

$$\begin{aligned} \Pr(T > d_k) &= \Pr(T > d_k \mid T > d_{k-1}) \Pr(T > d_{k-1}) \\ &\quad + \Pr(T > d_k \mid T \leq d_{k-1}) \Pr(T \leq d_{k-1}). \end{aligned}$$

Vì  $d_{k-1} < d_k$ , ta có  $\Pr(T > d_k \mid T \leq d_{k-1}) = 0$ . Do đó,

$$S(d_k) = \Pr(T > d_k) = \Pr(T > d_k \mid T > d_{k-1}) \Pr(T > d_{k-1}),$$

hay

$$S(d_k) = \Pr(T > d_k \mid T > d_{k-1}) S(d_{k-1}).$$

Suy ra

$$S(d_k) = \Pr(T > d_k \mid T > d_{k-1}) \times \dots \times \Pr(T > d_2 \mid T > d_1) \Pr(T > d_1).$$

Giờ ta chỉ cần thay các estimates vào từng term ở vế phải. Một estimator tự nhiên là

$$\widehat{\Pr}(T > d_j \mid T > d_{j-1}) = \frac{r_j - q_j}{r_j},$$

tức fraction của risk set tại time  $d_j$  sống qua  $d_j$ . Điều này dẫn đến Kaplan–Meier estimator của survival curve,

$$\widehat{S}(d_k) = \prod_{j=1}^k \left( \frac{r_j - q_j}{r_j} \right). \quad (11.3)$$

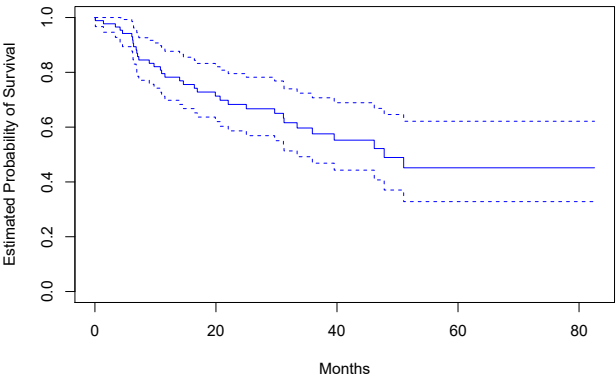
Với times  $t$  nằm giữa  $d_k$  và  $d_{k+1}$ , ta đặt  $\widehat{S}(t) = \widehat{S}(d_k)$ . Vì vậy, Kaplan–Meier survival curve có dạng step-like.

*Kaplan–Meier  
estimator; risk set*

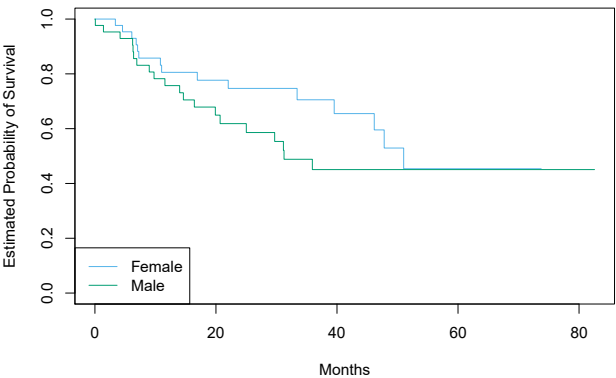
<sup>3</sup>Law of total probability phát biểu rằng với hai events  $A$  và  $B$ ,  $\Pr(A) = \Pr(A \mid B) \Pr(B) + \Pr(A \mid B^c) \Pr(B^c)$ , trong đó  $B^c$  là complement của event  $B$ .

Kaplan–Meier survival curve của **BrainCancer** data được hiển thị trong Figure 11.2. Mỗi point trên solid step-like curve cho biết estimated probability of surviving past the time trên horizontal axis. Estimated probability of survival past 20 months là 71%, cao hơn đáng kể so với hai naive estimates 55% và 68% nêu ở trên.

Sequential construction của Kaplan–Meier estimator—bắt đầu tại time zero và lần lượt theo dõi các observed events khi chúng xảy ra—là ý tưởng nền tảng của nhiều techniques quan trọng trong survival analysis. Trong số đó có log-rank test ở Section 11.4 và Cox’s proportional hazards model ở Section 11.5.2.



**HÌNH 11.2.** Với *BrainCancer* data, Kaplan–Meier survival curve được hiển thị bằng solid curve, cùng các standard-error bands bằng dashed curves.



**HÌNH 11.3.** Với *BrainCancer* data, Kaplan–Meier survival curves cho males và females được hiển thị.

## 11.4 The Log-Rank Test

Ta tiếp tục analysis của **BrainCancer** data được giới thiệu ở Section 11.3. Mục tiêu là compare survival của males và females. Figure 11.3 cho thấy Kaplan–Meier survival curves của hai groups. Females dường như có outcome tốt hơn một chút cho đến khoảng 50 months, nhưng sau đó cả hai

curves cùng level off quanh 50%. Làm thế nào để thực hiện một formal test về equality của hai survival curves?

Thoạt nhìn, two-sample  $t$ -test có vẻ là lựa chọn hiển nhiên: ta có thể test xem mean survival time của females có bằng mean survival time của males hay không. Nhưng censoring lại gây ra complication. Để vượt qua vấn đề này, ta dùng *log-rank test*,<sup>4</sup> một test xem xét cách các events trong mỗi group diễn tiến tuần tự theo time.

*log-rank test*

|          | Group 1           | Group 2           | Total       |
|----------|-------------------|-------------------|-------------|
| Died     | $q_{1k}$          | $q_{2k}$          | $q_k$       |
| Survived | $r_{1k} - q_{1k}$ | $r_{2k} - q_{2k}$ | $r_k - q_k$ |
| Total    | $r_{1k}$          | $r_{2k}$          | $r_k$       |

**BẢNG 11.1.** Trong risk set tại time  $d_k$ , bảng cho biết số patients died và survived trong mỗi of two groups.

Recall từ Section 11.3 rằng  $d_1 < d_2 < \dots < d_K$  là unique death times của non-censored patients,  $r_k$  là số patients at risk tại  $d_k$ , và  $q_k$  là số patients chết tại  $d_k$ . Ta định nghĩa thêm  $r_{1k}$  và  $r_{2k}$  là số patients thuộc groups 1 và 2, tương ứng, đang at risk tại  $d_k$ . Tương tự,  $q_{1k}$  và  $q_{2k}$  là số patients của hai groups chết tại  $d_k$ . Lưu ý rằng  $r_{1k} + r_{2k} = r_k$  và  $q_{1k} + q_{2k} = q_k$ . Tại mỗi death time  $d_k$ , ta lập một  $2 \times 2$  table như Table 11.1. Nếu các death times là unique, thì một trong  $q_{1k}$  và  $q_{2k}$  bằng 1, còn giá trị kia bằng 0.

Ý tưởng chính của log-rank test statistic như sau. Để test  $H_0 : E(X) = \mu$  cho một random variable  $X$ , một approach là xây dựng test statistic dạng

$$W = \frac{X - \mu}{\sqrt{\text{Var}(X)}}. \quad (11.4)$$

Để xây dựng log-rank test statistic, ta dùng đúng dạng (11.4), với  $X = \sum_{k=1}^K q_{1k}$ , trong đó  $q_{1k}$  nằm ở ô trên-trái của Table 11.1.

Cụ thể hơn, nếu không có difference in survival giữa hai groups, và conditional on row and column totals trong Table 11.1, expected value của  $q_{1k}$  là

$$\mu_k = \frac{r_{1k}}{r_k} q_k. \quad (11.5)$$

Do đó expected value của  $X = \sum_{k=1}^K q_{1k}$  là  $\mu = \sum_{k=1}^K (r_{1k}/r_k) q_k$ . Hơn nữa, có thể chứng minh<sup>5</sup> rằng

$$\text{Var}(q_{1k}) = \frac{q_k(r_{1k}/r_k)(1 - r_{1k}/r_k)(r_k - q_k)}{r_k - 1}. \quad (11.6)$$

Mặc dù  $q_{11}, \dots, q_{1K}$  có thể correlated, ta vẫn estimate

$$\text{Var}\left(\sum_{k=1}^K q_{1k}\right) \approx \sum_{k=1}^K \frac{q_k(r_{1k}/r_k)(1 - r_{1k}/r_k)(r_k - q_k)}{r_k - 1}. \quad (11.7)$$

<sup>4</sup>Log-rank test còn được gọi là Mantel–Haenszel test hoặc Cochran–Mantel–Haenszel test.

<sup>5</sup>Chi tiết xem Exercise 7 ở cuối chapter.

Vì vậy, để compute log-rank test statistic, ta chỉ cần áp dụng (11.4) với  $X = \sum_{k=1}^K q_{1k}$  và dùng (11.5), (11.7):

$$W = \frac{\sum_{k=1}^K (q_{1k} - \mu_k)}{\sqrt{\sum_{k=1}^K \text{Var}(q_{1k})}}. \quad (11.8)$$

Khi sample size lớn, log-rank test statistic  $W$  approximately có standard normal distribution; từ đó có thể compute p-value cho null hypothesis rằng hai survival curves không khác nhau.<sup>6</sup>

Comparison survival times giữa females và males trên **BrainCancer** data cho log-rank test statistic  $W = 1.2$ , tương ứng với two-sided p-value 0.2 theo theoretical null distribution và p-value 0.25 theo permutation null distribution với 1,000 permutations. Vì vậy, ta không thể reject null hypothesis rằng hai survival curves không khác nhau. Log-rank test có quan hệ chặt chẽ với Cox's proportional hazards model, sẽ được trình bày ở Section 11.5.2.

## 11.5 Regression Models With a Survival Response

Bây giờ ta xét bài toán fit một regression model cho survival data. Như trong Section 11.1, observations có dạng  $(Y, \delta)$ , với  $Y = \min(T, C)$  là survival time quan sát được (có thể censored), còn  $\delta$  là indicator bằng 1 nếu  $T \leq C$ . Ngoài ra,  $X \in \mathbb{R}^p$  là vector gồm  $p$  features. Mục tiêu là predict true survival time  $T$ .

Vì observed quantity  $Y$  dương và có thể có long right tail, ta có thể nghĩ đến việc fit linear regression của  $\log(Y)$  theo  $X$ . Nhưng censoring lại gây ra vấn đề, vì điều ta thực sự muốn predict là  $T$  chứ không phải  $Y$ . Để xử lý difficulty này, ta tiếp tục sử dụng một sequential construction tương tự Kaplan–Meier survival curve ở Section 11.3 và log-rank test ở Section 11.4.

### 11.5.1 The Hazard Function

*Hazard function* hoặc hazard rate—còn được gọi là force of mortality—được định nghĩa chính thức là

$$h(t) = \lim_{\Delta t \rightarrow 0} \frac{\Pr(t < T \leq t + \Delta t \mid T > t)}{\Delta t}. \quad (11.9)$$

Nó là death rate trong instant ngay sau time  $t$ , conditional on survival past that time.<sup>7</sup>

Tại sao ta quan tâm đến hazard function? Thứ nhất, nó có quan hệ chặt chẽ với survival curve (11.2), như ta sẽ thấy ngay sau đây. Thứ hai, một approach quan trọng để model survival data như một function

*hazard function;  
hazard rate*

<sup>6</sup>Alternatively, p-value có thể được estimate bằng permutations, sử dụng các ideas sẽ trình bày ở Section 13.5. Permutation distribution được tạo bằng cách randomly swapping labels của observations trong hai groups.

<sup>7</sup>Do  $\Delta t$  nằm ở denominator của (11.9), hazard function là một rate of death chứ không phải probability of death. Tuy nhiên, giá trị  $h(t)$  lớn hơn trực tiếp tương ứng với probability of death lớn hơn, tương tự như giá trị lớn hơn của probability density function tương ứng với outcomes có khả năng cao hơn. Thực ra,  $h(t)$  là probability density function của  $T$  conditional on  $T > t$ .

của covariates dựa rất nhiều vào hazard function; approach này là Cox's proportional hazards model ở Section 11.5.2.

Recall rằng với hai events  $A$  và  $B$ ,  $\Pr(A | B) = \Pr(A \cap B) / \Pr(B)$ , và từ (11.2),  $S(t) = \Pr(T > t)$ . Do đó,

$$h(t) = \frac{f(t)}{S(t)}, \quad (11.10)$$

trong đó

$$f(t) = \lim_{\Delta t \rightarrow 0} \frac{\Pr(t < T \leq t + \Delta t)}{\Delta t} \quad (11.11)$$

là probability density function gắn với  $T$ , tức instantaneous rate of death tại time  $t$ . Equality thứ hai trong derivation của (11.10) dùng fact rằng nếu  $t < T \leq t + \Delta t$  thì nhất thiết  $T > t$ . Equation (11.10) cho thấy mối quan hệ giữa hazard function  $h(t)$ , survival function  $S(t)$  và probability density function  $f(t)$ . Trên thực tế, đây là ba ways tương đương để mô tả distribution của  $T$ .<sup>8</sup>

Likelihood contribution của observation thứ  $i$  là

$$L_i = f(y_i)^{\delta_i} S(y_i)^{1-\delta_i}. \quad (11.12)$$

Intuition của (11.12) như sau: nếu  $Y = y_i$  và observation thứ  $i$  không censored, likelihood là probability of dying trong một interval rất nhỏ quanh time  $y_i$ . Nếu observation thứ  $i$  censored, likelihood là probability of surviving ít nhất đến time  $y_i$ . Giả sử  $n$  observations independent, likelihood của toàn data là

$$L = \prod_{i=1}^n f(y_i)^{\delta_i} S(y_i)^{1-\delta_i} = \prod_{i=1}^n h(y_i)^{\delta_i} S(y_i), \quad (11.13)$$

trong đó equality thứ hai follows from (11.10).

Ta có thể bắt đầu model survival times bằng cách assume exponential survival, tức probability density function của  $T$  có dạng  $f(t) = \lambda \exp(-\lambda t)$ . Khi đó việc estimate parameter  $\lambda$  bằng cách maximize likelihood (11.13) khá straightforward.<sup>9</sup> Ta cũng có thể assume survival times đến từ một family distributions flexible hơn, chẳng hạn Gamma hoặc Weibull; hoặc model survival times non-parametrically như Kaplan–Meier estimator ở Section 11.3.

Tuy nhiên, điều ta thực sự muốn là model survival time như một function của covariates. Để làm vậy, làm việc trực tiếp với hazard function thuận tiện hơn probability density function.<sup>10</sup> Một approach là assume một functional form cho  $h(t | x_i)$ , chẳng hạn  $h(t | x_i) = \exp(\beta_0 + \sum_{j=1}^p \beta_j x_{ij})$ , trong đó exponential function đảm bảo hazard không âm. Exponential hazard function là một trường hợp đặc biệt vì nó không thay đổi theo time.<sup>11</sup> Nếu biết  $h(t | x_i)$ , ta có thể calculate  $S(t | x_i)$  rồi plug các equations vào

<sup>8</sup>Xem Exercise 8.

<sup>9</sup>Xem Exercise 9.

<sup>10</sup>Do hazard function  $h(t)$  và density function  $f(t)$  có quan hệ chặt chẽ, việc giả định form của hazard function cũng gần với việc giả định form của density function.

<sup>11</sup>Notation  $h(t | x_i)$  biểu thị hazard function của observation thứ  $i$  conditional on covariate values  $x_i$ .

(11.13) để maximize likelihood và estimate  $\beta = (\beta_0, \beta_1, \dots, \beta_p)^T$ . Tuy nhiên, approach này khá restrictive vì phải áp đặt một assumption rất mạnh về form của hazard function. Section tiếp theo trình bày một approach flexible hơn nhiều.

### 11.5.2 Proportional Hazards

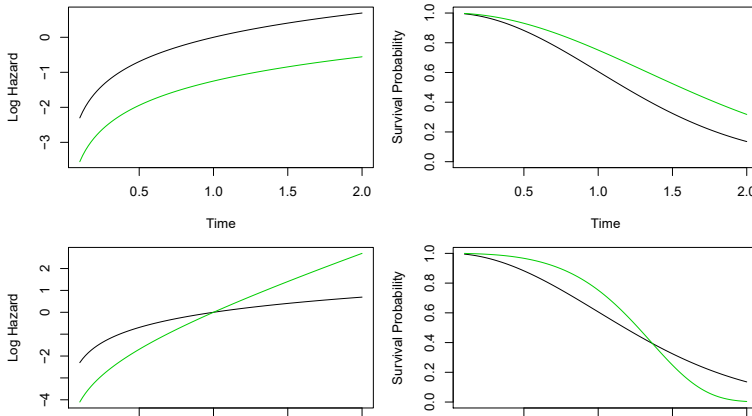
#### The Proportional Hazards Assumption

*Proportional hazards assumption* phát biểu rằng

$$h(t | x_i) = h_0(t) \exp\left(\sum_{j=1}^p x_{ij}\beta_j\right), \quad (11.14)$$

trong đó  $h_0(t) \geq 0$  là một unspecified function, được gọi là *baseline hazard*. Đây là hazard function của một individual có features  $x_{i1} = \dots = x_{ip} = 0$ . Tên “proportional hazards” xuất phát từ việc hazard function của một individual có feature vector  $x_i$  bằng unknown function  $h_0(t)$  nhân với factor  $\exp(\sum_j x_{ij}\beta_j)$ . Quantity  $\exp(\sum_j x_{ij}\beta_j)$  được gọi là *relative risk* của feature vector  $x_i = (x_{i1}, \dots, x_{ip})^T$  so với feature vector  $(0, \dots, 0)^T$ .

proportional  
hazards; baseline  
hazard; relative  
risk



**HÌNH 11.4.** Top: trong một simple example với  $p = 1$  và binary covariate  $x_i \in \{0, 1\}$ , log hazard và survival function dưới model (11.14) được hiển thị (green cho  $x_i = 0$  và black cho  $x_i = 1$ ). Do proportional hazards assumption, hai log-hazard functions khác nhau bởi một constant và hai survival functions không cross. Bottom: vẫn là một binary covariate, nhưng proportional hazards assumption không hold; cả log-hazard functions lẫn survival functions đều cross.

Việc  $h_0(t)$  là “unspecified” nghĩa là về cơ bản ta không đặt assumptions lên functional form của nó. Instantaneous probability of death tại time  $t$ , conditional on having survived ít nhất đến  $t$ , có thể nhận bất kỳ form nào. Điều này làm hazard function rất flexible và cho phép model nhiều dạng relationship giữa covariates và survival time. Assumption chính chỉ là mỗi one-unit increase trong  $x_{ij}$  làm  $h(t | x_i)$  tăng theo factor  $\exp(\beta_j)$ .

Figure 11.4 minh họa proportional hazards assumption trong một setting đơn giản với one binary covariate  $x_i \in \{0, 1\}$ . Ở top row, assumption

(11.14) hold. Vì vậy, hazard functions của hai groups là constant multiples của nhau, và trên log scale khoảng cách giữa chúng là constant. Survival curves không bao giờ cross và ban đầu thường tách xa nhau hơn theo time. Ngược lại, ở bottom row, (11.14) không hold: log-hazard functions cross, và survival curves cũng cross.

### Cox's Proportional Hazards Model

Vì form của  $h_0(t)$  trong proportional hazards assumption (11.14) là unknown, ta không thể đơn giản plug  $h(t | x_i)$  vào likelihood (11.13) rồi estimate  $\beta = (\beta_1, \dots, \beta_p)^T$  bằng maximum likelihood. Điểm đặc biệt của *Cox's proportional hazards model* là vẫn có thể estimate  $\beta$  mà không cần specify form của  $h_0(t)$ .

*Cox's proportional  
hazards model;  
partial likelihood*

Để làm điều này, ta dùng lại logic “sequential in time” đã dùng để derive Kaplan–Meier survival curve và log-rank test. Để đơn giản, giả sử không có ties giữa các failure times. Giả sử  $\delta_i = 1$ , tức observation thứ  $i$  uncensored và  $y_i$  là failure time của nó. Khi đó hazard của observation  $i$  tại  $y_i$  là  $h_0(y_i) \exp(\sum_j x_{ij}\beta_j)$ , còn total hazard tại  $y_i$  của toàn bộ observations đang at risk<sup>12</sup> là

$$\sum_{i': y_{i'} \geq y_i} h_0(y_i) \exp\left(\sum_{j=1}^p x_{i'j}\beta_j\right).$$

Do đó, probability rằng observation thứ  $i$  chính là observation failure tại time  $y_i$  là

$$\frac{\exp(\sum_{j=1}^p x_{ij}\beta_j)}{\sum_{i': y_{i'} \geq y_i} \exp(\sum_{j=1}^p x_{i'j}\beta_j)}, \quad (11.15)$$

bởi vì  $h_0(y_i)$  cancels khỏi numerator và denominator.

*Partial likelihood* đơn giản là product của các probabilities này trên toàn bộ uncensored observations,

$$PL(\beta) = \prod_{i: \delta_i=1} \frac{\exp(\sum_{j=1}^p x_{ij}\beta_j)}{\sum_{i': y_{i'} \geq y_i} \exp(\sum_{j=1}^p x_{i'j}\beta_j)}. \quad (11.16)$$

Điểm then chốt là partial likelihood này valid bất kể true value của  $h_0(t)$ , khiến model rất flexible và robust.<sup>13</sup> Để estimate  $\beta$ , ta maximize (11.16) theo  $\beta$ . Tương tự logistic regression ở Chapter 4, không có closed-form solution nên cần iterative algorithms. Ngoài estimates của  $\beta$ , ta cũng có thể obtain p-values cho các null hypotheses cụ thể, chẳng hạn  $H_0: \beta_j = 0$ , và confidence intervals của các coefficients.

### Connection With The Log-Rank Test

Giả sử chỉ có một predictor ( $p = 1$ ) và predictor này binary,  $x_i \in \{0, 1\}$ . Để xác định có difference giữa survival times của group  $\{i: x_i = 0\}$  và group  $\{i: x_i = 1\}$  hay không, ta có hai approaches: (1) fit Cox proportional

<sup>12</sup>Recall rằng observations “at risk” tại  $y_i$  là những observations vẫn còn có khả năng failure, tức chưa failure hoặc censored trước time  $y_i$ .

<sup>13</sup>Trong các settings mà full likelihood cho mọi parameters khó compute, partial likelihood được dùng để tạo likelihood chỉ cho các parameters được quan tâm chính. Ở đây đó là  $\beta_1, \dots, \beta_p$ . Có thể chứng minh rằng maximizing (11.16) cho các estimates tốt của các parameters này.

hazards model rồi test  $H_0 : \beta = 0$ ; hoặc (2) thực hiện log-rank test để compare hai groups như ở Section 11.4. Approach nào nên được ưu tiên?

Thực ra hai approaches có quan hệ rất chặt chẽ. Khi dùng Cox model, có nhiều ways để test  $H_0$ . Một trong số đó là score test. Với một binary covariate duy nhất, score test cho  $H_0 : \beta = 0$  trong Cox proportional hazards model chính xác bằng log-rank test. Nói cách khác, trong setting này hai approaches cho cùng kết quả.

### Additional Details

Phần trình bày về Cox's proportional hazards model ở trên đã lược qua một số subtleties:

- Không có intercept trong (11.14) hoặc các equations tiếp theo, vì intercept có thể được absorb vào baseline hazard  $h_0(t)$ .
- Ta giả sử không có tied failure times. Khi có ties, exact form của partial likelihood (11.16) phức tạp hơn và cần một số computational approximations.
- (11.16) được gọi là partial likelihood vì không hoàn toàn là likelihood; tức nó không tương ứng chính xác với probability của data dưới assumption (11.14). Tuy nhiên, nó là một approximation rất tốt.
- Ta mới tập trung vào estimation của coefficients  $\beta = (\beta_1, \dots, \beta_p)^T$ . Đôi khi ta cũng muốn estimate baseline hazard  $h_0(t)$ , chẳng hạn để estimate survival curve  $S(t | x)$  cho một individual có feature vector  $x$ . Chi tiết vượt quá phạm vi sách. Python package **lifelines**, được dùng ở Section 11.8, có implementation để estimate  $h_0(t)$ .

### 11.5.3 Example: Brain Cancer Data

Table 11.2 cho thấy kết quả fit proportional hazards model cho **BrainCancer** data được giới thiệu ở Section 11.3. Column Coefficient hiển thị  $\hat{\beta}_j$ . Chẳng hạn, estimated hazard của một male patient bằng  $e^{0.18} = 1.2$  lần estimated hazard của một female patient. Nói cách khác, giữ tất cả các features khác cố định, males có instantaneous chance of dying cao gấp 1.2 lần females tại bất kỳ point in time nào. Tuy nhiên, p-value 0.61 cho thấy difference này không significant.

Một example khác: mỗi one-unit increase trong Karnofsky index tương ứng với multiplier  $\exp(-0.05) = 0.95$  đối với instantaneous chance of dying. Nói cách khác, Karnofsky index càng cao thì chance of dying tại một time bất kỳ càng thấp. Effect này highly significant, với p-value 0.0027.

### 11.5.4 Example: Publication Data

Tiếp theo, xét **Publication** dataset về time to publication của các journal papers báo cáo results của clinical trials được National Heart, Lung, and Blood Institute tài trợ.<sup>14</sup> Với 244 trials, time tính bằng months cho

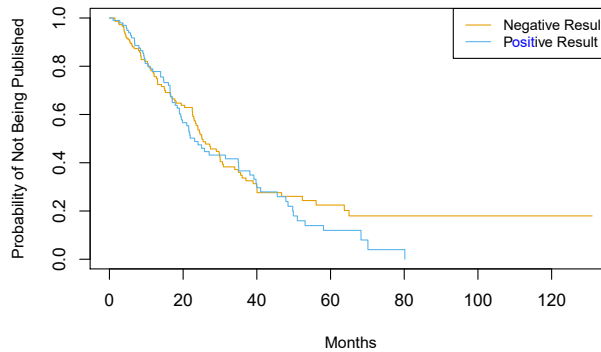
<sup>14</sup>Dataset này được mô tả trong Gordon et al. (2013), "Publication of trials funded by the National Heart, Lung, and Blood Institute", New England Journal of Medicine, 369(20):1926–



|                      | Coefficient | Std. error | z-statistic | p-value |
|----------------------|-------------|------------|-------------|---------|
| sex[Male]            | 0.18        | 0.36       | 0.51        | 0.61    |
| diagnosis[LG Glioma] | 0.92        | 0.64       | 1.43        | 0.15    |
| diagnosis[HG Glioma] | 2.15        | 0.45       | 4.78        | 0.00    |
| diagnosis[Other]     | 0.89        | 0.66       | 1.35        | 0.18    |
| loc[Supratentorial]  | 0.44        | 0.70       | 0.63        | 0.53    |
| ki                   | -0.05       | 0.02       | -3.00       | < 0.01  |
| gtv                  | 0.03        | 0.02       | 1.54        | 0.12    |
| stereo[SRT]          | 0.18        | 0.60       | 0.30        | 0.77    |

**BẢNG 11.2.** Kết quả Cox's proportional hazards model fit cho *BrainCancer* data, lần đầu được mô tả ở Section 11.3. Variable *diagnosis* là qualitative với four levels: *meningioma*, *LG glioma*, *HG glioma*, và *other*. Variables *sex*, *loc*, và *stereo* là binary.

đến publication được ghi lại. Trong 244 trials, chỉ 156 được published trong study period; các studies còn lại bị censored. Covariates gồm trial có tập trung vào clinical endpoint hay không (*clinend*), có involve multiple centers hay không (*multi*), funding mechanism trong National Institutes of Health (*mech*), trial sample size (*sampsize*), budget (*budget*), impact (*impact*, liên quan đến number of citations), và trial có cho positive/significant result hay không (*posres*). Covariate cuối đặc biệt đáng quan tâm vì một số studies đã gợi ý positive trials có publication rate cao hơn.



**HÌNH 11.5.** Survival curves cho time until publication của *Publication* data được mô tả ở Section 11.5.4, stratified theo việc study có cho positive result hay không.

Figure 11.5 cho thấy Kaplan–Meier curves của time until publication, stratified theo việc study có positive result hay không. Có một chút evidence rằng time until publication ngắn hơn với studies có positive result. Tuy nhiên, log-rank test cho p-value không ấn tượng là 0.36.

Bây giờ ta thực hiện một analysis cẩn thận hơn bằng cách dùng tất cả available predictors. Results của Cox's proportional hazards model được trình bày trong Table 11.3. Chance of publication của một study có positive result là  $e^{0.55} = 1.74$  lần chance of publication của một study có negative result tại bất kỳ point in time nào, giữ tất cả covariates khác cố định. P-value rất nhỏ của *posres* trong Table 11.3 cho thấy result này highly

significant. Điều này khá striking nếu so với finding trước đó rằng log-rank test giữa positive và negative results cho p-value 0.36. Sự khác biệt xuất phát từ fact rằng log-rank test không xét các covariates khác, còn Table 11.3 dựa trên Cox model dùng toàn bộ available covariates. Nói cách khác, sau khi adjust cho các covariates khác, việc study có positive result hay không trở thành predictor rất mạnh của time to publication.

Để hiểu thêm, Figure 11.6 hiển thị estimated survival curves của positive và negative results sau khi adjust cho các predictors khác. Để tạo các curves này, ta estimate underlying baseline hazard  $h_0(t)$  và chọn representative values cho các predictors còn lại: mean cho mỗi quantitative predictor, và category phổ biến nhất (R01) cho qualitative predictor **mech**. Sau adjustment, sự khác biệt giữa positive và negative results trở nên rõ ràng. Table 11.3 còn cho một số insights khác; chẳng hạn, studies có clinical endpoint có xu hướng được published sớm hơn studies không có clinical endpoint, trong khi funding mechanism dường như không significantly associated với time until publication.

|               | Coefficient | Std. error | z-statistic | p-value |
|---------------|-------------|------------|-------------|---------|
| posres[Yes]   | 0.55        | 0.18       | 3.02        | 0.00    |
| multi[Yes]    | 0.15        | 0.31       | 0.47        | 0.64    |
| clinend[Yes]  | 0.51        | 0.27       | 1.89        | 0.06    |
| mech[K01]     | 1.05        | 1.06       | 1.00        | 0.32    |
| mech[K23]     | -0.48       | 1.05       | -0.45       | 0.65    |
| mech[P01]     | -0.31       | 0.78       | -0.40       | 0.69    |
| mech[P50]     | 0.60        | 1.06       | 0.57        | 0.57    |
| mech[R01]     | 0.10        | 0.32       | 0.30        | 0.76    |
| mech[R18]     | 1.05        | 1.05       | 0.99        | 0.32    |
| mech[R21]     | -0.05       | 1.06       | -0.04       | 0.97    |
| mech[R24.K24] | 0.81        | 1.05       | 0.77        | 0.44    |
| mech[R42]     | -14.78      | 3414.38    | -0.00       | 1.00    |
| mech[R44]     | -0.57       | 0.77       | -0.73       | 0.46    |
| mech[RC2]     | -14.92      | 2243.60    | -0.01       | 0.99    |
| mech[U01]     | -0.22       | 0.32       | -0.70       | 0.48    |
| mech[U54]     | 0.47        | 1.07       | 0.44        | 0.66    |
| sampsize      | 0.00        | 0.00       | 0.19        | 0.85    |
| budget        | 0.00        | 0.00       | 1.67        | 0.09    |
| impact        | 0.06        | 0.01       | 8.23        | 0.00    |

**BẢNG 11.3.** Kết quả Cox's proportional hazards model fit cho *Publication* data bằng tất cả available features. Features **posres**, **multi**, và **clinend** là binary. Feature **mech** là qualitative với 14 levels và được coded với baseline level là *Contract*.

## 11.6 Shrinkage for the Cox Model

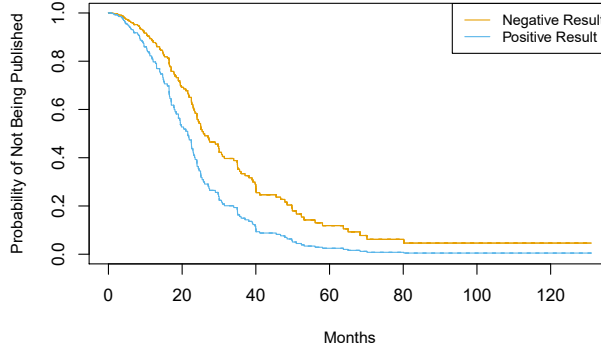
Trong section này, ta minh họa rằng shrinkage methods của Section 6.2 có thể áp dụng cho survival data. Xuất phát từ formulation “loss+penalty” ở Section 6.2, ta xét việc minimize một penalized version của negative log partial likelihood trong (11.16),

$$-\log \left[ \prod_{i: \delta_i=1} \frac{\exp(\sum_{j=1}^p x_{ij}\beta_j)}{\sum_{i': y_{i'} \geq y_i} \exp(\sum_{j=1}^p x_{i'j}\beta_j)} \right] + \lambda P(\beta), \quad (11.17)$$

đối với  $\beta = (\beta_1, \dots, \beta_p)^T$ . Ta có thể chọn  $P(\beta) = \sum_{j=1}^p \beta_j^2$ , tương ứng với ridge penalty, hoặc  $P(\beta) = \sum_{j=1}^p |\beta_j|$ , tương ứng với lasso penalty.

Trong (11.17),  $\lambda$  là một non-negative tuning parameter; thông thường ta minimize objective với nhiều values của  $\lambda$ . Khi  $\lambda = 0$ , minimizing

(11.17) tương đương maximizing ordinary Cox partial likelihood (11.16). Khi  $\lambda > 0$ , minimizing (11.17) cho một shrunken version của coefficient estimates. Với large  $\lambda$ , ridge penalty cho các coefficients nhỏ nhưng không exactly bằng zero; ngược lại, với sufficiently large  $\lambda$ , lasso penalty làm một số coefficients exactly zero.



**HÌNH 11.6.** Với *Publication* data, hiển thị survival curves của time until publication, stratified theo việc study có positive result hay không, sau khi adjust cho mọi covariates khác.

Bây giờ apply lasso-penalized Cox model cho *Publication* data. Trước hết, ta randomly split 244 trials thành training set và test set có size bằng nhau. Cross-validation results trên training set được hiển thị trong Figure 11.7. “Partial likelihood deviance” trên y-axis bằng hai lần cross-validated negative log partial likelihood và đóng vai trò như cross-validation error.<sup>15</sup> Curve của partial likelihood deviance có dạng chữ U: giống như ở các chapters trước, cross-validation error nhỏ nhất tại một mức model complexity trung gian. Cụ thể, điều này xảy ra khi chỉ hai predictors *budget* và *impact* có non-zero estimated coefficients.

Làm thế nào để áp dụng model này cho test set? Có một conceptual point quan trọng: về cơ bản không có cách đơn giản để compare predicted survival times với true survival times trên test set. Vấn đề thứ nhất là một số observations bị censored nên true survival times của chúng không observed. Vấn đề thứ hai là Cox model không predict một single survival time từ covariate vector  $x$ ; thay vào đó nó estimate cả một survival curve  $S(t | x)$  như một function của  $t$ .

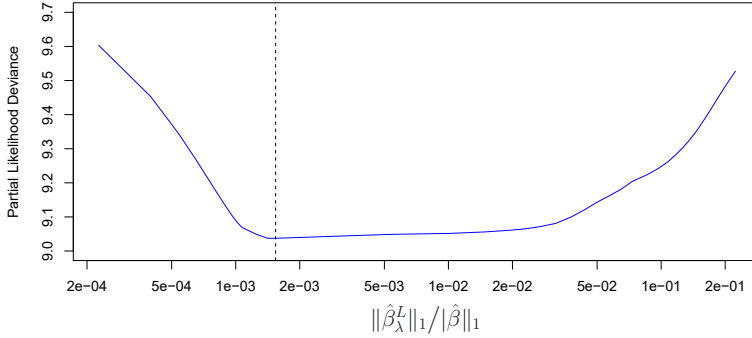
Vì vậy, để assess model fit, ta dùng một approach khác bằng cách stratify observations theo coefficient estimates. Với mỗi test observation, tính “risk” score

$$\text{budget}_i \hat{\beta}_{\text{budget}} + \text{impact}_i \hat{\beta}_{\text{impact}},$$

trong đó  $\hat{\beta}_{\text{budget}}$  và  $\hat{\beta}_{\text{impact}}$  được estimate từ training set. Sau đó dùng các risk scores để categorize observations theo risk. High-risk group gồm observations có risk score lớn nhất; theo (11.14), đó là những observations có instantaneous probability of being published lớn nhất tại bất kỳ moment in time nào. Nói cách khác, high-risk group gồm những trials có khả năng

<sup>15</sup>Cross-validation cho Cox model phức tạp hơn linear hoặc logistic regression vì objective function không phải là sum trên các observations.

được published sớm hơn. Trên *Publication* data, ta stratify observations thành tertiles low, medium và high risk. Figure 11.8 cho thấy survival curves của ba strata tách biệt rõ ràng và được ordered đúng theo mức risk of publication.



**HÌNH 11.7.** Với *Publication* data ở Section 11.5.4, Figure hiển thị cross-validation results của lasso-penalized Cox model. Y-axis là partial likelihood deviance, đóng vai trò cross-validation error. X-axis là  $\ell_1$  norm của coefficients của lasso-penalized Cox model với tuning parameter  $\lambda$ , chia cho  $\ell_1$  norm của coefficients của unpenalized Cox model. Dashed line chỉ minimum cross-validation error.

## 11.7 Additional Topics

### 11.7.1 Area Under the Curve for Survival Analysis

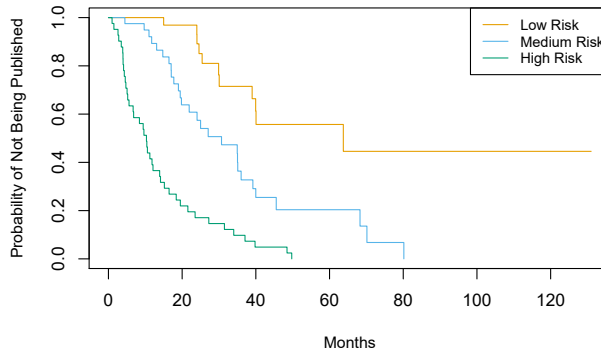
Trong Chapter 4, ta đã giới thiệu area under the ROC curve, thường gọi là AUC, để quantify performance của một two-class classifier. Gọi score của observation thứ  $i$  là estimate của classifier cho  $\Pr(Y = 1 \mid X = x_i)$ . Nếu xét mọi pairs gồm một observation thuộc Class 1 và một observation thuộc Class 2, thì AUC là fraction các pairs trong đó score của observation thuộc Class 1 lớn hơn score của observation thuộc Class 2.

Điều này gợi ý một cách generalize notion of AUC sang survival analysis. Ta tính estimated risk score  $\hat{\eta}_i = \hat{\beta}_1 x_{i1} + \dots + \hat{\beta}_p x_{ip}$ ,  $i = 1, \dots, n$ , từ Cox model coefficients. Nếu  $\hat{\eta}_{i'} > \hat{\eta}_i$ , model predict observation  $i'$  có hazard lớn hơn observation  $i$ , và do đó survival time  $t_i$  sẽ lớn hơn  $t_{i'}$ . Ta có thể muốn generalize AUC bằng proportion observations có  $t_i > t_{i'}$  và  $\hat{\eta}_{i'} > \hat{\eta}_i$ . Tuy nhiên, ta không observe  $t_1, \dots, t_n$ ; chỉ observe possibly-censored times  $y_1, \dots, y_n$  và censoring indicators  $\delta_1, \dots, \delta_n$ .

Vì vậy, Harrell's concordance index, hay C-index, tính proportion các observation pairs mà  $\hat{\eta}_{i'} > \hat{\eta}_i$  và  $y_i > y_{i'}$ :

$$C = \frac{\sum_{i, i': y_i > y_{i'}} I(\hat{\eta}_{i'} > \hat{\eta}_i) \delta_{i'}}{\sum_{i, i': y_i > y_{i'}} \delta_{i'}}.$$

Indicator  $I(\hat{\eta}_{i'} > \hat{\eta}_i)$  bằng one nếu inequality hold và zero nếu không. Numerator và denominator đều được nhân với status indicator  $\delta_{i'}$ , vì nếu observation  $i'$  uncensored ( $\delta_{i'} = 1$ ), thì  $y_i > y_{i'}$  suy ra  $t_i > t_{i'}$ . Ngược lại, nếu  $\delta_{i'} = 0$ , inequality  $y_i > y_{i'}$  không đảm bảo  $t_i > t_{i'}$ .



**HÌNH 11.8.** Với *Publication* data, ta chia test set thành các tertiles của “risk” bằng coefficients estimated trên training set. Các resulting survival curves tách biệt rõ ràng.

Fit Cox proportional hazards model trên training set của *Publication* data và compute C-index trên test set cho  $C = 0.733$ . Nói một cách gần đúng, nếu chọn ngẫu nhiên hai papers từ test set, model predict với accuracy 73.3% paper nào sẽ được published trước.

### 11.7.2 Choice of Time Scale

Trong các examples đến đây, cách define time khá rõ. Trong *Publication* example, time zero của mỗi paper được define là calendar time tại end of study, còn failure time là số months trôi qua từ end of study cho đến khi paper được published. Tuy nhiên, trong các settings khác, definitions của time zero và failure time có thể tinh tế hơn.

Ví dụ, khi xét association giữa risk factors và disease occurrence trong một epidemiological study, có thể dùng patient age để define time, tức time zero là date of birth. Với choice này, association giữa age và survival không thể được đo riêng, nhưng cũng không cần adjust cho age trong analysis. Khi xét covariates associated với disease-free survival, tức amount of time giữa treatment và disease recurrence, ta có thể dùng treatment date làm time zero.

### 11.7.3 Time-Dependent Covariates

Một feature mạnh của proportional hazards model là khả năng xử lý *time-dependent covariates*, tức predictors có value thay đổi theo time. Ví dụ, giả sử đo blood pressure của một patient mỗi tuần trong một medical study. Khi đó ta có thể xem blood pressure của observation thứ  $i$  không phải là một fixed  $x_i$ , mà là  $x_i(t)$  tại time  $t$ .

*time-dependent  
covariates*

Vì partial likelihood trong (11.16) được xây dựng sequentially in time, việc xử lý time-dependent covariates khá straightforward. Cụ thể, chỉ cần thay  $x_{ij}$  và  $x_{i'j}$  trong (11.16) bằng current values  $x_{ij}(y_i)$  và  $x_{i'j}(y_i)$ . Ngược lại, time-dependent covariates gây difficulty lớn hơn nhiều trong traditional parametric approach như (11.13).

Một example về time-dependent covariates xuất hiện trong analysis của

Stanford Heart Transplant Program. Patients cần heart transplant được đưa vào waiting list. Một số patients nhận transplant, nhưng một số khác chết khi vẫn còn trên waiting list. Primary objective là xác định transplant có associated với longer patient survival hay không.

Một naive approach sẽ dùng fixed covariate để represent transplant status:  $x_i = 1$  nếu patient thứ  $i$  từng nhận transplant, và  $x_i = 0$  nếu không. Nhưng cách này bỏ qua fact rằng patients phải sống đủ lâu mới có thể nhận transplant; do đó, trung bình, healthier patients có khả năng nhận transplant cao hơn. Vấn đề được giải quyết bằng time-dependent covariate:  $x_i(t) = 1$  nếu patient đã nhận transplant trước hoặc tại time  $t$ , và  $x_i(t) = 0$  nếu chưa.

#### 11.7.4 Checking the Proportional Hazards Assumption

Ta đã thấy Cox's proportional hazards model dựa trên proportional hazards assumption (11.14). Dù results của Cox model thường khá robust trước violations của assumption này, vẫn nên kiểm tra xem nó có hợp lý hay không. Với qualitative feature, ta có thể plot log hazard function cho từng level của feature. Nếu (11.14) hold, các log-hazard functions chỉ nên khác nhau bởi một constant, như ở top-left panel của Figure 11.4. Với quantitative feature, có thể dùng approach tương tự sau khi stratify feature thành các groups.

#### 11.7.5 Survival Trees

Trong Chapter 8, ta đã thảo luận các flexible và adaptive learning procedures như trees, random forests và boosting, áp dụng cho cả regression và classification. Hầu hết các approaches này có thể generalize sang survival-analysis setting. Ví dụ, *survival trees* là modification của classification và regression trees, dùng split criterion nhằm maximize difference giữa survival curves của resulting daughter nodes. Survival trees sau đó có thể được combine để tạo random survival forests.

*survival trees*

### 11.8 Lab: Survival Analysis

Trong lab này, chúng ta thực hiện survival analyses trên ba datasets riêng biệt. Ở Section 11.8.1, ta analyze **BrainCancer** data lần đầu được mô tả ở Section 11.3. Ở Section 11.8.2, ta xét **Publication** data từ Section 11.5.4. Cuối cùng, Section 11.8.3 explore một simulated call-center dataset.

Ta bắt đầu bằng cách import một số libraries quen thuộc ở top level. Điều này giúp code dễ đọc hơn, vì chỉ cần scan vài lines đầu của notebook là có thể thấy những libraries nào được dùng.

In [1]:

```
from matplotlib.pyplot import subplots
import numpy as np
import pandas as pd
from ISLP.models import ModelSpec as MS
from ISLP import load_data
```

Ta cũng tập hợp các imports mới cần cho lab này.

```
In [2]: from lifelines import \
 (KaplanMeierFitter,
 CoxPHFitter)
 from lifelines.statistics import \
 (logrank_test,
 multivariate_logrank_test)
 from ISLP.survival import sim_time
```

### 11.8.1 Brain Cancer Data

Ta bắt đầu với **BrainCancer** dataset có trong ISLP package.

```
In [3]: BrainCancer = load_data('BrainCancer')
 BrainCancer.columns
```

```
Out[3]: Index(['sex', 'diagnosis', 'loc', 'ki', 'gtv', 'stereo',
 'status', 'time'],
 dtype='object')
```

Các rows index 88 patients, còn 8 columns chứa predictors và outcome variables. Trước tiên, ta briefly examine data.

```
In [4]: BrainCancer['sex'].value_counts()
```

```
Out[4]: Female 45
 Male 43
 Name: sex, dtype: int64
```

```
In [5]: BrainCancer['diagnosis'].value_counts()
```

```
Out[5]: Meningioma 42
 HG glioma 22
 Other 14
 LG glioma 9
 Name: diagnosis, dtype: int64
```

```
In [6]: BrainCancer['status'].value_counts()
```

```
Out[6]: 0 53
 1 35
 Name: status, dtype: int64
```

Trước khi bắt đầu analysis, cần biết variable **status** được coded như thế nào. Hầu hết survival software dùng convention rằng status bằng 1 chỉ một uncensored observation, thường là death, còn status bằng 0 chỉ một censored observation. Tuy nhiên, một số scientists có thể dùng coding ngược lại. Với **BrainCancer** data, 35 patients chết trước khi study kết thúc, vì vậy dataset này dùng conventional coding.

Để bắt đầu analysis, ta re-create Kaplan–Meier survival curve trong Figure 11.2. Package chính dùng cho survival analysis là **lifelines**. Variable **time** tương ứng với  $y_i$ , tức time to the  $i$ th event (censoring hoặc death). Argument đầu của **km.fit()** là event time, còn argument thứ hai là cen-

soring/status variable, trong đó 1 chỉ một observed failure time. Method `plot()` tạo survival curve với pointwise confidence intervals. Theo default, đây là 90% confidence intervals, nhưng có thể thay đổi bằng cách đặt argument `alpha` bằng one minus desired confidence level.

In [7]:

```
fig, ax = subplots(figsize=(8,8))
km = KaplanMeierFitter()
km_brain = km.fit(BrainCancer['time'], BrainCancer['status'])
km_brain.plot(label='Kaplan Meier estimate', ax=ax)
```

Tiếp theo ta tạo Kaplan–Meier survival curves stratified by sex để reproduce Figure 11.3. Ta dùng method `groupby()` của dataframe. Method này trả về một generator có thể iterate trong for loop. Ở đây, mỗi item là một 2-tuple đại diện cho một group: entry thứ nhất là value của grouping column `sex`, còn entry thứ hai là dataframe gồm các rows có value `sex` tương ứng. Ta sẽ cần data này cho log-rank test ở dưới, vì vậy lưu các groups trong dictionary `by_sex`. Cuối cùng, ta dùng string interpolation để tự động label các curves khác nhau; Python có nhiều cách hỗ trợ việc format strings như vậy.

In [8]:

```
fig, ax = subplots(figsize=(8,8))
by_sex = {}
for sex, df in BrainCancer.groupby('sex'):
 by_sex[sex] = df
 km_sex = km.fit(df['time'], df['status'])
 km_sex.plot(label='Sex=%s' % sex, ax=ax)
```

Như đã thảo luận ở Section 11.4, ta có thể thực hiện log-rank test để compare survival của males và females. Ta dùng function `logrank_test()` từ module `lifelines.statistics`. Hai arguments đầu là event times của hai groups, còn hai arguments tiếp theo là các censoring indicators tương ứng.

In [9]:

```
logrank_test(by_sex['Male']['time'],
 by_sex['Female']['time'],
 by_sex['Male']['status'],
 by_sex['Female']['status'])
```

Out[9]:

|                    |     |              |
|--------------------|-----|--------------|
|                    | t_0 | -1           |
| null_distribution  |     | chi squared  |
| degrees_of_freedom |     | 1            |
| test_name          |     | logrank_test |

|                |      |          |
|----------------|------|----------|
| test_statistic | p    | -log2(p) |
| 1.44           | 0.23 | 2.12     |

Resulting p-value là 0.23, cho thấy không có evidence về difference in survival giữa hai sexes.

Tiếp theo, ta dùng estimator `CoxPHFitter()` từ `lifelines` để fit Cox proportional hazards models. Trước hết xét model chỉ dùng `sex` làm predictor.

In [10]:

```
coxph = CoxPHFitter # shorthand
sex_df = BrainCancer[['time', 'status', 'sex']]
model_df = MS(['time', 'status', 'sex'],
 intercept=False).fit_transform(sex_df)
```



```
cox_fit = coxph().fit(model_df,
 'time',
 'status')
cox_fit.summary[['coef', 'se(coef)', 'p']]
```

```
Out[10]:
```

|           | coef     | se(coef) | p        |
|-----------|----------|----------|----------|
| covariate |          |          |          |
| sex[Male] | 0.407667 | 0.342004 | 0.233263 |

Argument đầu của `fit()` phải là một dataframe chứa ít nhất event time (argument thứ hai, `time`, trong example này), cùng với optional censoring variable (argument `status`). Lưu ý Cox model không có intercept, vì vậy ta dùng `intercept=False` trong `ModelSpec`. Method `summary()` trả về nhiều columns; ở đây ta chỉ hiển thị một subset. Ta cũng có thể obtain likelihood-ratio test comparing model này với model không có features như sau.

```
In [11]: cox_fit.log_likelihood_ratio_test()
```

```
Out[11]:
```

|                 | chi squared               |
|-----------------|---------------------------|
| degrees_freedom | 1                         |
| test_name       | log-likelihood ratio test |

| test_statistic | p    | -log2(p) |
|----------------|------|----------|
| 1.44           | 0.23 | 2.12     |

Bất kể dùng test nào, ta đều thấy không có clear evidence về difference in survival giữa males và females. Như đã học trong chapter này, score test từ Cox model với một binary covariate chính xác bằng log-rank test statistic.

Bây giờ fit một model dùng thêm các predictors khác. Trước tiên, lưu ý có một missing value trong `diagnosis`, vì vậy ta drop observation đó trước khi tiếp tục.

```
In [12]: cleaned = BrainCancer.dropna()
all_MS = MS(cleaned.columns, intercept=False)
all_df = all_MS.fit_transform(cleaned)
fit_all = coxph().fit(all_df,
 'time',
 'status')
fit_all.summary[['coef', 'se(coef)', 'p']]
```

```
Out[12]:
```

|                       | coef      | se(coef) | p        |
|-----------------------|-----------|----------|----------|
| covariate             |           |          |          |
| sex[Male]             | 0.183748  | 0.360358 | 0.610119 |
| diagnosis[LG glioma]  | -1.239541 | 0.579557 | 0.032454 |
| diagnosis[Meningioma] | -2.154566 | 0.450524 | 0.000002 |
| diagnosis[Other]      | -1.268870 | 0.617672 | 0.039949 |
| loc[Supratentorial]   | 0.441195  | 0.703669 | 0.530664 |
| ki                    | -0.054955 | 0.018314 | 0.002693 |
| gtv                   | 0.034293  | 0.022333 | 0.124660 |
| stereo[SRT]           | 0.177778  | 0.601578 | 0.767597 |

Variable `diagnosis` được coded sao cho baseline là HG glioma. Results cho thấy risk associated với HG glioma cao hơn hơn tám lần risk associated với meningioma, vì  $e^{2.15} = 8.62$ . Nói cách khác, sau khi adjust cho các predictors khác, patients có HG glioma có survival tệ hơn nhiều so với

patients có meningioma. Ngoài ra, values lớn hơn của Karnofsky index `ki` associated với lower risk, tức longer survival.

Cuối cùng, ta plot estimated survival curves cho từng diagnosis category, đồng thời adjust cho các predictors khác. Để tạo các plots này, ta đặt quantitative variables bằng mean của chúng và categorical variables bằng mode. Ta dùng method `apply()` theo columns (`axis=0`) với một function `representative` để kiểm tra column có categorical hay không.

```
In [13]:
levels = cleaned['diagnosis'].unique()
def representative(series):
 if hasattr(series.dtype, 'categories'):
 return pd.Series.mode(series)
 else:
 return series.mean()
modal_data = cleaned.apply(representative, axis=0)
```

Sau đó tạo bốn copies của representative row và gán column `diagnosis` bằng bốn diagnosis levels khác nhau.

```
In [14]:
modal_df = pd.DataFrame(
 [modal_data.iloc[0] for _ in range(len(levels))])
modal_df['diagnosis'] = levels
modal_df
```

```
Out[14]:
 sex diagnosis loc ki gtv stereo ...
0 Female Meningioma Supratentorial 80.920 8.687 SRT ...
1 Female HG glioma Supratentorial 80.920 8.687 SRT ...
2 Female LG glioma Supratentorial 80.920 8.687 SRT ...
3 Female Other Supratentorial 80.920 8.687 SRT ...
```

Ta construct model matrix dựa trên model specification `all_MS` đã dùng để fit model, rồi đặt row names theo các diagnosis levels.

```
In [15]:
modal_X = all_MS.transform(modal_df)
modal_X.index = levels
modal_X
```

Ta có thể dùng method `predict_survival_function()` để obtain estimated survival function.

```
In [16]:
predicted_survival = fit_all.predict_survival_function(modal_X)
predicted_survival
```

```
Out[16]:
 Meningioma HG glioma LG glioma Other
0.070 0.998 0.982 0.995 0.995
1.180 0.998 0.982 0.995 0.995
1.410 0.996 0.963 0.989 0.990
1.540 0.996 0.963 0.989 0.990
...
67.380 0.689 0.040 0.394 0.405
73.740 0.689 0.040 0.394 0.405
78.750 0.689 0.040 0.394 0.405
82.560 0.689 0.040 0.394 0.405
85 rows × 4 columns
```

Kết quả trả về một dataframe; plot methods của dataframe tạo các survival curves khác nhau. Để tránh làm plot quá rối, ta không hiển thị confidence intervals.

```
In [17]: fig, ax = subplots(figsize=(8, 8))
 predicted_survival.plot(ax=ax);
```

### 11.8.2 Publication Data

Publication data được trình bày ở Section 11.5.4 có trong ISLP package. Trước tiên ta reproduce Figure 11.5 bằng cách plot Kaplan–Meier curves stratified theo variable `posres`, variable này ghi study có positive result hay không.

```
In [18]: fig, ax = subplots(figsize=(8,8))
 Publication = load_data('Publication')
 by_result = {}
 for result, df in Publication.groupby('posres'):
 by_result[result] = df
 km_result = km.fit(df['time'], df['status'])
 km_result.plot(label='Result=%d' % result, ax=ax)
```

Như đã thảo luận trước đó, p-values từ việc fit Cox's proportional hazards model chỉ với `posres` khá lớn, nên không có evidence về difference in time-to-publication giữa studies có positive và negative results.

```
In [19]: posres_df = MS(['posres',
 'time',
 'status'],
 intercept=False).fit_transform(Publication)
 posres_fit = coxph().fit(posres_df,
 'time',
 'status')
 posres_fit.summary[['coef', 'se(coef)', 'p']]
```

```
Out[19]:
```

|           | coef     | se(coef) | p        |
|-----------|----------|----------|----------|
| covariate |          |          |          |
| posres    | 0.148076 | 0.161625 | 0.359578 |

Tuy nhiên, results thay đổi đáng kể khi include các predictors khác trong model. Ở đây ta exclude funding mechanism variable.

```
In [20]: model = MS(Publication.columns.drop('mech'),
 intercept=False)
 coxph().fit(model.fit_transform(Publication),
 'time',
 'status').summary[['coef', 'se(coef)', 'p']]
```

```
Out[20]:
```

|           | coef      | se(coef) | p            |
|-----------|-----------|----------|--------------|
| covariate |           |          |              |
| posres    | 0.570774  | 0.175960 | 1.179606e-03 |
| multi     | -0.040863 | 0.251194 | 8.707727e-01 |
| clinend   | 0.546180  | 0.262001 | 3.710099e-02 |
| sampsize  | 0.000005  | 0.000015 | 7.506978e-01 |
| budget    | 0.004386  | 0.002464 | 7.511276e-02 |
| impact    | 0.058318  | 0.006676 | 2.426779e-18 |

Ta thấy một số variables statistically significant, gồm việc trial có clinical endpoint hay không, impact của study, và study có positive hay negative result.

### 11.8.3 Call Center Data

Trong section này, ta simulate survival data bằng mối quan hệ giữa cumulative hazard và survival function được explore trong Exercise 8. Simulated data đại diện cho observed wait times, tính bằng seconds, của 2,000 customers gọi đến một call center. Trong context này, censoring xảy ra nếu customer gác máy trước khi call được answered.

Có ba covariates: **Operators** (số call-center operators có sẵn tại thời điểm call, từ 5 đến 15), **Center** (A, B hoặc C), và **Time of day** (Morning, Afternoon hoặc Evening). Ta generate data sao cho mọi possibilities đều equally likely; chẳng hạn morning, afternoon và evening calls equally likely, và mỗi số operators từ 5 đến 15 cũng equally likely.

In [21]:

```
rng = np.random.default_rng(10)
N = 2000
Operators = rng.choice(np.arange(5, 16),
 N,
 replace=True)
```

In [22]:

```
Center = rng.choice(['A', 'B', 'C'],
 N,
 replace=True)
Time = rng.choice(['Morn.', 'After.', 'Even.'],
 N,
 replace=True)
D = pd.DataFrame({'Operators': Operators,
 'Center': pd.Categorical(Center),
 'Time': pd.Categorical(Time)})
```

Sau đó ta build model matrix, bỏ intercept.

In [23]:

```
model = MS(['Operators',
 'Center',
 'Time'],
 intercept=False)
X = model.fit_transform(D)
X[:5]
```

Out[23]:

|   | Operators | Center[B] | Center[C] | Time[Even.] | Time[Morn.] |
|---|-----------|-----------|-----------|-------------|-------------|
| 0 | 13        | 0.0       | 1.0       | 0.0         | 0.0         |
| 1 | 15        | 0.0       | 0.0       | 1.0         | 0.0         |
| 2 | 7         | 1.0       | 0.0       | 0.0         | 1.0         |
| 3 | 7         | 0.0       | 1.0       | 0.0         | 1.0         |
| 4 | 13        | 0.0       | 1.0       | 1.0         | 0.0         |

Nên xem qua model matrix  $X$  để chắc rằng ta hiểu các variables được coded như thế nào. Theo default, levels của categorical variables được sorted và, như thường lệ, first column của one-hot encoding bị dropped.

Tiếp theo, ta specify coefficients và hazard function.

In [24]:

```
true_beta = np.array([0.04, -0.3, 0, 0.2, -0.2])
true_linpred = X.dot(true_beta)
hazard = lambda t: 1e-5 * t
```

Coefficient associated với **Operators** được đặt bằng 0.04; nói cách khác, mỗi additional operator làm “risk” rằng call được answered tăng theo factor  $e^{0.04} = 1.041$ , conditional on **Center** và **Time**. Điều này hợp lý: càng nhiều

operators thì wait time càng ngắn. Coefficient của `Center == B` là  $-0.3$ , còn `Center == A` được coi là baseline. Vì vậy risk rằng call được answered tại Center B bằng 0.74 lần risk tại Center A; nói cách khác, wait times tại Center B dài hơn một chút.

Recall từ Section 2.3.7 việc dùng `lambda` để tạo các short functions on the fly. Ta dùng function `sim_time()` từ package `ISLP.survival`. Function này dùng relationship giữa survival function và cumulative hazard  $S(t) = \exp(-H(t))$ , cùng specific form của cumulative hazard trong Cox model, để simulate data từ values của linear predictor `true_linpred` và cumulative hazard. Ta cần cung cấp cumulative hazard function như sau.

```
In [25]: cum_hazard = lambda t: 1e-5 * t**2 / 2
```

Bây giờ ta có thể generate data dưới Cox proportional hazards model. Ta truncate maximum time tại 1000 seconds để simulated wait times ở mức hợp lý. Function `sim_time()` nhận linear predictor, cumulative hazard function và random number generator.

```
In [26]: W = np.array([sim_time(1, cum_hazard, rng)
 for l in true_linpred])
D['Wait time'] = np.clip(W, 0, 1000)
```

Tiếp theo simulate censoring variable, với assumption rằng 90% calls được answered (`Failed==1`) trước khi customer gác máy (`Failed==0`).

```
In [27]: D['Failed'] = rng.choice([1, 0],
 N,
 p=[0.9, 0.1])
D[:5]
```

```
Out[27]:
```

|   | Operators | Center | Time   | Wait time  | Failed |
|---|-----------|--------|--------|------------|--------|
| 0 | 13        | C      | After. | 525.064979 | 1      |
| 1 | 15        | A      | Even.  | 254.677835 | 1      |
| 2 | 7         | B      | Morn.  | 487.739224 | 1      |
| 3 | 7         | C      | Morn.  | 308.580292 | 1      |
| 4 | 13        | C      | Even.  | 154.174608 | 1      |

```
In [28]: D['Failed'].mean()
```

```
Out[28]: 0.8985
```

Bây giờ ta plot Kaplan–Meier survival curves. Trước hết stratify theo `Center`.

```
In [29]: fig, ax = subplots(figsize=(8,8))
by_center = {}
for center, df in D.groupby('Center'):
 by_center[center] = df
 km_center = km.fit(df['Wait time'], df['Failed'])
 km_center.plot(label='Center=%s' % center, ax=ax)
ax.set_title("Probability of Still Being on Hold")
```

Tiếp theo stratify theo `Time`.

```
In [30]: fig, ax = subplots(figsize=(8,8))
by_time = {}
for time, df in D.groupby('Time'):
 by_time[time] = df
 km_time = km.fit(df['Wait time'], df['Failed'])
 km_time.plot(label='Time=%s' % time, ax=ax)
ax.set_title("Probability of Still Being on Hold")
```

Có vẻ calls tại Call Center B mất nhiều time hơn mới được answered so với Centers A và C. Tương tự, wait times dường như dài nhất vào morning và ngắn nhất vào evening. Ta dùng log-rank test để xác định các differences này có statistically significant hay không, thông qua function `multivariate_logrank_test()`.

```
In [31]: multivariate_logrank_test(D['Wait time'],
 D['Center'],
 D['Failed'])
```

```
Out[31]:
```

|                    |       |                           |
|--------------------|-------|---------------------------|
|                    | t_0   | -1                        |
| null_distribution  |       | chi squared               |
| degrees_of_freedom |       | 2                         |
| test_name          |       | multivariate_logrank_test |
| test_statistic     | p     | -log2(p)                  |
|                    | 20.30 | <0.005                    |
|                    |       | 14.65                     |

Tiếp theo, xét effect của Time.

```
In [32]: multivariate_logrank_test(D['Wait time'],
 D['Time'],
 D['Failed'])
```

```
Out[32]:
```

|                    |       |                           |
|--------------------|-------|---------------------------|
|                    | t_0   | -1                        |
| null_distribution  |       | chi squared               |
| degrees_of_freedom |       | 2                         |
| test_name          |       | multivariate_logrank_test |
| test_statistic     | p     | -log2(p)                  |
|                    | 49.90 | <0.005                    |
|                    |       | 35.99                     |

Giống trường hợp categorical variable có 2 levels, những results này gần với likelihood-ratio test từ Cox proportional hazards model. Trước tiên xem results cho **Center**.

```
In [33]: X = MS(['Wait time',
 'Failed',
 'Center'],
 intercept=False).fit_transform(D)
F = coxph().fit(X, 'Wait time', 'Failed')
F.log_likelihood_ratio_test()
```

```
Out[33]:
```

|                 |       |                           |
|-----------------|-------|---------------------------|
|                 |       | chi squared               |
| degrees_freedom |       | 2                         |
| test_name       |       | log-likelihood ratio test |
| test_statistic  | p     | -log2(p)                  |
|                 | 20.58 | <0.005                    |
|                 |       | 14.85                     |

Tiếp theo xem results cho Time.

```
In [34]: X = MS(['Wait time',
 'Failed',
 'Time'],
 intercept=False).fit_transform(D)
F = coxph().fit(X, 'Wait time', 'Failed')
F.log_likelihood_ratio_test()
```

```
Out[34]: null_distribution chi squared
degrees_freedom 2
test_name log-likelihood ratio test

test_statistic p -log2(p)
 48.12 <0.005 34.71
```

Ta thấy differences giữa các centers highly significant, và differences giữa các times of day cũng vậy.

Cuối cùng, ta fit Cox's proportional hazards model cho toàn bộ data.

```
In [35]: X = MS(D.columns,
 intercept=False).fit_transform(D)
fit_queuing = coxph().fit(
 X,
 'Wait time',
 'Failed')
fit_queuing.summary[['coef', 'se(coef)', 'p']]
```

```
Out[35]: coef se(coef) p
covariate
Operators 0.043934 0.007520 5.143677e-09
Center[B] -0.236059 0.058113 4.864734e-05
Center[C] 0.012231 0.057518 8.316083e-01
Time[Even.] 0.268845 0.057797 3.294914e-06
Time[Morn.] -0.148215 0.057334 9.734378e-03
```

P-values của Center B và evening time đều rất nhỏ. Đồng thời, hazard—tức instantaneous risk rằng một call sẽ được answered—tăng theo number of operators. Vì data do chính ta generate, ta biết true coefficients của Operators, Center = B, Center = C, Time = Even. và Time = Morn. lần lượt là 0.04, −0.3, 0, 0.2, −0.2. Coefficient estimates từ fitted Cox model khá accurate.

## 11.9 Exercises

### Conceptual

1. Với từng ví dụ sau, hãy cho biết censoring mechanism có independent hay không. Giải thích câu trả lời.
  - (a) Trong một study về disease relapse, do một research scientist bất cẩn, tất cả patients có số điện thoại bắt đầu bằng chữ số “2” đều bị lost to follow-up.
  - (b) Trong một study về longevity, một formatting error khiến mọi patient age vượt quá 99 years bị mất (tức ta biết các patients đó lớn hơn 99 tuổi nhưng không biết exact age).
  - (c) Hospital A tiến hành một study về longevity. Tuy nhiên, các very sick patients có xu hướng được chuyển sang Hospital B và do đó bị lost to follow-up.
  - (d) Trong một study về unemployment duration, những người tìm được việc sớm ít có động lực giữ liên lạc với study investigators hơn, và vì vậy có probability bị lost to follow-up cao hơn.
  - (e) Trong một study về pregnancy duration, những women sinh pre-term có xu hướng sinh ở ngoài bệnh viện thông thường của họ và vì vậy có probability bị censored cao hơn so với women sinh full-term.
  - (f) Một researcher muốn model number of years of education của residents trong một small town. Residents theo học college ngoài thị trấn có probability bị lost to follow-up cao hơn, đồng thời cũng có probability học graduate school cao hơn, so với những người học college trong thị trấn.
  - (g) Researchers tiến hành một study về disease-free survival (tức time until disease relapse sau treatment). Patients không relapse trong vòng 5 years được xem là cured, và vì vậy survival time của họ bị censored tại 5 years.
  - (h) Ta muốn model failure time của một electrical component. Component này có thể được sản xuất tại Iowa hoặc Pittsburgh, không có difference về quality. Iowa factory mở 5 years trước nên components sản xuất tại Iowa bị censored tại 5 years. Pittsburgh factory mở 2 years trước nên components ở đó bị censored tại 2 years.
  - (i) Ta muốn model failure time của một electrical component được sản xuất tại two different factories, trong đó một factory mở sớm hơn factory kia. Có lý do để tin rằng components sản xuất tại factory mở sớm hơn có quality cao hơn.
2. Ta tiến hành một study với  $n = 4$  participants vừa mua cell phones, nhằm model time until phone replacement. Participant thứ nhất thay phone sau 1.2 years. Participant thứ hai vẫn chưa thay phone khi study period 2 years kết thúc. Participant thứ ba đổi phone number và bị lost to follow-up (nhưng vẫn chưa thay phone) sau 1.5 years. Participant



thứ tư thay phone sau 0.2 years. Với mỗi participant ( $i = 1, \dots, 4$ ), trả lời các câu hỏi sau bằng notation đã giới thiệu ở Section 11.1:

- (a) Cell phone replacement time của participant có bị censored không?
  - (b) Giá trị  $c_i$  có known không, và nếu có thì bằng bao nhiêu?
  - (c) Giá trị  $t_i$  có known không, và nếu có thì bằng bao nhiêu?
  - (d) Giá trị  $y_i$  có known không, và nếu có thì bằng bao nhiêu?
  - (e) Giá trị  $\delta_i$  có known không, và nếu có thì bằng bao nhiêu?
3. Với example trong Exercise 2, hãy report các giá trị

$$K, \quad d_1, \dots, d_K, \quad r_1, \dots, r_K, \quad q_1, \dots, q_K,$$

trong đó notation này được định nghĩa ở Section 11.3.

4. Problem này sử dụng Kaplan–Meier survival curve hiển thị trong Figure 11.9. Raw data dùng để plot survival curve này được cho trong Table 11.4. Column covariate của table không cần thiết cho problem này.
- (a) Estimated probability of survival past 50 days là bao nhiêu?

| Observation ( $Y$ ) | Censoring Indicator ( $\delta$ ) | Covariate ( $X$ ) |
|---------------------|----------------------------------|-------------------|
| 26.5                | 1                                | 0.1               |
| 37.2                | 1                                | 11                |
| 57.3                | 1                                | -0.3              |
| 90.8                | 0                                | 2.8               |
| 20.2                | 0                                | 1.8               |
| 89.8                | 0                                | 0.4               |

**BẢNG 11.4.** Data được dùng trong Exercise 4.

- (b) Viết một analytical expression cho estimated survival function. Chẳng hạn, answer có thể có dạng

$$\widehat{S}(t) = \begin{cases} 0.8, & t < 31, \\ 0.5, & 31 \leq t < 77, \\ 0.22, & 77 \leq t. \end{cases}$$

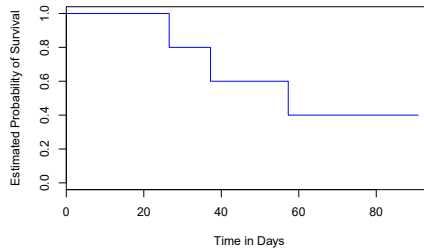
(Equation trên chỉ để minh họa; đó không phải correct answer.)

5. Phác thảo survival function cho bởi equation

$$\widehat{S}(t) = \begin{cases} 0.8, & t < 31, \\ 0.5, & 31 \leq t < 77, \\ 0.22, & 77 \leq t. \end{cases}$$

Answer nên trông tương tự Figure 11.9.

6. Problem này sử dụng data hiển thị trong Figure 11.1. Khi làm problem này, có thể ký hiệu observation times là  $y_1, \dots, y_4$ . Thứ tự của các observation times có thể đọc từ Figure 11.1; không cần exact values.



**HÌNH 11.9.** Một Kaplan–Meier survival curve được dùng trong Exercise 4.

- (a) Report các giá trị  $\delta_1, \dots, \delta_4, K, d_1, \dots, d_K, r_1, \dots, r_K$ , và  $q_1, \dots, q_K$ . Relevant notation được định nghĩa ở Sections 11.1 và 11.3.
  - (b) Phác thảo Kaplan–Meier survival curve tương ứng với dataset này. (Không cần dùng software; có thể sketch bằng tay dựa trên kết quả ở (a).)
  - (c) Dựa trên survival curve estimated ở (b), probability event xảy ra trong vòng 200 days là bao nhiêu? Probability event không xảy ra trong vòng 310 days là bao nhiêu?
  - (d) Viết một expression cho estimated survival curve từ (b).
7. Trong problem này, ta sẽ derive (11.5) và (11.6), cần thiết để construct log-rank test statistic (11.8). Recall notation trong Table 11.1.
- (a) Giả sử không có difference giữa survival functions của hai groups. Khi đó có thể xem  $q_{1k}$  là number of failures khi draw  $r_{1k}$  observations, without replacement, từ một risk set gồm  $r_k$  observations chứa tổng cộng  $q_k$  failures. Hãy lập luận rằng  $q_{1k}$  follows a hypergeometric distribution. Viết parameters của distribution này theo  $r_{1k}, r_k$ , và  $q_k$ .
  - (b) Dựa trên answer trước và properties của hypergeometric distribution, mean và variance của  $q_{1k}$  là gì? So sánh answer với (11.5) và (11.6).
8. Recall rằng survival function  $S(t)$ , hazard function  $h(t)$ , và density function  $f(t)$  được định nghĩa lần lượt trong (11.2), (11.9), và (11.11). Đồng thời define  $F(t) = 1 - S(t)$ . Chứng minh các relationships sau:

$$f(t) = \frac{dF(t)}{dt}, \quad S(t) = \exp\left(-\int_0^t h(u) du\right).$$

9. Trong exercise này, ta explore consequences của assumption rằng survival times follow an exponential distribution.
- (a) Giả sử survival time follows an  $\text{Exp}(\lambda)$  distribution, sao cho density function là  $f(t) = \lambda \exp(-\lambda t)$ . Dùng các relationships trong Exercise 8 để chứng minh  $S(t) = \exp(-\lambda t)$ .
  - (b) Bây giờ giả sử mỗi trong  $n$  independent survival times follows an  $\text{Exp}(\lambda)$  distribution. Viết expression cho likelihood function (11.13).

- (c) Chứng minh maximum likelihood estimator của  $\lambda$  là

$$\hat{\lambda} = \frac{\sum_{i=1}^n \delta_i}{\sum_{i=1}^n y_i}.$$

- (d) Dùng answer ở (c) để derive một estimator của mean survival time.

*Hint:* Với (d), recall rằng mean của một  $\text{Exp}(\lambda)$  random variable là  $1/\lambda$ .

### Applied

10. Exercise này tập trung vào brain tumor data có trong ISLP library.
  - (a) Plot Kaplan–Meier survival curve với  $\pm 1$  standard error bands, dùng estimator `KaplanMeierFitter()` trong package `lifelines`.
  - (b) Draw một bootstrap sample có size  $n = 88$  từ các pairs  $(y_i, \delta_i)$  và compute Kaplan–Meier survival curve tương ứng. Repeat process này  $B = 200$  times. Dùng results để estimate standard error của Kaplan–Meier survival curve tại mỗi timepoint. So sánh với standard errors ở (a).
  - (c) Fit Cox proportional hazards model dùng tất cả predictors để predict survival. Tóm tắt main findings.
  - (d) Stratify data theo value của `ki`. (Vì chỉ có one observation có `ki==40`, có thể group observation đó với observations có `ki==60`.) Plot Kaplan–Meier survival curves cho mỗi trong five strata, adjusted for other predictors.
11. Exercise này sử dụng data trong Table 11.4.
  - (a) Tạo two groups of observations. Trong Group 1,  $X < 2$ , còn trong Group 2,  $X \geq 2$ . Plot Kaplan–Meier survival curves tương ứng với hai groups. Hãy label curves rõ ràng để biết curve nào thuộc group nào. Bằng mắt, có vẻ tồn tại difference giữa survival curves của hai groups hay không?
  - (b) Fit Cox’s proportional hazards model, dùng group indicator làm covariate. Estimated coefficient là bao nhiêu? Viết một câu giải thích coefficient này theo hazard hoặc instantaneous probability of the event. Có evidence rằng true coefficient value khác zero hay không?
  - (c) Recall từ Section 11.5.2 rằng với single binary covariate, log-rank test statistic phải identical với score statistic của Cox model. Thực hiện log-rank test để xác định có difference giữa survival curves của hai groups hay không. p-value của log-rank test statistic so với p-value của score statistic cho Cox model ở (b) như thế nào?

# 12

## Unsupervised Learning

Phần lớn cuốn sách này tập trung vào các supervised learning methods như regression và classification. Trong supervised learning setting, ta thường có một tập gồm  $p$  features  $X_1, X_2, \dots, X_p$  được đo trên  $n$  observations, cùng một response  $Y$  cũng được đo trên chính  $n$  observations đó. Mục tiêu khi ấy là predict  $Y$  bằng  $X_1, X_2, \dots, X_p$ .

Chapter này thay vào đó tập trung vào *unsupervised learning*, một tập các statistical tools dành cho setting trong đó ta chỉ có features  $X_1, X_2, \dots, X_p$  được đo trên  $n$  observations. Ta không quan tâm đến prediction vì không có associated response variable  $Y$ . Thay vào đó, mục tiêu là khám phá những điều thú vị trong measurements của  $X_1, X_2, \dots, X_p$ . Có cách informative nào để visualize data không? Có thể discover subgroups giữa các variables hoặc giữa các observations hay không? Unsupervised learning chỉ một tập đa dạng các techniques nhằm trả lời những câu hỏi như vậy. Trong chapter này, ta tập trung vào hai loại unsupervised learning cụ thể: *principal components analysis*, một tool dùng cho data visualization hoặc data pre-processing trước khi áp dụng supervised techniques, và *clustering*, một broad class of methods để discover các unknown subgroups trong data.

### 12.1 The Challenge of Unsupervised Learning

Supervised learning là một area đã được hiểu khá rõ. Nếu đã đọc các chapters trước, đến đây bạn hẳn đã nắm được khá vững supervised learning. Chẳng hạn, nếu được yêu cầu predict một binary outcome từ data set, bạn có một bộ tools phát triển rất tốt để sử dụng, như logistic regression, linear discriminant analysis, classification trees, support vector machines, và nhiều methods khác; đồng thời bạn cũng hiểu khá rõ cách assess quality của results bằng cross-validation, validation trên independent test set, v.v.

Ngược lại, unsupervised learning thường khó hơn đáng kể. Bài toán có xu hướng subjective hơn và không có một goal đơn giản cho analysis như prediction của response. Unsupervised learning thường được thực hiện như một phần của *exploratory data analysis*. Hơn nữa, results từ unsupervised learning methods có thể khó assess vì không có một universally accepted mechanism để thực hiện cross-validation hoặc validate results trên independent data set. Lý do rất đơn giản. Nếu fit một predictive model bằng supervised learning technique, ta có thể check work bằng cách xem model predict response  $Y$  tốt đến đâu trên observations không dùng để fit model.

Nhưng trong unsupervised learning, không có cách tương tự để check vì ta không biết true answer — problem là unsupervised.

*exploratory data  
analysis*

Các unsupervised learning techniques ngày càng quan trọng trong nhiều fields. Một cancer researcher có thể assay gene-expression levels ở 100 patients bị breast cancer rồi tìm subgroups giữa các breast-cancer samples hoặc giữa các genes để hiểu disease tốt hơn. Một online shopping site có thể identify groups of shoppers có browsing và purchase histories tương tự, cũng như các items đặc biệt hấp dẫn đối với shoppers trong từng group. Khi đó, một individual shopper có thể được ưu tiên hiển thị những items mà người đó đặc biệt có khả năng quan tâm, dựa trên purchase histories của similar shoppers. Một search engine có thể chọn search results để hiển thị cho một individual dựa trên click histories của những người khác có search patterns tương tự. Các statistical learning tasks này, cùng nhiều tasks khác, đều có thể thực hiện bằng unsupervised learning techniques.

## 12.2 Principal Components Analysis

Principal components đã được thảo luận ở Section 6.3.1 trong context của principal components regression. Khi đối mặt với một large set of correlated variables, principal components cho phép summarize set này bằng một số nhỏ hơn các representative variables nhưng collectively explain phần lớn variability trong original set. Principal component directions được trình bày ở Section 6.3.1 như những directions trong feature space dọc theo đó original data có variability lớn. Các directions này cũng define những lines và subspaces gần data cloud nhất có thể. Để thực hiện principal components regression, ta chỉ cần dùng principal components làm predictors trong regression model thay cho original larger set of variables.

*Principal components analysis* (PCA) là process dùng để compute principal components và sau đó sử dụng các components này để understand data. PCA là một unsupervised approach vì chỉ involves một set of features  $X_1, X_2, \dots, X_p$  mà không có associated response  $Y$ . Ngoài việc tạo derived variables dùng trong supervised learning problems, PCA còn là một tool cho data visualization, cả visualization của observations lẫn visualization của variables. PCA cũng có thể dùng cho data imputation, tức fill missing values trong một data matrix.

*principal  
components  
analysis*

Ta sẽ xem PCA chi tiết hơn, tập trung vào việc dùng PCA như một tool cho unsupervised data exploration phù hợp với chủ đề của chapter này.

### 12.2.1 What Are Principal Components?

Giả sử ta muốn visualize  $n$  observations với measurements trên  $p$  features  $X_1, X_2, \dots, X_p$  như một phần của exploratory data analysis. Một cách là examine các two-dimensional scatterplots, mỗi plot chứa measurements của  $n$  observations trên hai features. Tuy nhiên, có tới  $\binom{p}{2} = p(p-1)/2$  scatterplots như vậy; chẳng hạn với  $p = 10$  đã có 45 plots. Nếu  $p$  lớn, rõ ràng không thể xem hết; hơn nữa, nhiều khả năng chẳng plot nào đặc biệt informative vì mỗi plot chỉ chứa một phần nhỏ total information trong data set. Ta cần một method tốt hơn để visualize  $n$  observations khi  $p$  lớn. Cụ thể, ta muốn tìm một low-dimensional representation của data giữ lại nhiều information nhất có thể. Ví dụ, nếu thu được một two-dimensional

representation giữ lại phần lớn information, ta có thể plot observations trong low-dimensional space đó.

PCA cung cấp đúng tool như vậy. Nó tìm một low-dimensional representation của data set chứa càng nhiều variation càng tốt. Ý tưởng là mỗi observation sống trong  $p$ -dimensional space, nhưng không phải mọi dimension đều equally interesting. PCA tìm một số nhỏ dimensions “interesting” nhất, trong đó mức interesting được đo bằng amount mà observations vary dọc theo mỗi dimension. Mỗi dimension mà PCA tìm ra là một linear combination của  $p$  features. Sau đây ta mô tả cách các dimensions, hay principal components, được tìm.

First principal component của features  $X_1, X_2, \dots, X_p$  là normalized linear combination

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p \quad (12.1)$$

có variance lớn nhất. “Normalized” nghĩa là  $\sum_{j=1}^p \phi_{j1}^2 = 1$ . Các elements  $\phi_{11}, \dots, \phi_{p1}$  được gọi là *loadings* của first principal component; cùng nhau chúng tạo thành principal component loading vector  $\phi_1 = (\phi_{11}, \phi_{21}, \dots, \phi_{p1})^T$ . Ta constrain loadings sao cho sum of squares bằng one, bởi nếu không, chỉ cần làm absolute values của các elements lớn tùy ý là có thể tạo variance lớn tùy ý.

loading

Với một  $n \times p$  data set  $X$ , làm thế nào compute first principal component? Vì chỉ quan tâm đến variance, giả sử mỗi variable trong  $X$  đã được centered để có mean zero, tức column means của  $X$  bằng zero. Ta tìm linear combination của sample feature values có dạng

$$z_{i1} = \phi_{11}x_{i1} + \phi_{21}x_{i2} + \dots + \phi_{p1}x_{ip} \quad (12.2)$$

có sample variance lớn nhất, subject to constraint  $\sum_{j=1}^p \phi_{j1}^2 = 1$ . Nói cách khác, first principal component loading vector solve optimization problem

$$\underset{\phi_{11}, \dots, \phi_{p1}}{\text{maximize}} \left\{ \frac{1}{n} \sum_{i=1}^n \left( \sum_{j=1}^p \phi_{j1}x_{ij} \right)^2 \right\} \quad \text{subject to} \quad \sum_{j=1}^p \phi_{j1}^2 = 1. \quad (12.3)$$

Từ (12.2), objective trong (12.3) có thể viết là  $n^{-1} \sum_{i=1}^n z_{i1}^2$ . Vì  $n^{-1} \sum_i x_{ij} = 0$ , average của  $z_{11}, \dots, z_{n1}$  cũng bằng zero. Do đó objective đang maximize chính là sample variance của  $n$  values  $z_{i1}$ . Ta gọi  $z_{11}, \dots, z_{n1}$  là *scores* của first principal component. Problem (12.3) có thể được solve bằng *eigen decomposition*, một standard technique trong linear algebra, nhưng details nằm ngoài scope của sách.<sup>1</sup>

score; eigen  
decomposition

Có một geometric interpretation đẹp cho first principal component. Loading vector  $\phi_1$  với elements  $\phi_{11}, \phi_{21}, \dots, \phi_{p1}$  define một direction trong feature space mà dọc theo đó data vary nhiều nhất. Nếu project  $n$  data points  $x_1, \dots, x_n$  lên direction này, projected values chính là principal component scores  $z_{11}, \dots, z_{n1}$ . Chẳng hạn Figure 6.14 ở page 254 hiển thị first principal component loading vector trên advertising data set. Vì data đó chỉ có two features, observations và loading vector đều có thể display dễ dàng. Từ (6.19),  $\phi_{11} = 0.839$  và  $\phi_{21} = 0.544$ .

<sup>1</sup>Một alternative cho eigen decomposition là một related technique gọi là singular value decomposition. Technique này sẽ được explore trong lab ở cuối chapter.

Sau khi first principal component  $Z_1$  được xác định, ta có thể tìm second principal component  $Z_2$ . Second principal component là linear combination của  $X_1, \dots, X_p$  có maximal variance trong tất cả linear combinations uncorrelated với  $Z_1$ . Second principal component scores  $z_{12}, z_{22}, \dots, z_{n2}$  có dạng

$$z_{i2} = \phi_{12}x_{i1} + \phi_{22}x_{i2} + \dots + \phi_{p2}x_{ip}, \quad (12.4)$$

trong đó  $\phi_2$  là second principal component loading vector. Constraining  $Z_2$  uncorrelated với  $Z_1$  tương đương constraining direction  $\phi_2$  orthogonal với direction  $\phi_1$ . Trong Figure 6.14, observations nằm trong two-dimensional space nên sau khi tìm  $\phi_1$  chỉ còn một possibility cho  $\phi_2$ , hiển thị bằng blue dashed line. Từ Section 6.3.1,  $\phi_{12} = 0.544$  và  $\phi_{22} = -0.839$ . Với larger data set có  $p > 2$ , có multiple distinct principal components được define tương tự. Để tìm  $\phi_2$ , ta solve problem giống (12.3) với  $\phi_2$  thay  $\phi_1$ , đồng thời thêm constraint  $\phi_2 \perp \phi_1$ .<sup>2</sup>

Sau khi compute principal components, ta có thể plot chúng against each other để tạo low-dimensional views của data. Chẳng hạn có thể plot score vector  $Z_1$  against  $Z_2$ ,  $Z_1$  against  $Z_3$ ,  $Z_2$  against  $Z_3$ , v.v. Geometrically, điều này tương đương project original data xuống subspace spanned by  $\phi_1, \phi_2, \phi_3$  rồi plot projected points.

Ta minh họa PCA trên **USArrests** data set. Với mỗi trong 50 states của United States, data set chứa number of arrests per 100,000 residents cho ba crimes: **Assault**, **Murder**, và **Rape**. Ta cũng record **UrbanPop**, percent population của mỗi state sống trong urban areas. Principal component score vectors có length  $n = 50$ , còn loading vectors có length  $p = 4$ . PCA được thực hiện sau khi standardize mỗi variable để có mean zero và standard deviation one.

|          | PC1       | PC2        |
|----------|-----------|------------|
| Murder   | 0.5358995 | -0.4181809 |
| Assault  | 0.5831836 | -0.1879856 |
| UrbanPop | 0.2781909 | 0.8728062  |
| Rape     | 0.5434321 | 0.1673186  |

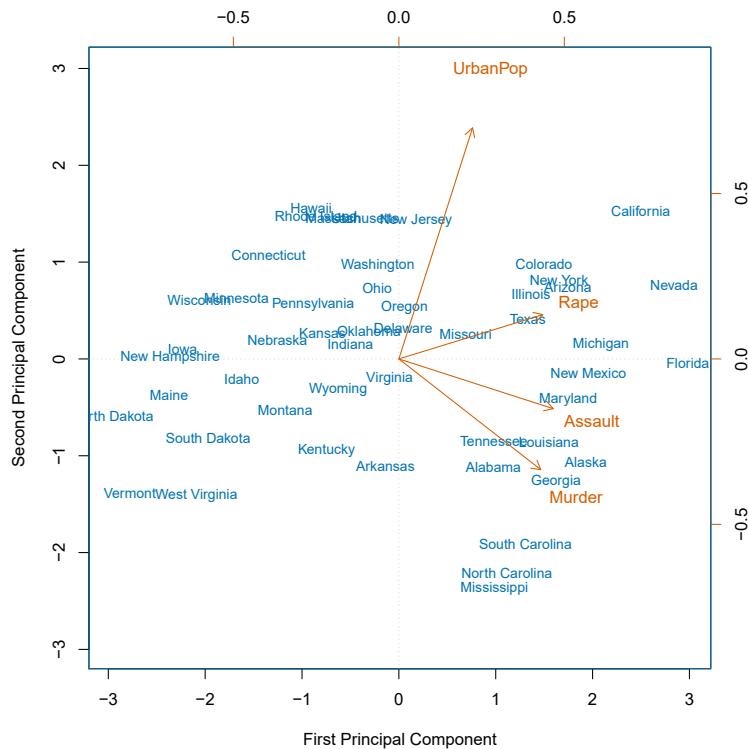
**BẢNG 12.1.** Principal component loading vectors  $\phi_1$  và  $\phi_2$  cho **USArrests** data. Chúng cũng được display trong Figure 12.1.

Figure 12.1 plot first two principal components của data này. Figure biểu diễn cả principal component scores và loading vectors trong một *biplot*. Loadings cũng được cho trong Table 12.1.

*biplot*

Trong Figure 12.1, first loading vector đặt approximately equal weight lên **Assault**, **Murder**, và **Rape**, nhưng much less weight lên **UrbanPop**. Vì vậy component này roughly tương ứng với một measure của overall rates of serious crimes. Second loading vector đặt phần lớn weight lên **UrbanPop** và ít hơn nhiều lên ba features còn lại, nên roughly tương ứng với level of urbanization của state. Nhìn chung, crime-related variables (**Murder**, **Assault**, và **Rape**) nằm gần nhau, còn **UrbanPop** nằm xa ba variables kia. Điều này cho thấy crime-related variables correlated với nhau — states có

<sup>2</sup>Về mặt technical, principal component directions  $\phi_1, \phi_2, \phi_3, \dots$  là ordered sequence of eigenvectors của matrix  $X^T X$ , còn variances của components là eigenvalues. Có nhiều nhất  $\min(n-1, p)$  principal components.



**HÌNH 12.1.** Hai principal components đầu tiên của *USArrests* data. Blue state names biểu diễn scores của first two principal components. Orange arrows biểu diễn first two principal component loading vectors, với axes ở top và right. Chẳng hạn loading của *Rape* trên first component là 0.54 và loading trên second component là 0.17; từ *Rape* được centered tại point (0.54, 0.17). Figure này được gọi là một biplot vì hiển thị cả principal component scores lẫn principal component loadings.

high murder rates có xu hướng cũng có high assault và rape rates — còn *UrbanPop* less correlated với ba variables kia.

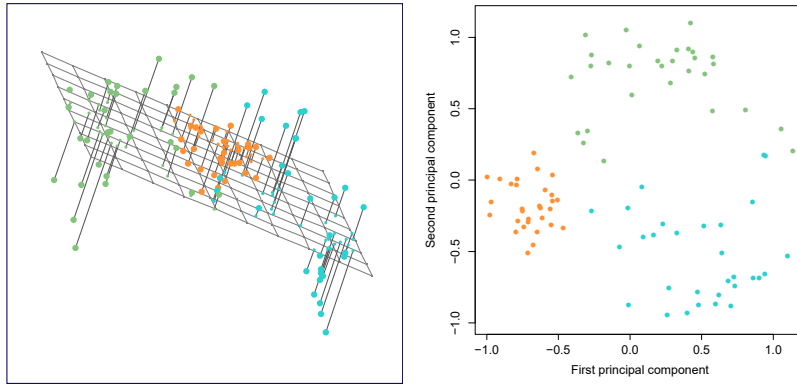
Ta cũng có thể examine differences giữa states qua two principal component score vectors trong Figure 12.1. Discussion về loadings gợi ý rằng states có large positive scores trên first component, như California, Nevada và Florida, có high crime rates, trong khi states như North Dakota với negative score có low crime rates. California cũng có high score trên second component, cho thấy high level of urbanization, trong khi Mississippi thì ngược lại. States gần zero trên cả two components, như Indiana, có levels gần average cho cả crime lẫn urbanization.

### 12.2.2 Another Interpretation of Principal Components

First two principal component loading vectors trong một simulated three-dimensional data set được hiển thị ở left-hand panel Figure 12.2; hai loading vectors này span một plane dọc theo đó observations có highest variance.



Ở section trước, ta mô tả principal component loading vectors như directions trong feature space mà data vary nhiều nhất, và principal component scores như projections dọc theo các directions này. Tuy nhiên còn một interpretation khác cũng hữu ích: principal components cung cấp các low-dimensional linear surfaces gần observations nhất. Ta mở rộng interpretation này ở đây.<sup>3</sup>



**HÌNH 12.2.** Ninety observations simulated trong three dimensions. Observations được display bằng color để dễ visualization. Left: first two principal component directions span plane fit data tốt nhất; plane được positioned để minimize sum of squared distances tới từng point. Right: first two principal component score vectors cho coordinates của projection của 90 observations lên plane.

First principal component loading vector có một property đặc biệt: đó là line trong  $p$ -dimensional space gần  $n$  observations nhất, dùng average squared Euclidean distance làm measure of closeness. Interpretation này có thể thấy trong left-hand panel của Figure 6.15; dashed lines chỉ distances giữa từng observation và line define bởi first principal component loading vector. Appeal của interpretation này rất rõ: ta tìm một single dimension của data nằm càng gần tất cả data points càng tốt, vì một line như vậy có khả năng summarize data tốt.

Ý tưởng principal components là những dimensions gần  $n$  observations nhất không chỉ áp dụng cho first component. Chẳng hạn first two principal components span plane gần  $n$  observations nhất theo average squared Euclidean distance, như left-hand panel Figure 12.2. First three principal components span three-dimensional hyperplane gần observations nhất, và cứ tiếp tục như vậy.

Dùng interpretation này, cùng nhau first  $M$  principal component score vectors và first  $M$  loading vectors tạo best  $M$ -dimensional approximation, theo Euclidean distance, cho observation  $x_{ij}$ :

$$x_{ij} \approx \sum_{m=1}^M z_{im} \phi_{jm}. \quad (12.5)$$

Ta có thể phát biểu hơn bằng một optimization problem. Giả

<sup>3</sup>Trong section này, ta tiếp tục assume rằng mỗi column của data matrix  $X$  đã centered để có mean zero, tức column mean đã được subtract khỏi mỗi column.

sử data matrix  $X$  đã column-centered. Trong tất cả approximations dạng  $x_{ij} \approx \sum_{m=1}^M a_{im} b_{jm}$ , ta tìm approximation có smallest residual sum of squares:

$$\underset{A \in \mathbb{R}^{n \times M}, B \in \mathbb{R}^{p \times M}}{\text{minimize}} \left\{ \sum_{j=1}^p \sum_{i=1}^n \left( x_{ij} - \sum_{m=1}^M a_{im} b_{jm} \right)^2 \right\}. \quad (12.6)$$

Ở đây  $A$  là  $n \times M$  matrix với element  $(i, m)$  là  $a_{im}$ , còn  $B$  là  $p \times M$  matrix với element  $(j, m)$  là  $b_{jm}$ .

Có thể chứng minh rằng với bất kỳ  $M$ , columns của matrices  $\widehat{A}$  và  $\widehat{B}$  solve (12.6) thực ra chính là first  $M$  principal component score và loading vectors. Nói cách khác, nếu  $\widehat{A}, \widehat{B}$  solve (12.6) thì  $\hat{a}_{im} = z_{im}$  và  $\hat{b}_{jm} = \phi_{jm}$ .<sup>4</sup> Vì vậy smallest possible value của objective trong (12.6) là

$$\sum_{j=1}^p \sum_{i=1}^n \left( x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2. \quad (12.7)$$

Tóm lại,  $M$  principal component score vectors cùng  $M$  loading vectors có thể cho một good approximation to data khi  $M$  đủ lớn. Khi  $M = \min(n-1, p)$ , representation là exact:  $x_{ij} = \sum_{m=1}^M z_{im} \phi_{jm}$ .

### 12.2.3 The Proportion of Variance Explained

Trong Figure 12.2, ta thực hiện PCA trên một three-dimensional data set và project data lên first two principal component loading vectors để thu two-dimensional view của data, tức principal component score vectors. Ta thấy representation 2D này capture tốt major pattern trong 3D data: orange, green và cyan observations gần nhau trong 3D space vẫn nằm gần nhau trong 2D representation. Tương tự, với **USArrests**, ta đã summarize 50 observations và 4 variables chỉ bằng first two score vectors và first two loading vectors.

Ta có thể đặt một câu hỏi tự nhiên: bao nhiêu information trong data set bị lost khi project observations lên first few principal components? Tức bao nhiêu variance của data không contained trong first few components? Tổng quát hơn, ta quan tâm đến *proportion of variance explained* (PVE) bởi mỗi principal component.

*proportion of  
variance explained*

Total variance trong một data set, assuming variables đã centered về mean zero, được define là

$$\sum_{j=1}^p \text{Var}(X_j) = \sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2, \quad (12.8)$$

còn variance explained bởi  $m$ th principal component là

$$\frac{1}{n} \sum_{i=1}^n z_{im}^2 = \frac{1}{n} \sum_{i=1}^n \left( \sum_{j=1}^p \phi_{jm} x_{ij} \right)^2. \quad (12.9)$$

<sup>4</sup>Về mặt technical, solution của (12.6) không unique. Chính xác hơn, bất kỳ solution nào của (12.6) cũng có thể dễ dàng transform để yield principal components.

Do đó PVE của  $m$ th principal component bằng

$$\frac{\sum_{i=1}^n z_{im}^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2} = \frac{\sum_{i=1}^n \left( \sum_{j=1}^p \phi_{jm} x_{ij} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2}. \quad (12.10)$$

PVE của mỗi principal component là positive. Để compute cumulative PVE của first  $M$  components, ta đơn giản sum (12.10) qua first  $M$  PVEs. Tổng cộng có  $\min(n-1, p)$  principal components và PVEs của chúng sum to one.

Ở Section 12.2.2, ta chỉ ra rằng first  $M$  loading và score vectors có thể được interpret là best  $M$ -dimensional approximation to data theo residual sum of squares. Variance của data có thể decompose thành variance của first  $M$  principal components cộng mean squared error của  $M$ -dimensional approximation:

$$\sum_{j=1}^p \frac{1}{n} \sum_{i=1}^n x_{ij}^2 = \sum_{m=1}^M \frac{1}{n} \sum_{i=1}^n z_{im}^2 + \frac{1}{n} \sum_{j=1}^p \sum_{i=1}^n \left( x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2. \quad (12.11)$$

$\underbrace{\hspace{1.5cm}}_{\text{Var. of data}} \{ \mathbf{Z} \}$

$\underbrace{\hspace{1.5cm}}_{\text{Var. of first } M \text{ PCs}} \{ \mathbf{Z} \}$

$\underbrace{\hspace{2.5cm}}_{\text{MSE of } M\text{-dimensional approximation}} \{ \mathbf{Z} \}$

Ba terms trong decomposition này được thảo luận lần lượt ở (12.8), (12.9), và (12.7). Vì first term fixed, maximizing variance của first  $M$  principal components tương đương minimizing MSE của  $M$ -dimensional approximation, và ngược lại. Điều này giải thích vì sao principal components có thể equivalently được xem là minimize approximation error hoặc maximize variance.

Ngoài ra, từ (12.11), cumulative PVE của first  $M$  components bằng

$$1 - \frac{\sum_{j=1}^p \sum_{i=1}^n \left( x_{ij} - \sum_{m=1}^M z_{im} \phi_{jm} \right)^2}{\sum_{j=1}^p \sum_{i=1}^n x_{ij}^2} = 1 - \frac{RSS}{TSS}.$$

Ở đây TSS là total sum of squared elements của  $X$ , còn RSS là residual sum of squares của  $M$ -dimensional approximation. Recall definition của  $R^2$  ở (3.17), điều này nghĩa là PVE có thể được interpret như  $R^2$  của approximation cho  $X$  bằng first  $M$  principal components.

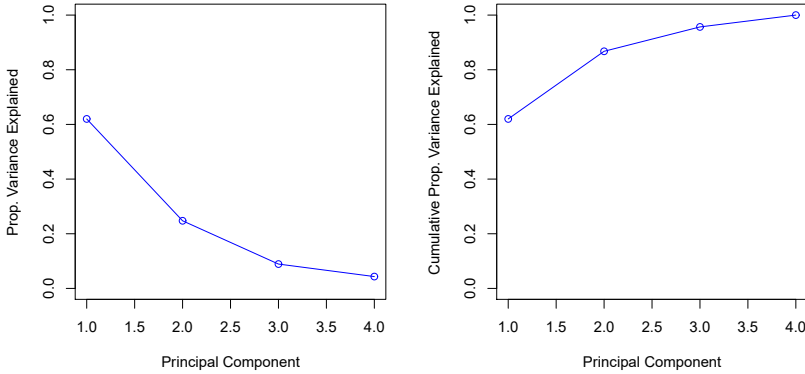
Trong **USArrests** data, first principal component explain 62.0% variance, và next principal component explain 24.7%. Together, first two principal components explain gần 87% variance, còn last two chỉ explain 13%. Điều này nghĩa là Figure 12.1 cung cấp một summary khá accurate của data chỉ bằng two dimensions. PVE của từng component và cumulative PVE được display trong Figure 12.3. Left-hand panel được gọi là *scree plot*, sẽ được thảo luận thêm sau trong chapter.

*scree plot*

### 12.2.4 More on PCA

#### Scaling the Variables

Ta đã nói rằng trước khi PCA được thực hiện, variables nên được centered để có mean zero. Hơn nữa, results của PCA còn phụ thuộc vào việc



**HÌNH 12.3.** Left: scree plot biểu diễn proportion of variance explained bởi từng trong four principal components của *USArrests* data. Right: cumulative proportion of variance explained bởi four principal components.

variables có được individually scaled hay không, tức mỗi variable được multiply bởi một different constant. Điều này khác với một số supervised và unsupervised learning techniques khác, như linear regression, nơi scaling variables không ảnh hưởng substantively đến model; multiply một variable bởi  $c$  chỉ làm corresponding coefficient estimate multiply bởi  $1/c$ .

Figure 12.1 được tạo sau khi scale mỗi variable để có standard deviation one, và được reproduce ở left-hand plot Figure 12.4. Tại sao scaling quan trọng? Trong data này, variables measured in different units: **Murder**, **Rape**, và **Assault** là number of occurrences per 100,000 people, còn **UrbanPop** là percentage dân số sống ở urban area. Four variables có variances lần lượt 18.97, 87.73, 6945.16, và 209.5. Vì vậy nếu thực hiện PCA trên unscaled variables, first loading vector sẽ có loading rất lớn cho **Assault**, do variable này có variance cao nhất.

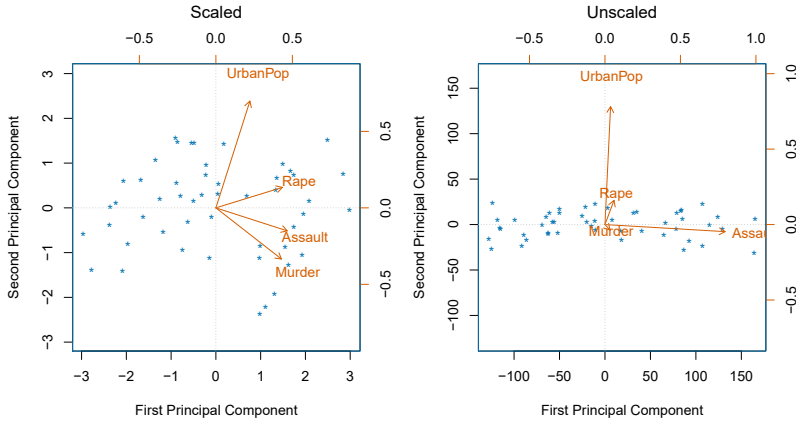
Right-hand plot Figure 12.4 hiển thị first two principal components của *USArrests* mà không scale variables về standard deviation one. Đúng như dự đoán, first loading vector đặt gần như toàn bộ weight lên **Assault**, còn second loading vector đặt gần như toàn bộ weight lên **UrbanPop**. So với left-hand plot, scaling rõ ràng có substantial effect lên results.

Tuy nhiên, result này chỉ là consequence của scales mà variables được measured. Chẳng hạn nếu **Assault** measured theo occurrences per 100 people thay vì per 100,000 people, tức divide mọi element của variable cho 1000, variance của variable sẽ rất nhỏ và first loading vector sẽ có value rất nhỏ cho variable đó. Vì không desirable để principal components phụ thuộc vào arbitrary choice of scaling, ta thường scale mỗi variable để có standard deviation one trước PCA.

Trong một số settings, variables có thể measured bằng cùng units. Khi đó ta có thể không muốn scale về standard deviation one. Ví dụ nếu variables là expression levels của  $p$  genes, vì expression measured trong cùng “units” cho mỗi gene, ta có thể chọn không scale từng gene về standard deviation one.

### Uniqueness of the Principal Components

Về theory, principal components không nhất thiết unique, nhưng trong



**HÌNH 12.4.** Hai principal component biplots cho *USArrests* data. Left: giống Figure 12.1, với variables scaled để có unit standard deviations. Right: principal components dùng unscaled data. *Assault* có loading lớn nhất rất nhiều trên first principal component vì nó có variance lớn nhất trong four variables. Nhìn chung, scaling variables để có standard deviation one được recommended.

gần như mọi practical setting chúng unique up to sign flips. Hai software packages khác nhau sẽ yield cùng loading vectors, dù signs có thể khác. Signs có thể khác vì mỗi loading vector specify một direction trong  $p$ -dimensional space; flip sign không đổi direction. Tương tự, score vectors unique up to a sign flip vì variance của  $Z$  bằng variance của  $-Z$ . Khi dùng (12.5) để approximate  $x_{ij}$ , ta multiply  $z_{im}$  với  $\phi_{jm}$ ; nếu flip sign của cả loading lẫn score vectors thì final product không đổi.

### Deciding How Many Principal Components to Use

Một  $n \times p$  data matrix  $X$  nói chung có  $\min(n-1, p)$  distinct principal components. Tuy nhiên, ta thường chỉ quan tâm first few components để visualize hoặc interpret data, và muốn dùng smallest number đủ để có good understanding. Bao nhiêu principal components là cần thiết? Không có một answer duy nhất hoặc đơn giản.

Thông thường ta decide number of components cần cho visualization bằng cách examine scree plot như left-hand panel Figure 12.3. Ta chọn smallest number of components cần để explain một sizable amount of variation, thường bằng cách eyeball scree plot và tìm point tại đó PVE của subsequent components drop off. Drop này thường được gọi là một *elbow*. Với Figure 12.3, có thể conclude rằng first two components explain khá nhiều variance và có elbow sau second component. Third component explain less than 10%, còn fourth explain less than half của third nên gần như worthless.

Tuy vậy, visual analysis này inherently ad hoc. Không có một well-accepted objective way để decide bao nhiêu components là enough; câu hỏi này inherently ill-defined và phụ thuộc application cùng data set. Trong practice, ta thường xem first few components để tìm interesting patterns. Nếu không có interesting patterns trong first few components thì subsequent components nhiều khả năng cũng không interesting. Ngược lại, nếu first

few components interesting, ta tiếp tục xem các components kế tiếp cho tới khi không còn patterns mới. Đây là subjective approach, phản ánh việc PCA thường được dùng như tool cho exploratory data analysis.

Mặt khác, nếu compute principal components để dùng trong supervised analysis, như principal components regression ở Section 6.3.1, có một cách simple và objective hơn để determine number of components: xem number of score vectors dùng trong regression như một tuning parameter được selected bằng cross-validation hoặc related approach. Simplicity này là một biểu hiện của việc supervised analyses thường được define rõ và evaluated objectively hơn unsupervised analyses.

### 12.2.5 Other Uses for Principal Components

Ở Section 6.3.1 ta thấy có thể perform regression bằng principal component score vectors làm features. Thực ra nhiều statistical techniques, như regression, classification, và clustering, có thể dễ dàng adapt để dùng  $n \times M$  matrix có columns là first  $M$  principal component score vectors, thay vì full  $n \times p$  data matrix. Điều này có thể cho less noisy results vì signal, khác với noise, thường concentrated trong first few principal components.

## 12.3 Missing Values and Matrix Completion

Datasets thường có missing values, gây nhiều bất tiện. Chẳng hạn muốn analyze **USArrests** và discover rằng 20 trong 200 values đã randomly corrupted và marked as missing. Statistical learning methods đã học trong sách nhìn chung không handle missing values. Nên làm gì?

Ta có thể remove rows chứa missing observations và perform analysis trên complete rows, nhưng cách này wasteful và có thể unrealistic tùy fraction missing. Hoặc nếu  $x_{ij}$  missing, ta có thể replace bằng mean của  $j$ th column tính trên non-missing entries. Đây là common và convenient strategy, nhưng thường có thể làm tốt hơn bằng cách exploit correlation giữa variables.

Trong section này, ta cho thấy principal components có thể dùng để *impute* missing values thông qua process gọi là *matrix completion*. Completed matrix sau đó có thể dùng trong statistical learning method như linear regression hoặc LDA.

Approach này phù hợp nếu missingness là random. Ví dụ nó appropriate nếu weight của một patient missing vì battery của electronic scale hết đúng lúc exam. Ngược lại, nếu weight missing vì patient quá nặng để bước lên scale, missingness là informative và không phải *missing at random*; approach ở đây không phù hợp.

Đôi khi data missing by necessity. Ví dụ, nếu tạo matrix ratings từ 1 đến 5 mà  $n$  customers cho toàn bộ Netflix catalog gồm  $p$  movies, phần lớn matrix sẽ missing vì không customer nào đã xem và rate hơn một tiny fraction của catalog. Nếu impute tốt missing values, ta có thể có ý tưởng mỗi customer sẽ nghĩ gì về movies chưa xem. Vì vậy matrix completion có thể power *recommender systems*.

### Principal Components with Missing Values

Section 12.2.2 cho thấy first  $M$  principal component score và loading

*impute;  
imputation; matrix  
completion*

*missing at random*

*recommender  
systems*

vectors provide “best” approximation to data matrix  $X$  theo (12.6). Giả sử một số observations  $x_{ij}$  missing. Ta có thể vừa impute missing values vừa solve principal component problem bằng modified optimization problem

$$\underset{A \in \mathbb{R}^{n \times M}, B \in \mathbb{R}^{p \times M}}{\text{minimize}} \left\{ \sum_{(i,j) \in \mathcal{O}} \left( x_{ij} - \sum_{m=1}^M a_{im} b_{jm} \right)^2 \right\}, \quad (12.12)$$

trong đó  $\mathcal{O}$  là set của all observed pairs of indices  $(i, j)$ , một subset của  $n \times p$  possible pairs.

Sau khi solve problem này:

- ta có thể estimate một missing observation  $x_{ij}$  bằng  $\hat{x}_{ij} = \sum_{m=1}^M \hat{a}_{im} \hat{b}_{jm}$ , trong đó  $\hat{a}_{im}$  và  $\hat{b}_{jm}$  là elements tương ứng của matrices  $\hat{A}$  và  $\hat{B}$  solve (12.12);
- ta có thể approximately recover  $M$  principal component scores và loadings như trong complete data.

Solve (12.12) exactly là difficult, khác complete-data case vì eigen decomposition không còn apply. Nhưng simple iterative approach trong Algorithm 12.1, được demonstrate ở Section 12.5.2, thường cung cấp good solution.<sup>56</sup>

Ta minh họa Algorithm 12.1 trên **USArrests**. Có  $p = 4$  variables và  $n = 50$  observations. Trước tiên standardize data để mỗi variable có mean zero và standard deviation one. Sau đó randomly select 20 trong 50 states và với mỗi state randomly set one of four variables to missing. Vì vậy 10% elements của data matrix missing. Ta apply Algorithm 12.1 với  $M = 1$  principal component. Figure 12.5 cho thấy recovery của missing elements khá accurate.

### Algorithm 12.1 Iterative Algorithm for Matrix Completion

1. Tạo complete data matrix  $\tilde{X}$  kích thước  $n \times p$  với element  $(i, j)$  bằng

$$\tilde{x}_{ij} = \begin{cases} x_{ij}, & (i, j) \in \mathcal{O}, \\ \bar{x}_j, & (i, j) \notin \mathcal{O}, \end{cases}$$

trong đó  $\bar{x}_j$  là average của observed values cho  $j$ th variable trong incomplete data matrix  $X$ .

2. Repeat (a)–(c) cho đến khi objective (12.14) không còn decrease:

(a) Solve

$$\underset{A \in \mathbb{R}^{n \times M}, B \in \mathbb{R}^{p \times M}}{\text{minimize}} \left\{ \sum_{j=1}^p \sum_{i=1}^n \left( \tilde{x}_{ij} - \sum_{m=1}^M a_{im} b_{jm} \right)^2 \right\} \quad (12.13)$$

bằng cách compute principal components của  $\tilde{X}$ .

<sup>5</sup>Algorithm này được gọi là “Hard-Impute” trong Mazumder, Hastie, and Tibshirani (2010), “Spectral regularization algorithms for learning large incomplete matrices”, *Journal of Machine Learning Research*, 2287–2322.

<sup>6</sup>Mỗi iteration của Step 2 làm decrease objective (12.14). Tuy nhiên algorithm không guaranteed đạt global optimum của (12.12).

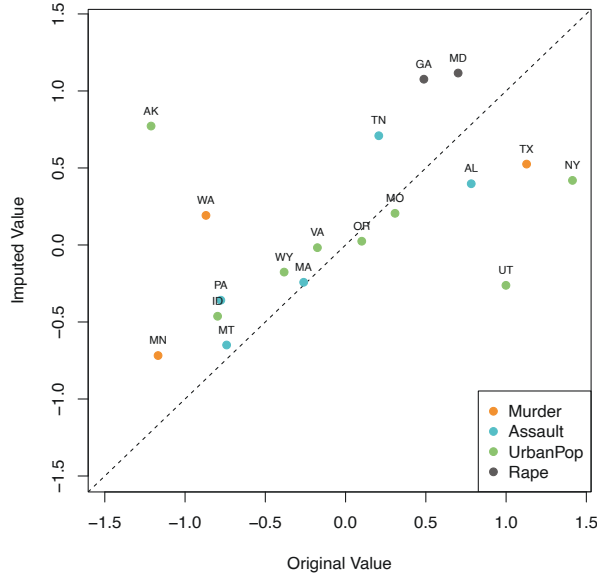
- (b) Với mỗi  $(i, j) \notin \mathcal{O}$ , set  $\tilde{x}_{ij} \leftarrow \sum_{m=1}^M \hat{a}_{im} \hat{b}_{jm}$ .
- (c) Compute objective

$$\sum_{(i,j) \in \mathcal{O}} \left( x_{ij} - \sum_{m=1}^M \hat{a}_{im} \hat{b}_{jm} \right)^2. \quad (12.14)$$

3. Return estimated missing entries  $\tilde{x}_{ij}$ ,  $(i, j) \notin \mathcal{O}$ .

Trên 100 random runs của experiment, average correlation giữa true và imputed values của missing elements là 0.63, standard deviation 0.11. Performance này tốt đến đâu? Có thể compare với unattainable benchmark dùng complete data: đơn giản compute  $\hat{x}_{ij} = z_{i1} \phi_{j1}$  từ first score và loading vectors của complete data.<sup>7</sup> Complete-data approach cho average correlation 0.79, standard deviation 0.08. Vậy imputation method kém hơn benchmark dùng all data ( $0.63 \pm 0.11$  so với  $0.79 \pm 0.08$ ), nhưng vẫn khá tốt. Và benchmark all-data dĩ nhiên không usable trong real-world setting có missing data.

Figure 12.6 cũng cho thấy Algorithm 12.1 hoạt động khá tốt trên dataset này.



**HÌNH 12.5.** Missing value imputation trên *USArrests*. Twenty values, tức 10% total matrix elements, được artificially set to missing rồi imputed bằng Algorithm 12.1 với  $M = 1$ . Figure hiển thị true value  $x_{ij}$  và imputed value  $\hat{x}_{ij}$  cho all twenty missing values. Color chỉ variable và label chỉ state. Correlation giữa true và imputed values khoảng 0.63.

Ta kết thúc phần này bằng vài observations:

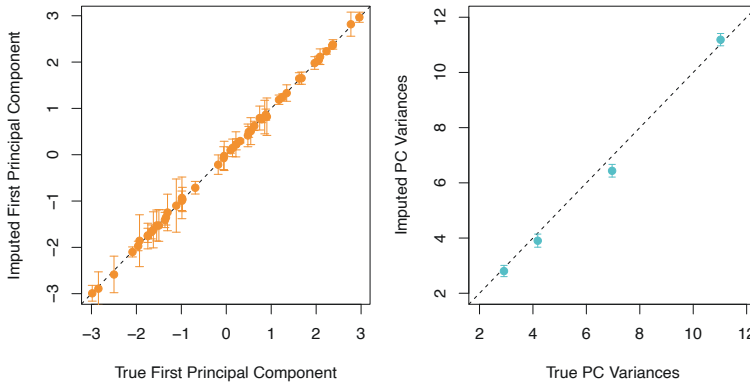
<sup>7</sup>Đây là một unattainable gold standard: khi có missing data, tất nhiên ta không thể compute principal components của complete data.



- **USArrests** chỉ có four variables, khá ít cho methods như Algorithm 12.1 hoạt động tốt. Vì vậy trong demonstration này ta randomly set nhiều nhất one variable per state to missing và chỉ dùng  $M = 1$  principal component.
- Nói chung, để apply Algorithm 12.1 ta phải select  $M$ , number of principal components dùng cho imputation. Một approach là randomly leave out thêm một vài known elements và select  $M$  dựa trên mức độ recover các known values đó. Điều này closely related với validation-set approach ở Chapter 5.

### Recommender Systems

Digital streaming services như Netflix và Amazon dùng data về content customer đã xem trước đây, cùng data từ other customers, để suggest content khác. Ví dụ cụ thể, nhiều năm trước Netflix cho customers rate mỗi movie đã xem từ 1–5. Điều này tạo một very large  $n \times p$  matrix, element  $(i, j)$  là rating customer  $i$  cho movie  $j$ .



**HÌNH 12.6.** Như mô tả trong text, trong mỗi trong 100 trials ta leave out 20 elements của **USArrests**. Mỗi trial apply Algorithm 12.1 với  $M = 1$  để impute missing elements và compute principal components. Left: với mỗi trong 50 states, imputed first principal component scores, averaged over 100 trials và displayed cùng standard deviation bar, được plot against first principal component scores computed using all data. Right: imputed principal component loadings, averaged over 100 trials và displayed cùng standard deviation bar, được plot against true principal component loadings.

Một early example cụ thể của matrix Netflix có  $n = 480,189$  customers và  $p = 17,770$  movies. Tuy nhiên, average mỗi customer chỉ xem khoảng 200 movies nên 99% matrix elements missing. Table 12.2 minh họa setup.

Để suggest một movie mà particular customer có thể thích, Netflix cần impute missing values của matrix. Key idea là set movies customer  $i$  đã xem sẽ overlap với những movies mà other customers đã xem, và một số other customers có movie preferences tương tự customer  $i$ . Vì vậy có thể dùng ratings của similar customers cho những movies customer  $i$  chưa xem để predict customer  $i$  có thích các movies đó hay không.

Cụ thể hơn, bằng Algorithm 12.1, ta predict rating của customer  $i$  cho

movie  $j$  bằng  $\hat{x}_{ij} = \sum_{m=1}^M \hat{a}_{im} \hat{b}_{jm}$ . Ta cũng có thể interpret  $M$  components theo “cliques” và “genres”:

- $\hat{a}_{im}$  biểu diễn strength mà user  $i$  thuộc  $m$ th clique, nơi clique là group customers thích movies thuộc  $m$ th genre;
- $\hat{b}_{jm}$  biểu diễn strength mà movie  $j$  thuộc  $m$ th genre.

Examples of genres gồm Romance, Western, và Action. Principal-component models tương tự Algorithm 12.1 nằm ở core của nhiều recommender systems.

|            | Jerry Maguire | Oceans | Road to Perdition | A Fortunate Man | Catch Me If You Can | Driving Miss Daisy | The Two Popes | The Laundromat | Code 8 | The Social Network | ... |
|------------|---------------|--------|-------------------|-----------------|---------------------|--------------------|---------------|----------------|--------|--------------------|-----|
| Customer 1 | •             | •      | •                 | •               | 4                   | •                  | •             | •              | •      | •                  | ... |
| Customer 2 | •             | •      | 3                 | •               | •                   | •                  | 3             | •              | •      | 3                  | ... |
| Customer 3 | •             | 2      | •                 | 4               | •                   | •                  | •             | •              | 2      | •                  | ... |
| Customer 4 | 3             | •      | •                 | •               | •                   | •                  | •             | •              | •      | •                  | ... |
| Customer 5 | 5             | 1      | •                 | •               | 4                   | •                  | •             | •              | •      | •                  | ... |
| Customer 6 | •             | •      | •                 | •               | •                   | 2                  | 4             | •              | •      | •                  | ... |
| Customer 7 | •             | •      | 5                 | •               | •                   | •                  | •             | 3              | •      | •                  | ... |
| Customer 8 | •             | •      | •                 | •               | •                   | •                  | •             | •              | •      | •                  | ... |
| Customer 9 | 3             | •      | •                 | •               | 5                   | •                  | •             | 1              | •      | •                  | ... |
| ...        | ...           | ...    | ...               | ...             | ...                 | ...                | ...           | ...            | ...    | ...                | ... |

**BẢNG 12.2.** Excerpt của Netflix movie rating data. Movies được rated từ 1 (worst) đến 5 (best). Symbol • biểu diễn missing value: movie không được corresponding customer rate.

Mặc dù data matrices involved thường massive, các algorithms đã được phát triển để exploit high level of missingness nhằm thực hiện efficient computations.

## 12.4 Clustering Methods

*Clustering* là một broad set of techniques để tìm subgroups, hay clusters, trong data set. Khi cluster observations, ta tìm cách partition chúng thành distinct groups sao cho observations within each group khá similar, còn observations ở different groups khá different. Để cụ thể hóa, phải define “similar” hoặc “different”, và đây thường là domain-specific consideration dựa trên knowledge của data đang study.

Ví dụ, giả sử có  $n$  observations với  $p$  features. Observations có thể là tissue samples của breast-cancer patients, còn features là measurements trên mỗi sample: clinical measurements như tumor stage hoặc grade, hoặc gene-expression measurements. Ta có thể tin rằng  $n$  tissue samples có heterogeneity, chẳng hạn tồn tại vài unknown subtypes of breast cancer. Clustering có thể dùng để find các subgroups này. Đây là unsupervised problem vì ta cố discover structure — distinct clusters — chỉ dựa trên data set. Ngược lại, supervised problems cố predict outcome vector như survival time hoặc response to drug treatment.

Clustering và PCA đều seek to simplify data bằng một số nhỏ summaries nhưng mechanisms khác nhau:

clustering

- PCA tìm low-dimensional representation của observations explain một good fraction of variance;
- clustering tìm homogeneous subgroups giữa observations.

Một application khác của clustering là marketing. Ta có thể có nhiều measurements như median household income, occupation, distance from nearest urban area, v.v. cho rất nhiều people. Goal là perform market segmentation bằng cách identify subgroups có khả năng receptive hơn với một form of advertising hoặc likely hơn để purchase particular product. Market segmentation về bản chất là clustering people trong data set.

Vì clustering popular trong nhiều fields, tồn tại rất nhiều clustering methods. Ở đây ta tập trung vào hai approaches nổi tiếng nhất: *K-means clustering* và *hierarchical clustering*. Với K-means, ta partition observations vào một pre-specified number of clusters. Với hierarchical clustering, ta không cần biết trước number of clusters; thay vào đó thu một tree-like representation gọi là *dendrogram*, cho phép view cùng lúc clusterings ứng với mọi possible number of clusters từ 1 đến  $n$ . Mỗi approach có advantages và disadvantages.

Nói chung, ta có thể cluster observations dựa trên features để identify subgroups among observations, hoặc cluster features dựa trên observations để discover subgroups among features. Trong phần sau, để đơn giản ta thảo luận clustering observations based on features; converse có thể thực hiện bằng cách transpose data matrix.

*K-means  
clustering;  
hierarchical  
clustering;  
dendrogram*

### 12.4.1 K-Means Clustering

K-means clustering là simple và elegant approach để partition data set thành  $K$  distinct, non-overlapping clusters. Trước hết phải specify desired number of clusters  $K$ ; sau đó K-means algorithm assign mỗi observation vào exactly one of  $K$  clusters. Figure 12.7 minh họa results trên simulated example gồm 150 observations trong two dimensions với three different values of  $K$ .

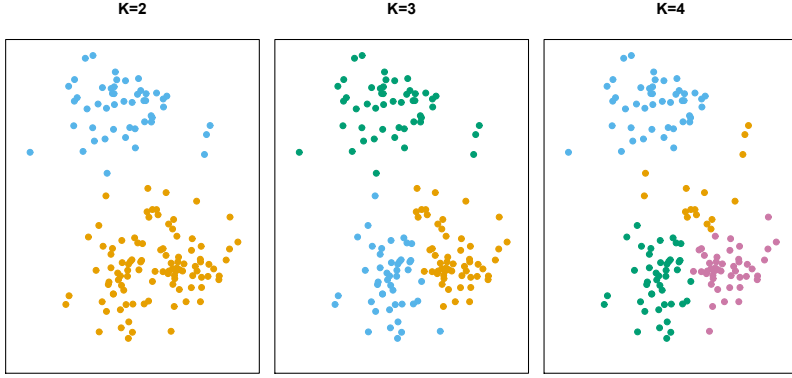
K-means procedure xuất phát từ một simple và intuitive mathematical problem. Gọi  $C_1, \dots, C_K$  là sets chứa indices của observations trong mỗi cluster. Các sets này thỏa hai properties:

1.  $C_1 \cup C_2 \cup \dots \cup C_K = \{1, \dots, n\}$ : mỗi observation thuộc ít nhất one cluster.
2.  $C_k \cap C_{k'} = \emptyset$  cho mọi  $k \neq k'$ : clusters non-overlapping, không observation nào thuộc more than one cluster.

Nếu observation  $i$  nằm trong  $k$ th cluster thì  $i \in C_k$ . Ý tưởng của K-means là một good clustering có within-cluster variation càng nhỏ càng tốt. Với cluster  $C_k$ , within-cluster variation là measure  $W(C_k)$  về mức observations trong cluster differ from each other. Vì vậy ta muốn solve

$$\underset{C_1, \dots, C_K}{\text{minimize}} \left\{ \sum_{k=1}^K W(C_k) \right\}. \quad (12.15)$$

Nói bằng lời, ta partition observations thành  $K$  clusters sao cho total within-cluster variation, summed over all  $K$  clusters, nhỏ nhất.



**HÌNH 12.7.** Một simulated data set với 150 observations trong two-dimensional space. Panels cho results của K-means clustering với different values of  $K$ . Color của observation chỉ cluster được assigned bởi K-means algorithm. Không có ordering của clusters nên coloring là arbitrary; labels này không được dùng trong clustering mà là outputs của procedure.

Để operationalize (12.15), cần define within-cluster variation. Choice phổ biến nhất dùng squared Euclidean distance:

$$W(C_k) = \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2, \quad (12.16)$$

trong đó  $|C_k|$  là number of observations trong  $k$ th cluster. Tức within-cluster variation là sum của all pairwise squared Euclidean distances giữa observations trong cluster, divided by number of observations trong cluster.

Combining (12.15) và (12.16) cho optimization problem define K-means clustering:

$$\text{minimize}_{C_1, \dots, C_K} \left\{ \sum_{k=1}^K \frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right\}. \quad (12.17)$$

Ta muốn một algorithm solve (12.17), tức partition observations thành  $K$  clusters minimize objective. Exact solution rất difficult vì có gần  $K^n$  ways để partition  $n$  observations vào  $K$  clusters, một enormous number trừ khi  $K, n$  rất nhỏ. May thay, một simple algorithm có thể provide local optimum, tức một pretty good solution. Approach được nêu trong Algorithm 12.2.

### Algorithm 12.2 K-Means Clustering

1. Randomly assign một number từ 1 đến  $K$  cho mỗi observation. Đây là initial cluster assignments.
2. Iterate cho đến khi cluster assignments stop changing:
  - (a) Với mỗi trong  $K$  clusters, compute cluster centroid.  $k$ th cluster centroid là vector của  $p$  feature means cho observations trong

$k$ th cluster.

- (b) Assign mỗi observation vào cluster có centroid closest, trong đó closest được define bằng Euclidean distance.

Algorithm 12.2 guaranteed decrease value của objective (12.17) tại each step. Identity sau giúp thấy vì sao:

$$\frac{1}{|C_k|} \sum_{i, i' \in C_k} \sum_{j=1}^p (x_{ij} - x_{i'j})^2 = 2 \sum_{i \in C_k} \sum_{j=1}^p (x_{ij} - \bar{x}_{kj})^2, \quad (12.18)$$

trong đó  $\bar{x}_{kj} = |C_k|^{-1} \sum_{i \in C_k} x_{ij}$  là mean của feature  $j$  trong cluster  $C_k$ . Trong Step 2(a), cluster means minimize sum-of-squared deviations; trong Step 2(b), reallocating observations chỉ có thể improve (12.18). Vì vậy khi algorithm chạy, clustering liên tục improve tới khi result không đổi; objective (12.17) không bao giờ increase. Khi result stable, local optimum đã đạt. Tên K-means đến từ việc Step 2(a) compute cluster centroids bằng mean của observations assigned to each cluster.

Vì K-means finds local thay vì global optimum, results depend on initial random cluster assignment trong Step 1. Do đó quan trọng là run algorithm multiple times từ different random initial configurations rồi chọn best solution, tức solution có smallest objective (12.17).

Figure 12.9 cho local optima thu được khi run K-means six times với six different initial cluster assignments trên toy data Figure 12.7. Trong case này, best clustering có objective value 235.8.

Để perform K-means, ta phải decide how many clusters expect trong data. Problem selecting  $K$  far from simple. Issue này, cùng other practical considerations, được addressed ở Section 12.4.3.

### 12.4.2 Hierarchical Clustering

Một potential disadvantage của K-means là phải pre-specify number of clusters  $K$ . Hierarchical clustering là alternative không cần commit trước vào particular  $K$ , và còn cho attractive tree-based representation của observations gọi là dendrogram.

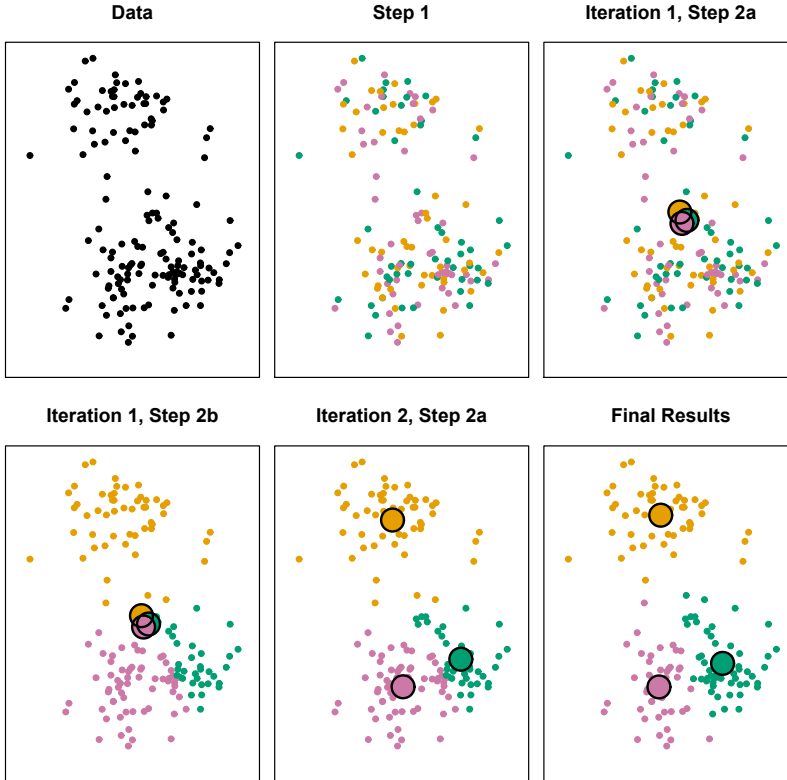
Trong section này ta mô tả *bottom-up* hay *agglomerative clustering*, loại hierarchical clustering phổ biến nhất. Dendrogram, thường drawn như upside-down tree, được build starting from leaves rồi combining clusters lên trunk. Trước tiên ta học cách interpret dendrogram, sau đó xem cách dendrogram được build.

#### Interpreting a Dendrogram

Ta bắt đầu với simulated data set Figure 12.10 gồm 45 observations trong two-dimensional space. Data được generated từ three-class model và true class labels hiển thị bằng distinct colors. Giả sử labels không observed và ta muốn hierarchical cluster data. Hierarchical clustering với complete linkage cho result ở left-hand panel Figure 12.11. Ta interpret dendrogram này như thế nào?

Mỗi leaf của dendrogram represent one observation. Khi move up tree, some leaves fuse thành branches; chúng tương ứng với observations similar nhau. Higher up, branches fuse với leaves hoặc other branches. Fusions

*bottom-up;  
agglomerative  
clustering*



**HÌNH 12.8.** Progress của K-means algorithm trên example Figure 12.7 với  $K = 3$ . Top left: observations. Top center: Step 1 randomly assigns observations to clusters. Top right: Step 2(a) computes cluster centroids, shown as large colored disks; initially centroids gần overlap vì assignments random. Bottom left: Step 2(b) assigns each observation to nearest centroid. Bottom center: Step 2(a) được thực hiện lại và cho new centroids. Bottom right: results sau ten iterations.

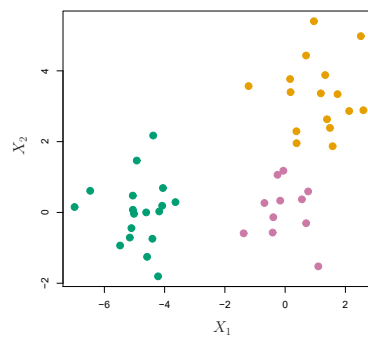
occur càng early, tức càng low in tree, thì groups of observations càng similar. Fusions occur later gần top có thể correspond to quite different observations. Precisely, với any two observations, tìm point nơi branches chứa hai observations first fuse; height của fusion trên vertical axis indicates how different chúng are. Vì vậy observations fuse near bottom khá similar, còn observations fuse near top có xu hướng quite different.

Điểm rất quan trọng thường bị hiểu sai là horizontal proximity trong dendrogram không trực tiếp indicate similarity. Figure 12.12 minh họa simple dendrogram từ nine observations.

Observations 5 và 7 khá similar vì fuse ở lowest point; observations 1 và 6 cũng similar. Tuy nhiên sẽ sai nếu conclude observations 9 và 2 similar chỉ vì nằm gần nhau horizontally trên dendrogram. Theo information trong dendrogram, observation 9 không similar với observation 2 hơn so với observations 8,5,7. Điều này thấy rõ từ raw data ở right-hand panel Figure 12.12. Mathematically, với  $n$  leaves có  $2^{n-1}$  possible reorderings of dendrogram vì tại mỗi trong  $n - 1$  fusion points, positions của two fused branches

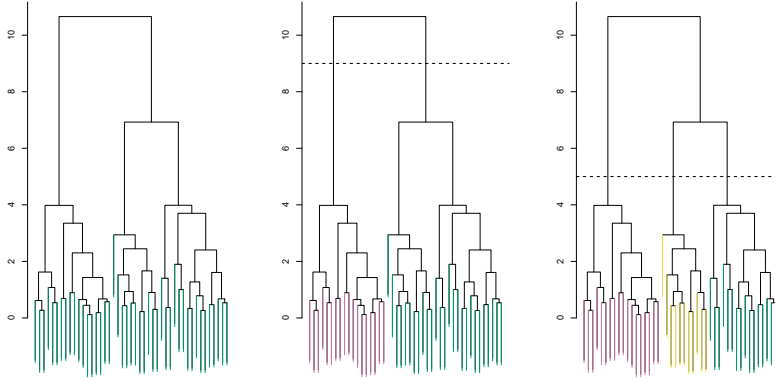


**HÌNH 12.9.** K-means clustering được thực hiện six times trên data Figure 12.7 với  $K = 3$ , mỗi lần dùng different random assignment ở Step 1. Trên mỗi plot là objective value (12.17). Thu được three different local optima; một optimum có smaller objective và better separation. Các plots labeled red đạt cùng best solution với objective 235.8.



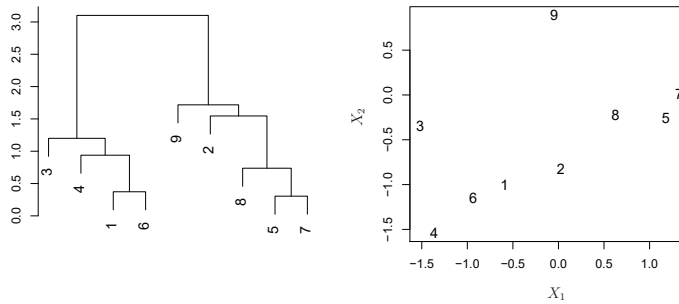
**HÌNH 12.10.** Forty-five observations generated trong two-dimensional space. Thực tế có three distinct classes, shown in separate colors. Tuy nhiên ta treat class labels là unknown và seek to cluster observations để discover classes từ data.

có thể swap mà không đổi meaning. Vì vậy similarity phải được infer từ vertical height nơi branches first fuse, không phải horizontal proximity.



**HÌNH 12.11.** Left: dendrogram từ hierarchical clustering data Figure 12.10 với complete linkage và Euclidean distance. Center: cùng dendrogram cut ở height 9, shown by dashed line, cho two clusters. Right: cut ở height 5 cho three clusters. Colors không được dùng trong clustering mà chỉ dùng cho display.

Để identify clusters từ dendrogram, ta make một horizontal cut như center và right panels Figure 12.11. Distinct sets of observations beneath cut được interpret là clusters. Cut ở height 9 cho two clusters; cut ở height 5 cho three clusters. Các cuts thấp hơn có thể cho bất kỳ number of clusters từ 1 đến  $n$ . Height of cut vì thế đóng same role như  $K$  trong K-means: control number of clusters obtained.



**HÌNH 12.12.** Minh họa cách interpret đúng một dendrogram với nine observations trong two-dimensional space. Left: dendrogram dùng Euclidean distance và complete linkage. Observations 5 và 7 khá similar, cũng như 1 và 6. Observation 9 không similar với observation 2 hơn so với 8,5,7, dù 9 và 2 gần nhau horizontally, vì all 2,8,5,7 fuse với 9 ở cùng height khoảng 1.8. Right: raw data confirm điều này.

Figure 12.11 highlights một attractive aspect của hierarchical clustering: single dendrogram có thể dùng để obtain any number of clusters. Trong practice, người ta thường look at dendrogram và select by eye sensible number of clusters dựa trên fusion heights và desired number. Với Figure 12.11 có thể chọn two hoặc three clusters, nhưng thường choice cut height không clear.

Term *hierarchical* nghĩa là clusters obtained bằng cut ở một height neces-



sarily nested trong clusters obtained bằng cut ở any greater height. Nhưng trên arbitrary data set, assumption hierarchical structure có thể unrealistic. Ví dụ observations là men và women evenly split giữa Americans, Japanese, và French. Best division into two groups có thể split by gender, còn best division into three groups split by nationality. True clusters không nested, nên situation này không thể represented tốt bằng hierarchical clustering. Vì vậy hierarchical clustering đôi khi yield worse results hơn K-means với cùng number of clusters.

### Algorithm 12.3 Hierarchical Clustering

1. Bắt đầu với  $n$  observations và một measure, chẳng hạn Euclidean distance, của all  $\binom{n}{2} = n(n-1)/2$  pairwise dissimilarities. Treat mỗi observation như own cluster.
2. Với  $i = n, n-1, \dots, 2$ :
  - (a) Examine all pairwise inter-cluster dissimilarities giữa  $i$  clusters và identify pair least dissimilar, tức most similar. Fuse hai clusters. Dissimilarity giữa chúng determines height trong dendrogram của fusion.
  - (b) Compute new pairwise inter-cluster dissimilarities giữa  $i-1$  remaining clusters.

#### The Hierarchical Clustering Algorithm

Dendrogram được obtain bằng một extremely simple algorithm. Bắt đầu bằng một dissimilarity measure giữa mỗi pair observations, thường Euclidean distance. Ở bottom của dendrogram, mỗi observation là own cluster. Fuse two most similar clusters để còn  $n-1$ , tiếp tục fuse two most similar để còn  $n-2$ , và lặp cho đến khi all observations thuộc one single cluster. Figure 12.13 depict first few steps cho data Figure 12.12. Algorithm 12.3 summarize process.

Một issue vẫn chưa addressed: trong bottom-right panel Figure 12.13, làm sao decide cluster  $\{5, 7\}$  nên fuse với cluster  $\{8\}$ ? Ta biết dissimilarity between pair observations, nhưng làm sao define dissimilarity between two clusters khi một hoặc cả hai chứa multiple observations? Concept này được extend bằng *linkage*, define dissimilarity giữa two groups of observations. Four common linkage types — complete, average, single, centroid — được mô tả trong Table 12.3. Average, complete, single phổ biến nhất trong statistics; average và complete thường preferred hơn single vì tend to yield more balanced dendrograms. Centroid linkage thường dùng trong genomics nhưng có drawback: *inversion* có thể xảy ra, khi two clusters fuse ở height lower than either individual cluster, gây difficulty cho visualization và interpretation.

*linkage; inversion*

Dissimilarities computed ở Step 2(b) của hierarchical clustering algorithm depend on linkage type và choice of dissimilarity measure. Vì vậy resulting dendrogram thường depend khá strongly on linkage type, như Figure 12.14.

#### Choice of Dissimilarity Measure

Các examples đến đây dùng Euclidean distance làm dissimilarity mea-

| Linkage  | Description                                                                                                                                                                                                                                     |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Complete | Maximal intercluster dissimilarity. Compute all pairwise dissimilarities giữa observations trong cluster A và cluster B, rồi record largest value.                                                                                              |
| Single   | Minimal intercluster dissimilarity. Compute all pairwise dissimilarities giữa observations trong A và B, rồi record smallest value. Single linkage có thể tạo extended, trailing clusters trong đó single observations được fuse one-at-a-time. |
| Average  | Mean intercluster dissimilarity. Compute all pairwise dissimilarities giữa observations trong A và B rồi record average.                                                                                                                        |
| Centroid | Dissimilarity giữa centroid của cluster A, một mean vector length $p$ , và centroid của cluster B. Centroid linkage có thể tạo undesirable inversions.                                                                                          |

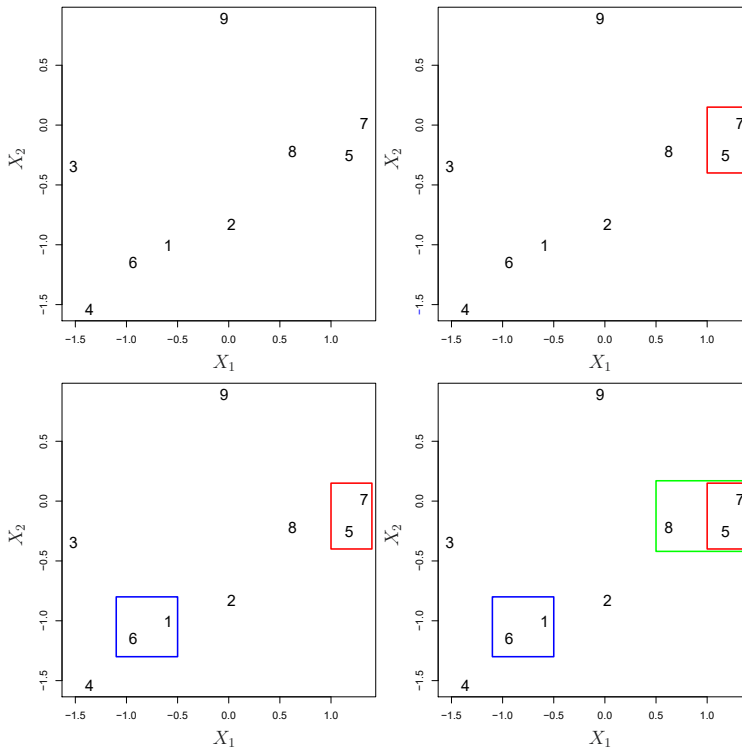
**BẢNG 12.3.** Summary của four most commonly-used linkage types trong hierarchical clustering.

sure. Nhưng sometimes other measures preferable. Chẳng hạn correlation-based distance xem two observations similar nếu feature profiles của chúng highly correlated, dù observed values có thể far apart theo Euclidean distance. Đây là một unusual use of correlation, vốn thường computed between variables; ở đây correlation computed giữa observation profiles của each pair observations. Figure 12.15 minh họa difference. Correlation-based distance focus on shapes of observation profiles hơn magnitudes.

Choice of dissimilarity measure rất important vì ảnh hưởng mạnh đến resulting dendrogram. Cần chú ý type of data đang cluster và scientific question; chúng nên determine dissimilarity measure dùng trong hierarchical clustering.

Ví dụ một online retailer muốn cluster shoppers theo past shopping histories để identify subgroups of similar shoppers và show items/advertisements likely to interest từng subgroup. Data matrix có rows là shoppers, columns là items, elements là number of times shopper purchased item. Nếu dùng Euclidean distance, shoppers mua rất few items overall, tức infrequent users, sẽ cluster together. Điều này có thể undesirable. Nếu dùng correlation-based distance, shoppers có similar preferences, chẳng hạn mua A và B nhưng không C hoặc D, sẽ cluster together dù some shoppers có higher purchase volume. Với application này, correlation-based distance có thể là better choice.

Ngoài careful selection của dissimilarity measure, còn phải consider có nên scale variables về standard deviation one trước khi compute inter-observation dissimilarities. Tiếp tục online-shopping example: một số items purchased thường xuyên hơn, như shopper có thể mua ten pairs of socks per year nhưng computer rất hiếm. High-frequency purchases như socks vì thế có much larger effect lên inter-shopper dissimilarities và resulting clustering hơn rare purchases như computers. Điều này có thể undesirable. Scaling variables về standard deviation one làm each variable effectively có equal importance. Ta cũng có thể muốn scale khi variables measured trên different scales; nếu không, choice of units như centimeters versus



**HÌNH 12.13.** First few steps của hierarchical clustering algorithm dùng data Figure 12.12, complete linkage và Euclidean distance. Top left: ban đầu có nine distinct clusters  $\{1\}, \dots, \{9\}$ . Top right: closest clusters  $\{5\}$  và  $\{7\}$  fuse. Bottom left:  $\{6\}$  và  $\{1\}$  fuse. Bottom right: theo complete linkage, cluster  $\{8\}$  fuse với cluster  $\{5, 7\}$ .

kilometers sẽ ảnh hưởng mạnh dissimilarity. Việc scale có tốt hay không phụ thuộc application. Issue này cũng apply cho K-means.

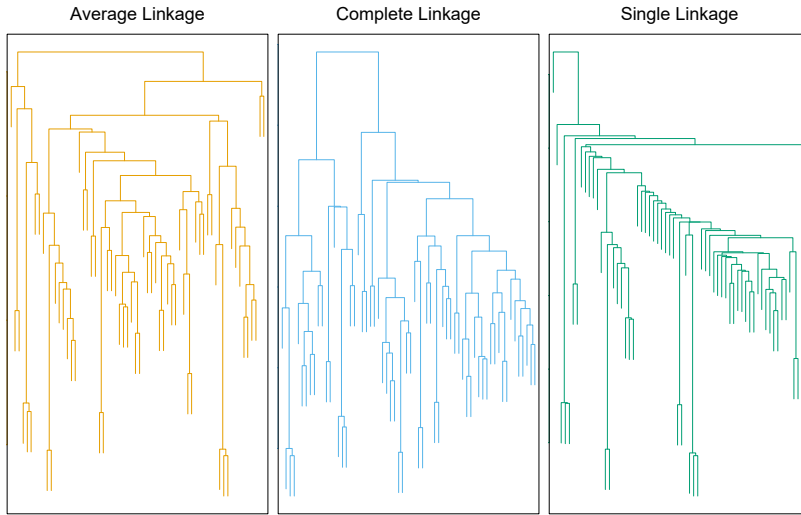
### 12.4.3 Practical Issues in Clustering

Clustering có thể là very useful tool cho unsupervised data analysis, nhưng có nhiều issues cần cân nhắc.

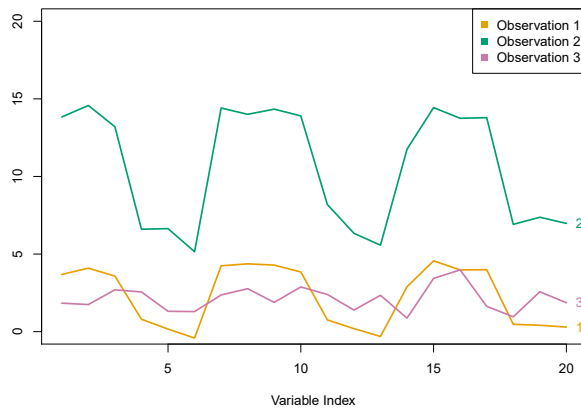
#### Small Decisions with Big Consequences

Để perform clustering, phải đưa ra một số decisions.

- Có nên standardize observations hoặc features trước theo một cách nào đó không? Ví dụ scale variables để có standard deviation one.
- Với hierarchical clustering:
  - nên dùng dissimilarity measure nào?
  - nên dùng linkage type nào?
  - nên cut dendrogram ở đâu để obtain clusters?



**HÌNH 12.14.** Average, complete, và single linkage applied cho một example data set. Average và complete linkage tend to yield more balanced clusters.



**HÌNH 12.15.** Three observations với measurements trên 20 variables. Observations 1 và 3 có similar values cho each variable nên small Euclidean distance, nhưng very weakly correlated nên large correlation-based distance. Observations 1 và 2 có quite different values nên large Euclidean distance, nhưng highly correlated nên small correlation-based distance.

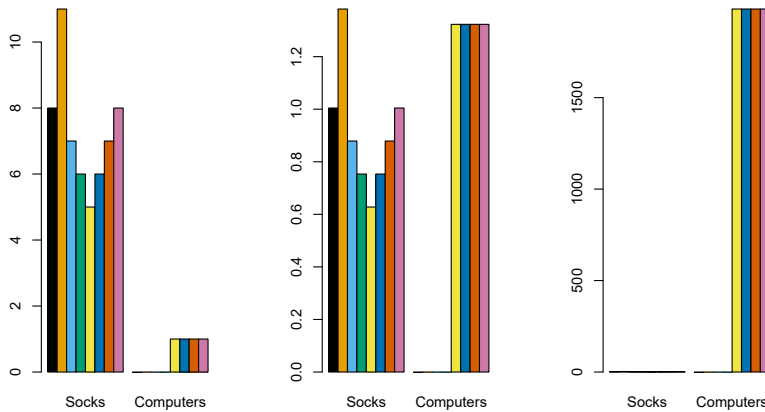
- Với K-means clustering, nên tìm bao nhiêu clusters trong data?

Mỗi decision có thể có strong impact lên results. Trong practice, ta thử several choices và tìm solution useful hoặc interpretable nhất. Với các methods này, không có single right answer — bất kỳ solution nào expose interesting aspects của data đều đáng consider.

### Validating the Clusters Obtained

Bất cứ khi nào cluster một data set ta đều sẽ find clusters. Nhưng điều thực sự muốn biết là clusters tìm được represent true subgroups trong data

hay chỉ là result của clustering noise. Chẳng hạn nếu có independent set of observations, chúng có display same set of clusters không? Đây là hard question. Có một số techniques assign p-value cho cluster để assess liệu evidence cho cluster có vượt mức expect by chance hay không, nhưng chưa có consensus về one best approach. More details có trong ESL.<sup>8</sup>



**HÌNH 12.16.** Một eclectic online retailer bán socks và computers. Left: numbers of socks và computers purchased bởi eight shoppers. Nếu dissimilarities computed bằng Euclidean distance trên raw variables, sock purchases dominate và computer purchases có little effect, điều có thể undesirable. Center: same data sau khi scale each variable by its standard deviation, nên two products có comparable effects. Right: same data nhưng y-axis là dollars spent; vì computers much more expensive, computer purchase history giữ dominate dissimilarities.

### Other Considerations in Clustering

Cả K-means lẫn hierarchical clustering assign each observation to a cluster, nhưng đôi khi điều này không appropriate. Giả sử most observations truly belong to một số nhỏ unknown subgroups, còn một small subset rất different from each other và from all other observations. Vì K-means và hierarchical clustering force every observation into a cluster, clusters có thể bị heavily distorted bởi outliers không thuộc any cluster. Mixture models là attractive approach để accommodate outliers; có thể xem chúng như soft version of K-means và được mô tả trong ESL.

Ngoài ra, clustering methods nói chung không very robust trước perturbations to data. Chẳng hạn cluster  $n$  observations rồi cluster lại sau khi randomly remove một subset. Ta có thể hy vọng two sets of clusters khá similar, nhưng thường không phải vậy.

### A Tempered Approach to Interpreting the Results of Clustering

Ta đã mô tả một số issues của clustering. Tuy nhiên, clustering vẫn là useful và valid statistical tool nếu used properly. Small decisions như cách standardize data hay linkage type có thể tạo large effect. Vì vậy, nên perform clustering với different choices of parameters và xem full set of results để nhận ra patterns nào consistently emerge. Vì clustering có thể

<sup>8</sup>ESL: *The Elements of Statistical Learning* by Hastie, Tibshirani and Friedman.

non-robust, cũng nên cluster subsets of data để đánh giá robustness của clusters obtained. Quan trọng nhất, phải careful khi report results: không nên xem chúng như absolute truth về data set. Thay vào đó, chúng nên là starting point cho development of a scientific hypothesis và further study, preferably trên independent data set.

## 12.5 Lab: Unsupervised Learning

Trong lab này, ta minh họa PCA và clustering trên một số datasets. Như ở các lab khác, ta import một số libraries ngay từ đầu. Cách này làm code dễ đọc hơn, vì chỉ cần nhìn vài dòng đầu của notebook là có thể biết những libraries nào được sử dụng.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.datasets import get_rdataset
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from ISLP import load_data
```

Ta cũng tập hợp các imports mới cần cho lab này.

```
In [2]: from sklearn.cluster import \
 (KMeans,
 AgglomerativeClustering)
from scipy.cluster.hierarchy import \
 (dendrogram,
 cut_tree)
from ISLP.cluster import compute_linkage
```

### 12.5.1 Principal Components Analysis

Trong lab này, ta thực hiện PCA trên **USArrests**, một data set trong R computing environment. Ta lấy data bằng `get_rdataset()`, hàm có thể fetch data từ nhiều standard R packages.

`get_rdataset()`

Các rows của data set chứa 50 states, theo thứ tự alphabet.

```
In [3]: USArrests = get_rdataset('USArrests').data
USArrests
```

```
Out[3]:
```

|           | Murder | Assault | UrbanPop | Rape |
|-----------|--------|---------|----------|------|
| Alabama   | 13.2   | 236     | 58       | 21.2 |
| Alaska    | 10.0   | 263     | 48       | 44.5 |
| Arizona   | 8.1    | 294     | 80       | 31.0 |
| ...       | ...    | ...     | ...      | ...  |
| Wisconsin | 2.6    | 53      | 66       | 10.8 |
| Wyoming   | 6.8    | 161     | 60       | 15.6 |

Các columns của data set chứa bốn variables.

```
In [4]: USArrests.columns
```

```
Out[4]: Index(['Murder', 'Assault', 'UrbanPop', 'Rape'], dtype='object')
```

Trước hết ta examine data một cách ngắn gọn. Ta thấy các variables có means rất khác nhau.

```
In [5]: USArrests.mean()
```

```
Out[5]: Murder 7.788
Assault 170.760
UrbanPop 65.540
Rape 21.232
dtype: float64
```

Dataframes có một số useful methods để compute column-wise summaries. Ta cũng có thể examine variance của bốn variables bằng method `var()`.

```
In [6]: USArrests.var()
```

```
Out[6]: Murder 18.970465
Assault 6945.165714
UrbanPop 209.518776
Rape 87.729159
dtype: float64
```

Không ngạc nhiên khi các variables cũng có variances rất khác nhau. Variable **UrbanPop** đo percentage của population trong mỗi state sống ở urban area, nên không thể so trực tiếp với number of rapes trên 100,000 individuals. PCA tìm các derived variables giải thích phần lớn variance trong data set. Nếu ta không scale variables trước khi thực hiện PCA, principal components sẽ chủ yếu bị chi phối bởi **Assault**, vì variable này có variance lớn hơn hẳn. Vì vậy, nếu variables được đo bằng different units hoặc khác nhau nhiều về scale, nên standardize để chúng có standard deviation bằng one trước khi thực hiện PCA. Thông thường ta cũng set means bằng zero.

Việc scaling này có thể thực hiện bằng transform `StandardScaler()` đã import ở trên. Trước hết ta fit scaler, qua đó compute các means và standard deviations cần thiết, rồi apply nó lên data bằng method `transform`. Như trước, ta combine hai bước bằng `fit_transform()`.

```
In [7]: scaler = StandardScaler(with_std=True,
 with_mean=True)
USArrests_scaled = scaler.fit_transform(USArrests)
```

*PCA()* Sau khi scale data, ta có thể thực hiện principal components analysis bằng transform `PCA()` từ package `sklearn.decomposition`.

```
In [8]: pcaUS = PCA()
```

Theo default, `PCA()` centers variables để có mean zero nhưng không scale chúng. Transform `pcaUS` có thể dùng để tìm PCA scores trả về sau khi fit. Khi method `fit()` đã được gọi, object `pcaUS` cũng chứa một số quantities hữu ích.

```
In [9]: pcaUS.fit(USArrests_scaled)
```

Sau khi fitting, attribute `mean_` tương ứng với means của variables. Trong trường hợp này, vì ta đã center và scale data bằng `scaler`, các means đều bằng 0.



```
In [10]: pcaUS.mean_
```

```
Out[10]: array([-0., 0., -0., 0.])
```

Scores có thể được compute bằng method `transform()` của `pcaUS` sau khi object đã được fit.

```
In [11]: scores = pcaUS.transform(USArrests_scaled)
```

Ta sẽ plot các scores này ở phía dưới. Attribute `components_` cung cấp principal component loadings: mỗi row của `pcaUS.components_` chứa corresponding principal component loading vector.

```
In [12]: pcaUS.components_
```

```
Out[12]: array([[0.53589947, 0.58318363, 0.27819087, 0.54343209],
 [0.41818087, 0.18798560, -0.87280619, -0.16731864],
 [-0.34123273, -0.26814843, -0.37801579, 0.81777791],
 [0.64922780, -0.74340748, 0.13387773, 0.08902432]])
```

Biplot là một visualization method thường dùng cùng PCA. Nó không được tích hợp như một standard part của `sklearn`, dù có các Python packages tạo loại plot này. Ở đây ta tự tạo một simple biplot.

```
In [13]: i, j = 0, 1 # which components
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(scores[:,0], scores[:,1])
ax.set_xlabel('PC%d' % (i+1))
ax.set_ylabel('PC%d' % (j+1))
for k in range(pcaUS.components_.shape[1]):
 ax.arrow(0, 0,
 pcaUS.components_[i,k],
 pcaUS.components_[j,k])
 ax.text(pcaUS.components_[i,k],
 pcaUS.components_[j,k],
 USArrests.columns[k])
```

Figure này là reflection của Figure 12.1 qua y-axis. Nhớ rằng principal components chỉ unique đến một sign change, nên ta có thể reproduce figure đó bằng cách flip signs của second set of scores và loadings. Ta cũng increase length của arrows để nhấn mạnh loadings.

```
In [14]: scale_arrow = s_ = 2
scores[:,1] *= -1
pcaUS.components_[1] *= -1 # flip the y-axis
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
ax.scatter(scores[:,0], scores[:,1])
ax.set_xlabel('PC%d' % (i+1))
ax.set_ylabel('PC%d' % (j+1))
for k in range(pcaUS.components_.shape[1]):
 ax.arrow(0, 0,
 s_*pcaUS.components_[i,k],
 s_*pcaUS.components_[j,k])
 ax.text(s_*pcaUS.components_[i,k],
 s_*pcaUS.components_[j,k],
 USArrests.columns[k])
```

Standard deviations của principal component scores như sau:

```
In [15]: scores.std(0, ddof=1)
```

```
Out[15]: array([1.5909, 1.0050, 0.6032, 0.4207])
```

Variance của mỗi score có thể được extract trực tiếp từ object `pcaUS` qua attribute `explained_variance_`.

```
In [16]: pcaUS.explained_variance_
```

```
Out[16]: array([2.5309, 1.01, 0.3638, 0.177])
```

Proportion of variance explained bởi mỗi principal component (PVE) được lưu trong `explained_variance_ratio_`:

```
In [17]: pcaUS.explained_variance_ratio_
```

```
Out[17]: array([0.6201, 0.2474, 0.0891, 0.0434])
```

Ta thấy first principal component giải thích 62.0% variance trong data, next principal component giải thích 24.7%, và cứ thế. Ta có thể plot PVE explained bởi mỗi component cũng như cumulative PVE. Trước hết plot proportion of variance explained.

```
In [18]: %%capture
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
ticks = np.arange(pcaUS.n_components_)+1
ax = axes[0]
ax.plot(ticks,
 pcaUS.explained_variance_ratio_,
 marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('Proportion of Variance Explained')
ax.set_ylim([0,1])
ax.set_xticks(ticks)
```

Lưu ý việc dùng `%%capture`, giúp suppress việc display một figure đang mới được build một phần.

```
In [19]: ax = axes[1]
ax.plot(ticks,
 pcaUS.explained_variance_ratio_.cumsum(),
 marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('Cumulative Proportion of Variance Explained')
ax.set_ylim([0, 1])
ax.set_xticks(ticks)
fig
```

Result tương tự Figure 12.3. Method `cumsum()` computes cumulative sum của các elements trong một numeric vector. Chẳng hạn:

*cumsum()*

```
In [20]: a = np.array([1,2,8,-3])
np.cumsum(a)
```

```
Out[20]: array([1, 3, 11, 8])
```

### 12.5.2 Matrix Completion

Bây giờ ta re-create analysis đã thực hiện trên `USArrests` data trong Section 12.3. Ta đã thấy ở Section 12.2.2 rằng solving optimization problem (12.6) trên một centered data matrix  $X$  tương đương với computing first  $M$  principal components của data. Ta dùng scaled và centered `USArrests` data làm  $X$  bên dưới. Singular value decomposition (SVD) là một general algorithm để solve (12.6).

*singular value  
decomposition;  
svd()*

```
In [21]: X = USArrests_scaled
 U, D, V = np.linalg.svd(X, full_matrices=False)
 U.shape, D.shape, V.shape
```

```
Out[21]: ((50, 4), (4,), (4, 4))
```

Function `np.linalg.svd()` returns three components,  $U$ ,  $D$  và  $V$ . Matrix  $V$  tương đương loading matrix từ principal components, ngoại trừ một unimportant sign flip. Option `full_matrices=False` đảm bảo rằng với một tall matrix, shape của  $U$  giống shape của  $X$ .

```
In [22]: V
```

```
Out[22]: array([[-0.53589947, -0.58318363, -0.27819087, -0.54343209],
 [0.41818087, 0.18798560, -0.87280619, -0.16731864],
 [-0.34123273, -0.26814843, -0.37801579, 0.81777791],
 [0.64922780, -0.74340748, 0.13387773, 0.08902432]])
```

```
In [23]:
```

```
pcaUS.components_
```

```
Out[23]: array([[0.53589947, 0.58318363, 0.27819087, 0.54343209],
 [0.41818087, 0.18798560, -0.87280619, -0.16731864],
 [-0.34123273, -0.26814843, -0.37801579, 0.81777791],
 [0.64922780, -0.74340748, 0.13387773, 0.08902432]])
```

Matrix  $U$  tương ứng với một standardized version của PCA score matrix, trong đó mỗi column được standardized để có sum-of-squares bằng one. Nếu multiply mỗi column của  $U$  với corresponding element của  $D$ , ta recover PCA scores exactly, ngoại trừ một meaningless sign flip.

```
In [24]:
```

```
(U * D[None,:])[:3]
```

```
Out[24]: array([[-0.9856, 1.1334, -0.4443, 0.1563],
 [-1.9501, 1.0732, 2.0400, -0.4386],
 [-1.7632, -0.7460, 0.0548, -0.8347]])
```

```
In [25]:
```

```
scores[:3]
```

```
Out[25]: array([[0.9856, -1.1334, -0.4443, 0.1563],
 [1.9501, -1.0732, 2.0400, -0.4386],
 [1.7632, 0.7460, 0.0548, -0.8347]])
```

Dù có thể thực hiện lab này bằng estimator `PCA()`, ở đây ta dùng function

`np.linalg.svd()` để minh họa cách sử dụng nó. Bây giờ ta omit 20 entries trong data matrix  $50 \times 4$  một cách random. Ta làm điều này bằng cách trước hết chọn random 20 rows (states), rồi chọn random one trong four entries của mỗi row. Như vậy every row có ít nhất three observed values.

In [26]:

```
n_omit = 20
np.random.seed(15)
r_idx = np.random.choice(np.arange(X.shape[0]),
 n_omit,
 replace=False)
c_idx = np.random.choice(np.arange(X.shape[1]),
 n_omit,
 replace=True)

Xna = X.copy()
Xna[r_idx, c_idx] = np.nan
```

Array `r_idx` chứa 20 integers từ 0 đến 49, đại diện cho states (rows của  $X$ ) được chọn để có missing values. `c_idx` chứa 20 integers từ 0 đến 3, đại diện cho features (columns trong  $X$ ) có missing values cho từng selected state.

Bây giờ ta viết code để implement Algorithm 12.1. Trước hết viết một function nhận một matrix và returns một approximation của matrix đó bằng function `svd()`. Function này cần cho Step 2 của Algorithm 12.1.

In [27]:

```
def low_rank(X, M=1):
 U, D, V = np.linalg.svd(X)
 L = U[:, :M] * D[None, :M]
 return L.dot(V[:, :M])
```

Để thực hiện Step 1 của algorithm, ta initialize  $\tilde{X}$ at — đây là  $\tilde{X}$  trong Algorithm 12.1 — bằng cách replace missing values bằng column means của non-missing entries. Các means này được lưu trong  $\bar{X}$  sau khi chạy `np.nanmean()` theo row axis. Ta tạo một copy để khi assign values cho  $\tilde{X}$ at bên dưới, ta không overwrite values trong  $X$ na.

*np.nanmean()*

In [28]:

```
Xhat = Xna.copy()
Xbar = np.nanmean(Xhat, axis=0)
Xhat[r_idx, c_idx] = Xbar[c_idx]
```

Trước khi bắt đầu Step 2, ta chuẩn bị để measure progress của iterations:

In [29]:

```
thresh = 1e-7
rel_err = 1
count = 0
ismiss = np.isnan(Xna)
mssold = np.mean(Xhat[~ismiss]**2)
mss0 = np.mean(Xna[~ismiss]**2)
```

Ở đây `ismiss` là một logical matrix có cùng dimensions với  $X$ na; một element là `True` nếu corresponding matrix element bị missing. Notation `~ismiss` negates boolean vector này, nhờ đó ta có thể access cả missing và non-missing entries. Ta lưu mean của squared non-missing elements trong `mss0`. Ta lưu mean squared error của non-missing elements trong old version của  $\tilde{X}$ at vào `mssold`; hiện tại quantity này bằng `mss0`. Ta sẽ lưu mean squared error của non-missing elements trong current version của  $\tilde{X}$ at vào `mss`, rồi iterate Step 2 của Algorithm 12.1 cho đến khi relative

error, defined là  $(mssold - mss) / mss0$ , nhỏ hơn  $thresh = 1e-7$ .<sup>9</sup>

Trong Step 2(a) của Algorithm 12.1, ta approximate  $X_{hat}$  bằng `low_rank()`; gọi kết quả là  $X_{app}$ . Trong Step 2(b), dùng  $X_{app}$  để update estimates cho elements trong  $X_{hat}$  bị missing trong  $X_{na}$ . Cuối cùng, ở Step 2(c), compute relative error. Ba steps này nằm trong while loop sau:

In [30]:

```
while rel_err > thresh:
 count += 1
 # Step 2(a)
 Xapp = low_rank(Xhat, M=1)
 # Step 2(b)
 Xhat[ismiss] = Xapp[ismiss]
 # Step 2(c)
 mss = np.mean((Xna - Xapp)[~ismiss]**2)
 rel_err = (mssold - mss) / mss0
 mssold = mss
 print("Iteration: {0}, MSS :{1:.3f}, Rel.Err {2:.2e}"
 .format(count, mss, rel_err))
```

Out[30]:

```
Iteration: 1, MSS :0.395, Rel.Err 5.99e-01
Iteration: 2, MSS :0.382, Rel.Err 1.33e-02
Iteration: 3, MSS :0.381, Rel.Err 1.44e-03
Iteration: 4, MSS :0.381, Rel.Err 1.79e-04
Iteration: 5, MSS :0.381, Rel.Err 2.58e-05
Iteration: 6, MSS :0.381, Rel.Err 4.22e-06
Iteration: 7, MSS :0.381, Rel.Err 7.65e-07
Iteration: 8, MSS :0.381, Rel.Err 1.48e-07
Iteration: 9, MSS :0.381, Rel.Err 2.95e-08
```

Ta thấy sau eight iterations, relative error đã giảm xuống dưới  $thresh = 1e-7$ , nên algorithm terminates. Khi đó mean squared error của non-missing elements bằng 0.381. Cuối cùng, ta compute correlation giữa 20 imputed values và actual values:

In [31]:

```
np.corrcoef(Xapp[ismiss], X[ismiss])[0,1]
```

Out[31]:

```
0.711
```

Trong lab này, ta tự implement Algorithm 12.1 vì mục đích didactic. Tuy nhiên, reader muốn apply matrix completion lên data của mình có thể tìm các specialized Python implementations.

### 12.5.3 Clustering

#### *K-Means Clustering*

Estimator `sklearn.cluster.KMeans()` thực hiện K-means clustering trong Python. Ta bắt đầu với một simple simulated example trong đó thực sự có two clusters trong data: first 25 observations có một mean shift relative to next 25 observations.

*KMeans()*

<sup>9</sup>Algorithm 12.1 yêu cầu iterate Step 2 đến khi (12.14) không còn decrease. Để xác định (12.14) có decrease hay không, chỉ cần theo dõi  $mssold - mss$ . Trong practice, ta theo dõi  $(mssold - mss) / mss0$ ; cách này làm number of iterations cần để Algorithm 12.1 converge không phụ thuộc vào việc raw data  $X$  có bị multiply bởi một constant factor hay không.

```
In [32]: np.random.seed(0)
X = np.random.standard_normal((50,2))
X[:25,0] += 3
X[:25,1] -= 4
```

Bây giờ thực hiện K-means clustering với  $K = 2$ .

```
In [33]: kmeans = KMeans(n_clusters=2,
 random_state=2,
 n_init=20).fit(X)
```

Ta specify `random_state` để results reproducible. Cluster assignments của 50 observations nằm trong `kmeans.labels_`.

```
In [34]: kmeans.labels_
```

```
Out[34]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int32)
```

K-means clustering đã separate observations thành two clusters gần như hoàn hảo dù ta không supply bất kỳ group information nào cho `KMeans()`. Ta có thể plot data, coloring mỗi observation theo cluster assignment.

```
In [35]: fig, ax = plt.subplots(1, 1, figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=kmeans.labels_)
ax.set_title("K-Means Clustering Results with K=2")
```

Ở đây observations có thể plot dễ dàng vì chúng two-dimensional. Nếu có nhiều hơn two variables, ta có thể perform PCA rồi plot first two principal component score vectors để represent clusters.

Trong example này ta biết thực sự có two clusters vì chính ta generated data. Với real data, ta không biết true number of clusters, thậm chí không biết chúng có tồn tại theo một precise way hay không. Thay vào đó, ta có thể đã performed K-means clustering với  $K = 3$ .

```
In [36]: kmeans = KMeans(n_clusters=3,
 random_state=3,
 n_init=20).fit(X)
fig, ax = plt.subplots(figsize=(8,8))
ax.scatter(X[:,0], X[:,1], c=kmeans.labels_)
ax.set_title("K-Means Clustering Results with K=3")
```

Khi  $K = 3$ , K-means clustering split two original clusters. Ta dùng argument `n_init` để run K-means với 20 initial cluster assignments; default là 10. Nếu `n_init > 1`, K-means clustering được performed với multiple random assignments ở Step 1 của Algorithm 12.2, và `KMeans()` chỉ report best result. Ở đây ta compare `n_init=1` với `n_init=20`.

```
In [37]: kmeans1 = KMeans(n_clusters=3,
 random_state=3,
 n_init=1).fit(X)
kmeans20 = KMeans(n_clusters=3,
 random_state=3,
 n_init=20).fit(X)
kmeans1.inertia_, kmeans20.inertia_
```

Out[37]: (78.06, 75.04)

Lưu ý `kmeans.inertia_` là total within-cluster sum of squares, quantity ta muốn minimize khi thực hiện K-means clustering (12.17). Ta strongly recommend luôn run K-means clustering với large value của `n_init`, chẳng hạn 20 hoặc 50, vì nếu không có thể obtain một undesirable local optimum. Khi perform K-means clustering, ngoài multiple initial cluster assignments, cũng quan trọng phải set random seed bằng argument `random_state` của `KMeans()`. Như vậy initial cluster assignments ở Step 1 có thể replicated và K-means output fully reproducible.

### Hierarchical Clustering

Class `AgglomerativeClustering()` từ package `sklearn.cluster` implements hierarchical clustering. Vì tên dài, ta dùng shorthand `HClust`. Điều này không thay return type nên instances vẫn thuộc class `AgglomerativeClustering`. Trong example sau, ta dùng data từ lab trước để plot hierarchical clustering dendrogram với complete, single, và average linkage, dùng Euclidean distance làm dissimilarity measure. Bắt đầu bằng complete linkage.

*Agglomerative-Clustering()*

```
In [38]: HClust = AgglomerativeClustering
hc_comp = HClust(distance_threshold=0,
 n_clusters=None,
 linkage='complete')
hc_comp.fit(X)
```

Lệnh này computes toàn bộ dendrogram. Ta cũng có thể perform hierarchical clustering với average hoặc single linkage:

```
In [39]: hc_avg = HClust(distance_threshold=0,
 n_clusters=None,
 linkage='average')
hc_avg.fit(X)
hc_sing = HClust(distance_threshold=0,
 n_clusters=None,
 linkage='single')
hc_sing.fit(X)
```

Để dùng một precomputed distance matrix, ta supply thêm argument `metric='precomputed'`. Trong code dưới, first four lines compute  $50 \times 50$  pairwise-distance matrix.

```
In [40]: D = np.zeros((X.shape[0], X.shape[0]))
for i in range(X.shape[0]):
 x_ = np.multiply.outer(np.ones(X.shape[0]), X[i])
 D[i] = np.sqrt(np.sum((X - x_)**2, 1))
hc_sing_pre = HClust(distance_threshold=0,
 n_clusters=None,
 metric='precomputed',
 linkage='single')
hc_sing_pre.fit(D)
```

Ta dùng `dendrogram()` từ `scipy.cluster.hierarchy` để plot dendrogram. Tuy nhiên `dendrogram()` expects một so-called linkage-matrix representation của clustering, không được `AgglomerativeClustering()` provide trực tiếp nhưng có thể compute. Function `compute_linkage()` trong

```
dendrogram();
compute_linkage()
```

package `ISLP.cluster` được cung cấp cho mục đích này.

Bây giờ ta có thể plot dendrograms. Numbers ở bottom của plot identify each observation. Function `dendrogram()` có default method để color các branches khác nhau của tree, gợi ý một pre-defined cut tại particular depth. Ta prefer overwrite default này bằng cách set threshold thành infinity. Vì muốn behavior này cho nhiều dendrograms, ta store các values trong dictionary `cargs` rồi pass nó như keyword arguments bằng notation `**cargs`.

```
In [41]: cargs = {'color_threshold':-np.inf,
 'above_threshold_color':'black'}
linkage_comp = compute_linkage(hc_comp)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp,
 ax=ax,
 **cargs)
```

Có thể ta muốn color branches phía trên và dưới một cut-threshold khác nhau. Điều này thực hiện bằng cách change `color_threshold`. Hãy cut tree ở height 4, coloring links merge above 4 bằng black.

```
In [42]: fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp,
 ax=ax,
 color_threshold=4,
 above_threshold_color='black')
```

Để determine cluster labels cho mỗi observation associated với một given cut của dendrogram, ta dùng function `cut_tree()` từ `scipy.cluster.hierarchy`:

```
cut_tree()
```

```
In [43]: cut_tree(linkage_comp, n_clusters=4).T
```

```
Out[43]: array([[0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 2, 0, 0, 0, 1, 1, 0, 0, 1,
 0, 0, 2, 0, 2, 2, 3, 2, 3, 3, 3, 3, 2, 3, 3, 3, 3, 2, 3,
 3, 3, 3, 2, 3, 3, 3, 3, 3, 3, 3, 3]])
```

Điều này cũng có thể đạt được bằng cách supply argument `n_clusters` cho `HClust()`; tuy nhiên mỗi cut sẽ require recomputing clustering. Tương tự, trees có thể cut theo distance threshold bằng argument `distance_threshold` cho `HClust()` hoặc `height` cho `cut_tree()`.

```
In [44]: cut_tree(linkage_comp, height=5)
```

```
Out[44]: array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 1, 0, 1, 1, 2, 1, 2, 2, 2, 2, 1, 2, 2, 2, 2, 1, 2,
 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2]])
```

Để scale variables trước khi hierarchical clustering observations, ta dùng `StandardScaler()` như trong PCA example:

```
In [45]: scaler = StandardScaler()
X_scale = scaler.fit_transform(X)
hc_comp_scale = HClust(distance_threshold=0,
 n_clusters=None,
```



```

linkage='complete').fit(X_scale)
linkage_comp_scale = compute_linkage(hc_comp_scale)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_comp_scale, ax=ax, **cargs)
ax.set_title("Hierarchical Clustering with Scaled Features")

```

Correlation-based distances giữa observations có thể dùng cho clustering. Correlation giữa two observations measures similarity của their feature values.<sup>10</sup> Với  $n$  observations,  $n \times n$  correlation matrix có thể dùng như similarity (hay affinity) matrix; do đó one minus correlation matrix là dissimilarity matrix dùng cho clustering.

Lưu ý rằng dùng correlation chỉ có ý nghĩa cho data có ít nhất three features, vì absolute correlation giữa any two observations với measurements trên two features luôn bằng one. Vì vậy ta cluster một three-dimensional data set.

In [46]:

```

X = np.random.standard_normal((30, 3))
corD = 1 - np.corrcoef(X)
hc_cor = HClust(linkage='complete',
 distance_threshold=0,
 n_clusters=None,
 metric='precomputed')
hc_cor.fit(corD)
linkage_cor = compute_linkage(hc_cor)
fig, ax = plt.subplots(1, 1, figsize=(8, 8))
dendrogram(linkage_cor, ax=ax, **cargs)
ax.set_title("Complete Linkage with Correlation-Based Dissimilarity")

```

### 12.5.4 NCI60 Data Example

Unsupervised techniques thường được dùng trong analysis của genomic data. Đặc biệt, PCA và hierarchical clustering là popular tools. Ta minh họa các techniques này trên NCI60 cancer cell line microarray data, gồm 6830 gene expression measurements trên 64 cancer cell lines.

In [47]:

```

NCI60 = load_data('NCI60')
nci_labs = NCI60['labels']
nci_data = NCI60['data']

```

Mỗi cell line được labeled bằng một cancer type. Ta không dùng cancer types khi perform PCA và clustering, vì đây là unsupervised techniques. Nhưng sau khi thực hiện PCA và clustering, ta sẽ check extent mà các cancer types này agree với results của unsupervised techniques. Data có 64 rows và 6830 columns.

In [48]:

```
nci_data.shape
```

Out[48]: (64, 6830)

Ta bắt đầu bằng examining cancer types của cell lines.

In [49]:

```
nci_labs.value_counts()
```

<sup>10</sup>Giả sử each observation có  $p$  features, mỗi feature là một numerical value. Ta measure similarity của two observations như vậy bằng cách compute correlation của  $p$  pairs of numbers.

```
Out[49]: label
NSCLC 9
RENAL 9
MELANOMA 8
BREAST 7
COLON 7
LEUKEMIA 6
OVARIAN 6
CNS 5
PROSTATE 2
K562A-repro 1
K562B-repro 1
MCF7A-repro 1
MCF7D-repro 1
UNKNOWN 1
dtype: int64
```

### *PCA on the NCI60 Data*

Trước hết ta perform PCA trên data sau khi scale variables (genes) để có standard deviation one, dù ở đây cũng có thể reasonably argue rằng không nên scale genes vì chúng được measured trong cùng units.

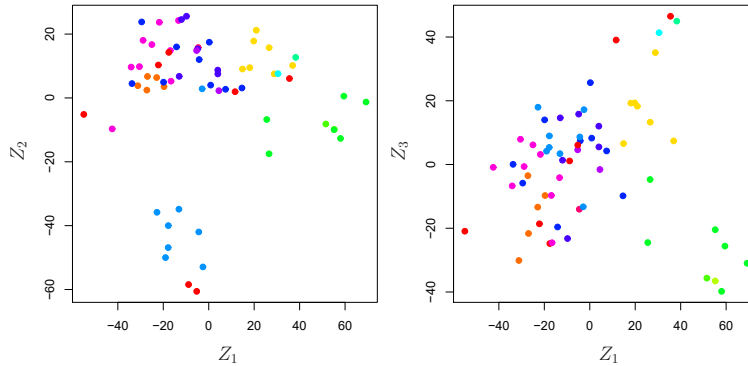
```
In [50]: scaler = StandardScaler()
nci_scaled = scaler.fit_transform(nci_data)
nci_pca = PCA()
nci_scores = nci_pca.fit_transform(nci_scaled)
```

Bây giờ ta plot first few principal component score vectors để visualize data. Observations (cell lines) tương ứng với một given cancer type sẽ được plot cùng color, để xem extent mà observations trong một cancer type similar với nhau.

```
In [51]: cancer_types = list(np.unique(nci_labs))
nci_groups = np.array([cancer_types.index(lab)
 for lab in nci_labs.values])
fig, axes = plt.subplots(1, 2, figsize=(15,6))
ax = axes[0]
ax.scatter(nci_scores[:,0],
 nci_scores[:,1],
 c=nci_groups,
 marker='o',
 s=50)
ax.set_xlabel('PC1'); ax.set_ylabel('PC2')
ax = axes[1]
ax.scatter(nci_scores[:,0],
 nci_scores[:,2],
 c=nci_groups,
 marker='o',
 s=50)
ax.set_xlabel('PC1'); ax.set_ylabel('PC3')
```

Resulting plots được shown trong Figure 12.17. Nhìn chung, cell lines tương ứng với một single cancer type có xu hướng có similar values trên first few principal component score vectors. Điều này cho thấy cell lines thuộc cùng cancer type thường có khá similar gene expression levels.

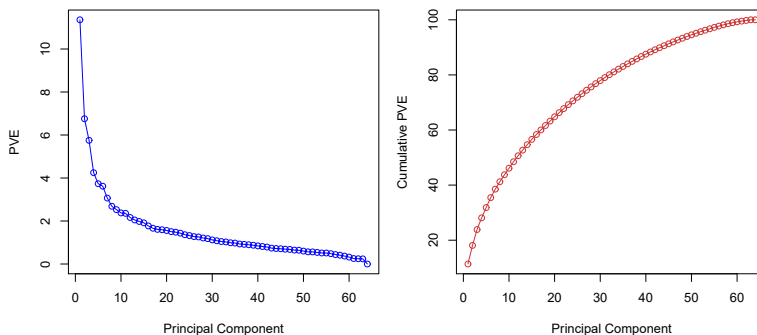
Ta cũng có thể plot percent variance explained bởi principal components cũng như cumulative percent variance explained. Điều này similar với plots đã tạo trước đó cho `USArrests` data.



**HÌNH 12.17.** Projections của NCI60 cancer cell lines lên first three principal components, hay scores của first three principal components. Nhìn chung, observations thuộc cùng một cancer type có xu hướng nằm gần nhau trong low-dimensional space này. Nếu không dùng một dimension reduction method như PCA thì không thể visualize data thuận tiện, vì full data set có  $\binom{6830}{2}$  possible scatterplots và none trong số đó đặc biệt informative.

In [52]:

```
fig, axes = plt.subplots(1, 2, figsize=(15,6))
ax = axes[0]
ticks = np.arange(nci_pca.n_components_)+1
ax.plot(ticks,
 nci_pca.explained_variance_ratio_,
 marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('PVE')
ax = axes[1]
ax.plot(ticks,
 nci_pca.explained_variance_ratio_.cumsum(),
 marker='o')
ax.set_xlabel('Principal Component')
ax.set_ylabel('Cumulative PVE')
```



**HÌNH 12.18.** PVE của principal components của NCI60 cancer cell line microarray data set. Left: PVE của từng principal component. Right: cumulative PVE của principal components. Together, all principal components explain 100% variance.

Resulting plots được shown trong Figure 12.18. Together, first seven

principal components explain khoảng 40% variance trong data. Đây không phải một amount quá lớn. Tuy nhiên, nhìn scree plot, ta thấy mỗi trong first seven principal components giải thích một substantial amount of variance, còn variance explained bởi further principal components giảm rõ rệt. Nói cách khác, có một elbow trong plot sau approximately seventh principal component. Điều này suggests có thể ít benefit khi examine nhiều hơn khoảng seven principal components, dù ngay cả examine seven components cũng có thể khó.

### *Clustering the Observations of the NCI60 Data*

Bây giờ ta perform hierarchical clustering của cell lines trong NCI60 data bằng complete, single, và average linkage. Một lần nữa, goal là tìm xem observations có cluster thành distinct cancer types hay không. Euclidean distance được dùng làm dissimilarity measure. Trước hết ta viết một short function để produce three dendrograms.

In [53]:

```
def plot_nci(linkage, ax, cut=-np.inf):
 cargs = {'above_threshold_color':'black',
 'color_threshold':cut}
 hc = HClust(n_clusters=None,
 distance_threshold=0,
 linkage=linkage.lower()).fit(nci_scaled)
 linkage_ = compute_linkage(hc)
 dendrogram(linkage_,
 ax=ax,
 labels=np.asarray(nci_labs),
 leaf_font_size=10,
 **cargs)
 ax.set_title('%s Linkage' % linkage)
 return hc
```

Hãy plot results.

In [54]:

```
fig, axes = plt.subplots(3, 1, figsize=(15,30))
ax = axes[0]; hc_comp = plot_nci('Complete', ax)
ax = axes[1]; hc_avg = plot_nci('Average', ax)
ax = axes[2]; hc_sing = plot_nci('Single', ax)
```

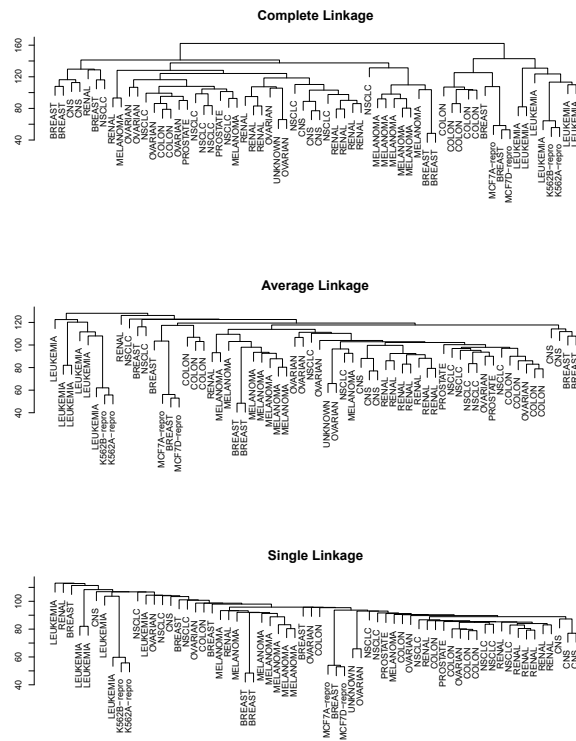
Results shown trong Figure 12.19 cho thấy choice of linkage thực sự affect results. Typically, single linkage tend to yield trailing clusters: very large clusters mà individual observations attach one-by-one. Ngược lại, complete và average linkage tend to yield more balanced, attractive clusters. Vì lý do này, complete và average linkage thường được preferred hơn single linkage. Rõ ràng cell lines trong một single cancer type có xu hướng cluster together, dù clustering không perfect. Ta sẽ dùng complete linkage hierarchical clustering cho analysis tiếp theo.

Ta có thể cut dendrogram ở height cho ra một particular number of clusters, chẳng hạn four:

In [55]:

```
linkage_comp = compute_linkage(hc_comp)
comp_cut = cut_tree(linkage_comp, n_clusters=4).reshape(-1)
pd.crosstab(nci_labs['label'],
 pd.Series(comp_cut.reshape(-1), name='Complete'))
```

Có một số clear patterns. All leukemia cell lines rơi vào one cluster, trong khi breast cancer cell lines spread trên three different clusters. Ta có thể



**HÌNH 12.19.** NCI60 cancer cell line microarray data, clustered bằng average, complete, và single linkage, dùng Euclidean distance làm dissimilarity measure. Complete và average linkage có xu hướng yield evenly sized clusters, trong khi single linkage có xu hướng yield extended clusters mà individual observations được fused one by one.

plot một cut trên dendrogram produce four clusters này:

```
In [56]: fig, ax = plt.subplots(figsize=(10,10))
 plot_nci('Complete', ax, cut=140)
 ax.axhline(140, c='r', linewidth=4)
```

Function `axhline()` draws một horizontal line trên existing axes. Argument 140 plot line tại height 140 trên dendrogram; đây là height result in four distinct clusters. Có thể verify rằng resulting clusters giống các clusters obtained trong `comp_cut`.

Trước đó ở Section 12.4.2 ta đã nói K-means clustering và hierarchical clustering với dendrogram cut để obtain cùng number of clusters có thể yield very different results. NCI60 hierarchical clustering results này compare thế nào với K-means clustering khi  $K = 4$ ?

```
In [57]: nci_kmeans = KMeans(n_clusters=4,
 random_state=0,
 n_init=20).fit(nci_scaled)
 pd.crosstab(pd.Series(comp_cut, name='HClust'),
 pd.Series(nci_kmeans.labels_, name='K-means'))
```

```
Out[57]: K-means 0 1 2 3
 HClust
0 28 3 9 0
1 7 0 0 0
2 0 0 0 8
3 0 9 0 0
```

Ta thấy four clusters obtained bằng hierarchical clustering và K-means clustering somewhat different. Trước hết cluster labels trong two clusterings là arbitrary: swapping identifier của cluster không change clustering. Ở đây Cluster 3 trong K-means clustering identical với cluster 2 trong hierarchical clustering. Nhưng các clusters khác differ; chẳng hạn cluster 0 trong K-means chứa một phần observations assigned to cluster 0 bởi hierarchical clustering, cũng như all observations assigned to cluster 1 bởi hierarchical clustering.

Thay vì hierarchical clustering trên entire data matrix, ta cũng có thể perform hierarchical clustering trên first few principal component score vectors, coi các first few components này như một less noisy version of data.

```
In [58]: hc_pca = HClust(n_clusters=None,
 distance_threshold=0,
 linkage='complete').fit(nci_scores[:, :5])
 linkage_pca = compute_linkage(hc_pca)
 fig, ax = plt.subplots(figsize=(8,8))
 dendrogram(linkage_pca,
 labels=np.asarray(nci_labs),
 leaf_font_size=10,
 ax=ax,
 **cargs)
 ax.set_title("Hier. Clust. on First Five Score Vectors")
 pca_labels = pd.Series(cut_tree(linkage_pca,
 n_clusters=4).reshape(-1),
 name='Complete-PCA')
 pd.crosstab(nci_labs['label'], pca_labels)
```

## 12.6 Exercises

### Conceptual

- Bài này liên quan đến K-means clustering algorithm.
  - Chúng minh (12.18).
  - Dựa trên identity này, hãy lập luận rằng K-means clustering algorithm (Algorithm 12.2) decreases objective (12.17) ở mỗi iteration.
- Giả sử ta có four observations, với dissimilarity matrix

$$\begin{bmatrix} 0 & 0.3 & 0.4 & 0.7 \\ 0.3 & 0 & 0.5 & 0.8 \\ 0.4 & 0.5 & 0 & 0.45 \\ 0.7 & 0.8 & 0.45 & 0 \end{bmatrix}.$$

Chẳng hạn, dissimilarity giữa first và second observations là 0.3, và giữa second và fourth observations là 0.8.

- Dựa trên dissimilarity matrix này, sketch dendrogram resulting from hierarchically clustering four observations bằng complete linkage. Hãy indicate height của mỗi fusion và observations corresponding to mỗi leaf.
  - Repeat (a), lần này dùng single linkage clustering.
  - Giả sử cut dendrogram trong (a) để result in two clusters. Observations nào thuộc từng cluster?
  - Giả sử cut dendrogram trong (b) để result in two clusters. Observations nào thuộc từng cluster?
  - Chapter đã nói rằng tại each fusion trong dendrogram, position của two clusters being fused có thể swap mà không change meaning. Draw một dendrogram equivalent với dendrogram ở (a), trong đó two or more leaves được repositioned nhưng meaning không đổi.
- Trong bài này, bạn sẽ perform K-means clustering manually với  $K = 2$  trên small example có  $n = 6$  observations và  $p = 2$  features. Observations như sau.

| Obs. | $X_1$ | $X_2$ |
|------|-------|-------|
| 1    | 1     | 4     |
| 2    | 1     | 3     |
| 3    | 0     | 4     |
| 4    | 5     | 1     |
| 5    | 6     | 2     |
| 6    | 4     | 0     |

- Plot observations.
- Randomly assign một cluster label cho mỗi observation. Có thể dùng `np.random.choice()`. Report cluster labels cho each observation.

- (c) Compute centroid của mỗi cluster.
  - (d) Assign mỗi observation cho centroid gần nhất theo Euclidean distance. Report cluster labels.
  - (e) Repeat (c) và (d) đến khi answers stop changing.
  - (f) Trong plot ở (a), color observations theo cluster labels obtained.
4. Giả sử với một particular data set, ta perform hierarchical clustering bằng single linkage và complete linkage, obtaining two dendrograms.
- (a) Tại một point trên single linkage dendrogram, clusters  $\{1, 2, 3\}$  và  $\{4, 5\}$  fuse. Trên complete linkage dendrogram, same clusters cũng fuse. Fusion nào occur higher trên tree, chúng occur at same height, hay không đủ information để biết?
  - (b) Tại một point trên single linkage dendrogram, clusters  $\{5\}$  và  $\{6\}$  fuse. Trên complete linkage dendrogram, same clusters cũng fuse. Fusion nào occur higher, same height, hay không đủ information?
5. Bằng lời, describe results bạn expect nếu perform K-means clustering của eight shoppers trong Figure 12.16 dựa trên sock và computer purchases với  $K = 2$ . Give three answers, one cho mỗi variable scaling displayed. Explain.
6. Ta đã thấy ở Section 12.2.2 rằng principal component loading và score vectors provide một approximation cho matrix theo sense của (12.5). Cụ thể, principal component score và loading vectors solve optimization problem (12.6).  
 Bây giờ giả sử  $M$  principal component score vectors  $z_{im}$ ,  $m = 1, \dots, M$ , đã known. Dùng (12.6), explain rằng mỗi trong first  $M$  principal component loading vectors  $\phi_{jm}$ ,  $m = 1, \dots, M$ , có thể obtain bằng cách perform  $p$  separate least squares linear regressions. Trong each regression, principal component score vectors là predictors và one feature của data matrix là response.

### Applied

7. Trong chapter này, ta đã nhắc đến correlation-based distance và Euclidean distance như dissimilarity measures cho hierarchical clustering. Hai measures này gần như equivalent: nếu mỗi observation đã centered để mean zero và standard deviation one, và  $r_{ij}$  là correlation giữa  $i$ th và  $j$ th observations, thì quantity  $1 - r_{ij}$  proportional với squared Euclidean distance giữa hai observations.  
 Trên USArrests data, show proportionality này. *Hint*: Euclidean distance có thể calculate bằng `pairwise_distances()` từ module `sklearn.metrics`, còn correlations bằng `np.corrcoef()`.
8. Ở Section 12.2.3, formula để calculate PVE được cho trong Equation 12.10. Ta cũng thấy PVE có thể obtain bằng attribute `explained_variance_ratio_` của fitted `PCA()` estimator. Trên USArrests data, calculate PVE theo two ways:
- (a) Dùng `explained_variance_ratio_` output của fitted `PCA()` estimator, như trong Section 12.2.3.

*pairwise-  
distances()*



- (b) Apply Equation 12.10 trực tiếp. Loadings được lưu trong attribute `components_` của fitted `PCA()` estimator. Dùng loadings này trong Equation 12.10 để obtain PVE.

Hai approaches nên give same results. *Hint:* Chỉ obtain same results ở (a) và (b) nếu same data được used trong cả hai. Chẳng hạn, nếu ở (a) performed `PCA()` với centered và scaled variables thì phải center và scale variables trước khi apply Equation 12.10 ở (b).

9. Xét `USArrests` data. Bây giờ ta perform hierarchical clustering trên states.
  - (a) Dùng hierarchical clustering với complete linkage và Euclidean distance, cluster states.
  - (b) Cut dendrogram tại một height result in three distinct clusters. States nào thuộc từng cluster?
  - (c) Hierarchically cluster states bằng complete linkage và Euclidean distance sau khi scale variables để có standard deviation one.
  - (d) Scaling variables ảnh hưởng thế nào đến hierarchical clustering obtained? Theo bạn, variables có nên scaled trước khi compute inter-observation dissimilarities không? Justify answer.
10. Trong bài này, bạn sẽ generate simulated data rồi perform PCA và K-means clustering.
  - (a) Generate simulated data set với 20 observations trong mỗi of three classes, tức 60 observations total, và 50 variables. *Hint:* Có nhiều functions trong Python để generate data, chẳng hạn method `normal()` của `random()` trong numpy; `uniform()` là option khác. Hãy add mean shift vào observations trong each class để có three distinct classes.
  - (b) Perform PCA trên 60 observations và plot first two principal component score vectors. Dùng different color để indicate observations trong three classes. Nếu classes appear separated thì continue (c); nếu không, return (a) và modify simulation để separation lớn hơn. Không continue đến (c) cho đến khi three classes show at least some separation trong first two score vectors.
  - (c) Perform K-means clustering với  $K = 3$ . Clusters obtained compare thế nào với true class labels? *Hint:* Có thể dùng `pd.crosstab()` để compare labels. Hãy cẩn thận khi interpret: K-means arbitrarily numbers clusters, nên không thể simply check true labels equal clustering labels.
  - (d) Perform K-means clustering với  $K = 2$ . Describe results.
  - (e) Perform K-means clustering với  $K = 4$ , và describe results.
  - (f) Perform K-means clustering với  $K = 3$  trên first two principal component score vectors thay vì raw data; tức cluster  $60 \times 2$  matrix whose first column là first PC score vector và second column là second PC score vector. Comment on results.
  - (g) Dùng `StandardScaler()` estimator, perform K-means clustering với  $K = 3$  sau khi scale each variable để standard deviation one.

Results compare thế nào với (b)? Explain.

11. Viết một Python function để perform matrix completion như Algorithm 12.1 và như outlined trong Section 12.5.2. Trong mỗi iteration, function phải keep track relative error và iteration count. Iterations tiếp tục cho đến khi relative error đủ small hoặc đạt some maximum number of iterations; hãy set một default value cho maximum. Ngoài ra phải có option print progress ở mỗi iteration.  
Test function trên **Boston** data. Trước hết standardize features để mean zero và standard deviation one bằng **StandardScaler()**. Run experiment trong đó randomly leave out an increasing và nested number of observations từ 5% đến 30%, step 5%. Apply Algorithm 12.1 với  $M = 1, 2, \dots, 8$ . Display approximation error như function của fraction observations missing và value of  $M$ , averaged over 10 repetitions.
12. Trong Section 12.5.2, Algorithm 12.1 được implemented bằng function **svd()** từ module **np.linalg**. Tuy nhiên, dựa trên connection giữa **svd()** và **PCA()** highlighted trong lab, ta có thể implement algorithm bằng **PCA()** thay thế. Viết một function implement Algorithm 12.1 dùng **PCA()** thay vì **svd()**.
13. Trên book website, [www.statlearning.com](http://www.statlearning.com), có gene expression data set **Ch12Ex13.csv** gồm 40 tissue samples với measurements trên 1,000 genes. First 20 samples từ healthy patients, second 20 từ diseased group.
  - (a) Load data bằng **pd.read\_csv()**. Cần select **header=None**.
  - (b) Apply hierarchical clustering cho samples bằng correlation-based distance và plot dendrogram. Genes có separate samples thành two groups không? Results có depend on linkage type không?
  - (c) Collaborator muốn biết genes nào differ nhiều nhất across two groups. Suggest một way để answer và apply ở đây.

# 13

## Multiple Testing

Cho đến đây, textbook chủ yếu tập trung vào estimation và người họ hàng gần của nó là prediction. Trong chapter này, ta chuyển sang hypothesis testing, một thành phần then chốt của inference. Nhắc lại rằng inference đã được discuss ngắn gọn ở Chapter 2.

Section 13.1 cung cấp một brief review về null hypotheses, p-values, test statistics, và các key ideas khác trong hypothesis testing. Tuy nhiên, chapter này assumes reader đã từng tiếp xúc với các topics đó. Cụ thể, ta sẽ không tập trung vào lý do tại sao hoặc cách conduct một hypothesis test — một topic mà cả cuốn sách có thể, và đã, được viết về nó. Thay vào đó, ta assume reader muốn test một particular set of null hypotheses và đã có một specific plan về cách conduct tests và obtain p-values.

Phần lớn emphasis trong classical statistics tập trung vào testing một single null hypothesis. Chẳng hạn, ta có thể test  $H_0$ : expected blood pressure của mice trong control group bằng expected blood pressure của mice trong treatment group. Dĩ nhiên, ta có thể hy vọng discover rằng mean blood pressure của two groups differs. Nhưng vì reasons sẽ sớm rõ, ta construct null hypothesis tương ứng với no difference.

Trong contemporary settings, ta thường đối mặt với huge amounts of data và vì thế có thể muốn test rất nhiều null hypotheses. Chẳng hạn, thay vì chỉ test  $H_0$ , ta có thể muốn test  $m$  null hypotheses  $H_{01}, \dots, H_{0m}$ , trong đó  $H_{0j}$  states rằng expected value của  $j$ th biomarker trong control group bằng expected value của  $j$ th biomarker trong treatment group. Khi conduct multiple testing, ta cần rất careful về cách interpret results để tránh erroneously rejecting far too many null hypotheses.

Chapter này discusses cả classical lẫn more contemporary ways để conduct multiple testing trong big-data setting. Section 13.2 highlights challenges associated với multiple testing. Classical solutions cho các challenges này được trình bày ở Section 13.3, còn more contemporary solutions ở Sections 13.4 và 13.5.

Đặc biệt, Section 13.4 tập trung vào false discovery rate. Notion này có từ thập niên 1990 và nhanh chóng trở nên popular vào đầu những năm 2000, khi large-scale datasets bắt đầu xuất hiện trong genomics. Các datasets đó đặc biệt không chỉ vì size lớn,<sup>1</sup> mà còn vì chúng thường được collected cho exploratory purposes: researchers collected chúng để test một huge number

---

<sup>1</sup>Microarray data từng được xem là “big data” vào thời điểm đó, dù theo standards ngày nay label này nghe khá khiêm tốn: một khá khiếm tốn: một microarray dataset có thể, và thường đã, được lưu trong một Microsoft Excel spreadsheet.

of null hypotheses, thay vì chỉ một very small number of pre-specified null hypotheses. Ngày nay, huge datasets được collected mà không có pre-specified null hypothesis trong gần như every field. Như ta sẽ thấy, false discovery rate rất well-suited với modern-day reality này.

Chapter này naturally centers on classical statistical technique của p-values, được dùng để quantify results của hypothesis tests. Vào thời điểm cuốn sách được viết (2020), p-values gần đây đã trở thành topic của extensive commentary trong social science research community, đến mức một số social science journals thậm chí ban việc dùng p-values. Ta chỉ comment rằng, khi được understand và apply đúng, p-values là một powerful tool để draw inferential conclusions từ data.

## 13.1 A Quick Review of Hypothesis Testing

Hypothesis tests provide một rigorous statistical framework để answer các simple “yes-or-no” questions về data, chẳng hạn:

1. True coefficient  $\beta_j$  trong linear regression của  $Y$  lên  $X_1, \dots, X_p$  có bằng zero không?<sup>2</sup>
2. Có difference trong expected blood pressure của laboratory mice ở control group và laboratory mice ở treatment group không?<sup>3</sup>

Trong Section 13.1.1, ta briefly review các steps involved trong hypothesis testing. Section 13.1.2 discusses different types of mistakes, hay errors, có thể occur trong hypothesis testing.

### 13.1.1 Testing a Hypothesis

Conducting một hypothesis test thường proceed qua four steps. First, define null và alternative hypotheses. Next, construct một test statistic summarize strength of evidence against null hypothesis. Sau đó compute p-value quantifies probability of having obtained một comparable hoặc more extreme value của test statistic under null hypothesis. Finally, dựa trên p-value, decide whether to reject null hypothesis. Ta briefly discuss từng step.

#### Step 1: Define the Null and Alternative Hypotheses

Trong hypothesis testing, ta divide world thành two possibilities: null hypothesis và alternative hypothesis. Null hypothesis, denoted  $H_0$ , là default state of belief về world.<sup>4</sup> Chẳng hạn, null hypotheses corresponding với two questions posed earlier là:

1. True coefficient  $\beta_j$  trong linear regression của  $Y$  lên  $X_1, \dots, X_p$  equals zero.
2. Không có difference giữa expected blood pressure của mice trong control và treatment groups.

null hypothesis;  
alternative  
hypothesis

<sup>2</sup>Hypothesis test này đã được discuss ở page 76 của Chapter 3.

<sup>3</sup>“Treatment group” là set of mice nhận experimental treatment, còn “control group” là những con không nhận treatment.

<sup>4</sup> $H_0$  được phát âm là “H naught” hoặc “H zero”.

Null hypothesis “boring” by construction: nó hoàn toàn có thể true, nhưng ta có thể hy vọng data sẽ tell us otherwise. Alternative hypothesis, denoted  $H_a$ , represents something different và unexpected; chẳng hạn expected blood pressure của mice trong two groups differs. Typically, alternative hypothesis simply posits rằng null hypothesis does not hold: nếu null nói A và B không khác nhau, alternative nói chúng khác nhau.

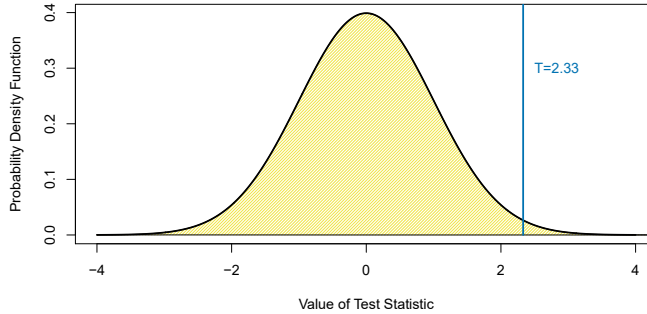
Điều quan trọng là treatment của  $H_0$  và  $H_a$  asymmetric.  $H_0$  được treated như default state of world, và ta focus on using data to reject  $H_0$ . Nếu reject  $H_0$ , điều đó provides evidence in favor of  $H_a$ . Có thể think of rejecting  $H_0$  như making a discovery about data: ta discover rằng  $H_0$  does not hold. By contrast, nếu fail to reject  $H_0$ , findings more nebulous: ta không biết có phải sample size quá small — trong trường hợp đó testing lại trên larger hoặc higher-quality dataset có thể lead to rejection — hay vì  $H_0$  thực sự holds.

### Step 2: Construct the Test Statistic

Tiếp theo, ta muốn use data để find evidence for or against null hypothesis. Để làm vậy, ta compute một test statistic, denoted  $T$ , summarize extent mà data consistent với  $H_0$ . Cách construct  $T$  depends on nature của null hypothesis đang test.

Để concrete, let  $x_1^t, \dots, x_{n_t}^t$  là blood pressure measurements cho  $n_t$  mice trong treatment group, và  $x_1^c, \dots, x_{n_c}^c$  cho  $n_c$  mice trong control group; đặt  $\mu_t = E(X^t)$  và  $\mu_c = E(X^c)$ . Để test  $H_0 : \mu_t = \mu_c$ , ta dùng two-sample t-statistic.<sup>5</sup>

test statistic;  
two-sample  
t-statistic



**HÌNH 13.1.** Density function của  $N(0,1)$  distribution, với vertical line tại value 2.33. Có 1% area under curve nằm bên phải line, nên chỉ có 2% chance quan sát một  $N(0,1)$  value lớn hơn 2.33 hoặc nhỏ hơn  $-2.33$ . Vì vậy, nếu một test statistic có  $N(0,1)$  null distribution, observed test statistic  $T = 2.33$  dẫn đến p-value 0.02.

$$T = \frac{\hat{\mu}_t - \hat{\mu}_c}{s \sqrt{\frac{1}{n_t} + \frac{1}{n_c}}}, \quad (13.1)$$

where  $\hat{\mu}_t = n_t^{-1} \sum_{i=1}^{n_t} x_i^t$ ,  $\hat{\mu}_c = n_c^{-1} \sum_{i=1}^{n_c} x_i^c$ , and

$$s = \sqrt{\frac{(n_t - 1)s_t^2 + (n_c - 1)s_c^2}{n_t + n_c - 2}}, \quad (13.2)$$

<sup>5</sup>T-statistic lấy tên từ fact rằng dưới  $H_0$ , nó follows một t-distribution.

is an estimator of pooled standard deviation của two samples.<sup>6</sup> Ở đây  $s_t^2$  và  $s_c^2$  là unbiased estimators của variance của blood pressure trong treatment và control groups. Một large absolute value của  $T$  provides evidence against  $H_0 : \mu_t = \mu_c$ , và therefore evidence in support of  $H_a : \mu_t \neq \mu_c$ .

### Step 3: Compute the p-Value

Ở previous section, ta noted rằng large absolute value của two-sample t-statistic provides evidence against  $H_0$ . Câu hỏi ngay lập tức là: large đến mức nào mới là large? Nói cách khác, một given value của test statistic provides bao nhiêu evidence against  $H_0$ ?

Notion của p-value cho ta cách formalize và answer question này. P-value được defined là probability of observing một test statistic equal to or more extreme than observed statistic, under assumption rằng  $H_0$  thật sự true. Vì vậy small p-value provides evidence against  $H_0$ .

Để concrete, suppose  $T = 2.33$  cho test statistic trong (13.1). Ta ask: nếu  $H_0$  holds, probability quan sát such a large value của  $T$  là bao nhiêu? Under  $H_0$ , distribution của  $T$  trong (13.1) approximately follows một  $N(0, 1)$  distribution<sup>7</sup> — normal distribution với mean 0 và variance 1. Figure 13.1 shows distribution này. Có 98% của  $N(0, 1)$  distribution nằm between  $-2.33$  và  $2.33$ , nghĩa là under  $H_0$  ta expect see such a large  $|T|$  chỉ 2% of the time. Vì vậy p-value corresponding to  $T = 2.33$  là 0.02.

Distribution của test statistic under  $H_0$ , còn gọi là null distribution của test statistic, depends on details của type of null hypothesis và type of test statistic. Nói chung, most commonly-used test statistics follow một well-known statistical distribution under null hypothesis — chẳng hạn normal, t-,  $\chi^2$ -, hoặc F-distribution — provided sample size sufficiently large và some other assumptions hold. Typically, R function dùng để compute test statistic sẽ use null distribution này để output p-value. Trong Section 13.5 ta sẽ thấy một approach estimate null distribution bằng re-sampling; trong many contemporary settings, đây là attractive option vì uses fast computers để avoid potentially problematic assumptions về data.

P-value có lẽ là một trong các notions được dùng và abused nhiều nhất trong statistics. Đôi khi người ta nói p-value là probability that  $H_0$  holds, tức probability null hypothesis true. Điều này incorrect. One and only correct interpretation của p-value là fraction of the time ta expect see such an extreme value của test statistic nếu repeated experiment many, many times, provided  $H_0$  holds.<sup>8</sup>

Trong Step 2 ta computed test statistic và noted large absolute value provides evidence against  $H_0$ . Trong Step 3, test statistic converted thành p-value, với small p-values providing evidence against  $H_0$ . Vậy conversion

<sup>6</sup>Equation (13.2) assumes control và treatment groups have equal variance. Without assumption này, (13.2) có một slightly different form.

<sup>7</sup>Precisely hơn, assuming observations drawn from a normal distribution,  $T$  follows a t-distribution với  $n_t + n_c - 2$  degrees of freedom. Provided  $n_t + n_c - 2$  larger than around 40, distribution này very well approximated by  $N(0, 1)$ . Trong Section 13.5 ta sẽ thấy alternative, thường attractive hơn, để approximate null distribution của  $T$  mà không cần stringent assumptions về data.

<sup>8</sup>Một one-sided p-value là probability of seeing such an extreme value của test statistic, ví dụ probability of a test statistic greater than or equal to  $T = 2.33$ . Một two-sided p-value là probability of seeing such an extreme absolute test statistic, ví dụ greater than or equal to 2.33 hoặc less than or equal to  $-2.33$ . Default recommendation là report two-sided p-value trừ khi có clear and compelling reason chỉ một direction có scientific interest.

p-value

null distribution

này accomplish gì? Suppose data analyst conducts statistical test và reports  $T = 17.3$ . Value này provide strong evidence against  $H_0$  không? Không thể biết without more information, đặc biệt cần biết expected value của test statistic under  $H_0$ . Chính p-value cho ta information đó. Nói cách khác, p-value transforms test statistic từ một arbitrary và hard-to-interpret scale thành number between 0 and 1 để interpret hơn.

#### Step 4: Decide Whether to Reject the Null Hypothesis

Sau khi compute p-value corresponding to  $H_0$ , ta cần decide whether to reject  $H_0$ . Ta thường không nói “accepting”  $H_0$ ; thay vào đó nói “failing to reject”  $H_0$ . Small p-value indicates such a large test statistic unlikely under  $H_0$  và thereby provides evidence against it. Nếu p-value sufficiently small, ta muốn reject  $H_0$ , tức make a “discovery”. Nhưng small đến mức nào mới đủ?

Answer depends heavily on data analyst. Smaller p-value means stronger evidence against  $H_0$ . Trong some fields, typical rule là reject  $H_0$  nếu p-value below 0.05, nghĩa là nếu  $H_0$  holds, ta expect such a small p-value no more than 5% of the time.<sup>9</sup> Trong other fields, burden of proof much higher: chẳng hạn some areas of physics chỉ reject  $H_0$  nếu p-value below  $10^{-9}$ .

Trong example Figure 13.1, nếu dùng threshold 0.05 làm cut-off thì reject null; nếu threshold 0.01 thì fail to reject. Các ideas này được formalize trong next section.

#### 13.1.2 Type I and Type II Errors

Nếu null hypothesis holds, ta gọi nó là true null hypothesis; otherwise là false null hypothesis. Chẳng hạn, nếu test  $H_0 : \mu_t = \mu_c$  như Section 13.1.1 và thật sự không có difference trong population mean blood pressure giữa mice treatment và control groups,  $H_0$  true; otherwise false. Dĩ nhiên a priori ta không biết  $H_0$  true hay false; đó là lý do cần hypothesis test.

true null  
hypothesis; false  
null hypothesis

| Decision            | Truth        |               |
|---------------------|--------------|---------------|
|                     | $H_0$        | $H_a$         |
| Reject $H_0$        | Type I Error | Correct       |
| Do Not Reject $H_0$ | Correct      | Type II Error |

**BẢNG 13.1.** Summary của possible scenarios associated với testing null hypothesis  $H_0$ . Type I errors còn gọi là false positives, và Type II errors là false negatives.

Table 13.1 summarizes possible scenarios associated với testing  $H_0$ .<sup>10</sup> Sau hypothesis test, row của table known dựa trên whether ta rejected  $H_0$ ; nhưng impossible biết column nào là truth. Nếu reject  $H_0$  khi false, tức  $H_a$  true, hoặc do not reject  $H_0$  khi true, ta arrive at correct result. Nếu erroneously reject  $H_0$  khi actually true, ta commit Type I error. Type I error rate là probability of making a Type I error given  $H_0$  holds, tức

<sup>9</sup>Threshold 0.05 để reject  $H_0$  rất ubiquitous trong some areas of science, nhưng authors advise against blind adherence to arbitrary choice này. Data analyst typically nên report p-value itself thay vì chỉ report whether nó exceeds một threshold.

<sup>10</sup>Có parallels giữa Table 13.1 và Table 4.6 về output của binary classifier. Recall rằng false positive từ Table 4.6 results from predicting positive (non-null) label khi true label actually negative (null). Điều này closely related to Type I error, xảy ra khi rejecting null hypothesis trong khi null actually holds.

probability of incorrectly rejecting  $H_0$ . Nếu do not reject  $H_0$  khi actually false, ta commit Type II error. Power của hypothesis test là probability of not making Type II error given  $H_a$  holds, tức probability of correctly rejecting  $H_0$ .

Type I error; Type  
I error rate; Type  
II error; power

Ideally, ta muốn cả Type I và Type II error rates small. Nhưng trong practice khó đạt cả hai. Usually có trade-off: có thể make Type I error small bằng cách chỉ reject  $H_0$  khi very sure it does not hold, nhưng điều đó increases Type II error. Hoặc make Type II error small bằng cách reject  $H_0$  ngay khi có modest evidence against it, nhưng Type I error sẽ large. Trong practice, Type I errors thường được xem serious hơn Type II errors vì former involves declaring một scientific finding không correct. Vì vậy hypothesis testing thường require low Type I error rate, ví dụ at most  $\alpha = 0.05$ , trong khi cố make Type II error small, equivalently power large.

Có direct correspondence giữa p-value threshold causing rejection của  $H_0$  và Type I error rate. Nếu chỉ reject  $H_0$  khi p-value below  $\alpha$ , ta ensure Type I error rate less than or equal to  $\alpha$ .

## 13.2 The Challenge of Multiple Testing

Ở previous section, ta thấy reject  $H_0$  khi p-value below, say, 0.01 cho một simple way to control Type I error cho  $H_0$  at level 0.01: nếu  $H_0$  true, probability reject nó no more than 1%. Nhưng suppose ta muốn test  $m$  null hypotheses  $H_{01}, \dots, H_{0m}$ . Có đủ chỉ reject tất cả null hypotheses có corresponding p-value below 0.01 không? Nói cách khác, nếu reject tất cả hypotheses có p-value below 0.01, ta expect make bao nhiêu Type I errors?

Bước đầu để answer question, consider một stockbroker muốn attract new clients bằng cách convince họ rằng she has trading acumen. She tells 1,024 potential clients —  $1,024 = 2^{10}$  — rằng có thể correctly predict whether Apple's stock price will increase or decrease trong 10 days liên tiếp. Có  $2^{10}$  possibilities cho sequence tăng/giảm qua 10 days. Vì vậy she emails mỗi client one trong  $2^{10}$  possibilities. Vast majority sẽ thấy predictions no better than chance, thậm chí worse. Nhưng a broken clock is right twice a day, và one potential client sẽ very impressed vì predictions đúng all 10 days, nên stockbroker gains a new client.

Điều gì happened? Stockbroker có actual insight nào về Apple stock price không? Không. Vậy làm sao she predicted perfectly trong 10 days? Answer là she made a lot of guesses và one guess happened to be exactly right.

Điều này liên quan multiple testing thế nào? Suppose ta flip 1,024 fair coins<sup>11</sup> ten times each. Ta expect on average one coin come up all tails. Probability một individual coin cho all tails là  $1/2^{10} = 1/1,024$ , nên với 1,024 coins ta expect one all-tails coin on average. Nếu one coin all tails, ta có thể conclude particular coin không fair. In fact, một standard hypothesis test cho null rằng coin fair cho p-value below 0.002.<sup>12</sup> Nhưng conclude coin unfair sẽ incorrect: null hypothesis actually holds; ta chỉ happened to get ten tails in a row by chance.

<sup>11</sup>Fair coin là coin có equal chance landing heads hoặc tails.

<sup>12</sup>Recall p-value là probability of observing data at least this extreme under null. Nếu coin fair, probability observing at least ten tails là  $(1/2)^{10} = 1/1,024 < 0.001$ . P-value therefore  $2/1,024 < 0.002$ , vì đây là probability of observing ten heads hoặc ten tails.



Các examples này illustrate main challenge của multiple testing: khi testing huge number of null hypotheses, ta bound to get some very small p-values by chance. Nếu decide whether to reject each null independently mà không account for fact rằng đã performed very large number of tests, ta có thể reject great number of true null hypotheses — tức make many Type I errors.

multiple testing

Problem severe đến đâu? Recall nếu reject single  $H_0$  khi p-value less than, say,  $\alpha = 0.01$ , thì có 1% chance of false rejection if  $H_0$  true. Bây giờ nếu test  $m$  null hypotheses  $H_{01}, \dots, H_{0m}$ , all true, thì mỗi individual null có 1% chance bị rejected; do đó expect falsely reject approximately  $0.01 \times m$  null hypotheses. Nếu  $m = 10,000$ , expect falsely reject 100 hypotheses by chance. Đó là rất nhiều Type I errors.

Crux của issue là: rejecting một null hypothesis khi p-value below  $\alpha$  controls probability of falsely rejecting that particular null at level  $\alpha$ . Tuy nhiên nếu làm vậy cho  $m$  null hypotheses, chance of falsely rejecting at least one trong  $m$  hypotheses sẽ much higher.

|                     | $H_0$ is True | $H_0$ is False | Total   |
|---------------------|---------------|----------------|---------|
| Reject $H_0$        | $V$           | $S$            | $R$     |
| Do Not Reject $H_0$ | $U$           | $W$            | $m - R$ |
| Total               | $m_0$         | $m - m_0$      | $m$     |

**BẢNG 13.2.** Summary của results khi testing  $m$  null hypotheses. Một given null hypothesis either true or false, và một test có thể reject hoặc fail to reject nó. Trong practice, individual values của  $V$ ,  $S$ ,  $U$ , và  $W$  unknown. Tuy nhiên ta observe  $V + S = R$  và  $U + W = m - R$ , là numbers of null hypotheses rejected và not rejected.

Ta sẽ investigate issue này chi tiết hơn và đưa ra một solution trong Section 13.3.

### 13.3 The Family-Wise Error Rate

Trong các section tiếp theo, ta sẽ discuss việc testing multiple hypotheses đồng thời control probability của việc mắc ít nhất một Type I error.

#### 13.3.1 What is the Family-Wise Error Rate?

Nhắc lại rằng Type I error rate là probability of rejecting  $H_0$  khi  $H_0$  true. Family-wise error rate (FWER) generalizes notion này sang setting có  $m$  null hypotheses,  $H_{01}, \dots, H_{0m}$ , và được defined là probability of making at least one Type I error. Dùng notation trong Table 13.2,  $V$  là number of Type I errors (còn gọi là false positives hay false discoveries),  $S$  là number of true positives,  $U$  là number of true negatives, và  $W$  là number of Type II errors (false negatives). Khi đó

$$\text{FWER} = \Pr(V \geq 1). \quad (13.3)$$

Một strategy là reject bất kỳ null hypothesis nào có p-value dưới  $\alpha$  — tức control Type I error của từng test ở level  $\alpha$ . Strategy này dẫn đến

$$\text{FWER}(\alpha) = 1 - \Pr(V = 0) \quad (13.3)$$

$$= 1 - \Pr(\text{không falsely reject bất kỳ null hypothesis nào}) \quad (13.4)$$

$$= 1 - \Pr\left(\bigcap_{j=1}^m \{\text{không falsely reject } H_{0j}\}\right). \quad (13.4)$$

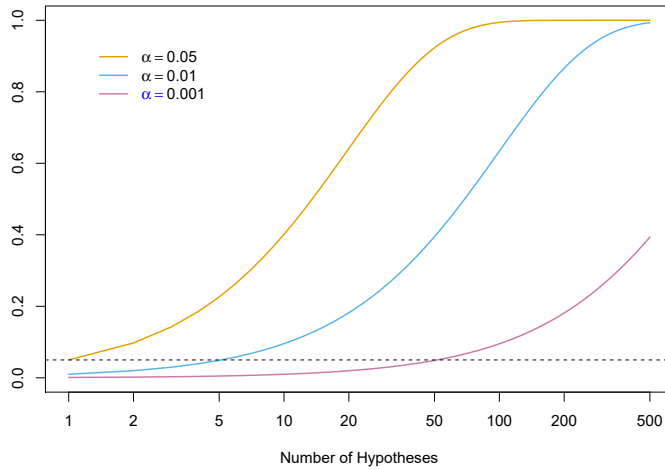
Nhắc lại rằng nếu two events  $A$  và  $B$  independent thì  $\Pr(A \cap B) = \Pr(A)\Pr(B)$ . Vì vậy, nếu thêm hai assumptions khá mạnh rằng  $m$  tests independent và tất cả  $m$  null hypotheses đều true, thì

$$\text{FWER}(\alpha) = 1 - \prod_{j=1}^m (1 - \alpha) = 1 - (1 - \alpha)^m. \quad (13.5)$$

Do đó, nếu chỉ test một null hypothesis thì  $\text{FWER}(\alpha) = \alpha$ , nên Type I error rate và FWER bằng nhau. Nhưng nếu perform  $m = 100$  independent tests, thì với  $\alpha = 0.05$  ta có  $1 - (1 - 0.05)^{100} = 0.994$ : gần như guaranteed sẽ mắc ít nhất một Type I error.

Figure 13.2 displays (13.5) cho nhiều values của  $m$  và  $\alpha$ . Setting  $\alpha = 0.05$  tạo high FWER ngay cả với moderate  $m$ . Với  $\alpha = 0.01$ , ta test được không quá five null hypotheses trước khi FWER vượt 0.05. Chỉ với very small values như  $\alpha = 0.001$  mới giữ được small FWER cho moderately-sized  $m$ .

Quay lại example ở Section 13.1.1, ta test  $H_0 : \mu_t = \mu_c$  bằng two-sample t-statistic. Figure 13.1 cho thấy để Type I error không exceed 0.02, ta reject khi  $|T| \geq 2.33$ . Nếu thay vì one test, ta test 10 null hypotheses, Section 13.3.2 sẽ cho thấy có thể guarantee FWER không exceed 0.02 bằng cách chỉ reject p-values dưới 0.002. Điều này tương ứng cutpoint much more stringent là 3.09: reject  $H_{0j}$  chỉ nếu  $|T_j| \geq 3.09$ . Nói cách khác, control FWER ở level  $\alpha$  đặt một bar cao hơn nhiều về evidence cần để reject từng null hypothesis so với chỉ control Type I error của từng test ở level  $\alpha$ .



**HÌNH 13.2.** Family-wise error rate như một function của number of hypotheses tested (displayed trên log scale), cho ba values của  $\alpha$ : 0.05, 0.01, và 0.001. Dashed line chỉ 0.05. Chẳng hạn, để control FWER ở 0.05 khi testing  $m = 50$  null hypotheses, ta phải control Type I error của từng null hypothesis ở level  $\alpha = 0.001$ .

| Manager | Mean, $\bar{x}$ | Standard Deviation, $s$ | t-statistic | p-value |
|---------|-----------------|-------------------------|-------------|---------|
| One     | 3.0             | 7.4                     | 2.86        | 0.006   |
| Two     | -0.1            | 6.9                     | -0.10       | 0.918   |
| Three   | 2.8             | 7.5                     | 2.62        | 0.012   |
| Four    | 0.5             | 6.7                     | 0.53        | 0.601   |
| Five    | 0.3             | 6.8                     | 0.31        | 0.756   |

**BẢNG 13.3.** Hai columns đầu là sample mean và sample standard deviation của percentage excess return trong  $n = 50$  months cho five managers đầu tiên của **Fund** dataset. Hai columns cuối cho t-statistic  $\sqrt{n} \bar{X}/S$  và p-value associated với test  $H_{0j} : \mu_j = 0$ .

### 13.3.2 Approaches to Control the Family-Wise Error Rate

Ta briefly survey một số approaches để control FWER, minh họa bằng **Fund** dataset, nơi monthly percentage excess returns được recorded cho 2,000 fund managers qua  $n = 50$  months.<sup>13</sup> Table 13.3 cung cấp summary statistics cho five managers đầu tiên.

Trước hết là Bonferroni method và Holm's step-down procedure, hai general-purpose approaches có thể apply whenever  $m$  p-values đã computed, bất kể form của null hypotheses, choice of test statistics, hay dependence giữa p-values. Sau đó ta briefly discuss Tukey's method và Scheffé's method để minh họa rằng trong certain situations, specialized approaches có thể preferable.

#### The Bonferroni Method

Giả sử ta muốn test  $H_{01}, \dots, H_{0m}$ . Gọi  $A_j$  là event rằng ta make a Type

<sup>13</sup>Excess return là phần return mà fund manager đạt thêm beyond overall market return. Nếu market tăng 5% trong một period và manager đạt 7%, excess return là  $7\% - 5\% = 2\%$ .

I error cho  $j$ th null hypothesis. Khi đó

$$\text{FWER} = \Pr(\text{falsely reject at least one null hypothesis}) \quad (13.5)$$

$$= \Pr\left(\bigcup_{j=1}^m A_j\right) \quad (13.6)$$

$$\leq \sum_{j=1}^m \Pr(A_j). \quad (13.6)$$

Inequality này theo union bound và không require independence. Bonferroni method, hay Bonferroni correction, đặt threshold cho each test là  $\alpha/m$ , nên  $\Pr(A_j) \leq \alpha/m$  và vì thế  $\text{FWER}(\alpha/m) \leq m(\alpha/m) = \alpha$ .

Chẳng hạn, để control FWER ở 0.1 với  $m = 100$ , Bonferroni yêu cầu Type I error của từng null hypothesis ở 0.001, tức reject p-values dưới 0.001.

Trong Table 13.3, nếu control Type I error ở  $\alpha = 0.05$  cho từng manager riêng rẽ, ta reject  $H_{01} : \mu_1 = 0$  và  $H_{03} : \mu_3 = 0$ . Nhưng procedure này không account for multiple testing nên FWER exceeds 0.05. Với Bonferroni để control FWER ở 0.05, threshold là  $0.05/5 = 0.01$ ; khi đó chỉ Manager One bị reject. Bonferroni gives peace of mind về false rejections, nhưng price là reject ít hypotheses và thường incur nhiều Type II errors.

Bonferroni là multiplicity correction nổi tiếng và commonly used nhất, nhờ dễ hiểu, dễ implement, và controls Type I error regardless of dependence. Tuy nhiên nó thường không phải most powerful hay best approach. Bonferroni có thể khá conservative: true FWER thường thấp hơn nominal FWER do inequality trong (13.6). Một less conservative procedure có thể control FWER trong khi reject nhiều hypotheses hơn và do đó make fewer Type II errors.

### *Holm's Step-Down Procedure*

Holm's method, còn gọi Holm's step-down procedure hay Holm-Bonferroni method, là alternative cho Bonferroni. Nó controls FWER nhưng less conservative, nên thường reject more null hypotheses, tạo fewer Type II errors và greater power. Procedure nằm trong Algorithm 13.1. Threshold dùng để reject từng hypothesis phụ thuộc vào all  $m$  p-values, khác với Bonferroni. Holm không assume independence và uniformly more powerful than Bonferroni.

*Holm's method*

#### **Algorithm 13.1 Holm's Step-Down Procedure to Control the FWER**

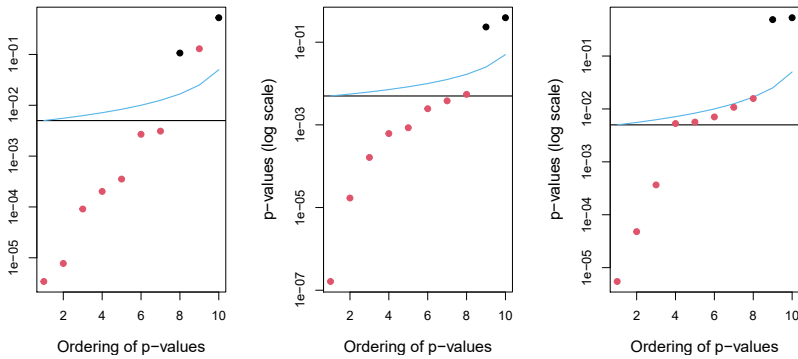
1. Specify  $\alpha$ , level cần control FWER.
2. Compute p-values  $p_1, \dots, p_m$  cho  $H_{01}, \dots, H_{0m}$ .
3. Order chúng:  $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$ .
4. Define

$$L = \min \left\{ j : p_{(j)} > \frac{\alpha}{m+1-j} \right\}. \quad (13.7)$$

5. Reject mọi  $H_{0j}$  sao cho  $p_j < p_{(L)}$ .

Áp dụng Holm cho five managers trong Table 13.3 với FWER 0.05: ordered p-values là 0.006, 0.012, 0.601, 0.756, 0.918. Ta có  $0.006 < 0.01$  và  $0.012 < 0.0125$ , nhưng  $0.601 > 0.0167$ , nên  $L = 3$ . Holm reject null hypotheses cho Managers One và Three, trong khi Bonferroni chỉ reject Manager One.

Figure 13.3 illustrates Bonferroni và Holm trên three simulated datasets với  $m = 10$  tests, trong đó  $m_0 = 2$  null hypotheses true. Ten p-values được ordered và plotted trên log scale; red points là false nulls, black points là true nulls. Để control FWER ở 0.05, Bonferroni reject dưới black horizontal line, Holm reject dưới blue line. Blue line luôn nằm above black line. Trong left panel cả hai reject 7/8 false nulls; center panel Holm reject cả 8 còn Bonferroni miss one; right panel Bonferroni chỉ reject three còn Holm reject all eight. Neither makes Type I errors trong examples này.



**HÌNH 13.3.** Mỗi panel hiển thị sorted p-values của  $m = 10$  null hypotheses trong một simulation. Hai true null hypotheses ( $m_0 = 2$ ) màu black, còn lại red. Khi control FWER ở 0.05, Bonferroni reject dưới black line, Holm reject dưới blue line. Region giữa hai lines là hypotheses được Holm reject nhưng Bonferroni không reject. Center panel: Holm reject thêm one null hypothesis; right panel: thêm five.

### Two Special Cases: Tukey's Method and Scheffé's Method

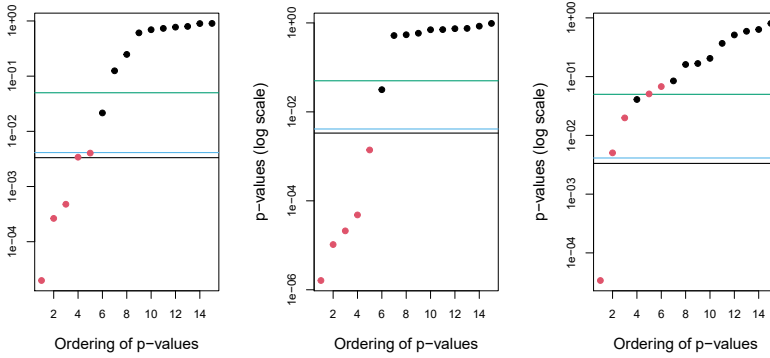
Bonferroni và Holm có thể dùng trong gần như mọi setting vì không assume nature của null hypotheses, test statistic, hay dependence. Tuy vậy, trong very specific settings, tailored approaches có thể achieve higher power. Tukey's method và Scheffé's method là two examples.

Table 13.3 cho thấy Managers One và Two có largest difference trong sample mean returns. Nếu ta test  $H_0: \mu_1 = \mu_2$  bằng two-sample t-test, p-value là 0.0349, có vẻ là modest evidence. Nhưng p-value này misleading vì comparison chỉ được chosen after examining all five managers; effectively ta đã perform  $m = 5(5 - 1)/2 = 10$  pairwise tests và select smallest p-value. Bonferroni ở FWER 0.05 would require p-value below 0.005, nên không reject được equality của Managers One và Two.

Trong setting này Bonferroni hơi too stringent vì 10 pairwise tests related với nhau; p-values không independent. Tukey's method exploits structure này: khi perform  $m = G(G - 1)/2$  pairwise comparisons của  $G$  means, nó cho threshold  $\alpha_T > \alpha/m$  trong khi vẫn control FWER ở  $\alpha$ .

*Tukey's method*

Figure 13.4 dùng  $G = 6$  với  $\mu_1 = \dots = \mu_5 \neq \mu_6$ , nên trong 15 pairwise hypotheses có ten true và five false. Horizontal lines cho thấy Tukey luôn



**HÌNH 13.4.** Ba simulations của  $m = 15$  hypotheses tương ứng pairwise tests về equality của  $G = 6$  means.  $m_0 = 10$  true nulls màu black, còn lại red. Bonferroni reject dưới black line, Tukey dưới blue line; green line là control Type I error không adjust multiple testing. Tukey có slightly higher power.

reject at least as many as Bonferroni; left panel Tukey correctly rejects two more false nulls.

Bây giờ nhìn Table 13.3 và notice Managers One và Three có higher mean returns hơn Managers Two, Four, Five. Điều đó có thể motivate test

$$H_0 : \frac{1}{2}(\mu_1 + \mu_3) = \frac{1}{3}(\mu_2 + \mu_4 + \mu_5). \quad (13.8)$$

Một variant của two-sample t-test cho p-value 0.004. Tuy nhiên hypothesis này cũng được chosen after peeking at data, nên có implicit multiple testing.

Scheffé's method được designed cho exactly setting này. Với Fund example, để control Type I error at  $\alpha = 0.05$ , threshold cần là  $\alpha_S = 0.002$ , nên p-value 0.004 không đủ để reject (13.8). Advantage quan trọng là same threshold có thể dùng để compare bất kỳ split nào của managers thành two groups, chẳng hạn  $\frac{1}{3}(\mu_1 + \mu_2 + \mu_3) = \frac{1}{2}(\mu_4 + \mu_5)$  hay  $\frac{1}{4}(\mu_1 + \mu_2 + \mu_3 + \mu_4) = \mu_5$ , mà không cần additional multiplicity adjustment.<sup>14</sup>

Scheffé's method

Tóm lại, Holm và Bonferroni là very general multiple testing corrections. Trong special cases, more powerful corrections có thể control FWER với higher power và fewer Type II errors; Tukey và Scheffé minh họa two such cases.

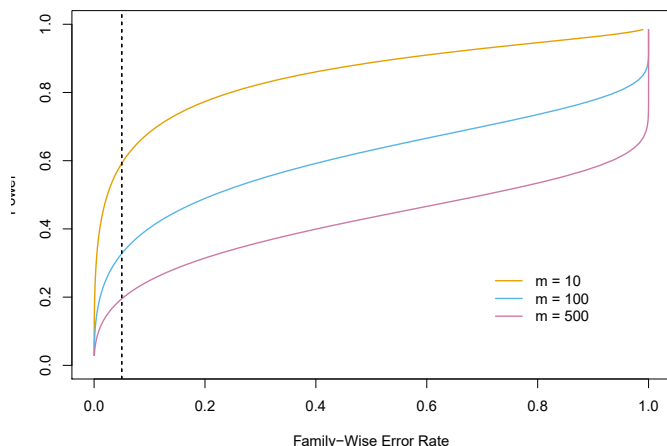
### 13.3.3 Trade-Off Between the FWER and Power

Có trade-off giữa FWER threshold và power. Power là number of false null hypotheses rejected chia cho total false null hypotheses, tức  $S/(m - m_0)$ . Figure 13.5 mô phỏng  $m$  hypotheses, 90% true và 10% false. Với  $m = 10$ , FWER 0.05 cho power khoảng 60%. Khi  $m$  tăng, power decreases; với  $m = 500$ , power dưới 0.2 tại FWER 0.05.

Vì vậy control FWER reasonable khi  $m$  small như 5 hay 10. Nhưng với  $m = 100$  hay 1,000, FWER control làm gần như impossible để reject false nulls; power extremely low. Reason là  $\text{FWER} = \Pr(V \geq 1)$  guarantees rất

<sup>14</sup>Thực tế, calculating “correct” value của  $m$  trong setting này khá technical và outside scope của book.

low probability của even one false positive. Khi  $m$  large, guarantee đó có thể force analyst reject very few hypotheses hoặc none at all.



**HÌNH 13.5.** Trong simulation có 90% của  $m$  null hypotheses true, power được displayed như function của FWER cho  $m = 10, 100, 500$ . Khi  $m$  tăng, power decreases. Vertical dashed line chỉ FWER 0.05.

Điều này scientifically uninteresting và thường dẫn tới very low power. Trong practice, khi  $m$  large, ta có thể willing to tolerate một few false positives để make more discoveries. Đây là motivation cho false discovery rate.

## 13.4 The False Discovery Rate

### 13.4.1 Intuition for the False Discovery Rate

Khi  $m$  large, cố prevent any false positives như FWER control là too stringent. Thay vào đó, ta có thể muốn ratio false positives  $V$  trên total positives  $R = V + S$  sufficiently low, để most rejected null hypotheses không phải false positives. Ratio  $V/R$  gọi là false discovery proportion (FDP).

false discovery  
proportion

Có vẻ tempting để yêu cầu analyst control FDP, chẳng hạn guarantee không quá 20% rejected hypotheses là false positives. Nhưng trên một particular dataset, analyst không biết chắc hypotheses nào true hay false, nên impossible to guarantee FDP. Tương tự, analyst có thể control FWER in probability nhưng không guarantee  $V = 0$  trên từng dataset trừ khi reject none.

Vì vậy ta control false discovery rate (FDR),<sup>15</sup> defined là

$$\text{FDR} = E(\text{FDP}) = E(V/R). \quad (13.9)$$

Khi control FDR ở  $q = 20\%$ , ta reject as many null hypotheses as possible while guaranteeing rằng no more than 20% của rejected nulls là false positives, on average.

false discovery rate

Expectation trong (13.9) lấy over population generating the data. Nếu repeat experiment many times và mỗi lần control FDR ở 0.2, then on

<sup>15</sup>Nếu  $R = 0$ , ratio  $V/R$  được replace bằng 0 để tránh 0/0. Formally,  $\text{FDR} = E(V/R \mid R > 0) \Pr(R > 0)$ .

average 20% rejected nulls sẽ là false positives. Trên any given dataset, fraction có thể above hoặc below 20%.

FDR cũng align với exploratory data collection hiện đại. Ví dụ genomic researcher có thể test association cho 20,000 genes mà không có specific pre-specified hypothesis. Nếu simply investigate all p-values below 0.05, even khi no genes associated ta expect  $0.05 \times 20,000 = 1,000$  small p-values by chance. Control FDR ở 20% guarantees rằng, on average, không quá 20% genes selected for follow-up là false positives.

Không giống p-values, FDR không có universally accepted threshold. Choice thường context- hoặc dataset-dependent: 10% có thể phù hợp nếu follow-up expensive, trong khi 30% có thể suitable nếu follow-up inexpensive.

### 13.4.2 The Benjamini–Hochberg Procedure

Ta cần decide hypotheses nào reject while guaranteeing  $\text{FDR} \leq q$ . Algorithm 13.2 provides một simple connection giữa  $m$  p-values và desired FDR level.

#### Algorithm 13.2 Benjamini–Hochberg Procedure to Control the FDR

1. Specify  $q$ , level cần control FDR.
2. Compute p-values  $p_1, \dots, p_m$  cho  $H_{01}, \dots, H_{0m}$ .
3. Order:  $p_{(1)} \leq \dots \leq p_{(m)}$ .
4. Define

$$L = \max\{j : p_{(j)} < qj/m\}. \quad (13.10)$$

5. Reject mọi  $H_{0j}$  sao cho  $p_j \leq p_{(L)}$ .

Với five managers trong Table 13.3,  $m = 5$ . Ta có  $p_{(1)} = 0.006 < 0.05(1/5)$ ,  $p_{(2)} = 0.012 < 0.05(2/5)$ , còn three p-values còn lại fail inequalities tương ứng. Do đó để control FDR ở 5%, ta reject null hypotheses rằng Managers One và Three perform no better than chance.

*Benjamini–  
Hochberg procedure*

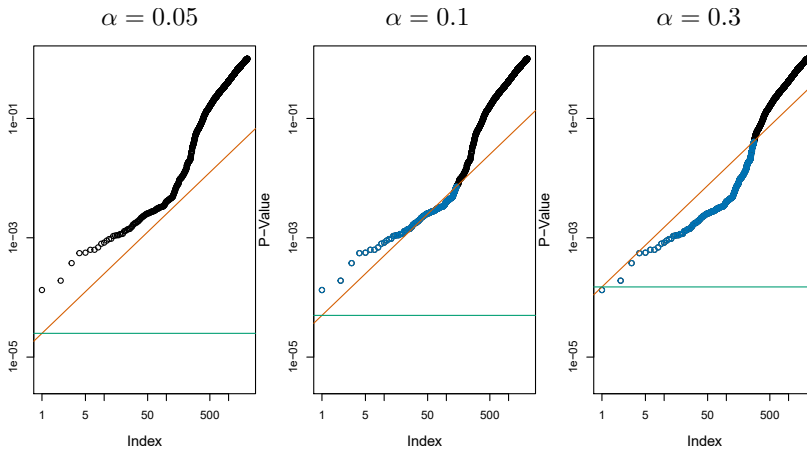
Miễn là  $m$  p-values independent hoặc only mildly dependent, Benjamini–Hochberg guarantees<sup>16</sup>  $\text{FDR} \leq q$ . Remarkably, guarantee này không depend on number of true null hypotheses hay distribution của p-values corresponding to false nulls.

Có fundamental difference giữa Bonferroni và Benjamini–Hochberg. Bonferroni threshold  $\alpha/m$  không depend on data beyond  $m$ . BH dùng data-dependent threshold  $p_{(L)}$ , với  $L$  là function của all p-values. Do đó ta không biết rejection threshold trước khi seeing data. Holm procedure cũng share property này.

Figure 13.6 applies both methods cho all 2,000 Fund managers. Bonferroni ở FWER 0.3 chỉ reject one hypothesis, dù 13 managers có p-values below 0.001. BH ở FDR 0.3 reject 279 managers; expected no more than around  $279 \times 0.3 = 83.7$  là chance successes. FDR control vì thế milder và

<sup>16</sup>Proof nằm well beyond scope của book.





**HÌNH 13.6.** Cùng  $m = 2,000$  ordered  $p$ -values của *Fund* data. Green lines là Bonferroni thresholds cho FWER levels 0.05, 0.1, 0.3; orange lines là Benjamini–Hochberg thresholds cho FDR levels tương ứng. Ở  $q = 0.1$ , 146 null hypotheses bị reject; ở  $q = 0.3$ , 279 bị reject.

more powerful than FWER control, đổi lại substantially more false positives.

Benjamini–Hochberg có từ mid-1990s. Dù nhiều alternative FDR procedures được proposed cho particular scenarios, BH vẫn là highly useful và widely applicable.

## 13.5 A Re-Sampling Approach to p-Values and False Discovery Rates

Cho đến đây ta assumed test statistic  $T$  có known hoặc assumed distribution under  $H_0$ , như normal,  $t$ ,  $\chi^2$ , hoặc  $F$  distribution. Đây gọi là theoretical null distribution, thường được dùng để obtain p-value. Với most familiar hypotheses, theoretical null exists nếu willing to make stringent assumptions.

*theoretical null  
distribution*

Nếu  $H_0$  hoặc  $T$  unusual, theoretical null distribution có thể unavailable. Hoặc distribution exists nhưng assumptions của nó bị violated, chẳng hạn sample size too small. Section này presents một general computer-based framework để approximate null distribution và obtain p-values. Ta focus on specific example testing equality of two means bằng two-sample t-test. Material này more challenging và có thể skipped nếu reader content với theoretical null distributions.

### 13.5.1 A Re-Sampling Approach to the p-Value

Xét  $H_0 : E(X) = E(Y)$  versus  $H_a : E(X) \neq E(Y)$  với  $n_X$  independent observations của  $X$  và  $n_Y$  của  $Y$ . Two-sample t-statistic là

$$T = \frac{\hat{\mu}_X - \hat{\mu}_Y}{s\sqrt{1/n_X + 1/n_Y}}, \quad (13.11)$$

where  $\hat{\mu}_X = n_X^{-1} \sum_{i=1}^{n_X} x_i$  và  $\hat{\mu}_Y = n_Y^{-1} \sum_{i=1}^{n_Y} y_i$ , còn pooled standard-deviation estimator là

$$s = \sqrt{\frac{(n_X - 1)s_X^2 + (n_Y - 1)s_Y^2}{n_X + n_Y - 2}}.$$

Large  $|T|$  provides evidence against  $H_0$ .

Nếu  $n_X, n_Y$  large,  $T$  approximately follows  $N(0, 1)$ . Nếu samples small và ta không make strong distributional assumptions, theoretical null của  $T$  unknown.<sup>17</sup> Ta có thể approximate nó bằng re-sampling, specifically a permutation approach.

re-sampling;  
permutation

Thought experiment: nếu  $H_0$  holds và distributions của  $X, Y$  actually identical, swapping observations between groups leaves distribution of  $T$  invariant. Vì vậy randomly permute all  $n_X + n_Y$  observations  $B$  times; mỗi lần compute (13.11), yielding  $T^{*1}, \dots, T^{*B}$ . Đây approximates null distribution. Re-sampling p-value là fraction permuted datasets có statistic at least as extreme as observed:

$$\text{p-value} = \frac{\sum_{b=1}^B \mathbf{1}(|T^{*b}| \geq |T|)}{B}. \quad (13.12)$$

### Algorithm 13.3 Re-Sampling p-Value for a Two-Sample t-Test

1. Compute  $T$  in (13.11) on original  $x_1, \dots, x_{n_X}$  and  $y_1, \dots, y_{n_Y}$ .
2. For  $b = 1, \dots, B$  (e.g.  $B = 10,000$ ):
  - (a) Randomly permute all observations; first  $n_X$  become  $x_1^*, \dots, x_{n_X}^*$  and remaining  $n_Y$  become  $y_1^*, \dots, y_{n_Y}^*$ .
  - (b) Compute (13.11) trên permuted data, call result  $T^{*b}$ .
3. p-value bằng expression trong (13.12).

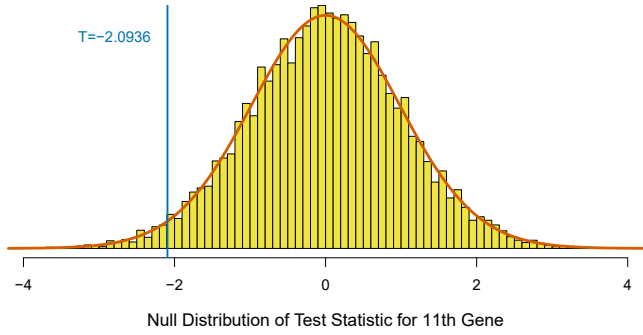
Apply procedure cho **Khan** dataset, expression measurements của 2,308 genes trong four sub-types của small round blood cell tumors. Ta restrict vào rhabdomyosarcoma ( $n_X = 29$ ) và Burkitt's lymphoma ( $n_Y = 25$ ). Với 11th gene, two-sample t-test gives  $T = -2.09$ . Theoretical  $t_{52}$  null gives p-value 0.041; Algorithm 13.3 với  $B = 10,000$  gives 0.042. Figure 13.7 cho thấy almost no difference giữa two null distributions.

Figure 13.8 shows analogous results cho 877th gene, nơi theoretical và re-sampling null distributions differ substantially, leading to different p-values. Differences tend to be more pronounced với smaller samples hoặc more skewed data. Ở gene 877, một observation rất far from others creates strong skewness.

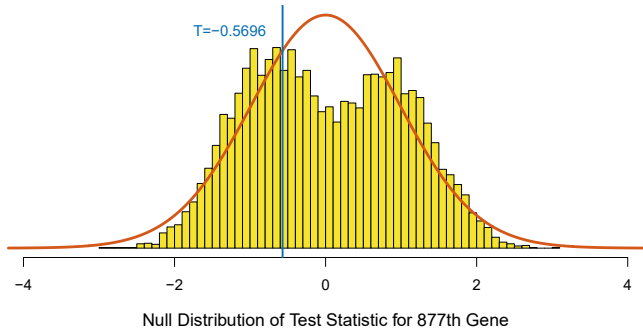
### 13.5.2 A Re-Sampling Approach to the False Discovery Rate

Giả sử ta muốn control FDR cho  $m$  hypotheses nhưng không muốn rely on theoretical null. Ta có thể compute  $m$  re-sampling p-values rồi apply BH;

<sup>17</sup>Nếu assume  $X$  và  $Y$  normally distributed,  $T$  follows a t-distribution với  $n_X + n_Y - 2$  degrees of freedom under  $H_0$ . Trong practice distribution thường unknown, nên re-sampling có thể preferable. Nếu re-sampling và theoretical approach disagree, re-sampling results are more trustworthy.



**HÌNH 13.7.** 11th gene trong *Khan* dataset có  $T = -2.09$ . Theoretical và re-sampling null distributions gần như identical. Theoretical p-value là 0.041 và re-sampling p-value là 0.042.



**HÌNH 13.8.** 877th gene trong *Khan* dataset có  $T = -0.57$ . Theoretical và re-sampling null distributions khá different. Theoretical p-value là 0.571 và re-sampling p-value là 0.673.

tuy nhiên có một more direct way không cần compute p-values.

Using notation Table 13.2,  $\text{FDR} = E(V/R)$ . Re-sampling approach bắt đầu với approximation

$$\text{FDR} = E\left(\frac{V}{R}\right) \approx \frac{E(V)}{R}. \quad (13.13)$$

Nếu reject hypotheses với  $|T_j| \geq c$ , denominator straightforward:  $R = \sum_{j=1}^m \mathbf{1}(|T_j| \geq c)$ .

Numerator  $E(V)$  khó hơn vì ta không biết nulls nào truly true. Solution là simulate data under all null hypotheses. For each hypothesis, randomly permute its  $n_X + n_Y$  observations and compute t-statistic; under permuted data all nulls hold. Number of permuted statistics exceeding  $c$  estimates  $V$ . Repeat  $B$  times và average để improve estimate.

Algorithm 13.4 formalizes procedure, yielding a plug-in estimate của FDR: plug observed  $R$  vào denominator và estimated  $E(V)$  vào numerator.<sup>18</sup>

<sup>18</sup>Để implement Algorithm 13.4 efficiently, same set of permutations in Step 2(b)i should

Apply Algorithm 13.4 và BH using theoretical p-values cho all  $m = 2,308$  Khan genes. Figure 13.9 shows, for a given number of rejections, estimated FDRs almost identical.

Nếu define re-sampling p-value bằng pooled information across all  $m$  tests,

$$p_j = \frac{\sum_{j'=1}^m \sum_{b=1}^B \mathbf{1}(|T_{j'}^{*b}| \geq |T_j|)}{Bm}, \quad (13.14)$$

then applying BH to these p-values is exactly equivalent to Algorithm 13.4. Equation (13.14) is an alternative to (13.12) that pools information across tests when approximating null distribution.

#### Algorithm 13.4 Plug-In FDR for a Two-Sample T-Test

1. Select threshold  $c > 0$ .
2. For  $j = 1, \dots, m$ :
  - (a) Compute  $T^{(j)}$ , the two-sample t-statistic for  $H_{0j}$  on original data.
  - (b) For  $b = 1, \dots, B$ : randomly permute  $n_X + n_Y$  observations and compute  $T^{(j),*b}$  on the permuted data.
3. Compute  $R = \sum_{j=1}^m \mathbf{1}(|T^{(j)}| \geq c)$ .
4. Compute  $\widehat{V} = B^{-1} \sum_{b=1}^B \sum_{j=1}^m \mathbf{1}(|T^{(j),*b}| \geq c)$ .
5. Estimated FDR associated with  $c$  is  $\widehat{V}/R$ .

### 13.5.3 When Are Re-Sampling Approaches Useful?

Sections 13.5.1 và 13.5.2 cho thấy re-sampling có thể differ strongly from theoretical p-values khi theoretical null inaccurate, nhưng agree closely khi approximation good. Re-sampling particularly useful trong two settings:

1. No theoretical null distribution is available, vì  $H_0$  hoặc test statistic unusual.
2. A theoretical null exists nhưng assumptions required for validity fail. Chẳng hạn t-statistic (13.11) follows  $t_{n_X+n_Y-2}$  only under normality, và only approximates  $N(0, 1)$  khi sample sizes large. Với non-normal small samples, theoretical p-values may not properly control Type I error.

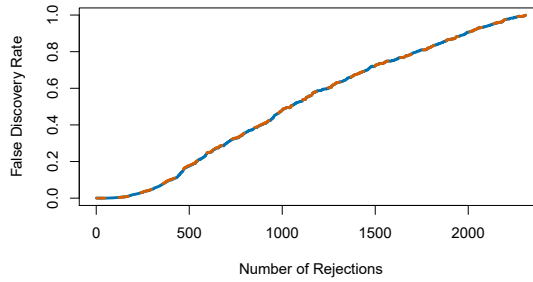
Generally, nếu có cách re-sample hoặc permute observations để generate data following null distribution, variants of Algorithms 13.3 và 13.4 cho phép compute p-values hoặc estimate FDR. Đây là powerful tool khi no out-of-box hypothesis test exists hoặc key assumptions của standard tests bị violated.

## 13.6 Lab: Multiple Testing

Ta bắt đầu bằng các usual imports đã thấy trong earlier labs.

---

be used cho all  $m$  null hypotheses.



**HÌNH 13.9.** Với  $m = 2,308$  Khan genes,  $y$ -axis là estimated FDR khi reject hypotheses tương ứng với  $k$  smallest  $p$ -values,  $k = 1, \dots, 2308$ . Orange dashed curve dùng Benjamini-Hochberg; blue solid curve dùng re-sampling Algorithm 13.4 với  $B = 10,000$ . Hai estimates gần như identical; rejecting 500 smallest  $p$ -values corresponds to FDR khoảng 17.7%.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
from ISLP import load_data
```

Ta cũng collect các new imports cần cho lab này.

```
In [2]: from scipy.stats import \
 (ttest_1samp,
 ttest_rel,
 ttest_ind,
 t as t_dbn)
from statsmodels.stats.multicomp import \
 pairwise_tukeyhsd
from statsmodels.stats.multitest import \
 multipletests as mult_test
```

### 13.6.1 Review of Hypothesis Tests

Ta bắt đầu bằng one-sample t-tests. Trước hết create 100 variables, mỗi variable có 10 observations. First 50 variables có mean 0.5 và variance 1, còn remaining variables có mean 0 và variance 1.

```
In [3]: rng = np.random.default_rng(12)
X = rng.standard_normal((10, 100))
true_mean = np.array([0.5]*50 + [0]*50)
X += true_mean[None,:]
```

Đầu tiên dùng `ttest_1samp()` từ `scipy.stats` để test  $H_0 : \mu_1 = 0$ .

`ttest_1samp()`

```
In [4]: result = ttest_1samp(X[:,0], 0)
result.pvalue
```

Out[4]: 0.931

P-value 0.931 không small enough để reject ở  $\alpha = 0.05$ . Ở đây  $\mu_1 = 0.5$ ,

nên  $H_0$  false; fail to reject là một Type II error.

Bây giờ test  $H_{0j} : \mu_j = 0$  cho  $j = 1, \dots, 100$ . Ta compute 100 p-values, rồi record decision dựa trên threshold 0.05.

```
In [5]: p_values = np.empty(100)
for i in range(100):
 p_values[i] = ttest_1samp(X[:,i], 0).pvalue
decision = pd.cut(p_values,
 [0, 0.05, 1],
 labels=['Reject H0',
 'Do not reject H0'])
truth = pd.Categorical(true_mean == 0,
 categories=[True, False],
 ordered=True)
```

Vì đây là simulated data, ta có thể tạo  $2 \times 2$  table tương tự Table 13.2.

```
In [6]: pd.crosstab(decision,
 truth,
 rownames=['Decision'],
 colnames=['H0'])
```

```
Out[6]: H0 True False
Decision
Reject H0 5 15
Do not reject H0 45 35
```

Ở level 0.05, ta reject 15/50 false null hypotheses và incorrectly reject 5 true null hypotheses. Theo notation Section 13.3,  $V = 5, S = 15, U = 45, W = 35$ . Vì  $\alpha = 0.05$ , ta expect reject khoảng 5% true nulls; observed 5/50 phù hợp với expectation này.

Trong simulation trên, signal-to-noise ratio cho false nulls chỉ  $0.5/1 = 0.5$ , nên Type II errors nhiều. Nếu tăng mean shift lên 1, signal mạnh hơn và chỉ còn 10 Type II errors.

```
In [7]: true_mean = np.array([1]*50 + [0]*50)
X = rng.standard_normal((10, 100))
X += true_mean[None,:]
for i in range(100):
 p_values[i] = ttest_1samp(X[:,i], 0).pvalue
decision = pd.cut(p_values,
 [0, 0.05, 1],
 labels=['Reject H0',
 'Do not reject H0'])
truth = pd.Categorical(true_mean == 0,
 categories=[True, False],
 ordered=True)

pd.crosstab(decision,
 truth,
 rownames=['Decision'],
 colnames=['H0'])
```

```
Out[7]: H0 True False
Decision
Reject H0 2 40
Do not reject H0 48 10
```

### 13.6.2 Family-Wise Error Rate

Từ (13.5), nếu all  $m$  null hypotheses true và tests independent thì FWER =  $1 - (1 - \alpha)^m$ . Ta compute expression này cho  $m = 1, \dots, 500$  và  $\alpha = 0.05, 0.01, 0.001$  để reproduce Figure 13.2.

```
In [8]: m = np.linspace(1, 501)
fig, ax = plt.subplots()
[ax.plot(m,
 1 - (1 - alpha)**m,
 label=r'$\alpha = %s$' % str(alpha))
 for alpha in [0.05, 0.01, 0.001]]
ax.set_xscale('log')
ax.set_xlabel('Number of Hypotheses')
ax.set_ylabel('Family-Wise Error Rate')
ax.legend()
ax.axhline(0.05, c='k', ls='--');
```

Ngay cả với  $m = 50$ , FWER exceeds 0.05 trừ khi  $\alpha$  rất small như 0.001; nhưng threshold quá low tạo many Type II errors và low power.

Ta conduct one-sample t-test cho first five managers của Fund, testing  $H_{0j} : \mu_j = 0$ .

```
In [9]: Fund = load_data('Fund')
fund_mini = Fund.iloc[:, :5]
fund_mini_pvals = np.empty(5)
for i in range(5):
 fund_mini_pvals[i] = ttest_1samp(fund_mini.iloc[:, i], 0).pvalue
fund_mini_pvals
```

```
Out[9]: array([0.006, 0.918, 0.012, 0.601, 0.756])
```

P-values low cho Managers One và Three, high cho three managers còn lại. Nhưng không thể simply reject  $H_{01}$  và  $H_{03}$  vì phải account for multiple testing. Ta dùng `multiplerejects()` từ `statsmodels` (abbreviated `mult_test()`). Với methods như Holm và Bonferroni, function returns adjusted p-values; nếu adjusted p-value  $\leq \alpha$ , hypothesis có thể reject while maintaining FWER  $\leq \alpha$ .

*multiplerejects();  
adjusted p-values*

```
In [10]: reject, bonf = mult_test(fund_mini_pvals, method="bonferroni")[:2]
reject
```

```
Out[10]: array([True, False, False, False, False])
```

Bonferroni adjusted p-values chính là raw p-values multiplied by 5 rồi truncated ở 1.

```
In [11]: bonf, np.minimum(fund_mini_pvals * 5, 1)
```

```
Out[11]: (array([0.03, 1. , 0.06, 1. , 1.]),
 array([0.03, 1. , 0.06, 1. , 1.]))
```

Do đó Bonferroni chỉ reject Manager One ở FWER 0.05. Holm thì reject Managers One và Three.

```
In [12]: mult_test(fund_mini_pvals, method="holm", alpha=0.05)[:2]
```

```
Out[12]: (array([True, False, True, False, False]),
 array([0.03, 1. , 0.05, 1. , 1.]))
```

Manager One seems particularly strong, còn Manager Two poor.

```
In [13]: fund_mini.mean()
```

```
Out[13]: Manager1 3.0
 Manager2 -0.1
 Manager3 2.8
 Manager4 0.5
 Manager5 0.3
 dtype: float64
```

Có evidence of meaningful difference giữa Managers One và Two không? Paired t-test bằng `ttest_rel()` cho:

*paired t-test;*  
*ttest\_rel()*

```
In [14]: ttest_rel(fund_mini['Manager1'],
 fund_mini['Manager2']).pvalue
```

```
Out[14]: 0.038
```

P-value 0.038 suggests statistically significant difference. Nhưng test này chỉ được chosen sau khi examining data và noticing highest/lowest means. Effectively ta implicitly considered  $\binom{5}{2} = 10$  pairwise tests. Vì vậy ta dùng Tukey's method qua `pairwise_tukeyhsd()` để adjust multiple testing. Function treats response là monthly excess returns và predictor là manager identity, tương đương ANOVA regression với qualitative predictor.

*pairwise\_*  
*tukeyhsd();*  
*ANOVA*

```
In [15]: returns = np.hstack([fund_mini.iloc[:,i] for i in range(5)])
 managers = np.hstack([[i+1]*50 for i in range(5)])
 tukey = pairwise_tukeyhsd(returns, managers)
 print(tukey.summary())
```

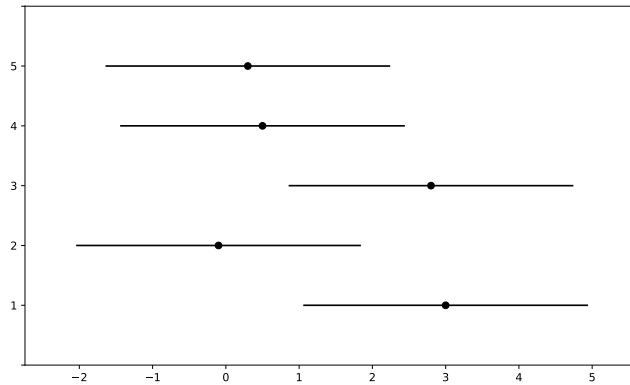
Multiple Comparison of Means - Tukey HSD, FWER=0.05

```
=====
group1 group2 meandiff p-adj lower upper reject

1 2 -3.1 0.1862 -6.9865 0.7865 False
1 3 -0.2 0.9999 -4.0865 3.6865 False
1 4 -2.5 0.3948 -6.3865 1.3865 False
1 5 -2.7 0.3152 -6.5865 1.1865 False
2 3 2.9 0.2453 -0.9865 6.7865 False
2 4 0.6 0.9932 -3.2865 4.4865 False
2 5 0.4 0.9986 -3.4865 4.2865 False
3 4 -2.3 0.4820 -6.1865 1.5865 False
3 5 -2.5 0.3948 -6.3865 1.3865 False
4 5 -0.2 0.9999 -4.0865 3.6865 False
=====
```

Tukey-adjusted p-value cho difference giữa Managers One và Two tăng từ 0.038 lên 0.186, nên evidence không còn clear. `plot_simultaneous()` plots confidence intervals; non-overlap would indicate significance at nom-





**HÌNH 13.10.** 95% confidence intervals cho each manager trong *Fund* data, dùng Tukey's method để adjust multiple testing. All intervals overlap, nên no differences among managers are statistically significant khi control FWER ở 0.05.

inal 0.05. Ở đây none are significant.<sup>19</sup>

```
In [16]: fig, ax = plt.subplots(figsize=(8,8))
 tukey.plot_simultaneous(ax=ax);
```

### 13.6.3 False Discovery Rate

Bây giờ perform hypothesis tests cho all 2,000 Fund managers, each one testing  $H_{0j} : \mu_j = 0$ .

```
In [17]: fund_pvalues = np.empty(2000)
 for i, manager in enumerate(Fund.columns):
 fund_pvalues[i] = ttest_1samp(Fund[manager], 0).pvalue
```

Có quá many managers để realistically control FWER, nên ta control FDR. `mult_test()` thực hiện Benjamini–Hochberg.

```
In [18]: fund_qvalues = mult_test(fund_pvalues, method="fdr_bh")[1]
 fund_qvalues[:10]
```

```
Out[18]: array([0.09, 0.99, 0.12, 0.92, 0.96, 0.08, 0.08, 0.08, 0.08,
 0.08])
```

Q-values có thể interpreted như smallest FDR threshold tại đó particular null hypothesis would be rejected. Q-value 0.1 nghĩa là reject được ở FDR 10% hoặc greater, nhưng không dưới 10%.

*q-values*

```
In [19]: (fund_qvalues <= 0.1).sum()
```

```
Out[19]: 146
```

<sup>19</sup>Traditionally plot này shows intervals cho each paired difference. Với many groups, displaying one interval per group convenient và equivalent; differencing all pairs recovers traditional plot.

Có 146 managers có q-value below 0.1, nên ta conclude 146 managers beat market at FDR 10%. Khoảng 15 trong số này expected là false discoveries. Ngược lại, Bonferroni control FWER 0.1 sẽ reject none.

```
In [20]: (fund_pvalues <= 0.1 / 2000).sum()
```

```
Out[20]: 0
```

Ta implement BH manually để reproduce middle panel of Figure 13.6: order p-values, find all  $p_{(j)} < qj/m$ , rồi select p-values không exceed largest qualifying value.

```
In [21]: sorted_ = np.sort(fund_pvalues)
m = fund_pvalues.shape[0]
q = 0.1
sorted_set_ = np.where(sorted_ < q * np.linspace(1, m, m) / m)[0]
if sorted_set_.shape[0] > 0:
 selected_ = fund_pvalues < sorted_[sorted_set_].max()
 sorted_set_ = np.arange(sorted_set_.max())
else:
 selected_ = []
 sorted_set_ = []
```

Bây giờ reproduce middle panel of Figure 13.6.

```
In [22]: fig, ax = plt.subplots()
ax.scatter(np.arange(0, sorted_.shape[0]) + 1,
 sorted_, s=10)
ax.set_yscale('log')
ax.set_xscale('log')
ax.set_ylabel('P-Value')
ax.set_xlabel('Index')
ax.scatter(sorted_set_+1, sorted_[sorted_set_], c='r', s=20)
ax.axline((0, 0), (1,q/m), c='k', ls='--', linewidth=3);
```

### 13.6.4 A Re-Sampling Approach

Ta implement re-sampling hypothesis testing trên `Khan` dataset. Merge training và testing data tạo observations cho 83 patients và 2,308 genes.

```
In [23]: Khan = load_data('Khan')
D = pd.concat([Khan['xtrain'], Khan['xtest']])
D['Y'] = pd.concat([Khan['ytrain'], Khan['ytest']])
D['Y'].value_counts()
```

```
Out[23]: 2 29
 4 25
 3 18
 1 11
 Name: Y, dtype: int64
```

Có four cancer classes. Với each gene, ta compare mean expression ở class 2 (rhabdomyosarcoma) và class 4 (Burkitt's lymphoma). Standard two-sample t-test trên 11th gene gives statistic  $-2.09$  và p-value 0.041.

`ttest_ind()`

```
In [24]: D2 = D[lambdas df:df['Y'] == 2]
 D4 = D[lambdas df:df['Y'] == 4]
```

```
gene_11 = 'G0011'
observedT, pvalue = ttest_ind(D2[gene_11],
 D4[gene_11],
 equal_var=True)

observedT, pvalue
```

Out[24]: (-2.094, 0.041)

Theoretical p-value assumes  $t_{52}$  null. Thay vào đó, randomly split 54 patients into groups 29 và 25, compute statistic, repeat 10,000 times, rồi estimate p-value bằng fraction re-sampled statistics more extreme than observed.

```
In [25]: B = 10000
Tnull = np.empty(B)
D_ = np.hstack([D2[gene_11], D4[gene_11]])
n_ = D2[gene_11].shape[0]
D_null = D_.copy()
for b in range(B):
 rng.shuffle(D_null)
 ttest_ = ttest_ind(D_null[:n_],
 D_null[n_:],
 equal_var=True)
 Tnull[b] = ttest_.statistic
(np.abs(Tnull) > np.abs(observedT)).mean()
```

Out[25]: 0.0398

Re-sampling p-value 0.0398 gần như identical với theoretical 0.0412. Ta plot histogram để reproduce Figure 13.7.

```
In [26]: fig, ax = plt.subplots(figsize=(8,8))
ax.hist(Tnull,
 bins=100,
 density=True,
 facecolor='y',
 label='Null')
xval = np.linspace(-4.2, 4.2, 1001)
ax.plot(xval,
 t_dbn.pdf(xval, D_.shape[0]-2),
 c='r')
ax.axvline(observedT,
 c='b',
 label='Observed')
ax.legend()
ax.set_xlabel("Null Distribution of Test Statistic");
```

Cuối cùng implement plug-in re-sampling FDR từ Algorithm 13.4. Vì all 2,308 genes có thể slow, ta randomly choose 100 genes; for each gene compute observed statistic và 10,000 re-sampled statistics.

```
In [27]: m, B = 100, 10000
idx = rng.choice(Khan['xtest'].columns, m, replace=False)
T_vals = np.empty(m)
Tnull_vals = np.empty((m, B))

for j in range(m):
 col = idx[j]
 T_vals[j] = ttest_ind(D2[col],
 D4[col],
 equal_var=True).statistic
```

```

D_ = np.hstack([D2[col], D4[col]])
D_null = D_.copy()
for b in range(B):
 rng.shuffle(D_null)
 ttest_ = ttest_ind(D_null[:n_],
 D_null[n_:],
 equal_var=True)
 Tnull_vals[j,b] = ttest_.statistic

```

Compute number rejected  $R$ , estimated false positives  $\widehat{V}$ , và estimated FDR cho range of cutoffs  $c$  chosen từ absolute observed statistics.

```

In [28]: cutoffs = np.sort(np.abs(T_vals))
FDRs, Rs, Vs = np.empty((3, m))
for j in range(m):
 R = np.sum(np.abs(T_vals) >= cutoffs[j])
 V = np.sum(np.abs(Tnull_vals) >= cutoffs[j]) / B
 Rs[j] = R
 Vs[j] = V
 FDRs[j] = V / R

```

Ở FDR 0.1, 15/100 hypotheses được reject, expected one or two false discoveries. Ở FDR 0.2, 28 genes được reject, expected around six false discoveries.

```

In [29]: sorted(idx[np.abs(T_vals) >= cutoffs[FDRs < 0.1].min()])

```

Threshold 0.2 selects more genes at the cost of higher expected proportion of false discoveries.

```

In [30]: sorted(idx[np.abs(T_vals) >= cutoffs[FDRs < 0.2].min()])

```

Dòng tiếp theo generates Figure 13.11, analogous to Figure 13.9 nhưng based on only 100 genes.

```

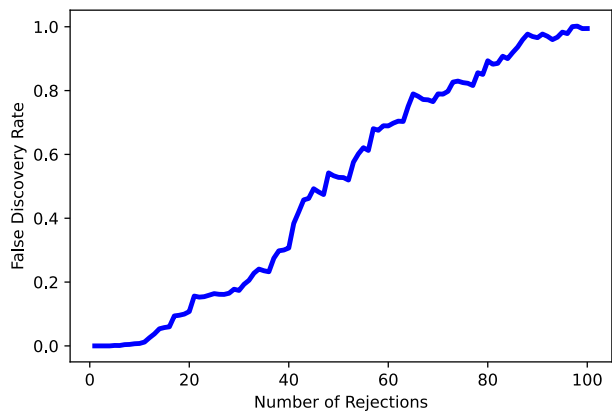
In [31]: fig, ax = plt.subplots()
ax.plot(Rs, FDRs, 'b', linewidth=3)
ax.set_xlabel("Number of Rejections")
ax.set_ylabel("False Discovery Rate");

```

## 13.7 Exercises

### Conceptual

- Giả sử test  $m$  null hypotheses, tất cả đều true. Ta control Type I error cho each null hypothesis ở level  $\alpha$ . Với mỗi part, justify answer.
  - Tổng cộng expected bao nhiêu Type I errors?
  - Nếu  $m$  tests independent, family-wise error rate của  $m$  tests là gì? *Hint:* nếu events  $A, B$  independent thì  $\Pr(A \cap B) = \Pr(A) \Pr(B)$ .
  - Với  $m = 2$ , p-values positively correlated. FWER qualitatively compare thế nào với answer (b) khi  $m = 2$ ? *Hint:* first suppose two p-values perfectly correlated.
  - Với  $m = 2$  nhưng p-values negatively correlated, FWER compare thế nào với (b)? *Hint:* suppose whenever one p-value  $< \alpha$ , the



**HÌNH 13.11.** Estimated false discovery rate versus number of rejected null hypotheses, cho 100 genes randomly selected từ *Khan* dataset.

other  $> \alpha$ , so never reject both.

2. Giả sử test  $m$  hypotheses, control Type I error của each ở level  $\alpha$ . Assume all p-values independent và all null hypotheses true.
  - (a) Let  $A_j = 1$  nếu  $j$ th null rejected, 0 otherwise. Distribution của  $A_j$  là gì?
  - (b) Distribution của  $\sum_{j=1}^m A_j$  là gì?
  - (c) Standard deviation của number of Type I errors là gì?
3. Giả sử test  $m$  null hypotheses và control Type I error của  $j$ th test ở  $\alpha_j$ . Argue rằng FWER không lớn hơn  $\sum_{j=1}^m \alpha_j$ .

| Null Hypothesis | p-value |
|-----------------|---------|
| $H_{01}$        | 0.0011  |
| $H_{02}$        | 0.031   |
| $H_{03}$        | 0.017   |
| $H_{04}$        | 0.32    |
| $H_{05}$        | 0.11    |
| $H_{06}$        | 0.90    |
| $H_{07}$        | 0.07    |
| $H_{08}$        | 0.006   |
| $H_{09}$        | 0.004   |
| $H_{10}$        | 0.0009  |

**BẢNG 13.4.** P-values cho Exercise 4.

4. Giả sử test  $m = 10$  hypotheses và obtain p-values trong Table 13.4.
  - (a) Control Type I error cho each null ở  $\alpha = 0.05$ : reject hypotheses nào?
  - (b) Control FWER ở  $\alpha = 0.05$ : reject hypotheses nào? Justify.
  - (c) Control FDR ở  $q = 0.05$ : reject hypotheses nào? Justify.

- (d) Control FDR ở  $q = 0.2$ : reject hypotheses nào? Justify.
  - (e) Trong hypotheses rejected ở FDR  $q = 0.2$ , approximately bao nhiêu là false positives? Justify.
5. Make up p-values dẫn đến specified rejection counts bằng Bonferroni và Holm.
- (a) Give five p-values sao cho both Bonferroni và Holm reject exactly one null hypothesis khi control FWER ở 0.1.
  - (b) Give five p-values sao cho Bonferroni rejects one null hypothesis và Holm rejects more than one at 0.1.
6. Với each of three panels trong Figure 13.3, answer:
- (a) Bao nhiêu false positives, false negatives, true positives, true negatives, Type I errors, Type II errors khi Bonferroni control FWER ở  $\alpha = 0.05$ ?
  - (b) Same quantities cho Holm ở  $\alpha = 0.05$ ?
  - (c) FDP associated với Bonferroni ở  $\alpha = 0.05$ ?
  - (d) FDP associated với Holm ở  $\alpha = 0.05$ ?
  - (e) Answers (a) và (c) change thế nào nếu Bonferroni controls FWER ở  $\alpha = 0.001$ ?

### Applied

7. Dùng **Carseats** dataset trong ISLP.
- (a) Với each quantitative variable besides **Sales**, fit linear model predicting **Sales** using that variable. Report p-value associated với slope  $\beta_1$  trong  $Y = \beta_0 + \beta_1 X + \varepsilon$ .
  - (b) Control Type I error ở  $\alpha = 0.05$  cho p-values ở (a): reject null hypotheses nào?
  - (c) Control FWER ở 0.05: reject hypotheses nào?
  - (d) Control FDR ở 0.2: reject hypotheses nào?
8. Trong bài này, simulate data từ  $m = 100$  fund managers.

```
rng = np.random.default_rng(1)
n, m = 20, 100
X = rng.normal(size=(n, m))
```

Data represent percentage returns của each manager over  $n = 20$  months. Test null rằng each manager's population mean return equals zero. Simulation được generated sao cho all  $m$  null hypotheses true.

- (a) Conduct one-sample t-test cho each manager và plot histogram của p-values.
- (b) Nếu control Type I error cho each null ở  $\alpha = 0.05$ , reject bao nhiêu null hypotheses?
- (c) Nếu control FWER ở 0.05, reject bao nhiêu?
- (d) Nếu control FDR ở 0.05, reject bao nhiêu?

- (e) Bây giờ “cherry-pick” 10 fund managers perform best trong data. Nếu control FWER cho only these 10 ở 0.05, reject bao nhiêu? Nếu control FDR cho only these 10 ở 0.05, reject bao nhiêu?
- (f) Explain vì sao analysis ở (e) misleading. *Hint:* standard FWER/FDR methods assume all tested null hypotheses are adjusted for multiplicity và no cherry-picking of smallest p-values. What goes wrong if we cherry-pick?

# Chỉ mục

- accuracy, 415
- activation, 400
- activation function, 401
- additive, 11, 94–98, 110–111
- additivity, 305, 306
- adjusted  $R^2$ , 87, 231, 232, 236–238
- Advertising data set, 15, 16, 19, 69, 71–73, 77, 78, 80, 82, 83, 85, 87–90, 95, 96, 109–111
- agglomerative clustering, 525
- Akaike information criterion, 87, 231, 232, 236–238
- alternative hypothesis, 76, 559
- analysis of variance, 312
- ANOVA, 587
- area under the curve, 155, 486–487
- argument, 40
- array, 42
- attribute, 42
- AUC, 155
- Auto data set, 12, 66, 98–101, 129, 197, 202–207, 327, 398
- auto-correlation, 421
- autoregression, 423
- axes, 48
- backfitting, 307, 328
- backpropagation, 429
- backward stepwise selection, 87, 234–235
- bag-of- $n$ -grams, 415
- bag-of-words, 414
- bagging, 11, 24, 331, 343–346, 354, 360–361
- BART, 343, 350, 353, 354, 362–363
- baseline, 93, 145, 161
- basis function, 293–294, 296
- Bayes
  - classifier, 35–37, 147
  - decision boundary, 148
  - error, 35–37
- Bayes’ theorem, 146, 250
- Bayesian, 250–251, 353
- Bayesian additive regression trees,
  - 331, 343, 350, 350, 353, 354, 362–363
- Bayesian information criterion, 87, 231, 232, 236–238
- Benjamini–Hochberg procedure, 575–577
- Bernoulli distribution, 172
- best subset selection, 231, 246
- bias, 31–34, 74, 90, 159, 405
- bias-variance
  - decomposition, 32



- trade-off, 31–34, 38, 111–112, 157, 159, 163, 164, 242, 254, 263, 266, 301, 336, 376, 385
- bidirectional, 425
- Bikeshare** data set, 12, 167–172
- binary, 27, 138
- biplot, 507, 508
- Bonferroni method, 575–577, 585
- Boolean, 53, 176
- boosting, 11, 24, 331, 343, 347–350, 354, 361–362
- bootstrap, 11, 201, 212–214, 343
- Boston** data set, 12, 67, 117, 122, 133, 199, 227, 287, 327, 364, 556
- bottom-up clustering, 525
- boxplot, 62
- BrainCancer** data set, 12, 472–474, 476, 482
- branch, 333
- burn-in, 352
- C-index, 487
- Caravan** data set, 12, 184, 366
- Carseats** data set, 12, 126, 130, 364
- categorical, 2, 27
- censored data, 469–502
- censoring
  - independent, 471
  - interval, 471
  - left, 471
  - mechanism, 471
  - non-informative, 471
  - right, 471
  - time, 470
- chain rule, 429
- channel, 407
- CIFAR100** data set, 406, 409–411, 448, 449
- classification, 2, 11, 27, 34–39, 135–199, 367–382
  - error rate, 338
  - tree, 337–341, 355–358
- classifier, 135
- cluster analysis, 25–26
- clustering, 4, 25–26, 520–535
  - agglomerative, 525
  - bottom-up, 525
  - hierarchical, 521, 525–535
  - $K$ -means, 11, 521–524
- Cochran–Mantel–Haenszel test, 475
- coefficient, 71
- College** data set, 12, 65, 286, 328
- collinearity, 106–110
- concatenation, 41
- conditional probability, 35
- confidence interval, 75–76, 90, 110, 292
- confounding, 144
- confusion matrix, 153, 176
- continuous, 2
- contour, 246
- contour plot, 50
- contrast, 94
- convenience function, 53
- convolution filter, 407
- convolution layer, 407
- convolutional neural network, 406–413
- correlation, 79, 82–83, 530
- count data, 167, 170
- Cox’s proportional hazards model, 480, 483–486
- $C_p$ , 87, 231, 232, 236–238
- Credit** data set, 12, 91, 92, 94, 97, 98, 106–109
- cross-entropy, 405
- cross-validation, 11, 31, 34, 201–211, 231, 252, 270
  - $k$ -fold, 206–209
  - leave-one-out, 204–206
- curse of dimensionality, 115, 193, 266
- data augmentation, 411
- data frame, 55
- Data sets
  - Advertising**, 15, 16, 19, 69, 71–73, 77, 78, 80, 82, 83, 85, 87–90, 95, 96, 109–111
  - Auto**, 12, 66, 98–101, 129, 197, 202–207, 327, 398
  - Bikeshare**, 12, 167–172

- Boston**, 12, 67, 117, 122, 133, 199, 227, 287, 327, 364, 556
- BrainCancer**, 12, 472–474, 476, 482
- Caravan**, 12, 184, 366
- Carseats**, 12, 126, 130, 364
- CIFAR100**, 406, 409–411, 448, 449
- College**, 12, 65, 286, 328
- Credit**, 12, 91, 92, 94, 97, 98, 106–109
- Default**, 12, 136–139, 141–144, 152–156, 160, 161, 225, 226, 466
- Fund**, 12, 567–570, 572, 575, 576, 585, 588, 589
- Heart**, 339, 340, 344–347, 352, 353, 382, 383
- Hitters**, 12, 332, 333, 336, 338, 339, 366, 425, 426, 437, 446
- IMDb**, 413, 415, 416, 418, 420, 437, 458, 467
- Income**, 16–18, 21–23
- Khan**, 12, 579–581, 583, 590, 593
- MNIST**, 402–404, 406, 430, 431, 441, 444, 445, 448
- NCI60**, 4, 5, 12, 546, 548–550
- NYSE**, 12, 422–424, 466, 467
- OJ**, 12, 365, 398
- Portfolio**, 12
- Publication**, 12, 482–487
- Smarket**, 2, 3, 12, 173, 184, 196
- USArrests**, 12, 507, 508, 510, 512, 513, 515, 516, 518, 519
- Wage**, 1, 2, 8, 9, 12, 290, 291, 293, 295, 297–300, 302–306, 309, 315, 327
- Weekly**, 12, 196, 226
- data type, 42
- decision function, 387
- decision tree, 11, 331–342
- deep learning, 399
- Default** data set, 12, 136–139, 141–144, 152–156, 160, 161, 225, 226, 466
- degrees of freedom, 30, 266, 295, 296, 301
- dendrogram, 521, 525–530
- density function, 146
- dependent variable, 15
- derivative, 296, 300
- detector layer, 410
- deviance, 232
- dictionary, 66
- dimension reduction, 230, 253–262
- discriminant function, 149
- discriminant method, 146–161
- dissimilarity, 530–532
- distance
  - correlation-based, 530–532, 554
  - Euclidean, 509, 522, 523, 529–532
- double descent, 431–435
- double-exponential distribution, 251
- dropout, 406, 431
- dummy variable, 91–94, 138, 142, 292
- early stopping, 430
- effective degrees of freedom, 301
- eigen decomposition, 506, 516
- elbow, 548
- embedding, 418
- embedding layer, 419
- ensemble, 343–354
- entropy, 337–339, 363
- epochs, 430
- error
  - irreducible, 17, 30
  - rate, 34
  - reducible, 17
  - term, 16
- Euclidean distance, 509, 522, 523, 529–532, 554
- event time, 470
- exception, 45
- expected value, 18
- exploratory data analysis, 504
- exponential, 173
- exponential family, 173
- F-statistic, 84

- factor, 92
- factorial, 170
- failure time, 470
- false
  - discovery proportion, 155, 573
  - discovery rate, 558, 573–577, 579–582
  - negative, 155, 562
  - positive, 155, 562, 563
  - positive rate, 155, 156, 382
- family-wise error rate, 565–573, 577
- feature, 15
- feature map, 406
- feature selection, 230
- featurize, 414
- feed-forward neural network, 400
- figure, 48
- fit, 21
- fitted value, 101
- flattening, 424
- flexible, 21
- floating point, 43
- forward stepwise selection, 86, 87, 233–234, 268
- function, 40
- Fund** data set, 12, 567–570, 572, 575, 576, 585, 588, 589
- Gamma, 173
- Gaussian (normal) distribution, 146, 147, 150, 172, 561
- generalized additive model, 5, 24, 162, 289, 290, 305–309, 319
- generalized linear model, 5, 135, 167–174, 217
- generative model, 146–161
- Gini index, 337–339, 345, 346, 363
- global minimum, 427
- gradient, 428
- gradient descent, 427
- Harrell’s concordance index, 487
- hazard function, 476–478
  - baseline, 478
- hazard rate, 476
- Heart** data set, 339, 340, 344–347, 352, 353, 382, 383
- heatmap, 50
- helper, 311
- heteroscedasticity, 103, 168
- hidden layer, 400
- hidden units, 400
- hierarchical clustering, 525–530
  - dendrogram, 525–528
  - inversion, 529
  - linkage, 529–530
- hierarchical principle, 96
- high-dimensional, 86, 234, 263
- hinge loss, 385
- Hitters** data set, 12, 332, 333, 336, 338, 339, 366, 425, 426, 437, 446
- hold-out set, 202
- Holm’s method, 568, 576, 585
- hypergeometric distribution, 501
- hyperparameter, 187
- hyperplane, 367–372
- hypothesis test, 76–77, 84, 103, 558–583
- IMDb** data set, 413, 415, 416, 418, 420, 437, 458, 467
- imputation, 515
- Income** data set, 16–18, 21–23
- increment, 60
- independent variable, 15
- indexable, 186
- indicator function, 292
- inference, 17, 18
- inner product, 379, 380
- input layer, 400
- input variable, 15
- integral, 301
- interaction, 70, 89, 95–98, 110–111, 308
- intercept, 71, 72
- interpolate, 432
- interpretability, 229
- inversion, 529
- irreducible error, 17, 36, 90, 110
- iterator, 312
- joint distribution, 158
- K-means clustering, 11, 521–524
- K-nearest neighbors, 135, 164–167
  - classifier, 11, 36–37
  - regression, 111–115

- Kaplan–Meier survival curve, 472–474, 483
- kernel, 379–382, 384, 394
  - linear, 380
  - non-linear, 377–382
  - polynomial, 380, 382
  - radial, 381–383, 390
- kernel density estimator, 159
- keyword, 46
- Khan** data set, 12, 579–581, 583, 590, 593
- knot, 290, 294, 296–299
- $\ell_1$  norm, 244
- $\ell_2$  norm, 242
- lag, 422
- Laplace distribution, 251
- lasso, 11, 24, 244–251, 265–266, 336, 385, 484
- leaf, 333, 526
- learning rate, 429
- least squares, 5, 21, 71–72, 140, 141, 229
  - line, 73
  - weighted, 103
- level, 92
- leverage, 104–106
- likelihood function, 141
- linear, 2, 69–115
- linear combination, 128, 230, 253, 505
- linear discriminant analysis, 5, 11, 135, 138, 147–155, 164–167, 377, 382
- linear kernel, 380
- linear model, 20, 69–115
- linear regression, 5, 11, 69–115, 172–173
  - multiple, 80–90
  - simple, 70–80
- link function, 172, 173
- linkage, 529–530, 548
  - average, 529–530
  - centroid, 529–530
  - complete, 526, 529–530
  - single, 529–530
- list, 41
- list comprehension, 123
- local minimum, 427
- local regression, 290
- log odds, 145
- log-rank test, 474–476, 483
- logistic function, 139
- logistic regression, 5, 11, 25, 135, 138–144, 164–167, 172–173, 308–309, 377, 384–385
  - multinomial, 145, 163
  - multiple, 142–144
- logit, 140
- loss function, 300, 385
- low-dimensional, 262
- LSTM RNN, 420
- main effects, 96
- majority vote, 344
- Mallow’s  $C_p$ , 87, 231, 232, 236–238
- Mantel–Haenszel test, 475
- margin, 370, 385
- marginal distribution, 158
- Markov chain Monte Carlo, 353
- matrix completion, 515
- matrix multiplication, 10
- maximal margin
  - classifier, 367–372
  - hyperplane, 370
- maximum likelihood, 139–141, 143, 170
- mean squared error, 28
- mesh, 53
- method, 43
- minibatch, 429
- misclassification error, 35
- missing at random, 515
- missing data, 56, 515–520
- mixed selection, 87
- MNIST** data set, 402–404, 406, 430, 431, 441, 444, 445, 448
- model assessment, 201
- model selection, 201
- module, 42
- multicollinearity, 108, 266
- multinomial logistic regression, 145, 163
- multiple testing, 557–583
- multi-task learning, 403
- multivariate Gaussian, 150

- multivariate normal, 150
- naive Bayes, 135, 158–161, 164–167
- namespace, 116
- natural spline, 297, 298, 301, 317
- NCI60 data set, 4, 5, 12, 546, 548–550
- negative binomial, 173
- negative predictive value, 155, 156
- neural network, 5, 399
- node
  - internal, 333
  - purity, 337–339
  - terminal, 333
- noise, 21, 252
- non-linear, 2, 11, 289–329
  - decision boundary, 377–382
  - kernel, 377–382
- non-parametric, 20, 22–23, 111–115, 193
- normal (Gaussian) distribution, 146, 147, 150, 172, 476, 561
- notebook, 40
- null, 152
  - distribution, 561, 578
  - hypothesis, 76, 559
  - model, 87, 231, 245
- null rate, 186
- NYSE data set, 12, 422–424, 466, 467
- Occam’s razor, 426
- odds, 140, 145, 195
- OJ data set, 12, 365, 398
- one-hot encoding, 92, 126, 403
- one-standard-error rule, 240
- one-versus-all, 384
- one-versus-one, 384
- one-versus-rest, 384
- optimal separating hyperplane, 370
- optimism of training error, 30
- ordered categorical variable, 315
- orthogonal, 257, 506
  - basis, 125
- out-of-bag, 345
- outlier, 103–104
- output variable, 15
- over-parametrized, 465
- overdispersion, 172
- overfitting, 21, 23, 25, 30–31, 88, 152, 233, 371
- p-value, 77, 82, 560–562, 578–579
  - adjusted, 586
- package, 42
- parameter, 71
- parametric, 20–22, 111–115
- partial least squares, 254, 260–262, 282
- partial likelihood, 480
- path algorithm, 249
- permutation, 578
- permutation approach, 577–582
- perpendicular, 257
- Poisson distribution, 169, 172
- Poisson regression, 135, 167–173
- polynomial
  - kernel, 380, 382
  - regression, 98–99, 289–292, 294–295
- pooling, 410
- population regression line, 73
- Portfolio data set, 12
- positive predictive value, 155, 156
- posterior
  - distribution, 251
  - mode, 251
  - probability, 147
- power, 108, 155, 563
- precision, 155
- prediction, 17
  - interval, 90, 110
- predictor, 15
- principal components, 505
  - analysis, 11, 254–260, 504–515
  - loading vector, 505, 506
  - missing values, 515–520
  - proportion of variance explained, 510–515, 547
  - regression, 11, 254–260, 280–282, 504, 515
  - score vector, 506
  - scree plot, 514–515
- prior
  - distribution, 251
  - probability, 146
- probability density function, 477, 478

- projection, 230
- proportional hazards assumption, 478
- pruning, 336
  - cost complexity, 336
  - weakest link, 336
- Publication data set, 12, 482–487
- Python objects and functions
  - %capture, 458
  - iloc[], 58
  - loc[], 57
  - AgglomerativeClustering(), 543
  - anova(), 313
  - anova\_lm(), 125, 129, 312, 313
  - axhline(), 122, 551
  - axline(), 121, 129, 329
  - BART(), 362
  - biplot, 537
  - boot\_SE(), 223
  - boxplot(), 62, 66
  - bs(), 315, 327
  - BSpline(), 315
  - clone(), 222
  - columns.drop(), 122
  - compute\_linkage(), 544
  - confusion\_table(), 176
  - contour(), 50
  - corr(), 129, 174
  - cost\_complexity\_pruning\_path(), 357
  - CoxPHFitter(), 491
  - cross\_val\_predict(), 270
  - cross\_validate(), 218, 219, 226
  - cumsum(), 539
  - cut\_tree(), 545
  - data.frame(), 227
  - Dataset, 440
  - decision\_function(), 392
  - DecisionTreeClassifier(), 354, 355
  - DecisionTreeRegressor(), 354
  - def, 121
  - dendrogram(), 544
  - describe(), 62, 66
  - dir(), 116
  - drop(), 179
  - dropna(), 56, 268, 461
  - DTC(), *see* DecisionTreeClassifier()
  - DTR(), *see* DecisionTreeRegressor()
  - dtype, 43
  - ElasticNetCV(), 279
  - enumerate(), 217
  - export\_text(), 356
  - export\_tree(), 365
  - fit(), 118, 181, 218
  - fit\_transform(), 119
  - for, 59
  - GaussianNB(), 182
  - GBR(), *see* GradientBoostingRegressor()
  - get\_dummies(), 461
  - get\_influence(), 121
  - get\_prediction(), 120, 314
  - get\_rdataset(), 535
  - glm(), 313
  - glob(), 437
  - GradientBoostingClassifier(), 361
  - GradientBoostingRegressor(), 354, 361
  - GridSearchCV(), 276
  - groupby(), 490
  - hist(), 62
  - iloc[], 58, 59
  - import, 42
  - imshow(), 50, 449
  - ISLP.bart, 362
  - ISLP.cluster, 544
  - json, 437
  - KaplanMeierFitter(), 502
  - keras, 437
  - KFold(), 219
  - KMeans(), 542, 543
  - Kmeans(), 542
  - KNeighborsClassifier(), 183
  - lambda, 58
  - LDA(), *see* LinearDiscriminantAnalysis()
  - legend(), 132
  - lifelines, 490
  - LinearDiscriminantAnalysis(), 174, 179
  - LinearGAM(), 317
  - LinearRegression(), 280

- `load_data()`, 117
- `loc[]`, 58, 59, 177
- `log_loss()`, 355
- `LogisticGAM()`, 323
- `logrank_test()`, 490
- `lowess()`, 324
- `matplotlib`, 48
- `max()`, 66
- `mean()`, 48
- `median()`, 197
- `min()`, 66
- `MNIST()`, 444
- `ModelSpec()`, 116–118, 122, 124, 267
- `MS()`, *see* `ModelSpec()`
- `mult_test()`, *see* `multipletests()`
- `multipletests()`, 586
- `multipletests()`, 583, 589
- `multivariate_logrank_test()`, 496
- `NaturalSpline()`, 317, 319
- `ndim`, 42
- `nn.RNN()`, 461
- `normal()`, 132, 286, 555
- `np`, *see* `numpy`
- `np.all()`, 54, 180
- `np.allclose()`, 190
- `np.any()`, 54
- `np.arange()`, 51
- `np.argmax()`, 122
- `np.array()`, 42
- `np.concatenate()`, 133
- `np.corrcoef()`, 46, 554
- `np.empty()`, 224
- `np.isnan()`, 268
- `np.ix_()`, 53
- `np.linalg.svd()`, 539
- `np.linspace()`, 50
- `np.logspace()`, 318
- `np.mean()`, 47, 176
- `np.nan`, 60
- `np.nanmean()`, 541
- `np.percentile()`, 228
- `np.power()`, 219
- `np.random.choice()`, 553
- `np.random.default_rng()`, 46, 47
- `np.random.normal()`, 45
- `np.sqrt()`, 45
- `np.squeeze()`, 457
- `np.std()`, 47
- `np.sum()`, 43
- `np.var()`, 47
- `np.where()`, 180
- `ns()`, 317
- `numpy`, 42, 555
- `os.chdir()`, 55
- `outer()`, 219
- `pairwise_distances()`, 554
- `pairwise_tukeyhsd()`, 587
- `pandas`, 55
- `params`, 175
- `partial()`, 222, 269
- `PCA()`, 280, 537, 540, 554
- `pd`, *see* `pandas`
- `pd.crosstab()`, 555
- `pd.cut()`, 315
- `pd.get_dummies()`, 314
- `pd.plotting.scatter_matrix()`, 62
- `pd.qcut()`, 314, 315
- `pd.read_csv()`, 55, 556
- `pd.Series()`, 62
- `Pipeline()`, 275
- `plot()`, 48, 61, 356, 490
- `plot.scatter()`, 120
- `plot_gam()`, 321
- `plot_svm()`, 398
- `PLSRegression()`, 282
- `poly()`, 125, 313, 327
- `predict()`, 175, 178, 181, 216, 218, 323, 358
- `predict_survival_function()`, 493
- `print()`, 40
- `pvalues`, 175
- `pygam`, 307, 317
- `pytorch_lightning`, 435
- `QDA()`, *see* `QuadraticDiscriminantAnalysis()`
- `QuadraticDiscriminantAnalysis()`, 174, 181
- `random()`, 555
- `RandomForestRegressor()`, 354, 360
- `read_image()`, 436
- `reindex()`, 461
- `reshape()`, 43

- `return`, 198
- `RF()`, *see* `RandomForestRegressor()`
- `rng`, *see* `np.random.default_rng()`
- `rng.choice()`, 60
- `rng.standard_normal()`, 60
- `roc_curve()`, 392
- `RocCurveDisplay.from_estimator()`, 387
- `savefig()`, 50
- `scatter()`, 49, 61
- `scipy.interpolate`, 315
- `score()`, 218, 461
- `seed_everything()`, 436
- `set_index()`, 57
- `set_title()`, 49
- `set_xlabel()`, 49
- `set_xscale()`, 198
- `set_ylabel()`, 49
- `set_yscale()`, 198
- `shape`, 43
- `ShuffleSplit()`, 219
- `sim_time()`, 495
- `SimpleDataModule()`, 441
- `SimpleModule.classification()`, 446
- `SimpleModule.regression()`, 442
- `skl`, *see* `sklearn.linear_model`
- `skl.ElasticNet()`, 273, 277
- `skl.ElasticNet.path`, 274
- `skl.ElasticNet.path()`, 273
- `sklearn`, 118, 181
- `sklearn.ensemble`, 360
- `sklearn.linear_model`, 267
- `sklearn.model_selection`, 267
- `sklearn_selected()`, 269
- `sklearn_selection_path()`, 270
- `sklearn_sm()`, 218
- `skm`, *see* `sklearn.model_selection`
- `skm.cross_val_predict()`, 271
- `skm.KFold()`, 271
- `skm.ShuffleSplit()`, 272
- `slice()`, 51, 462
- `sm`, *see* `statsmodels`
- `sm.GLM()`, 174, 192, 226
- `sm.Logit()`, 174
- `sm.OLS()`, 118, 129, 174, 319
- `StandardScaler()`, 185, 438, 537, 555
- `statsmodels`, 116, 173
- `std()`, 186
- `Stepwise()`, 269
- `str.contains()`, 59
- `subplots()`, 48
- `sum()`, 43, 268
- `summarize()`, 118, 129, 223, 226
- `summary()`, 119, 322, 587
- `super()`, 440
- `SupportVectorClassifier()`, 387, 389–391, 393
- `SupportVectorRegression()`, 394
- `SVC()`, *see* `SupportVectorClassifier()`
- `svd()`, 539
- `SVR()`, *see* `SupportVectorRegression()`
- `TensorDataset()`, 441
- `to_numpy()`, 437
- `torch`, 435
- `torchinfo`, 436
- `torchmetrics`, 436
- `torchvision`, 436
- `ToTensor()`, 444
- `train_test_split()`, 186, 216
- `transform()`, 118, 119
- `ttest_1samp()`, 584
- `ttest_ind()`, 590
- `ttest_rel()`, 587
- `tuple`, 43
- `uniform()`, 555
- `value_counts()`, 66
- `var()`, 536
- `variance_inflation_factor()`, 116, 124
- `VIF()`, *see* `variance_inflation_factor()`
- `where()`, 355
- `zip()`, 60, 312
- q-values, 589
- quadratic, 98
- quadratic discriminant analysis, 4, 135, 156–157, 164–167



- qualitative, 2, 27, 91, 135, 167, 202
  - variable, 91–94
- quantitative, 2, 27, 91, 135, 167, 202
- radial kernel, 381, 383, 390
- random forest, 11, 331, 343, 346–347, 354, 360–361
- random seed, 46
- re-sampling, 577–582
- recall, 155
- receiver operating characteristic (ROC), 154, 382–383
- recommender systems, 516
- rectified linear unit, 401
- recurrent neural network, 416–427
- recursive binary splitting, 334, 337, 338
- reducible error, 17, 90
- regression, 2, 11, 27
  - local, 289, 290, 304–305
  - piecewise polynomial, 294–295
  - polynomial, 289–292, 299
  - spline, 289, 294
  - tree, 331–337, 358–360
- regularization, 230, 240, 406, 484–486
- ReLU, 401
- resampling, 201–214
- residual, 71, 81
  - plot, 100
  - standard error, 75, 77–78, 88–89, 109
  - studentized, 104
  - sum of squares, 71, 79, 81
- residuals, 263, 348
- response, 15
- ridge regression, 11, 240–244, 385, 484
- risk set, 473
- robust, 374, 376, 535
- ROC curve, 154, 382–383, 486–487
- $R^2$ , 77–80, 88, 109, 238
- rug plot, 314
- scale equivariant, 242
- Scheffé’s method, 572
- scree plot, 512, 514–515
- elbow, 514
- semi-supervised learning, 27
- sensitivity, 153, 155, 156
- separating hyperplane, 367–372
- Seq2Seq, 425
- sequence, 41
- shrinkage, 230, 240, 484–486
  - penalty, 240
- sigmoid, 401
- signal, 252
- signature, 45
- singular value decomposition, 539
- slack variable, 375
- slice, 51
- slope, 71, 72
- Smarket** data set, 2, 3, 12, 173, 184, 196
- smoother, 308
- smoothing spline, 290, 300–303
- soft margin classifier, 372–374
- soft-thresholding, 250
- softmax, 145, 405
- sparse, 244, 252
- sparse matrix format, 414
- sparsity, 244
- specificity, 153, 155, 156
- spline, 289, 294–303
  - cubic, 296
  - linear, 296
  - natural, 297, 301
  - regression, 289, 294–299
  - smoothing, 30, 290, 300–303
  - thin-plate, 22
- standard error, 75, 101
- standardize, 185
- statistical model, 1
- step function, 111, 289, 292–293
- stepwise model selection, 11, 231, 233
- stochastic gradient descent, 429
- string, 41
- string interpolation, 490
- stump, 349
- subset selection, 230–240
- subtree, 336
- supervised learning, 25–27, 261
- support vector, 371, 376, 385
  - classifier, 367, 372–377
  - machine, 5, 11, 24, 377–386

- regression, 386
- survival
  - analysis, 469–502
  - curve, 472, 483
  - function, 472
  - time, 470
- synergy, 70, 89, 95–98, 110–111
- systematic, 16
- t-distribution, 77, 165
- t-statistic, 76
- t-test
  - one-sample, 583, 584, 588
  - paired, 587
  - two-sample, 559, 570, 571, 577–581, 584, 590
- test
  - error, 35, 37, 176
  - MSE, 28–32
  - observations, 28
  - set, 30
  - statistic, 559
- theoretical null distribution, 577
- time series, 101
- total sum of squares, 79
- tracking, 102
- train, 21
- training
  - data, 20
  - error, 35, 37, 176
  - MSE, 28–31
- transformer, 311
- tree, 331–342
- tree-based method, 331
- true negative, 155
- true positive, 155
- true positive rate, 155, 156, 382
- truncated power basis, 296
- Tukey’s method, 571, 585, 587
- tuning parameter, 187, 240, 484
- two-sample *t*-test, 474
- Type I error, 155, 562–565
- Type I error rate, 563
- Type II error, 155, 563, 568, 584
- unsupervised learning, 25–27, 255, 260, 503–552
- USArrests data set, 12, 507, 508, 510, 512, 513, 515, 516, 518, 519
- validation set, 202
  - approach, 202–204
- variable, 15
  - dependent, 15
  - dummy, 91–94, 97–98
  - importance, 346, 360
  - independent, 15
  - indicator, 35
  - input, 15
  - output, 15
  - qualitative, 91–94, 97–98
  - selection, 86, 230, 244
- variance, 18, 31–34, 159
  - inflation factor, 108–110, 123
- varying coefficient model, 305
- Wage data set, 1, 2, 8, 9, 12, 290, 291, 293, 295, 297–300, 302–306, 309, 315, 327
- weak learner, 343
- weakest link pruning, 336
- Weekly data set, 12, 196, 226
- weight freezing, 412, 419
- weight sharing, 418
- weighted least squares, 103, 304
- weights, 404
- with replacement, 214
- within class covariance, 150
- wrapper, 217